

# ASIC Design

## From Specification to Silicon

### The Complete Guide - From Beginner to Expert

Devices & CMOS · Process & PDKs · Standard Cells & Liberty · Architecture · RTL for ASIC · Verification · Synthesis · STA · DFT · Low Power & UPF · AMS & ESD · Floorplan & Power Grid · Placement · CTS · Routing & SI · Sign-off · DRC/LVS · ECO · Tapeout · Packaging & Chiplets · Test & Yield · Reliability · Bring-up · Economics · Open-Source Flow

|                           |                                                                                                                                                                                                                                 |
|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Contents</b>           | 26 chapters in 6 parts, plus 4 appendices                                                                                                                                                                                       |
| <b>Scope</b>              | The whole life of a chip: business case, device physics and process, front end, back end, sign-off, tapeout, packaging, manufacturing test, yield, reliability and bring-up                                                     |
| <b>Real numbers</b>       | Synthesis was run with <b>Yosys</b> against the real <b>SkyWater SKY130</b> standard-cell library; timing, power, IR drop, placement, routing, yield and cost were modelled in <b>Python</b> . Green cards show the real output |
| <b>Honest about tools</b> | Commercial sign-off tools cannot be run here; any report excerpt that did not come from a real run is labelled illustrative                                                                                                     |
| <b>Series</b>             | Fourth book, with <i>RTL Design for SoC &amp; ASIC</i> , <i>Verilog &amp; SystemVerilog for SoC &amp; ASIC</i> and <i>Communication Protocols for SoC &amp; ASIC</i> in this repository                                         |
| <b>Assumed</b>            | Basic digital logic. Everything else is built up from first principles.                                                                                                                                                         |

## How To Use This Book

An **ASIC** - an application-specific integrated circuit - is a chip built for one job, from a phone's application processor to a network switch, an automotive controller or an AI accelerator. Designing one is a relay race run by a dozen specialist teams over one to three years: architects, RTL designers, verification engineers, DFT, synthesis and timing, physical design, physical verification, analog, packaging, test and product engineers. Each team hands the next a precise set of files, and a mistake at any hand-off can cost a mask set, a quarter of schedule, or the product.

This book follows that relay from start to finish. It explains what each team does and why, the physics and algorithms underneath its tools, the numbers it works to, the files it hands on, and the mistakes that reach silicon. The three companion books cover RTL design, the Verilog and SystemVerilog languages, and communication protocols in depth; this one covers everything else, and how it all fits together into a working chip.

### The six parts

| Part                                      | Chapters | What you get out of it                                                                                                                                                                                                                                |
|-------------------------------------------|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>I - Foundations</b>                    | 1-5      | The ASIC business and the end-to-end flow; transistors, CMOS, delay, power and scaling; how chips are manufactured and what a PDK contains; standard cells, Liberty, LEF, memories and IP; turning requirements into an architecture and PPA targets. |
| <b>II - Front-End Design</b>              | 6-10     | What the back end needs from RTL; verification strategy and sign-off; logic synthesis; static timing analysis; design for test.                                                                                                                       |
| <b>III - Power, AMS and PD Setup</b>      | 11-14    | Low-power techniques and UPF; analog, I/O and ESD integration; the physical-design data and flow; floorplanning, the power grid and the I/O ring.                                                                                                     |
| <b>IV - Implementation &amp; Sign-off</b> | 15-19    | Placement; clock tree synthesis; routing, signal integrity and DFM; timing, power, reliability and physical-verification sign-off; ECOs, tapeout and mask making.                                                                                     |
| <b>V - After Tapeout</b>                  | 20-23    | Packaging, 2.5D/3D and chiplets; manufacturing test, yield and reliability; bring-up, characterisation and qualification; methodology, project management and cost.                                                                                   |
| <b>VI - Practice &amp; Roadmap</b>        | 24-26    | The open-source flow on open PDKs; a complete RTL-to-GDSII case study; the career roadmap and interview preparation.                                                                                                                                  |

### Reading paths

- **Beginner:** read Part I, then Chapters 8, 9, 13 and 25 for a first complete pass through the flow, then fill in the rest.
- **RTL or verification engineer:** 1, 4, 6-11, 13 and 18 - what happens to your code after you hand it off.
- **Aspiring physical-design engineer:** 2-4, 8, 9, then Parts III and IV in order, then 24-25.
- **Test, product or quality engineer:** 3, 10, 20-22.
- **Manager or architect:** 1, 5, 19, 21 and 23.
- **Interview preparation:** 2, 8, 9, 10, 14-18, then Appendix C.

### Conventions

| Style                 | Meaning                                                                                                       |
|-----------------------|---------------------------------------------------------------------------------------------------------------|
| <b>A grey card</b>    | A script, netlist, constraint file, library excerpt or program, exactly as it was run or written              |
| <b>A green card</b>   | Real output from Yosys, Icarus Verilog, Verilator or Python, produced while this book was written             |
| <b>"Illustrative"</b> | A report excerpt showing the shape of a commercial tool's output; not from a real run, and always labelled so |

| Style                 | Meaning                                                                                                                                                                                                                            |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Coloured boxes</b> | KEY IDEA = the one thing to remember. MATH = an equation or calculation. INTUITION = the mental picture. PRACTICAL TIP = what to do. PITFALL = a mistake that reaches silicon. EXPERT CORNER = depth you can skip on a first read. |
| <b>Numbers</b>        | Process and cost figures vary by foundry, node and year; where they do, they are given as typical values or orders of magnitude, never as foundry data                                                                             |

#### RUN A REAL FLOW YOURSELF

The open-source SKY130, GF180MCU and IHP SG13G2 PDKs, together with Yosys, OpenROAD, Magic, KLayout and Netgen, let anyone take a design from RTL to a manufacturable GDSII on a laptop - and shuttle programmes have put such designs on real silicon. Chapter 24 shows how, and Chapter 25 walks a block through every step.

#### THE ONE IDEA THAT TIES THE FLOW TOGETHER

Every step trades **performance, power and area** (PPA) against **schedule and risk**, using models that get more accurate as the design gets more physical. Early steps estimate; later steps measure. The skill of ASIC design is making early estimates good enough that the measurements at sign-off hold no surprises.

# Table of Contents

|                                                                                                                        |           |
|------------------------------------------------------------------------------------------------------------------------|-----------|
| <b>PART I - Foundations</b>                                                                                            | <b>14</b> |
| 1. The ASIC Landscape: What an ASIC Is, Design Styles, Economics and the End-to-End Flow . . . . .                     | 15        |
| 1.1 ASIC, SoC, ASSP: the vocabulary . . . . .                                                                          | 15        |
| 1.2 Design styles: from full custom to FPGA . . . . .                                                                  | 16        |
| 1.3 When does building an ASIC make sense? . . . . .                                                                   | 17        |
| 1.4 The industry: fabless, foundry, IDM, OSAT and the EDA/IP ecosystem . . . . .                                       | 18        |
| 1.5 The end-to-end ASIC flow . . . . .                                                                                 | 19        |
| 1.6 What a typical project schedule looks like . . . . .                                                               | 21        |
| 1.7 Who does what: the roles in an ASIC team . . . . .                                                                 | 21        |
| 1.8 Roadmap: how this book is organised . . . . .                                                                      | 22        |
| 1.9 Summary . . . . .                                                                                                  | 22        |
| 1.10 Exercises . . . . .                                                                                               | 22        |
| 2. Semiconductor Device Fundamentals: MOSFETs, CMOS, Delay, Power and Scaling . . . . .                                | 23        |
| 2.1 Silicon, doping and the pn junction . . . . .                                                                      | 23        |
| 2.2 The MOSFET: structure and operation . . . . .                                                                      | 23        |
| 2.3 Threshold voltage, body effect and subthreshold conduction . . . . .                                               | 25        |
| 2.4 Short-channel effects and DIBL . . . . .                                                                           | 26        |
| 2.5 The CMOS inverter . . . . .                                                                                        | 27        |
| 2.6 Switching delay: RC and Elmore . . . . .                                                                           | 27        |
| 2.7 Logical effort: sizing a path by hand . . . . .                                                                    | 28        |
| 2.8 Power: dynamic, short-circuit and leakage . . . . .                                                                | 29        |
| 2.9 Scaling: Dennard and its end . . . . .                                                                             | 30        |
| 2.10 Device architectures: planar, FinFET, GAA/nanosheet, CFET . . . . .                                               | 30        |
| 2.11 Multi-Vt, SOI/FD-SOI and body bias . . . . .                                                                      | 31        |
| 2.12 Summary . . . . .                                                                                                 | 31        |
| 2.13 Exercises . . . . .                                                                                               | 31        |
| 3. Process Technology and Fabrication: Wafers, Lithography, FEOL/MEOL/BEOL, Design Rules, PDKs and Variation . . . . . | 32        |
| 3.1 From sand to wafers . . . . .                                                                                      | 32        |
| 3.2 The unit processes . . . . .                                                                                       | 32        |
| 3.3 Photolithography . . . . .                                                                                         | 33        |
| 3.4 FEOL: building the transistors . . . . .                                                                           | 34        |
| 3.5 MEOL: the local interconnect . . . . .                                                                             | 34        |
| 3.6 BEOL: the metal stack . . . . .                                                                                    | 34        |
| 3.7 Node names versus real dimensions . . . . .                                                                        | 35        |
| 3.8 Design rules . . . . .                                                                                             | 36        |
| 3.9 The process design kit (PDK) . . . . .                                                                             | 36        |
| 3.10 Process variation . . . . .                                                                                       | 37        |

|                                                                                                                   |           |
|-------------------------------------------------------------------------------------------------------------------|-----------|
| 3.11 Summary . . . . .                                                                                            | 38        |
| 3.12 Exercises . . . . .                                                                                          | 39        |
| <b>4. Standard Cells, Libraries and IP: Cell Design, Liberty, LEF, Memories, I/O and IP Integration . . . . .</b> | <b>40</b> |
| 4.1 Standard-cell architecture . . . . .                                                                          | 40        |
| 4.2 Cell families . . . . .                                                                                       | 40        |
| 4.3 Drive strengths, Vt flavours and variants . . . . .                                                           | 41        |
| 4.4 Characterisation: how a Liberty file is made . . . . .                                                        | 42        |
| 4.5 Liberty in depth . . . . .                                                                                    | 43        |
| 4.6 Looking up an NLDM table . . . . .                                                                            | 46        |
| 4.7 Using a library: dont_use lists and a real synthesis . . . . .                                                | 47        |
| 4.8 LEF: the physical abstract . . . . .                                                                          | 48        |
| 4.9 Memories: SRAM compilers, register files and ROM . . . . .                                                    | 50        |
| 4.10 I/O libraries . . . . .                                                                                      | 50        |
| 4.11 IP: soft, firm and hard . . . . .                                                                            | 51        |
| 4.12 Summary . . . . .                                                                                            | 51        |
| 4.13 Exercises . . . . .                                                                                          | 52        |
| <b>5. Specification, Architecture and Micro-architecture: From Requirements to PPA Targets . . . . .</b>          | <b>53</b> |
| 5.1 Requirements: what the chip must do . . . . .                                                                 | 53        |
| 5.2 Architecture exploration . . . . .                                                                            | 53        |
| 5.3 Hardware/software partitioning . . . . .                                                                      | 55        |
| 5.4 Memory hierarchy and bandwidth budgets . . . . .                                                              | 55        |
| 5.5 Clock and power domain planning . . . . .                                                                     | 56        |
| 5.6 IP selection . . . . .                                                                                        | 56        |
| 5.7 Early die-size and power estimation . . . . .                                                                 | 57        |
| 5.8 PPA targets and the frequency sanity check . . . . .                                                          | 58        |
| 5.9 The micro-architecture specification . . . . .                                                                | 58        |
| 5.10 Reviews, sign-offs and requirement traceability . . . . .                                                    | 59        |
| 5.11 Summary . . . . .                                                                                            | 59        |
| 5.12 Exercises . . . . .                                                                                          | 59        |

## PART II - Front-End Design

61

|                                                                                |           |
|--------------------------------------------------------------------------------|-----------|
| <b>6. RTL Design for ASIC: What the Back End Needs from Your RTL . . . . .</b> | <b>62</b> |
| 6.1 Who consumes your RTL . . . . .                                            | 62        |
| 6.2 ASIC versus FPGA coding: the differences that bite . . . . .               | 63        |
| 6.3 Reset strategy: which flops need a reset . . . . .                         | 63        |
| 6.4 Memories: macros and wrappers instead of inferred arrays . . . . .         | 65        |
| 6.5 Clock gating in RTL . . . . .                                              | 66        |
| 6.6 No internal tri-states, and synchronous design . . . . .                   | 67        |
| 6.7 Hierarchy for physical design . . . . .                                    | 67        |
| 6.8 Clocks, resets, CDC and RDC in one page . . . . .                          | 68        |
| 6.9 Pipelining for timing: logic depth per stage . . . . .                     | 68        |

|                                                                               |           |
|-------------------------------------------------------------------------------|-----------|
| 6.10 DFT-friendly RTL . . . . .                                               | 69        |
| 6.11 Low-power-friendly RTL . . . . .                                         | 69        |
| 6.12 RTL quality gates . . . . .                                              | 69        |
| 6.13 The hand-off package to synthesis . . . . .                              | 70        |
| 6.14 Summary . . . . .                                                        | 71        |
| 6.15 Exercises . . . . .                                                      | 71        |
| <b>7. Functional Verification Strategy and Sign-off . . . . .</b>             | <b>72</b> |
| 7.1 Why verification dominates, and how it is organised . . . . .             | 72        |
| 7.2 The verification plan . . . . .                                           | 72        |
| 7.3 Testbench tiers . . . . .                                                 | 73        |
| 7.4 UVM in one page . . . . .                                                 | 73        |
| 7.5 Constrained-random stimulus and functional coverage: a real run . . . . . | 74        |
| 7.6 Formal verification . . . . .                                             | 76        |
| 7.7 Emulation and FPGA prototyping . . . . .                                  | 77        |
| 7.8 Hardware/software co-verification . . . . .                               | 77        |
| 7.9 Coverage closure . . . . .                                                | 77        |
| 7.10 Bug tracking and trend curves . . . . .                                  | 78        |
| 7.11 Regression management . . . . .                                          | 78        |
| 7.12 Gate-level simulation . . . . .                                          | 79        |
| 7.13 Power-aware simulation . . . . .                                         | 79        |
| 7.14 Verification sign-off . . . . .                                          | 79        |
| 7.15 Summary . . . . .                                                        | 80        |
| 7.16 Exercises . . . . .                                                      | 80        |
| <b>8. Logic Synthesis in Depth . . . . .</b>                                  | <b>81</b> |
| 8.1 The production synthesis flow . . . . .                                   | 81        |
| 8.2 Elaboration and the generic (GTECH) netlist . . . . .                     | 82        |
| 8.3 Boolean optimisation and technology mapping . . . . .                     | 82        |
| 8.4 Timing-driven optimisation . . . . .                                      | 83        |
| 8.5 A real run: the area-delay trade-off of a 32-bit adder . . . . .          | 83        |
| 8.6 Datapath synthesis . . . . .                                              | 85        |
| 8.7 Retiming, clock-gating insertion and sequential optimisations . . . . .   | 86        |
| 8.8 Hierarchy, boundary optimisation and ungrouping . . . . .                 | 87        |
| 8.9 From wire-load models to physical-aware synthesis . . . . .               | 87        |
| 8.10 Synthesis constraints and design rules . . . . .                         | 88        |
| 8.11 Reading QoR reports . . . . .                                            | 88        |
| 8.12 Common synthesis warnings and what they mean . . . . .                   | 89        |
| 8.13 Logic equivalence checking after synthesis . . . . .                     | 90        |
| 8.14 Summary . . . . .                                                        | 91        |
| 8.15 Exercises . . . . .                                                      | 91        |
| <b>9. Static Timing Analysis in Depth . . . . .</b>                           | <b>92</b> |
| 9.1 The timing graph and timing arcs . . . . .                                | 92        |

|                                                                                     |            |
|-------------------------------------------------------------------------------------|------------|
| 9.2 Delay calculation: NLDM tables, slew propagation and beyond . . . . .           | 93         |
| 9.3 Wire models: from lumped C to extracted RC . . . . .                            | 94         |
| 9.4 Setup and hold: the full equations . . . . .                                    | 94         |
| 9.5 A worked example: STA of a real SKY130 path in Python . . . . .                 | 95         |
| 9.6 Reading a timing report line by line . . . . .                                  | 97         |
| 9.7 Constraints: clocks, I/O, exceptions . . . . .                                  | 98         |
| 9.8 Latches and time borrowing . . . . .                                            | 99         |
| 9.9 On-chip variation: derates, AOCV, POCV/SOCV and LVF . . . . .                   | 99         |
| 9.10 Multi-corner multi-mode (MCM) analysis . . . . .                               | 100        |
| 9.11 Signal integrity: crosstalk delay and glitch . . . . .                         | 100        |
| 9.12 Hold fixing and timing ECOs . . . . .                                          | 101        |
| 9.13 Constraint quality checks . . . . .                                            | 101        |
| 9.14 Summary . . . . .                                                              | 101        |
| 9.15 Exercises . . . . .                                                            | 102        |
| <b>10. Design for Test: Scan, ATPG, Compression, BIST and Test Access . . . . .</b> | <b>103</b> |
| 10.1 Why test: defects, faults and quality . . . . .                                | 103        |
| 10.2 Fault models . . . . .                                                         | 103        |
| 10.3 Scan design . . . . .                                                          | 104        |
| 10.4 Scan chains, shift and capture . . . . .                                       | 104        |
| 10.5 At-speed test: launch-on-capture, launch-on-shift and OCCs . . . . .           | 105        |
| 10.6 ATPG: automatic test pattern generation . . . . .                              | 105        |
| 10.7 Fault coverage versus test coverage . . . . .                                  | 107        |
| 10.8 Test compression . . . . .                                                     | 107        |
| 10.9 Memory BIST and repair . . . . .                                               | 108        |
| 10.10 Logic BIST . . . . .                                                          | 109        |
| 10.11 Test access: IEEE 1149.1, 1500 and 1687 . . . . .                             | 110        |
| 10.12 Test modes and pin muxing . . . . .                                           | 110        |
| 10.13 DFT sign-off . . . . .                                                        | 111        |
| 10.14 DFT impact on area, timing and power . . . . .                                | 111        |
| 10.15 Summary . . . . .                                                             | 112        |
| 10.16 Exercises . . . . .                                                           | 112        |
| <b>PART III - Power, Mixed-Signal and Physical Design Setup . . . . .</b>           | <b>113</b> |
| <b>11. Low-Power Design and Power Intent (UPF) . . . . .</b>                        | <b>114</b> |
| 11.1 Where the power goes . . . . .                                                 | 114        |
| 11.2 Power analysis flow . . . . .                                                  | 115        |
| 11.3 Techniques by level of abstraction . . . . .                                   | 116        |
| 11.4 UPF (IEEE 1801) in depth . . . . .                                             | 119        |
| 11.5 UPF through the flow . . . . .                                                 | 123        |
| 11.6 Power-aware verification . . . . .                                             | 124        |
| 11.7 Rush current and power-up sequencing . . . . .                                 | 124        |

|                                                                                |            |
|--------------------------------------------------------------------------------|------------|
| 11.8 Summary . . . . .                                                         | 125        |
| 11.9 Exercises . . . . .                                                       | 126        |
| <b>12. Analog/Mixed-Signal, I/O and ESD Integration . . . . .</b>              | <b>127</b> |
| 12.1 The analog blocks in a typical SoC . . . . .                              | 127        |
| 12.2 PLL fundamentals . . . . .                                                | 127        |
| 12.3 The AMS design flow . . . . .                                             | 129        |
| 12.4 AMS verification at the SoC level . . . . .                               | 129        |
| 12.5 Integration: analog-on-top vs digital-on-top . . . . .                    | 130        |
| 12.6 Noise isolation . . . . .                                                 | 131        |
| 12.7 I/O cells . . . . .                                                       | 131        |
| 12.8 ESD: models and protection networks . . . . .                             | 132        |
| 12.9 Summary . . . . .                                                         | 134        |
| 12.10 Exercises . . . . .                                                      | 134        |
| <b>13. Physical Design Overview: Data, Flows and Design Planning . . . . .</b> | <b>136</b> |
| 13.1 From netlist to GDSII . . . . .                                           | 136        |
| 13.2 Inputs: the data PD needs . . . . .                                       | 136        |
| 13.3 Outputs and databases . . . . .                                           | 137        |
| 13.4 Flat vs hierarchical implementation . . . . .                             | 138        |
| 13.5 Design planning and die-size estimation . . . . .                         | 140        |
| 13.6 MCMM: multi-corner, multi-mode setup . . . . .                            | 141        |
| 13.7 The PD tool landscape . . . . .                                           | 142        |
| 13.8 Teams and hand-offs . . . . .                                             | 143        |
| 13.9 Summary . . . . .                                                         | 143        |
| 13.10 Exercises . . . . .                                                      | 143        |
| <b>14. Floorplanning, Power Planning and the I/O Ring . . . . .</b>            | <b>144</b> |
| 14.1 Die, core, rows and sites . . . . .                                       | 144        |
| 14.2 The I/O ring . . . . .                                                    | 144        |
| 14.3 Macro placement . . . . .                                                 | 145        |
| 14.4 Power delivery network design . . . . .                                   | 146        |
| 14.5 Power domains and power switches in the floorplan . . . . .               | 149        |
| 14.6 Well taps, endcaps and physical-only cells . . . . .                      | 149        |
| 14.7 Pin assignment . . . . .                                                  | 149        |
| 14.8 A floorplan script . . . . .                                              | 150        |
| 14.9 Floorplan quality checks . . . . .                                        | 150        |
| 14.10 Summary . . . . .                                                        | 151        |
| 14.11 Exercises . . . . .                                                      | 151        |

## **PART IV - Implementation and Sign-off 152**

|                                                                   |            |
|-------------------------------------------------------------------|------------|
| <b>15. Placement and Optimisation . . . . .</b>                   | <b>153</b> |
| 15.1 The placement problem . . . . .                              | 153        |
| 15.2 The placement flow: global, legalisation, detailed . . . . . | 156        |



|                                                                                               |            |
|-----------------------------------------------------------------------------------------------|------------|
| 15.3 Timing-driven and congestion-driven placement . . . . .                                  | 157        |
| 15.4 Pre-CTS optimisation . . . . .                                                           | 158        |
| 15.5 Scan-chain reordering . . . . .                                                          | 159        |
| 15.6 Spare cells, tie cells and density screens . . . . .                                     | 159        |
| 15.7 Placement quality metrics and the go/no-go for CTS . . . . .                             | 160        |
| 15.8 Summary . . . . .                                                                        | 161        |
| 15.9 Exercises . . . . .                                                                      | 161        |
| <b>16. Clock Tree Synthesis and Clock Distribution . . . . .</b>                              | <b>162</b> |
| 16.1 Clock tree goals and the vocabulary . . . . .                                            | 162        |
| 16.2 Clock tree structures . . . . .                                                          | 162        |
| 16.3 The CTS flow . . . . .                                                                   | 165        |
| 16.4 Clock gating inside the tree . . . . .                                                   | 166        |
| 16.5 OCV, CPPR, jitter and duty cycle . . . . .                                               | 167        |
| 16.6 Generated clocks, clock muxes and multiple clocks . . . . .                              | 167        |
| 16.7 Clock power . . . . .                                                                    | 168        |
| 16.8 Post-CTS optimisation and hold fixing . . . . .                                          | 168        |
| 16.9 Summary . . . . .                                                                        | 170        |
| 16.10 Exercises . . . . .                                                                     | 170        |
| <b>17. Routing, Signal Integrity and DFM . . . . .</b>                                        | <b>171</b> |
| 17.1 Routing layers, tracks and preferred directions . . . . .                                | 171        |
| 17.2 Global routing, track assignment and detailed routing . . . . .                          | 171        |
| 17.3 Vias, redundant vias and non-default rules . . . . .                                     | 174        |
| 17.4 Crosstalk: coupling, delta delay and glitches . . . . .                                  | 174        |
| 17.5 Antenna effect . . . . .                                                                 | 176        |
| 17.6 Electromigration on signal nets . . . . .                                                | 176        |
| 17.7 Design for manufacturability (DFM) . . . . .                                             | 177        |
| 17.8 Post-route optimisation . . . . .                                                        | 177        |
| 17.9 RC extraction and SPEF . . . . .                                                         | 178        |
| 17.10 Summary . . . . .                                                                       | 179        |
| 17.11 Exercises . . . . .                                                                     | 179        |
| <b>18. Sign-off: Timing, Power Integrity, Reliability and Physical Verification . . . . .</b> | <b>180</b> |
| 18.1 Timing sign-off . . . . .                                                                | 180        |
| 18.2 Power integrity: IR drop and voltage-drop-aware timing . . . . .                         | 181        |
| 18.3 Reliability sign-off: EM, self-heating and aging . . . . .                               | 182        |
| 18.4 Physical verification: DRC . . . . .                                                     | 184        |
| 18.5 Physical verification: LVS . . . . .                                                     | 185        |
| 18.6 ERC, antenna and density checks . . . . .                                                | 187        |
| 18.7 Formal equivalence checking . . . . .                                                    | 187        |
| 18.8 Low-power sign-off (UPF static checks) . . . . .                                         | 188        |
| 18.9 The sign-off criteria checklist . . . . .                                                | 188        |
| 18.10 Summary . . . . .                                                                       | 189        |

|                                                                     |            |
|---------------------------------------------------------------------|------------|
| 18.11 Exercises . . . . .                                           | 189        |
| <b>19. ECOs, Tapeout and Mask Making . . . . .</b>                  | <b>190</b> |
| 19.1 Why ECOs, and what kinds exist . . . . .                       | 190        |
| 19.2 Functional ECO flow . . . . .                                  | 190        |
| 19.3 Timing ECOs . . . . .                                          | 191        |
| 19.4 Metal-only ECOs, spare cells and ECO cells . . . . .           | 191        |
| 19.5 The tapeout process . . . . .                                  | 192        |
| 19.6 Mask data preparation: OPC, RET, SRAF and fracturing . . . . . | 194        |
| 19.7 MPW shuttles . . . . .                                         | 195        |
| 19.8 Reticles, dies per wafer and cost per die . . . . .            | 195        |
| 19.9 From tapeout to first silicon . . . . .                        | 196        |
| 19.10 Summary . . . . .                                             | 197        |
| 19.11 Exercises . . . . .                                           | 197        |

## **PART V - After Tapeout 198**

|                                                                          |            |
|--------------------------------------------------------------------------|------------|
| <b>20. Packaging, 2.5D/3D Integration and Chiplets . . . . .</b>         | <b>199</b> |
| 20.1 What a package does . . . . .                                       | 199        |
| 20.2 Package families . . . . .                                          | 199        |
| 20.3 Wire bond versus flip chip . . . . .                                | 200        |
| 20.4 Substrates, RDL and the package design flow . . . . .               | 201        |
| 20.5 2.5D integration: interposers and bridges . . . . .                 | 201        |
| 20.6 3D integration: TSVs and hybrid bonding . . . . .                   | 202        |
| 20.7 Chiplets, UCIe and HBM . . . . .                                    | 202        |
| 20.8 Package-chip co-design: bumps, power and signal integrity . . . . . | 203        |
| 20.9 Thermal design . . . . .                                            | 205        |
| 20.10 Package reliability and qualification . . . . .                    | 206        |
| 20.11 Summary . . . . .                                                  | 206        |
| 20.12 Exercises . . . . .                                                | 207        |
| <b>21. Manufacturing Test, Yield and Reliability . . . . .</b>           | <b>208</b> |
| 21.1 The test flow: from wafer to shipped part . . . . .                 | 208        |
| 21.2 ATE architecture and the test program . . . . .                     | 208        |
| 21.3 Test time and test cost . . . . .                                   | 209        |
| 21.4 Binning, speed binning and adaptive test . . . . .                  | 210        |
| 21.5 Burn-in and system-level test . . . . .                             | 210        |
| 21.6 Yield: defect density and yield models . . . . .                    | 210        |
| 21.7 Test quality: coverage and DPPM . . . . .                           | 212        |
| 21.8 Reliability physics . . . . .                                       | 213        |
| 21.9 The bathtub curve, FIT and MTBF . . . . .                           | 214        |
| 21.10 Accelerated testing and the Arrhenius model . . . . .              | 214        |
| 21.11 Qualification: JEDEC, ESD, latch-up and AEC-Q100 . . . . .         | 216        |
| 21.12 Summary . . . . .                                                  | 216        |

|                                                                                       |            |
|---------------------------------------------------------------------------------------|------------|
| 21.13 Exercises . . . . .                                                             | 217        |
| <b>22. Silicon Bring-up, Validation, Characterisation and Qualification . . . . .</b> | <b>218</b> |
| 22.1 Preparing for silicon: the bring-up plan . . . . .                               | 218        |
| 22.2 First power-on and smoke tests . . . . .                                         | 219        |
| 22.3 Post-silicon validation . . . . .                                                | 219        |
| 22.4 Debug infrastructure: seeing inside the silicon . . . . .                        | 220        |
| 22.5 Characterisation: shmoo plots, Vmin and Fmax . . . . .                           | 220        |
| 22.6 Managing validation: bug tracking and exit criteria . . . . .                    | 224        |
| 22.7 Errata, workarounds and respins . . . . .                                        | 224        |
| 22.8 From samples to production: ramp, qualification and datasheets . . . . .         | 225        |
| 22.9 Summary . . . . .                                                                | 225        |
| 22.10 Exercises . . . . .                                                             | 225        |
| <b>23. Methodology, Project Management and the Economics of an ASIC . . . . .</b>     | <b>226</b> |
| 23.1 Project phases and milestones . . . . .                                          | 226        |
| 23.2 Gating reviews . . . . .                                                         | 226        |
| 23.3 Team structure and headcount . . . . .                                           | 227        |
| 23.4 Schedule risks . . . . .                                                         | 228        |
| 23.5 The EDA tool landscape and licensing . . . . .                                   | 228        |
| 23.6 Compute and infrastructure . . . . .                                             | 229        |
| 23.7 Methodology: flows, scripts, CI and version control . . . . .                    | 229        |
| 23.8 The cost model: NRE . . . . .                                                    | 230        |
| 23.9 The cost model: unit cost and cost per good die . . . . .                        | 231        |
| 23.10 Make versus buy . . . . .                                                       | 232        |
| 23.11 Risk management . . . . .                                                       | 232        |
| 23.12 Functional safety and security certification . . . . .                          | 233        |
| 23.13 Summary . . . . .                                                               | 234        |
| 23.14 Exercises . . . . .                                                             | 234        |

## PART VI - Practice and Roadmap

**235**

|                                                                                        |            |
|----------------------------------------------------------------------------------------|------------|
| <b>24. The Open-Source ASIC Flow: Yosys, OpenROAD/OpenLane and Open PDKs . . . . .</b> | <b>236</b> |
| 24.1 Why an open flow matters, and who uses it . . . . .                               | 236        |
| 24.2 A short history: from closed PDKs to open silicon . . . . .                       | 236        |
| 24.3 The open PDKs . . . . .                                                           | 237        |
| 24.4 The tools and what each one does . . . . .                                        | 241        |
| 24.5 The flows: OpenLane, OpenLane 2 / LibreLane and ORFS . . . . .                    | 242        |
| 24.6 Run: synthesis with Yosys on the real SKY130 library . . . . .                    | 244        |
| 24.7 Tiny Tapeout, shuttles and getting a design onto silicon . . . . .                | 245        |
| 24.8 Installing the flow . . . . .                                                     | 245        |
| 24.9 Open-source versus commercial: an honest gap analysis . . . . .                   | 246        |
| 24.10 Summary . . . . .                                                                | 247        |
| 24.11 Exercises . . . . .                                                              | 247        |

|                                                                      |            |
|----------------------------------------------------------------------|------------|
| <b>25. Case Study: A Block from RTL to GDSII</b>                     | <b>248</b> |
| 25.1 Step 1 - Specification                                          | 248        |
| 25.2 Step 2 - Micro-architecture and RTL                             | 249        |
| 25.3 Step 3 - Simulation                                             | 250        |
| 25.4 Step 4 - Lint                                                   | 252        |
| 25.5 Step 5 - Synthesis on SKY130                                    | 253        |
| 25.6 Step 6 - Constraints (SDC)                                      | 255        |
| 25.7 Step 7 - Timing across corners (pre-layout estimate)            | 255        |
| 25.8 Step 8 - Floorplan arithmetic                                   | 257        |
| 25.9 Step 9 - Power estimate                                         | 258        |
| 25.10 Step 10 - Place, CTS and route (commands, not run)             | 259        |
| 25.11 Step 11 - Sign-off and the GDSII hand-off (checklist, not run) | 260        |
| 25.12 What the case study teaches                                    | 261        |
| 25.13 Summary                                                        | 261        |
| 25.14 Exercises                                                      | 262        |
| <b>26. The ASIC Career Roadmap and Interview Preparation</b>         | <b>263</b> |
| 26.1 The roles on an ASIC team                                       | 263        |
| 26.2 Skills tables by role                                           | 264        |
| 26.3 Where ASIC engineers work                                       | 264        |
| 26.4 A staged roadmap: student to principal                          | 265        |
| 26.5 A 12-month study plan                                           | 265        |
| 26.6 Portfolio projects that get a resume read                       | 266        |
| 26.7 How ASIC interviews are structured                              | 267        |
| 26.8 Mini-problems by role, with model answers                       | 268        |
| 26.9 Soft skills that decide careers                                 | 269        |
| 26.10 Your first 90 days on an ASIC team                             | 269        |
| 26.11 Final checklist                                                | 270        |
| 26.12 Summary                                                        | 270        |
| 26.13 Exercises                                                      | 270        |

## **APPENDICES 272**

|                                                                   |            |
|-------------------------------------------------------------------|------------|
| <b>Appendix A. Sign-off Checklists and File Formats Reference</b> | <b>273</b> |
| How to use the sign-off checklists                                | 273        |
| Front-end gates                                                   | 273        |
| Back-end gates                                                    | 275        |
| Sign-off gates                                                    | 275        |
| File formats at a glance                                          | 277        |
| Gate-level Verilog netlist                                        | 277        |
| Liberty (.lib)                                                    | 279        |
| LEF (Library Exchange Format)                                     | 281        |
| DEF (Design Exchange Format)                                      | 284        |

|                                                                               |            |
|-------------------------------------------------------------------------------|------------|
| SDC (Synopsys Design Constraints) . . . . .                                   | 285        |
| SPEF (Standard Parasitic Exchange Format) . . . . .                           | 286        |
| SDF (Standard Delay Format) . . . . .                                         | 287        |
| UPF (Unified Power Format, IEEE 1801) . . . . .                               | 288        |
| VCD (Value Change Dump) and SAIF (Switching Activity) . . . . .               | 289        |
| GDSII and OASIS . . . . .                                                     | 291        |
| Tcl in EDA tools . . . . .                                                    | 292        |
| <b>Appendix B. Glossary . . . . .</b>                                         | <b>293</b> |
| <b>Appendix C. 100 ASIC Design Interview Questions with Answers . . . . .</b> | <b>299</b> |
| Device physics and technology . . . . .                                       | 299        |
| CMOS circuits and logic . . . . .                                             | 300        |
| Logic synthesis . . . . .                                                     | 300        |
| Static timing analysis . . . . .                                              | 301        |
| Low power . . . . .                                                           | 302        |
| Design for test . . . . .                                                     | 303        |
| Floorplanning and power grid . . . . .                                        | 304        |
| Placement and clock tree synthesis . . . . .                                  | 304        |
| Routing, signal integrity and DFM . . . . .                                   | 305        |
| Sign-off and physical verification . . . . .                                  | 306        |
| Packaging, manufacturing test, yield and reliability . . . . .                | 307        |
| Project, methodology and economics . . . . .                                  | 307        |
| Puzzles . . . . .                                                             | 308        |
| <b>Appendix D. Standards, Tools and Resources . . . . .</b>                   | <b>309</b> |
| Design-language and design-data standards . . . . .                           | 309        |
| Test, reliability and safety standards . . . . .                              | 309        |
| Commercial EDA tools by flow step . . . . .                                   | 310        |
| Open-source tools . . . . .                                                   | 311        |
| Foundries, open PDKs and prototyping routes . . . . .                         | 311        |
| Books . . . . .                                                               | 312        |
| Courses and communities . . . . .                                             | 312        |

## PART I

# Foundations

What an ASIC is and why companies build one; the economics and the end-to-end flow from an idea to a shipping part; the physics of the MOSFET and the CMOS gate that set every delay and every milliwatt; how wafers are actually made and what a PDK hands you; the standard cells, Liberty and LEF files and IP blocks you build with; and how a specification becomes an architecture with PPA targets.

# 1. The ASIC Landscape: What an ASIC Is, Design Styles, Economics and the End-to-End Flow

An **application-specific integrated circuit** (ASIC) is a chip designed to do one job, or one family of jobs, for one product line: the image signal processor in a phone camera, the controller in an SSD, a bitcoin miner, a 5G baseband, the neural accelerator in a car. It is the opposite of a general-purpose part such as an off-the-shelf microcontroller or an FPGA, which is designed once and sold to thousands of customers who program it for their own needs.

Building an ASIC is one of the most capital-intensive and schedule-critical activities in engineering. A modern project involves architects, RTL designers, verification engineers, DFT, physical design, analog, package, test, software and product engineers; it consumes millions of dollars of tool licences, IP and masks before the first chip exists; and a single escaped bug can cost a re-spin of several months. This chapter draws the whole map: what an ASIC and an SoC are, the ways a chip can be built, when building one makes economic sense, who does what in the industry, and the end-to-end flow that the rest of this book walks through step by step.

## 1.1 ASIC, SoC, ASSP: the vocabulary

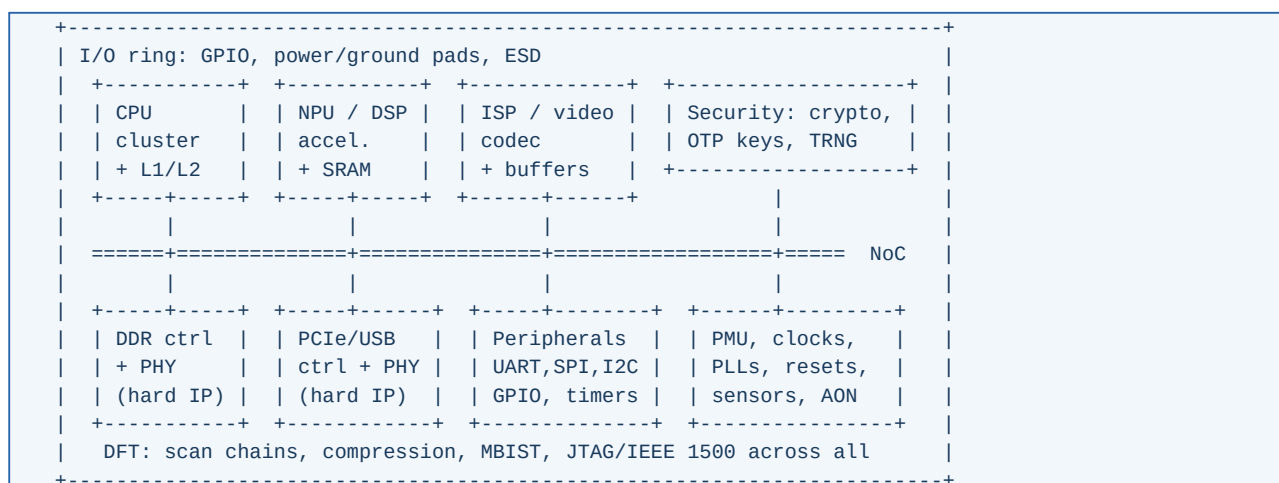
The terms overlap and are used loosely, so it pays to be precise:

| Term                        | Meaning                                                                                                                                                                                 | Example                                              |
|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------|
| <b>ASIC</b>                 | A chip designed for a specific application, usually for one customer or one product line. Strictly, the term says nothing about complexity: a 10k-gate sensor interface is an ASIC too. | An automotive radar front-end controller             |
| <b>ASSP</b>                 | Application-specific standard product: application-specific, but sold on the open market to many customers.                                                                             | A USB-to-Ethernet bridge chip, a Wi-Fi combo chip    |
| <b>SoC</b>                  | System-on-chip: an ASIC or ASSP that integrates processors, memories, interconnect, peripherals and often analog/RF on one die, so that it runs software and forms a whole system.      | A phone application processor; an edge-AI camera SoC |
| <b>Custom silicon / XPU</b> | In-house ASICs built by system companies (hyperscalers, phone makers, carmakers) for their own products.                                                                                | Datacentre AI accelerators, video transcoders        |
| <b>Chiplet</b>              | A die designed to be packaged together with other dies (Chapter 20) rather than as a stand-alone chip.                                                                                  | CPU compute die plus separate I/O die                |

In this book 'ASIC' means any chip that goes through the cell-based implementation flow described here - which covers almost every digital or mixed-signal SoC made today.

### Anatomy of a typical SoC

Every SoC contains the same broad ingredients: processing (CPU, DSP, accelerators), memory (on-chip SRAM plus an external DRAM interface), an interconnect (buses or a network-on-chip), high-speed and low-speed peripherals, infrastructure (clocks, resets, power management, always-on logic, sensors), security, and test/debug logic. The differentiating block - the thing that justifies the chip - is often a small fraction of the area; the rest is infrastructure that must nevertheless be integrated, verified and signed off with the same rigour.



Block diagram of a representative SoC. Soft IP, in-house RTL, compiled memories and hard analog/mixed-signal IP all meet on one die

## 1.2 Design styles: from full custom to FPGA

There are several ways to turn a logic design into a physical chip. They differ in how many mask layers are unique to your design (and therefore in non-recurring engineering cost, NRE), how long it takes, and how dense, fast and power-efficient the result is.

## Full custom

Every transistor is drawn by hand (or by a generator) and every wire is placed deliberately. All mask layers are unique. This gives the highest density and speed, but costs enormous engineering effort. Today full custom is used for things that are replicated millions of times or that are extremely performance-critical: SRAM bitcells, register files, high-speed SerDes, analog blocks, the datapaths of flagship CPUs, and the standard cells themselves.

## Standard cell (cell-based) design

The dominant style for digital logic. A library of pre-designed, pre-characterised gates (standard cells, Chapter 4) of equal height is placed in rows and connected by automatic place-and-route tools. All masks are still unique to the design, so NRE is the same order as full custom, but engineering effort is far lower because the design is described in RTL and implemented largely by EDA tools. Nearly everything in this book is about the standard-cell flow.

## Gate array and structured ASIC

A **gate array** pre-fabricates wafers with a regular sea of transistors (the 'base' layers) and only the top metal layers are customised per design. Fewer unique masks means lower NRE and a faster turnaround (weeks instead of months from tapeout to parts), at the cost of density and speed. A **structured ASIC** extends the idea: the base contains pre-built logic cells, memories, PLLs and I/Os, and only a few via/metal layers are customised. Structured ASICs were popular as an FPGA-to-ASIC conversion path; they survive in niche offerings (e.g. via-programmable platforms) but have largely been squeezed out by cheaper FPGAs and mature-node cell-based ASICs.

## FPGA and eFPGA

A **field-programmable gate array** is a standard product with configurable logic blocks (look-up tables and flip-flops), DSP blocks, block RAM and programmable routing, configured by a bitstream. There is zero silicon NRE and a design can be changed in minutes, but the per-unit price is high and the fabric is typically an order of magnitude or more worse than an ASIC in area, several times worse in dynamic power, and noticeably slower for the same logic, because most of the silicon is programmable routing and configuration memory. An **embedded FPGA** (eFPGA) is an FPGA fabric delivered as a hard IP block inside an ASIC, used where part of



the logic must stay changeable after tapeout (evolving protocols, customer-specific accelerators, security patches).

| Style                  | Custom masks    | NRE        | Time to parts                                                                   | Density / speed / power               | Typical use                                                        |
|------------------------|-----------------|------------|---------------------------------------------------------------------------------|---------------------------------------|--------------------------------------------------------------------|
| <b>Full custom</b>     | All             | Highest    | Months                                                                          | Best                                  | Bitcells, SerDes, analog, critical datapaths, the cells themselves |
| <b>Standard cell</b>   | All             | High       | Months (tapeout -> parts ~2-4 months in mature nodes, longer in advanced nodes) | Very good                             | Almost all digital ASICs and SoCs                                  |
| <b>Gate array</b>      | Metal only      | Medium     | Weeks                                                                           | Moderate                              | Legacy, rad-hard, quick-turn                                       |
| <b>Structured ASIC</b> | A few via/metal | Low-medium | Weeks                                                                           | Moderate                              | FPGA conversion, niche volumes                                     |
| <b>FPGA</b>            | None            | None       | Minutes                                                                         | Worst (large area and power overhead) | Prototyping, low volume, fast-changing standards                   |
| <b>eFPGA in ASIC</b>   | (part of ASIC)  | IP licence | -                                                                               | FPGA-like inside the block            | Post-silicon programmability                                       |

Design styles compared. Ratios vary by generation and design; the ordering is what matters.

#### MOST REAL SOCS ARE A MIX OF STYLES

A typical SoC is cell-based digital logic, plus compiler-generated SRAMs (full-custom bitcells tiled by a generator), plus hard IP (PLLs, SerDes, DDR PHYs, ADCs) that were designed full custom by specialists, plus a custom I/O ring. Integrating these pieces correctly is a large part of ASIC engineering.

## 1.3 When does building an ASIC make sense?

An ASIC is justified when it wins on at least one of: **unit cost** at volume, **power** (battery life, thermals, datacentre electricity), **performance** (functions no FPGA or CPU can reach), **form factor** (one chip instead of a board), or **differentiation and security** (IP that competitors cannot buy). The cost argument is the one that can be computed, and it is the classic interview question.

### NRE versus unit cost

Total cost for a volume V is simply  $NRE + V \times (\text{unit cost})$ . **NRE** includes engineering salaries, EDA licences, IP licences, the mask set, prototype wafers, package and test development, and qualification. **Unit cost** includes the die (wafer cost divided by good dies), package, assembly, test time and yield loss. An FPGA has almost no NRE and a high unit price; an ASIC has a large NRE and a low unit price; an advanced-node ASIC has an even larger NRE.

$$\text{Total}(V) = NRE + V * C_{\text{unit}}$$

$$\text{Break-even volume } V^* = (NRE_{\text{ASIC}} - NRE_{\text{FPGA}}) / (C_{\text{unit,FPGA}} - C_{\text{unit,ASIC}})$$

Cost model and break-even volume between two implementation options.

```
# Total cost of ownership: FPGA vs ASIC in a mature node vs ASIC in an advanced node.
# All numbers are illustrative round figures in US dollars.
options = {
    # name                NRE (design+masks+IP)    unit cost (die+package+test)
    "FPGA (mid-range)": ( 0.3e6,                120.0),
    "ASIC 28 nm":      ( 15.0e6,                6.0),
    "ASIC 5 nm":       ( 120.0e6,               9.0),    # bigger masks, pricier wafers
}
```

```
def total(nre, unit, volume):
    return nre + unit * volume

def breakeven(a, b):
    # volume where option a and option b cost the same (None if never)
    (n1, u1), (n2, u2) = options[a], options[b]
    return None if u1 == u2 else (n2 - n1) / (u1 - u2)

print("volume      " + "".join("%18s" % k for k in options))
for vol in (1e4, 1e5, 1e6, 1e7):
    row = "".join("%16.1f M" % (total(*options[k], vol) / 1e6) for k in options)
    print("%-10.0e%s" % (vol, row))
v = breakeven("FPGA (mid-range)", "ASIC 28 nm")
print("FPGA vs 28 nm ASIC break-even : %9.0f units" % v)
print("per-unit cost at 1M units, 28 nm: $%.2f" % (total(*options["ASIC 28 nm"], 1e6) / 1e6))
print("per-unit cost at 1M units, 5 nm : $%.2f" % (total(*options["ASIC 5 nm"], 1e6) / 1e6))
```

breakeven.py - total cost of three options (illustrative round numbers, not quotes).

| volume                                       | FPGA (mid-range) | ASIC 28 nm | ASIC 5 nm |
|----------------------------------------------|------------------|------------|-----------|
| 1e+04                                        | 1.5 M            | 15.1 M     | 120.1 M   |
| 1e+05                                        | 12.3 M           | 15.6 M     | 120.9 M   |
| 1e+06                                        | 120.3 M          | 21.0 M     | 129.0 M   |
| 1e+07                                        | 1200.3 M         | 75.0 M     | 210.0 M   |
| FPGA vs 28 nm ASIC break-even : 128947 units |                  |            |           |
| per-unit cost at 1M units, 28 nm: \$21.00    |                  |            |           |
| per-unit cost at 1M units, 5 nm : \$129.00   |                  |            |           |

Real output of breakeven.py.

With these (illustrative) numbers the 28 nm ASIC beats the FPGA above roughly 130k units, while the 5 nm ASIC never makes sense for *this* product: its NRE is so large that only a design that genuinely needs 5 nm density, speed or power - and sells in the millions or tens of millions - can recover it. In practice the advanced node is chosen for performance per watt (a datacentre accelerator, a phone SoC), not because it makes a simple chip cheaper.

#### PITFALL: COMPARING UNIT PRICES ONLY

Teams often compare the FPGA's price with the ASIC die cost and forget NRE, schedule and risk. Include: the probability of a re-spin (a second mask set and 3-6 months), the cost of delay to market, the engineering team for 18-30 months, and the qualification cost. Conversely, FPGA-based products often forget the board cost, the configuration flash and the power supply rails an FPGA needs.

#### INTERVIEW INSIGHT: THE THREE QUESTIONS BEHIND 'ASIC OR NOT?'

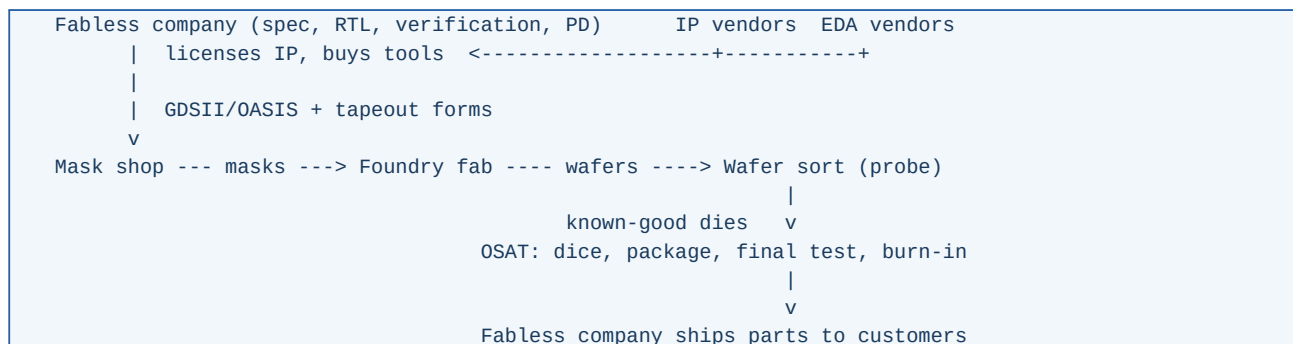
1) What volume and over how many years? 2) Is there a hard PPA requirement an FPGA or merchant chip cannot meet (power in a battery device, throughput per watt in a datacentre)? 3) Is the specification stable enough to freeze in silicon for the product lifetime? A 'no' to all three means do not build an ASIC.

## 1.4 The industry: fabless, foundry, IDM, OSAT and the EDA/IP ecosystem

Very few companies do everything themselves. The modern semiconductor industry is a layered supply chain:

| Player                                      | What they do                                                                                            | Examples of the category                                         |
|---------------------------------------------|---------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| <b>IDM (integrated device manufacturer)</b> | Design and manufacture their own chips in their own fabs (often also offering foundry services).        | Intel, Samsung, Texas Instruments, Infineon                      |
| <b>Foundry</b>                              | Manufacture wafers for others; publish a PDK (Chapter 3) and design rules; do not sell their own chips. | TSMC, GlobalFoundries, UMC, SMIC, Samsung Foundry, Intel Foundry |

| Player                                      | What they do                                                                                     | Examples of the category                                   |
|---------------------------------------------|--------------------------------------------------------------------------------------------------|------------------------------------------------------------|
| <b>Fabless company</b>                      | Design chips, own the product and the customer; outsource all manufacturing.                     | NVIDIA, Qualcomm, AMD, Broadcom, MediaTek, most start-ups  |
| <b>System company with in-house silicon</b> | Designs chips for its own products, usually fabless.                                             | Apple, Google, Amazon, Tesla                               |
| <b>Design-services house / ASIC vendor</b>  | Takes a customer's RTL or spec through physical design, tapeout and production ('turnkey' ASIC). | Various; many foundries have partner ecosystems            |
| <b>IP vendor</b>                            | Licenses pre-designed blocks: CPUs, interfaces, PHYs, memory compilers, standard-cell libraries. | Arm, Synopsys, Cadence, Rambus, Alphawave                  |
| <b>EDA vendor</b>                           | Sells the design and verification tools.                                                         | Synopsys, Cadence, Siemens EDA; open-source Yosys/OpenROAD |
| <b>OSAT</b>                                 | Outsourced semiconductor assembly and test: packaging and production test (Chapters 20-21).      | ASE, Amkor, JCET                                           |
| <b>Mask shop</b>                            | Writes the photomasks from the tapeout data (Chapter 19).                                        | Captive foundry mask shops, Photonics, Toppan              |



The fabless supply chain: design data flows down, parts flow back.

## 1.5 The end-to-end ASIC flow

The flow below is the backbone of this book. Each box is a stage with an owning team, inputs, outputs (the 'hand-off' files) and a sign-off criterion. Arrows back upward - timing or area that cannot be met, bugs found late, power over budget - are the iterations that dominate real schedules.

| STAGE                                            | OWNER                    | HAND-OFF / KEY OUTPUTS                                                 |
|--------------------------------------------------|--------------------------|------------------------------------------------------------------------|
| 1 Product requirements                           | Marketing/PM             | MRD, PRD (features, cost, power)                                       |
|                                                  |                          |                                                                        |
| 2 Architecture                                   | Architects               | Arch spec, perf model, PPA target                                      |
|                                                  |                          |                                                                        |
| 3 Micro-architecture                             | Design leads             | uArch specs, interface specs                                           |
|                                                  |                          |                                                                        |
| 4 RTL design                                     | RTL designers            | Verilog/SV RTL, SDC, UPF, lint                                         |
| <-----+----->                                    |                          |                                                                        |
| 5 Functional verification                        | DV engineers             | Testbench, coverage, sign-off rpt<br>  bugs                            |
|                                                  |                          |                                                                        |
| 6 Logic synthesis                                | Synthesis/PD             | Gate netlist, reports, SDC                                             |
|                                                  |                          |                                                                        |
| 7 DFT insertion + ATPG                           | DFT engineers            | Scan netlist, test protocol, pats                                      |
|                                                  |                          |                                                                        |
| 8 Floorplan + power plan                         | Physical design          | DEF/ODB floorplan, PG grid, pins                                       |
|                                                  |                          |                                                                        |
| 9 Placement + optimisation                       | Physical design          | Placed DB, timing/congestion                                           |
|                                                  |                          |                                                                        |
| 10 Clock tree synthesis                          | PD / clock team          | Clock tree, skew/latency reports                                       |
|                                                  |                          |                                                                        |
| 11 Routing                                       | Physical design          | Routed DB, SPEF (parasitics)                                           |
| <-----+----->                                    |                          |                                                                        |
| 12 Sign-off: STA, IR/EM,<br>DRC/LVS, equivalence | STA, PI, PV<br>engineers | Clean STA, IR/EM, DRC/LVS/ERC,<br>LEC reports; ECOs if needed<br>  ECO |
|                                                  |                          |                                                                        |
| 13 Tapeout                                       | PD + mgmt                | GDSII/OASIS, tapeout checklist                                         |
|                                                  |                          |                                                                        |
| 14 Mask making + fab                             | Foundry                  | Masks, wafers (8-16+ weeks)                                            |
|                                                  |                          |                                                                        |
| 15 Wafer sort + packaging                        | Test/OSAT                | Probe results, packaged parts                                          |
|                                                  |                          |                                                                        |
| 16 Final test + bring-up                         | Test, validation         | Test program, first-silicon rpt                                        |
|                                                  |                          |                                                                        |
| 17 Char., qual, production                       | Product eng.             | Datasheet, qual report, PPAP                                           |

The end-to-end ASIC flow with owners and hand-offs. Stages 4-12 run with heavy overlap and iteration.

The front end (stages 2-7, Part II of this book) turns a specification into a verified gate-level netlist. The back end (stages 8-13, Parts III and IV) turns the netlist into polygons on masks. Post-silicon (stages 14-17, Part V) turns masks into qualified, tested parts in customers' hands.

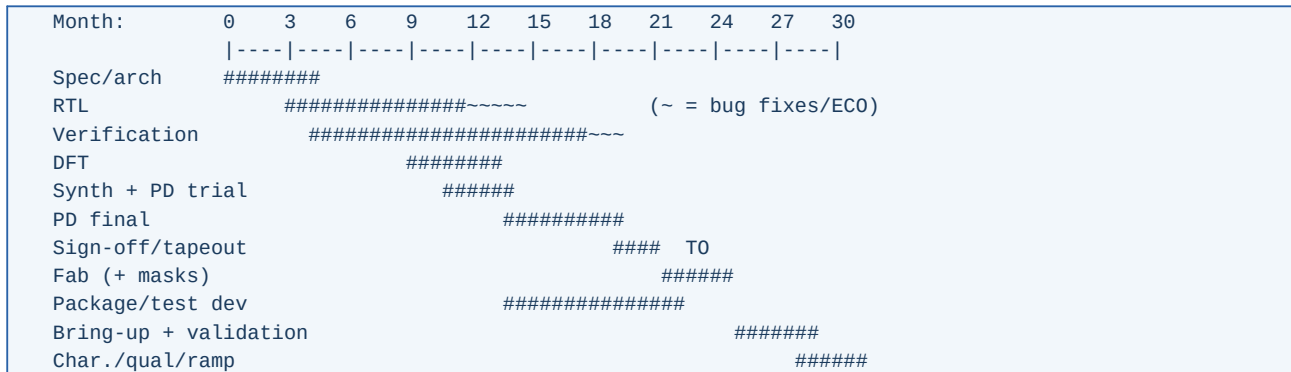
### The files that move between teams

| File / format                    | What it carries                                       | Produced by -> used by                     |
|----------------------------------|-------------------------------------------------------|--------------------------------------------|
| <b>Verilog/SystemVerilog RTL</b> | Behaviour at register-transfer level                  | RTL -> DV, synthesis, DFT, emulation       |
| <b>SDC</b>                       | Timing constraints: clocks, I/O delays, exceptions    | RTL/arch -> synthesis, PD, STA             |
| <b>UPF (IEEE 1801)</b>           | Power intent: domains, switches, isolation, retention | Arch/RTL -> DV, synthesis, PD, sign-off    |
| <b>Liberty (.lib/.db)</b>        | Cell timing, power, function                          | Library/IP vendor -> synthesis, STA, power |
| <b>LEF</b>                       | Abstract cell/macro geometry, routing layers          | Library/IP vendor -> PD                    |
| <b>Gate-level netlist (.v)</b>   | Cells and nets                                        | Synthesis/DFT -> PD, LEC, gate-level sim   |
| <b>DEF / OpenDB / tool DB</b>    | Placement, routing, floorplan                         | PD -> PD, extraction                       |
| <b>SPEF</b>                      | Extracted parasitics (R, C)                           | Extraction -> STA, power                   |
| <b>SDF</b>                       | Annotated delays                                      | STA -> gate-level timing simulation        |
| <b>GDSII / OASIS</b>             | Final mask polygons                                   | PD -> PV, foundry                          |
| <b>STIL / WGL</b>                | Test patterns                                         | ATPG -> tester                             |

Hand-off files. Appendix A has a format reference.

## 1.6 What a typical project schedule looks like

Schedules vary enormously - a small mature-node mixed-signal chip can take 12 months with a team of ten, a flagship SoC 24-36 months with hundreds - but the shape is always the same: long front-end and verification, overlapping back-end, a fixed fab cycle and a bring-up/qualification tail.



An illustrative 30-month schedule for a mid-size SoC. The critical path is almost always verification -> sign-off -> fab.

### MILESTONES YOU WILL HEAR ABOUT

**Arch freeze / spec freeze:** features locked. **RTL freeze** (or 'RTL 0.9/1.0'): all features coded, only bug fixes after. **Netlist freeze:** only ECOs after. **Tapeout (TO):** GDSII sent to the foundry. **First silicon / A0:** first wafers back. **ES / CS / MP:** engineering samples, customer samples, mass production. Stepping names such as A0, A1, B0 identify silicon revisions: a metal-only fix usually bumps the number (A0 -> A1), a base-layer change bumps the letter (A1 -> B0).

## 1.7 Who does what: the roles in an ASIC team

| Role                            | Main responsibility                               | Key skills                                | Chapters    |
|---------------------------------|---------------------------------------------------|-------------------------------------------|-------------|
| <b>Architect</b>                | Features, performance/power targets, partitioning | Modelling, workloads, system trade-offs   | 1, 5, 23    |
| <b>RTL / design engineer</b>    | Micro-architecture and RTL of blocks              | SV, timing-aware RTL, CDC, low power      | 5, 6, 11    |
| <b>Design verification (DV)</b> | Prove the RTL matches the spec                    | UVM, SVA, coverage, formal                | 7           |
| <b>DFT engineer</b>             | Scan, ATPG, compression, MBIST, JTAG              | Test theory, fault models, test protocols | 10, 21      |
| <b>Synthesis / STA engineer</b> | Netlist, constraints, timing sign-off             | SDC, Liberty, OCV, corners                | 4, 8, 9, 18 |
| <b>Physical design (PD)</b>     | Floorplan to GDSII for blocks/top                 | P&R tools, CTS, routing, ECO              | 13-17, 19   |
| <b>Power integrity / PV</b>     | IR drop, EM, DRC, LVS, ERC                        | Extraction, rule decks, debugging layout  | 3, 18       |
| <b>Low-power / UPF</b>          | Power intent and its verification                 | IEEE 1801, power analysis                 | 11          |
| <b>Analog / mixed-signal</b>    | PLLs, ADCs, regulators, I/O, ESD                  | Circuit design, SPICE, layout             | 2, 12       |
| <b>Package / SI-PI</b>          | Package, substrate, signal/power integrity        | 3D EM, thermal                            | 20          |
| <b>Test / product engineer</b>  | Test programs, yield, qualification               | ATE, statistics, reliability              | 21, 22      |
| <b>Post-silicon validation</b>  | Bring-up, debug, characterisation                 | Lab, firmware, debug infrastructure       | 22          |
| <b>CAD / methodology</b>        | Flows, scripts, tool infrastructure               | Tcl/Python, EDA internals                 | 13, 23, 24  |

## 1.8 Roadmap: how this book is organised

| Part                                  | Chapters | What you learn                                                                                        | Most relevant to                      |
|---------------------------------------|----------|-------------------------------------------------------------------------------------------------------|---------------------------------------|
| <b>I Foundations</b>                  | 1-5      | Flow and economics, device physics, fabrication and PDKs, libraries and IP, spec to architecture      | Everyone                              |
| <b>II Front-End</b>                   | 6-10     | RTL for implementation, verification strategy, synthesis, STA, DFT                                    | RTL, DV, synthesis, STA, DFT          |
| <b>III Power, AMS, PD setup</b>       | 11-14    | UPF, mixed-signal/I/O/ESD, PD data and flows, floorplan and power grid                                | Low-power, AMS, PD                    |
| <b>IV Implementation and sign-off</b> | 15-19    | Placement, CTS, routing/SI, sign-off, ECO, tapeout and masks                                          | PD, STA, PV                           |
| <b>V After tapeout</b>                | 20-23    | Packaging and chiplets, test/yield/reliability, bring-up and qualification, methodology and economics | Test, product, validation, management |
| <b>VI Practice and roadmap</b>        | 24-26    | Open-source flow, full case study, career and interviews                                              | Everyone                              |

RTL coding style, SystemVerilog language details and on-chip protocols (AXI, APB, PCIe and friends) are covered in the companion RTL Design, Verilog & SystemVerilog and Communication Protocols guides; this book focuses on everything else and on how the whole flow fits together.

## 1.9 Summary

- An ASIC is a chip built for a specific application; an SoC integrates processors, memory, interconnect and peripherals into one die; an ASSP is an application-specific chip sold to many customers.
- Design styles trade NRE and turnaround against density, speed and power: full custom > standard cell > structured ASIC/gate array > FPGA. Almost every digital SoC is standard-cell logic plus custom/compiled macros and hard IP.
- The ASIC decision is driven by volume (NRE amortisation), hard PPA requirements and specification stability; compute the break-even volume  $V^* = dNRE / dC_{unit}$ , and include re-spin risk.
- The industry is layered: fabless designers, foundries, IDMs, IP and EDA vendors, mask shops and OSATs.
- The flow runs spec -> architecture -> RTL -> verification -> synthesis -> DFT -> floorplan -> place -> CTS -> route -> sign-off -> tapeout -> fab -> package -> test -> bring-up -> production, with well-defined owners and hand-off files (RTL, SDC, UPF, Liberty, LEF, netlist, DEF, SPEF, GDSII, test patterns).

## 1.10 Exercises

- 1 Using breakeven.py, find the break-even volume between the 28 nm ASIC and the FPGA if the ASIC needs one full re-spin (add \$3M to its NRE). How sensitive is the answer to the FPGA unit price?
- 2 List three products for which an FPGA is the right answer even at high volume, and explain why in terms of the three questions in Section 1.4.
- 3 For each stage of the flow diagram, name one thing that can force an iteration back to RTL.
- 4 Explain why a metal-only ECO is cheaper and faster than a base-layer change, and which stepping name each would produce.
- 5 A start-up has \$20M of funding. Sketch the NRE budget of a 28 nm mixed-signal SoC (engineering, EDA, IP, masks, package, test, qualification) and decide whether a re-spin is affordable.

## 2. Semiconductor Device Fundamentals: MOSFETs, CMOS, Delay, Power and Scaling

Every number an ASIC engineer works with - a cell delay in a Liberty table, a leakage figure in a power report, the supply voltage of a low-power mode, the reason a hold violation appears only at the fast corner - comes from the physics of the MOSFET. You do not need to be a device physicist, but you do need a correct mental model of how a transistor conducts, what makes it fast or leaky, and how that changed as transistors went from planar to FinFET to nanosheet. This chapter builds that model and connects it to the gate-level quantities used everywhere else in the book.

### 2.1 Silicon, doping and the pn junction

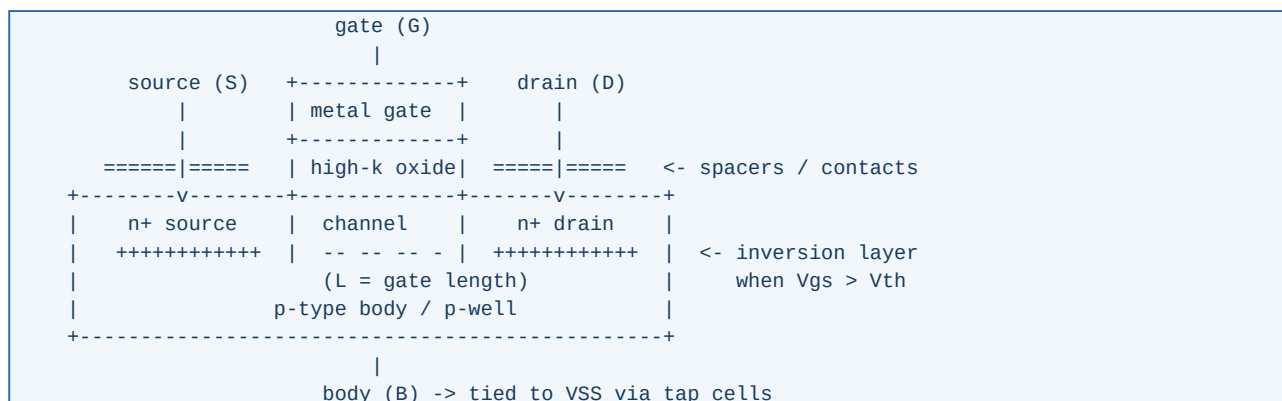
Silicon has four valence electrons and forms a diamond-cubic crystal. At room temperature pure (intrinsic) silicon has very few free carriers - about  $10^{10}$  per  $\text{cm}^3$  - so it is a poor conductor. Its **band gap** (about 1.12 eV) is the energy an electron needs to break free from a bond into the conduction band, leaving a mobile 'hole' behind.

**Doping** replaces a tiny fraction of silicon atoms with impurities. Group-V donors (phosphorus, arsenic) add a free electron: **n-type**. Group-III acceptors (boron) create a hole: **p-type**. Typical doping levels are  $10^{15}$  to  $10^{20}$  atoms/ $\text{cm}^3$ , i.e. from about one in ten million to one in a few hundred silicon atoms, and they change conductivity by many orders of magnitude.

Where p-type meets n-type, electrons and holes diffuse across and recombine, leaving a **depletion region** of fixed ionised dopants and a built-in potential (roughly 0.6-0.9 V). Forward bias shrinks the barrier and current grows exponentially (the diode equation  $I = I_s (e^{V/(n kT/q)} - 1)$ ); reverse bias widens the depletion region and only a small leakage flows. Every MOSFET contains two such junctions (source-body and drain-body) that must stay reverse biased - which is why the p-substrate is tied to the lowest supply and n-wells to the highest, through the **tap cells** of Chapter 4. The parasitic pnpn structure formed by neighbouring n- and p-MOSFETs can latch up into a low-impedance path if those taps are too far apart; the foundry's latch-up rules set the maximum tap spacing.

### 2.2 The MOSFET: structure and operation

A metal-oxide-semiconductor field-effect transistor has four terminals: **gate**, **source**, **drain** and **body** (bulk). In an nMOS device the source and drain are heavily n-doped regions in a p-type body. The gate sits on a thin insulator above the channel region between them. A positive gate voltage attracts electrons to the surface; above the **threshold voltage**  $V_{th}$  the surface inverts to n-type and forms a conducting channel from source to drain. A pMOS device is the complement: p+ source/drain in an n-well, conducting when the gate is pulled below the source by more than  $|V_{th}|$ .



Cross-section of a planar nMOS transistor. W (width) is into the page.

## Regions of operation

| Region                       | Condition (nMOS)                               | Behaviour                                                                 | Where it matters                                      |
|------------------------------|------------------------------------------------|---------------------------------------------------------------------------|-------------------------------------------------------|
| <b>Cutoff / subthreshold</b> | $V_{gs} < V_{th}$                              | Only weak diffusion current, exponential in $V_{gs}$                      | Leakage power, retention, sleep modes                 |
| <b>Linear (triode)</b>       | $V_{gs} > V_{th}, V_{ds} < V_{gs} - V_{th}$    | Acts like a voltage-controlled resistor                                   | On-resistance of a switch, pass gates, power switches |
| <b>Saturation</b>            | $V_{gs} > V_{th}, V_{ds} \geq V_{gs} - V_{th}$ | Current roughly independent of $V_{ds}$ (pinch-off / velocity saturation) | Charging and discharging loads: gate delay            |

## The square-law (long-channel) model

**Linear:**  $I_d = \mu C_{ox} (W/L) [ (V_{gs} - V_{th}) V_{ds} - V_{ds}^2 / 2 ]$

**Saturation:**  $I_d = (1/2) \mu C_{ox} (W/L) (V_{gs} - V_{th})^2 (1 + \lambda V_{ds})$

$\mu$  = carrier mobility,  $C_{ox} = \epsilon_{ox} / t_{ox}$  (gate capacitance per area),

$\lambda$  = channel-length modulation

Shockley square-law model. Accurate for long channels only.

Two consequences are worth memorising. Current is proportional to **W/L**, which is why wider transistors (or more fins) drive harder and why cells come in drive strengths x1, x2, x4... And electrons have roughly 2-3x the mobility of holes in bulk silicon, which is why planar pMOS devices were made wider than nMOS to balance rise and fall. (In FinFETs strain engineering narrows the gap, so modern cells often use equal fin counts.)

## Velocity saturation and the alpha-power law

In short channels the lateral field is so high that carriers reach a saturation velocity (about  $10^7$  cm/s in silicon) before the drain. Current then grows roughly linearly, not quadratically, with overdrive. Sakurai and Newton's **alpha-power law** captures this with a fitted exponent  $\alpha$  between 1 (fully velocity saturated) and 2 (square law); modern devices are typically around 1.1-1.4.

$I_{d,sat} = K (W/L) (V_{gs} - V_{th})^\alpha$   $1 < \alpha < 2$

$t_d \sim C_L * V_{dd} / I_{d,sat} \sim C_L * V_{dd} / (V_{dd} - V_{th})^\alpha$

Alpha-power law and the resulting gate-delay dependence on supply.

```
# Alpha-power law (Sakurai-Newton): Id_sat ~ (Vdd - Vth)^alpha, so
# t_d ~ C * Vdd / (Vdd - Vth)^alpha      E_switch = C * Vdd^2
VTH, ALPHA, VNOM = 0.35, 1.3, 0.80      # generic FinFET-like numbers

def delay(vdd, vth=VTH, a=ALPHA):
    return vdd / (vdd - vth) ** a

d0, e0 = delay(VNOM), VNOM ** 2
print(" Vdd  delay(norm)  energy(norm)  EDP(norm)")
for vdd in (0.50, 0.55, 0.60, 0.70, 0.80, 0.90, 1.00):
    d, e = delay(vdd) / d0, vdd ** 2 / e0
    print("%4.2f  %9.2f  %10.2f  %9.2f" % (vdd, d, e, d * e))
# Sensitivity: how much does a 30 mV Vth shift hurt at low vs nominal voltage?
for vdd in (0.55, 0.80):
    s = delay(vdd, VTH + 0.03) / delay(vdd) - 1
    print("Vdd=%2.2f V: +30 mV Vth -> delay +%1.1f %%" % (vdd, 100 * s))
```

alpha.py - normalised delay, energy and energy-delay product versus Vdd.



| Vdd                                     | delay(norm) | energy(norm) | EDP(norm) |
|-----------------------------------------|-------------|--------------|-----------|
| 0.50                                    | 2.61        | 0.39         | 1.02      |
| 0.55                                    | 1.97        | 0.47         | 0.93      |
| 0.60                                    | 1.61        | 0.56         | 0.91      |
| 0.70                                    | 1.21        | 0.77         | 0.93      |
| 0.80                                    | 1.00        | 1.00         | 1.00      |
| 0.90                                    | 0.87        | 1.27         | 1.10      |
| 1.00                                    | 0.77        | 1.56         | 1.21      |
| Vdd=0.55 V: +30 mV Vth -> delay +23.5 % |             |              |           |
| Vdd=0.80 V: +30 mV Vth -> delay +9.4 %  |             |              |           |

Real output of alpha.py.

The table shows the essence of voltage scaling: dropping from 0.8 V to 0.6 V costs about 60 % in speed but saves 44 % of switching energy; the energy-delay product is flat over a wide range, which is why DVFS (Chapter 11) is so effective. It also shows that at low voltage the same 30 mV threshold shift hurts delay two and a half times more than at nominal - the reason near-threshold designs suffer badly from variation and why low-voltage corners dominate sign-off for such designs.

## 2.3 Threshold voltage, body effect and subthreshold conduction

Vth is set by the gate work function, the channel doping and the oxide. Foundries offer several **threshold flavours** of the same transistor - typically ULVT, LVT, SVT (RVT) and HVT - by changing the work-function metal or implants. Lower Vth means faster and leakier devices (see below).

### Body effect

If the source is above the body (as in the upper nMOS of a stack), the depletion charge increases and Vth rises:

$$V_{th} = V_{th0} + \gamma \left( \sqrt{2 \phi_F + V_{sb}} - \sqrt{2 \phi_F} \right)$$

Body effect.  $\gamma$  = body-effect coefficient,  $\phi_F$  = Fermi potential.

The body effect is why stacked transistors are slower, and it is the physical basis of **body biasing** (Section 2.11): applying a reverse body bias raises Vth and cuts leakage, forward body bias lowers Vth and speeds the device up. In FinFETs the thin, fully depleted fin makes the body effect weak, so body biasing is largely ineffective there.

### Subthreshold conduction and the 60 mV/decade limit

Below Vth the transistor does not switch off abruptly. Current flows by diffusion and falls exponentially with gate voltage:

$$I_{d,sub} = I_0 * 10^{(V_{gs} - V_{th}) / S}$$

$$S = n * (kT/q) * \ln(10) \quad [\text{mV per decade of current}]$$

$$kT/q = 25.9 \text{ mV at } 300 \text{ K} \rightarrow S \geq 59.6 \text{ mV/dec at } 300 \text{ K } (n = 1)$$

Subthreshold swing.  $n \geq 1$  depends on how well the gate controls the channel.

S is the gate voltage needed to change the off-current by 10x. Its theoretical minimum at room temperature is about 60 mV/decade ( $n = 1$ ), set by the Boltzmann distribution of carriers - no conventional MOSFET can beat it, which is the fundamental reason supply voltage stopped scaling. Planar bulk devices at advanced nodes degraded to 90-100 mV/dec; FinFETs and nanosheets recover roughly 65-75 mV/dec because the gate wraps the channel.

```
# Subthreshold leakage:  $I_{off} = I_0 * 10^{(-V_{th} / S)}$ ,  $S = n * (kT/q) * \ln(10)$ 
import math
k, q = 1.380649e-23, 1.602176634e-19
for tc in (25, 125):
    vt = k * (tc + 273.15) / q
    print("T=%3d C: kT/q = %.1f mV, ideal S (n=1) = %.1f mV/dec, n=1.3 -> %.1f mV/dec"
          % (tc, 1e3 * vt, 1e3 * vt * math.log(10), 1e3 * 1.3 * vt * math.log(10)))
S = 1.3 * k * 298.15 / q * math.log(10) # 77 mV/decade at 25 C
I0 = 5e-6 # A/um at Vgs = Vth (constant-current Vth definition)
print(" Vth(mV) Ioff (nA/um) vs 400 mV")
for vth in (200, 250, 300, 350, 400, 450):
    ioff = I0 * 10 ** (-vth / 1e3 / S)
    ref = I0 * 10 ** (-0.400 / S)
    print(" %5d %11.3f %8.1fx" % (vth, ioff * 1e9, ioff / ref))
print("leakage ratio per 100 mV of Vth: %.1fx" % 10 ** (0.1 / S))
```

leak.py - subthreshold swing versus temperature, and off-current versus Vth.

```
T= 25 C: kT/q = 25.7 mV, ideal S (n=1) = 59.2 mV/dec, n=1.3 -> 76.9 mV/dec
T=125 C: kT/q = 34.3 mV, ideal S (n=1) = 79.0 mV/dec, n=1.3 -> 102.7 mV/dec
Vth(mV) Ioff (nA/um) vs 400 mV
200      12.544      398.6x
250       2.807       89.2x
300       0.628      20.0x
350       0.141       4.5x
400       0.031       1.0x
450       0.007       0.2x
leakage ratio per 100 mV of Vth: 20.0x
```

Real output of leak.py.

#### THE RULE OF THUMB EVERY POWER ENGINEER USES

With S around 75-100 mV/dec, each 100 mV reduction of Vth costs roughly 10-20x more subthreshold leakage, and leakage grows strongly with temperature (both kT/q and Vth itself change). This is why HVT cells are the default in leakage-sensitive designs and why leakage is always signed off at the hot corner.

### Other leakage mechanisms

- **Gate leakage:** tunnelling through the gate dielectric. It exploded with 1-2 nm SiO<sub>2</sub> and was tamed by high-k/metal-gate (HKMG) stacks from the 45 nm generation onward.
- **Junction leakage and GIDL** (gate-induced drain leakage): band-to-band tunnelling at the heavily doped drain, worse at high Vdg; it limits how far reverse body bias can reduce leakage.
- **Punch-through:** in very short channels the source and drain depletion regions merge.

## 2.4 Short-channel effects and DIBL

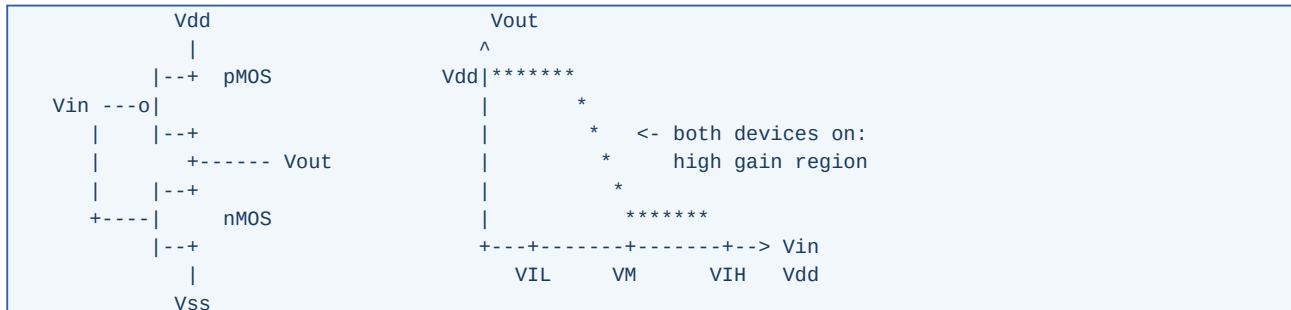
As L shrinks, the drain's electric field reaches into the channel and helps the gate to lower the source barrier. The results are collectively called **short-channel effects**:

- **Vth roll-off:** Vth decreases as L decreases, so gate-length variation becomes Vth variation (and leakage variation).
- **DIBL** (drain-induced barrier lowering): Vth drops as Vds rises, typically quoted in mV/V (tens of mV/V in a good FinFET, much more in a poor planar device). High DIBL means high off current at full Vds and poor output resistance.
- **Channel-length modulation:** the pinch-off point moves, so saturation current keeps rising with Vds.
- **Mobility degradation** from high vertical fields and **velocity saturation** from high lateral fields (previous section).
- **Hot-carrier injection** (HCI): energetic carriers damage the oxide near the drain, shifting Vth over the lifetime (Chapter 21).

Good electrostatic control - the gate dominating the channel potential rather than the drain - is the single goal behind every device-architecture change from planar to FinFET to gate-all-around (Section 2.10).

## 2.5 The CMOS inverter

Complementary MOS combines a pMOS pull-up and an nMOS pull-down. In either stable state one device is off, so ideally no static current flows - the property that made CMOS the dominant logic family.



CMOS inverter and its voltage-transfer characteristic (VTC).

The **switching threshold**  $V_M$  is the input voltage where  $V_{out} = V_{in}$ ; it is centred ( $V_M = V_{dd}/2$ ) when pull-up and pull-down strengths are equal. The unity-gain points  $V_{IL}$  and  $V_{IH}$  (where  $dV_{out}/dV_{in} = -1$ ) define the **noise margins**:

$$NM_H = V_{OH} - V_{IH} \qquad NM_L = V_{IL} - V_{OL}$$

(for a full-swing CMOS gate  $V_{OH} = V_{dd}$ ,  $V_{OL} = 0$ )

Noise margins: how much noise a logic level can absorb.

Because each CMOS gate restores levels to the rails, noise does not accumulate along a logic path. Noise-margin erosion from crosstalk and supply droop is nevertheless a sign-off concern (Chapters 17-18), and low-voltage designs must check that margins survive local variation - the classic failure being an SRAM or a latch that no longer flips.

## 2.6 Switching delay: RC and Elmore

To first order a switching transistor is an effective resistance  $R_{eff}$  charging or discharging a capacitance  $C_L$  made of the next gates' input capacitance, the wire, and the driver's own drain (parasitic) capacitance. For a step input the 50 % delay of a single RC is  $\ln(2) RC = 0.69 RC$ .

For an RC tree (a wire with branches, or a transistor stack) the **Elmore delay** is a simple, pessimistic but surprisingly good estimate: sum over every capacitor the capacitance times the resistance shared between the source-to-output path and the source-to-that-capacitor path.

$$t_{Elmore}(out) = \sum_k C_k * R_{shared}(k, out)$$

$$\text{Uniform wire of length } l \text{ (r, c per unit length): } t = r c l^2 / 2$$

$$\text{Driver } R_d + \text{wire} + \text{load } C_L: t = R_d (c l + C_L) + r l (c l / 2 + C_L)$$

Elmore delay. The quadratic  $l^2$  term is why long wires need repeaters.

```
# Elmore delay of a driver + distributed RC wire + load, wire cut into n pi-segments.
# Elmore: t = sum over every capacitor of C_k * (resistance shared with the path to it)
R_DRV, C_LOAD = 2000.0, 2e-15 # driver 2 kohm, receiver 2 fF
r_um, c_um = 2.0, 0.2e-15 # ~2 ohm/um, ~0.2 fF/um for a thin local metal

def elmore(length_um, n=50):
    rs, cs = r_um * length_um / n, c_um * length_um / n
    t, r_path = 0.0, R_DRV
    for _ in range(n):
        r_path += rs
        t += r_path * cs # each segment's cap sees all R upstream of it
    return t + r_path * C_LOAD

print(" length(um) Elmore(ps) wire-only RC/2 (ps)")
for L in (100, 250, 500, 1000, 2000):
    wire = 0.5 * r_um * L * c_um * L
    print("%10d %10.1f %12.1f" % (L, 1e12 * elmore(L), 1e12 * wire))
# Doubling length ~quadruples the wire term: this is why long wires get repeaters.
```

elmore.py - Elmore delay of a driver driving a distributed RC wire (generic thin-metal values).

| length(um) | Elmore(ps) | wire-only RC/2 (ps) |
|------------|------------|---------------------|
| 100        | 46.4       | 2.0                 |
| 250        | 117.8      | 12.5                |
| 500        | 257.0      | 50.0                |
| 1000       | 612.0      | 200.0               |
| 2000       | 1628.0     | 800.0               |

Real output of elmore.py.

Up to a few hundred microns the driver resistance dominates; beyond that the wire's own  $r c l^2/2$  grows quadratically and takes over. Inserting repeaters makes delay linear in length again - optimum repeater spacing is a classic physical-design calculation revisited in Chapters 15 and 17. STA tools use more accurate models than Elmore (effective capacitance, current-source models, Chapter 9), but Elmore remains the right mental model and the basis of many placement and CTS heuristics.

## 2.7 Logical effort: sizing a path by hand

Sutherland and Sproull's **logical effort** method expresses the delay of a gate, in units of tau (the delay of an ideal inverter driving an identical inverter with no parasitics), as:

$$d = g * h + p$$

$g$  = logical effort (input cap / inverter input cap for equal drive)  
 $h$  = electrical effort =  $C_{out} / C_{in}$        $p$  = parasitic delay

Path:  $G = \prod g_i$ ,  $B = \prod b_i$  (branching),  $H = C_{load} / C_{in}$   
 $F = G B H$ , optimal stage effort  $f = F^{(1/N)}$ ,  
 $D_{min} = N F^{(1/N)} + P$ , best  $N \sim \log_4(F)$

Logical effort. With a pMOS/nMOS mobility ratio of 2:  $g(INV) = 1$ ,  $g(NAND2) = 4/3$ ,  $g(NOR2) = 5/3$ ,  $g(NAND3) = 5/3$ .

The key result is that a path is fastest when every stage bears the same effort  $f$ , and that the optimal  $f$  is about 4 (the famous 'fanout-of-4', FO4). The worked example sizes a four-stage path INV -> NAND2 -> NOR2 -> INV driving a load 64x its input capacitance:

```
# Logical-effort sizing of a 4-stage path: INV -> NAND2 -> NOR2 -> INV -> load.
# g = logical effort, p = parasitic delay (units of an inverter's), gamma = 2 (Pmos/Nmos)
import math
stages = [("INV", 1.0, 1.0), ("NAND2", 4 / 3, 2.0), ("NOR2", 5 / 3, 2.0), ("INV", 1.0, 1.0)]
c_in, c_load, branching = 1.0, 64.0, 1.0 # capacitances in unit-inverter inputs
```

```

G = math.prod(g for _, g, _ in stages)
H = c_load / c_in
F = G * branching * H
N = len(stages)
f_hat = F ** (1 / N)
P = sum(p for _, _, p in stages)
print("G=%.3f H=%.0f F=%.1f stage effort f=%.3f" % (G, H, F, f_hat))
print("minimum delay D = N*f + P = %.2f (x tau)" % (N * f_hat + P))
# Work backwards from the load: Cin_i = g_i * Cout_i / f_hat
c_out, sizes = c_load, []
for name, g, p in reversed(stages):
    cin = g * c_out / f_hat
    sizes.append((name, cin, g * c_out / cin + p))
    c_out = cin
for name, cin, d in reversed(sizes):
    print(" %-6s input cap = %6.2f stage delay = %.2f" % (name, cin, d))
print("best number of stages ~ log4(F) = %.2f" % math.log(F, 4))

```

le.py - logical-effort sizing of a four-stage path.

```

G=2.222 H=64 F=142.2 stage effort f=3.453
minimum delay D = N*f + P = 19.81 (x tau)
  INV   input cap =  1.00 stage delay = 4.45
  NAND2 input cap =  3.45 stage delay = 5.45
  NOR2  input cap =  8.94 stage delay = 5.45
  INV   input cap = 18.53 stage delay = 4.45
best number of stages ~ log4(F) = 3.58

```

Real output of le.py.

Each stage gets an effort of 3.45 and the minimum path delay is 19.8 tau; sizes grow geometrically towards the load. Since  $\log_4(F)$  is 3.6, four stages is close to optimal; forcing it into two stages would need a stage effort near 12 and be much slower. Synthesis and placement tools do this sizing automatically, but logical effort explains *why* they insert buffers, why high-fanout nets become buffer trees and why NOR-heavy logic is slow.

#### FO4 DELAY AS A TECHNOLOGY YARDSTICK

Designers quote cycle times in FO4 inverter delays because the number is roughly technology independent: a high-performance CPU pipeline stage is on the order of 10-20 FO4, a relaxed ASIC pipeline 30-60 FO4. Knowing the FO4 delay of your library at the sign-off corner lets you sanity-check a target frequency before writing any RTL (Chapter 5).

## 2.8 Power: dynamic, short-circuit and leakage

$$P_{\text{total}} = P_{\text{dynamic}} + P_{\text{short-circuit}} + P_{\text{leakage}}$$

$$P_{\text{dynamic}} = \alpha * C_{\text{sw}} * V_{\text{dd}}^2 * f \quad (\alpha = \text{activity factor})$$

$$P_{\text{short}} \sim \text{proportional to input slew, } V_{\text{dd}} \text{ and } f \text{ (both devices briefly on)}$$

$$P_{\text{leakage}} = V_{\text{dd}} * (I_{\text{sub}} + I_{\text{gate}} + I_{\text{junction/GIDL}})$$

$$\text{Energy per transition from the supply} = C V_{\text{dd}}^2 \text{ (half stored, half burned)}$$

The three components of CMOS power.

**Dynamic power** dominates active operation: every  $0 \rightarrow 1$  transition draws  $C V_{\text{dd}}^2$  from the supply, half dissipated in the pull-up, and the stored half is dissipated in the pull-down on the next  $1 \rightarrow 0$  transition. The quadratic dependence on  $V_{\text{dd}}$  makes voltage the strongest power lever; clock gating attacks  $\alpha$ ; careful floorplanning attacks wire  $C$ . **Short-circuit power** is usually 5-10 % of dynamic if slews are controlled - one reason Liberty libraries impose max\_transition limits. **Leakage** flows whenever the chip is powered, dominates in standby, and rises steeply with temperature; it is attacked with HVT cells, power gating and body bias

(Chapter 11). In Liberty, dynamic power appears as internal-power tables plus switching of the load, and leakage as state-dependent leakage\_power values (Chapter 4).

## 2.9 Scaling: Dennard and its end

In 1974 Dennard et al. showed that if all dimensions and voltages shrink by a factor  $k$  (about 1.4 per generation), transistors get faster by  $k$ , density rises by  $k^2$ , and power density stays constant. For thirty years this 'free lunch' powered the industry.

| Quantity                     | Ideal Dennard scaling | Post-2005 reality                                                                  |
|------------------------------|-----------------------|------------------------------------------------------------------------------------|
| Dimensions (L, W, $t_{ox}$ ) | $1/k$                 | Scaled, but gate oxide stalled (replaced by high-k)                                |
| Supply Vdd                   | $1/k$                 | Nearly flat around 0.7-1.0 V (subthreshold slope limit, variation)                 |
| Vth                          | $1/k$                 | Cannot drop without exponential leakage                                            |
| Density                      | $k^2$                 | Still improves, but more slowly and via DTCO tricks (fewer tracks, backside power) |
| Gate delay                   | $1/k$                 | Modest gains; wires increasingly dominate                                          |
| Power density                | Constant              | Rises -> 'dark silicon', power-limited design                                      |

Once Vdd stopped scaling (roughly the 90-65 nm era), power density rose and frequency plateaued. The industry responded with multi-core, specialised accelerators, aggressive power management (clock and power gating, DVFS), and new device structures. Wire resistance also rises sharply as cross-sections shrink (Chapter 3), so interconnect, not transistors, often limits performance today.

## 2.10 Device architectures: planar, FinFET, GAA/nanosheet, CFET

| PLANAR (bulk)                                                             | FinFET                                                                                                                | GAA / NANOSHEET                                                                                             | CFET<br>(stacked n/p)                                                              |
|---------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| gate on top<br>+-----+<br>  gate  <br>+-----+<br>~~channel~~<br>substrate | gate wraps 3 sides<br>+--+ +--+<br>+--   ---   ---+<br>   f   f   <br>   i   i   <br>   n   n   <br>+---+---+---+---+ | gate wraps all 4<br>+-----+<br>  ==sheet==  <br>  ==sheet==  <br>  ==sheet==  <br>+-----+<br>sheets stacked | nFET sheets<br>over pFET<br>sheets<br>+-----+<br>  n n n  <br>  p p p  <br>+-----+ |
| Control: 1 side<br><= 28/20 nm                                            | Control: 3 sides<br>22/16 nm .. 3 nm                                                                                  | Control: 4 sides<br>~3/2 nm and below                                                                       | Footprint /2<br>research                                                           |

Evolution of the transistor for better electrostatic control.

| Architecture                           | Key idea                                       | Design implications                                                                                                      |
|----------------------------------------|------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Planar bulk                            | Gate above a flat channel                      | Continuous W; body biasing works; poor control below ~28/20 nm                                                           |
| FD-SOI (planar)                        | Very thin silicon on buried oxide              | Good control without fins; strong body bias via back gate                                                                |
| FinFET                                 | Channel is a vertical fin; gate on three sides | Width quantised in fins; cells sized in fins; strong drive, low leakage; higher gate/parasitic capacitance; self-heating |
| GAA / nanosheet<br>(RibbonFET, MBCFET) | Stacked horizontal sheets surrounded by gate   | Sheet width is variable again; better control; often paired with backside power delivery                                 |
| CFET                                   | nFET stacked on pFET                           | Roughly halves cell footprint; research/early development                                                                |

For the ASIC engineer, the practical consequences are: in FinFET nodes transistor width comes in integer fins, so drive strengths and cell heights are quantised; parasitic capacitance and wire resistance matter more than intrinsic device speed; and *self-heating* and electromigration limits are tighter (Chapter 18).

## 2.11 Multi-Vt, SOI/FD-SOI and body bias

**Multi-Vt design** is the most widely used leakage technique: synthesis and placement use LVT/ULVT cells on critical paths and HVT/SVT everywhere else. Typical ratios per Vt step are roughly 2-3x in leakage and 10-20 % in delay, but these vary by node. Most flows cap the fraction of LVT/ULVT cells (for example a few percent to twenty percent) and track it as a design metric.

**Silicon-on-insulator** places the transistor in a thin silicon layer on a buried oxide (BOX). **FD-SOI** (fully depleted, e.g. 28 nm and 22 nm offerings) uses a film only a few nanometres thick, so the channel is fully depleted and short-channel control is good. Because the BOX is thin, the substrate under it acts as a second gate: **body bias** can shift Vth by tens of mV per volt over a wide range (on the order of  $\pm 1$  V or more in some flavours, far beyond what junction limits allow in bulk). This allows forward body bias for speed in active mode and reverse body bias for low leakage in standby, or compensation of process corners after manufacture - a powerful knob for IoT and automotive designs.

### PITFALL: ASSUMING BODY BIAS WORKS EVERYWHERE

Body bias is strong in FD-SOI, moderate in older planar bulk and weak in FinFET/GAA. Also, bias generators, well isolation (deep n-well), extra bias pins on every cell (the VPB/VNB pins seen in the SKY130 LEF in Chapter 4) and multi-bias sign-off corners all cost area and schedule. Decide at architecture time (Chapter 5).

## 2.12 Summary

- Doping creates n and p regions; pn junctions must be kept reverse biased with well/substrate taps, and tap spacing prevents latch-up.
- A MOSFET works in cutoff, linear or saturation. The square law fails in short channels; the alpha-power law  $t_d \sim C_{Vdd} / (V_{dd} - V_{th})^\alpha$  captures velocity saturation and explains why low-voltage operation is slow and variation-sensitive.
- Subthreshold swing cannot beat  $\sim 60$  mV/dec at room temperature, so every 100 mV of Vth trades  $\sim 10$ - $20$ x in leakage; leakage rises steeply with temperature.
- DIBL and other short-channel effects drove the move from planar to FinFET to GAA; CFET is next.
- Gate delay is  $R_{eff} \times C_{load}$  to first order; Elmore delay handles RC trees and shows quadratic wire delay; logical effort sizes paths for minimum delay (equal stage effort  $\sim 4$ ).
- Power =  $\alpha C_{Vdd}^2 f + \text{short-circuit} + \text{leakage}$ . Dennard scaling ended when Vdd and Vth stopped scaling.
- Multi-Vt libraries, FD-SOI body bias and power gating are the main device-level power knobs.

## 2.13 Exercises

- 1 Using alpha.py, find the Vdd that minimises the energy-delay product for  $\alpha = 1.3$  and  $V_{th} = 0.35$  V. How does it move if Vth is 0.25 V?
- 2 A design has 20 mW of leakage at 25 deg C with  $S = 80$  mV/dec. Estimate the leakage change if all cells are swapped from a Vth of 0.30 V to 0.38 V. What else changes?
- 3 Size a path NAND2  $\rightarrow$  INV  $\rightarrow$  NAND3  $\rightarrow$  INV driving 100x the input capacitance with logical effort. Would adding two more inverters help?
- 4 Compute the Elmore delay at the far end of a 2 mm wire (2 ohm/um, 0.2 fF/um) driven by a 1 kohm driver, then with one repeater (same driver) at the midpoint. Ignore the repeater's own delay first, then add 20 ps.
- 5 Explain, using the subthreshold-swing limit, why supply voltage stopped scaling while dimensions kept shrinking.
- 6 Why is body biasing ineffective in FinFETs but effective in FD-SOI?



### 3. Process Technology and Fabrication: Wafers, Lithography, FEOL/MEOL/BEOL, Design Rules, PDKs and Variation

A design engineer never touches a wafer, yet the fab shapes every decision: the design rules that the router must obey, the metal stack that sets wire delay, the variation that sets timing margins, the density rules that force fill, the mask count that sets NRE. This chapter walks through how a chip is actually made, then through what the foundry hands the design team - the process design kit - and how manufacturing variation is modelled.

#### 3.1 From sand to wafers

Chip-grade silicon starts as quartz sand reduced to metallurgical silicon, purified through a gas phase (trichlorosilane) to 'eleven nines' (99.99999999 %) polysilicon, then melted and grown into a single-crystal **ingot** by the Czochralski process: a seed crystal is dipped into the melt and slowly pulled and rotated. Dopant added to the melt sets the substrate type (lightly p-type for most CMOS). The ingot is ground to diameter, notched to mark the crystal orientation, sliced with wire saws into wafers (about 775  $\mu\text{m}$  thick for 300 mm), then lapped, etched and polished to a mirror finish. Some processes add an epitaxial layer or use SOI wafers (Chapter 2).

| Wafer diameter        | Typical use today                                                                                       |
|-----------------------|---------------------------------------------------------------------------------------------------------|
| <b>150 mm (6 in)</b>  | Power, MEMS, compound semiconductors, old analog                                                        |
| <b>200 mm (8 in)</b>  | Mature nodes (350-90 nm): analog, power, sensors, many automotive and IoT parts                         |
| <b>300 mm (12 in)</b> | All leading-edge logic and memory, and many mature nodes (e.g. 65-28 nm). Area is 2.25x a 200 mm wafer. |

Wafers are processed in **cleanrooms** in which the air is filtered to ISO class 1-5 levels (a few particles of 0.1  $\mu\text{m}$  or larger per cubic metre in the best areas), temperature and humidity are tightly controlled, and lithography bays use yellow light. Modern 300 mm fabs move wafers in sealed pods (FOUPs) on overhead transport so that the wafer itself sees an even cleaner mini-environment. A leading-edge wafer passes through well over a thousand process steps and spends roughly three months or more in the fab.

#### 3.2 The unit processes

Every structure on a chip is made by repeating a handful of unit processes: add material, pattern it, remove material, modify it, and flatten.

| Process                 | What it does                                                                                              | Examples                                                                          |
|-------------------------|-----------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| <b>Oxidation</b>        | Grow $\text{SiO}_2$ by heating in $\text{O}_2/\text{H}_2\text{O}$                                         | Pad oxide, legacy gate oxide, liners                                              |
| <b>Deposition</b>       | Add thin films: CVD, PECVD, ALD (atomic-layer, angstrom control), PVD/sputtering, electroplating, epitaxy | High-k by ALD, barrier/seed by PVD, copper by electroplating, SiGe S/D by epitaxy |
| <b>Lithography</b>      | Transfer a mask pattern into photoresist                                                                  | Every patterned layer (Section 3.3)                                               |
| <b>Etch</b>             | Remove material where resist is open. Dry (plasma, anisotropic) or wet (isotropic)                        | Fin etch, gate etch, via and trench etch                                          |
| <b>Ion implantation</b> | Accelerate dopant ions into the silicon, then anneal to activate and repair damage                        | Wells, Vt adjust, source/drain extensions, halos                                  |
| <b>Anneal</b>           | Heat to activate dopants (rapid thermal, spike, laser)                                                    | After implants; thermal budget limits later steps                                 |
| <b>CMP</b>              | Chemical-mechanical polishing: planarise the wafer                                                        | After STI fill, after every copper layer                                          |



| Process                  | What it does                                                           | Examples                 |
|--------------------------|------------------------------------------------------------------------|--------------------------|
| <b>Clean / metrology</b> | Remove particles and residues; measure thickness, CD, overlay, defects | Between almost all steps |

### 3.3 Photolithography

Lithography is the most expensive and most critical step. A wafer is coated with photoresist, exposed through a **photomask** (reticle) in a scanner that projects a reduced (typically 4x) image, then developed so that the pattern remains in resist. Etch or implant then transfers the pattern into the wafer. A scanner exposes one field (at most 26 mm x 33 mm for standard 4x systems - this **reticle limit** caps the size of a single die, about 858 mm<sup>2</sup>) then steps to the next.

$$\text{Resolution (half-pitch)} \quad R = k_1 * \lambda / NA$$

$$\text{Depth of focus} \quad DOF = k_2 * \lambda / NA^2$$

$$\lambda = \text{wavelength}, NA = \text{numerical aperture (n sin theta)}, k_1 \geq 0.25$$

Rayleigh criteria. Everything in lithography is a fight to lower k1 or lambda or to raise NA.

```
# Rayleigh resolution: half-pitch = k1 * lambda / NA (k1 >= 0.25 physical limit)
tools = [("KrF dry", 248, 0.93), ("ArF dry", 193, 0.93), ("ArF immersion", 193, 1.35),
        ("EUV (0.33 NA)", 13.5, 0.33), ("High-NA EUV", 13.5, 0.55)]
print("%-15s %6s %5s   hp@k1=0.40   hp@k1=0.28" % ("tool", "lambda", "NA"))
for name, lam, na in tools:
    print("%-15s %6.1f %5.2f   %7.1f nm   %7.1f nm" % (name, lam, na, 0.4 * lam / na,
                                                         0.28 * lam / na))

# A 28 nm metal pitch (14 nm half-pitch) printed with 193i in a single exposure:
k1 = 14 / (193 / 1.35)
print("193i: k1 needed for 14 nm half-pitch = %.3f (below the 0.25 limit)" % k1)
print("pitch must be split by ~%.1fx at k1=0.28 -> quadruple patterning (SAQP)"
      % (0.28 / k1))
```

litho.py - printable half-pitch for each lithography generation.

| tool          | lambda | NA   | hp@k1=0.40 | hp@k1=0.28 |
|---------------|--------|------|------------|------------|
| KrF dry       | 248.0  | 0.93 | 106.7 nm   | 74.7 nm    |
| ArF dry       | 193.0  | 0.93 | 83.0 nm    | 58.1 nm    |
| ArF immersion | 193.0  | 1.35 | 57.2 nm    | 40.0 nm    |
| EUV (0.33 NA) | 13.5   | 0.33 | 16.4 nm    | 11.5 nm    |
| High-NA EUV   | 13.5   | 0.55 | 9.8 nm     | 6.9 nm     |

193i: k1 needed for 14 nm half-pitch = 0.098 (below the 0.25 limit)  
pitch must be split by ~2.9x at k1=0.28 -> quadruple patterning (SAQP)

Real output of litho.py.

Immersion lithography (ArF, 193 nm, with water between lens and wafer so NA can exceed 1) printed everything down to about 80 nm pitch in a single exposure. Tighter pitches needed **multi-patterning**: LELE (litho-etch-litho-etch, two masks with the layout split into two 'colours'), and self-aligned double or quadruple patterning (SADP/SAQP) where spacers deposited on a mandrel define lines at half or a quarter of the printed pitch. Multi-patterning adds masks, cost and layout restrictions: **colouring** rules (no two same-colour shapes too close), unidirectional metal, and cut masks.

**EUV** lithography (13.5 nm, reflective optics in vacuum, NA 0.33) entered volume production around the 7 nm/5 nm generation and replaced many multi-patterning steps with single exposures; high-NA EUV (0.55) is being introduced for the following nodes. EUV brings its own issues: stochastic defects (few photons per feature), mask defects that cannot be covered by a pellicle as easily, and high tool cost.

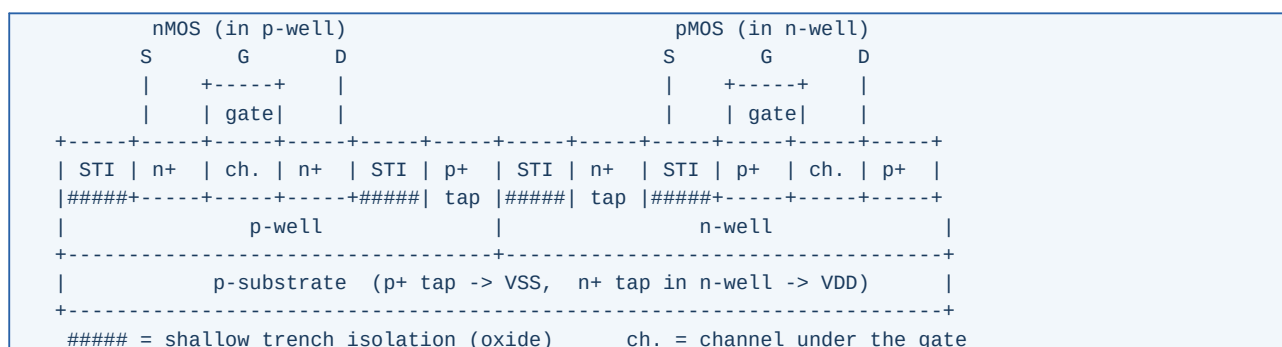
## Resolution enhancement techniques

- **OPC** (optical proximity correction): the mask shapes are deliberately distorted - serifs, hammerheads, biasing - so that the printed shape matches the drawn one. Done by the foundry or mask flow after tapeout (Chapter 19).
- **SRAFs** (sub-resolution assist features): tiny non-printing shapes that improve the process window of isolated lines.
- **Phase-shift masks** and **off-axis / source-mask optimised illumination** improve contrast.
- **Restricted design rules**: fixed pitches, unidirectional layers and gridded placement make patterns regular and printable.

## 3.4 FEOL: building the transistors

Front-end-of-line (FEOL) processing builds the devices in the silicon. A simplified modern sequence (details vary widely by foundry and node):

- 1 **Wells**: implant n-wells (for pMOS) and p-wells (for nMOS); deep n-well where a p-well must be isolated from the substrate.
- 2 **Isolation**: shallow trench isolation (STI) - etch trenches, fill with oxide, CMP flat - separates active areas. In FinFET processes the fins are etched first (often by SADP/SAQP) and oxide fills between them.
- 3 **Gate stack**: modern nodes use high-k dielectric (hafnium-based) and metal gates. In the common **replacement-metal-gate** (gate-last) flow a sacrificial polysilicon gate is patterned, spacers are formed, source/drain are built, then the dummy gate is removed and replaced by high-k and work-function metals (whose choice sets the  $V_t$  flavours).
- 4 **Source/drain**: extension and halo implants, spacers, then raised S/D grown by epitaxy - SiGe for pMOS (compressive strain boosts hole mobility), Si:P for nMOS.
- 5 **Anneal and silicide**: activate dopants; form a low-resistance silicide at contacts.



Simplified FEOL cross-section: wells, STI, gates and source/drain.

## 3.5 MEOL: the local interconnect

Middle-of-line (MEOL) connects the devices to the first metal. It consists of **trench contacts** to source/drain, **gate contacts**, and sometimes a **local interconnect** layer. In SKY130 this is the 'li1' local-interconnect layer (visible in the LEF of Chapter 4) with 'mcon' contacts up to met1. At advanced nodes the MEOL contacts are tiny and highly resistive; contact resistance has become a large fraction of total device resistance, and tricks such as **contact over active gate** (COAG) are used to shrink cells.

## 3.6 BEOL: the metal stack

Back-end-of-line (BEOL) builds the wiring. Since the late 1990s copper has been used, patterned by the **damascene** process because copper cannot be dry-etched easily: etch trenches (and vias, in **dual damascene**) into the dielectric, line them with a barrier (Ta/TaN) and seed, electroplate copper to overfill, then CMP back to the dielectric. Dielectrics are **low-k** (carbon-doped oxide, porous films, k around 2.5-3.0 versus 3.9 for  $\text{SiO}_2$ ) to reduce capacitance, at the cost of mechanical weakness that matters for packaging (Chapter

20).

| Layer group       | pitch                                   | used for                            |
|-------------------|-----------------------------------------|-------------------------------------|
| RDL / AP (Al pad) | ~ several um                            | bond pads, bumps, redistribution    |
| Mz top thick Cu   | ~ 1-10 um                               | global power grid, clock spines     |
| Mx intermediate   | ~ 2-4x min                              | block routing, long signals         |
| M2..M4 thin (1x)  | minimum                                 | local routing inside std-cell areas |
| M1                | minimum                                 | cell internal wiring, pins          |
| ---- MEOL ----    | contacts / local interconnect           |                                     |
| ---- FEOL ----    | transistors (fins / sheets, gates, S/D) |                                     |
| thick             |                                         | (low R: power, clock)               |
| intermediate      |                                         |                                     |
| thin (1x)         |                                         | (high R, dense)                     |
| contacts          | . . . . .                               |                                     |
| devices           | ====                                    |                                     |

A generic BEOL stack: thin, dense layers at the bottom; thick, low-resistance layers at the top.

Wire resistance per unit length grows roughly as  $1/(\text{width} \times \text{thickness})$ , and at advanced nodes even faster because the barrier does not scale and electrons scatter at surfaces and grain boundaries (effective copper resistivity at the thinnest layers is several times bulk). A typical advanced-node stack has 10-18 metal layers; the physical-design engineer chooses which layers carry power, clocks and long signals (Chapters 14-17). Alternative metals (cobalt, ruthenium) and **backside power delivery** - moving the power grid under the transistors - are recent answers to interconnect resistance.

### A real stack: SKY130

The open SkyWater 130 nm process has one local interconnect (li1) and five aluminium metal layers. From its technology LEF ([sky130\\_fd\\_sc\\_hd.tlef](#)) the routing pitches are:

| Layer | Direction  | Pitch (um)  | Min width (um) |
|-------|------------|-------------|----------------|
| li1   | vertical   | 0.46 x 0.34 | 0.17           |
| met1  | horizontal | 0.34        | 0.14           |
| met2  | vertical   | 0.46        | 0.14           |
| met3  | horizontal | 0.68        | 0.30           |
| met4  | vertical   | 0.92        | 0.30           |
| met5  | horizontal | 3.40        | 1.60           |

SKY130 routing layers as defined in the real [sky130\\_fd\\_sc\\_hd.tlef](#) (pitches and widths in um).

Note the alternating preferred directions (each layer routes mostly in one direction, and its neighbours in the other) and the thick, wide top layer used for the power grid.

## 3.7 Node names versus real dimensions

Up to roughly 0.35 um the node name matched the drawn gate length. Since then it has become a **marketing label** for a generation: a '5 nm' or '3 nm' process has no 5 nm or 3 nm feature that defines it. Meaningful density metrics are the **contacted poly pitch** (CPP, gate-to-gate spacing), the **minimum metal pitch** (MMP) and the **cell height** (tracks x metal pitch).

| Node label (approximate) | CPP (order of) | Min metal pitch (order of) | Device        |
|--------------------------|----------------|----------------------------|---------------|
| 130 nm (e.g. SKY130)     | ~ 0.4-0.5 um   | ~ 0.3-0.4 um               | Planar        |
| 28 nm                    | ~ 110-120 nm   | ~ 90 nm                    | Planar (HKMG) |
| 16/14 nm                 | ~ 80-90 nm     | ~ 64 nm                    | FinFET        |
| 7 nm                     | ~ 55-60 nm     | ~ 36-40 nm                 | FinFET        |

| Node label (approximate) | CPP (order of) | Min metal pitch (order of) | Device       |
|--------------------------|----------------|----------------------------|--------------|
| 5 nm                     | ~ 50 nm        | ~ 28-30 nm                 | FinFET (EUV) |
| 3 nm / 2 nm class        | ~ 45-48 nm     | ~ 21-24 nm                 | FinFET / GAA |

Published/estimated ranges from public sources; exact values differ per foundry. Always use your PDK's numbers.

#### PITFALL: COMPARING NODES ACROSS FOUNDRIES BY NAME

Two foundries' '7 nm' processes can differ substantially in density and performance. Compare CPP x MMP, cell height, SRAM bitcell area and published PPA of a reference design - not the label.

## 3.8 Design rules

The foundry guarantees yield only for layouts that obey its **design rules**, documented in the **design rule manual** (DRM) and encoded in the DRC rule deck (Chapter 18). The basic rule types:

| WIDTH     | SPACING        | ENCLOSURE            | EXTENSION | AREA        |
|-----------|----------------|----------------------|-----------|-------------|
| +---+     | +---+ +---+    | +-----+              | +---+     | +-----+     |
|           | <->            | +-----+              |           | #####       |
| <-->      | s              | via                  | == =-->   | #####       |
| w         | +---+ +---+    | +-----+              | e         | +-----+     |
| +---+     |                | +--<-e->-----+       | +---+     | >= A_min    |
| w >= Wmin | s >= Smin(w,l) | metal encloses via   | poly past | min polygon |
|           | (wide-metal    | by >= e (end-of-line | diffusion | area        |
|           | & EOL rules)   | rules differ)        | by >= e   |             |

Basic design-rule types.

- **Width, spacing, enclosure, extension, area** - geometric rules per layer and between layers.
- **Wide-metal and end-of-line spacing** - wider or longer wires need more space; line ends need extra space for printability.
- **Density rules** - each layer must have a minimum and maximum pattern density within sliding windows (for uniform CMP and etch); met by inserting **fill** (dummy metal/poly/diffusion).
- **Antenna rules** - the ratio of metal area connected to a gate during manufacturing (before the upper layers connect it to a diffusion) must stay below a limit to avoid plasma charging damage to the gate oxide; fixed with layer jumping or diodes (Chapters 4 and 17).
- **Multi-patterning (colouring) rules, grid rules, via rules** - advanced nodes add hundreds of context-dependent rules.
- **Latch-up and ESD rules** - tap spacing, guard rings, I/O rules (Chapter 12).

### DFM: design for manufacturability

Beyond the mandatory rules, foundries publish **recommended rules** and DFM guidelines: redundant (double) vias, wire spreading, wider line ends, litho-friendly patterns checked by litho simulation or pattern matching (hotspot detection), and critical area analysis to estimate random-defect yield. DFM work improves yield and reliability and is discussed in Chapters 17 and 21.

## 3.9 The process design kit (PDK)

The PDK is the contract between the foundry and the design team. It is usually licensed under NDA and qualified for specific tool versions. Open PDKs such as SKY130, GF180MCU and IHP SG13G2 let anyone see what one contains.

| PDK component                   | Content                                                | Used by        |
|---------------------------------|--------------------------------------------------------|----------------|
| <b>Design rule manual (DRM)</b> | All layout rules, layer definitions, recommended rules | Layout, PD, PV |

| PDK component                         | Content                                                                                                                            | Used by                                     |
|---------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------|
| <b>Device models</b>                  | SPICE models (BSIM-CMG for FinFET, BSIM4 for planar, BSIM-IMG for FD-SOI), corner and Monte Carlo statistical models, aging models | Circuit designers, library characterisation |
| <b>Layer map and tech files</b>       | GDS layer numbers; technology files for layout editors and P&R (tech LEF, routing rules)                                           | Layout, PD                                  |
| <b>Rule decks</b>                     | DRC, LVS, antenna, ERC, fill, parasitic extraction (RC tech files, e.g. ITF/ICT/qrcTechFile), DFM                                  | PV, extraction                              |
| <b>PCells</b>                         | Parameterised layout generators for transistors, resistors, capacitors, inductors                                                  | Analog/custom layout                        |
| <b>Primitive libraries</b>            | Symbols, schematics, device cells                                                                                                  | Analog design                               |
| <b>Foundation IP (often separate)</b> | Standard cells, I/Os, memory compilers (Chapter 4)                                                                                 | Everyone                                    |
| <b>Reliability docs</b>               | EM limits, TDDDB, HCI, BTI, self-heating guidelines                                                                                | Sign-off (Chapter 18)                       |

#### PDK VERSIONS MATTER

PDKs are released in versions (0.1, 0.5, 0.9, 1.0...). Early versions carry preliminary models and rules; designs started on 0.5 must be re-verified when 1.0 arrives. Track the PDK version in every sign-off report, and read the release notes - a changed rule deck can create thousands of DRC violations late in a project.

## 3.10 Process variation

No two transistors are identical. Variation is classified two ways:

|                   | Global (die-to-die, wafer-to-wafer, lot-to-lot)          | Local (within-die, device-to-device)                                                       |
|-------------------|----------------------------------------------------------|--------------------------------------------------------------------------------------------|
| <b>Systematic</b> | Equipment drift, wafer-level gradients, lens aberrations | Layout-dependent effects: well proximity, STI stress, density, litho context, OPC residue  |
| <b>Random</b>     | Lot-to-lot fluctuations                                  | Random dopant fluctuation, line-edge roughness, work-function granularity, fin variability |

### Corners

Global variation is captured by **process corners**, named by the speed of nMOS then pMOS: **TT** (typical), **FF** (fast-fast), **SS** (slow-slow), **FS** and **SF** (skewed). Combined with voltage (e.g. nominal +/- 10 %) and temperature (e.g. -40 to 125 deg C) they form the **PVT corners** at which libraries are characterised and timing is signed off (Chapter 9). Setup is usually worst at SS/low-V, hold at FF/high-V, leakage at FF/high-T, dynamic power at FF/high-V. Skewed corners stress ratioed circuits (SRAM write/read, level shifters, latches).

### Temperature inversion

Higher temperature lowers mobility (slower) but also lowers  $V_{th}$  (faster). At high  $V_{dd}$  mobility wins and hot is slow. At low  $V_{dd}$  - where overdrive  $V_{dd} - V_{th}$  is small - the  $V_{th}$  effect wins and **cold is slow**. This **temperature inversion** is pronounced in FinFET nodes and low-voltage designs, so sign-off must include the cold corners (e.g. SS/0.72 V/-40 C) as potential worst-setup corners, not just the hot ones.

### Local variation: Pelgrom's law and Monte Carlo

Random local variation of  $V_{th}$  between two identically drawn devices follows **Pelgrom's law**: the standard deviation is inversely proportional to the square root of the gate area.

$$\sigma(\Delta V_{th}) = A_{VT} / \sqrt{W * L}$$

Pelgrom mismatch.  $A_{VT}$  (mV\* $\mu$ m) is a technology constant from the PDK.

```
# Pelgrom mismatch:  $\sigma(dV_{th}) = A_{VT} / \sqrt{W * L}$  (pair of identical devices)
#  $A_{VT}$  is typically ~1-5 mV*um for modern nodes; 1.5 mV*um is used here.
import math, random
A_VT = 1.5 # mV*um
print(" W(um)  L(um)  area(um2)  sigma_dVth(mV)  6-sigma(mV)")
for w, l in ((0.1, 0.03), (0.2, 0.03), (0.5, 0.05), (1.0, 0.1), (4.0, 0.5), (10, 1)):
    s = A_VT / math.sqrt(w * l)
    print("%7.2f %7.2f %10.4f %13.1f %12.1f" % (w, l, w * l, s, 6 * s))
# Monte Carlo check for the smallest device: draw 100k pairs, compare with formula.
random.seed(1)
s_pair = A_VT / math.sqrt(0.1 * 0.03)
s_dev = s_pair / math.sqrt(2) # each device carries 1/sqrt(2)
d = [random.gauss(0, s_dev) - random.gauss(0, s_dev) for _ in range(100000)]
mc = math.sqrt(sum(x * x for x in d) / len(d))
print("Monte Carlo sigma(dVth) = %.1f mV (formula %.1f mV)" % (mc, s_pair))
```

pelgrom.py - Vth mismatch versus device size, checked with a Monte Carlo draw.

| W(um) | L(um) | area(um2) | sigma_dVth(mV) | 6-sigma(mV) |
|-------|-------|-----------|----------------|-------------|
| 0.10  | 0.03  | 0.0030    | 27.4           | 164.3       |
| 0.20  | 0.03  | 0.0060    | 19.4           | 116.2       |
| 0.50  | 0.05  | 0.0250    | 9.5            | 56.9        |
| 1.00  | 0.10  | 0.1000    | 4.7            | 28.5        |
| 4.00  | 0.50  | 2.0000    | 1.1            | 6.4         |
| 10.00 | 1.00  | 10.0000   | 0.5            | 2.8         |

Monte Carlo sigma(dVth) = 27.5 mV (formula 27.4 mV)

Real output of pelgrom.py.

A minimum-size device has tens of millivolts of random Vth mismatch; a 6-sigma excursion is well over 100 mV - comparable to the overdrive at low voltage. Halving sigma costs 4x the area. This is why SRAM bitcells (millions of minimum devices, each needing to work at 6-sigma or beyond) set the minimum operating voltage of many chips, why analog designers use large devices, and why STA uses statistical on-chip variation models (AOCV, POCV/LVF, Chapter 9). SPICE **Monte Carlo** simulation draws global and local parameters from the PDK's statistical models to verify such circuits; library characterisation uses it to produce LVF tables (Chapter 4).

#### INTERVIEW INSIGHT: WHY DOES VARIATION GET WORSE WITH SCALING?

Fewer dopant atoms per channel (a few tens in a planar device), line-edge roughness that does not shrink with the feature, and smaller gate area (Pelgrom). FinFETs reduced random dopant fluctuation (undoped fins) but added fin-width and work-function granularity. Meanwhile lower Vdd means the same mV of variation is a larger fraction of overdrive.

## 3.11 Summary

- Wafers are grown as single-crystal ingots, sliced and polished; 300 mm is standard for advanced and many mature nodes. Processing takes 1000+ steps and roughly three months.
- Lithography resolution is  $k_1 \lambda / NA$ . 193i needs multi-patterning below ~80 nm pitch; EUV (13.5 nm) restored single-exposure patterning for critical layers. The reticle limits a die to about 26 x 33 mm.
- FEOL builds transistors (wells, STI/fins, HKMG gates, epitaxial S/D); MEOL makes contacts/local interconnect; BEOL builds a copper damascene stack from thin, dense layers to thick top layers.
- Node names are labels; compare CPP, metal pitch and cell height.
- Design rules (width, spacing, enclosure, density, antenna, colouring) and DFM guidelines protect yield; the PDK packages rules, models, tech files, rule decks and PCells.
- Variation is global or local, systematic or random; corners model global variation, temperature inversion makes cold corners critical, and Pelgrom's law  $\sigma = A_{VT} / \sqrt{WL}$  sets local mismatch.

## 3.12 Exercises

---

- 1 With 193i immersion ( $NA = 1.35$ ) and  $k_1 = 0.30$ , what is the finest single-exposure pitch? How many exposures (or which self-aligned scheme) would a 32 nm pitch need?
- 2 Using the SKY130 pitches above, how many met2 tracks cross a 100  $\mu\text{m}$  wide channel? How many met4 tracks?
- 3 An SRAM needs  $\sigma(\Delta V_{th}) \leq 15 \text{ mV}$  between the two pull-down devices. With  $A_{VT} = 1.5 \text{ mV } \mu\text{m}$  and  $L = 30 \text{ nm}$ , what minimum  $W$  is required? What does that do to bitcell area?
- 4 Explain why the worst setup corner of a 0.6 V FinFET design may be SS/-40 C rather than SS/125 C.
- 5 List five PDK deliverables and the team that consumes each.
- 6 Why must metal density be kept within minimum and maximum limits, and what does the PD flow do to meet them?

## 4. Standard Cells, Libraries and IP: Cell Design, Liberty, LEF, Memories, I/O and IP Integration

Synthesis, placement, clock-tree synthesis, routing, STA, power analysis and ATPG never look at transistors. They see a design as instances of **cells** and **macros**, each described by a handful of abstract views: a Liberty file for timing, power and function, a LEF file for geometry, a Verilog model for simulation, and GDS for the final layout. The quality of these libraries bounds the quality of the chip. This chapter looks inside them, using the real SkyWater SKY130 high-density library ([sky130\\_fd\\_sc\\_hd](#)) wherever possible.

## 4.1 Standard-cell architecture

A standard cell is a small, fixed-height layout implementing one logic function. Cells have the same height so that they abut in **rows**, sharing power rails at the top (VDD) and bottom (VSS). Alternate rows are flipped (mirrored about the x-axis) so that adjacent rows share a rail and an n-well.

```

----- width = N x placement site (0.46 um in SKY130 hd) ----->
+=====+
| n-well          pMOS devices (pull-up network)          | VPWR (met1 rail)
|      ==|==|==      ==|==|==      |
|      |      |      |      |      |
|      o A      poly gates run vertically          o Y (output pin) | height
|      o B      |      |      |      |      |      | 2.72 um
|      p-well    ==|==|==      ==|==|==      | (8 met1
|      nMOS devices (pull-down network)          | pitches)
+=====+
|
+=====+ VGND (met1 rail)

next row is flipped: it shares the VGND rail (or VPWR rail) with this row

```

## Anatomy of a standard cell: rails, wells, gates and pins.

## Key architectural parameters

- **Cell height** in routing **tracks**: height / M1 (or M2) pitch. Typical libraries range from about 12 tracks (older, high-drive) down to 6 or fewer tracks at advanced nodes. SKY130 hd cells are 2.72  $\mu\text{m}$  tall = 8 met1 pitches of 0.34  $\mu\text{m}$ . Taller cells drive harder and route more easily; shorter cells are denser.
- **Placement site**: the width quantum. Cells are an integer number of sites wide (0.46  $\mu\text{m}$  in SKY130 hd; one contacted poly pitch in FinFET libraries).
- **Fins per device** (FinFET): a 6-track cell might allow only 2 fins per device, a 7.5-track cell 3, which quantises drive strength.
- **Pin access**: at advanced nodes pins are small shapes on M1 or M0 and the router must reach each with a limited number of via positions; poor pin access is a major cause of routing congestion (Chapter 17).
- **Power rails**: on M1 (or buried/backside at the newest nodes); the PD flow later straps them to the power grid (Chapter 14).

## 4.2 Cell families

| Family              | Purpose                                                                                      | SKY130 hd examples                                                                                                                                            |
|---------------------|----------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Combinational logic | INV, BUF, NAND/NOR, AND/OR, XOR/XNOR, AOI/OAI complex gates, MUX, full/half adders, majority | <code>inv</code> , <code>nand2</code> , <code>a21oi</code> , <code>o22ai</code> , <code>xor2</code> , <code>mux2</code> , <code>fa</code> , <code>maj3</code> |
| Flip-flops          | D flops with/without reset/set, enable, scan mux (scan flops, Chapter 10)                    | <code>dfxtp</code> (plain), <code>dfrrtp</code> (async reset), <code>sdfxtp</code> (scan), <code>edfxtp</code> (enable)                                       |
| Latches             | Level-sensitive storage                                                                      | <code>dlxtp</code> , <code>dlrrtp</code>                                                                                                                      |
| Clock cells         | Balanced clock buffers/inverters, integrated clock gates (ICG)                               | <code>clkbuf</code> , <code>clkinv</code> , <code>dlclkp</code> (ICG), <code>sdlclkp</code>                                                                   |



| Family                     | Purpose                                                                                          | SKY130 hd examples                                                        |
|----------------------------|--------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| <b>Delay cells</b>         | Deliberate delay for hold fixing                                                                 | <code>dlygate4sd3, clkdlybuf4s50</code>                                   |
| <b>Low-power cells</b>     | Level shifters, isolation, retention, always-on buffers, power switches (Chapter 11)             | <code>lpflow_isobufsrc, lpflow_inputiso0p, lpflow_lsbuf_lh_isowell</code> |
| <b>Tie cells</b>           | Drive constant 0/1 through a device, never directly from the rail (protects gate oxide from ESD) | <code>conb</code> (tie hi/lo)                                             |
| <b>Physical-only cells</b> | Fill, decap (on-die decoupling), well-tap, endcap/boundary, antenna diodes                       | <code>decap_4, diode_2, tapvpwrvgnd_1, fill_1</code>                      |
| <b>Spare cells</b>         | Unused gates sprinkled for metal-only ECOs (Chapter 19)                                          | <code>macro_sparecell</code>                                              |

#### WHY SO MANY SPECIAL CELLS?

The physical-only cells exist because of the process: taps bias wells and prevent latch-up (Chapter 2); endcaps terminate rows so that the edge devices see the same environment as inner ones (layout-dependent effects, Chapter 3); decaps supply charge for local current spikes (Chapter 18); antenna diodes discharge plasma charge (Chapter 3); fill keeps densities legal.

### 4.3 Drive strengths, Vt flavours and variants

Each function comes in several **drive strengths** (x1, x2, x4...) built with wider devices or parallel fingers/fins. Larger drives have proportionally larger input capacitance and area - the logical-effort trade-off of Chapter 2. A library usually also comes in several **Vt flavours** with identical footprints (so they can be swapped late without re-placement) and sometimes in several **channel lengths** (e.g. a 'long-L' variant for lower leakage) or track heights. The real library summary below was produced by parsing the SKY130 hd Liberty file:

```
# Summarise the real SKY130 high-density library (typical corner, 25 C, 1.80 V).
import re, collections
lib = open("sky130_fd_sc_hd__tt_025C_1v80.lib").read()
cells = re.findall(r'cell \("sky130_fd_sc_hd__(\w+)\)", lib)
print("cells in library:", len(cells))
fam = collections.Counter(re.sub(r"_\d+$", "", c) for c in cells)
print("distinct functions:", len(fam))
for f in ("inv", "clkbuf", "nand2", "dfxtp", "sdfxtp", "dlclkp", "decap",
         "lpflow_isobufsrc", "diode"):
    sizes = sorted(int(c.split("_")[-1]) for c in cells if re.fullmatch(f + r"_\d+", c))
    print(" %17s drive/size variants: %s" % (f, " ".join(map(str, sizes))))

def cell_block(name):
    i = lib.index('cell ("sky130_fd_sc_hd__%s")' % name)
    depth, j = 0, lib.index("{", i)
    for k in range(j, len(lib)):
        depth += {"{": 1, "}": -1}.get(lib[k], 0)
        if depth == 0:
            return lib[i:k + 1]

print("cell          area(um2)  in cap(fF)  leakage(nW)")
for c in ("inv_1", "nand2_1", "nand2_2", "nand2_4", "nand2_8", "dfxtp_1"):
    b = cell_block(c)
    area = float(re.search(r"area : ([\d.]+)", b).group(1))
    pin = "A" if "nand" in c or "inv" in c else "D"
    cap = float(re.search(r'pin \("%s"\) \{s*capacitance : ([\d.]+)' % pin, b).group(1))
    leak = float(re.search(r"cell_leakage_power : ([\d.e-]+)", b).group(1))
    print("%-10s %9.3f %10.2f %12.4f" % (c, area, cap * 1e3, leak))
```

libsum.py - summarise the real sky130\_fd\_sc\_hd typical-corner Liberty file.

```

cells in library: 428
distinct functions: 158
  inv          drive/size variants: 1 2 4 6 8 12 16
  clkbuf       drive/size variants: 1 2 4 8 16
  nand2        drive/size variants: 1 2 4 8
  dfxtp       drive/size variants: 1 2 4
  sdfxtp      drive/size variants: 1 2 4
  dlclkp      drive/size variants: 1 2 4
  decap       drive/size variants: 3 4 6 8 12
  lpflow_isobufsrc drive/size variants: 1 2 4 8 16
  diode       drive/size variants: 2
cell      area(um2)  in cap(fF)  leakage(nW)
inv_1     3.754      2.30      0.0053
nand2_1   3.754      2.31      0.0021
nand2_2   6.256      4.43      0.0025
nand2_4   11.261     8.54      0.0071
nand2_8   20.019     16.93     0.0046
dfxtp_1   20.019     1.68      0.0084

```

Real output of libsum.py (area in  $\mu\text{m}^2$ , capacitance converted from pF to fF, leakage in the library's nW).

Note how area and input capacitance scale almost linearly with drive (nand2\_1 -> nand2\_8: area x5.3, input capacitance x7.3), and that a flip-flop is about five times the area of a NAND2. Area divided by the NAND2 area gives the **NAND2-equivalent gate count** (GE) used to size designs (Chapter 5).

## 4.4 Characterisation: how a Liberty file is made

Each cell is simulated in SPICE (with the PDK's device models and the extracted parasitics of the cell layout) at every PVT corner, sweeping **input slew** and **output load**. For every timing arc (input pin -> output pin, and for sequential cells clock -> output and data -> clock constraints) the tool measures:

- **Delay**: from the input crossing its threshold (typically 50 % of Vdd) to the output crossing its threshold.
- **Output transition (slew)**: typically 10-90 % or 20-80 % of the swing (scaled back to full swing using slew\_derate\_from\_library).
- **Setup, hold, recovery, removal** and **minimum pulse width** constraints of sequential cells, found by bisection on the data-to-clock offset until the output degrades by a defined criterion (in SKY130 note [violation\\_delay\\_degrade\\_pct : 10](#)).
- **Internal energy** per transition, **leakage** per input state, and **pin capacitance**.

```

cell netlist + extracted RC      PDK SPICE models      PVT corner list
      \              |              /
      +-----> characterisation tool <-----+
                (SPICE runs: slews x loads x arcs x states)
                |
      +-----+-----+-----+-----+
      v             v             v             v             v
NLDM tables      CCS / ECSM      LVF / POCV sigma  power tables  noise
(delay, slew)    (current/volt. (local variation) (internal,    (CCSN)
                  waveforms)
      +-----> Liberty .lib per corner -----> compiled .db / tool DB

```

Library characterisation flow.

A full library at an advanced node is characterised at dozens of corners (process x voltage x temperature x extraction corner), consuming very large compute farms; every new corner requested late in a project costs schedule.

## 4.5 Liberty in depth

Liberty (.lib) is a human-readable, group-and-attribute text format originally from Synopsys and now a de-facto standard. The structure is hierarchical: **library** -> **cell** -> **pin** -> **timing** / **internal\_power** groups, with lookup-table templates declared at the library level.

### The library header

```
library ("sky130_fd_sc_hd__tt_025C_1v80") {
    ...
    technology("cmos");
    delay_model : "table_lookup";
    bus_naming_style : "%s[%d]";
    time_unit : "1ns";
    voltage_unit : "1V";
    leakage_power_unit : "1nW";
    current_unit : "1mA";
    pulling_resistance_unit : "1kohm";
    capacitive_load_unit(1.0000000000, "pf");
    revision : 1.0000000000;
    ...
    operating_conditions ("tt_025C_1v80") {
        voltage : 1.8000000000;
        process : 1.0000000000;
        temperature : 25.000000000;
        tree_type : "balanced_tree";
    }
    ...
    lu_table_template ("del_1_7_7") {
        variable_1 : "input_net_transition";
        variable_2 : "total_output_net_capacitance";
        index_1("1, 2, 3, 4, 5, 6, 7");
        index_2("1, 2, 3, 4, 5, 6, 7");
    }
}
```

Real excerpt: header of the SKY130 hd typical-corner Liberty file (units, operating condition, a table template).

Units matter: in this library time is in ns, capacitance in pF and leakage in nW. A wrong unit assumption is a classic cause of absurd reports when mixing libraries. The template **del\_1\_7\_7** declares a 7 x 7 table indexed by input transition (**index\_1**) and total output capacitance (**index\_2**).

### A combinational cell: the SKY130 nand2\_1

```
cell ("sky130_fd_sc_hd__nand2_1") {
    leakage_power () {
        value : 0.0002796000;
        when : "!A&B";
    }
    ... (3 more state-dependent leakage_power groups)
    area : 3.7536000000;
    cell_footprint : "sky130_fd_sc_hd__nand2";
    cell_leakage_power : 0.0021179600;
    ... (pg_pin groups VGND, VNB, VPB, VPWR)
    pin ("A") {
        capacitance : 0.0023150000;
        clock : "false";
        direction : "input";
        fall_capacitance : 0.0022540000;
        ... (internal_power tables)
        max_transition : 1.5000000000;
        related_ground_pin : "VGND";
        related_power_pin : "VPWR";
    }
}
```

```

    rise_capacitance : 0.0023750000;
}
... (pin B is similar)
pin ("Y") {
    direction : "output";
    function : "(!A) | (!B)";
    ... (internal_power, one group per related_pin)
    max_capacitance : 0.1666360000;
    max_transition : 1.4963760000;
    timing () {
        cell_fall ("del_1_7_7") {
            index_1("0.0100000000, 0.0230506000, 0.0531329000, 0.1224740000, 0.282311 ...
            index_2("0.0005000000, 0.0013165500, 0.0034665900, 0.0091278700, 0.024034 ...
            values("0.0206305000, 0.0250594000, 0.0363371000, 0.0651531000, 0.1403625 ...
                "0.0243797000, 0.0289316000, 0.0403352000, 0.0696727000, 0.1447142000 ...
                "0.0327052000, 0.0384095000, 0.0504824000, 0.0797753000, 0.1551681000 ...
                "0.0428315000, 0.0514229000, 0.0698132000, 0.1038626000, 0.1794323000 ...
                "0.0525334000, 0.0659547000, 0.0937839000, 0.1461446000, 0.2370110000 ...
                "0.0550564000, 0.0754976000, 0.1173683000, 0.1976946000, 0.3328764000 ...
                "0.0332144000, 0.0632894000, 0.1271843000, 0.2473768000, 0.4578195000 ...
            ... (cell_rise, fall_transition tables)
        }
        related_pin : "A";
        ... (rise_transition table)
        timing_sense : "negative_unate";
        timing_type : "combinational";
    }
}
... (second timing group for related_pin "B")

```

Real excerpt of the nand2\_1 cell group (long lines cut at '...'; everything else verbatim).

| Construct                           | Meaning                                                                                                                                                                                     |
|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| area                                | Cell area in $\mu\text{m}^2$ (here $1.38 \times 2.72 \mu\text{m} = 3.7536$ )                                                                                                                |
| cell_footprint                      | Cells with the same footprint are interchangeable in layout (used for sizing/Vt swaps)                                                                                                      |
| leakage_power + when                | State-dependent leakage: 7.9 pW with both inputs high (A&B, output low) versus 0.03 pW with both low - stacking two off nMOS devices cuts leakage by orders of magnitude (the stack effect) |
| pin / direction / capacitance       | Pin capacitance in pF, separately for rise and fall                                                                                                                                         |
| function                            | Boolean function (used by synthesis, LEC, ATPG)                                                                                                                                             |
| max_transition,<br>max_capacitance  | Design-rule limits: the tables are valid only inside them; exceeding them is a DRV (design-rule violation) that STA reports and PD must fix                                                 |
| timing() + related_pin              | A timing arc from related_pin to this pin                                                                                                                                                   |
| timing_sense                        | positive_unate, negative_unate or non_unate (e.g. XOR)                                                                                                                                      |
| timing_type                         | combinational, rising_edge, setup_rising, hold_rising, recovery/removal, min_pulse_width, three_state_enable...                                                                             |
| cell_rise/cell_fall                 | Delay tables for output rising/falling                                                                                                                                                      |
| rise_transition/fall_transi<br>tion | Output slew tables                                                                                                                                                                          |
| internal_power                      | Energy tables per transition (excludes the $C V_{dd}^2$ of the external load, which power tools add from the net capacitance)                                                               |

## Sequential cells and constraint arcs

```

cell ("sky130_fd_sc_hd__dfxtp_1") {
    ...
    ff ("IQ", "IQ_N") {
        clocked_on : "CLK";
    }
}

```

```

    next_state : "D";
}

...
pin ("D") {
    capacitance : 0.0016780000;
    ...
    timing () {
        fall_constraint ("vio_3_3_1") {
            index_1("0.0100000000, 0.5000000000, 1.5000000000");
            index_2("0.0100000000, 0.5000000000, 1.5000000000");
            values("0.1033184000, 0.3187643000, 0.6206849000", \
                "-0.010809200, 0.1997539000, 0.4980123000", \
                "-0.096665400, 0.1090150000, 0.4036113000");
            ... (rise_constraint table)
        }
        timing_type : "setup_rising";
    }
    timing () { ... related_pin : "CLK" ...
        timing_type : "hold_rising";
    }
}

```

Real excerpt: the D-pin constraint arcs of the SKY130 hd flip-flop dftxp\_1.

The **ff** group declares the internal state (IQ, IQ\_N) and its next-state function. Constraint tables live on the *data* pin with **related\_pin** : "CLK"; **setup\_rising** and **hold\_rising** mean checks against the rising clock edge. The template **vio\_3\_3\_1** is indexed by related-pin (clock) transition and constrained-pin (data) transition. Negative hold values are normal - they mean data may change slightly *before* the clock edge without violation - because of the flop's internal clock-path delay.

## Integrated clock gate

```

cell ("sky130_fd_sc_hd__dlclkp_1") {
    ...
    clock_gating_integrated_cell : "latch_posedge";
    ...
    pin ("CLK") {
        clock_gate_clock_pin : "true";
        direction : "input";
        ...
    }
    pin ("GATE") {
        clock_gate_enable_pin : "true";
        direction : "input";
        ...
        timing_type : "setup_rising";
    }
    ...
    pin ("GCLK") {
        clock_gate_out_pin : "true";
        direction : "output";
        ...
        state_function : "(CLK*M0)";
        ...
        timing_type : "combinational";
    }
    ...
    pin ("M0") {
        direction : "internal";
        internal_node : "M0";
        ...
    }
    statetable ("CLK GATE", "M0") {
        table : "L L : - : L, L H : - : H, H - : - : N";
    }
}

```

Real excerpt: the ICG cell the SKY130 hd cell dlclkp\_1 (latch-based, rising-edge clock gate).

Attributes such as `clock_gating_integrated_cell`, `clock_gate_enable_pin` and `clock_gate_out_pin` tell synthesis this cell can implement clock gating (Chapter 11) and tell STA to perform clock-gating checks on GATE. The `statetable` models the internal latch: M0 follows GATE while CLK is low (the 'L L -> L', 'L H -> H' rows) and holds (N = no change) while CLK is high.

## 4.6 Looking up an NLDM table

An STA tool computes a cell delay by finding the input slew and the output load of the instance, then interpolating (bilinearly, inside the grid; extrapolating outside, which is inaccurate and flagged) in the relevant table. The script below does exactly that on the real SKY130 tables:

```
# NLDM lookup exactly as an STA tool does it: find the surrounding 2x2 grid points
# in (input slew, output load) and interpolate bilinearly (extrapolate at the edges).
import re
lib = open("sky130_fd_sc_hd__tt_025C_1v80.lib").read()

def cell_block(name):
    # same brace matcher as before
    i, depth = lib.index('cell ("sky130_fd_sc_hd__%s")' % name), 0
    for k in range(lib.index("{", i), len(lib)):
        depth += {"{": 1, "}": -1}.get(lib[k], 0)
        if depth == 0:
            return lib[i:k + 1]

def table(block, pin, related, kind):
    # returns (index_1, index_2, values) of table `kind` on the arc related -> pin
    out = block[block.index('pin ("%s")' % pin):]
    for arc in out.split("timing ()")[1:]:
        if 'related_pin : "%s"' % related not in arc:
            continue
        t = arc[arc.index(kind + " ("):]
        nums = lambda key: [float(x) for x in
                             re.search(key + r'\("[^"]+\)"', t).group(1).split(",")]
        i1, i2 = nums("index_1"), nums("index_2")
        rows = re.search(r'values\((.*?)\);', t, re.S).group(1)
        vals = [[float(x) for x in r.split(",")] for r in re.findall(r'"([^\"]+)"', rows)]
        return i1, i2, vals

def seg(axis, x):
    i = max(0, min(len(axis) - 2, sum(1 for a in axis if a <= x) - 1))
    return i, (x - axis[i]) / (axis[i + 1] - axis[i])

def lookup(i1, i2, v, x1, x2):
    i, u = seg(i1, x1)
    j, w = seg(i2, x2)
    return ((1 - u) * (1 - w) * v[i][j] + u * (1 - w) * v[i + 1][j]
            + (1 - u) * w * v[i][j + 1] + u * w * v[i + 1][j + 1])

nand = cell_block("nand2_1")
i1, i2, v = table(nand, "Y", "A", "cell_fall")
print("index_1 (slew, ns):", i1[:4], "...")
print("index_2 (load, pF):", i2[:4], "...")
slew, load = 0.080, 0.0050 # 80 ps input slew, 5 fF load
print("grid corners used: slew %.4f..%.4f load %.5f..%.5f" % (i1[2], i1[3], i2[2], i2[3]))
print("A->Y cell_fall at (80 ps, 5 fF) = %.1f ps" % (1e3 * lookup(i1, i2, v, slew, load)))
for kind in ("cell_rise", "fall_transition", "rise_transition"):
    t = table(nand, "Y", "A", kind)
    print("A->Y %-15s = %.1f ps" % (kind, 1e3 * lookup(*t, slew, load)))
ff = cell_block("dfxtp_1")
s1, s2, sv = table(ff, "D", "CLK", "rise_constraint") # first D arc is setup_rising
print("dfxtp_1 setup (D rise), CLK slew 0.1 ns, D slew 0.1 ns = %.1f ps"
```

```
% (1e3 * lookup(s1, s2, sv, 0.1, 0.1)))
```

nldm.py - parse a Liberty timing table and interpolate it like an STA tool.

```
index_1 (slew, ns): [0.01, 0.0230506, 0.0531329, 0.122474] ...
index_2 (load, pF): [0.0005, 0.00131655, 0.00346659, 0.00912787] ...
grid corners used: slew 0.0531..0.1225 load 0.00347..0.00913
A->Y cell_fall at (80 ps, 5 fF) = 66.4 ps
A->Y cell_rise = 82.7 ps
A->Y fall_transition = 50.7 ps
A->Y rise_transition = 61.2 ps
dfxtp_1 setup (D rise), CLK slew 0.1 ns, D slew 0.1 ns = 58.7 ps
```

Real output of nldm.py on the real SKY130 typical-corner tables.

$$u = (x1 - x1_i) / (x1_{(i+1)} - x1_i), \quad w = (x2 - x2_j) / (x2_{(j+1)} - x2_j)$$

$$D = (1-u)(1-w) T[i][j] + u(1-w) T[i+1][j] + (1-u)w T[i][j+1] + uw T[i+1][j+1]$$

Bilinear interpolation between the four surrounding grid points.

So a nand2\_1 driven with an 80 ps slew into 5 fF has a falling delay of about 66 ps and a rising delay of about 83 ps at the typical corner (the pMOS pull-up is weaker). The output slew then becomes the input slew of the next stage - which is how slew propagates through a path (Chapter 9).

## Beyond NLDM: CCS, ECSM, LVF and power

| Model                                  | What it adds                                                                                                | Why                                                                                                                              |
|----------------------------------------|-------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <b>NLDM</b>                            | Delay and slew tables vs (slew, load)                                                                       | Simple, fast; assumes a ramp input and a lumped C load                                                                           |
| <b>CCS (Synopsys) / ECSM (Cadence)</b> | Output current (CCS) or voltage (ECSM) waveforms vs time for each (slew, load); receiver capacitance tables | Accurate with resistive wires (effective capacitance), non-ramp waveforms, and below ~65 nm; also CCS noise (CCSN) and CCS power |
| <b>LVF (Liberty Variation Format)</b>  | Sigma tables (ocv_sigma_cell_rise, ...) per arc, optionally with mean shift and skewness                    | Statistical local variation for POCV/SOCV analysis (Chapter 9)                                                                   |
| <b>Power tables</b>                    | internal_power (energy per event vs slew/load), state-dependent leakage_power                               | Dynamic and leakage power analysis                                                                                               |
| <b>Aging / reliability</b>             | Libraries characterised with aged models (e.g. 10-year BTI/HCI) or EM limits per pin                        | Lifetime sign-off (Chapter 18)                                                                                                   |

## 4.7 Using a library: dont\_use lists and a real synthesis

Not every cell in a library should be used by synthesis. Clock cells should appear only in clock trees, delay cells only for hold fixing, isolation and level-shifter cells only where the UPF puts them, and some cells are simply poor or unreliable at a given corner. Flows therefore maintain a **dont\_use** list. To show why, a 16x16 multiply-accumulate block was synthesised with Yosys against the real SKY130 Liberty, first as-is:

```
module mac16 (input clk, input rst_n, input en,
              input [15:0] a, b, output reg [39:0] acc);
  always @(posedge clk or negedge rst_n)
    if (!rst_n) acc <= 40'd0;
    else if (en) acc <= acc + a * b;
endmodule
```

mac.v - a registered 16 x 16 -> 40-bit MAC.

```

Number of cells:          1401
...
sky130_fd_sc_hd__dfntp_1      40
sky130_fd_sc_hd__lpflow_inputiso1p_1    15
sky130_fd_sc_hd__lpflow_isobufsrc_1     49
...
Chip area for module '\mac16': 10720.281600

```

Real Yosys 0.33 'stat -liberty' excerpt, full library: the mapper picked lpflow isolation cells as ordinary gates.

ABC chose `lpflow_inputiso1p` and `lpflow_isobufsrc` cells - isolation cells that happen to compute an AND/OR function - for ordinary logic. In a real flow those cells would confuse power-intent verification and may have unsuitable characteristics. Marking them `dont_use : true` fixes it:

```

# Mark special-purpose cells dont_use so the mapper never picks them for plain logic.
import re
DONT_USE = r"lpflow\w+|probe\w+|macro_sparecell|clkdybuf\w+|dlygate\w+|dlymetal\w+"
src = open("sky130_fd_sc_hd__tt_025C_1v80.lib").read()
pat = re.compile(r'(cell \("sky130_fd_sc_hd__(?:%s)_d+\)" \{) ' % DONT_USE)
dst, n = pat.subn(r'\1\n          dont_use : true;', src)
open("hd_tt_dontuse.lib", "w").write(dst)
print("cells marked dont_use:", n)

```

dontuse.py - mark special-purpose cells dont\_use in a copy of the Liberty file.

```
cells marked dont_use: 50
```

Real output of dontuse.py.

```

read_verilog mac.v
synth -top mac16 -flatten
dfflibmap -liberty hd_tt_dontuse.lib
abc -D 5000 -liberty hd_tt_dontuse.lib
opt_clean
stat -liberty hd_tt_dontuse.lib

```

mac2.py - the Yosys script (synthesis, flop mapping, ABC mapping with a 5 ns delay target, statistics).

```

Number of cells:          1416
...
sky130_fd_sc_hd__dfntp_1      40
sky130_fd_sc_hd__maj3_1      101
sky130_fd_sc_hd__nand2_1     226
...
sky130_fd_sc_hd__xnor2_1     211
...
Chip area for module '\mac16': 10767.827200

```

Real Yosys 0.33 'stat -liberty' excerpt with dont\_use applied.

The result is 1416 cells and 10768  $\mu\text{m}^2$ , i.e. about 2870 NAND2 equivalents (10768 / 3.7536), of which the 40 reset flops (`dfntp_1`) are a significant share. We will reuse this number in Chapter 5.

#### PITFALL: TRUSTING THE FULL LIBRARY

Always review the dont\_use list shipped with the library and add your own (weak x0/x1 cells at the slow corner, cells with known silicon issues, clock and delay cells in data paths). Also check that dont\_use is applied consistently in synthesis, PD and ECO tools - a mismatch lets an ECO reintroduce a forbidden cell.

## 4.8 LEF: the physical abstract

Place-and-route tools do not need the full transistor layout; they need outlines, pins and blockages. LEF (Library Exchange Format) provides this in two parts:



- **Technology LEF:** units, manufacturing grid, placement **SITES**, routing **LAYERS** (direction, pitch, width, spacing tables, resistance and capacitance per square), **VIA** definitions and via rules. Produced from the PDK.
- **Cell/macro LEF:** for each cell a **MACRO** with **CLASS** (CORE, BLOCK, PAD, ENDCAP...), **SIZE**, **SITE**, **SYMMETRY**, **PINS** with their **PORT** geometry and direction/use, antenna information, and **OBS** (obstructions: metal the router must avoid).

```

SITE unithd
  SYMMETRY Y ;
  CLASS CORE ;
  SIZE 0.46 BY 2.72 ;
END unithd

LAYER met1
  TYPE ROUTING ;
  DIRECTION HORIZONTAL ;
  PITCH 0.34 ;
  OFFSET 0.17 ;
  WIDTH 0.14 ;                      # Met1 1
  # SPACING 0.14 ;                  # Met1 2

```

Real excerpt of sky130\_fd\_sc\_hd.tlef: the placement site and the met1 routing layer.

```

MACRO sky130_fd_sc_hd__nand2_1
  CLASS CORE ;
  FOREIGN sky130_fd_sc_hd__nand2_1 ;
  ORIGIN 0.000000 0.000000 ;
  SIZE 1.380000 BY 2.720000 ;
  SYMMETRY X Y R90 ;
  SITE unithd ;
  PIN A
    ANTENNAGATEAREA 0.247500 ;
    DIRECTION INPUT ;
    USE SIGNAL ;
    PORT
      LAYER li1 ;
      RECT 0.940000 1.075000 1.275000 1.325000 ;
    END
  END A
  ... (PIN B and PIN Y are similar)
  PIN VPWR
    DIRECTION INOUT ;
    SHAPE ABUTMENT ;
    USE POWER ;
    PORT
      LAYER met1 ;
      RECT 0.000000 2.480000 1.380000 2.960000 ;
    END
  END VPWR
  OBS
    LAYER li1 ;
    RECT 0.000000 -0.085000 1.380000 0.085000 ;
    RECT 0.000000 2.635000 1.380000 2.805000 ;
    ...
  END
END sky130_fd_sc_hd__nand2_1

```

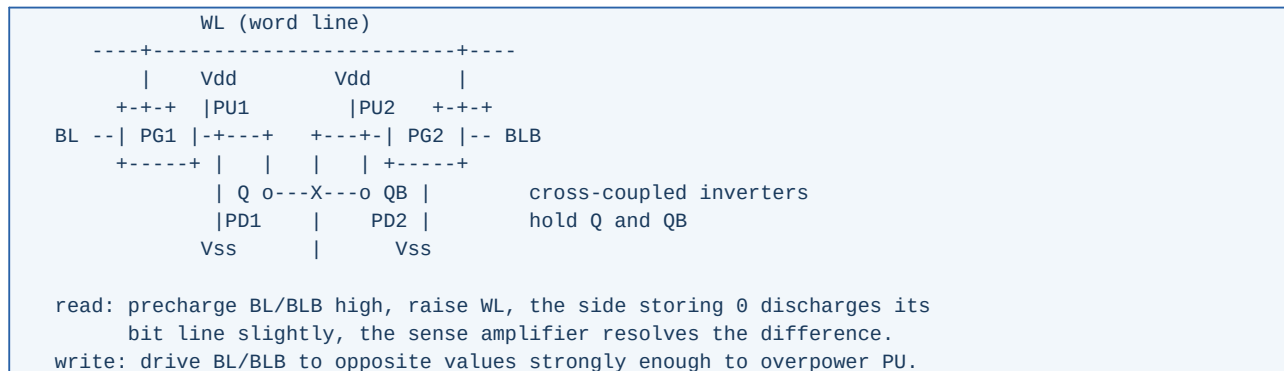
Real excerpt of the cell LEF for the SKY130 hd nand2\_1.

The nand2\_1 is 1.38  $\mu\text{m}$  wide = 3 sites of 0.46  $\mu\text{m}$ . Its signal pins are on li1 (the local interconnect), its power pins are met1 rails that extend 0.24  $\mu\text{m}$  beyond the cell edge so that they merge with the neighbour row (**SHAPE ABUTMENT**), **ANTENNAGATEAREA** feeds the router's antenna checks (Chapter 17), and the VPB/VNB pins are the separate n-well and p-well bias connections mentioned in Chapter 2. A cell's LEF, Liberty, Verilog model and

GDS must agree exactly on names, pins and size - checking this is part of library QA.

## 4.9 Memories: SRAM compilers, register files and ROM

Memories typically occupy a large fraction of an SoC's area - often a third to more than half. They are delivered by **memory compilers**: generators that tile a full-custom bitcell array with custom periphery (decoders, sense amplifiers, write drivers, timing control) for a requested words x bits configuration and emit all views.



The 6T SRAM bitcell: two cross-coupled inverters (PU/PD) and two pass gates (PG).

The bitcell is sized for a delicate balance: PD must be stronger than PG for **read stability** (a read must not flip the cell), and PG stronger than PU for **writability**. Local mismatch (Pelgrom, Chapter 3) disturbs both, so bitcells set the minimum voltage ( $V_{min}$ ) of many chips and often need **assist circuits** (word-line underdrive, negative bit line) and sometimes a separate, higher memory supply. Foundry bitcells use special 'pushed' design rules that are denser than logic rules - which is why you cannot build a competitive SRAM out of standard cells.

| Memory type                            | Characteristics                                                    | Typical use                     |
|----------------------------------------|--------------------------------------------------------------------|---------------------------------|
| <b>High-density SRAM (single-port)</b> | Smallest bitcell, slower, one access per cycle                     | Large buffers, caches L2/L3     |
| <b>High-speed / dual-port SRAM</b>     | Larger 8T/dual-port cells, separate read/write ports               | L1 caches, FIFOs, frame buffers |
| <b>Register file (compiler)</b>        | Small, fast, multi-port arrays                                     | CPU register files, small FIFOs |
| <b>Flop/latch-based arrays</b>         | Built from standard cells; large but flexible and testable by scan | Tiny memories (< ~1 kbit)       |
| <b>ROM</b>                             | Contents programmed by a mask layer (via/diffusion)                | Boot code, look-up tables       |
| <b>OTP / eFuse / eFlash / MRAM</b>     | Non-volatile; special process options or IP                        | Trimming, keys, firmware        |

A memory compiler delivers: Liberty (per corner, often with separate array and periphery supplies), LEF/abstract, GDS, behavioural Verilog, a **BIST/repair** interface (MBIST and redundant rows/columns, Chapter 10), and datasheets. Memory power-down modes (light sleep, deep sleep, shut-down) must be wired to the power-management logic and described in UPF (Chapter 11).

## 4.10 I/O libraries

I/O cells connect the core to the package: digital I/O buffers (with programmable drive, slew, pull-up/down, Schmitt trigger), analog pads, power/ground pads for core and I/O supplies, corner cells, filler/spacer cells and **power-cut** cells that split the ring into voltage domains. Each I/O contains ESD protection and level shifting between the thick-oxide I/O voltage (1.8, 2.5, 3.3 V) and the core voltage. I/O rings, bump-pitch versus pad-pitch limits and ESD are covered in Chapters 12 and 14.

## 4.11 IP: soft, firm and hard

| Type           | Delivered as                                          | Flexibility                                  | Examples                                                           |
|----------------|-------------------------------------------------------|----------------------------------------------|--------------------------------------------------------------------|
| <b>Soft IP</b> | Synthesisable RTL + constraints + testbench           | Portable to any node; you own timing closure | CPU cores, interconnect, controllers (USB, DDR controller), crypto |
| <b>Firm IP</b> | Gate-level netlist, sometimes with placement guidance | Node/library specific; some flexibility      | Pre-hardened subsystems, obfuscated cores                          |
| <b>Hard IP</b> | GDS + LEF abstract + Liberty (+ models)               | Fixed layout for one process; no changes     | PLLs, SerDes/PHYs, ADCs, SRAMs, I/Os, eFuse                        |

### IP qualification and integration

Buying IP transfers design effort, not responsibility: if the PHY fails, your chip fails. A disciplined team qualifies each IP before committing to it:

#### IP qualification and integration deliverables

- ☐ Silicon proven? In which exact process/PDK version and metal stack, with test-chip characterisation data and errata.
- ☐ Complete views: RTL or netlist, Liberty for every sign-off corner you need, LEF, GDS, Verilog/behavioural models, SPICE/IBIS for analog and I/O, CDL netlist for LVS.
- ☐ Constraints: SDC for soft IP, timing models (ETM) for hard IP, UPF/power-intent fragments, CDC/RDC waivers.
- ☐ Verification collateral: testbenches, VIP, coverage reports, assertion packs, protocol compliance results.
- ☐ DFT: scan-ready, MBIST, test modes, JTAG/IEEE 1500 wrappers, test patterns for hard IP.
- ☐ Integration guide: floorplan and routing blockages, power supply and decap requirements, noise sensitivity, keep-out zones, bump/pad assignments.
- ☐ Clean DRC/LVS/antenna reports in the target PDK version, and the EM/IR data needed for chip-level sign-off.
- ☐ Deliverable quality checks: consistent pin names across all views, correct units, dont\_touch/dont\_use requirements, licence terms (per-design, royalty), support and export control.

#### INTERVIEW INSIGHT: THE INTEGRATION BUGS THAT ESCAPE

Common real-world IP problems: Liberty corners that do not match your sign-off corners (so STA extrapolates), a LEF abstract whose OBS does not match the GDS (routing shorts found only at full-chip DRC), reset or clock requirements hidden in a footnote, and an IP that was 'silicon proven' in a different metal stack. Qualify early and run the IP through your own flow in a test harness.

## 4.12 Summary

- Standard cells are fixed-height layouts in rows sharing rails; key parameters are track height, site width, fins per device and pin access. SKY130 hd: 0.46 x 2.72 um site.
- Libraries contain logic, sequential, clock, delay, low-power, tie and physical-only cells, in several drive strengths and Vt flavours with common footprints.
- Characterisation runs SPICE over slews x loads x corners; Liberty stores function, pin caps, NLDM (or CCS/ECSM) delay and slew tables, constraints (setup/hold via timing\_type and related\_pin), state-dependent power (when) and LVF sigma tables.
- STA interpolates tables bilinearly; the real SKY130 nand2\_1 gives ~66 ps fall delay at 80 ps slew and 5 fF.
- Use dont\_use lists - a real Yosys run mapped logic onto isolation cells without one.
- LEF provides the tech (sites, layers, vias) and cell/macro abstracts (size, pins, OBS, antenna data).
- Memories come from compilers around 6T/8T bitcells that set Vmin; I/O libraries provide pads with ESD; IP is soft, firm or hard and must be qualified with a full deliverable checklist.

## 4.13 Exercises

- 1 Using `nldm.py`, compute the `nand2_1` A->Y rise and fall delays at (20 ps, 2 fF) and (400 ps, 50 fF). Which is limited by the input slew and which by the load?
- 2 Why does the Liberty leakage of `nand2_1` differ by more than two orders of magnitude between input states A&B and !A&!B?
- 3 Explain the meaning of a negative hold constraint in the `dfxtp_1` table. Under which conditions would STA report a hold violation anyway?
- 4 List the LEF and Liberty attributes that must agree for a hard macro, and describe the failure that results if each disagrees.
- 5 Estimate how many SKY130 hd sites and rows a block of 10768  $\mu\text{m}^2$  needs at 70 % utilisation, and the side of a square core.
- 6 Write a `dont_use` policy for a design with UPF power domains and explain each entry.

## 5. Specification, Architecture and Micro-architecture: From Requirements to PPA Targets

Most expensive ASIC failures are not caused by a timing violation or a DRC error. They are caused by building the wrong chip: a feature the customer needed was missing, the memory bandwidth could not feed the compute, the power budget was exceeded in a mode nobody modelled, the die came out 30 % larger than the cost model assumed. The front of the flow - requirements, architecture and micro-architecture - is where those decisions are made, cheaply, on paper and in models. This chapter describes how professional teams do it and how to put numbers on power, performance and area (PPA) before any RTL exists.

### 5.1 Requirements: what the chip must do

Requirements come from customers, marketing, standards and regulations. They are captured in a **market requirements document** (MRD, the 'what and why': target market, use cases, competitors, price point, volumes, launch window) and refined into a **product requirements document** (PRD, the 'what exactly': measurable requirements the chip is designed and tested against).

| Category                       | Examples of measurable requirements                                                                              |
|--------------------------------|------------------------------------------------------------------------------------------------------------------|
| <b>Functional</b>              | Supported interfaces and modes; algorithms; software/boot flow; debug features                                   |
| <b>Performance</b>             | Throughput (frames/s, TOPS, Gb/s), latency, clock frequencies, number of concurrent streams                      |
| <b>Power</b>                   | Active power per use case, idle and standby power, peak current, thermal design power, battery life              |
| <b>Area / cost</b>             | Die-size target, package size and type, bill of materials, target selling price and margin                       |
| <b>Schedule</b>                | Tapeout, sample, qualification and production dates                                                              |
| <b>Standards / compliance</b>  | PCIe/USB/MIPi/Ethernet compliance, JEDEC memory standards, EMC, RoHS                                             |
| <b>Quality and reliability</b> | Operating temperature grade, lifetime (e.g. 10 years at a mission profile), DPPM target, AEC-Q100 for automotive |
| <b>Safety</b>                  | ISO 26262 ASIL level (automotive), IEC 61508 (industrial): safety mechanisms, diagnostic coverage                |
| <b>Security</b>                | Secure boot, key storage, side-channel and fault attack resistance, certifications (e.g. Common Criteria, FIPS)  |

#### WRITE REQUIREMENTS YOU CAN TEST

'Low power' is not a requirement. 'Average power  $\leq$  350 mW in use case UC-3 (1080p30 inference, 25 deg C ambient, typical silicon), measured at the board supply' is. Every PRD line should have an ID, a priority (must/should/may), a verification method (simulation, emulation, silicon measurement, analysis) and an owner.

### 5.2 Architecture exploration

The architect's job is to find the cheapest structure that meets the requirements with margin. Exploration uses models of increasing fidelity:

| Model                                 | Fidelity / speed           | Answers                                                       |
|---------------------------------------|----------------------------|---------------------------------------------------------------|
| <b>Spreadsheet / analytical</b>       | Seconds; first-order       | Rough bandwidth, compute, memory size, power and area budgets |
| <b>Roofline and queueing models</b>   | Seconds-minutes            | Compute- vs memory-bound, buffer depths, latency under load   |
| <b>Python / C++ performance model</b> | Minutes; cycle-approximate | Workload-driven throughput, utilisation, what-if studies      |

| Model                                                  | Fidelity / speed                                  | Answers                                                                  |
|--------------------------------------------------------|---------------------------------------------------|--------------------------------------------------------------------------|
| <b>SystemC/TLM virtual platform</b>                    | Near real time;<br>transaction-level              | Software bring-up before RTL, interconnect contention,<br>early firmware |
| <b>RTL simulation / emulation /<br/>FPGA prototype</b> | Cycle-accurate; slow (sim) to<br>fast (emulation) | Final performance validation, power estimation with real<br>activity     |

SystemC (IEEE 1666) with TLM-2.0 lets architects build a virtual platform in which CPU instruction-set simulators run real software against loosely-timed or approximately-timed models of the accelerators and interconnect. It is also the input language of high-level synthesis tools.

## The roofline model

For compute engines the **roofline** answers the most important early question: is the design limited by compute or by memory bandwidth? Attainable throughput is the minimum of peak compute and bandwidth x **arithmetic intensity** (operations per byte moved to/from memory). The **ridge point** peak / bandwidth is the intensity above which the engine becomes compute-bound.

$$\text{Attainable} = \min(\text{Peak\_ops}, \text{BW\_mem} * \text{AI}) \quad \text{AI} = \text{ops} / \text{bytes}$$

$$\text{Ridge point AI}^* = \text{Peak\_ops} / \text{BW\_mem}$$

Roofline model.

```
# Roofline check for an edge NPU: attainable = min(peak compute, bandwidth * intensity)
PEAK_OPS = 4e12                                # 4 TOPS INT8 (2048 MACs x 2 ops x ~1 GHz)
BW = 2 * 4 * 4266e6                             # LPDDR4x, 2 channels x 32 bit (4 B) x 4266 MT/s
EFF = 0.70                                       # sustained DRAM efficiency (refresh, page misses)
bw = BW * EFF
print("peak %.1f TOPS, DRAM %.1f GB/s (sustained %.1f), ridge = %.0f ops/byte"
      % (PEAK_OPS / 1e12, BW / 1e9, bw / 1e9, PEAK_OPS / bw))
# (name, ops, DRAM bytes moved) per layer, weights + activations, INT8
layers = [("conv3x3 64->64 @56x56", 2 * 56 * 56 * 64 * 64 * 9, 56 * 56 * 64 * 2 + 64 * 64 * 9),
          ("depthwise 3x3 @112x112x32", 2 * 112 * 112 * 32 * 9, 112 * 112 * 32 * 2 + 288),
          ("FC 4096->4096 (batch 1)", 2 * 4096 * 4096, 4096 * 4096 + 2 * 4096),
          ("FC 4096->4096 (batch 32)", 2 * 32 * 4096 * 4096, 4096 * 4096 + 64 * 4096)]
print("%-28s %8s %9s %8s" % ("layer", "ops/B", "TOPS", "bound"))
for name, ops, byts in layers:
    ai = ops / byts
    att = min(PEAK_OPS, bw * ai)
    print("%-28s %8.1f %9.2f %8s" % (name, ai, att / 1e12,
                                     "compute" if att == PEAK_OPS else "memory"))
```

roofline.py - which neural-network layers can keep a 4 TOPS edge NPU busy on an LPDDR4x interface?

```
peak 4.0 TOPS, DRAM 34.1 GB/s (sustained 23.9), ridge = 167 ops/byte
layer                ops/B      TOPS    bound
conv3x3 64->64 @56x56  527.6    4.00  compute
depthwise 3x3 @112x112x32  9.0    0.21  memory
FC 4096->4096 (batch 1)  2.0    0.05  memory
FC 4096->4096 (batch 32) 63.0    1.51  memory
```

Real output of roofline.py.

Only the dense convolution reaches peak; depthwise convolutions and batch-1 fully connected layers are badly memory-bound. This single table drives major architectural decisions: add on-chip SRAM so weights and activations stay local (raising effective intensity via reuse and layer fusion), widen the DRAM interface, add weight compression, or accept lower utilisation and shrink the MAC array to save area and leakage.

## 5.3 Hardware/software partitioning

Every function can be implemented in software on a CPU, on a DSP or programmable accelerator, or in fixed-function hardware. The trade-off is flexibility versus efficiency: fixed-function logic is often orders of magnitude more energy-efficient than a general-purpose CPU for the same task, but it cannot be changed after tapeout.

- Put in hardware: high-throughput, stable, regular kernels (video codecs, crypto, DSP filters, NN MAC arrays, packet parsing at line rate) and anything with hard real-time or security requirements.
- Keep in software: control, evolving algorithms, protocol upper layers, anything likely to change or rarely executed.
- Middle ground: programmable accelerators, DSPs with custom instructions, microcoded engines, eFPGA (Chapter 1).
- Define the HW/SW interface precisely: register map, interrupts, DMA descriptors, memory ordering, firmware responsibilities. The register map is typically generated from a single source (IP-XACT or SystemRDL) into RTL, C headers and documentation.

## 5.4 Memory hierarchy and bandwidth budgets

Memory decisions dominate area, power and performance of most SoCs. The architect sizes each level - registers, local SRAM buffers, shared L2/L3 or system cache, external DRAM - from the workload's working set and reuse, and then budgets bandwidth at every interface.

|                                                                    |          |              |                          |                   |
|--------------------------------------------------------------------|----------|--------------|--------------------------|-------------------|
| DRAM (LPDDR/DDR/HBM) -- 34 GB/s peak, ~24 GB/s sustained (example) |          |              |                          |                   |
|                                                                    |          |              |                          |                   |
| memory controller + PHY                                            |          |              |                          |                   |
|                                                                    |          |              |                          |                   |
| system interconnect (NoC / AXI)                                    |          |              | budget per master (GB/s) |                   |
|                                                                    |          |              |                          |                   |
| CPU L2                                                             | NPU SRAM | ISP / video  | CPU cluster              | 4                 |
|                                                                    | (4 MB)   | line buffers | NPU                      | 14                |
| CPU L1                                                             |          |              | ISP + video              | 4                 |
|                                                                    |          |              | display                  | 1.5               |
|                                                                    |          |              | misc. DMA                | 0.5               |
|                                                                    |          |              | -----                    |                   |
|                                                                    |          |              | total                    | 24.0 <= sustained |

A memory hierarchy with a bandwidth budget: the sum of concurrent demands must fit the sustained, not the peak, DRAM bandwidth.

- Use **sustained** bandwidth: DRAM efficiency is typically 60-85 % of peak depending on access pattern, refresh and read/write turnarounds.
- Budget **concurrent** use cases: display refresh is real-time and must never starve; QoS and priority schemes in the interconnect follow from this analysis.
- Budget **latency** too: a CPU cache miss or a real-time DMA may need a bound, which drives outstanding-transaction counts and buffer depths (Little's law: outstanding = bandwidth x latency).
- Every on-chip SRAM adds area and leakage; every DRAM access costs roughly one to two orders of magnitude more energy than an on-chip SRAM access. Data movement, not arithmetic, usually dominates energy in data-intensive designs.

Little's law turns the bandwidth budget into hardware: to sustain bandwidth B through a memory system with round-trip latency T, a master must keep B x T bytes in flight, which sets the number of outstanding transactions its bus interface and the interconnect must support, and the size of reorder and data buffers. Real-time masters also need FIFOs deep enough to ride through the worst-case stall.

```
# Little's law sizing: outstanding data = bandwidth x latency
import math
BURST = 64 # bytes per AXI burst (e.g. 16 beats x 4 B)
print("%-22s %8s %9s %12s %10s" % ("master", "GB/s", "lat (ns)", "in flight", "bursts"))
for name, gbps, lat_ns in (("CPU cache refills", 4.0, 150), ("NPU weight fetch", 14.0, 300),
                           ("ISP/video DMA", 4.0, 400), ("display (worst lat.)", 1.5, 2000)):
    inflight = gbps * lat_ns # GB/s * ns = bytes
    print("%-22s %8.1f %9d %10.0f B %10d" % (name, gbps, lat_ns, inflight,
                                             math.ceil(inflight / BURST)))
# Display FIFO: must hide the worst DRAM stall (e.g. refresh + other masters) of 2 us
stall_ns, drain_gbps = 2000, 1.5
print("display FIFO to hide a %d ns stall: %.0f bytes -> %.1f KB SRAM"
      % (stall_ns, drain_gbps * stall_ns, drain_gbps * stall_ns / 1024))
```

little.py - outstanding transactions and FIFO depth from bandwidth and latency.

| master                                                          | GB/s | lat (ns) | in flight | bursts |
|-----------------------------------------------------------------|------|----------|-----------|--------|
| CPU cache refills                                               | 4.0  | 150      | 600 B     | 10     |
| NPU weight fetch                                                | 14.0 | 300      | 4200 B    | 66     |
| ISP/video DMA                                                   | 4.0  | 400      | 1600 B    | 25     |
| display (worst lat.)                                            | 1.5  | 2000     | 3000 B    | 47     |
| display FIFO to hide a 2000 ns stall: 3000 bytes -> 2.9 KB SRAM |      |          |           |        |

Real output of little.py.

The NPU needs about 66 outstanding 64-byte bursts to reach its 14 GB/s share - an interconnect or memory controller that supports only 16 outstanding reads per master would silently cap it at roughly a quarter of the budget. Numbers like these go straight into the uArch specs of the DMA engines and the NoC configuration.

## 5.5 Clock and power domain planning

Clocking and power architecture are decided at architecture time because they shape RTL, verification, DFT and physical design.

| Decision                             | Considerations                                                                                                      | Detailed in                    |
|--------------------------------------|---------------------------------------------------------------------------------------------------------------------|--------------------------------|
| <b>Clock sources and frequencies</b> | PLL count, reference clocks, frequency plan per block, integer ratios vs asynchronous domains, jitter needs of PHYs | Chapters 12, 16                |
| <b>Clock domain crossings</b>        | Number of asynchronous domains, synchroniser strategy, CDC verification                                             | Companion RTL guide, Chapter 6 |
| <b>Voltage domains</b>               | Which blocks need independent supplies or DVFS; level shifters; I/O voltages                                        | Chapter 11                     |
| <b>Power gating</b>                  | Which blocks shut off, retention needs, wake-up latency, rush current                                               | Chapters 11, 18                |
| <b>Power modes</b>                   | A mode table: which domains are on/off, at which voltage/frequency, in each system state                            | Chapter 11                     |
| <b>Reset architecture</b>            | Reset domains, sequencing, synchronisers, reset-domain crossings                                                    | Chapter 6                      |

## 5.6 IP selection

Buy-versus-build is decided per block. Buy standard-heavy, verification-heavy or analog IP (CPU cores, PCIe/USB/DDR controllers and PHYs, PLLs, memory compilers) unless it is your differentiation. Build what differentiates you. Evaluate vendors with the qualification checklist of Chapter 4, and note that IP choice constrains node choice: the PHY you need may exist in only one foundry's process or one metal stack.



## 5.7 Early die-size and power estimation

Long before synthesis, the architect must predict die size (it drives cost, package and yield) and power (it drives package, thermal solution and battery life). The method is simple and every input is refined as the project proceeds:

- **Logic:** estimate each block in NAND2-equivalent gates (GE) from similar past designs or quick synthesis of key blocks.  $\text{Area} = \text{GE} \times \text{NAND2 area} / \text{utilisation}$ . In Chapter 4 a real SKY130 synthesis of a 16-bit MAC gave about 2870 GE ( $10768 \text{ um}^2 / 3.7536 \text{ um}^2$ ), so a block of that complexity costs about  $0.015 \text{ mm}^2$  at 70 % utilisation in SKY130.
- **Memory:** bits x bitcell area / array efficiency, or better, the compiler's datasheet for the exact configurations.
- **Hard IP:** the vendor's dimensions (PHYs are large and often fixed to die edges).
- **I/O ring and overheads:** pad ring or bump field, seal ring, scribe line; check whether the die is **pad-limited** (the perimeter needed by the I/O count exceeds the core-limited size).

```
# Early die-size, cost and power estimate for an edge-AI SoC. Every input is an
# ASSUMPTION to be replaced by your foundry/IP data; the method is what matters.
import math
NAND2_UM2 = 0.20          # NAND2-equivalent footprint, 16/12 nm-class order of magnitude
UTIL      = 0.65          # std-cell placement utilisation after CTS/buffering headroom
logic_ge  = {"CPU cluster": 6.0e6, "NPU": 9.0e6, "interconnect+periph": 3.0e6,
             "DDR/USB controllers": 1.5e6}
sram_bits = 8 * 8 * 2**20    # 8 MB of on-chip SRAM
BIT_UM2, ARRAY_EFF = 0.074, 0.55 # HD 6T bitcell; periphery shrinks usable fraction
hard_ip_mm2 = {"LPDDR4x PHY": 2.4, "USB3 PHY": 0.8, "PLLs+sensors": 0.4}
logic = sum(logic_ge.values()) * NAND2_UM2 / UTIL / 1e6
sram = sram_bits * BIT_UM2 / ARRAY_EFF / 1e6
core = logic + sram + sum(hard_ip_mm2.values())
die = core * 1.12           # IO ring, seal ring, scribe share
print("logic %.1f mm2 (%.1f M GE)  SRAM %.1f mm2  hard IP %.1f mm2"
      % (logic, sum(logic_ge.values()) / 1e6, sram, sum(hard_ip_mm2.values())))
print("die area ~ %.1f mm2  (%.2f mm x %.2f mm)" % (die, math.sqrt(die), math.sqrt(die)))
d = 300.0                  # wafer diameter, mm
gross = math.pi * (d / 2) ** 2 / die - math.pi * d / math.sqrt(2 * die)
D0 = 0.10                  # defects/cm2 on a mature line
y = (1 + die / 100 * D0 / 3) ** -3 # negative binomial, alpha = 3
print("gross dies/wafer %.0f, yield %.1f %, good dies %.0f" % (gross, 100 * y, gross * y))
print("die cost at $9000/wafer: $%.2f" % (9000 / (gross * y)))
# Dynamic power P = a * C * V^2 * f with ~1 fF switched per GE (cell + wire)
a, c_ge, v, f = 0.12, 1.0e-15, 0.80, 1.0e9
p_logic = a * c_ge * sum(logic_ge.values()) * v * v * f
p_clock = 0.35 * p_logic / 0.65 # clock tree ~35 % of dynamic
p_leak = 0.10 * (p_logic + p_clock)
print("power: logic %.2f W + clock %.2f W + leakage %.2f W = %.2f W (+ SRAM, IO, PHY)"
      % (p_logic, p_clock, p_leak, p_logic + p_clock + p_leak))
```

diesize.py - early die area, dies per wafer, yield, die cost and logic power for an edge-AI SoC. All inputs are stated assumptions.

```
logic 6.0 mm2 (19.5 M GE)  SRAM 9.0 mm2  hard IP 3.6 mm2
die area ~ 20.9 mm2  (4.57 mm x 4.57 mm)
gross dies/wafer 3242, yield 97.9 %, good dies 3175
die cost at $9000/wafer: $2.83
power: logic 1.50 W + clock 0.81 W + leakage 0.23 W = 2.53 W (+ SRAM, IO, PHY)
```

Real output of diesize.py.

$$\begin{aligned} \text{Dies per wafer} &\sim \pi (d/2)^2 / A - \pi d / \sqrt{2 A} \\ \text{Yield (negative binomial)} \quad Y &= (1 + A D0 / \alpha)^{-\alpha} \\ \text{Die cost} &= \text{wafer cost} / (\text{dies per wafer} * Y) \end{aligned}$$

Classic early cost formulas ( $A$  = die area,  $D0$  = defect density,  $\alpha$  = clustering parameter). Chapter 21 treats yield in depth.

The estimate says: roughly 21 mm<sup>2</sup>, a few dollars of die cost, and about 2.5 W if all logic toggled at 1 GHz simultaneously. The power figure is a red flag for a fan-less edge device, and exactly the kind of early warning this exercise exists to produce: the architect now builds a per-use-case power table (which blocks are active, at what frequency and voltage), adds clock gating and power gating assumptions (Chapter 11), and negotiates the target with product management - long before RTL is written.

#### PITFALL: FORGETTING THE OVERHEADS

First estimates routinely miss: DFT logic (scan muxes, compression, MBIST: often ~5-15 % of logic area), clock tree and buffering, power-switch and isolation cells, decap and fill, memory BIST collars, spare cells, I/O and power pads, keep-out margins around macros, and the utilisation drop caused by routing congestion. Always carry an explicit margin (commonly 10-20 %) and revisit it at each milestone.

## 5.8 PPA targets and the frequency sanity check

Architecture concludes with numeric targets per block: frequency (at the sign-off corner, not typical), area, power per mode, and latency. A useful sanity check converts the clock period into gate delays: if the library's FO4 delay at the slow corner is, say, 25 ps, a 1 GHz clock leaves 1000 ps - clock uncertainty and flop overhead (clock-to-Q + setup, perhaps 100-150 ps) = roughly 34-36 FO4 of logic per stage. A 64-bit adder plus a mux and a comparator fits; a 32-bit multiplier does not, and must be pipelined. These decisions belong in the micro-architecture, not in a late timing fire-fight.

## 5.9 The micro-architecture specification

Each block then gets a micro-architecture specification (uArch spec) written by the design owner and reviewed by architecture, verification, DFT and physical design. A good template:

### Micro-architecture specification template

- ☐ Overview: purpose, features, block diagram, references to PRD IDs.
- ☐ Interfaces: every port with direction, width, protocol, timing (which clock), reset value; bus protocols (see the companion Communication Protocols guide).
- ☐ Clocks and resets: domains, frequencies, CDC/RDC crossings and their synchronisation schemes, clock gating.
- ☐ Datapath: pipeline diagram with stage-by-stage operations, widths, latency and throughput; numerical formats and rounding.
- ☐ Control: FSMs with state diagrams, arbitration, flow control, back-pressure, error handling.
- ☐ Memories: sizes, types, ports, ECC/parity, power-down modes, BIST.
- ☐ Register map: addresses, fields, access types, reset values (generated from SystemRDL/IP-XACT).
- ☐ Power: domains, gating, retention, estimated power per mode.
- ☐ PPA targets: frequency, estimated GE and memory area, floorplan hints (aspect ratio, pin placement), critical paths.
- ☐ DFT: scan, test modes, MBIST, observability; debug hooks.
- ☐ Verification plan hooks: features to verify, assertions, coverage points, known corner cases.
- ☐ Safety/security mechanisms if applicable (ECC, lockstep, parity, access control).
- ☐ Open issues and revision history.

## 5.10 Reviews, sign-offs and requirement traceability

Specifications are living documents under version control, and each major milestone has a formal review: architecture review, uArch reviews per block, RTL/verification-plan reviews, and later the tapeout review. Reviews include the downstream teams because they catch the problems the author cannot see: DFT sees untestable structures, PD sees a 2000-bit bus crossing the die, verification sees an unspecified corner case.

**Traceability** links each PRD requirement to architecture features, uArch sections, RTL blocks, verification items (tests, assertions, coverage points) and finally silicon validation tests. It is mandatory for safety-critical designs (ISO 26262 requires bidirectional traceability) and good practice everywhere: at tapeout, the question 'is every must-have requirement verified?' must have a data-backed answer.

```
PRD-017 'Decode 4 x 1080p60 H.264 streams'
|---> ARCH 3.2 video engine: 2 cores @ 600 MHz, 1.5 GB/s budget
|      |---> uARCH vdec 4.1 pipeline, 8 KB line buffer
|      |      |---> RTL vdec_top.sv, vdec_mc.sv
|      |      |---> DV vplan item VDEC-12: 4-stream test, cov 100 %
|      |      |---> PERF model run #231: 4 streams at 63 fps (margin 5 %)
|      |---> VALIDATION silicon test VAL-VDEC-04 (post-silicon, Chapter 22)
```

A requirement traced from PRD to silicon validation.

### INTERVIEW INSIGHT: 'HOW WOULD YOU ARCHITECT X?'

Architecture interview questions reward a structured answer: clarify requirements and use cases; estimate compute, memory and bandwidth with numbers; identify the bottleneck (roofline); propose a block diagram; state PPA estimates and the key trade-offs; and say how you would validate the architecture (model, prototype). Numbers - even rough ones - distinguish a senior answer.

## 5.11 Summary

- Requirements (functional, performance, power, area/cost, schedule, standards, quality, safety, security) are captured in the MRD and PRD and must be measurable and testable.
- Architecture exploration moves from spreadsheets and roofline models to performance models and SystemC/TLM virtual platforms. The roofline ridge point (peak / bandwidth) tells whether a workload is compute- or memory-bound.
- HW/SW partitioning trades flexibility for efficiency; the register map is the contract, ideally generated from one source.
- Size the memory hierarchy from working sets and budget sustained DRAM bandwidth across concurrent use cases.
- Plan clocks, resets, voltage and power domains and the IP list at architecture time.
- Estimate die size (GE x NAND2 area / utilisation + memories + hard IP + ring), dies per wafer, yield, die cost and power early, with explicit margins.
- Write a uArch spec per block, review it with all downstream teams, and maintain traceability from PRD to verification and silicon validation.

## 5.12 Exercises

- 1 Using roofline.py, how much on-chip reuse (factor on intensity) would make the depthwise layer compute-bound? What SRAM size does that imply for a 112 x 112 x 32 INT8 tensor?
- 2 Change diesize.py to a 28 nm-class node (NAND2 ~0.5  $\mu\text{m}^2$ , bitcell ~0.13  $\mu\text{m}^2$ , wafer cost lower) and compare die area and cost. Which node is cheaper per good die for this design?
- 3 Build a mode table (off/retention/on, voltage, frequency) for the edge-AI SoC with CPU, NPU, ISP and always-on sensor hub domains, and estimate standby power.
- 4 Write the interface section of a uArch spec for a simple DMA engine with an AXI master port and an APB register port.

- 5 Pick three PRD-level requirements of a product you know and show their traceability chain down to verification items.

## PART II

# Front-End Design

From RTL that the back end can implement, through verification strategy and sign-off, logic synthesis and static timing analysis, to design for test - the steps that turn an architecture into a verified, timed, testable netlist.

## 6. RTL Design for ASIC: What the Back End Needs from Your RTL

The companion RTL Design guide teaches how to write correct, synthesizable RTL. This chapter asks a different question: once the RTL simulates correctly, what does everybody **downstream** of the RTL designer need from it? In an ASIC project the RTL is read by at least eight other tools and five other teams - synthesis, static timing, DFT insertion, formal equivalence, power analysis, floorplanning, emulation and the software team's models. An RTL block that is functionally perfect but ignores their needs will cost weeks in implementation, and in the worst case forces a late RTL change that re-opens verification.

An FPGA engineer moving to ASIC usually discovers the differences the hard way: there is no block RAM to infer, flip-flops do not power up to the value in your **initial** statement, every reset flop costs area and a slice of a very large reset tree, and nothing can be fixed with a bitstream update after tapeout. We go through these differences one by one, measure some of them with a real synthesis run against the SkyWater SKY130 library, and finish with the quality gates and the hand-off package that front-end design delivers to synthesis.

### 6.1 Who consumes your RTL

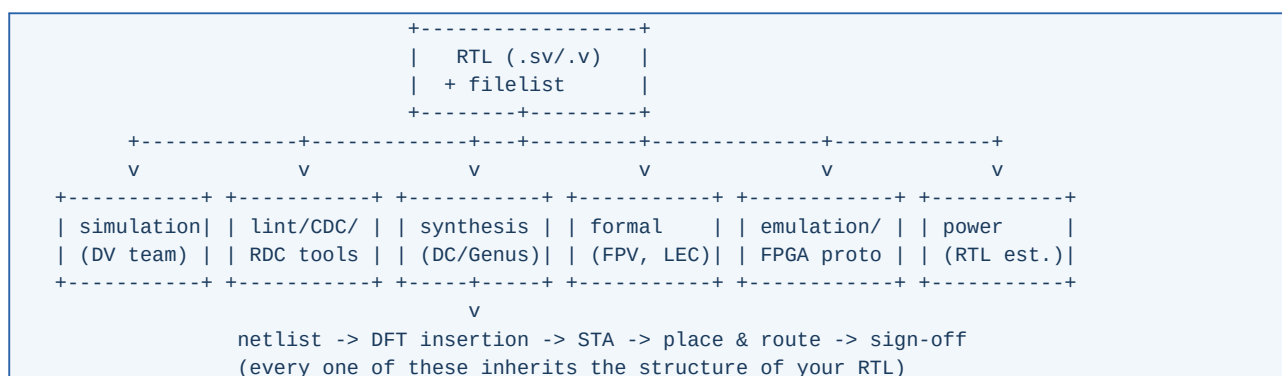


Figure 6.1 - The RTL is a shared source. Its structure (hierarchy, clocks, resets, memories, register boundaries) propagates all the way to layout.

| Consumer (owner)                                       | What it needs from the RTL                                                                                                                 | Symptom when the RTL ignores it                                              |
|--------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| <b>Synthesis (implementation / synthesis engineer)</b> | Synthesizable subset, no simulation-only constructs, clean clock and reset structure, memories as macros, reasonable logic depth per stage | Latches, huge mux trees, unmet timing that no tool option fixes              |
| <b>STA (timing sign-off)</b>                           | Few, well-defined clocks from known generator modules; registered block boundaries; documented false/multicycle paths                      | Hundreds of exceptions, unconstrained paths, clock-on-data warnings          |
| <b>DFT (DFT engineer)</b>                              | Controllable clocks and resets in test mode, no combinational loops, no clock-gating without test enable, memory BIST ports                | Low scan coverage, un-scannable flops, ATPG DRC violations                   |
| <b>Formal equivalence (LEC)</b>                        | Stable register names, no functionality hidden in <code>translate_off</code> , synthesis-compatible semantics                              | Non-equivalent points, aborted compare points on big datapaths               |
| <b>Physical design (PD team)</b>                       | Hierarchy that maps to a floorplan, registered interfaces between distant blocks, no giant high-fanout control nets                        | Congestion, long wires that cannot meet timing, reset/enable fanout problems |
| <b>Power (power / low-power architect)</b>             | Clock enables that can become clock gates, clean power-domain boundaries, operand isolation opportunities                                  | High dynamic power, isolation cells on thousands of signals                  |

Table 6.1 - Downstream consumers of RTL and what each one expects.

## 6.2 ASIC versus FPGA coding: the differences that bite

Most RTL constructs mean the same thing on both targets, but an FPGA is a pre-fabricated sea of look-up tables, flip-flops with configurable initial values, block RAMs, DSP slices and a dedicated global clock network. An ASIC has none of these ready-made: every flip-flop, every clock buffer and every memory is something you (or the flow) must choose, place and verify. Table 6.2 lists the differences that most often break a design that was 'proven on FPGA'.

| Topic                    | FPGA habit                                                                         | ASIC reality and rule                                                                                                                  |
|--------------------------|------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| <b>Initial values</b>    | <code>reg [7:0] cnt = 8'd0;</code> works: the bitstream loads the value            | Ignored by synthesis (or rejected). Silicon flops power up to a random value. <b>Every state that matters needs a reset.</b>           |
| <b>Reset style</b>       | Often no reset at all (GSR does it), or synchronous reset (cheaper in LUT fabrics) | Explicit reset strategy: which flops, sync or async, assertion/de-assertion synchronisation, reset tree built by CTS                   |
| <b>Memories</b>          | Arrays infer block RAM or distributed RAM automatically                            | Arrays become thousands of flops + muxes. Real memories are <b>compiled SRAM macros</b> instantiated through a wrapper                 |
| <b>Clock gating</b>      | Discouraged; use clock enables (CE pins)                                           | Encouraged, but only through <b>integrated clock-gating (ICG) cells</b> , inserted by the tool or instantiated via a wrapper           |
| <b>Clocks</b>            | PLL/MMCM primitives, global buffers (BUFG)                                         | Clock generation is analog IP (PLL) plus RTL dividers and muxes built from dedicated glitch-free cells; the tree is synthesised by CTS |
| <b>Tri-states</b>        | Internal tri-states converted to muxes by the tool                                 | No internal tri-state buses. Tri-states exist only in I/O pads                                                                         |
| <b>Arithmetic</b>        | DSP slices do multiply-accumulate for free                                         | Every multiplier is real area and power; datapath synthesis chooses architectures (chapter 8)                                          |
| <b>Vendor primitives</b> | Direct instantiation of LUT6, SRL16, IDDR...                                       | Only library cells or technology-independent wrappers; keep technology cells out of functional RTL                                     |
| <b>Bug fixing</b>        | Re-synthesise and reload in minutes                                                | A post-tapeout bug costs a metal or full respin: months and a large fraction of a mask set cost (chapter 19)                           |

Table 6.2 - FPGA habits versus ASIC rules.

### PITFALL - THE INITIAL VALUE THAT ONLY EXISTED ON FPGA

A classic escape: a state machine written with `reg [2:0] state = IDLE;` and no reset works for months on the FPGA prototype. On silicon the register powers up to a random code, often an unused one, and the FSM never leaves it. Gate-level simulation shows X, but only if somebody runs it with the right power-up sequence. Lint rules (for example 'initial statement in synthesizable code') catch this in seconds - enable them.

## 6.3 Reset strategy: which flops need a reset

Resets are the first place where ASIC RTL differs in cost. A resettable flop is a larger cell, and the reset net itself is one of the largest nets on the chip: it reaches a large fraction of all flops, must be buffered like a clock, and its de-assertion must be timed (recovery and removal checks) against every clock domain it reaches. The standard strategy in industry is:

- **Reset control state, not data.** FSM state, valid bits, counters, pointers, configuration registers and anything that is observed before it is written must be reset. Pure datapath registers whose content is qualified by a valid bit do not need one.
- **Asynchronous assertion, synchronous de-assertion.** The reset reaches the flops immediately (works without a clock), but is released through a 2-flop reset synchroniser per clock domain so that de-assertion meets recovery/removal timing. The companion RTL Design guide shows the synchroniser circuit.
- **One reset synchroniser per clock domain and per reset domain**, owned by a dedicated reset-controller module - never sprinkled in functional blocks.

- **Reset sequencing** is a system decision: which domains come out of reset first (PLL lock, memory repair load, fuse read), documented in the architecture spec (chapter 5).

How much does 'reset everything' cost? The two variants below implement the same 4-stage, 32-bit pipeline. Variant A resets all 100 flops; variant B resets only the four valid bits. Both were synthesised with Yosys against the real SKY130 high-density library (typical corner, 25 C, 1.8 V).

```
// Variant A: every flop has an asynchronous reset (FPGA-style 'reset everything')
always @(posedge clk or negedge rst_n)
  if (!rst_n) begin
    {v1, v2, v3, out_vld} <= 4'b0;
    d1 <= 32'd0; d2 <= 32'd0; d3 <= 32'd0; out_dat <= 32'd0;
  end else begin
    v1 <= in_vld; v2 <= v1; v3 <= v2; out_vld <= v3;
    d1 <= in_dat; d2 <= d1 + 32'd1; d3 <= d2 ^ {d2[15:0], d2[31:16]};
    out_dat <= d3;
  end

// Variant B: only the control (valid) bits are reset; the datapath is not
always @(posedge clk or negedge rst_n)
  if (!rst_n) {v1, v2, v3, out_vld} <= 4'b0;
  else begin v1 <= in_vld; v2 <= v1; v3 <= v2; out_vld <= v3; end
always @(posedge clk) begin // no reset: data qualified by valid
  d1 <= in_dat; d2 <= d1 + 32'd1; d3 <= d2 ^ {d2[15:0], d2[31:16]};
  out_dat <= d3;
end
```

Two reset strategies for the same pipeline (pipe\_a.v / pipe\_b.v).

```
read_verilog pipe_a.v
synth -top pipe_a -flatten
dfflibmap -liberty hd_tt_du.lib      # map flops to real SKY130 flip-flops
abc -liberty hd_tt_du.lib           # map logic to real SKY130 gates
opt_clean
stat -liberty hd_tt_du.lib          # area from the Liberty 'area' attributes
```

Yosys script. hd\_tt\_du.lib is the SKY130 hd tt Liberty with clock, delay, probe and lpflop cells removed - a crude dont\_use list.

```
=== pipe_b ===
Number of cells:                190
sky130_fd_sc_hd__a21oi_1        7
sky130_fd_sc_hd__a31oi_1        1
sky130_fd_sc_hd__and3_1         3
sky130_fd_sc_hd__and4_1         1
sky130_fd_sc_hd__dfrrtp_1       4
sky130_fd_sc_hd__dfxtp_1       96
sky130_fd_sc_hd__inv_1          13
sky130_fd_sc_hd__nand2_1         3
sky130_fd_sc_hd__nand3_1         2
sky130_fd_sc_hd__nand4_1         5
sky130_fd_sc_hd__nor2_1         12
sky130_fd_sc_hd__nor4_1          4
sky130_fd_sc_hd__xnor2_1        14
sky130_fd_sc_hd__xor2_1         25
Chip area for module '\pipe_b': 2603.747200
```

Real Yosys 0.33 stat -liberty output for variant B (areas in  $\mu\text{m}^2$ ).

```
=== pipe_a ===
Number of cells:                190
sky130_fd_sc_hd__dfrrtp_1       100
(all 12 combinational cell types and counts identical to pipe_b)
Chip area for module '\pipe_a': 3084.208000
```

Real Yosys output for variant A (excerpt: only the lines that differ).



The combinational logic is identical; the only change is that 96 flops moved from `dfrrtp_1` (D flop with active-low async reset, 25.02  $\mu\text{m}^2$  in the Liberty file) to `dfxtp_1` (plain D flop, 20.02  $\mu\text{m}^2$ ). That is 5.0  $\mu\text{m}^2$  per flop, 480  $\mu\text{m}^2$  in total, **15.6% of the block area**. In a real chip the saving is larger than the cell area suggests, because every removed reset pin is one less sink on the reset tree that CTS must buffer and balance, and one less recovery/removal timing check.

#### INTERVIEW INSIGHT - SYNC OR ASYNC RESET?

Asynchronous reset works without a running clock (needed at power-up and for clock-gated or PLL-bypassed domains), and it keeps the reset out of the D-input logic cone, so it does not add delay to the data path. Its costs are a larger flop, a reset tree that must be timed for recovery/removal, and glitch sensitivity. Synchronous reset uses a plain flop plus an AND gate in the data path and needs the clock running. The safe default in most ASIC teams is async assert / sync de-assert, applied only to the flops that need it - but the decision belongs to the project's RTL guidelines, and **mixing styles inside a block is the real mistake**.

## 6.4 Memories: macros and wrappers instead of inferred arrays

On an FPGA, `reg [15:0] mem [0:63]` becomes a block RAM. In an ASIC flow, a synthesis tool has no memory primitives: the array becomes discrete flip-flops plus write-enable and read muxes. This is acceptable for small register files (tens to a few hundred bits); beyond that it is a disaster for area, power and routability. The same 64x16 array synthesised against SKY130:

```
=== rf64x16 ===
Number of cells:          2596
sky130_fd_sc_hd__dfxtp_1  1040
sky130_fd_sc_hd__mux2_1   1024
sky130_fd_sc_hd__mux2i_1  144
sky130_fd_sc_hd__mux4_2   288
... (93 small decode gates omitted)
Chip area for module '\rf64x16': 40767.849600
```

Real Yosys output (excerpt): a 1 Kbit memory written as an array costs about 40,800  $\mu\text{m}^2$  in SKY130 - roughly 40  $\mu\text{m}^2$  per bit.

Look at the cells. 1040 flops (1024 storage bits plus the 16-bit read register), **1024 mux2 cells** that implement 'hold the old value unless written' (because this mapping flow could not use an enable flop), and 432 mux2/mux4 cells forming the 64:1 read multiplexer. A compiled SRAM macro stores a bit in a 6-transistor bitcell that is typically several times to an order of magnitude smaller than a flop plus its muxes, and its peripheral circuits (sense amplifiers, decoders) amortise over thousands of bits. The rule most teams use: **above a few hundred bits, use a memory compiler macro**; below that, a latch- or flop-based register file generated by a register-file compiler or by synthesis.

### The memory wrapper

Functional RTL should never instantiate a foundry memory macro directly. Instead it instantiates a **technology-independent wrapper** with a clean interface. Inside the wrapper, a `generate/ifdef` selects a behavioural model for simulation and FPGA, or the real macro(s) for the ASIC netlist. The wrapper is where all the ASIC-only plumbing lives:

```
functional RTL          mem_wrap_1024x32 (owned by the memory/DFT team)
+-----+ +-----+
| ce, we, addr |-->| +-----+ +-----+ +-----+ |
| wdata, bwe  | | BIST mux|-->| ASIC: SRAM  |-->| ECC / parity  |-->| rdata
| rdata <-----| | (test / | | macro(s), e.g. | | check (option) | |
+-----+ | | func) | | 2 x 512x32  | +-----+ |
          | +-----+ +-----+ +-----+ |
MBIST controller +-----+ | | +-- repair bus (from fuses) |
(chapter 10)    | | +----- power: retention/sleep |
                | | +----- margin / timing adjust |
                | | FPGA/sim: behavioural array model in the same wrapper |
                +-----+ +-----+ +-----+ +-----+
```

Figure 6.2 - A memory wrapper hides the macro and carries the BIST, repair, power-management and ECC hooks.

```

module mem_wrap_1024x32 (input  logic clk, ce, we,
                        input  logic [9:0]  addr,
                        input  logic [31:0] wdata, bwe,      // bit-write enable
                        output logic [31:0] rdata,
                        mbist_if.mem      bist,              // test port
                        input  logic      ls, ds);           // light/deep sleep
`ifdef ASIC
    // two 512x32 compiler instances + BIST mux, repair and power pins
    sram_sp_512x32 u_bank0 (...); sram_sp_512x32 u_bank1 (...);
`else
    logic [31:0] mem [0:1023];          // behavioural model
    always_ff @(posedge clk) if (ce) begin
        if (we) for (int i = 0; i < 32; i++) if (bwe[i]) mem[addr][i] <= wdata[i];
        rdata <= mem[addr];              // read-first, 1-cycle latency
    end
`endif
endmodule

```

Skeleton of a memory wrapper. Port names of real compiled macros differ per memory compiler; the wrapper isolates the design from them.

- **Match the macro's behaviour exactly** in the model: read latency, read-during-write behaviour, bit-write granularity and what happens to `rdata` when `ce` is low. Mismatches here are a favourite source of silicon bugs because the model is what DV verified.
- **BIST and repair ports** are planned from day one (chapter 10). Adding them after floorplanning moves macros and wires.
- **Power pins** (light sleep, deep sleep, shut-down, retention) are controlled by the power controller defined in the UPF (chapter 11).
- **Macro count and aspect ratio** are a floorplan decision: two 512-deep instances may place better than one 1024-deep.

## 6.5 Clock gating in RTL

Clock gating is the single most effective dynamic-power technique in digital ASICs (chapter 11): a register bank that loads only occasionally should not see a toggling clock. In ASIC RTL you almost never build the gate yourself. Instead you write a clean **enable** and let synthesis replace the feedback multiplexer with an **integrated clock-gating (ICG) cell** - a latch plus AND gate characterised as one cell with a setup/hold check on the enable. SKY130 provides `dlclkp` (17.5  $\mu\text{m}^2$  for the `_1` size) and the test-enabled `sdclkp` (18.8  $\mu\text{m}^2$ ).

RTL you write:

```

always_ff @(posedge clk)
    if (en) q <= d;

```

What synthesis builds (ICG cell, e.g. `sdclkp`):

```

+-----+
| en --+ | OR  +-----+ |
|      | | +----->| D   Q |---+ +-----+ |
|      | | latch | +-->|    | |
| te --+ | (open |    | AND |---> gclk
|      | | G low) | +-->|    | |
| clk ---+>|    | +-----+ |
|      | +-----+ |
+-----+

```

Without gating: a mux in front of every flop, and the clock toggles every cycle.

With gating: one ICG drives N flops, the muxes disappear, `gclk` toggles only when `en=1`.

Figure 6.3 - A latch-based ICG. The latch is transparent while the clock is low, so the enable cannot change while `gclk` is high: no glitches. `te` (test enable) forces the clock on during scan shift.

- Write enables at the **group level** (a whole 32-bit register loads together) - synthesis gates only banks whose width exceeds a minimum (commonly 3-8 flops).
- Never build a clock gate from an AND or OR gate in RTL. A combinational gate on the clock glitches, is invisible to STA's clock-gating checks unless constrained, and breaks scan.

- If you must gate explicitly (for example to gate an entire sub-system), instantiate a **clock-gate wrapper** module (`cg_wrap`) that maps to the library ICG and has a `test_en` port tied to the DFT scan-enable/test-mode signal.
- Clock muxes and dividers likewise use dedicated glitch-free wrapper cells owned by the clocking team.

## 6.6 No internal tri-states, and synchronous design

Internal tri-state buses were common in the 1990s. Modern flows forbid them: they cannot be timed cleanly, create contention during scan shift (two drivers enabled by random scan data), float to mid-rail and burn crowbar current when no driver is enabled, and complicate equivalence checking. Use multiplexers or AND-OR buses. Tri-states survive only in I/O pad cells.

The ASIC flow also assumes a **fully synchronous** design style: registers clocked by a small number of well-defined clocks, with all asynchronous interactions confined to reviewed synchroniser cells. Things that are illegal or strongly discouraged:

- Combinational feedback loops (STA cannot time them, ATPG cannot handle them).
- Using data signals as clocks (ripple counters, 'clock = counter[3]') outside the clock-generation module.
- Latches, except intentional ones (ICGs, latch-based register files, time-borrowing designs) that are reviewed and constrained.
- Delay chains built from buffers to 'fix' a race: delay varies by 2x or more over PVT corners (chapter 9).
- Asynchronous set **and** reset on the same flop, and resets generated from combinational logic without a synchroniser.

## 6.7 Hierarchy for physical design

The RTL hierarchy becomes the starting point of the floorplan. Synthesis may flatten small modules, but large blocks usually keep their hierarchy (for runtime, for hierarchical implementation, and for ECO traceability). A few structural rules make physical design dramatically easier:

```
chip_top (pads, I/O ring, only instantiations - no logic)
|-- chip_core
|   |-- clk_rst_gen      PLL wrapper, dividers, clock muxes, reset synchronisers
|   |-- pmu / power_ctl  power switches, isolation/retention control (UPF)
|   |-- dft_top          TAP, test-mode decode, OCCs, compression, MBIST ctrl
|   |-- cpu_ss           <- hard partition: implemented separately, reused
|   |-- npu_ss           <- hard partition, its own power domain
|   |-- noc / interconnect registered at every partition boundary
|   |-- periph_ss        UART/SPI/I2C/timers (soft, flattened)
|   +-- mem_ss           SRAM wrappers grouped where the floorplan wants them
+-- io_ring              pad cells, ESD, level shifters (chapter 12)
```

Figure 6.4 - A physically aware chip hierarchy. The top level only connects; each subsystem is a candidate floorplan region or hard partition.

- **No glue logic at the top level.** Top-level logic becomes orphan cells that the floorplan must place somewhere and time against every partition.
- **Register the interfaces** of blocks that will be far apart. A wire across a 10 mm die can take a large fraction of a nanosecond even with repeaters (chapter 17); flops at both ends keep the path budget local.
- **Keep clocks, resets and test control in dedicated modules** so their special handling (dont\_touch, CTS exceptions, DFT overrides) is applied to one place.
- **Group by power domain and by clock domain:** one module should not straddle two power domains.
- **Stable, meaningful instance and register names:** timing constraints, UPF, LEC mapping and post-silicon debug all refer to them by name.
- **Parameterise, but elaborate once:** the implementation team needs to know exactly which parameter set is taped out.

## 6.8 Clocks, resets, CDC and RDC in one page

Clock-domain crossing (CDC) and reset-domain crossing (RDC) are covered in depth in the companion RTL Design guide; here is what matters for the ASIC flow and sign-off:

| Item                               | RTL rule                                                                                                         | Checked by                                               |
|------------------------------------|------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| <b>Single-bit control crossing</b> | 2- or 3-flop synchroniser cell from a library wrapper (named, so STA can find it)                                | Structural CDC tool (Spyglass CDC, Questa CDC, Meridian) |
| <b>Multi-bit data crossing</b>     | Gray-coded pointers (async FIFO) or req/ack handshake with data held stable                                      | CDC tool + formal                                        |
| <b>Reconvergence</b>               | Never synchronise related bits separately and recombine them                                                     | CDC tool (reconvergence rule)                            |
| <b>Reset de-assertion</b>          | Per-domain reset synchroniser; async assert, sync de-assert                                                      | RDC tool, STA recovery/removal                           |
| <b>Reset-domain crossing</b>       | Flop in domain reset A feeding a flop in reset domain B must be isolated or B must be reset too                  | RDC tool                                                 |
| <b>Timing of crossings</b>         | Constrained with <code>set_max_delay -datapath_only</code> or clock groups, never left unconstrained by accident | STA constraint review                                    |

Table 6.3 - CDC/RDC rules and the tools that sign them off.

## 6.9 Pipelining for timing: logic depth per stage

Synthesis can size gates, restructure logic and even retime registers, but it cannot rescue a design whose pipeline stage holds three times more logic than the clock period allows. The front-end designer must budget **logic depth per stage** early. A useful technology-independent unit is the **FO4 delay**: the delay of an inverter driving four copies of itself.

$$\text{cycle time} \sim (\text{logic depth in FO4}) \times t_{\text{FO4}} + \text{clk} \rightarrow \text{Q} + \text{setup} + \text{clock skew/uncertainty} \quad (\text{roughly } 3\text{-}6 \text{ FO4})$$

**typical budgets: very high-speed CPU pipelines ~ 12-20 FO4 per stage;**  
**ASIC SoC logic ~ 25-50 FO4; low-power MCUs ~ 60+ FO4**

Rule-of-thumb cycle budget (the numbers are indicative, they vary by design style and team).

In SKY130 at the typical corner, a small gate with a realistic load is on the order of 50-100 ps (the real NLDL lookups in chapter 9 give 40-100 ps per stage), so a 100 MHz clock (10 ns) comfortably fits 50+ gate levels, while at 500 MHz the whole budget, including clock-to-Q and setup, is 2 ns. Guidelines:

- **Register outputs of every block** and, for high-speed blocks, inputs as well.
- Split wide comparators, priority encoders and multi-level muxes into two stages when they sit between other heavy logic.
- Keep **retiming-friendly** structure: a pipeline register placed after a large arithmetic block with no reset and no observation in between can be moved by synthesis retiming (chapter 8); a register with reset, enable and debug visibility usually cannot.
- Watch **high-fanout control** (stall, flush, global enables): an enable driving 2,000 flops needs a buffer tree that alone costs several gate delays. Replicate or pipeline it.
- Put memories on stage boundaries: SRAM access time is often the longest single delay in a pipeline; drive macro inputs directly from flops and register the outputs.

## 6.10 DFT-friendly RTL

Scan insertion (chapter 10) turns every flop into a scan flop and stitches them into chains. It works automatically **if** the RTL obeys a short list of rules; otherwise the ATPG tool reports DRC violations and the affected flops are excluded, lowering test coverage.

| Rule                                             | Why                                                                            | Typical fix in RTL                                                  |
|--------------------------------------------------|--------------------------------------------------------------------------------|---------------------------------------------------------------------|
| <b>All clocks controllable from a test clock</b> | ATPG must pulse the clock at will during shift and capture                     | Clock mux (test_mode) in clk_rst_gen; OCC for at-speed              |
| <b>Async set/reset controllable</b>              | A reset driven by functional logic can fire during shift and corrupt the chain | Gate internally generated resets with test_mode / scan_en           |
| <b>ICGs have a test enable</b>                   | Otherwise gated flops cannot be shifted                                        | Use <code>sdclkp</code> -style ICG, TE tied to scan_en or test_mode |
| <b>No combinational loops</b>                    | ATPG cannot evaluate loops                                                     | Break with a flop, or with a test-mode mux                          |
| <b>Avoid latches in data paths</b>               | Latches need special handling (transparent in test)                            | Replace with flops, or make them transparent in test mode           |
| <b>Memories bypassable</b>                       | Macro outputs are X-sources for logic test                                     | Memory wrapper with bypass / 'observe-through' mode                 |
| <b>Black boxes and analog outputs</b>            | Unknown values (X) propagate and destroy compression                           | Tie-off or register analog outputs in test mode (X-bounding)        |

Table 6.4 - RTL rules that keep scan and ATPG clean.

## 6.11 Low-power-friendly RTL

Most power is decided by the architecture and the RTL, long before any implementation tool sees the design (chapter 11 goes into UPF, power gating and DVFS). The RTL designer controls:

- **Clock enables everywhere they are natural** - load registers only when their value changes, so synthesis can insert ICGs. Clock-tree and flop power are often a third or more of dynamic power in SoC logic.
- **Operand isolation**: hold the inputs of a large multiplier or adder stable when its result is not used, so the datapath does not toggle for nothing.
- **Memory access discipline**: do not assert chip-enable every cycle 'because it is simpler'; an SRAM read typically costs far more energy than the logic around it.
- **Avoid glitchy structures**: long chains of XORs and ripple arithmetic generate spurious transitions; balanced trees and pipelining reduce glitch power.
- **Power-domain-clean interfaces**: signals leaving a domain that can be switched off must be isolatable (clamp value defined in the spec), and retention requirements known per register.

## 6.12 RTL quality gates

Before RTL is handed to synthesis for a milestone, it must pass a set of automated quality gates. Mature teams run them in continuous integration on every commit, and track violations and waivers per block.

| Gate        | What it checks                                                                                         | Typical tools                                              |
|-------------|--------------------------------------------------------------------------------------------------------|------------------------------------------------------------|
| <b>Lint</b> | Synthesizability, width mismatches, latches, multiple drivers, unused signals, naming and coding rules | Spyglass Lint, Ascent Lint, Verilator --lint-only, Verible |
| <b>CDC</b>  | Unsynchronised crossings, reconvergence, glitchy synchroniser inputs, data stability of handshakes     | Spyglass CDC, Questa CDC, Meridian CDC                     |
| <b>RDC</b>  | Reset-domain crossings, reset synchroniser structure, reset ordering                                   | Spyglass RDC, Questa RDC, Meridian RDC                     |

| Gate                        | What it checks                                                                                                                                                   | Typical tools                             |
|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------|
| <b>Synthesis sanity</b>     | Elaborates in the real synthesis tool, no unexpected latches, no black boxes, area/timing roughly as budgeted, no constant-propagated logic you expected to keep | DC/Genus quick run, Yosys                 |
| <b>X-propagation</b>        | Reset values complete; X from uninitialised state does not reach outputs under X-pessimistic/optimistic semantics                                                | Simulator X-prop modes, formal X-prop app |
| <b>DFT rule check (RTL)</b> | Clock/reset controllability, loops, ICG test enables                                                                                                             | Tessent / DFT Compiler RTL DRC            |
| <b>Power lint</b>           | Enable efficiency, UPF consistency                                                                                                                               | PowerArtist, Joules, SpyGlass Power       |

Table 6.5 - RTL quality gates.

#### PITFALL - THE WAIVER FILE NOBODY READS

Lint and CDC tools produce thousands of messages on a large design. Teams respond with waiver files; after a few months the waivers hide real bugs. Good practice: every waiver has an owner, a reason and a pattern that is as narrow as possible (instance + signal, never a whole rule), and waivers are reviewed at each milestone like code.

## 6.13 The hand-off package to synthesis

The RTL freeze for a milestone is not 'the Git tag'. It is a package with a defined content that the implementation team can run without asking questions.

| Deliverable                    | Content                                                                                     | Owner                       |
|--------------------------------|---------------------------------------------------------------------------------------------|-----------------------------|
| <b>RTL + filelist</b>          | Tagged source, one filelist per top, all defines/parameters for the taped-out configuration | RTL lead                    |
| <b>Constraints (SDC)</b>       | Clocks, generated clocks, I/O delays, clock groups, exceptions with justification           | RTL designer + timing owner |
| <b>Power intent (UPF)</b>      | Domains, supplies, isolation, retention, level shifters (chapter 11)                        | Low-power architect         |
| <b>Memory and IP list</b>      | Macro configurations, compiler versions, hard IP views (LEF, Liberty, GDS, models)          | IP/memory owner             |
| <b>DFT specification</b>       | Test modes, scan chain count, compression ratio, MBIST grouping, OCC locations              | DFT lead                    |
| <b>Quality reports</b>         | Lint/CDC/RDC clean or waived, synthesis sanity area/timing, coverage status                 | RTL + DV leads              |
| <b>Known issues / ECO list</b> | Open bugs, planned late changes, blocks expected to change                                  | Project lead                |

Table 6.6 - The front-end hand-off package.

### RTL ready-for-synthesis checklist

- ☐ No **initial** blocks, delays (#), **force**, or simulation-only constructs outside **translate\_off** regions that do not change function.
- ☐ Every flop that holds control state has a documented reset; datapath flops without reset are qualified by valid.
- ☐ All memories above the agreed size go through wrappers with BIST, repair and power ports.
- ☐ No clock or reset generated outside **clk\_rst\_gen**; all clock gating via ICG wrappers or tool-inferred enables.
- ☐ No internal tri-states, combinational loops or unreviewed latches.
- ☐ Lint, CDC, RDC and DFT-RTL checks clean or waived with owner and reason.
- ☐ Quick synthesis meets area and timing budgets within the agreed margin; SDC and UPF delivered and reviewed.

## 6.14 Summary

- RTL is read by synthesis, STA, DFT, LEC, PD, power and emulation; its structure propagates all the way to layout.
- ASIC silicon has no initial values, no block RAM and no global clock buffers: resets, memory macros and clock gating are explicit design decisions.
- Reset only what needs it: in our SKY130 run, removing reset from 96 datapath flops saved 15.6% of block area.
- A 1 Kbit array synthesised to flops cost about  $40 \text{ um}^2$  per bit; memories go through wrappers that carry BIST, repair, power and ECC hooks.
- Code clock enables and let synthesis insert ICGs; never gate clocks with plain logic.
- Plan hierarchy for the floorplan: no top-level glue, registered interfaces, dedicated clock/reset/test/power modules.
- Budget logic depth per stage, follow DFT and low-power rules, and pass lint/CDC/RDC/X-prop gates before the hand-off package goes to synthesis.

## 6.15 Exercises

- 1 A block has 12,000 flops, of which 1,800 hold control state. Using the SKY130 areas from this chapter ( $\text{dfrtp\_1} = 25.02 \text{ um}^2$ ,  $\text{dfxtp\_1} = 20.02 \text{ um}^2$ ), estimate the area saved by resetting only the control flops. What other costs go away?
- 2 Rewrite `reg [3:0] state = 4'd0;` for an ASIC and explain what happens on silicon and in gate-level simulation if you do not.
- 3 Sketch the memory wrapper ports for a 4096x64 single-port SRAM with byte-write enables, MBIST, redundancy repair and a light-sleep mode. Which team owns each port?
- 4 Explain why a clock gate built from an AND gate glitches, and why a latch-based ICG does not. Draw the waveforms.
- 5 Your block must run at 800 MHz in a node where  $t_{\text{FO4}}$  is about 15 ps. How many FO4 of logic fit in a stage after 5 FO4 of overhead? Is a 32-bit ripple-carry adder feasible in one stage?
- 6 List five RTL constructs that make scan insertion fail or lower coverage, and the fix for each.



## 7. Functional Verification Strategy and Sign-off

An ASIC cannot be patched. Every functional bug that escapes to silicon costs either a software workaround (lost features or performance), a metal-layer ECO respin, or a full respin - months of schedule and, at advanced nodes, mask sets costing millions of dollars (chapter 19). This is why **functional verification is typically the largest single effort on an ASIC project**: industry surveys consistently report that verification engineers outnumber design engineers, and that verification consumes on the order of half to two-thirds of the front-end effort.

The companion Verilog & SystemVerilog guide teaches the language machinery: classes, constraints, covergroups, assertions and UVM. This chapter is about **strategy**: how a project decides what to verify, at which level, with which technique (simulation, formal, emulation, prototyping), how it measures progress, and what evidence it must show before the verification lead signs the tapeout checklist.

### 7.1 Why verification dominates, and how it is organised

| Metric                            | Typical range (indicative)                                   | Comment                                                                        |
|-----------------------------------|--------------------------------------------------------------|--------------------------------------------------------------------------------|
| <b>DV : design engineer ratio</b> | about 1:1 to 3:1                                             | Higher for CPUs, GPUs, safety-critical and protocol-heavy IP                   |
| <b>Share of front-end effort</b>  | roughly 50-70%                                               | Includes testbench development, debug, coverage closure                        |
| <b>Bugs found per phase</b>       | most in block level; the costly ones at SoC level            | Integration bugs: connectivity, clocks, resets, address maps, power sequencing |
| <b>First-silicon success</b>      | a minority of projects are fully functional on first silicon | Logic/functional bugs are the leading cause of respins in published surveys    |

Table 7.1 - Verification in numbers. Figures are ranges commonly quoted in industry surveys, not precise statistics.

Ownership is clear in a mature organisation: the **DV lead** owns the verification plan and sign-off; **block DV engineers** own testbenches per IP; an **SoC DV team** owns integration tests, often together with **firmware/validation engineers** who run real software on emulation; a **formal team** (or formal-trained DV engineers) owns property proofs and formal apps; the **methodology/infrastructure team** owns the regression system, coverage database and verification IP (VIP) licences.

### 7.2 The verification plan

Everything starts from the **verification plan** (vPlan, testplan). It is written in parallel with the micro-architecture specification and reviewed by designers, architects and DV together. It turns each feature of the spec into measurable verification goals.

| Specification<br>(features) | Verification plan<br>(what + how + done-criteria) | Evidence<br>(metrics) |
|-----------------------------|---------------------------------------------------|-----------------------|
| +-----+                     | +-----+                                           | +-----+               |
| F1 DMA burst                | F1.1 all burst lengths 1..256                     | -->  covergroup bins  |
| transfers                   | F1.2 4KB boundary crossing                        | -->  cover property   |
|                             | F1.3 error response mid-burst                     | -->  directed test ID |
| F2 low-power                | F2.1 entry/exit with traffic                      | -->  emulation run    |
| modes                       | F2.2 isolation clamps correct                     | -->  PA-sim + formal  |
| F3 register                 | F3.1 reset values, access policies                | -->  UVM RAL test     |
| map                         | F3.2 connectivity to SoC map                      | -->  formal conn. app |
| +-----+                     | +-----+                                           | +-----+               |
|                             |                                                   |                       |
|                             | +----> tracked in the vPlan tool <-----+          |                       |
|                             | (% of items closed = progress)                    |                       |

Figure 7.1 - From spec features to plan items to coverage evidence. Tools such as Verisium Manager, VC Execution Manager or in-house dashboards link each plan item to coverage.



- **Feature list** extracted from the spec, including error and corner cases, configuration parameters, clock/reset/power scenarios and performance requirements.
- **Technique per feature**: constrained-random simulation, directed test, formal proof, emulation with software, or 'covered by reuse' of pre-verified IP.
- **Level per feature**: block, subsystem or SoC - verify each thing at the lowest level where it is observable.
- **Done criteria**: coverage targets, number of clean regression cycles, bug-rate thresholds, reviewed waivers.
- **Resources and schedule**: VIP to buy, testbench components to build, emulation capacity, who owns what.

## 7.3 Testbench tiers

A single monolithic SoC testbench would be too slow to reach corner cases in each block and too late to start. Verification is therefore layered, and components (agents, scoreboards, reference models, VIP) are built for reuse from one level to the next.

| Tier              | Scope and typical approach                                                                                                                                | Speed / what it finds                                                                           |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| <b>Unit</b>       | One module or small function, often by the designer: directed tests, formal property checks, quick assertions                                             | Very fast; syntax-level and micro-architectural bugs                                            |
| <b>Block (IP)</b> | Complete IP with UVM environment: agents per interface, reference model, scoreboard, functional coverage, constrained-random stimulus                     | Fast (thousands of tests per night); most functional bugs, corner cases                         |
| <b>Subsystem</b>  | Several IPs plus interconnect (e.g. CPU cluster + caches + coherent fabric); reuses block agents in passive mode                                          | Medium; interaction and performance bugs                                                        |
| <b>SoC / chip</b> | Full chip, often with real CPU running C tests or an embedded test program; checks connectivity, boot, clocks, resets, power modes, pin muxing, DFT modes | Slow in simulation (Hz to kHz of chip clock); integration bugs; moves to emulation for software |
| <b>System</b>     | Chip + firmware + OS on emulator or FPGA prototype, or silicon                                                                                            | MHz; software-visible and performance bugs                                                      |

Table 7.2 - Testbench tiers.

## 7.4 UVM in one page

The Universal Verification Methodology (UVM, standardised as IEEE 1800.2) is the SystemVerilog class library used by nearly every ASIC DV team for block and subsystem environments. The companion SystemVerilog guide builds a complete UVM environment; the architecture to remember is:

```

uvm_test (selects config + sequences)
+-- env
+-- agent (active) -----+ +-----+
|   sequencer -> driver --> [ interface ] --> |       DUT       |
|   monitor <--- [ interface ] <--- |       |
+-----+-----+ +-----+
+-- agent (passive): monitor only |       ^
+-- scoreboard <-----+-----+ |       |
|   (compares DUT output with reference model predictions)
+-- coverage collector (covergroups fed by monitors)
+-- register model (RAL) - front-door / back-door register access

```

Figure 7.2 - The UVM environment structure. Agents are reused from block to SoC level; sequences are layered to build scenarios.

### WHEN UVM IS OVERKILL

For small blocks, a SystemVerilog testbench with a task-based driver, a reference model function and a few covergroups is faster to write and easier to debug. For many control blocks, formal property verification replaces most simulation. Use UVM where reuse and scale pay off: standard interfaces, complex protocols, subsystem integration.

## 7.5 Constrained-random stimulus and functional coverage: a real run

Constrained-random verification generates legal but random stimulus, checks every result against a reference model, and measures **functional coverage** - have we exercised the scenarios listed in the plan? The small example below captures the whole loop. The DUT is a signed 8-bit saturating add/subtract unit. A buggy variant computes  $a - b$  as  $a + (-b)$  in 8 bits - wrong exactly when  $b = -128$ , because  $-(-128)$  overflows back to  $-128$ . This is the kind of bug that random testing hits only with probability  $1/512$  per transaction.

```
// satalu.v - signed 8-bit saturating add/subtract (DUT)
module satalu (input clk, input op,           // op: 0 = add, 1 = sub
              input signed [7:0] a, b, output reg signed [7:0] y, output reg sat);
`ifdef BUG
    wire signed [7:0] bn = op ? -b : b;      // BUG: -(-128) overflows to -128
    wire signed [8:0] s = a + bn;
`else
    wire signed [8:0] s = op ? a - b : a + b; // 9-bit exact result
`endif
    always @(posedge clk) begin
        sat <= (s > 127) || (s < -128);
        y <= (s > 127) ? 8'sd127 : (s < -128) ? -8'sd128 : s[7:0];
    end
endmodule
```

The DUT, with the bug selectable by a define.

```
// tb.sv - self-checking constrained-random testbench with functional coverage
module tb;
    reg clk = 0; always #5 clk = ~clk;
    reg op; reg signed [7:0] a, b; wire signed [7:0] y; wire sat;
    satalu dut (.clk(clk), .op(op), .a(a), .b(b), .y(y), .sat(sat));
    integer seed, seed0, n, i, bias, errs = 0, exp; integer cov [0:13]; // 14 coverage bins
    function signed [7:0] pick(input integer r); // corner-biased operand
        reg signed [7:0] c [0:4];
        begin c[0] = -128; c[1] = -1; c[2] = 0; c[3] = 1; c[4] = 127;
        pick = (bias && r % 4 == 0) ? c[($unsigned($random(seed))) % 5] : $random(seed); end
    endfunction
    initial begin
        if (!$value$plusargs("seed=%d", seed)) seed = 1;
        if (!$value$plusargs("n=%d", n)) n = 200;
        seed0 = seed;
        if (!$value$plusargs("bias=%d", bias)) bias = 1;
        for (i = 0; i < 14; i = i + 1) cov[i] = 0;
        for (i = 0; i < n; i = i + 1) begin
            op = $random(seed); a = pick($random(seed)); b = pick($random(seed));
            @(posedge clk); #1;
            exp = op ? a - b : a + b; // reference model
            exp = (exp > 127) ? 127 : (exp < -128) ? -128 : exp;
            if (y !== exp[7:0] || sat !== (op ? (a-b > 127 || a-b < -128)
                                           : (a+b > 127 || a+b < -128))) begin
                errs = errs + 1;
                if (errs <= 2) $display("ERROR seed=%0d op=%0d a=%0d b=%0d y=%0d exp=%0d",
                                       seed0, op, a, b, y, exp);
            end
            cov[op*3 + (exp == 127 && sat ? 1 : exp == -128 && sat ? 2 : 0)]++; // 0-5
            cov[6 + (a == -128 ? 0 : a == 127 ? 1 : a == 0 ? 2 : 3)]++; // 6-9
            if (b == -128) cov[10]++; if (b == 127) cov[11]++;
            if (op && b == -128) cov[12]++; if (y == 0) cov[13]++;
        end
        $write("RESULT seed=%0d n=%0d errors=%0d bins=", seed0, n, errs);
        for (i = 0; i < 14; i = i + 1) $write("%0d", cov[i] > 0);
        $display(""); $finish;
    end
end
```

**endmodule**

A compact self-checking testbench: random stimulus with an optional 25% bias towards corner values, a reference model, a checker and 14 hand-rolled coverage bins (a covergroup in a real SV environment).

```
# regress.py - run a seed regression, merge coverage, report closure and failures
import subprocess, re
NAMES = ["add/none", "add/+sat", "add/-sat", "sub/none", "sub/+sat", "sub/-sat",
        "a=-128", "a=127", "a=0", "a=other", "b=-128", "b=127", "sub&b=-128", "y=0"]
def regress(build, bias, seeds=20, n=40):
    merged, fails = [0] * len(NAMES), 0
    print("== %s, corner bias=%d, %d seeds x %d txns == " % (build, bias, seeds, n))
    for seed in range(1, seeds + 1):
        r = subprocess.run(["vvp", "-n", build, "+seed=%d" % seed, "+n=%d" % n,
                           "+bias=%d" % bias], capture_output=True, text=True).stdout
        m = re.search(r"errors=(\d+) bins=([01]+)", r)
        merged = [x | int(c) for x, c in zip(merged, m.group(2))]
        fails += int(m.group(1)) > 0
        if seed in (1, 2, 5, 10, 20):
            print("  %d seeds: coverage %2d/%d (%5.1f%%)  failing seeds: %d"
                  % (seed, sum(merged), len(NAMES), 100.0 * sum(merged) / len(NAMES), fails))
            print("  holes: %s\n" % ([k for k, h in zip(NAMES, merged) if not h] or "none"))
    regress("sim_ok", 1); regress("sim_bug", 1); regress("sim_bug", 0)
```

A miniature regression manager: runs seeds, merges coverage across runs (union of bins), counts failing seeds and lists holes.

```
$ iverilog -g2012 -Wall -o sim_ok satalu.v tb.sv
$ iverilog -g2012 -DEBUG -o sim_bug satalu.v tb.sv
$ python3 regress.py
== sim_ok, corner bias=1, 20 seeds x 40 txns ==
  1 seeds: coverage 11/14 ( 78.6%)  failing seeds: 0
  2 seeds: coverage 14/14 (100.0%)  failing seeds: 0
  5 seeds: coverage 14/14 (100.0%)  failing seeds: 0
 10 seeds: coverage 14/14 (100.0%)  failing seeds: 0
 20 seeds: coverage 14/14 (100.0%)  failing seeds: 0
holes: none

== sim_bug, corner bias=1, 20 seeds x 40 txns ==
  1 seeds: coverage 11/14 ( 78.6%)  failing seeds: 0
  2 seeds: coverage 14/14 (100.0%)  failing seeds: 1
  5 seeds: coverage 14/14 (100.0%)  failing seeds: 4
 10 seeds: coverage 14/14 (100.0%)  failing seeds: 8
 20 seeds: coverage 14/14 (100.0%)  failing seeds: 15
holes: none

== sim_bug, corner bias=0, 20 seeds x 40 txns ==
  1 seeds: coverage  8/14 ( 57.1%)  failing seeds: 0
  2 seeds: coverage  9/14 ( 64.3%)  failing seeds: 0
  5 seeds: coverage 11/14 ( 78.6%)  failing seeds: 0
 10 seeds: coverage 11/14 ( 78.6%)  failing seeds: 0
 20 seeds: coverage 13/14 ( 92.9%)  failing seeds: 0
holes: ['sub&b=-128']

$ vvp -n sim_bug +seed=3 +n=40
ERROR seed=3 op=1 a=77 b=-128 y=-51 exp=127
RESULT seed=3 n=40 errors=1 bins=1111111011110
tb.sv:36: $finish called at 396 (1s)
```

Real Icarus Verilog 12 runs driven by the Python regression script.

The three regressions tell the whole story of coverage-driven verification:

- With corner bias, the correct DUT reaches **100% functional coverage after 2 seeds** and never fails - but that alone does not prove anything: coverage only says the scenarios were exercised.

- The same biased stimulus on the buggy DUT exposes the bug in 15 of 20 seeds. The checker (reference model + compare) detects the failure; the coverage model told us the triggering scenario (**sub&b=-128**) was reached.
- With **pure random** stimulus the buggy DUT **passes every test** - 800 transactions, zero failures. A team that only tracked the pass rate would sign off a broken design. The coverage report shows exactly one hole: the scenario that contains the bug. **Pass rate measures the DUT; coverage measures the testbench.** You need both.

#### THE TWO QUESTIONS OF VERIFICATION

**Did we check?** - checkers, scoreboards and assertions must detect every wrong behaviour that is stimulated. A test with no checker that 'passes' is worthless.

**Did we stimulate?** - coverage (code and functional) must show that every planned scenario was actually exercised. A coverage hole is a place where bugs can hide.

## 7.6 Formal verification

Simulation checks the behaviours that the stimulus happens to produce. **Formal verification** proves properties for **all** possible input sequences, using SAT/BDD-based model checking. It has moved from a niche technique to a standard part of the DV plan, in two forms: **formal property verification (FPV)** on design-specific assertions, and push-button **formal apps**.

```
// Example SVA properties for an arbiter, checked by a formal tool
a_onehot:  assert property (@(posedge clk) disable iff (!rst_n) $onehot0(gnt));
a_no_gnt:  assert property (@(posedge clk) disable iff (!rst_n)
              (gnt & ~req) == '0);           // only grant requesters
a_fair:    assert property (@(posedge clk) disable iff (!rst_n)
              req[0] |-> ##[0:3] gnt[0]);     // bounded latency
m_req_hold: assume property (@(posedge clk) req[1] && !gnt[1] |=> req[1]); // env
c_all:     cover  property (@(posedge clk) gnt == 4'b1000 ##1 gnt == 4'b0001);
```

Assertions, assumptions and covers: the three kinds of formal properties.

| Result                          | Meaning                                   | What to do                                                                                             |
|---------------------------------|-------------------------------------------|--------------------------------------------------------------------------------------------------------|
| <b>Proven</b>                   | Holds for all reachable states from reset | Check the assumptions are not over-constraining (covers must still be reachable)                       |
| <b>Failed (CEX)</b>             | Tool shows a counter-example trace        | Debug like a simulation failure; fix RTL or the property                                               |
| <b>Bounded proof (k cycles)</b> | No failure within k cycles of reset       | Acceptable if k exceeds the design's sequential depth; otherwise add abstractions or helper assertions |
| <b>Inconclusive</b>             | State-space explosion                     | Decompose, black-box datapaths, use abstractions or invariants                                         |

Table 7.3 - Interpreting formal results.

| Formal app                          | What it proves                                                                                           | Typical use                                                  |
|-------------------------------------|----------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|
| <b>Connectivity</b>                 | Signal A at the top reaches pin B of an instance under condition C (pin-mux, test mux, interrupt wiring) | SoC integration; replaces thousands of directed tests        |
| <b>X-propagation</b>                | X from uninitialised flops or X-sources cannot reach specified outputs                                   | Reset-value sign-off, low-power corruption                   |
| <b>Sequential equivalence (SEC)</b> | Two designs produce the same outputs even though their state encoding/pipelining differs                 | Clock-gating insertion, retiming, power optimisations in RTL |
| <b>Register / CSR</b>               | Access policies and reset values match the register spec (IP-XACT)                                       | Register map sign-off                                        |
| <b>Unreachability / coverage</b>    | Code coverage items that can never be hit                                                                | Justify coverage waivers automatically                       |
| <b>Security / taint</b>             | Secret data cannot reach insecure outputs                                                                | Crypto keys, debug locks                                     |

Table 7.4 - Formal apps.

**INTERVIEW INSIGHT - WHERE FORMAL WINS**

Formal is strongest on **control logic with deep corner cases and narrow interfaces**: arbiters, FIFOs, credit counters, cache-coherence protocol controllers, interrupt controllers, clock-gating and power-control sequencers. It struggles with wide datapaths (multipliers, floating point) where the state space is huge - there, simulation plus specialised datapath equivalence tools are used. A strong answer names both the technique and the kind of logic it fits.

## 7.7 Emulation and FPGA prototyping

A full SoC in an RTL simulator runs at roughly hertz to a few kilohertz of chip clock. Booting Linux takes billions of cycles - years in simulation. Two hardware-assisted platforms close the gap:

| Platform                                           | Typical speed        | Strengths                                                                                                                      | Weaknesses                                                                                                                   |
|----------------------------------------------------|----------------------|--------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| <b>RTL simulation</b>                              | ~1 Hz - 10 kHz (SoC) | Full visibility, 4-state, fast compile, cheap per seat                                                                         | Too slow for software                                                                                                        |
| <b>Emulation (Palladium, Veloce, ZeBu)</b>         | ~0.5 - 5 MHz         | Large capacity, fast compile (hours), good debug visibility, transaction-based testbenches, power estimation on real workloads | Expensive, shared capacity                                                                                                   |
| <b>FPGA prototype (HAPS, S2C, in-house boards)</b> | ~5 - 100 MHz         | Runs real software and real I/O at near-system speed; can be shipped to software teams                                         | Long bring-up, partitioning across FPGAs, limited visibility, ASIC-only structures (memories, clock gating) need remodelling |

Table 7.5 - Verification platforms. Speeds are orders of magnitude and depend strongly on design size.

Here the ASIC RTL rules of chapter 6 pay off again: a design with memories behind wrappers and clock gating behind ICG wrappers maps to an emulator or FPGA by swapping the wrapper implementation, without touching functional RTL.

## 7.8 Hardware/software co-verification

Modern SoCs are defined as much by firmware as by RTL. Boot ROM code, power management firmware, drivers and the hardware abstraction layer must be verified **against the RTL before tapeout** - a boot ROM bug is a mask bug. Typical approaches:

- **Virtual platforms** (SystemC/TLM models) let software start months before RTL exists; they must be kept consistent with the register map (generated from the same IP-XACT/SystemRDL source).
- **Hybrid emulation**: CPU subsystem as a fast model, the rest as RTL in the emulator.
- **C tests on the SoC testbench**: the real CPU RTL executes compiled test code, with a small mailbox to the SV testbench for pass/fail and stimulus coordination.
- **Boot ROM sign-off**: every boot path (fuse settings, boot devices, secure boot, recovery modes) is exercised on RTL and gate-level netlist.

## 7.9 Coverage closure

| Coverage type                 | Measures                                                                            | Pitfall                                                            |
|-------------------------------|-------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| <b>Line / statement</b>       | Each executable line was executed                                                   | 100% line coverage says nothing about values                       |
| <b>Branch</b>                 | Each if/case branch taken                                                           | Default branches of full cases are unreachable - waive with reason |
| <b>Condition / expression</b> | Each sub-condition independently affected the outcome (focused expression coverage) | Grows quickly; needs targeted tests                                |

| Coverage type                     | Measures                                                            | Pitfall                                                      |
|-----------------------------------|---------------------------------------------------------------------|--------------------------------------------------------------|
| <b>Toggle</b>                     | Each bit toggled 0->1 and 1->0                                      | Useful for connectivity and tie-offs at SoC level            |
| <b>FSM</b>                        | States and transitions visited                                      | Illegal states must be unreachable, not merely unvisited     |
| <b>Functional (covergroups)</b>   | Planned scenarios, values and crosses from the vPlan                | Only as good as the plan: cannot find what nobody thought of |
| <b>Assertion / cover property</b> | Temporal scenarios happened; assertions were active and not vacuous | Vacuous passes (antecedent never true)                       |

Table 7.6 - Code and functional coverage types.

Closure is iterative: run regressions, merge coverage from all tests and seeds, analyse holes, then either **write a targeted test or constraint**, **prove the hole unreachable** (formal unreachability) and waive it, or **file a bug** if the hole reveals dead or broken logic. Typical sign-off targets are 100% of functional coverage items in the plan (with reviewed waivers) and very high code coverage (often 95-100% after justified exclusions), but the exact targets are a project decision.

## 7.10 Bug tracking and trend curves

Every failure goes into a bug tracker (JIRA, Bugzilla, in-house) with severity, owner, block and the test and seed that reproduce it. Project management watches two curves: the **cumulative bugs found** and the **open bug count**. A healthy project shows an S-shaped cumulative curve that flattens as the design matures.

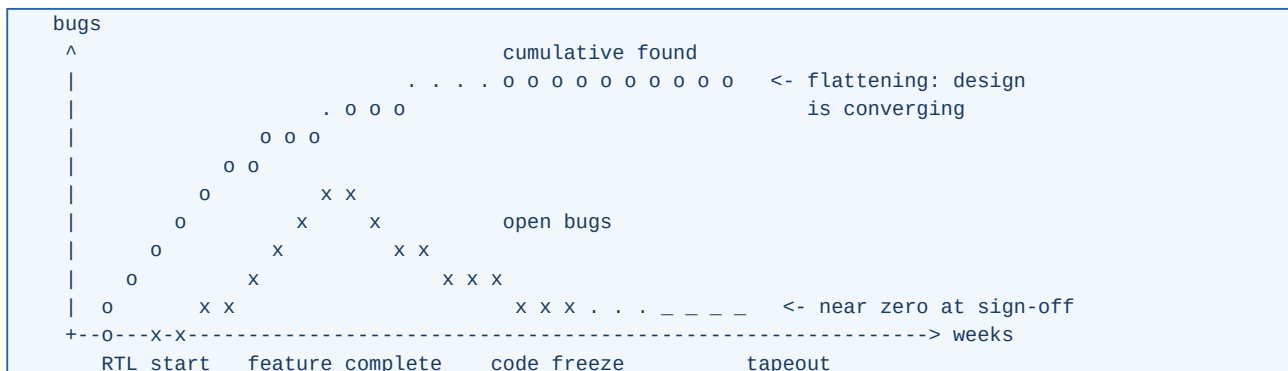


Figure 7.3 - Bug trend curves (schematic). A late spike in found bugs after 'feature complete' is a warning that verification started late or the design is still changing.

- **Bug rate per week** should trend down over the last few weeks before sign-off; many teams require no new high-severity bugs for N consecutive regression cycles.
- **Bugs per block** point to risky IP - a block with many late bugs deserves a formal review or extra testing.
- **Root-cause categories** (spec ambiguity, RTL coding, integration, testbench bug) show where the process leaks.

## 7.11 Regression management

- **Tiers of regression:** a smoke/sanity set on every commit (minutes), a nightly regression (thousands of tests, random seeds), a weekly full regression including gate-level and power-aware runs.
- **Seed management:** failing seeds are saved and replayed; random stability (the same seed reproduces the same stimulus after unrelated code changes) matters for debug.
- **Triage:** failures are auto-bucketed by error signature so that one RTL bug that fails 500 tests is one ticket.
- **Compute farm:** regressions run on hundreds to thousands of cores under a job scheduler (LSF, Slurm, cloud); licences are often the real bottleneck.
- **Metrics dashboard:** pass rate, coverage trend, runtime, flaky tests - reviewed weekly by the DV lead.

## 7.12 Gate-level simulation

After synthesis and after place-and-route, the netlist is simulated again. Formal equivalence (LEC, chapter 8) already proves the netlist implements the RTL, so gate-level simulation (GLS) is **not** for functional coverage. It catches what LEC and STA cannot:

| GLS flavour                                 | What it catches                                                                                                                                                                                                                               |
|---------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Zero-delay (unit-delay) netlist</b>      | Reset and initialisation problems hidden by RTL X-optimism (an <code>if (x)</code> in RTL takes the else-branch; gates propagate X), missing resets, power-up sequences, scan/DFT mode functionality, netlist connectivity to macros and pads |
| <b>SDF back-annotated (min/max corners)</b> | Timing-check violations on asynchronous interfaces, reset release, synchronisers, un-timed paths with wrong exceptions, glitches on clock muxes; validates the SDC exceptions                                                                 |
| <b>Scan / ATPG pattern simulation</b>       | The test patterns work on the real netlist with timing (chapter 10) before they go to the tester                                                                                                                                              |

Table 7.7 - Gate-level simulation flavours.

### PITFALL - 'GLS PASSES' WITH TIMING CHECKS DISABLED

SDF simulation with `+notimingchecks`, or with every synchroniser's first flop excluded from X-generation carelessly, hides the problems GLS exists to find. The correct approach is to disable timing checks only on the **first stage of known synchronisers** (by instance list generated from the CDC report) and keep them everywhere else.

## 7.13 Power-aware simulation

When a design has switchable power domains (chapter 11), the simulator reads the UPF and models what RTL alone cannot: when a domain is off, its registers and outputs are **corrupted** (driven to X); isolation cells clamp outputs to their specified value; retention registers save and restore state; level shifters and supply states are checked. Power-aware simulation finds missing isolation, wrong isolation enables, sequencing errors in the power controller and software that accesses a switched-off block. It runs at RTL (with the UPF) and at gate level (with the implemented power network and cells).

## 7.14 Verification sign-off

### Functional verification sign-off checklist (tapeout gate)

- ☐ Verification plan reviewed and 100% of plan items closed, deferred with sign-off from architecture, or covered by reuse.
- ☐ Functional coverage targets met; all holes analysed; waivers reviewed and signed.
- ☐ Code coverage (line, branch, condition, toggle, FSM) at target with justified exclusions, preferably backed by formal unreachability.
- ☐ Formal: all planned properties proven or bounded beyond the sequential depth; connectivity and X-prop apps clean.
- ☐ Regression: N consecutive clean nightly regressions on the final RTL tag; no open severity-1/2 bugs; bug rate trend flat.
- ☐ SoC tests: boot paths, clocks, resets, power modes, pin-mux, DFT and debug modes exercised.
- ☐ Software: boot ROM and key firmware run on emulation/prototype on the final RTL.
- ☐ GLS: zero-delay and SDF (min/max) runs clean; scan patterns simulated.
- ☐ Power-aware simulation clean for all power-state transitions.
- ☐ LEC RTL-to-netlist clean (chapter 8), ECO changes re-verified.



## 7.15 Summary

- Verification is typically the largest front-end effort; DV engineers often outnumber designers.
- The verification plan maps every spec feature to a technique, a level and measurable done-criteria.
- Testbenches are tiered unit -> block -> subsystem -> SoC -> system, with UVM components reused across tiers.
- Our Icarus run showed a buggy DUT passing 100% of purely random tests while coverage showed the one hole containing the bug: pass rate measures the DUT, coverage measures the testbench.
- Formal proves properties exhaustively and its apps (connectivity, X-prop, SEC, unreachability) replace large amounts of directed simulation.
- Emulation and FPGA prototypes provide the MHz speed needed for software; GLS and power-aware simulation catch what RTL simulation, LEC and STA cannot.
- Sign-off is an evidence-based checklist: plan closure, coverage, formal results, regression and bug trends, software, GLS and power-aware results.

## 7.16 Exercises

- 1 Write verification plan entries (feature, technique, level, done-criteria) for an asynchronous FIFO with almost-full and almost-empty flags.
- 2 In the saturating-ALU example, compute the probability that 800 purely random transactions contain at least one subtraction with  $b = -128$ . Why did our 20-seed run miss it?
- 3 Give two bugs that 100% line and branch coverage cannot find, and the functional coverage or assertion that would.
- 4 Your arbiter proof returns 'bounded proof,  $k = 25$ '. What do you need to know to decide whether this is sufficient?
- 5 Explain why gate-level simulation still exists when LEC proves RTL and netlist equivalent. Give three bug classes found only in GLS.
- 6 A project has 20 open bugs one week before tapeout and the weekly find rate is 8. What would you recommend, and what data would you ask for?



## 8. Logic Synthesis in Depth

Logic synthesis translates RTL into a gate-level netlist of standard cells that meets timing, area, power and design-rule constraints. The companion RTL Design guide explains what each RTL idiom becomes; this chapter opens the tool itself: the steps of a production flow, the algorithms behind mapping and optimisation, how datapaths are synthesised, why wire-load models failed and physical-aware synthesis replaced them, how to read QoR reports and warnings, and how the result is proven equivalent to the RTL.

Synthesis is owned by the **synthesis / implementation engineer** (in some companies the RTL designer runs it for blocks, and a central team runs the full chip). Inputs: RTL + filelist, SDC, UPF, Liberty (.lib/.db) for every corner used, physical data (LEF/tech file, floorplan DEF) for physical-aware runs, and the DFT specification. Outputs: netlist, updated SDC, UPF, scan definition (SCANDEF), reports and a LEC setup. All numbers in this chapter's runs are from **Yosys 0.33 with ABC** against the real **SKY130 sky130\_fd\_sc\_hd tt\_025C\_1v80** Liberty file.

### 8.1 The production synthesis flow

| RTL + filelist<br>  defines, params                                                                                                     | SDC<br>  (+ UPF) | Liberty (all<br>  used corners) | LEF / tech / DEF<br>  (physical-aware) |
|-----------------------------------------------------------------------------------------------------------------------------------------|------------------|---------------------------------|----------------------------------------|
| 1 analyze (parse)                                                                                                                       |                  |                                 |                                        |
| 2 elaborate (generic/GTECH netlist, inferred flops, operators)                                                                          |                  |                                 |                                        |
| 3 link (resolve cells/macros against libraries)                                                                                         |                  |                                 |                                        |
| <-----+ 4 read constraints, check_design/timing                                                                                         |                  |                                 |                                        |
| 5 compile: generic opt -> technology mapping -> timing/area/power ---+<br>opt (sizing, buffering, restructuring, Vt swap, clock gating) |                  |                                 |                                        |
| 6 DFT: scan replacement + insert_dft (chains, compression, OCC hooks)                                                                   |                  |                                 |                                        |
| 7 incremental compile (fix what DFT and physical estimates broke)                                                                       |                  |                                 |                                        |
| 8 reports: QoR, timing, area, power, constraints, check_design                                                                          |                  |                                 |                                        |
| 9 write: netlist (.v), SDC, UPF, SCANDEF, SAIF mapping, LEC guide file                                                                  |                  |                                 |                                        |

Figure 8.1 - Steps of a production synthesis flow.

```
# Illustrative Design Compiler (Tcl) flow - commands are standard, paths generic
set_app_var target_library "stdcell_ss_0p72v_125c.db"      ;# mapping + timing
set_app_var link_library   "*" $target_library sram_macros.db"
analyze -format sverilog -vcs "-f rtl.f"
elaborate soc_top
link
read_sdc constraints/soc_top.sdc
load_upf soc_top.upf
check_design ; check_timing
set_dont_use [get_lib_cells */*_X0P5*]                    ;# no weak cells
compile_ultra -gate_clock -retime -spg                     ;# topographical / physical guide
insert_dft
compile_ultra -incremental -scan
report_qor > rpt/qor.rpt ; report_timing -max_paths 50 > rpt/timing.rpt
write -format verilog -hierarchy -output out/soc_top.vg
write_sdc out/soc_top.sdc ; write_scan_def -output out/soc_top.scandef
```

A typical Design Compiler script. The Cadence Genus equivalent is read\_hdl / elaborate / read\_sdc / syn\_generic / syn\_map / syn\_opt / write\_hdl; Yosys uses read\_verilog / synth / dfflibmap / abc.

## 8.2 Elaboration and the generic (GTECH) netlist

Elaboration builds a technology-independent netlist. Operators such as `+`, `*` and `<` become **high-level operator cells** (Synopsys keeps them as DesignWare/GTECH operators, Yosys as `$add`, `$alu`, `$mul`); flops become generic sequential cells with reset and enable attributes; control logic becomes generic Boolean gates. Yosys lets us watch the 32-bit adder `assign s = a + b;` pass through these stages:

```
# after the 'coarse' stage (operators still abstract)
Number of cells:          1
  $alu                    1
# after the 'fine' stage (techmap to generic gates, before library mapping)
Number of cells:          235
  $_ANDNOT_               53
  $_AND_                  11
  $_NAND_                 21
  $_NOR_                  34
  $_ORNOT_                11
  $_OR_                   42
  $_XNOR_                 27
  $_XOR_                  36
```

Real Yosys stat output for add32 at two stages of synth. The generic gate library (`$_AND_`, `$_XOR_`...) plays the role of GTECH.

Keeping arithmetic abstract as long as possible is what allows **datapath synthesis** (section 8.6) to choose an architecture per operator later, with timing information. Premature flattening into gates loses that choice.

## 8.3 Boolean optimisation and technology mapping

Most modern synthesis engines represent combinational logic as an **And-Inverter Graph (AIG)**: a DAG of 2-input AND nodes with optional inversion on edges. The AIG is a compact, canonical-ish form on which fast rewriting algorithms run:

- **Structural hashing** merges identical nodes; **constant propagation** and **SAT sweeping / fraiging** merge nodes proven functionally equivalent.
- **Rewriting and refactoring** replace small sub-graphs (for example 4-input cuts) by smaller pre-computed equivalents; **balancing** reduces logic depth.
- **Don't-care optimisation** uses satisfiability and observability don't-cares to simplify logic whose value does not matter.
- **Technology mapping** covers the optimised AIG with library cells. The classic formulation is tree covering by dynamic programming (DAGON, 1987); modern mappers enumerate **cuts** (small sub-graphs with  $k$  inputs), match each cut's Boolean function against library cells, and pick the cover minimising delay, then recover area on non-critical paths.

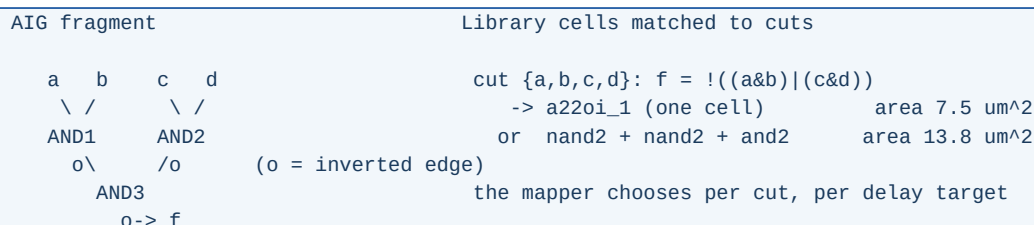


Figure 8.2 - Cut-based mapping: the same AIG sub-graph can be covered by one complex cell (AOI) or several simple ones. (Areas: SKY130 a22oi\_1 = 7.51  $\mu\text{m}^2$ ; nand2\_1 = 3.75  $\mu\text{m}^2$ ; and2\_1 = 6.26  $\mu\text{m}^2$ .)

ABC, the open-source engine from UC Berkeley that Yosys uses, exposes these steps as commands (`strash`, `dc2`, `dch`, `map`, `&nf`, `buffer`, `upsized`, `dnsize`, `stime`). Commercial engines use the same ideas with far larger libraries of transformations, better timing models and tight coupling to placement.

## 8.4 Timing-driven optimisation

After mapping, the tool times the netlist with its internal STA engine (the same NLDM delay calculation as chapter 9) and repeatedly applies transformations to the worst paths:

| Transformation                      | What it does                                                                                                                                   | Cost                                 |
|-------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------|
| <b>Gate sizing</b>                  | Swap a cell for a stronger/weaker drive of the same function (nand2_1 -> nand2_4); upsizing speeds the driven net but loads the previous stage | Area, leakage, input cap             |
| <b>Buffering / fanout splitting</b> | Insert buffers or build a buffer tree on high-fanout or long nets; isolate critical sinks from non-critical load                               | Area, power                          |
| <b>Restructuring</b>                | Re-express logic so late-arriving signals pass through fewer levels (e.g. Shannon expansion on a late input, re-balancing a chain)             | Area                                 |
| <b>Pin swapping</b>                 | Connect the latest signal to the fastest input pin of a cell (pins of a NAND stack are not equal)                                              | None                                 |
| <b>Multi-Vt assignment</b>          | Use low-Vt (fast, leaky) cells only on critical paths, high-Vt (slow, low leakage) elsewhere                                                   | Leakage (roughly exponential in Vth) |
| <b>Logic duplication</b>            | Duplicate a driver so each copy drives fewer, better-placed sinks                                                                              | Area                                 |
| <b>Area recovery</b>                | Downsize and re-map cells on paths with positive slack                                                                                         | Timing margin                        |

Table 8.1 - Timing-driven optimisation moves.

### WHY LOW-VT EVERYWHERE IS NOT THE ANSWER

Subthreshold leakage grows roughly exponentially as Vth drops (about a decade per 60-100 mV of Vth at room temperature, chapter 2), and grows further with temperature. A typical multi-Vt flow starts with the high- or standard-Vt library and allows low-Vt swaps only where the setup slack is negative, often with a budget such as 'no more than a few percent of cells in ulvt'. Leakage is then reported per Vt class.

## 8.5 A real run: the area-delay trade-off of a 32-bit adder

To see timing-driven mapping at work, we synthesised two 32-bit adders against the SKY130 library with a sweep of ABC delay targets (`abc -D <ps>`). Both compute the same function: `add32` is `assign s = a + b`; (Yosys decomposes `$alu` into its default carry-lookahead structure), and `ks32` is a hand-written **Kogge-Stone** parallel-prefix adder.

```
// 32-bit Kogge-Stone parallel-prefix adder (structural generate/propagate tree)
module ks32 (input [31:0] a, b, output [32:0] s);
  wire [31:0] g0 = a & b, p0 = a ^ b;
  wire [31:0] g [0:5];
  wire [31:0] p [0:5];
  assign g[0] = g0; assign p[0] = p0;
  genvar l, i;
  generate for (l = 0; l < 5; l = l + 1) begin : lvl
    for (i = 0; i < 32; i = i + 1) begin : bit
      if (i >= (1 << l)) begin : op
        assign g[l+1][i] = g[l][i] | (p[l][i] & g[l][i-(1<<l)]);
        assign p[l+1][i] = p[l][i] & p[l][i-(1<<l)];
      end else begin : pass
        assign g[l+1][i] = g[l][i];
        assign p[l+1][i] = p[l][i];
      end
    end
  end
end generate
assign s[0] = p0[0];
assign s[31:1] = p0[31:1] ^ g[5][30:0];
assign s[32] = g[5][31];
endmodule
```

ks32.v:  $\log_2(32) = 5$  prefix levels; every level has up to 32 black cells, which gives minimum depth at the price of area and wiring.

```
# sweep.sh (core): one Yosys run per delay target D
read_verilog $f
synth -top $t -flatten
dfflibmap -liberty hd_tt_du.lib
abc -fast -liberty hd_tt_du.lib -constr constr.txt -D $D # -D omitted for 'none'
opt_clean
stat -liberty hd_tt_du.lib

# constr.txt - driving cell and output load seen by ABC's timer
set_driving_cell sky130_fd_sc_hd__inv_2
set_load 0.01
```

Delay-target sweep. With -constr, the ABC script is 'strash; dretime; map {D}; buffer; upsize {D}; dnsz {D}; stime -p', i.e. mapping, buffering and gate sizing towards the target D.

|       |      |           |                      |                      |
|-------|------|-----------|----------------------|----------------------|
| add32 | none | cells=238 | area_um2=1326.272000 | abc_delay_ps=1460.14 |
| add32 | 4000 | cells=233 | area_um2=1204.905600 | abc_delay_ps=2246.78 |
| add32 | 2000 | cells=232 | area_um2=1212.412800 | abc_delay_ps=1962.67 |
| add32 | 1800 | cells=230 | area_um2=1213.664000 | abc_delay_ps=1750.37 |
| add32 | 1600 | cells=223 | area_um2=1251.200000 | abc_delay_ps=1591.88 |
| add32 | 1400 | cells=229 | area_um2=1296.243200 | abc_delay_ps=1558.49 |
| add32 | 1200 | cells=238 | area_um2=1326.272000 | abc_delay_ps=1460.14 |
| add32 | 1000 | cells=238 | area_um2=1326.272000 | abc_delay_ps=1460.14 |
| add32 | 900  | cells=238 | area_um2=1326.272000 | abc_delay_ps=1460.14 |
| ks32  | none | cells=548 | area_um2=3178.048000 | abc_delay_ps=1014.37 |
| ks32  | 4000 | cells=422 | area_um2=2019.436800 | abc_delay_ps=1329.48 |
| ks32  | 2000 | cells=422 | area_um2=2019.436800 | abc_delay_ps=1329.48 |
| ks32  | 1800 | cells=422 | area_um2=2019.436800 | abc_delay_ps=1329.48 |
| ks32  | 1600 | cells=423 | area_um2=2026.944000 | abc_delay_ps=1409.97 |
| ks32  | 1400 | cells=397 | area_um2=1948.118400 | abc_delay_ps=1289.69 |
| ks32  | 1200 | cells=387 | area_um2=2036.953600 | abc_delay_ps=1192.82 |
| ks32  | 1000 | cells=468 | area_um2=2418.569600 | abc_delay_ps=1061.02 |
| ks32  | 900  | cells=548 | area_um2=3178.048000 | abc_delay_ps=1014.37 |

Real results: cell count and stat -liberty area ( $\mu\text{m}^2$ ) from Yosys, delay (ps) from ABC's 'stime' timer with SKY130 NLDM tables.

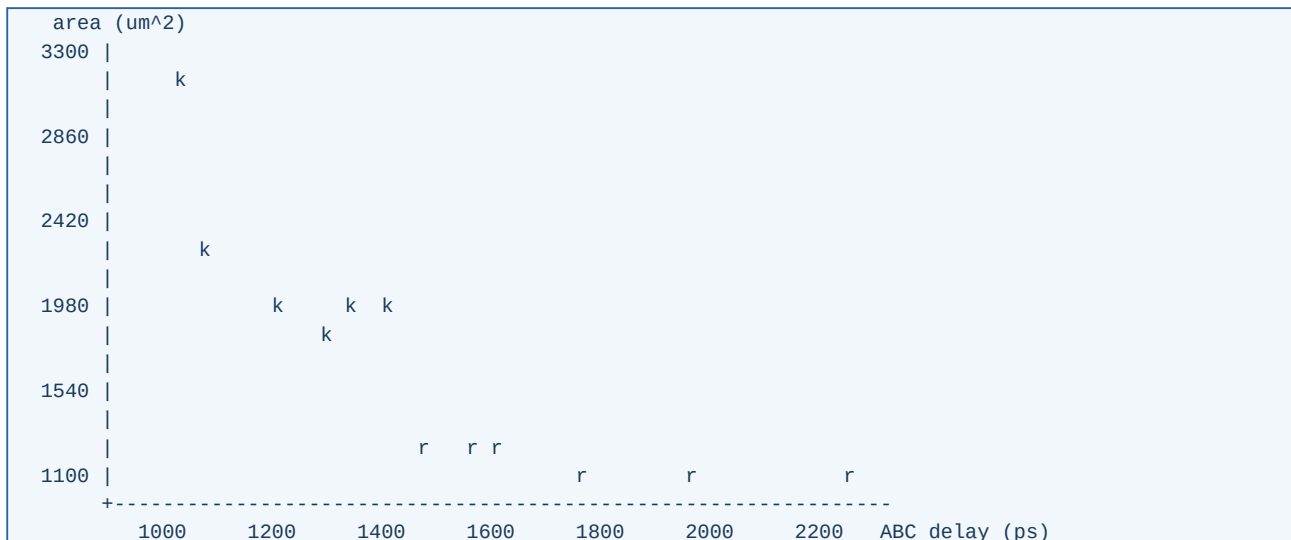


Figure 8.3 - The same data plotted: r = add32 (a + b), k = ks32 (Kogge-Stone). Each architecture traces its own area-delay curve; the fastest point costs about 63% more area than the smallest Kogge-Stone point and about 2.6x the smallest add32 point.

What the numbers teach:

- **Tight targets cost area.** For add32 the delay-optimal mapping (no target, or targets below what is reachable) gives 1460 ps at  $1326 \mu\text{m}^2$ ; relaxing the target to 4000 ps lets the mapper recover area down to  $1205 \mu\text{m}^2$  at 2247 ps. That is the classic 'banana curve' every synthesis engineer knows.

- **Architecture beats optimisation.** No delay target made add32 faster than about 1.46 ns, while the Kogge-Stone structure reached 1.01 ns - at 2.4x the area. The mapper optimises locally around the structure it is given; choosing the adder architecture is a datapath synthesis decision (next section).
- **Heuristics are noisy.** The ks32 points at 1600 ps and 1400 ps are not monotonic: a tighter target produced a smaller netlist. Real tools show the same effect, which is why teams run several strategies and keep the best result, and why 1-2% QoR differences between runs are not significant.
- **Unconstrained is not 'smallest'.** ABC's default (no -D) aims for minimum delay. A production tool without a clock constraint behaves differently (it optimises area), which is one reason check\_timing must report zero unconstrained endpoints.

```

=== add32 === (D = 1600 ps)
Number of cells:                223
  sky130_fd_sc_hd__a21boi_0      3
  sky130_fd_sc_hd__a21oi_1     11
  sky130_fd_sc_hd__and2_1        9
  sky130_fd_sc_hd__inv_1        26
  sky130_fd_sc_hd__nand2_1       40
  sky130_fd_sc_hd__nor2_1       54
  sky130_fd_sc_hd__o21ai_0       8
  sky130_fd_sc_hd__o21ai_4       2
  sky130_fd_sc_hd__xnor2_1      11
  sky130_fd_sc_hd__xor2_1       27
  ... (22 further cell types with 1-3 instances each)
Chip area for module '\add32': 1251.200000
ABC: ... Gates = 223 ... Area = 1251.20 ... Delay = 1591.88 ps

```

Real stat -liberty excerpt: note the mix of drive strengths (\_0, \_1, \_4) chosen by sizing, and the dominance of NAND/NOR/AOI/OAI - inverting CMOS cells are the fastest and smallest per function.

## 8.6 Datapath synthesis

Arithmetic dominates the area of many blocks (DSP, ML accelerators, GPUs). Commercial tools therefore contain a dedicated **datapath synthesis** engine (Synopsys DesignWare Foundation / 'DW' components and datapath extraction; Cadence Genus datapath synthesis) which: extracts clusters of arithmetic (sums of products, comparators, shifters) from the RTL; merges them into a single **carry-save** tree so that only one carry-propagate adder is needed at the end; and picks each operator's architecture from a library of implementations according to the timing context.

| Structure                                   | Delay                       | Area                              | When chosen                           |
|---------------------------------------------|-----------------------------|-----------------------------------|---------------------------------------|
| <b>Ripple-carry adder</b>                   | $O(n)$                      | $O(n)$ , smallest                 | Non-critical, narrow adders           |
| <b>Carry-lookahead / carry-select</b>       | $O(\log n)$ / $O(\sqrt{n})$ | Moderate                          | Medium widths, moderate timing        |
| <b>Parallel prefix: Brent-Kung</b>          | $2 \log_2 n - 1$ levels     | $O(n)$ , low fanout               | Area/power-aware fast adders          |
| <b>Parallel prefix: Kogge-Stone</b>         | $\log_2 n$ levels           | $O(n \log n)$ , many wires        | Fastest adders, when wiring allows    |
| <b>Parallel prefix: Sklansky</b>            | $\log_2 n$ levels           | $O(n/2 \log n)$ , high fanout     | Fast with buffering                   |
| <b>Array multiplier</b>                     | $O(n)$                      | $O(n^2)$                          | Rare in ASIC today                    |
| <b>Booth (radix-4) + Wallace/Dadda tree</b> | $O(\log n)$                 | $O(n^2)$ , fewer partial products | Standard for fast multipliers         |
| <b>Carry-save accumulation (MAC)</b>        | one CPA per result          | Saves adders                      | $a*b + c$ , dot products, FIR filters |

Table 8.2 - Datapath architectures a synthesis tool chooses from.

A 16x16 signed multiply-accumulate with a 40-bit accumulator, synthesised the same way, shows how much a datapath costs and how strongly its area depends on the timing target:

```

module mac16 (input clk, rst_n, clr, en, input signed [15:0] a, b,
              output reg signed [39:0] acc);
  always @(posedge clk or negedge rst_n)
    if (!rst_n)   acc <= 40'sd0;
    else if (clr) acc <= 40'sd0;
    else if (en)  acc <= acc + a * b;
endmodule

```

mac16.v

|       |      |            |                       |                      |
|-------|------|------------|-----------------------|----------------------|
| mac16 | none | cells=3855 | area_um2=21052.691200 | abc_delay_ps=3731.33 |
| mac16 | 8000 | cells=2410 | area_um2=14776.672000 | abc_delay_ps=5745.17 |
| mac16 | 6000 | cells=2410 | area_um2=14776.672000 | abc_delay_ps=5745.17 |
| mac16 | 5000 | cells=2404 | area_um2=14869.260800 | abc_delay_ps=4971.30 |
| mac16 | 4000 | cells=2407 | area_um2=15292.166400 | abc_delay_ps=4242.06 |
| mac16 | 3500 | cells=2882 | area_um2=16605.926400 | abc_delay_ps=3975.57 |
| mac16 | 3000 | cells=3647 | area_um2=19942.876800 | abc_delay_ps=3740.92 |

Real sweep for mac16 (40 dfrtp\_1 flops included in every point; delay is the combinational register-to-register logic as seen by ABC).

Going from 5.75 ns to 3.73 ns (-35% delay) costs +42% area. Yosys has no Booth recoding or carry-save merging in this version, so these numbers are for a simple array-style multiplier followed by an adder; a commercial datapath engine would merge the accumulator into the partial-product tree and do considerably better. The lesson for the RTL designer: **write arithmetic as expressions** (`acc + a * b`) so the tool can merge it, rather than hand-instantiating adders, unless you need a specific architecture.

## 8.7 Retiming, clock-gating insertion and sequential optimisations

### Retiming

**Retiming** moves registers across combinational logic without changing the cycle-level behaviour at the outputs, to balance stage delays. It is invaluable for deep arithmetic pipelines: write `result = f(x)` followed by N register stages, and let the tool distribute the registers inside f. Restrictions: registers with different reset/enable conditions, registers visible to software/debug, and registers crossing hierarchy or used by constraints usually cannot move; LEC then needs sequential equivalence checking or retiming-aware guidance.

|                                                                         |                                          |                         |
|-------------------------------------------------------------------------|------------------------------------------|-------------------------|
| before:                                                                 | --[comb 6 ns]--[comb 2 ns]-- FF -- FF -- | stage delays 8 ns, 0 ns |
| after :                                                                 | --[comb 4 ns]-- FF --[comb 4 ns]-- FF -- | stage delays 4 ns, 4 ns |
| (same latency, same function at the outputs, register count may change) |                                          |                         |

Figure 8.4 - Retiming balances pipeline stages.

### Clock-gating insertion

With clock gating enabled (`compile_ultra -gate_clock` in DC, `lp_insert_clock_gating` in Genus), the tool looks for groups of flops that share an enable, removes the feedback multiplexers and inserts an ICG cell. Controls: minimum bit-width per gate (e.g. 3 or 4), maximum fanout per ICG, the ICG cell to use, and whether the test-enable pin connects to scan-enable or test-mode. Timing of the enable path to the ICG (clock-gating setup check) is often critical because the ICG sits early in the clock tree.

### Other sequential optimisations

- **Constant-flop removal:** flops that can never change (tied inputs) are removed - a frequent source of 'my register disappeared' questions and of LEC unmapped points.
- **Equivalent-flop merging:** two flops with identical inputs are merged unless marked `dont_touch` (redundant flops used for fanout or safety must be protected).
- **FSM re-encoding:** extracted FSMs may be re-encoded (one-hot, Gray); keep it off for safety-critical or observed state registers.

- **Scan-flop replacement:** in DFT-aware compile, flops are mapped directly to scan flops so timing sees the mux delay from the start (chapter 10).

## 8.8 Hierarchy, boundary optimisation and ungrouping

Synthesis can keep the RTL hierarchy or dissolve it. Each choice has consequences downstream:

| Option                                                   | Effect                                                                                          | Trade-off                                                     |
|----------------------------------------------------------|-------------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| <b>Keep hierarchy, no boundary optimisation</b>          | Each module optimised in isolation; ports preserved exactly                                     | Worst QoR; easiest LEC, ECO and debug                         |
| <b>Boundary optimisation</b>                             | Constants propagated and unused ports removed across boundaries; inverters pushed through ports | Better QoR; LEC needs guidance; port behaviour changes        |
| <b>Ungroup / flatten (auto-ungroup small modules)</b>    | Hierarchy dissolved, global optimisation                                                        | Best QoR; names change, harder ECO and UPF/constraint mapping |
| <b>Hard boundaries (dont_touch, hierarchical blocks)</b> | Block implemented separately, abstract model at the top                                         | Required for hierarchical implementation and IP reuse         |

Table 8.3 - Hierarchy handling.

### PITFALL - HIERARCHY THAT UPF AND SDC DEPEND ON

Power domains in UPF and many SDC exceptions reference hierarchical instance names. If synthesis ungroups a module that defines a power domain boundary, or a path exception points to a pin that no longer exists, the constraint silently stops applying. Mark power-domain and constraint-anchor hierarchies as preserved, and check 'ignored constraints' reports after every compile.

## 8.9 From wire-load models to physical-aware synthesis

Before placement, synthesis does not know how long each wire will be. Classic flows used **wire-load models (WLMs)**: a statistical table giving wire length (hence R and C) as a function of fanout and block size. The SKY130 Liberty file still contains them:

```
wire_load("Small") {
  capacitance : 1.42e-05;
  resistance : 0.0745;
  slope : 8.3631;
  fanout_length( 1, 23.2746);
  fanout_length( 2, 32.1136);
  fanout_length( 3, 48.4862);
  fanout_length( 4, 64.0974);
  fanout_length( 5, 86.2649);
  fanout_length( 6, 84.2649);
}
```

Wire-load table from `sky130_fd_sc_hd__tt_025C_1v80.lib` (verbatim): length per fanout, plus capacitance and resistance per unit length. Note the non-monotonic entry at fanout 6 - such tables are statistical fits, not physics.

WLMs worked when gate delay dominated. From roughly the 130-90 nm generations on, wire delay became a large share of path delay and WLMs failed for fundamental reasons:

- A net's length depends on **where its pins are placed**, not on its fanout. A fanout-2 net between distant blocks can be millimetres long; a fanout-10 net inside a cluster can be tiny.
- WLMs are **pessimistic on average and optimistic on the worst nets** - precisely the ones that fail after placement.
- Result: synthesis timing no longer correlated with place-and-route timing, and designs looped between synthesis and layout ('timing closure crisis').



The replacement is **physical-aware synthesis**: the synthesis tool runs a fast, coarse placement internally, using the real floorplan (die size, macro positions, blockages, pin locations) and the technology's RC per layer, and estimates each wire from its placed pins. Examples: Synopsys **DC Topographical** (DC-T), **DC-NXT/DC Graphical** (adds congestion awareness and can pass placement guidance to ICC2 - 'SPG'), **Fusion Compiler** (synthesis and P&R in one data model), and Cadence **Genus iSpatial** (runs Innovus placement/optimisation engines inside Genus). Typical claims are timing correlation to post-placement within a few percent, versus tens of percent with WLMs.

```
WLM flow:      RTL -> synth (WLM guess) -> netlist -> place -> timing surprise
                ^                               |
                +----- re-synthesise <-----+

physical-aware: RTL + floorplan DEF + tech/LEF -> synth with virtual placement
                -> netlist (+ placement guidance) -> P&R starts close to closure
```

Figure 8.5 - Why the industry moved to physical-aware synthesis.

## 8.10 Synthesis constraints and design rules

Synthesis reads the same SDC as STA (chapter 9 explains each command) plus design-rule and optimisation constraints. **Design rule constraints (DRCs)** have priority over timing: the tool fixes them first, because a violation means the library tables are being used outside their characterised range.

```
create_clock -name clk -period 2.0 [get_ports clk]
set_clock_uncertainty -setup 0.15 [get_clocks clk]      ;# jitter + margin pre-CTS
set_clock_transition 0.10 [get_clocks clk]             ;# ideal clock slew pre-CTS
set_input_delay 0.8 -clock clk [remove_from_collection [all_inputs] [get_ports clk]]
set_output_delay 0.6 -clock clk [all_outputs]
set_driving_cell -lib_cell sky130_fd_sc_hd__buf_2 [all_inputs]
set_load 0.02 [all_outputs]                            ;# pF, from the next block
# design rule constraints
set_max_transition 1.0 [current_design]                ;# ns - within Liberty table range
set_max_capacitance 0.2 [current_design]               ;# pF
set_max_fanout 20 [current_design]                     ;# optimisation guide, not a physical law
# optimisation controls
set_dont_use [get_lib_cells */sky130_fd_sc_hd__probe_*]
set_dont_touch [get_cells u_sync*/u_meta_ff]           ;# synchroniser flops
group_path -name io -from [all_inputs] -to [all_outputs]
```

Synthesis constraint set (SDC plus Synopsys-style optimisation commands). Values are examples, not recommendations.

| Design rule            | Source                                                                                    | Why it matters                                                                                         |
|------------------------|-------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| <b>max_transition</b>  | Liberty default_max_transition (1.5 ns in the SKY130 hd tt file), pin max_transition, SDC | Beyond the table the delay is extrapolated; slow slews raise short-circuit power and noise sensitivity |
| <b>max_capacitance</b> | Liberty per output pin (e.g. 0.162 pF for dfxtp_1 Q in SKY130 tt), SDC                    | Cell driving beyond its characterised load; also EM limits                                             |
| <b>max_fanout</b>      | SDC / library fanout_load                                                                 | Proxy for load and placement spread; modern flows rely more on cap and transition                      |

Table 8.4 - Design rule constraints.

## 8.11 Reading QoR reports

After every compile, the synthesis engineer reads a small set of reports in a fixed order: constraint problems first, then design rules, then timing, then area and power. The excerpt below shows the structure of a Design Compiler `report_qor`.



```
# ---- ILLUSTRATIVE report_qor excerpt (generic numbers, not from a real run) ----
Timing Path Group 'clk'
-----
Levels of Logic:          24.00      <- depth of the worst path
Critical Path Length:     1.93      <- ns, data arrival
Critical Path Slack:      -0.07     <- WNS for this group
Critical Path Clk Period: 2.00
Total Negative Slack:     -1.84     <- TNS: sum over failing endpoints
No. of Violating Paths:   61.00
Cell Count / Area
Combinational Area:       182345.1
Noncombinational Area:    96210.4   <- flops, latches, ICGs
Macro/Black Box Area:     410000.0
Design Rules
Total Number of Nets:     61234
Nets With Violations:     3         <- max_trans / max_cap: fix before timing
```

How to read a QoR report (illustrative, not from a real run).

- **WNS / TNS / number of violating paths** per path group. A small WNS with a large TNS means many near-critical paths - a structural problem, not a single bad path.
- **Levels of logic** on the worst path: compare with the FO4 budget of chapter 6. Twice the budget means RTL must change.
- **Area split**: combinational vs sequential vs macro; watch for unexpected sequential area (inferred memories).
- **DRC violations**: max transition/capacitance must be zero or explained (e.g. ideal nets before CTS).
- **check\_timing**: unconstrained endpoints, missing input delays, combinational loops, multiple clocks on a register.

## 8.12 Common synthesis warnings and what they mean

Real synthesis logs contain thousands of lines. Knowing which warnings are harmless and which indicate broken RTL is a core skill. A deliberately broken module run through Yosys:

```
module warn (input clk, input [1:0] sel, input [7:0] a, b, output reg [7:0] y,
             output [7:0] z, output w);
    wire [7:0] nc;                      // declared, never driven
    always @* case (sel)                // incomplete: no default, sel=3 not covered
        2'd0: y = a; 2'd1: y = b; 2'd2: y = a & b;
    endcase
    assign z = nc ^ a;
    assign w = a[0];
    assign w = b[0];                    // second driver on w
endmodule
```

warn.v - three classic RTL mistakes.

```
Latch inferred for signal `warn.y' from process `warn.$proc$warn.v:4$1': $auto$...
Warning: multiple conflicting drivers for warn.a [0]:
Warning: Wire warn.nc [7] is used but has no driver.
Warning: Wire warn.nc [6] is used but has no driver.
... (same for bits 5..0)
Found and reported 9 problems.
$DLATCH_N_ 8
```

Real Yosys 0.33 messages (excerpt; long internal names shortened). Note that the conflicting-driver warning names a[0], because w and a[0] were merged into one net before the check ran.

| Message (paraphrased, any tool) | Meaning                                                        | Action                                                              |
|---------------------------------|----------------------------------------------------------------|---------------------------------------------------------------------|
| <b>Latch inferred</b>           | A combinational block does not assign a variable on every path | Add default assignment or default case unless the latch is intended |

| Message (paraphrased, any tool)               | Meaning                                                 | Action                                                                       |
|-----------------------------------------------|---------------------------------------------------------|------------------------------------------------------------------------------|
| <b>Multiple drivers / conflicting drivers</b> | Two assignments or instances drive the same net         | Always a bug in ASIC RTL (no internal tri-states)                            |
| <b>Net has no driver / undriven input</b>     | Floating wire; tools tie it to 0 or X                   | Connect or tie explicitly                                                    |
| <b>Constant register removed</b>              | Flop input is constant                                  | Check intent; a tied-off mode pin may have been wrong                        |
| <b>Unloaded / unused output, cell removed</b> | Logic with no observable output is deleted              | Usually fine; bad if the logic was a debug or safety feature (dont_touch it) |
| <b>Sensitivity list incomplete</b>            | Simulation and synthesis disagree                       | Use always_comb / always @*                                                  |
| <b>Width mismatch / truncation</b>            | Operand sizes differ                                    | Fix in RTL; lint catches it earlier                                          |
| <b>Timing loop detected / broken</b>          | Combinational loop; the tool disables an arc to time it | Never acceptable unless it is a known false loop, constrained explicitly     |
| <b>Unresolved reference / black box</b>       | Module or macro view missing at link                    | Fix the filelist or library setup; never tape out with black boxes           |

Table 8.5 - Warnings every synthesis engineer must recognise.

## 8.13 Logic equivalence checking after synthesis

Synthesis is a complex optimiser, and synthesis tools have bugs too. Before any netlist is used, **logic equivalence checking (LEC)** formally proves that the netlist implements the RTL. Tools: Synopsys Formality, Cadence Conformal LEC (and open-source Yosys `equiv_*` / `eqy` for small cases).

| reference (RTL)                                                                                                                                                                                                                                                                                              | implementation (netlist)                                                                                                         |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <pre> +-----+   in -&gt; [cone] -&gt; FF_a   &lt;== map ==&gt;   in -&gt; [gates] -&gt; FF_a_reg          [cone] -&gt; FF_b   &lt;== map ==&gt;        [gates] -&gt; FF_b_reg          [cone] -&gt; out    &lt;== map ==&gt;        [gates] -&gt; out        +-----+ </pre>                                  | <pre> +-----+   in -&gt; [gates] -&gt; FF_a_reg          [gates] -&gt; FF_b_reg          [gates] -&gt; out        +-----+ </pre> |
| <ol style="list-style-type: none"> <li>1. map compare points (flops, latches, outputs, black-box pins) by name + structure + function</li> <li>2. prove each pair of logic cones equivalent with SAT/BDD engines</li> <li>3. report: equivalent / non-equivalent (with counter-example) / aborted</li> </ol> |                                                                                                                                  |

Figure 8.6 - Combinational equivalence checking: map state points, then compare the combinational cones between them.

- **Guidance files** (Synopsys SVF, Cadence mapping files) record the transformations synthesis made - register merges, inversions, retiming, FSM re-encoding - so the LEC tool can map points that changed name or polarity.
- **Retiming, clock gating and sequential optimisations** break the one-to-one flop mapping; they need guidance or sequential equivalence checking.
- **Aborted points** (typically large multipliers) are resolved with datapath-aware LEC, partitioning or cut-points - never ignored.
- LEC is run RTL -> synthesised netlist, synthesised -> DFT-inserted netlist (in test-mode-off setup), and netlist -> post-layout netlist, and after every ECO (chapter 19).

### INTERVIEW INSIGHT - LEC VERSUS SIMULATION

LEC proves that two designs are the same; it says nothing about whether either is correct. A bug in the RTL is faithfully reproduced in the netlist and LEC passes. That is why DV signs off the RTL and LEC carries that sign-off to the netlist and layout.

## 8.14 Summary

- A production flow: analyze, elaborate, link, constrain, compile (generic opt, mapping, timing/area/power optimisation), DFT insertion, incremental compile, reports, netlist + SDC + UPF + SCANDEF.
- Logic is optimised as an AIG and mapped by cut enumeration and library matching; timing-driven moves are sizing, buffering, restructuring, pin swapping and multi-Vt assignment.
- Real SKY130 runs: a 32-bit adder spans 1205-1326  $\mu\text{m}^2$  and 1.46-2.25 ns depending on the delay target; a Kogge-Stone structure reaches 1.01 ns at 2.4x the area. Architecture beats optimisation.
- Datapath synthesis picks adder/multiplier architectures and merges arithmetic into carry-save trees; retiming and clock-gating insertion are sequential optimisations that LEC must be told about.
- Wire-load models failed because wire length depends on placement, not fanout; physical-aware synthesis (DC-T/NXT, Fusion Compiler, Genus iSpatial) uses a real floorplan.
- Read reports in order: constraint checks, DRCs, WNS/TNS/levels, area; know the classic warnings; prove every netlist with LEC.

## 8.15 Exercises

- 1 In the add32 sweep, why does a target of 900 ps produce the same netlist as no target at all? What would a commercial tool report for that run?
- 2 Estimate, from Table 8.2, the number of prefix (black) cells in 32-bit Kogge-Stone and Brent-Kung adders. Relate the difference to the ks32 area in the sweep.
- 3 A design has WNS = -0.05 ns and TNS = -350 ns with 12,000 violating endpoints. What does this suggest and what would you try first?
- 4 Explain why a synthesised netlist's timing with wire-load models can be optimistic for some nets and pessimistic for others.
- 5 Your LEC run reports 40 unmapped flops after synthesis with retiming enabled. List the likely causes and fixes.
- 6 Write the SDC design-rule constraints for a library whose tables stop at 1.2 ns input slew and whose largest output load index is 0.25 pF, keeping a 20% margin.

## 9. Static Timing Analysis in Depth

Static timing analysis (STA) proves that every path in the design meets its timing requirements at every operating corner and mode, without simulating any vectors. It is the timing sign-off method for every digital ASIC. STA engines run inside synthesis and place-and-route for optimisation; sign-off STA is run in a dedicated tool - Synopsys **PrimeTime**, Cadence **Tempus**, Siemens/other tools, or the open-source **OpenSTA** used by OpenROAD. Timing sign-off is owned by the **STA / timing sign-off engineer**, with constraints owned jointly with the RTL designers who know the intent.

The companion RTL Design guide introduces setup/hold and SDC from the RTL designer's side. Here we go inside the engine: the timing graph, how a cell delay is computed from Liberty tables, wire models, the full setup and hold equations with clock latency, uncertainty and CRPR, exceptions, latches, on-chip variation models, multi-corner multi-mode analysis, signal integrity and hold fixing. Along the way we compute a real path by hand - in Python, using the actual SKY130 NLDM tables.

### 9.1 The timing graph and timing arcs

STA converts the netlist into a directed acyclic **timing graph**: nodes are pins, edges are **timing arcs**. Arcs come from two sources:

| Arc type                             | From - to                                                                                                                                                                           | Source of its delay / constraint                                            |
|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| <b>Cell arc, combinational</b>       | Input pin -> output pin of a gate (A -> Y of nand2), with a <b>timing sense</b> : positive unate (buffer, AND), negative unate (inverter, NAND, NOR) or non-unate (XOR, mux select) | Liberty <b>cell_rise/cell_fall</b> and <b>rise/fall_transition</b> tables   |
| <b>Cell arc, sequential (launch)</b> | Clock pin -> Q of a flop (timing_type rising_edge), or D -> Q of a transparent latch                                                                                                | Liberty tables, as above                                                    |
| <b>Timing check (constraint) arc</b> | Clock pin -> D pin: setup, hold; recovery/removal on async pins; clock-gating checks; min pulse width                                                                               | Liberty <b>rise/fall_constraint</b> tables (setup_rising, hold_rising...)   |
| <b>Net arc</b>                       | Driver pin -> each load pin                                                                                                                                                         | Wire RC: estimated, or extracted (SPEF) and reduced by the delay calculator |

Table 9.1 - Timing arcs.

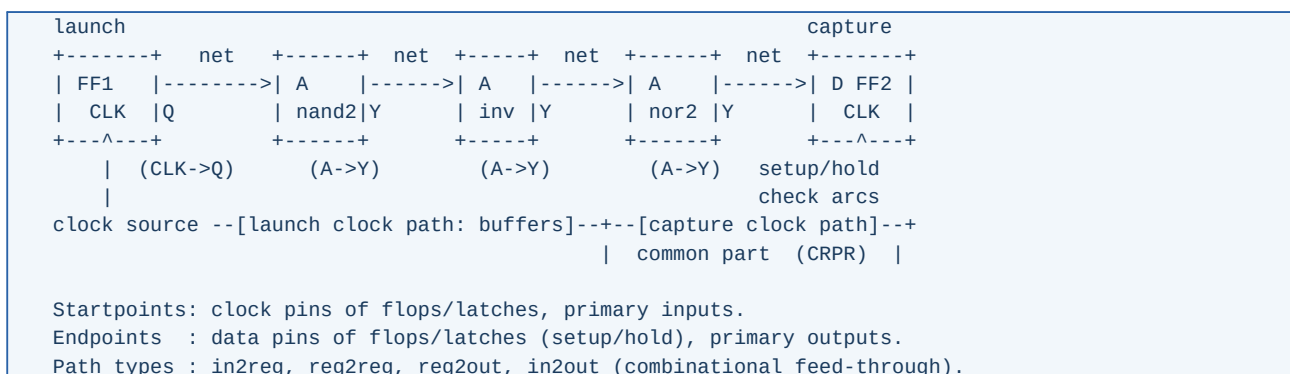


Figure 9.1 - A register-to-register timing path as the STA engine sees it: cell arcs, net arcs, check arcs and the launch/capture clock paths.

Because the graph is acyclic, arrival times are propagated forward in topological order (latest arrival for setup, earliest for hold) and required times backward; slack at each pin is required minus arrival. This is linear in the size of the graph, which is why STA handles billion-gate designs where simulation could not. The price is pessimism: STA assumes every path can be sensitised, which is why **false-path** and **multicycle** exceptions exist.

## 9.2 Delay calculation: NLDM tables, slew propagation and beyond

A cell's delay depends mostly on two quantities: the **input transition time** (slew) and the **output load capacitance**. In the Non-Linear Delay Model (NLDM), the Liberty file stores for each arc a 2-D table of delay and a 2-D table of output transition, indexed by exactly those two variables. Here is the real `cell_fall` table of the SKY130 nand2\_1 arc A -> Y (a negative-unate arc, so a rising A causes a falling Y):

|           |        |        |        |        |        |        |        |
|-----------|--------|--------|--------|--------|--------|--------|--------|
| slew\load | 0.0005 | 0.0013 | 0.0035 | 0.0091 | 0.0240 | 0.0633 | 0.1666 |
| 0.0100    | 0.0206 | 0.0251 | 0.0363 | 0.0652 | 0.1404 | 0.3379 | 0.8628 |
| 0.0231    | 0.0244 | 0.0289 | 0.0403 | 0.0697 | 0.1447 | 0.3426 | 0.8634 |
| 0.0531    | 0.0327 | 0.0384 | 0.0505 | 0.0798 | 0.1552 | 0.3529 | 0.8773 |
| 0.1225    | 0.0428 | 0.0514 | 0.0698 | 0.1039 | 0.1794 | 0.3767 | 0.8969 |
| 0.2823    | 0.0525 | 0.0660 | 0.0938 | 0.1461 | 0.2370 | 0.4342 | 0.9574 |
| 0.6507    | 0.0551 | 0.0755 | 0.1174 | 0.1977 | 0.3329 | 0.5653 | 1.0821 |
| 1.5000    | 0.0332 | 0.0633 | 0.1272 | 0.2474 | 0.4578 | 0.8093 | 1.3854 |

negative\_unate

SKY130 hd tt nand2\_1, pin Y related to A, cell\_fall (ns), rows = input slew (ns), columns = load (pF). Printed by the Python Liberty reader used later in this chapter, rounded to 4 decimals.

Reading the table: with a sharp input (10 ps) and a light load (0.5 fF), the NAND falls in 21 ps; driving 166 fF it takes 863 ps. Delay grows almost linearly with load (the output resistance charging the load) and more weakly with input slew. Delay calculation for an arbitrary point uses **bilinear interpolation** (and extrapolation outside the table - a reason to keep max\_transition and max\_capacitance inside the characterised range):

given  $x_1 \leq x \leq x_2$  (slew) and  $y_1 \leq y \leq y_2$  (load), table values Q11..Q22:

$$f_x = (x - x_1) / (x_2 - x_1), \quad f_y = (y - y_1) / (y_2 - y_1)$$

$$d(x, y) = Q_{11} (1-f_x)(1-f_y) + Q_{21} f_x (1-f_y) + Q_{12} (1-f_x) f_y + Q_{22} f_x f_y$$

Bilinear interpolation of NLDM tables.

**Slew propagation:** each arc also has an output transition table. The output slew of stage k becomes the input slew of stage k+1, so a weak driver early in a path slows every later stage. Pessimistic engines propagate the worst slew among all inputs of a gate ('worst-slew propagation'); more accurate modes propagate the slew belonging to the path being reported ('path-based analysis', PBA, versus graph-based analysis, GBA).

### Beyond NLDM: effective capacitance and current-source models

- **Resistive shielding / effective capacitance.** When a driver sees a wire with significant resistance, the far-end capacitance is partly shielded; the driver behaves as if it drove a smaller **C\_eff**. Delay calculators iterate to find C\_eff that matches the charge delivered in the first part of the transition, and then look up the NLDM table at C\_eff.
- **Current-source models (CCS from Synopsys, ECSM from Cadence)** store the output current waveform versus time for each (slew, load) point instead of a single delay number. The delay calculator drives the actual RC network with that current and gets an accurate waveform. This matters at advanced nodes where waveforms are far from ideal ramps and for noise analysis (CCS noise).
- **Liberty Variation Format (LVF)** adds per-arc sigma tables for statistical OCV (section 9.9).

#### WHERE THE NUMBERS COME FROM

Every delay in STA comes from the Liberty tables that the library team characterised with SPICE at a given PVT corner (chapter 4). STA never simulates transistors; it interpolates. If the tables are wrong or used outside their range, sign-off is wrong - which is why library qualification and extrapolation checks are part of the sign-off methodology.

### 9.3 Wire models: from lumped C to extracted RC

The net between two cells contributes capacitance (loading the driver) and, when long, resistance (adding delay along the wire and degrading slew). The model depends on the stage of the flow:

| Model                                | Used when                                 | Delay                                                                                          |
|--------------------------------------|-------------------------------------------|------------------------------------------------------------------------------------------------|
| <b>Lumped C</b>                      | Short nets; early synthesis               | Only adds C to the driver load; no wire delay                                                  |
| <b>Wire-load model</b>               | Legacy synthesis (chapter 8)              | Statistical R and C by fanout                                                                  |
| <b>Lumped RC / Elmore</b>            | Estimation after placement; hand analysis | Elmore delay: sum over resistors of $R_i$ times downstream C                                   |
| <b>Distributed RC network (SPEF)</b> | Post-route sign-off                       | Reduced order models (e.g. pi-model for driver, AWE/moment matching) for each driver-load pair |

Table 9.2 - Interconnect models.

**Elmore delay to node k in an RC tree:**

$$T_D(k) = \sum \text{over resistors } R_i \text{ on the path from the driver to } k \text{ of} \\ R_i \times (\text{total capacitance downstream of } R_i)$$

**uniform wire, length L, r and c per unit length:  $T_{\text{Elmore}} = 0.5 r c L^2$**   
**(the 50% point of the distributed line's step response is  $\sim 0.38 r c L^2$ )**

Elmore delay. The quadratic dependence on length is why long wires need repeaters (chapter 17).

```
*SPEF "IEEE 1481-1998"                                // illustrative structure only
*T_UNIT 1 NS *C_UNIT 1 FF *R_UNIT 1 OHM
*D_NET n42 5.23                                         // net n42, total cap 5.23 fF
*CONN
*I u1:Y 0 *L 0 *D sky130_fd_sc_hd__nand2_1           // driver pin
*I u2:A I *L 2.3                                       // load pin, pin cap
*CAP
1 n42:1 0.91
2 n42:2 1.12
3 n42:1 n77:3 0.20                                     // coupling cap to aggressor net n77
*RES
1 u1:Y n42:1 12.5
2 n42:1 n42:2 38.0
3 n42:2 u2:A 9.1
*END
```

Structure of a SPEF (Standard Parasitic Exchange Format, IEEE 1481) net from an extraction tool: ground caps, coupling caps and resistors forming the RC network (illustrative values).

### 9.4 Setup and hold: the full equations

For a path launched by clock edge at time 0 at FF1 and captured by the next edge (period T) at FF2, with clock insertion delays (latencies)  $L_{\text{launch}}$  and  $L_{\text{capture}}$  from the clock source to each flop's clock pin:

```
arrival A = L launch + t clk->q + t logic + t wire
```

**SETUP (max delays):**

$$\text{required } R_s = T + L_{\text{capture}} - t_{\text{setup}} - U_{\text{setup}} + CRPR$$
$$\text{slack } s = R s - A \max \geq 0$$

**HOLD** (min delays, same clock edge):

required  $R_h = L_{\text{capture}} + t_{\text{hold}} + U_{\text{hold}} - CRPR$

$$\text{slack } h = A_{\min} - R_h \geq 0$$
$$\text{skew} = L_{\text{capture}} - L_{\text{launch}} \quad (\text{positive skew helps setup, hurts hold})$$

Setup and hold equations. U = clock uncertainty (jitter + margin; before CTS also estimated skew). CRPR = clock reconvergence pessimism removal (below).

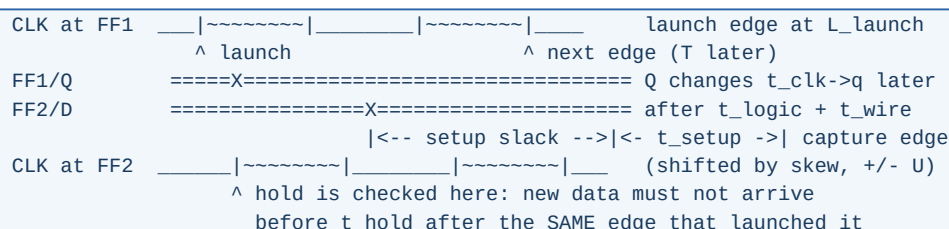


Figure 9.2 - Setup is checked against the next capture edge, hold against the same edge.

## CRPR - clock reconvergence pessimism removal

With on-chip variation, STA uses the **late** delay for the launch clock path and the **early** delay for the capture clock path in a setup check (and the reverse for hold). But the launch and capture clock paths share a common segment from the clock source to the point where they diverge. A physical buffer cannot be both slow and fast at the same moment, so the difference between its late and early delay on the common segment is pure pessimism. **CRPR** (also called CPPR) adds that difference back as a credit. It is significant: on a deep clock tree with derates, CRPR can be tens of picoseconds. For hold checks, where launch and capture use the same edge, the full common-path difference is removed; for setup (different edges), jitter-related components are not removed.

## 9.5 A worked example: STA of a real SKY130 path in Python

To make the equations concrete we time the path of Figure 9.1 - FF1 (dfxtp\_1) -> nand2\_1 -> inv\_1 -> nor2\_1 -> FF2 (dfxtp\_1) - using the real NLDM tables from the SKY130 tt Liberty file. The script parses the Liberty groups it needs, computes every load as pin capacitance of the next gate plus 2 fF of wire, propagates slew stage by stage, looks up the setup and hold constraint tables of the capturing flop (indexed by clock slew and data slew), and applies the equations above with a 1 ns clock, 0.30/0.35 ns launch/capture latency (50 ps of useful skew), and 80/30 ps setup/hold uncertainty. Both edges are analysed because the rise and fall delays differ.

```
# sta3.py - tiny NLDL static timing analysis of one register-to-register path
import re, bisect
LIB = open("hd_tt.lib").read()

def group(text, head):
    # return body of "head { ... }"
    i = text.index(head); j = text.index("{", i) + 1; d = 1; k = j
    while d:
        d += {"{": 1, "}": -1}.get(text[k], 0); k += 1
```

```

    return text[j:k - 1]

def nums(s): return [float(x) for x in re.findall(r"-?[\d.]+(?:e-?\d+)?", s)]

def table(body, kind):
    # index_1, index_2, values of one table
    t = group(body, kind + " (")
    ix = re.findall(r'index_\d\("[^"]*\)"', t)
    v = nums(re.search(r"values\((.*?)\);", t, re.S).group(1))
    i1 = nums(ix[0]); i2 = nums(ix[1]) if len(ix) > 1 else [0.0]
    return i1, i2, [v[r * len(i2):(r + 1) * len(i2)] for r in range(len(i1))]

def lookup(tab, x, y):
    # bilinear inter/extrapolation
    i1, i2, v = tab
    def seg(ax, a):
        k = min(max(bisect.bisect(ax, a) - 1, 0), len(ax) - 2)
        return k, (a - ax[k]) / (ax[k + 1] - ax[k])
    r, fx = seg(i1, x); c, fy = seg(i2, y)
    return (v[r][c] * (1 - fx) * (1 - fy) + v[r + 1][c] * fx * (1 - fy)
            + v[r][c + 1] * (1 - fx) * fy + v[r + 1][c + 1] * fx * fy)

def arc(cell, pin, rel, ttype=None):
    # the timing() group for pin<-rel
    body = group(group(LIB, 'cell ("sky130_fd_sc_hd_%s")' % cell), 'pin ("%s")' % pin)
    for t in re.split(r"\btiming \(\)", body)[1:]:
        if 'related_pin : "%s"' % rel in t and (ttype is None or ttype in t):
            return t

def cap(cell, pin):
    body = group(group(LIB, 'cell ("sky130_fd_sc_hd_%s")' % cell), 'pin ("%s")' % pin)
    return float(re.search(r"\bcapacitance : ([\d.]+)", body).group(1))

# ---- the path: FF1/CLK -> Q -> nand2_1 -> inv_1 -> nor2_1 -> FF2/D -----
T, LAT_L, LAT_C, UNC_S, UNC_H = 1.00, 0.30, 0.35, 0.08, 0.03 # ns: period, latencies, unc.
CLK_SLEW, CWIRE = 0.10, 0.002 # ns, pF (2 fF per net)
stages = [("dfxtp_1", "CLK", "Q", "nand2_1", "A"), ("nand2_1", "A", "Y", "inv_1", "A"),
           ("inv_1", "A", "Y", "nor2_1", "A"), ("nor2_1", "A", "Y", "dfxtp_1", "D")]
INV = {"rise": "fall", "fall": "rise"}

def run(q_edge):
    t, slew, edge = LAT_L, CLK_SLEW, None
    print("%-22s %6s %6s %7s %7s %s" % ("point", "load", "slew", "incr", "path", "edge"))
    print("%-22s %6s %6.3f %7.3f %7.3f %s"
          % ("clock CLK (launch)", "", CLK_SLEW, LAT_L, t, "r"))
    for cell, i, o, nxt, npin in stages:
        edge = q_edge if edge is None else (edge if cell.startswith("buf") else INV[edge])
        load = cap(nxt, npin) + CWIRE
        a = arc(cell, o, i)
        d = lookup(table(a, "cell_" + edge), slew, load)
        slew = lookup(table(a, edge + "_transition"), slew, load)
        t += d
        print("%-22s %6.4f %6.3f %7.3f %7.3f %s"
              % (cell + "/" + o, load, slew, d, t, edge[0]))
    su = lookup(table(arc("dfxtp_1", "D", "CLK", "setup_rising"), edge + "_constraint"),
                CLK_SLEW, slew)
    ho = lookup(table(arc("dfxtp_1", "D", "CLK", "hold_rising"), edge + "_constraint"),
                CLK_SLEW, slew)
    req = T + LAT_C - UNC_S - su
    print("data arrival time %.3f | required = %.2f + %.2f - %.2f - setup %.3f = %.3f"
          % (t, T, LAT_C, UNC_S, su, req))
    print("SETUP slack (%s at D) = %+.3f ns %s"
          % (edge, req - t, "MET" if req >= t else "VIOLATED"))
    hreq = LAT_C + UNC_H + ho

```



```

print("HOLD slack          = %.3f - (%.2f + %.2f + hold %.3f) = %+.3f ns\n"
      % (t, LAT_C, UNC_H, ho, t - hreq))
return req - t

w = min(run("rise"), run("fall"))
print("worst setup slack %.3f ns -> max frequency ~ %.0f MHz" % (w, 1000 / (T - w)))

```

sta3.py - a complete (if tiny) NLDM timing engine: Liberty reader, bilinear lookup, slew propagation, setup and hold checks.

```

point          load  slew  incr  path  edge
clock CLK (launch)          0.100  0.300  0.300  r
dfxtp_1/Q      0.0043  0.055  0.330  0.630  r
nand2_1/Y      0.0043  0.042  0.055  0.686  f
inv_1/Y        0.0044  0.047  0.056  0.742  r
nor2_1/Y       0.0037  0.029  0.044  0.786  f
data arrival time 0.786 | required = 1.00 + 0.35 - 0.08 - setup 0.091 = 1.179
SETUP slack (fall at D) = +0.393 ns  MET
HOLD slack      = 0.786 - (0.35 + 0.03 + hold -0.034) = +0.440 ns

point          load  slew  incr  path  edge
clock CLK (launch)          0.100  0.300  0.300  r
dfxtp_1/Q      0.0043  0.034  0.318  0.618  f
nand2_1/Y      0.0043  0.051  0.059  0.677  r
inv_1/Y        0.0044  0.028  0.043  0.720  f
nor2_1/Y       0.0037  0.093  0.101  0.821  r
data arrival time 0.821 | required = 1.00 + 0.35 - 0.08 - setup 0.057 = 1.213
SETUP slack (rise at D) = +0.392 ns  MET
HOLD slack      = 0.821 - (0.35 + 0.03 + hold -0.034) = +0.475 ns

worst setup slack 0.392 ns -> max frequency ~ 1645 MHz

```

Real output of sta3.py against the SKY130 hd tt\_025C\_1v80 Liberty file.

Points worth noticing, all from real library data:

- **Clock-to-Q dominates** this short path: 318-330 ps of a total logic delay of about 490-520 ps. In real designs clock-to-Q plus setup is a fixed tax of every pipeline stage (chapter 6).
- **Rise and fall differ.** The nor2\_1 output **rise** takes 101 ps with a 93 ps slew, versus 44 ps / 29 ps for its fall: a NOR pulls up through two series PMOS transistors (weak holes, stacked). Libraries and synthesis tools prefer NAND-based logic for exactly this reason.
- **The setup constraint depends on the data edge and slews:** 91 ps for a falling D, 57 ps for a rising D.
- **Negative hold time** (-34 ps) is normal for flip-flops whose internal clock path delays capture; it makes hold easier to meet.
- **The useful skew** (capture clock 50 ps later) added 50 ps of setup slack and took 50 ps from hold slack - the trade-off CTS exploits deliberately (chapter 16).
- Real sign-off adds what the script leaves out: wire resistance (Elmore/C<sub>eff</sub>), multiple fan-outs, OCV derates, CRPR, SI delta-delays and every other corner.

## 9.6 Reading a timing report line by line

A sign-off report has exactly the structure the script printed. The excerpt below is in PrimeTime's `report_timing` format with generic numbers:

```

# ---- ILLUSTRATIVE report_timing excerpt (generic numbers, not from a real run) ----
Startpoint: u_core/u_alu/res_reg_12_ (rising edge-triggered flip-flop
      clocked by core_clk)
Endpoint: u_core/u_wb/data_reg_3_ (rising edge-triggered flip-flop clocked
      by core_clk)
Path Group: core_clk      Path Type: max      Scenario: func_ss0p72v_125c

Point          Fanout  Cap  Trans  Incr  Path
clock core_clk (rise edge)          0.000  0.000

```

```

clock network delay (propagated)                                0.412  0.412
u_core/u_alu/res_reg_12_/CK (DFFQ_X1)                        0.045  0.000  0.412 r
u_core/u_alu/res_reg_12_/Q (DFFQ_X1)                        2    3.1  0.038  0.091  0.503 f
u_core/u_alu/U3021/ZN (NAND2_X2)                            1    1.9  0.027  0.024 & 0.527 r
... (14 more stages)
u_core/u_wb/data_reg_3_/D (DFFQ_X1)                        0.041  0.006 & 0.981 f
data arrival time                                           0.981

clock core_clk (rise edge)                                  1.000  1.000
clock network delay (propagated)                            0.398  1.398
clock reconvergence pessimism                               0.012  1.410
clock uncertainty                                           -0.030  1.380
library setup time                                          -0.035  1.345
data required time                                          1.345
slack (MET)                                                 0.364

```

Anatomy of a PrimeTime path report (illustrative). '&' marks increments that include annotated parasitics; r/f = rise/fall.

| Report item                                 | What to check                                                                                                  |
|---------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| <b>Startpoint / endpoint / clock</b>        | Are these the clocks you expect? A path between asynchronous clocks means a missing clock group                |
| <b>Path type max/min, scenario</b>          | Setup vs hold, and which corner/mode                                                                           |
| <b>Clock network delay ideal/propagated</b> | Pre-CTS reports use ideal clocks; post-CTS must be propagated                                                  |
| <b>Trans and Cap columns</b>                | Large transitions point to weak drivers or long wires; compare with max_transition                             |
| <b>Incr per stage</b>                       | Which stage dominates: a single big increment is a sizing/buffering fix, many small ones a logic-depth problem |
| <b>CRPR, uncertainty, library setup</b>     | Plausible values for the stage of the flow                                                                     |
| <b>Slack</b>                                | Sign and magnitude; compare GBA vs PBA for pessimism                                                           |

Table 9.3 - How an STA engineer reads a path.

## 9.7 Constraints: clocks, I/O, exceptions

STA is only as good as its constraints. The Synopsys Design Constraints (SDC) format is the industry standard, read by synthesis, P&R and sign-off tools alike.

```

# primary and generated clocks
create_clock -name core_clk -period 1.25 [get_ports clk_core]      ;# 800 MHz
create_clock -name ref_clk -period 40.0 [get_ports clk_ref]       ;# 25 MHz
create_generated_clock -name core_div2 -source [get_ports clk_core] \
    -divide_by 2 [get_pins u_crg/u_div2/q_reg/Q]
set_clock_groups -asynchronous -group {core_clk core_div2} -group {ref_clk}
set_clock_uncertainty -setup 0.05 [get_clocks core_clk]          ;# post-CTS: jitter+margin
set_clock_latency -source 0.2 [get_clocks core_clk]               ;# off-chip / PLL latency
# I/O budgets relative to the board / neighbour block
set_input_delay -max 0.50 -clock core_clk [get_ports din*]
set_input_delay -min 0.10 -clock core_clk [get_ports din*]
set_output_delay -max 0.40 -clock core_clk [get_ports dout*]
# exceptions (each one needs a written justification)
set_false_path -from [get_ports test_mode]                        ;# static in functional mode
set_multicycle_path -setup 2 -from [get_cells u_mac/acc*] -to [get_cells u_out/r*]
set_multicycle_path -hold 1 -from [get_cells u_mac/acc*] -to [get_cells u_out/r*]
set_max_delay 1.0 -datapath_only -from [get_cells u_fifo/wptra_gray*] \
    -to [get_cells u_fifo/u_sync_w/meta*]                        ;# CDC bus skew bound
set_case_analysis 0 [get_ports scan_mode]                        ;# functional mode

```

A representative SDC. The multicycle pair is the classic idiom: move setup to the 2nd edge, then move hold back by 1 so it is checked at the original launch edge.

**PITFALL - MULTICYCLE WITHOUT THE HOLD ADJUSTMENT**

`set_multicycle_path -setup 2` alone also moves the default hold check one cycle later, so the tool now demands that data be held for a whole cycle - producing hundreds of hold 'violations' that P&R will 'fix' by inserting large delay chains. Always pair it with `-hold 1` (in the usual same-clock case) and verify the RTL really holds the source stable for two cycles.

| Exception                                           | Meaning                                                                                      | Risk                                                    |
|-----------------------------------------------------|----------------------------------------------------------------------------------------------|---------------------------------------------------------|
| <code>set_false_path</code>                         | Path is never functionally exercised (static config, mutually exclusive modes, async groups) | If wrong, a real path goes untimed - silicon failure    |
| <code>set_multicycle_path</code>                    | Data is allowed N cycles (enable-based slow logic)                                           | RTL must guarantee it; verify with assertions           |
| <code>set_max_delay / min_delay</code>              | Point-to-point bounds (CDC buses, asynchronous interfaces)                                   | -datapath_only ignores clock latency; use deliberately  |
| <code>set_clock_groups</code>                       | Clocks are asynchronous/exclusive                                                            | Wrong grouping hides real synchronous paths             |
| <code>set_case_analysis / set_disable_timing</code> | Constant values or disabled arcs define a mode                                               | Mode-specific; must match the chip's real configuration |

Table 9.4 - Timing exceptions.

## 9.8 Latches and time borrowing

A positive level-sensitive latch is transparent while its clock is high. If data arrives at a latch **after** the opening edge but before the closing edge (minus setup), it passes through and the path simply continues into the next stage: the latch **borrow**s time from the next stage. STA handles this by checking the arrival against the closing edge and passing the borrowed amount to the next path.

```

latch opens at t_open, closes at t_close (= t_open + high phase)
if A <= t_open                : no borrowing, next path starts at t_open
if t_open < A <= t_close - t_su : borrow = A - t_open, next path starts at A
if A > t_close - t_su          : setup violation at the latch
max borrow is often further limited by set_max_time_borrow

```

Time borrowing through a transparent latch.

Latch-based design (two-phase clocking, pulsed latches) tolerates unbalanced stages and clock skew better than flop-based design and is used in high-performance CPU datapaths and some register files; the cost is harder hold analysis, more complex DFT and a much more careful constraint set.

## 9.9 On-chip variation: derates, AOCV, POCV/SOCV and LVF

Transistors on the same die differ: random dopant fluctuation, line-edge roughness, local voltage drop and temperature gradients (chapters 2-3). A corner library describes the **global** process corner; **on-chip variation (OCV)** models the local spread around it. Four generations of models:

| Model                      | How it works                                                                                                                                     | Weakness                                                                                          |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| <b>Flat OCV derate</b>     | Multiply all late delays by, e.g., 1.05-1.10 and early delays by 0.90-0.95                                                                       | Over-pessimistic on long paths (random variation partly cancels), may be optimistic on short ones |
| <b>AOCV (advanced OCV)</b> | Derate from a table indexed by <b>path depth</b> (and distance/bounding box): deep paths get small derates because random variation averages out | Table-based approximation; tables per cell type                                                   |

| Model                                             | How it works                                                                                                          | Weakness                                              |
|---------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------|
| <b>POCV / SOCV (parametric / statistical OCV)</b> | Each arc has a mean and a sigma; sigmas add in quadrature along a path; path delay = mean + n x sigma (n typically 3) | Needs per-arc sigma data                              |
| <b>LVF (Liberty Variation Format)</b>             | Liberty tables of sigma per arc, per slew/load point (and for constraints), used by POCV                              | Characterisation cost; now standard at advanced nodes |

Table 9.5 - Evolution of OCV modelling.

```

flat OCV      : D_late = 1.08 x sum(d_i)           (example derate)
POCV (path)   : mu_path = sum(mu_i),  sigma_path = sqrt( sum(sigma_i^2) )
                D_late  = mu_path + 3 sigma_path

N identical stages, each mu, sigma: D = N mu + 3 sqrt(N) sigma
-> relative margin 3 sigma / (sqrt(N) mu) shrinks as 1/sqrt(N)

```

Why statistical OCV is less pessimistic on deep paths (independent random variation assumed).

## 9.10 Multi-corner multi-mode (MCMM) analysis

A chip must work at every combination of process, voltage and temperature (PVT) and in every functional and test **mode**. Each combination with its RC extraction corner is a **scenario**. Setup is usually worst at slow process, low voltage, and high temperature - except that at advanced nodes **temperature inversion** can make low temperature the slow corner at low voltage. Hold is usually worst at fast process, high voltage, with the minimum-RC extraction.

| Scenario                           | Process / V / T                   | RC corner       | Checks                                   |
|------------------------------------|-----------------------------------|-----------------|------------------------------------------|
| <b>func_setup_ss</b>               | SS / Vnom-10% / 125 C (and -40 C) | Cworst, RCworst | Setup, max transition/cap                |
| <b>func_hold_ff</b>                | FF / Vnom+10% / -40 C and 125 C   | Cbest, RCbest   | Hold                                     |
| <b>func_hold_ss</b>                | SS / low V                        | Cworst          | Hold (clock skew grows at slow corners)  |
| <b>func_typ</b>                    | TT / Vnom / 25-85 C               | typical         | Power analysis, correlation              |
| <b>scan_shift</b>                  | SS and FF                         | worst/best      | Shift at test frequency: mostly hold     |
| <b>scan_capture_atspeed</b>        | SS / FF                           | worst/best      | At-speed capture paths with OCC clocks   |
| <b>low_power_mode (DVFS point)</b> | SS / reduced V                    | Cworst          | Setup at the lower frequency and voltage |

Table 9.6 - A typical MCMM scenario table (voltages and temperatures are examples; the real list comes from the foundry sign-off guide and the product spec).

Modern tools analyse all scenarios concurrently. A mid-size SoC at an advanced node commonly has tens of scenarios; the implementation flow optimises with a representative subset and sign-off runs them all.

## 9.11 Signal integrity: crosstalk delay and glitch

Adjacent wires are coupled by sidewall capacitance, which at advanced nodes is a large share of total wire capacitance. When a neighbour (**aggressor**) switches at the same time as the **victim** net:

- **Opposite direction:** the effective coupling capacitance is up to doubled (Miller effect) - the victim slows down: a positive **delta delay** hurting setup.
- **Same direction:** the victim speeds up - negative delta delay hurting hold.

- **Quiet victim:** the aggressor injects a **glitch**. If the glitch is large and wide enough to propagate through the receiving gate and reach a flop or a clock/reset pin, it can flip state - a functional failure with no timing path involved.

SI-aware STA (PrimeTime SI, Tempus SI) uses coupling capacitances from SPEF and **timing windows**: an aggressor can only affect a victim if their switching windows overlap. Fixes are routing-level: spacing, shielding (especially clocks), layer changes, upsizing the victim driver, or downsizing aggressors (chapter 17).

## 9.12 Hold fixing and timing ECOs

Setup violations are fixed by making paths faster; hold violations by making them **slower** - inserting delay cells or buffers on the data path near the capturing flop, or using slower (high-Vt, smaller) cells. Rules of thumb:

- Fix hold **after CTS**, with propagated clocks: before CTS the skew is unknown and hold fixes are guesses.
- Fix hold at the **fast corner** while checking that the inserted delay does not break **setup at the slow corner** on the same path. Paths that fail both are a clock-tree (skew) problem.
- Prefer inserting delay at the endpoint side so the fix does not disturb other paths sharing the start of the path.
- Very large hold violations usually mean wrong constraints (missing clock groups, a multicycle path without -hold) or huge skew between clocks - fix the cause, not the symptom.

After sign-off STA, remaining violations are fixed by **timing ECOs**: the sign-off tool (PrimeTime ECO, Tempus ECO) proposes cell sizing, Vt swaps, buffer insertion and hold buffers that fit in the existing placement, writes a change list, and the P&R tool implements it with minimal disturbance (chapter 19). Late ECOs prefer **metal-only** changes using pre-placed spare cells.

## 9.13 Constraint quality checks

### SDC / STA setup quality checklist

- ☐ check\_timing clean: no unconstrained endpoints, no unlocked registers, no missing input/output delays, no combinational loops.
- ☐ Every clock has a defined source, period and waveform; generated clocks point to the right master and pin; clock relationships (groups) reviewed with the clocking architect.
- ☐ Every exception has a written reason and owner; false paths verified with CDC tool results or formal; wildcards reviewed.
- ☐ No 'ignored' or 'partially applied' constraints in the tool reports (objects renamed or removed by synthesis).
- ☐ I/O delays derived from a documented budget with the neighbouring block or board; not placeholder 0 or 50% values.
- ☐ set\_case\_analysis matches each mode; all modes and corners from the sign-off plan are present as scenarios.
- ☐ Design rule limits (max transition/capacitance) set within library table ranges; no extrapolation warnings.
- ☐ Correlation: synthesis vs P&R vs sign-off timing within the agreed margin on a reference set of paths.

## 9.14 Summary

- STA builds a timing graph of cell, net and check arcs, propagates arrival and required times, and proves every path meets setup and hold without vectors.
- Cell delays and slews come from Liberty NLDL tables indexed by input slew and load, interpolated bilinearly; CCS/ECSM and effective capacitance improve accuracy with resistive wires.
- Setup:  $L_{\text{launch}} + t_{\text{cq}} + t_{\text{logic}} \leq T + L_{\text{capture}} - t_{\text{setup}} - U (+ \text{CRPR})$ ; hold:  $L_{\text{launch}} + t_{\text{cq}} + t_{\text{logic}} \geq L_{\text{capture}} + t_{\text{hold}} + U (- \text{CRPR})$ .
- Our Python STA on real SKY130 tables found 0.39 ns setup slack at 1 GHz for a 3-gate path, with clock-to-Q the largest single delay and NOR rise transitions the slowest gate edge.

- Constraints (clocks, I/O delays, exceptions, modes) determine correctness; every exception is a risk that needs justification.
- OCV evolved from flat derates to AOCV and POCV/LVF; sign-off runs many MCMM scenarios, includes SI delta-delay and glitch, and closes remaining violations with timing ECOs.

## 9.15 Exercises

---

- 1 Using the nand2\_1 cell\_fall table, interpolate the delay for an input slew of 0.08 ns and a load of 0.005 pF. Check your result by running `lookup()` from sta3.py.
- 2 Modify sta3.py to give nor2\_1 a 20 fF load (a long wire). How do the nor2 delay, the D slew and the setup time change? Why?
- 3 Derive the hold equation for a path where launch and capture flops are clocked by opposite edges of the same clock.
- 4 A path has 20 identical stages with  $\mu = 30$  ps and  $\sigma = 3$  ps. Compare the late delay with a flat 8% derate and with POCV at 3 sigma.
- 5 Explain why `set_multicycle_path -setup 2` must usually be paired with `-hold 1`. Draw the default and adjusted check edges.
- 6 List four constraint errors that make STA optimistic (silicon fails although STA passes).

## 10. Design for Test: Scan, ATPG, Compression, BIST and Test Access

Functional verification (chapter 7) asks whether the **design** is correct. Manufacturing test asks a different question for every single die: **was this particular chip built correctly?** A wafer contains random defects - particles, shorts, opens, voids, resistive vias - and a fraction of the dies are faulty. Test must find them quickly and cheaply on automatic test equipment (ATE), because every defective part that ships becomes a field return. Design for test (DFT) is the set of structures added to the chip to make that possible: scan chains, test compression, on-chip clock controllers, memory and logic BIST, and standard test access ports.

DFT is owned by the **DFT engineer / DFT team**: they write the DFT specification with the architects, insert the structures (usually during or right after synthesis), generate test patterns with ATPG, sign off coverage, and hand patterns to the **test/product engineering** team who run them on the ATE (chapter 21). The companion RTL Design guide shows scan and BIST from the RTL designer's perspective; this chapter covers the full DFT discipline.

### 10.1 Why test: defects, faults and quality

A **defect** is a physical imperfection. A **fault** is a logical model of the defect's effect that a tool can reason about. A **failure** is the observable wrong behaviour. Test quality is measured in defective parts per million shipped (**DPPM**). The classic Williams-Brown model relates DPPM to yield Y and fault coverage T:

$$\text{defect level } DL = 1 - Y^{(1 - T)}$$

$$\text{example: } Y = 0.80, \quad T = 0.99 \quad \rightarrow \quad DL = 1 - 0.8^{0.01} = 0.00223 = 2230 \text{ DPPM}$$

$$Y = 0.80, \quad T = 0.999 \quad \rightarrow \quad DL = 1 - 0.8^{0.001} = 0.000223 = 223 \text{ DPPM}$$

Williams-Brown (1981). Automotive customers typically ask for single-digit DPPM or 'zero defects' programmes, which is why their test coverage targets are so high.

Every tenth of a percent of coverage matters at the high end, and stuck-at coverage alone is not enough: many real defects in modern processes are timing-related (resistive opens and vias) and only show at speed. Hence the portfolio of fault models below.

### 10.2 Fault models

| Fault model                           | Models                                                                                         | Test requirement                                                                              |
|---------------------------------------|------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| <b>Stuck-at (SA0/SA1)</b>             | A net permanently at 0 or 1 (shorts to supply, many opens)                                     | Activate the opposite value, propagate to an observable point; the workhorse, static patterns |
| <b>Transition (slow-to-rise/fall)</b> | A node that switches too slowly (resistive defects)                                            | Two-pattern test: launch a transition, capture one functional clock period later (at speed)   |
| <b>Path delay</b>                     | Accumulated delay along a specific path exceeds the period                                     | Sensitise a whole path; used for the critical paths from STA                                  |
| <b>Bridging</b>                       | Two nets shorted (wired-AND, wired-OR, dominant)                                               | Drive the two nets to opposite values; uses layout-extracted neighbour pairs                  |
| <b>Cell-aware (CAT)</b>               | Defects <b>inside</b> standard cells (transistor opens/shorts) characterised by SPICE per cell | Cell-specific input combinations; catches defects that SA/transition miss                     |
| <b>IDDQ</b>                           | Defects that cause elevated quiescent supply current                                           | Measure current in a static state; less useful at nodes with high leakage                     |
| <b>Memory faults</b>                  | Stuck cells, transition, coupling, address decoder, retention                                  | March algorithms in MBIST (section 10.9)                                                      |



Table 10.1 - Fault models used in production test.

## 10.3 Scan design

The fundamental problem of testing sequential logic is that the internal state is neither controllable nor observable from the pins. **Scan** solves it by replacing every flip-flop with a **scan flip-flop** that has a second data input, and connecting them into **shift registers (scan chains)** in test mode. Then any state can be shifted in, one functional clock applied, and the captured state shifted out: sequential test reduces to combinational test of the logic between flops.

```
Mux-D scan flip-flop (e.g. SKY130 sdfxtp: pins D, SCD = SI, SCE = SE, CLK, Q)
      +-----+
D  -----|0   |      +-----+
      | MUX |----->| D   Q |-----> Q (to functional logic)
SI  -----|1   |      | FF  |      +---> S0 (to SI of the next scan cell)
      +---+---+      |>CK  |
SE  -----+      +-----+

Scan chain: scan_in -> [SFF] -> [SFF] -> [SFF] -> ... -> [SFF] -> scan_out
SE = 1 (shift): the chain is a shift register, one bit per clock
SE = 0 (capture): every SFF loads its functional D (the logic's response)
```

Figure 10.1 - The mux-D scan cell and a scan chain. SE selects shift (SI) or capture (D).

The **mux-D** style is used in almost all ASICs. The alternative, **LSSD** (level-sensitive scan design, from IBM), uses two non-overlapping scan clocks and latches (master/slave) and is robust against hold problems but needs more clock routing; it survives in some high-performance and latch-based designs. The cost of scan in SKY130 is visible directly in the Liberty file:

| SKY130 hd cell      | Function                    | Area (um <sup>2</sup> ) | vs dfxtp_1 |
|---------------------|-----------------------------|-------------------------|------------|
| dfxtp_1             | D flip-flop                 | 20.02                   | -          |
| sdfxtp_1            | mux-D scan flip-flop        | 26.28                   | +31%       |
| dfrtp_1             | D flip-flop, async reset    | 25.02                   | +25%       |
| sdftrtp_1           | scan flip-flop, async reset | 31.28                   | +56%       |
| dlclkp_1 / sdclkp_1 | ICG / ICG with scan enable  | 17.52 / 18.77           | -          |

Table 10.2 - Scan overhead read from the real SKY130 tt Liberty file (area attribute).

With flops often making up roughly 30-50% of logic area in SoC blocks, scan replacement alone costs on the order of 5-15% of standard-cell area in this library, plus the scan-enable tree and a small delay (the mux) on every flop's D path.

## 10.4 Scan chains, shift and capture

```
SE  ~~~~~
CLK  _|_|_|_|_|_ ... _|_|_|_|_|_
      <--- shift L cycles (slow) ---> capture <--- shift out L --->
      load pattern k                (1 pulse,  unload response k while
                                   or 2 at-  loading pattern k+1
                                   speed)
```

Figure 10.2 - Scan test protocol. Unload of pattern k overlaps with load of pattern k+1, so each pattern costs about L shift cycles plus capture.

- **Chain count and length** trade tester pins against test time: N scan cells split into c chains need about N/c shift cycles per pattern. Chains are balanced to equal length.
- **Shift frequency** is limited by scan-path timing and by shift power (section 10.14) - typically tens of MHz, well below functional speed.
- **Chain ordering** is done by the P&R tool after placement (scan reordering using SCANDEF) to minimise wiring; the netlist order from synthesis is only a starting point.



- **Lockup latches** are inserted where a chain crosses from one clock domain or clock-tree branch to another, to avoid hold violations in shift.
- **Scan-enable** is a high-fanout net treated almost like a clock: buffered, and sometimes pipelined for at-speed tests.

## 10.5 At-speed test: launch-on-capture, launch-on-shift and OCCs

Transition and path-delay faults need two patterns: the first sets up the initial value, the second **launches** a transition, and the result is **captured** exactly one functional period later. Two launch styles exist:

| Style                                       | How the launch happens                                                                                                    | Pros                                                      | Cons                                                                                     |
|---------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|------------------------------------------------------------------------------------------|
| <b>Launch-on-capture (LOC, 'broadside')</b> | Shift slowly, then two fast functional clock pulses: the first launches from the functional response, the second captures | SE can be slow (not timed at speed); standard in industry | Launch values restricted to functional responses: more patterns, somewhat lower coverage |
| <b>Launch-on-shift (LOS, 'skewed-load')</b> | The last shift pulse launches; SE drops within one fast cycle; next pulse captures                                        | Higher coverage, fewer patterns                           | SE must switch at speed: timing-critical global net; may test non-functional paths       |

Table 10.3 - At-speed launch styles.

Testers cannot deliver GHz clocks with exact two-pulse bursts to every domain. The chip uses its own PLL, and an **on-chip clock controller (OCC)** per clock domain chooses between the slow shift clock from the tester and a burst of exactly N at-speed PLL pulses for capture, triggered by the tester.

```

+-----+
PLL clk --->| sync + pulse counter (programmable 1..N pulses)|
ATE shift ->| glitch-free clock mux                        |---> domain clock
scan_en --->| control: shift -> slow clk, capture -> burst  |   (to CTS root)
test_mode ->| functional mode: PLL clock straight through  |
+-----+

```

Figure 10.3 - An OCC sits at the root of each clock domain; it is part of the clock architecture and must be planned with the clocking and CTS teams.

## 10.6 ATPG: automatic test pattern generation

For each fault, ATPG must find an input pattern (scan state plus primary inputs) that **activates** the fault (drives the faulty net to the opposite of the stuck value) and **propagates** the difference to an observable scan cell or output. Classic algorithms:

- **D-algorithm** (Roth, 1966): uses the 5-valued algebra {0, 1, X, D, D-bar}, where D means '1 in the good circuit, 0 in the faulty one'. It drives D forward (D-frontier) and justifies required values backward, backtracking on conflicts.
- **PODEM** (Goel, 1981): searches only over **primary input** assignments, simulating forward after each decision - a much smaller search space; the basis of most later algorithms (FAN, SOCRATES), now combined with SAT solvers.
- **Fault simulation** does the bulk of the work: every generated pattern is simulated against all remaining faults, and every fault it happens to detect is dropped. Random patterns first detect the easy faults cheaply; deterministic ATPG targets the rest.
- **Compaction**: static (merge compatible patterns) and dynamic (fill don't-care bits of a pattern to target more faults) keep the pattern count low.

The Python model below does all of this on two small circuits: ISCAS-85 **c17** (6 NAND gates) and a function with a redundant consensus term,  $f = ab + a'c + bc$ . It enumerates single stuck-at faults on every net (stems only, for brevity - real tools also model fanout branches and collapse equivalent faults), applies random patterns with fault dropping, then tops up with a deterministic search, which here is exhaustive because the circuits are tiny; real ATPG uses PODEM/SAT-style search.

```
# faults.py - single stuck-at fault simulation + a brute-force "ATPG" top-up
```

```

import random, itertools
C17 = [("n10", "NAND", "i1", "i3"), ("n11", "NAND", "i3", "i6"),      # ISCAS-85 c17
       ("n16", "NAND", "i2", "n11"), ("n19", "NAND", "n11", "i7"),
       ("o22", "NAND", "n10", "n16"), ("o23", "NAND", "n16", "n19")]
CONS = [("t1", "AND", "a", "b"), ("na", "NOT", "a", None),           # f = ab + a'c + bc
        ("t2", "AND", "na", "c"), ("t3", "AND", "b", "c"),         # (bc is redundant:
        ("t4", "OR", "t1", "t2"), ("f", "OR", "t4", "t3"))         # consensus term)
OPS = {"NAND": lambda x, y: 1 - (x & y), "AND": lambda x, y: x & y,
       "OR": lambda x, y: x | y, "NOT": lambda x, y: 1 - x}

def simulate(ckt, pis, vec, fault=None):      # fault = (net, stuck_value)
    v = dict(zip(pis, vec))
    if fault and fault[0] in v: v[fault[0]] = fault[1]
    for out, op, a, b in ckt:
        v[out] = OPS[op](v[a], v[b] if b else 0)
        if fault and fault[0] == out: v[out] = fault[1]
    return v

def run(name, ckt, pis, pos, n_random, seed=1):
    nets = pis + [g[0] for g in ckt]
    faults = [(n, s) for n in nets for s in (0, 1)]
    good = lambda vec: [simulate(ckt, pis, vec)[o] for o in pos]
    detects = lambda vec, f: good(vec) != [simulate(ckt, pis, vec, f)[o] for o in pos]
    rng, left, pats = random.Random(seed), set(faults), []
    print("%s: %d nets, %d stuck-at faults, %d PIs, %d POs" %
          (name, len(nets), len(faults), len(pis), len(pos)))
    for k in range(1, n_random + 1):          # random-pattern phase
        vec = [rng.randint(0, 1) for _ in pis]
        hit = {f for f in left if detects(vec, f)}
        if hit: pats.append(vec); left -= hit
        print(" random pattern %d %s detected %d new FC=%5.1f%%" %
              (k, "".join(map(str, vec)), len(hit), 100 * (1 - len(left) / len(faults))))
    redundant = []
    for f in sorted(left):                    # deterministic top-up (exhaustive)
        if f not in left: continue            # already caught by a top-up pattern
        vec = next((list(v) for v in itertools.product((0, 1), repeat=len(pis))
                    if detects(list(v), f)), None)
        if vec is None: redundant.append(f); continue
        pats.append(vec); left -= {g for g in left if detects(vec, g)}
        print(" ATPG top-up for %s/SA%d: %s" % (f[0], f[1], "".join(map(str, vec))))
    fc = 100 * (len(faults) - len(left)) / len(faults)
    tc = 100 * (len(faults) - len(left)) / (len(faults) - len(redundant))
    print(" patterns=%d untestable(redundant)=%s" % (len(pats), redundant or "none"))
    print(" fault coverage=%5.1f%% test coverage=%5.1f%%\n" % (fc, tc))

run("c17", C17, ["i1", "i2", "i3", "i6", "i7"], ["o22", "o23"], n_random=4)
run("consensus", CONS, ["a", "b", "c"], ["f"], n_random=3)

```

faultsim.py - fault list, fault simulation with fault dropping, deterministic top-up and redundancy identification.

```

c17: 11 nets, 22 stuck-at faults, 5 PIs, 2 POs
random pattern 1 00101 detected 10 new FC= 45.5%
random pattern 2 11100 detected  3 new FC= 59.1%
random pattern 3 10110 detected  5 new FC= 81.8%
random pattern 4 11001 detected  0 new FC= 81.8%
ATPG top-up for i3/SA1: 00011
ATPG top-up for i6/SA0: 00111
ATPG top-up for i7/SA1: 00000
patterns=6 untestable(redundant)=none
fault coverage=100.0% test coverage=100.0%

consensus: 9 nets, 18 stuck-at faults, 3 PIs, 1 POs

```

```

random pattern 1 001 detected 6 new FC= 33.3%
random pattern 2 011 detected 0 new FC= 33.3%
random pattern 3 110 detected 3 new FC= 50.0%
ATPG top-up for b/SA1: 100
ATPG top-up for c/SA1: 000
ATPG top-up for na/SA1: 101
patterns=5 untestable(redundant)=[('t3', 0)]
fault coverage=94.4% test coverage=100.0%

```

Real output of faultsimpl.py.

- Random patterns reached 82% coverage on c17 with three patterns; the fourth random pattern detected nothing new - the typical saturation of random testing. Three deterministic patterns closed the rest: **6 patterns for 22 faults**.
- In the consensus circuit, **t3 stuck-at-0 is undetectable**: the term bc is logically redundant (ab + a'c already covers it), so no input can show a difference. ATPG proves such faults **untestable (redundant)**; they lower fault coverage but not test coverage.
- Redundant logic is a design smell: besides costing area it hides defects. (Here the consensus term is sometimes intentional - it removes a static hazard - which is why test coverage excludes provably untestable faults.)

## 10.7 Fault coverage versus test coverage

**fault coverage FC = detected / total faults**

**test coverage TC = detected / (total faults - untestable faults)**

**consensus example: FC = 17/18 = 94.4 % TC = 17/17 = 100 %**

**ATPG fault classes: DT detected, PT possibly detected (X at output),**

**AU ATPG-untestable (constraints), UD undetectable (redundant, tied, unused),**

**ND not detected (aborted / no pattern found)**

Coverage definitions. Tool vendors differ slightly in how possibly-detected and ATPG-untestable faults are credited; always state the definition when quoting a number.

Sign-off targets are project-specific. Commonly quoted ranges are stuck-at test coverage above about 98-99% and transition coverage above about 90-95% for consumer products, with automotive (AEC-Q100 / ISO 26262 contexts) pushing higher and adding cell-aware and in-system test requirements.

## 10.8 Test compression

A modern SoC has millions of scan cells. Without compression, test data volume and ATE time explode. **Compression** puts a **decompressor** between a few scan-in pins and hundreds of short internal chains, and a **compactor** (XOR tree / MISR) between the chains and the scan-out pins. It works because ATPG patterns are mostly don't-care bits: typically only a few percent of bits in a pattern are specified.

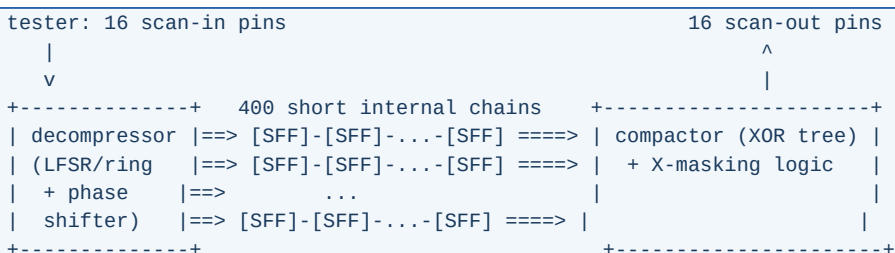


Figure 10.4 - Scan compression architecture (e.g. Siemens Tessent TestKompress/EDT, Synopsys DFTMAX / TestMAX, Cadence Modus).

- **EDT** (embedded deterministic test) solves linear equations so that the decompressor produces the specified care bits; DFTMAX uses a combinational broadcast/mux network plus sequential variants. Compression ratios of tens to over a hundred are common.
- **X-masking**: unknown values (uninitialised memories, analog outputs, false/multicycle paths captured at speed, bus contention) corrupt the compactor output; X-sources must be bounded in the design or masked per pattern, otherwise compression and coverage collapse.
- Compression slightly increases pattern count (encoding limitations) but reduces data and time by roughly the chain-length ratio.

The arithmetic of test time and data volume, for a 2-million-flop design with 12,000 patterns, 16 scan-in and 16 scan-out pins at 50 MHz shift:

```
# testtime.py - scan test time and data volume, with and without compression
import math
FLOPS, PATTERNS, F_SHIFT = 2_000_000, 12_000, 50e6    # scan cells, ATPG patterns, Hz
SCAN_PINS, COST_PER_S = 16, 0.03                    # scan-in pins (= scan-out), $/s tester
def report(name, chains, pin_pairs):
    L = math.ceil(FLOPS / chains)                      # longest chain length
    cycles = (PATTERNS + 1) * L + PATTERNS * 4        # shift in/out overlapped + capture
    t = cycles / F_SHIFT
    vol = 2 * PATTERNS * L * pin_pairs / 8 / 2**20    # stimulus + expected response, MiB
    print("%-16s chains=%5d L=%6d cycles=%10d t=%6.3f s $%.3f data=%6.1f MiB"
          % (name, chains, L, cycles, t, t * COST_PER_S, vol))
    return t
t0 = report("no compression", SCAN_PINS, SCAN_PINS)
for ratio in (25, 50, 100):
    t = report("compressed %3dx" % ratio, SCAN_PINS * ratio, SCAN_PINS)
    print("%-16s -> test time and data volume reduced %.0fx" % ("", t0 / t))
```

testtime.py - shift cycles, test time, tester cost and data volume (the tester cost per second is an assumed example value).

```
no compression chains= 16 L=125000 cycles=1500173000 t=30.003 s $0.900 data=5722.0 MiB
compressed 25x chains= 400 L= 5000 cycles= 60053000 t= 1.201 s $0.036 data= 228.9 MiB
-> test time and data volume reduced 25x
compressed 50x chains= 800 L= 2500 cycles= 30050500 t= 0.601 s $0.018 data= 114.4 MiB
-> test time and data volume reduced 50x
compressed 100x chains= 1600 L= 1250 cycles= 15049250 t= 0.301 s $0.009 data= 57.2 MiB
-> test time and data volume reduced 100x
```

Real output of testtime.py.

Without compression the test would take 30 s per die and 5.6 GiB of vector memory - far beyond any production budget, where logic scan time is typically a fraction of a second to a few seconds. 100x compression brings it to 0.3 s. In reality compressed pattern counts grow somewhat, and at-speed (transition) patterns are usually several times more numerous than stuck-at patterns, so both sets are budgeted separately.

## 10.9 Memory BIST and repair

Embedded SRAMs often occupy a large share of SoC area and have the densest layout on the die, so they have the highest defect density. They are tested by **memory BIST (MBIST)**: an on-chip controller generates addresses and data, runs **March algorithms** and compares read data, at full speed, with only a few control pins. A March test is a sequence of **March elements**, each visiting every address in ascending or descending order and applying a fixed sequence of reads and writes.

```
# march.py - March C- on a 16-word x 1-bit memory model with injected faults
N = 16
MARCH_CM = [("up", ["w0"]), ("up", ["r0", "w1"]), ("up", ["r1", "w0"]),
            ("down", ["r0", "w1"]), ("down", ["r1", "w0"]), ("any", ["r0"])]
class Mem:
    def __init__(s, fault=None): s.m, s.f = [0] * N, fault or {}
    def write(s, a, v):
```

```

        if s.f.get("saf") == (a, 1 - v): v = 1 - v          # stuck-at cell
        if s.f.get("tf") == (a, v) and s.m[a] != v: return # transition fault
        old, s.m[a] = s.m[a], v
        c = s.f.get("cfin")                                # inversion coupling:
        if c and c[0] == a and old == 0 and v == 1:        # aggressor 0->1 flips victim
            s.m[c[1]] ^= 1
    def read(s, a): return s.m[a]
def march(mem):
    for order, ops in MARCH_CM:
        addrs = range(N - 1, -1, -1) if order == "down" else range(N)
        for a in addrs:
            for op in ops:
                if op[0] == "w": mem.write(a, int(op[1]))
                elif mem.read(a) != int(op[1]): return "FAIL at addr %2d, op %s" % (a, op)
    return "pass"
print("March C- = {up(w0); up(r0,w1); up(r1,w0); dn(r0,w1); dn(r1,w0); any(r0)}: 10N ops")
for name, f in [("fault-free", None), ("stuck-at-0 cell 5", {"saf": (5, 0)}),
                ("stuck-at-1 cell 9", {"saf": (9, 1)}),
                ("up-transition fault cell 3", {"tf": (3, 1)}),
                ("coupling 2 -> 11 (aggr. lower)", {"cfin": (2, 11)}),
                ("coupling 12 -> 4 (aggr. higher)", {"cfin": (12, 4)})]:
    print(" %-32s %s" % (name, march(Mem(f))))

```

march.py - March C- on a behavioural memory with injected stuck-at, transition and inversion-coupling faults.

```

March C- = {up(w0); up(r0,w1); up(r1,w0); dn(r0,w1); dn(r1,w0); any(r0)}: 10N ops
fault-free           pass
stuck-at-0 cell 5    FAIL at addr  5, op r1
stuck-at-1 cell 9    FAIL at addr  9, op r0
up-transition fault cell 3    FAIL at addr  3, op r1
coupling 2 -> 11 (aggr. lower)  FAIL at addr 11, op r0
coupling 12 -> 4 (aggr. higher) FAIL at addr  4, op r1

```

Real output of march.py: every injected fault is detected, and the failing address and operation are reported (the basis for repair analysis).

March C- needs 10 operations per address (10N) and detects stuck-at, transition, address-decoder and a large class of coupling faults - note that the coupling fault with the aggressor at a **higher** address is caught by a descending element. Production MBIST uses several algorithms (March C-, March LR, checkerboards, retention tests with pauses) selected per memory type.

## Repair

- Large memories include **redundant rows and/or columns**. MBIST with **built-in redundancy analysis (BIRA)** collects the failing addresses and computes a repair solution.
- The solution is programmed into **fuses** (laser fuses in older technologies, **eFuses** or anti-fuse OTP today) at wafer test. At every power-up a fuse controller reads the fuse box and shifts the repair data into each memory's repair register before the memories are used (**soft repair** can also be done at every boot without fuses).
- Repair turns memory defects from yield loss into good dies; for large-memory SoCs it is a first-order yield lever (chapter 21).

## 10.10 Logic BIST

**Logic BIST (LBIST)** tests the logic without an external tester: a **PRPG** (pseudo-random pattern generator, an LFSR) feeds the scan chains, a **MISR** (multiple-input signature register) compacts the responses, and at the end the signature is compared with the expected golden value.

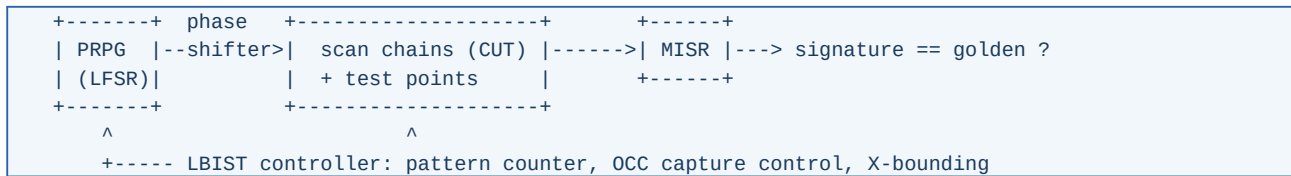


Figure 10.5 - Logic BIST architecture.

- Used where in-field or power-on self-test is required: **automotive functional safety (ISO 26262)** requires periodic latent-fault checks, and LBIST at key-on/key-off is a standard safety mechanism.
- Random patterns miss **random-pattern-resistant faults** (wide AND/OR cones). **Test points** (control and observe points) are inserted to raise coverage.
- Any X reaching the MISR corrupts the signature, so LBIST requires strict **X-bounding** of memories, analog blocks and non-scan flops.
- Hybrid schemes use the same compression logic for both ATPG and LBIST.

## 10.11 Test access: IEEE 1149.1, 1500 and 1687

| Standard                                 | Purpose                                                              | Key elements                                                                                                                                                                         |
|------------------------------------------|----------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>IEEE 1149.1 (JTAG, boundary scan)</b> | Board-level interconnect test and a standard chip test/debug port    | TAP with TCK, TMS, TDI, TDO (optional TRST); 16-state TAP controller; instruction register; boundary-scan cells on I/Os; instructions such as BYPASS, EXTEST, SAMPLE/PRELOAD, IDCODE |
| <b>IEEE 1149.6</b>                       | AC-coupled and differential I/O test                                 | Extension of 1149.1 for high-speed links                                                                                                                                             |
| <b>IEEE 1500</b>                         | Core-level test wrapper for embedded IP                              | Wrapper boundary register, wrapper instruction register, serial and parallel test access; isolates a core for test                                                                   |
| <b>IEEE 1687 (IJTAG)</b>                 | Access to embedded instruments (BIST controllers, sensors, PLL test) | ICL (instrument connectivity language), PDL (procedural description language), segment insertion bits (SIBs) for reconfigurable scan networks                                        |

Table 10.4 - Test-access standards.

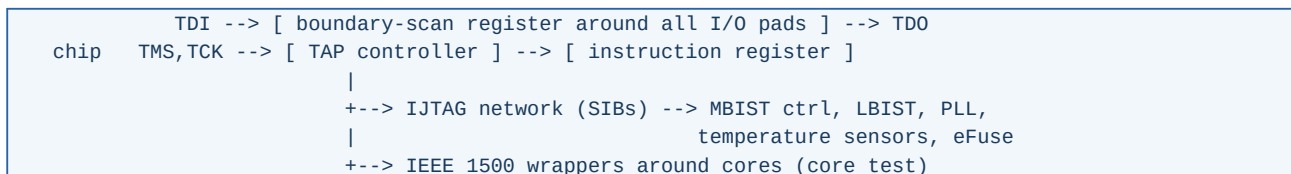


Figure 10.6 - Test access built around the JTAG TAP.

## 10.12 Test modes and pin muxing

A chip has far fewer pins available for test than it has test features, so functional pins are shared. A **test-mode controller** (entered via TAP instructions or dedicated test pins, usually locked after production for security) configures the chip:

| Mode                                     | Pin usage                                 | Notes                                                         |
|------------------------------------------|-------------------------------------------|---------------------------------------------------------------|
| <b>Functional (mission)</b>              | All pins functional                       | DFT logic transparent; must be proven by LEC/STA in this mode |
| <b>Scan (stuck-at / at-speed)</b>        | GPIOs become scan-in/out, SE, test clocks | Compression active; OCCs control capture                      |
| <b>MBIST</b>                             | JTAG only                                 | Controller runs algorithms, reports pass/fail and repair      |
| <b>Boundary scan</b>                     | JTAG + all I/O boundary cells             | Board test (EXTEST)                                           |
| <b>IDDQ / burn-in / characterisation</b> | Special                                   | Static states, stress modes                                   |

| Mode                       | Pin usage   | Notes                                                                  |
|----------------------------|-------------|------------------------------------------------------------------------|
| <b>Debug / secure lock</b> | JTAG, trace | Debug must be disabled or authenticated in production parts (security) |

Table 10.5 - Typical test modes.

**PITFALL - DFT PINS THAT NOBODY TIMED**

Scan-out pins multiplexed onto GPIOs add a mux in the functional path; test clocks enter through functional input pads; OCC muxes sit in the clock tree. Each test mode needs its own SDC and STA scenario (chapter 9, Table 9.6). Missing test-mode constraints are a classic reason why first-silicon scan fails while functional mode works.

## 10.13 DFT sign-off

### DFT sign-off checklist

- ☐ DFT DRC clean (or waived): all flops scannable, clocks and resets controllable, no combinational loops, X-sources bounded.
- ☐ Scan chains traced and verified in the final netlist (after reordering); chain lengths balanced; lockup latches present.
- ☐ Stuck-at, transition (and, if required, cell-aware, path-delay, bridging) coverage at target; untestable faults reviewed.
- ☐ Pattern counts and test time within budget; data volume fits the tester memory.
- ☐ Patterns simulated on the post-layout netlist with SDF at min and max corners (serial and parallel load); no mismatches.
- ☐ MBIST: all memories covered, algorithms reviewed, repair flow verified end-to-end including fuse programming and reload.
- ☐ LBIST (if present): signature stable across corners, X-bounding verified.
- ☐ Test modes: STA scenarios for shift and capture clean; LEC in functional mode clean; boundary scan BSDL generated and verified.
- ☐ Test-power analysis: shift and capture IR drop within limits (chapter 18).

## 10.14 DFT impact on area, timing and power

| Aspect               | Impact                                                                                                                                                       | Mitigation                                                                                                                                     |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Area</b>          | Scan flops (+31% per flop in SKY130), compression logic, OCCs, MBIST controllers and wrappers, TAP; commonly a few percent to over ten percent of logic area | Share MBIST controllers; use compression to reduce pins, not logic                                                                             |
| <b>Timing</b>        | Scan mux on every D input; SE tree; OCC muxes in the clock path; wrapper muxes on memory interfaces                                                          | Scan-aware synthesis (map to scan flops from the start); timing-aware chain ordering                                                           |
| <b>Shift power</b>   | In shift, a large fraction of flops toggle every cycle - far more than in functional operation - causing IR drop and heating                                 | Lower shift frequency, low-power fill of don't-care bits (fill with adjacent values), gating flop outputs during shift, staggered chain groups |
| <b>Capture power</b> | At-speed capture of random patterns can switch more logic than any real workload: IR drop slows paths and causes false failures (yield loss)                 | Power-aware ATPG (switching limits per pattern), clock-domain staggering, IR-drop-aware pattern validation                                     |

Table 10.6 - The cost of DFT.



**INTERVIEW INSIGHT - THE MOST ASKED DFT QUESTIONS**

Why is launch-on-shift harder to implement than launch-on-capture? (Scan-enable must switch at functional speed.) Why do we need lockup latches? (To avoid shift hold violations across clock skews.) Why can scan cause yield loss? (Test power - IR drop in capture slows paths that are fine in the application.) What is the difference between fault coverage and test coverage? (Whether provably untestable faults are in the denominator.) Be ready to draw a scan cell, a scan chain waveform and an OCC.

## 10.15 Summary

- Test detects manufacturing defects in every die; quality is measured in DPPM and follows  $DL = 1 - Y^{(1-T)}$ .
- Fault models: stuck-at, transition, path delay, bridging, cell-aware, IDDQ and memory faults, each with its own pattern type.
- Scan turns sequential test into combinational test; in SKY130 a scan flop costs 31% more area than a plain flop.
- At-speed test uses LOC or LOS launch with OCCs delivering PLL pulse bursts.
- ATPG = fault activation + propagation (D-algorithm, PODEM, SAT) plus fault simulation; our model reached 100% test coverage on c17 with 6 patterns and found the redundant consensus fault.
- Compression (EDT, DFTMAX) cut our 2M-flop example from 30 s to 0.3 s of scan time; X-sources must be bounded or masked.
- MBIST with March algorithms, repair via eFuses, LBIST with PRPG/MISR, and IEEE 1149.1/1500/1687 access complete the DFT architecture; DFT sign-off covers DRC, coverage, patterns, timing modes and test power.

## 10.16 Exercises

- 1 A product has 85% yield. What test coverage is needed for 100 DPPM according to Williams-Brown? For 10 DPPM?
- 2 Extend faults.py to model fanout branches of c17 separately (e.g. i3 feeds two gates). How many faults are there now, and does the pattern count change?
- 3 Using testtime.py, find the compression ratio needed to bring scan time below 0.5 s when the pattern count grows by 30% with compression.
- 4 Show a coupling fault that March C- detects only with the descending elements, and explain why both address orders are needed.
- 5 Explain why an uninitialised SRAM output reaching a compactor destroys coverage, and give two design fixes.
- 6 Compare LOC and LOS: draw the SE and CLK waveforms for each and state which paths each can test that the other cannot.



## PART III

# Power, Mixed-Signal and Physical Design Setup

Power intent and low-power techniques, the analog, clocking, I/O and ESD blocks every SoC must integrate, and the start of the back end: physical-design data, flows, design planning, floorplanning, power grids and the I/O ring.

## 11. Low-Power Design and Power Intent (UPF)

Power is no longer a number you look at after timing closes. For a phone SoC it sets battery life and the thermal envelope; for a data-centre accelerator it sets how many chips fit in a rack and what the electricity bill is; for an IoT sensor it decides whether the product runs for ten years on a coin cell or for ten weeks. In every modern ASIC, power is a first-class constraint that shapes the architecture (Chapter 5), the RTL (Chapter 6), the library choice (Chapter 4) and the whole back end (Chapters 13-18).

The companion RTL Design guide showed how to write clock-gating and operand-isolation RTL and gave a first UPF file. This chapter takes the implementation engineer's view: where every milliwatt comes from, how power is estimated at each stage and how accurate each estimate is, the full menu of techniques from architecture down to body bias, and the **power intent** file (UPF, IEEE 1801) in enough depth to write one for a multi-voltage, power-gated SoC and to follow it through simulation, synthesis, place-and-route and sign-off.

### 11.1 Where the power goes

Total power splits into **dynamic** power (charging and discharging capacitance, plus short-circuit current while both networks conduct during a transition) and **static** power (leakage that flows whether or not anything toggles). Chapter 2 derived each term from device physics; here are the forms an implementation engineer uses daily.

$$\begin{aligned}
 P_{\text{total}} &= P_{\text{switching}} + P_{\text{internal}} + P_{\text{leakage}} \\
 P_{\text{switching}} &= \alpha * C_{\text{load}} * V_{\text{DD}}^2 * f \quad (\text{net/pin capacitance, 'switching power'}) \\
 P_{\text{internal}} &= \text{sum over toggles of } E_{\text{int}}(\text{slew, load}) * \text{toggle\_rate} \\
 &\quad (\text{short-circuit + cell-internal node charging, from Liberty tables}) \\
 P_{\text{leakage}} &= V_{\text{DD}} * (I_{\text{sub}} + I_{\text{gate}} + I_{\text{GIDL}} + I_{\text{junction}}) \\
 I_{\text{sub}} &\sim \exp((V_{\text{GS}} - V_{\text{th}}) / (n * v_{\text{T}})), \quad v_{\text{T}} = kT/q = 26 \text{ mV at } 300 \text{ K}
 \end{aligned}$$

Equation 11.1 - The three power components as reported by power tools. 'alpha' is the activity factor (toggles per clock), f the clock frequency.

| Component              | Typical share                                          | Grows with                                                          | Main levers                                                              |
|------------------------|--------------------------------------------------------|---------------------------------------------------------------------|--------------------------------------------------------------------------|
| <b>Switching (net)</b> | 30-50% of dynamic                                      | activity, wire C, $V^2$ , f                                         | clock gating, data gating, lower V, shorter wires, low-C routing         |
| <b>Internal (cell)</b> | 30-50% of dynamic                                      | slew, activity, flop count                                          | fewer flops, good slews, multi-bit flops, gating                         |
| <b>Clock network</b>   | 20-45% of dynamic power                                | flop count, tree depth, f                                           | clock gating, CTS with fewer buffers, multi-bit flops, lower clock swing |
| <b>Leakage</b>         | a few % (HVT-heavy, cold) to over 30% (LVT-heavy, hot) | temperature (exponential), low $V_t$ , $V_{\text{DD}}$ (DIBL), area | multi- $V_t$ , power gating, body bias, lower $V_{\text{DD}}$            |

Table 11.1 - Power components and their levers. Shares vary widely with design style and node; the clock is often the single biggest consumer in a synchronous design.

#### WHY $V^2$ DOMINATES EVERY STRATEGY

Frequency enters dynamic power linearly, but achievable frequency itself falls roughly linearly with voltage above threshold. Scaling V and f together therefore reduces dynamic power roughly with the **cube** of V, and energy per operation (power / f) with the **square**. That single fact motivates DVFS, multi-voltage domains, near-threshold computing and 'race-to-idle' vs 'slow-and-steady' debates.

## 11.2 Power analysis flow

A power number is only as good as its **activity** and its **parasitics**. As a project progresses, both become more accurate, and the power team (often part of the physical-design or methodology group, working with the architects) re-runs the analysis at each stage.

| stage        | netlist           | parasitics       | activity          | accuracy |
|--------------|-------------------|------------------|-------------------|----------|
| architecture | spreadsheet model | none             | use-case profile  | +50%     |
| RTL          | RTL (+ quick map) | wire-load/none   | RTL sim (FSDB)    | +20-30%  |
| synthesis    | mapped netlist    | wire-load / est. | SAIF / FSDB       | +15-20%  |
| post-place   | placed netlist    | virtual route    | SAIF / vectorless | +10-15%  |
| sign-off     | routed netlist    | extracted SPEF   | gate-level FSDB   | +5-10%   |

```

RTL/gate sim --> FSDB/VCD (per-cycle toggles) --+
      \-> SAIF (toggle counts, T0/T1) ----> power tool --> average,
vectorless (set_switching_activity defaults) ----+ (PrimePower, peak, per-
netlist + Liberty (power tables) + SPEF -----+ Voltus, Joules instance,
                                                    PowerArtist) per-domain
  
```

Figure 11.1 - Power estimation through the project. Accuracy ranges are typical rules of thumb, not guarantees; the dominant error is almost always unrepresentative activity, not the tool.

### Activity sources

- **Vectorless** (statistical): the tool propagates default toggle rates and static probabilities from the inputs (e.g. 10-20% toggle rate on data, 100%/2 per cycle on clocks). Fast and needs no testbench, but can be wildly off for control-dominated logic. Used early and for power-grid design where a pessimistic assumption is acceptable.
- **SAIF** (Switching Activity Interchange Format): per-net toggle count (TC) and time at 0/1 (T0/T1) over a window. Small files, gives accurate **average** power, cannot give peak or cycle-by-cycle power.
- **FSDB/VCD** (waveforms): every transition with a time stamp; enables **time-based** (peak, per-cycle) power and the vectors used for **dynamic IR drop** (Chapter 18). Big files: pick short, meaningful windows.
- **Emulation-based** power: an emulator (Palladium, ZeBu, Veloce) runs real software for billions of cycles; activity is summarised per window to find the power hot-spots, then a short FSDB window is extracted for detailed analysis.

### Average vs peak, and which scenario to sign off

Average power drives battery life and the thermal design (with time constants of milliseconds to seconds); **peak** power and **di/dt** drive the package, the voltage regulator and the power grid (time constants of nanoseconds). They need different vectors: average power wants a representative workload (e.g. video playback, a benchmark), while peak wants the worst sustainable burst (all cores running a power virus, a DFT scan-shift pattern) and the worst **step** (from idle to full activity in a few cycles). Scan shift is notorious: with all flops toggling at 50% it can draw several times functional power, which is why DFT uses low-power shift patterns and slower shift clocks (Chapter 10).

#### PITFALL: A 'POWER NUMBER' WITHOUT ITS CONDITIONS

'The block is 120 mW' means nothing unless you state the corner (process, voltage, temperature), the activity source and scenario, the frequency, and whether it is average or peak. Leakage at the fast corner and 125 C can be 10-50x the typical value at 25 C; a dynamic number at typical voltage can be 30% below the worst case at high voltage. Always quote a power number with its PVT, mode and activity.

### Early estimation

Before RTL exists, the architect estimates power from similar blocks (mW per MHz per mm<sup>2</sup> or energy per operation), from the gate count and an average energy per gate toggle, and from memory access counts multiplied by energy per access from the memory compiler datasheet. RTL power tools (e.g. PowerArtist, Joules, PrimePower RTL) then do a quick internal synthesis and give per-module power with 20-30%

accuracy, which is enough to decide where to add clock gating.

### 11.3 Techniques by level of abstraction

The higher the level, the bigger the saving and the cheaper it is to apply early. A rough rule quoted in many low-power courses: architecture and system decisions can save 10x or more, RTL techniques 20-50%, and gate/physical optimisation 10-20%.

| Level                 | Technique                                                                                                                                    | What it saves                 | Cost / risk                                                            |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------|------------------------------------------------------------------------|
| <b>System</b>         | DVFS, adaptive voltage scaling (AVS), race-to-idle, power states (sleep, retention, off)                                                     | dynamic ( $V^2$ ) and leakage | PMIC/regulator, software governor, characterisation at many V/f points |
| <b>Architecture</b>   | parallelism/pipelining at lower V, specialised accelerators, memory hierarchy (fewer DRAM accesses), data compression, near-memory compute   | dynamic, often by 10x+        | area, design effort                                                    |
| <b>RTL</b>            | clock gating (module/register level), operand isolation, memory chip-select/light-sleep gating, glitch reduction, one-hot vs binary encoding | dynamic 20-50%                | gating logic timing, DFT observability                                 |
| <b>Synthesis</b>      | automatic clock-gate insertion, multi-Vt assignment, multi-bit flops, low-power mapping                                                      | dynamic and leakage 10-30%    | timing closure, hold fixes                                             |
| <b>Implementation</b> | power gating (header/footer switches), retention, multi-voltage domains, level shifters, isolation                                           | leakage 90%+ in off domains   | area 5-15%, verification complexity, rush current, wake-up latency     |
| <b>Device/circuit</b> | body bias (forward/reverse), high-Vt / long channel cells, SRAM retention voltage                                                            | leakage                       | bias generators, triple wells, limited in FinFET                       |

Table 11.2 - The low-power toolbox by abstraction level.

#### System and architecture: DVFS, AVS and parallelism

**Dynamic voltage and frequency scaling (DVFS)** chooses an operating performance point (OPP) - a (V, f) pair - according to the load. The hardware must be characterised and signed off at every OPP (each is an STA corner), the PLL must re-lock or a glitch-free clock switch is needed, and the voltage must ramp **before** the frequency goes up and **after** it comes down. **Adaptive voltage scaling (AVS)** closes the loop: on-die monitors (ring oscillators, critical-path replicas, timing-error detectors) tell the regulator how much margin the particular die at its current temperature really has, recovering the guard band that a fixed OPP table must keep for slow silicon. **Parallelism**: two copies of a unit at half the frequency can run at a lower voltage; for the same throughput the energy per operation falls with  $V^2$  while area doubles. This is why GPUs and AI accelerators are wide and relatively slow.

The following model uses an alpha-power-law delay ( $f$  proportional to  $(V - V_{th})^\alpha / V$ ) and a leakage current that falls exponentially with V (DIBL) to show the energy per operation of one block across the voltage range. The parameters are illustrative, not from a specific process.

```
# dvfs.py - energy per operation vs supply voltage (alpha-power-law delay model)
import numpy as np
C_eff, Vth, alpha = 50e-12, 0.35, 1.3 # F switched per op, V, velocity-sat. exponent
f_nom, I_leak0 = 1.0e9, 5e-3 # 1 GHz and 5 mA leakage at 0.90 V
def fmax(V): k = f_nom / ((0.9 - Vth)**alpha / 0.9); return k * (V - Vth)**alpha / V
def ileak(V): return I_leak0 * np.exp((V - 0.9) / 0.3)
for V in [0.90, 0.80, 0.70, 0.60, 0.55, 0.50, 0.45, 0.42, 0.40]:
    f = fmax(V); Ed = C_eff * V**2 * 1e12; El = V * ileak(V) / f * 1e12
    ... # print P_dyn, P_leak, E/op
Vs = np.linspace(0.37, 0.9, 2000)
E = C_eff*Vs**2*1e12 + Vs*ileak(Vs)/fmax(Vs)*1e12 # pJ/op
i = np.argmin(E) # minimum-energy point
```

Listing 11.1 - Core of the DVFS energy model (printing lines elided).

| Vdd<br>[V] | fmax<br>[MHz] | P_dyn<br>[mW] | P_leak<br>[mW] | E_dyn/op<br>[pJ] | E_leak/op<br>[pJ] | E_total/op<br>[pJ] | leak% |
|------------|---------------|---------------|----------------|------------------|-------------------|--------------------|-------|
| 0.90       | 1000          | 40.50         | 4.500          | 40.50            | 4.50              | 45.00              | 10%   |
| 0.80       | 867           | 27.73         | 2.866          | 32.00            | 3.31              | 35.31              | 9%    |
| 0.70       | 714           | 17.50         | 1.797          | 24.50            | 2.52              | 27.02              | 9%    |
| 0.60       | 538           | 9.69          | 1.104          | 18.00            | 2.05              | 20.05              | 10%   |
| 0.55       | 439           | 6.64          | 0.856          | 15.13            | 1.95              | 17.07              | 11%   |
| 0.50       | 332           | 4.16          | 0.659          | 12.50            | 1.98              | 14.48              | 14%   |
| 0.45       | 218           | 2.21          | 0.502          | 10.12            | 2.30              | 12.43              | 19%   |
| 0.42       | 147           | 1.30          | 0.424          | 8.82             | 2.89              | 11.71              | 25%   |
| 0.40       | 100           | 0.80          | 0.378          | 8.00             | 3.79              | 11.79              | 32%   |

minimum-energy point: V = 0.412 V, f = 127 MHz, E = 11.65 pJ/op  
energy saving vs 0.90 V: 3.9x; speed penalty: 7.9x

Output of dvfs.py (python3 + numpy). Power falls 50x from 0.9 V to 0.4 V, but energy per operation only 3.9x, and below about 0.41 V it rises again because leakage integrates over ever longer cycles.

Three lessons generalise. First, lowering V pays off quadratically only while dynamic energy dominates. Second, there is a **minimum-energy point** near threshold where leakage energy per operation starts to win; running slower than that wastes energy. Third, if the work has a deadline and the chip can be power-gated when idle, 'race to idle' at high V can beat 'slow and steady' only when leakage in the active state is large and the idle state is truly off - which is exactly the trade-off OS power governors evaluate. (The alpha-power law is inaccurate close to threshold; real near-threshold design uses characterised libraries.)

## RTL: clock gating, operand isolation, memory gating

Clock gating removes the clock from registers whose value will not change. Synthesis tools infer an **integrated clock-gating cell** (ICG: a latch plus an AND, glitch-free by construction) from enable-style RTL, typically for groups of at least 3-8 flops; architects add coarse module-level gates by hand. Operand isolation holds the inputs of an expensive datapath (multiplier, adder tree) constant when its result is not used. Memories are the other big win: drive chip-select only when accessing, and use the memory's **light-sleep** (periphery off), **deep-sleep** (array at retention voltage) and **shut-down** pins from the power controller. The coding details are in the companion RTL Design guide; what matters downstream is that every ICG is a timing endpoint (the enable has a setup check to the gating latch - the 'clock gating check'), sits in the clock tree and must be scan-controllable (test enable pin) (Chapters 9, 10 and 16).

## Implementation: multi-Vt, power gating and multi-voltage

- **Multi-Vt:** libraries come in several threshold flavours (e.g. ULVT, LVT, SVT, HVT; on the order of 2-4x leakage change and 10-30% speed change per step). Synthesis and P&R start with high-Vt everywhere it meets timing and swap to lower Vt only on critical paths. Many teams cap the LVT/ULVT fraction (e.g. a few percent of cells) as a leakage budget.
- **Power gating:** a switch transistor in series with the supply - a PMOS **header** between the real VDD and a virtual VDD, or an NMOS **footer** between virtual VSS and VSS - disconnects a whole domain. Headers are more common (easier with a common ground and standard-cell rows). Leakage of the domain drops by 10-100x to that of the switches themselves.

- **Retention:** a retention flop has a small always-on 'balloon' latch on the always-on supply that saves the state before power-down and restores it after power-up, so software does not need to save/restore registers. It costs area (often 20-40% bigger than a plain flop) and an always-on supply route.
- **Isolation:** outputs of a powered-down domain float to unknown values; an isolation cell (an AND/OR clamp on an always-on supply) forces them to a known safe value while the domain is off.
- **Level shifters:** a signal crossing from a low-voltage to a high-voltage domain cannot fully turn off the receiver's PMOS, causing crowbar current and slow edges, so it needs a level shifter (L->H is mandatory; H->L is often just a buffer on the low supply, but many methodologies insert shifters both ways). Combined **enable level shifters** also isolate.
- **Multi-voltage domains:** each voltage island has its own supply and can be at a different, fixed or DVFS-scaled voltage; SRAM arrays often have a separate, higher, rail (VDDM) because bit cells need more voltage than logic for stability.
- **Body bias:** reverse body bias (RBB) raises  $V_t$  and cuts leakage in standby; forward body bias (FBB) lowers  $V_t$  for speed. Very effective in bulk planar and especially FD-SOI (wide bias range through the thin BOX); weak in FinFET, where the fin is poorly coupled to the substrate.

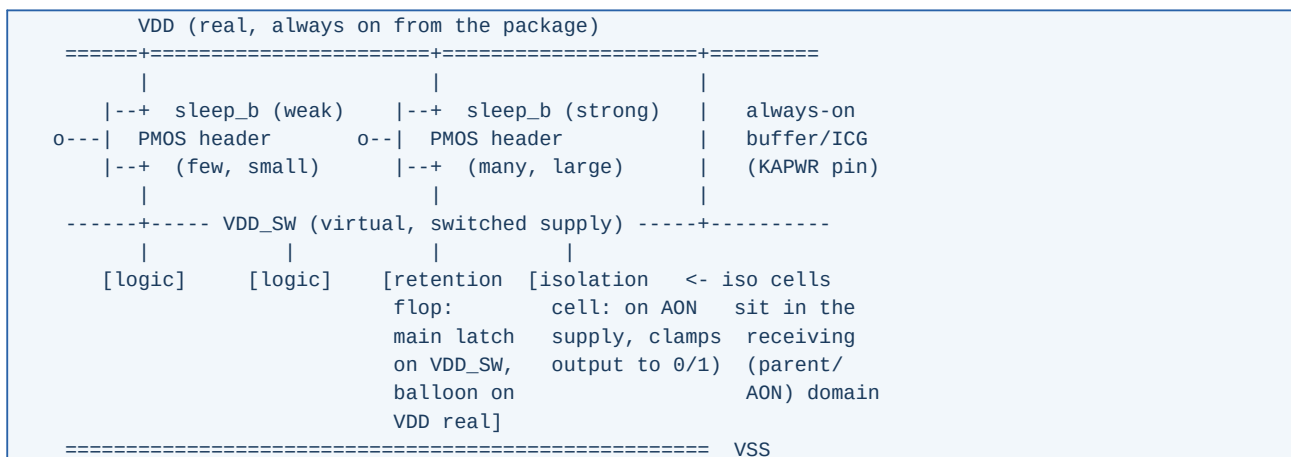


Figure 11.2 - A header-switched domain: weak switches turn on first to limit in-rush current, strong switches after; retention and always-on cells need a connection to the real (unswitched) supply.

## Real low-power cells in an open library

Low-power cells are ordinary Liberty cells with extra attributes that tell the tools what they are. The following script reads the real SkyWater SKY130 high-density library (typical corner, 25 C, 1.80 V) and prints the special cells next to a plain inverter and flip-flop.

```
# lp_cells.py - find low-power special cells in the SKY130 HD Liberty file
import re
txt = open("sky130_fd_sc_hd__tt_025C_1v80.lib").read()
def cell_block(name):
    # brace-matched text of one cell group
    i = txt.index('cell ("%s")' % name); j = txt.index("{", i); d = 0
    for k in range(j, len(txt)):
        d += (txt[k] == "{") - (txt[k] == "}")
        if d == 0: return txt[i:k]
for c in ["inv_1", "dfxtp_1", "dlclkp_1", "sdlclkp_1", "lpflow_inputiso0p_1", ...]:
    b = cell_block("sky130_fd_sc_hd__" + c)
    # print area, cell_leakage_power and any of: clock_gating_integrated_cell,
    # is_isolation_cell, is_level_shifter, plus the output function and the
    # primary_power pg_pins when there is more than one
```

Listing 11.2 - Liberty attribute scan (abridged).

| cell                      | area                    | leak_nW | special attribute      | function [power pins] |
|---------------------------|-------------------------|---------|------------------------|-----------------------|
| inv_1                     | 3.75                    | 0.0053  | -                      | (!A)                  |
| dfxtp_1                   | 20.02                   | 0.0084  | -                      | IQ                    |
| dlclkp_1                  | 17.52                   | 0.0088  | icg=posedge            | -                     |
| sdlclkp_1                 | 18.77                   | 0.0104  | icg=posedge_precontrol | -                     |
| lpflow_inputiso0p_1       | 7.51                    | 0.0053  | isolation_cell=true    | (!SLEEP&A)            |
| lpflow_inputiso1n_1       | 7.51                    | 0.0036  | isolation_cell=true    | (A)   (!SLEEP_B)      |
| lpflow_isobufsrc_1        | 6.26                    | 0.0022  | isolation_cell=true    | (A&!SLEEP)            |
| lpflow_lsbuf_lh_isowell_4 | 40.04                   | 0.0112  | level_shifter=true     | (A) [LOWLVPWR, VPWR]  |
| lpflow_clkbufkapwr_1      | 3.75                    | 0.0012  | -                      | (A) [KAPWR, VPWR]     |
| lpflow_decapkapwr_4       | 5.00                    | 0.0032  | -                      | - [KAPWR, VPWR]       |
| decap_4                   | 5.00                    | 0.0032  | -                      | -                     |
| tapvpwrvgnd_1             | (not in timing library) |         |                        |                       |

Output of lp\_cells.py on the real SKY130 HD Liberty (area in  $\mu\text{m}^2$ , leakage in nW as declared by leakage\_power\_unit; icg=posedge is the attribute value latch\_posedge).

- **dlclkp\_1** is the ICG: **clock\_gating\_integrated\_cell : latch\_posedge** tells synthesis it may use it for clock gating; **sdlclkp** adds a test-enable (precontrol) input for scan.
- The **lpflow\_inputiso\*** and **isobufsrc** cells carry **is\_isolation\_cell : true** and pins flagged **isolation\_cell\_enable\_pin** - the tools only use them where UPF asks for isolation.
- The level shifter has two primary power pins (LOWLVPWR for the input side, VPWR for the output side), **level\_shifter\_type : LH** and **input\_voltage\_range / output\_voltage\_range** (1.2-2.1 V here).
- **kapwr** cells have a **keep-alive** power pin (KAPWR): they sit physically inside a switched domain but are powered from the always-on rail - used for always-on clock buffers and feed-throughs.
- The tap cell is a physical-only cell (in the LEF, not in the timing library); Chapter 14 places it.

#### PITFALL: LETTING SYNTHESIS USE SPECIAL CELLS AS LOGIC

Synthesising a small MAC/register-file block with Yosys against the full SKY130 library, the mapper happily used 45 **lpflow\_isobufsrc\_1** and 5 **lpflow\_inputiso1p\_1** cells as ordinary AND gates (real run, Chapter 13 shows the design). Functionally correct, but the P&R and low-power checkers then see 'isolation cells' that no UPF strategy asked for, and some of these cells have unusual power pins. Commercial flows mark such cells **dont\_use** (and **dont\_touch** where inserted); open flows use a trimmed library. The same applies to clock buffers/inverters in data paths, delay cells, and any cell the foundry lists as 'not for synthesis'.

## 11.4 UPF (IEEE 1801) in depth

RTL describes function; **UPF** describes power intent: which supplies exist, which logic is powered by which supply, which domains can switch off or change voltage, and what protection is needed at every crossing. It is a Tcl-based format standardised as IEEE 1801 (UPF 1.0 was the Accellera version; IEEE 1801-2009 is UPF 2.0, -2013 UPF 2.1, -2015 UPF 3.0, -2018 UPF 3.1, with a later revision published since). Cadence's older CPF format expressed the same concepts; most flows today are UPF. Tool support lags the standard, so teams pin a version (commonly 2.1 or 3.x subsets) and a coding style guide.

### The object model



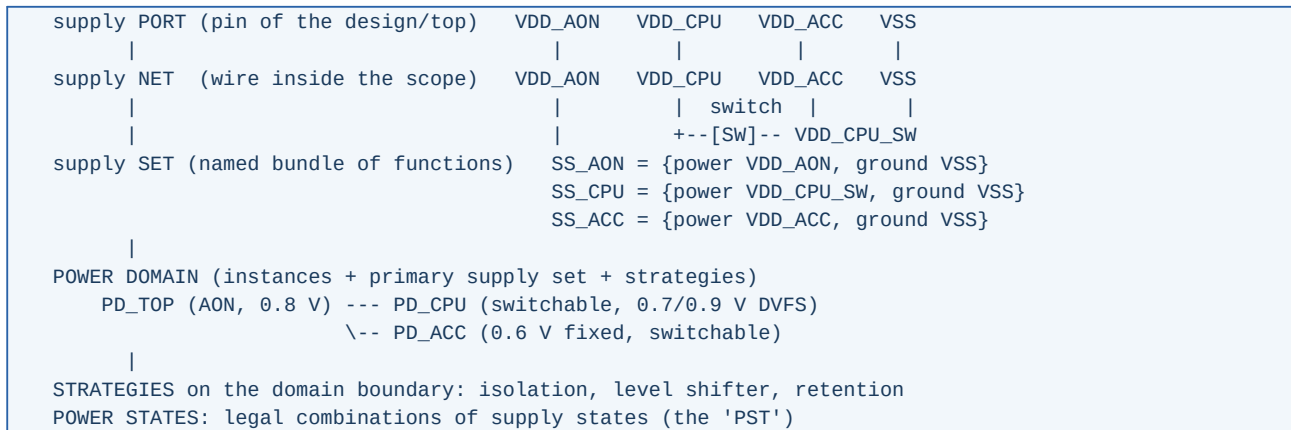


Figure 11.3 - UPF objects. Strategies are written against domains and supply sets, never against individual cells; the tools choose and place the cells.

| Command                                                                                           | Purpose                                                   | Key options                                                                                                                                                                                                                                            |
|---------------------------------------------------------------------------------------------------|-----------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>upf_version</code> , <code>set_scope</code> , <code>load_upf</code>                         | Version declaration, hierarchy scope, include a block UPF | <code>load_upf blk.upf -scope u_blk</code> for hierarchical UPF                                                                                                                                                                                        |
| <code>create_power_domain</code>                                                                  | Group instances with a primary supply                     | <code>-elements</code> , <code>-include_scope</code> , <code>-supply {primary SS}</code> , <code>-supply {default_isolation SS}</code> , <code>-atomic</code>                                                                                          |
| <code>create_supply_port / net</code> , <code>connect_supply_net</code>                           | Supply topology                                           | <code>-direction in/out</code> , <code>-domain</code> , <code>-resolve (rare)</code>                                                                                                                                                                   |
| <code>create_supply_set</code>                                                                    | Bundle of power/ground (and n/p-well) functions           | <code>-function {power NET}</code> , <code>-function {ground NET}</code> , <code>-update</code>                                                                                                                                                        |
| <code>create_power_switch</code>                                                                  | Switch topology, control and ack                          | <code>-input_supply_port</code> , <code>-output_supply_port</code> , <code>-control_port</code> , <code>-ack_port</code> , <code>-on_state</code> , <code>-off_state</code>                                                                            |
| <code>set_isolation</code>                                                                        | Where and how to clamp                                    | <code>-applies_to inputs/outputs/both</code> , <code>-clamp_value 0/1/latch</code> , <code>-isolation_signal</code> , <code>-isolation_sense</code> , <code>-location self/parent</code> , <code>-diff_supply_only</code> , <code>-source/-sink</code> |
| <code>set_level_shifter</code>                                                                    | Voltage-crossing rules                                    | <code>-rule low_to_high/high_to_low/both</code> , <code>-location</code> , <code>-threshold</code> , <code>-no_shift</code>                                                                                                                            |
| <code>set_retention (+ set_retention_elements)</code>                                             | Which state is kept, with what controls                   | <code>-retention_supply_set</code> , <code>-save_signal</code> , <code>-restore_signal</code> , <code>-elements</code>                                                                                                                                 |
| <code>add_power_state / legacy create_pst</code> , <code>add_pst_state</code>                     | Legal power states of supply sets and domains             | <code>-supply_expr</code> , <code>-logic_expr</code> , <code>-simstate</code>                                                                                                                                                                          |
| <code>map_retention_cell</code> , <code>use_interface_cell</code> , <code>map_power_switch</code> | Constrain which library cells implement a strategy        | <code>-lib_cells</code> , <code>-strategy</code>                                                                                                                                                                                                       |
| <code>set_port_attributes</code> , <code>set_design_attributes</code>                             | Boundary information for block-level UPF                  | <code>-driver_supply</code> , <code>-receiver_supply</code> , <code>-is_analog</code>                                                                                                                                                                  |

Table 11.3 - The UPF commands you will actually write. Option names changed between versions (UPF 3.0 prefers `-isolation_supply/-retention_supply`); follow the version your tools accept.

## A complete example: three domains, DVFS and power gating

The SoC below has an always-on top domain at 0.8 V (power controller, interrupt logic, wake-up timers), a CPU domain that runs at 0.7 or 0.9 V (DVFS) and can be switched off with state retention, and an accelerator domain at a fixed 0.6 V that can be switched off without retention. The PMU (power management unit) in the AON domain drives all controls.



```

## ===== soc.upf : IEEE 1801 (UPF 2.1 style) =====
upf_version 2.1
set_scope /
set_design_top soc_top

## ---- 1. supply ports and nets (top level) ----
foreach s {VDD_AON VDD_CPU VDD_ACC VSS} {
    create_supply_port $s -direction in
    create_supply_net $s
    connect_supply_net $s -ports $s
}
create_supply_net VDD_CPU_SW      ;# switched (virtual) CPU supply
create_supply_net VDD_ACC_SW      ;# switched (virtual) ACC supply

## ---- 2. supply sets ----
create_supply_set SS_AON -function {power VDD_AON} -function {ground VSS}
create_supply_set SS_CPU -function {power VDD_CPU_SW} -function {ground VSS}
create_supply_set SS_CPU_RET -function {power VDD_CPU} -function {ground VSS}
create_supply_set SS_ACC -function {power VDD_ACC_SW} -function {ground VSS}

## ---- 3. power domains ----
create_power_domain PD_TOP -include_scope -supply {primary SS_AON}
create_power_domain PD_CPU -elements {u_cpu} -supply {primary SS_CPU}
create_power_domain PD_ACC -elements {u_acc} -supply {primary SS_ACC}

```

Listing 11.3a - Supplies and domains.

```

## ---- 4. power switches (headers placed in the switched domains) ----
create_power_switch SW_CPU -domain PD_CPU \
    -input_supply_port {vin VDD_CPU} -output_supply_port {vout VDD_CPU_SW} \
    -control_port {en u_pmu/cpu_pwr_en} \
    -ack_port {ack u_pmu/cpu_pwr_ack} \
    -on_state {ON vin {en}} -off_state {OFF {!en}}
create_power_switch SW_ACC -domain PD_ACC \
    -input_supply_port {vin VDD_ACC} -output_supply_port {vout VDD_ACC_SW} \
    -control_port {en u_pmu/acc_pwr_en} \
    -ack_port {ack u_pmu/acc_pwr_ack} \
    -on_state {ON vin {en}} -off_state {OFF {!en}}
map_power_switch SW_CPU -domain PD_CPU -lib_cells {HDRSW_X4}

## ---- 5. isolation: outputs of each switchable domain, cells in the parent (AON) --
set_isolation iso_cpu -domain PD_CPU -applies_to outputs \
    -isolation_supply_set SS_AON -clamp_value 0 \
    -isolation_signal u_pmu/cpu_iso_en -isolation_sense high -location parent
set_isolation iso_cpu_irqn -domain PD_CPU -elements {u_cpu/irq_n} \
    -isolation_supply_set SS_AON -clamp_value 1 \
    -isolation_signal u_pmu/cpu_iso_en -isolation_sense high -location parent
set_isolation iso_acc -domain PD_ACC -applies_to outputs \
    -isolation_supply_set SS_AON -clamp_value 0 \
    -isolation_signal u_pmu/acc_iso_en -isolation_sense high -location parent

```

Listing 11.3b - Switches and isolation. **HDRSW\_X4** stands for whatever header cell the library provides. Note the separate strategy for the active-low interrupt, which must clamp to 1.

```
## ---- 6. level shifters: every crossing between different voltages -----
set_level_shifter ls_cpu_out -domain PD_CPU -applies_to outputs -rule both \
    -location parent
set_level_shifter ls_cpu_in  -domain PD_CPU -applies_to inputs  -rule both \
    -location self
set_level_shifter ls_acc_out -domain PD_ACC -applies_to outputs -rule low_to_high \
    -location parent
set_level_shifter ls_acc_in  -domain PD_ACC -applies_to inputs  -rule high_to_low \
    -location self

## ---- 7. retention: architectural state of the CPU only -----
set_retention ret_cpu -domain PD_CPU -retention_supply_set SS_CPU_RET \
    -elements {u_cpu/u_regfile u_cpu/u_csr u_cpu/u_pc} \
    -save_signal {u_pmu/cpu_save high} \
    -restore_signal {u_pmu/cpu_restore high}
map_retention_cell ret_cpu -domain PD_CPU -lib_cells {RDFFX1 RDFFX2}
```

Listing 11.3c - Level shifting and retention. Isolation cells at the parent location for a crossing that is also a voltage change are usually merged into **enable level shifters**.

```
## ---- 8. power states -----
add_power_state SS_AON -state ON    {-supply_expr {power == `{FULL_ON, 0.80}}}}
add_power_state SS_CPU -state HV    {-supply_expr {power == `{FULL_ON, 0.90}}}}
add_power_state SS_CPU -state LV    {-supply_expr {power == `{FULL_ON, 0.70}}}}
add_power_state SS_CPU -state OFF   {-supply_expr {power == `{OFF}} -simstate CORRUPT}
add_power_state SS_ACC -state ON    {-supply_expr {power == `{FULL_ON, 0.60}}}}
add_power_state SS_ACC -state OFF   {-supply_expr {power == `{OFF}} -simstate CORRUPT}

add_power_state PD_TOP -state RUN_FAST {-logic_expr {SS_CPU == HV  && SS_ACC == ON}}
add_power_state PD_TOP -state RUN_ECO  {-logic_expr {SS_CPU == LV  && SS_ACC == ON}}
add_power_state PD_TOP -state CPU_ONLY {-logic_expr {SS_CPU == HV  && SS_ACC == OFF}}
add_power_state PD_TOP -state SLEEP    {-logic_expr {SS_CPU == OFF && SS_ACC == OFF}}
## (any combination not listed, e.g. CPU OFF with ACC ON, is illegal)
```

Listing 11.3d - Power states. Static checkers compute every legal pair of (driver, receiver) states from this table and verify that each crossing has the isolation and shifting it needs.

The legacy **power state table (PST)** expresses the same information as a matrix over supply ports. You will still meet it in older IP deliveries and in some tool flows:

```
add_port_state VDD_AON -state {ON 0.80}
add_port_state VDD_CPU -state {HV 0.90} -state {LV 0.70} -state {OFF off}
add_port_state VDD_ACC -state {ON 0.60} -state {OFF off}
create_pst soc_pst -supplies {VDD_AON VDD_CPU VDD_ACC}
add_pst_state RUN_FAST -pst soc_pst -state {ON HV ON }
add_pst_state RUN_ECO  -pst soc_pst -state {ON LV ON }
add_pst_state CPU_ONLY -pst soc_pst -state {ON HV OFF}
add_pst_state SLEEP    -pst soc_pst -state {ON OFF OFF}
```

Listing 11.4 - The same states as a legacy PST (on real supply ports; switched nets inherit their state from the switch).

| Crossing (driver -> receiver) | Voltages       | Can driver be off while receiver on? | Needs                                                  |
|-------------------------------|----------------|--------------------------------------|--------------------------------------------------------|
| PD_CPU -> PD_TOP              | 0.7/0.9 -> 0.8 | yes (SLEEP)                          | isolation + LS both ways (enable level shifter)        |
| PD_TOP -> PD_CPU              | 0.8 -> 0.7/0.9 | no (AON never off)                   | LS only (both directions depending on OPP)             |
| PD_ACC -> PD_TOP              | 0.6 -> 0.8     | yes (CPU_ONLY, SLEEP)                | isolation + L->H shifter                               |
| PD_TOP -> PD_ACC              | 0.8 -> 0.6     | no                                   | H->L shifter (or buffer on the 0.6 V supply)           |
| PD_CPU -> PD_ACC              | 0.7/0.9 -> 0.6 | no (CPU off => ACC off)              | H->L shifter; no isolation required by the state table |

Table 11.4 - What the state table implies at every crossing. A good exercise: add the state CPU OFF with ACC ON and see which row changes.

#### INTERVIEW INSIGHT: ISOLATION LOCATION AND SUPPLY

Where do isolation cells go? With **-location parent** they sit in the receiving (usually always-on) domain and are powered by it, which is physically simple. With **-location self** they sit inside the switchable domain but must be powered from an always-on supply routed into that domain (a secondary power pin, like the KAPWR cells above). The isolation **enable** must itself come from an always-on domain and must be asserted before the supply drops and released only after it is stable again.

#### PITFALL: CLAMP VALUES THAT BREAK THE RECEIVER

Clamping everything to 0 is the default and it is wrong for active-low signals (a request\_n or reset\_n clamped to 0 asserts it), for handshake signals where 0 means 'busy', and for bus signals where the receiver needs an IDLE encoding. Clamp values are a functional decision the RTL owner must review, signal by signal.

## 11.5 UPF through the flow

One UPF (with per-stage additions) travels with the design from RTL to sign-off. Each tool reads it, implements its part, and writes out an updated UPF that describes what was built.

```
RTL + UPF (golden, 'power intent')
|---> static LP checks (VC LP, Conformal Low Power): syntax, consistency
|---> power-aware RTL simulation (VCS NLP, Xcelium, Questa PA)
v
synthesis (DC/FC, Genus): inserts ISO, LS, retention flops, maps ICGs;
writes netlist + UPF' (with implementation details, e.g. inserted cells)
|---> static LP checks on netlist (every crossing covered? cells legal?)
|---> LP-aware equivalence (RTL+UPF vs netlist+UPF')
v
P&R (Innovus, ICC2/FC, OpenROAD): creates domain regions, places switches,
routes secondary PG pins, builds per-domain grids, connects always-on;
writes netlist WITH PG connections + UPF''
|---> static LP checks on PG netlist; LVS knows the real supplies
v
sign-off: MV STA (per-supply voltages from the states), power/IR per domain,
rush-current analysis, power-aware gate-level simulation of the sequences
```

Figure 11.4 - UPF through the flow. The low-power static checker is run at every hand-off, the same way STA and equivalence are.

| Stage            | Owner                              | What UPF drives                                                                                                                   | Typical checks                                           |
|------------------|------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| <b>RTL</b>       | RTL/architecture + power architect | Power-aware simulation: supplies turn on/off, corruption of off domains, isolation/retention behaviour                            | UPF lint; missing isolation; PMU sequence tests          |
| <b>Synthesis</b> | synthesis/implementation           | Cell insertion; domain-aware optimisation (no buffers from the wrong domain); multi-voltage timing                                | LP check netlist vs UPF; LEC with UPF                    |
| <b>P&amp;R</b>   | physical design                    | Voltage areas / domain regions, switch arrays, secondary PG routing, always-on buffering, level-shifter placement at the boundary | PG connectivity; always-on paths; buffer supply legality |
| <b>Sign-off</b>  | STA, power-integrity, verification | Per-state timing, IR drop per domain, rush current, PA-GLS                                                                        | MV STA, IR/EM, LVS with PG netlist, LP check final       |

Table 11.5 - Who uses the power intent at each stage.

Static checkers report issues with rule names such as missing isolation, isolation enable driven from a switchable domain, level shifter missing, retention control not always-on, or 'buffer powered by off domain on an always-on path'. An excerpt of the kind of summary they print:

```
# ---- LP static check summary (ILLUSTRATIVE, not from a real run) ----
Rule          Severity  Count  Example
ISO_MISSING   error      3    u_acc/done -> u_noc/acc_done
ISO_EN_NOT_AON error      1    u_cpu/u_pmu_if/iso_q (PD_CPU)
LS_MISSING_LH error      2    u_acc/irq -> u_gic/irq[5]
ISO_CLAMP_ACTIVE_LOW warning  1    u_cpu/irq_n clamp 0
AON_BUF_IN_SWITCHED error     4    inserted by CTS in PD_CPU
RET_CTRL_CORRUPTIBLE error     1    u_pmu/cpu_save from PD_CPU
Total: 11 errors, 1 warning
```

Illustrative LP-checker summary with generic rule names (every commercial tool has its own names and format).

## 11.6 Power-aware verification

A normal simulator assumes every gate is always powered. A **power-aware** simulator reads the UPF, models supply nets as having a state and a voltage, and **corrupts** (drives X onto) every register and net of a domain whose supply is off (simstate CORRUPT). Isolation cells clamp, retention registers save and restore, and anything the UPF forgot shows up as X propagating into the live domains. The verification plan (Chapter 7) must include:

- Every legal power-state transition, including the ones software rarely uses (e.g. RUN\_ECO -> SLEEP -> RUN\_FAST).
- Correct control sequences from the PMU (Figure 11.5), including wake-up events arriving at every step of a power-down sequence.
- Retention: state saved, domain off, domain back on, state restored and **non-retained** state reset correctly.
- Isolation clamps visible and correct in the receiving domain.
- Illegal states: the PMU must never create a state that is not in the state table (assertions on the supply states).
- Gate-level power-aware simulation of the sequences with the PG-connected netlist before tape-out.

|                                                                                                                                                                                                                                                             |                                                                                                                                                                                                                                                                                                                         |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>power DOWN</p> <ol style="list-style-type: none"> <li>1. stop clocks to the domain (ICG off)</li> <li>2. assert save (retention)</li> <li>3. assert isolation enable</li> <li>4. (optional) assert reset</li> <li>5. switch off; wait for ack</li> </ol> | <p>power UP</p> <ol style="list-style-type: none"> <li>1. switch enable (weak chain first)</li> <li>2. wait for ack (supply stable)</li> <li>3. assert reset of non-retained logic</li> <li>4. assert restore (retention)</li> <li>5. release reset</li> <li>6. release isolation</li> <li>7. restart clocks</li> </ol> |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Figure 11.5 - A typical power-down / power-up sequence. The exact order of reset and restore depends on the retention cell style; the invariants are: isolate before power goes away, un-isolate only after state is valid, and never clock a domain without power.

## 11.7 Rush current and power-up sequencing

When a switched domain turns on, its whole decoupling and gate capacitance charges from (nearly) zero to VDD. If all switches close at once the in-rush current  $I = C \cdot dV/dt$  can be amperes for nanoseconds, collapsing the supply of the **neighbouring, active** domains through the shared grid and package inductance.

$$I_{\text{rush}} = C_{\text{domain}} * dV/dt$$

example:  $C_{\text{domain}} = 5 \text{ nF}$ ,  $V = 0.8 \text{ V}$ , ramp in  $20 \text{ ns} \rightarrow I = 5\text{e-}9 * 0.8 / 20\text{e-}9 = 0.2 \text{ A}$

same domain ramped in  $2 \text{ us}$  through the weak chain  $\rightarrow I = 2 \text{ mA}$

energy to charge:  $E = C V^2 = 3.2 \text{ nJ}$  per wake-up (half is lost in the switches)

Equation 11.2 - Rush current and wake-up energy. Wake-up energy sets the minimum idle time for which power gating pays off (the break-even time).

- **Daisy-chained switches:** the enable ripples through the switch cells (each has an enable-out pin); the chain delay spreads the turn-on over many nanoseconds to microseconds.
- **Two-stage (trickle + main):** a chain of small, weak switches charges the domain slowly; when the virtual supply is near VDD the strong chain closes. The ack at the end of the last chain tells the PMU the domain is ready.
- Rush current and the resulting droop in neighbouring domains is a **sign-off check** (Chapter 18) done with dynamic power-integrity tools; the number and size of switches (IR drop when on) and the chain arrangement (rush when turning on) trade against each other.
- Chip-level **power-up sequencing** (which external rails come up first: typically core before I/O or as the I/O library requires; power-on reset; PLL lock; release of resets in order) is written into the spec with the board and PMIC team.

### Low-power sign-off checklist

- ☐ UPF lint clean; the same UPF version accepted by all tools in the flow.
- ☐ Every crossing covered by isolation / level shifting in every legal state.
- ☐ Isolation, retention and switch controls all driven from always-on logic.
- ☐ Clamp values reviewed by RTL owners (active-low and handshake signals).
- ☐ Power-aware simulation of all state transitions; PA-GLS of the sequences.
- ☐ LP-aware equivalence between RTL+UPF and final netlist+UPF.
- ☐ Always-on buffering and secondary PG pins routed; LVS with PG netlist.
- ☐ Multi-voltage STA at every OPP; level shifter timing and voltage ranges.
- ☐ Rush current and wake-up latency within budget; IR drop of switches in ON state.
- ☐ Leakage and dynamic power reported per domain, per state, at stated PVT.

## 11.8 Summary

- Power = switching + internal + leakage; the clock network and memories are usually the biggest dynamic consumers, leakage grows exponentially with temperature and lower  $V_t$ .
- Power estimates need activity (vectorless, SAIF, FSDB) and parasitics; accuracy improves from about  $\pm 50\%$  at architecture to  $\pm 5\text{-}10\%$  at sign-off. Quote power with PVT, mode and activity.
- The biggest savings are architectural (DVFS, parallelism, specialisation); RTL gating saves tens of percent; implementation techniques (multi- $V_t$ , power gating, multi-voltage, body bias) attack leakage.
- Energy per operation has a minimum near threshold; below it, leakage dominates.
- UPF (IEEE 1801) describes supplies, domains, switches, isolation, level shifters, retention and legal power states; the same intent drives simulation, synthesis, P&R and sign-off, with static LP checks at each hand-off.
- Power gating needs a carefully ordered sequence (stop clocks, save, isolate, switch off; and the reverse) and controlled in-rush current through daisy-chained weak and strong switches.

## 11.9 Exercises

---

- 1 A block has 2 nF of switched capacitance per cycle at 0.8 V and 1 GHz with activity 0.15, and 30 mW of leakage at 0.8 V. Compute the dynamic power. If DVFS moves it to 0.65 V and 700 MHz and leakage scales as  $\exp((V - 0.8)/0.25)$ , what is the new total power and the new energy per cycle?
- 2 Modify the UPF of Listings 11.3a-d to add a legal state where the CPU is OFF and the accelerator is ON. Which isolation and level-shifter strategies must change, and why?
- 3 Explain why an isolation enable generated by a flop inside PD\_CPU is a bug even though simulation of the 'happy path' passes.
- 4 A domain has 8 nF of capacitance at 0.75 V. The neighbouring domain tolerates at most 30 mA of extra transient current. What is the minimum ramp time, and how would you implement it with switch chains?
- 5 Using the dvfs.py model, change the leakage slope to  $\exp((V - 0.9)/0.15)$  (a leakier process) and predict qualitatively how the minimum-energy point moves. Then run it.
- 6 List three reasons a synthesis library should mark special cells (isolation, level shifters, always-on buffers, clock inverters) as `dont_use` for general mapping.

## 12. Analog/Mixed-Signal, I/O and ESD Integration

No SoC is purely digital. Even a 'digital' ASIC needs a clock generator, a reference voltage, a way to know that power is good, I/O cells that talk to the outside world at a different voltage, and protection against the thousands of volts a human finger or a handling machine can deliver. Most of these blocks are designed by analog/mixed-signal (AMS) specialists or bought as hard IP, but the digital implementation team integrates them: it floorplans them, routes their supplies, closes timing to their digital pins, verifies their interaction with the logic and signs off the chip-level ESD network. Most tape-out delays caused by 'the analog block' are really integration problems: missing views, wrong abstracts, noisy neighbours, or an ESD path nobody owned.

### 12.1 The analog blocks in a typical SoC

| Block                                   | What it does                                                                                                               | Integration concerns                                                                        |
|-----------------------------------------|----------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| <b>Crystal oscillator (XO) pad cell</b> | Drives an external quartz crystal (e.g. 19.2/24/25/38.4 MHz) - the clean reference                                         | Dedicated pads, quiet supply, keep digital switching away                                   |
| <b>PLL (phase-locked loop)</b>          | Multiplies the reference to GHz clocks, sets frequency for DVFS                                                            | Jitter spec, lock time, quiet supply (often own LDO), clock output routing, test/bypass mux |
| <b>DLL (delay-locked loop)</b>          | Aligns phases (DDR DQS centring, de-skew); no VCO so no jitter accumulation                                                | Placement next to the PHY; delay-line matching                                              |
| <b>RC / ring oscillators</b>            | Low-accuracy clocks for boot, always-on timers, watchdogs                                                                  | Trimming, temperature drift                                                                 |
| <b>Bandgap reference</b>                | Temperature-stable voltage (about 1.2 V in silicon) and bias currents                                                      | Everything analog depends on it; trim bits in OTP/fuses                                     |
| <b>LDO / DC-DC (buck) regulators</b>    | Generate internal rails; LDO is quiet but lossy (efficiency $\sim V_{out}/V_{in}$ ), buck is efficient but switching-noisy | Large currents, pads for inductors, decap, stability, power-up order                        |
| <b>POR / brown-out detector</b>         | Holds the chip in reset until the supply is valid                                                                          | Must work with no clock and at any ramp rate                                                |
| <b>ADC / DAC</b>                        | Sensor, audio, RF baseband conversion (SAR, sigma-delta, pipeline, flash)                                                  | Reference and clock purity, substrate noise, digital calibration logic                      |
| <b>Temperature / voltage sensors</b>    | Thermal throttling, AVS, reliability monitors                                                                              | Distributed placement near hot spots                                                        |
| <b>SerDes / high-speed PHY</b>          | PCIe, Ethernet, USB, DDR, MIPI links (TX FFE, RX CTLE/DFE, CDR)                                                            | Bump/ball assignment, package channel, power isolation, huge IP deliverable set             |
| <b>eFuse / OTP</b>                      | Trim, keys, repair data                                                                                                    | High-voltage programming supply, security                                                   |

Table 12.1 - Common analog and mixed-signal blocks. The protocol side of SerDes and DDR PHYs is covered in the companion Communication Protocols guide.

### 12.2 PLL fundamentals

A charge-pump PLL is a feedback loop that forces the divided-down output of a voltage-controlled oscillator to match the reference in phase and therefore in frequency:  $f_{out} = N * f_{ref}$  (integer-N) or  $(N + \text{fraction}) * f_{ref}$  (fractional-N, where a sigma-delta modulator dithers the divider).

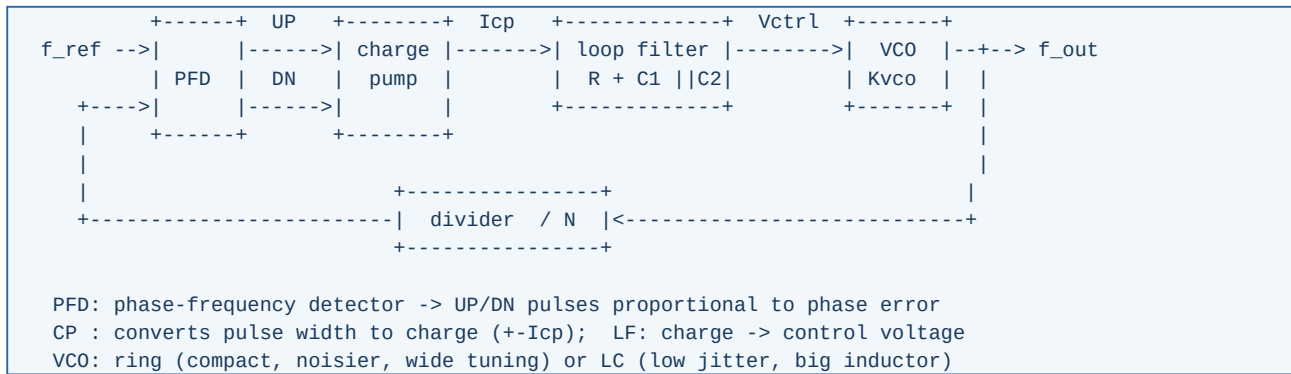


Figure 12.1 - Charge-pump PLL. The loop filter's resistor creates the stabilising zero; C2 smooths the ripple the charge pump injects every reference cycle.

$$\begin{aligned} \omega_n &= \sqrt{I_{cp} \cdot K_{vco} / (2\pi \cdot N \cdot C_1)} && \text{(natural frequency)} \\ \zeta &= (R/2) \cdot \sqrt{I_{cp} \cdot K_{vco} \cdot C_1 / (2\pi \cdot N)} && \text{(damping)} \\ \text{zero} : f_z &= 1/(2\pi \cdot R \cdot C_1) && \text{ripple pole: } f_{p3} = 1/(2\pi \cdot R \cdot C_1 \cdot C_2 / (C_1 + C_2)) \\ \text{rule of thumb: loop bandwidth} &< f_{ref}/10 && \text{(discrete-time sampling limit)} \end{aligned}$$

Equation 12.1 - Second-order approximation of a type-II charge-pump PLL ( $K_{vco}$  in rad/s/V).

```
# pll.py - type-II third-order charge-pump PLL loop parameters
import numpy as np
f_ref, N = 25e6, 80 # 25 MHz reference, f_vco = 2 GHz
Icp, Kvco = 100e-6, 2*np.pi*1.0e9 # 100 uA, 1 GHz/V
R, C1, C2 = 8.0e3, 100e-12, 5e-12
wn = np.sqrt(Icp*Kvco/(2*np.pi*N*C1))
zeta = R/2*np.sqrt(Icp*Kvco*C1/(2*np.pi*N))
w = 2*np.pi*np.logspace(3, 9, 200000); s = 1j*w # exact open loop with C2
Z = (1 + s*R*C1) / (s*(C1+C2)*(1 + s*R*C1*C2/(C1+C2)))
G = Icp/(2*np.pi) * Z * Kvco/s / N
k = np.argmin(abs(abs(G) - 1)); pm = 180 + np.degrees(np.angle(G[k]))
```

Listing 12.1 - PLL loop calculation (print statements elided).

```
f_vco = 2.000 GHz
natural freq fn = 0.56 MHz
damping zeta = 1.41
zero fz = 0.20 MHz
pole fp3 = 4.18 MHz
unity-gain fc = 1.45 MHz (f_ref/fc = 17)
phase margin = 63.1 deg
lock time ~ 4/(zeta*wn) = 0.80 us (to ~2% settling)
```

Output of pll.py. A 63-degree phase margin and a bandwidth 17x below the reference are healthy; the lock-time figure is only a rough second-order estimate (real lock includes frequency acquisition).

## Jitter, and how it reaches STA

- **Period jitter**: deviation of each period from the ideal; **cycle-to-cycle jitter**: difference between adjacent periods; **long-term jitter / TIE**: accumulated edge displacement. PLL datasheets quote these as RMS and peak (peak ~ 6-7x RMS for Gaussian jitter at typical bit-error-style confidence levels, depending on sample count).
- Inside the loop bandwidth the output follows the reference (and its noise); outside it the VCO's own phase noise dominates. The loop bandwidth is therefore chosen to balance reference noise vs VCO noise, and supply noise sensitivity of ring VCOs makes the PLL supply one of the quietest on the chip.
- For digital timing, the PLL's period jitter (plus duty-cycle distortion and the clock tree's own supply-induced jitter) becomes part of **set\_clock\_uncertainty** for setup (Chapter 9). Hold checks on the same edge see no PLL jitter, only tree jitter.



Clocking architecture

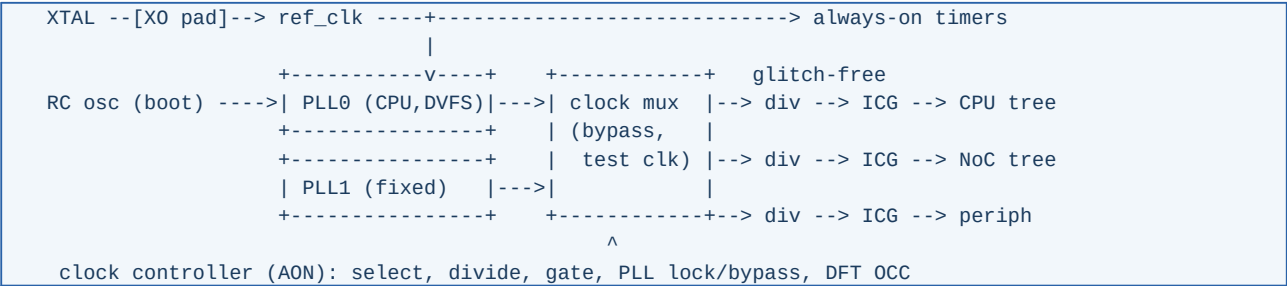


Figure 12.2 - A typical clock generation and distribution scheme. The clock controller is digital RTL owned by the SoC team; the PLLs are hard macros. Test clocks enter through on-chip clock controllers (OCC, Chapter 10).

12.3 The AMS design flow

Analog blocks are designed at the transistor level with a completely different flow from the digital one, usually in Cadence Virtuoso (schematic and custom layout) with SPICE-class simulators (Spectre, HSPICE, Siemens AFS, Synopsys PrimeSim).

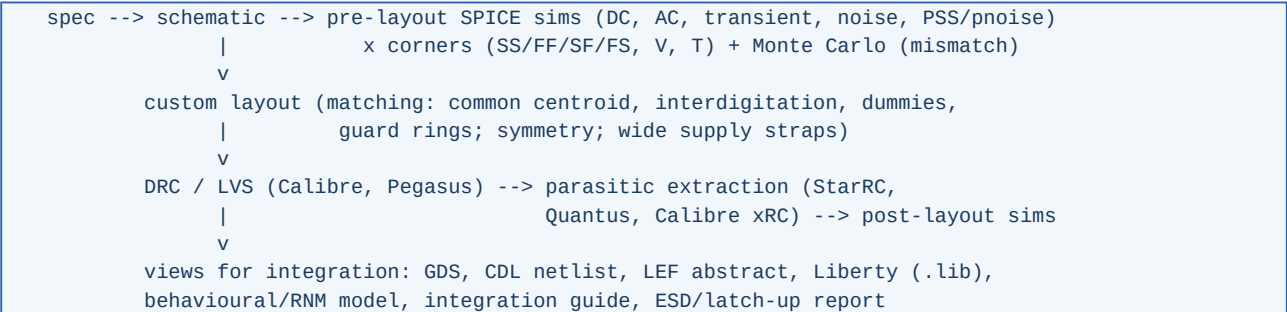


Figure 12.3 - Analog design flow and the views it must deliver to the SoC team.

Where digital sign-off relies on STA and static checks, analog sign-off relies on simulation across corners and Monte Carlo (mismatch between 'identical' transistors is the dominant accuracy limit of comparators, current mirrors and DACs). Layout is performance: a differential pair that is not matched and symmetric does not meet offset specs no matter what the schematic says.

12.4 AMS verification at the SoC level

A full-chip SPICE simulation of an SoC is impossible, yet the digital logic that configures, calibrates and monitors the analog blocks must be verified with them. The industry answer is a ladder of model abstractions:

| Model                   | Language                                                                   | Speed                       | Use                                                                                 |
|-------------------------|----------------------------------------------------------------------------|-----------------------------|-------------------------------------------------------------------------------------|
| Pure digital stub       | SV/Verilog                                                                 | fastest                     | Connectivity only; dangerous if it hides behaviour                                  |
| Real-number model (RNM) | SV <i>real</i> , user-defined nettypes (SV-2012 UDN), Cadence <i>wreal</i> | near-digital (event-driven) | Default for SoC regressions: voltages/currents as real values that change at events |
| Behavioural analog      | Verilog-A / Verilog-AMS                                                    | 10-100x slower than digital | Continuous-time behaviour (filters, loops) in an analog solver                      |
| Transistor level        | SPICE netlist                                                              | slowest                     | Block sign-off; selected co-simulation of critical interactions                     |

Table 12.2 - The abstraction ladder. Mixed-signal simulators (Xcelium with Spectre AMS Designer, VCS with PrimeSim, Questa with Symphony) co-simulate digital and analog engines.

The key discipline is **model validation**: every RNM must be checked against the transistor-level schematic (same stimulus, compare outputs within tolerance), otherwise the SoC regression verifies the model writer's assumptions rather than the circuit. Below is a small RNM of a power-on-reset block - a comparator with

hysteresis on a real-valued supply plus a digital delay counter - simulated with Icarus Verilog.

```
`timescale 1ns/1ps
module por_rnm #(parameter real VTH_R = 1.35, VTH_F = 1.25, parameter int DLY = 8)
    (input real vdd, input logic osc_clk, output logic por_n);
    logic vok = 1'b0;           // analog comparator output (digital)
    int cnt = 0;
    always @(vdd) begin         // re-evaluated whenever the real supply changes
        if (!vok && vdd > VTH_R) vok = 1'b1;
        else if (vok && vdd < VTH_F) vok = 1'b0;
    end
    always @(posedge osc_clk or negedge vok)
        if (!vok) begin cnt <= 0; por_n <= 1'b0; end
        else if (cnt < DLY) cnt <= cnt + 1;
        else por_n <= 1'b1;
    initial por_n = 1'b0;
endmodule
// tb: vdd ramps 0 -> 1.8 V in 0.1 V/100 ns steps, stays, dips to 1.1 V, recovers;
//      a 10 MHz osc_clk; $display on every change of por_n or the comparator
```

Listing 12.2 - Real-number POR model (testbench summarised in the last comment lines).

```
time(ns)  vdd(V)  por_n
      0      0.00  0   (comparator=0)
    1400      1.40  0   (comparator=1)
    2250      1.80  1   (comparator=1)
    3900      1.20  1   (comparator=0)
    3900      1.20  0   (comparator=0)
    4300      1.40  0   (comparator=1)
    5150      1.80  1   (comparator=1)
por_rnm.sv:28: $finish called at 6200000 (1ps)
```

iverilog -g2012 -Wall, vvp: reset is released 8 oscillator cycles after the rising threshold is crossed, asserted at once when the brown-out drops below the falling threshold, and the hysteresis (1.35/1.25 V) avoids chatter.

#### PITFALL: THE 'ALWAYS POWERED' ANALOG MODEL

The most common SoC-level AMS bug is a behavioural model that ignores its own supply and enable pins: in simulation the PLL locks while its LDO is still off, or an ADC converts while its reference is disabled. Models must check supply/bias/enable pins and produce X or invalid outputs when they are not valid - and those checks must be validated against the schematic.

## 12.5 Integration: analog-on-top vs digital-on-top

|                                | Digital-on-top (DoT)                                                | Analog-on-top (AoT)                                                             |
|--------------------------------|---------------------------------------------------------------------|---------------------------------------------------------------------------------|
| <b>Top-level owner</b>         | Digital P&R tool (Innovus, ICC2/FC)                                 | Custom layout tool (Virtuoso); digital blocks are hard macros                   |
| <b>Typical chip</b>            | Large SoC with a few analog macros                                  | Mostly analog chip (PMIC, sensor AFE, RF transceiver) with some digital control |
| <b>Analog block form</b>       | Hard macro with LEF/Liberty/GDS views                               | Native schematic/layout; digital comes in as placed-and-routed macros           |
| <b>Chip-level verification</b> | STA + digital checks; analog treated as black boxes plus AMS co-sim | Chip-level SPICE/AMS simulation; digital timing closed at block level           |
| <b>Risk</b>                    | Noise coupling and supply mistakes around macros                    | Digital timing and DFT integration handled by hand                              |

Table 12.3 - The two integration styles. Mixed flows (OpenAccess-based interoperability between Virtuoso and Innovus) let each team edit its own part of one database.

## Views a hard macro must deliver

### Hard-macro (analog IP) deliverables

- ☐ GDSII/OASIS layout and a CDL/SPICE netlist for LVS (with black-box rules if needed).
- ☐ LEF abstract: size, pins (layer, shape, direction, USE SIGNAL/POWER/GROUND/ANALOG), obstructions (OBS) per layer, and the routing layers the chip may use over it.
- ☐ Liberty timing model for digital pins (setup/hold to its clocks, output delays, pin capacitance, leakage/power, pg\_pin and related\_power\_pin attributes).
- ☐ Behavioural/RNM model validated against the schematic; UPF for its supplies.
- ☐ Integration guide: supply sequencing, decap, keep-outs, shielding, pin constraints.
- ☐ DRC/LVS/antenna clean reports; ESD and latch-up compliance statement.
- ☐ Test and trim: DFT access, BIST/loopback, trim fuses, characterisation data.

Timing models for hard macros come in two flavours: a **Liberty abstract** written by the IP team (black-box timing arcs per pin), or for digital hard blocks an **ETM** (extracted timing model) or **ILM** (interface logic model) generated from the block's own implementation (Chapter 13).

## 12.6 Noise isolation

Digital logic injects noise into the shared substrate and supplies every clock edge; sensitive analog circuits (PLL VCOs, ADC comparators, bandgaps, LNAs) pick it up. Isolation is a combination of layout and planning:

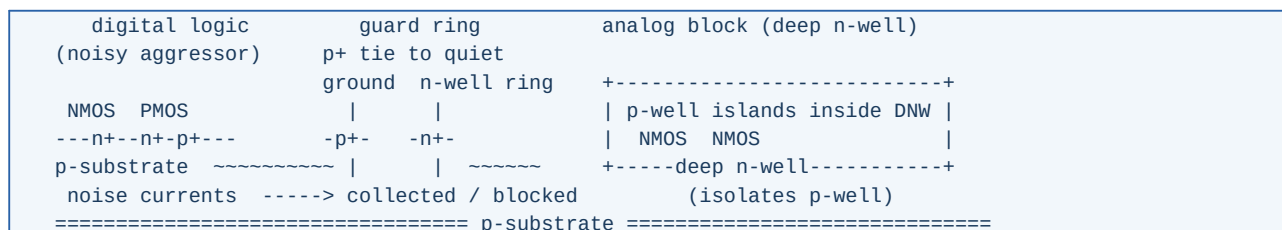


Figure 12.4 - Substrate noise isolation: guard rings collect substrate current before it reaches the victim; a deep n-well (triple-well process) isolates the p-well of sensitive NMOS devices from the common substrate.

- **Separate supplies and grounds** for analog (AVDD/AVSS), separate pads and package balls, joined only at the board or through back-to-back diodes for ESD.
- **Distance and floorplan**: keep high-activity digital (CPU cores, clock roots) and I/O drivers away from the PLL/ADC; put analog at the die edge next to its pads.
- **Guard rings** (p+ substrate ties, n-well rings) around both aggressors and victims; **deep n-well** under sensitive blocks.
- **Routing keep-outs and shielding**: no digital routing over analog macros (blockages on all or some layers), grounded shields beside sensitive analog nets, differential routing.
- **Clock and supply hygiene**: spread-spectrum or de-synchronised switching, on-chip decap, dedicated LDOs for the most sensitive blocks.

## 12.7 I/O cells

I/O cells (pad cells) translate between the core's thin-oxide, low-voltage transistors and the outside world at 1.2, 1.8, 2.5 or 3.3 V using thick-oxide devices. They are delivered as an **I/O library** from the foundry or an IP vendor, with Liberty, LEF, GDS and SPICE/IBIS models (IBIS is used by board engineers for signal integrity).

| core side (VDD, 0.8 V)                                                                                                                                                                                                                                                                                                                                                                                                                                                           | pad side (VDDIO, 1.8/3.3 V) |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------|
| <pre> A (data out) --[level  ]--[pre-driver]---+--[big PMOS]--+ OE (enable) --[shifter ]--[ + slew  ]---+--[big NMOS]---+-----[ PAD ]--- bond/bump DS[1:0] (drive strength) -&gt; selects driver fingers     SR (slew rate)          -&gt; pre-driver strength   [pull-up/down R, keeper] PU/PD (pulls)               Y (data in) &lt;--[level shift]&lt;--[Schmitt/CMOS rx]&lt;---+ [ESD diodes to VDDIO/VSSIO] IE (input enable, blocks floating-input crowbar current) </pre> |                             |

Figure 12.5 - A bidirectional GPIO cell. The ESD diodes and the rail clamp elsewhere in the ring are part of every signal pad.

| Feature                                  | Purpose                                                   | Design consideration                                              |
|------------------------------------------|-----------------------------------------------------------|-------------------------------------------------------------------|
| <b>Drive strength (e.g. 2/4/8/12 mA)</b> | Meet load/speed at the board                              | Stronger drive = more SSO noise and EMI                           |
| <b>Slew-rate control</b>                 | Slow edges reduce ringing and SSO                         | Must still meet the interface timing                              |
| <b>Pull-up / pull-down / keeper</b>      | Define floating inputs, straps                            | Weak (tens of kohm); check leakage in sleep                       |
| <b>Schmitt trigger input</b>             | Hysteresis for slow or noisy edges (resets, buttons)      | Slightly slower; mandatory on reset pins                          |
| <b>Input enable / fail-safe</b>          | Avoid crowbar current and back-powering when VDDIO is off | Required when the other side can be powered while the chip is off |
| <b>Power-on control (POC)</b>            | Keeps outputs tristated while the core supply ramps       | A global signal on the I/O ring                                   |

Table 12.4 - GPIO features the SoC team configures through control pins or registers.

### Pad types in the ring

- **Signal pads:** digital GPIO, specialised (LVDS, I2C open-drain, oscillator), and **analog pads** (direct connection with ESD but no buffers; low series resistance).
- **Core supply pads** (VDD/VSS) feeding the core power ring or grid, and **I/O supply pads** (VDDIO/VSSIO) feeding the pad ring's own rails. Their number is set by current, EM, IR drop and SSO (the rule 'one VDDIO/VSSIO pair per N signal pads' comes from the I/O library's SSO guideline).
- **Corner cells** that continue the ring rails around corners; **filler/spacer cells** between pads that keep the rails and ESD bus continuous; **power-cut (break) cells** that split the ring into separately supplied segments (e.g. 1.8 V and 3.3 V banks) while keeping the ESD ground bus connected.
- Flip-chip designs place I/O and supply cells in the core area or an array; bumps connect through a redistribution layer (Chapter 14).

## 12.8 ESD: models and protection networks

Electrostatic discharge is a short, high-current event: a charged person or machine touching a pin, or the charged package itself discharging to a grounded tool. Thin gate oxides break down at a few volts and junctions melt at amperes, so every pin needs a path that carries amperes for nanoseconds while keeping the voltage across sensitive devices below their breakdown.

| Model                                                     | What it represents                                       | Circuit / waveform                                                       | Typical component-level target                                       |
|-----------------------------------------------------------|----------------------------------------------------------|--------------------------------------------------------------------------|----------------------------------------------------------------------|
| <b>HBM (Human Body Model), ANSI/ESDA/JEDEC JS-001</b>     | Charged person touching a pin                            | 100 pF through 1.5 kohm; ~2-10 ns rise, ~150 ns decay; ~0.67 A per kV    | 1-2 kV (industry councils argue 1 kV is safe with basic ESD control) |
| <b>CDM (Charged Device Model), ANSI/ESDA/JEDEC JS-002</b> | The package/die itself charged, discharging from one pin | Very fast: sub-ns rise, peak currents of amperes, total duration ~1-2 ns | ~250-500 V; large packages see higher currents                       |

| Model                               | What it represents            | Circuit / waveform                   | Typical component-level target                                      |
|-------------------------------------|-------------------------------|--------------------------------------|---------------------------------------------------------------------|
| <b>MM (Machine Model)</b>           | Charged tool                  | 200 pF, ~0 ohm                       | Obsolete for qualification; still seen in old specs                 |
| <b>IEC 61000-4-2 (system level)</b> | ESD into the finished product | 150 pF / 330 ohm, up to 8 kV contact | Board-level (TVS diodes); on-chip only for exposed pins such as USB |

Table 12.5 - ESD models. Exact targets are set by the product specification and the foundry's ESD guidelines.

[illegible]

Listing 12.3 - HBM waveform model.

V=2000 V: I<sub>peak</sub>=1.22 A (V/R=1.33 A), rise<sub>10-90</sub>=8.6 ns, decay(1/e)=150 ns  
V=4000 V: I<sub>peak</sub>=2.44 A (V/R=2.67 A), rise<sub>10-90</sub>=8.6 ns, decay(1/e)=150 ns  
RC time constant = 150 ns; energy 0.5CV<sup>2</sup> at 2 kV = 200 uJ

Output of hbm.py: a 2 kV HBM zap pushes about 1.2-1.3 A for ~150 ns through the protection network. The rise time depends on the assumed inductance; JS-001 specifies acceptable ranges for the tester waveform.

## The protection network

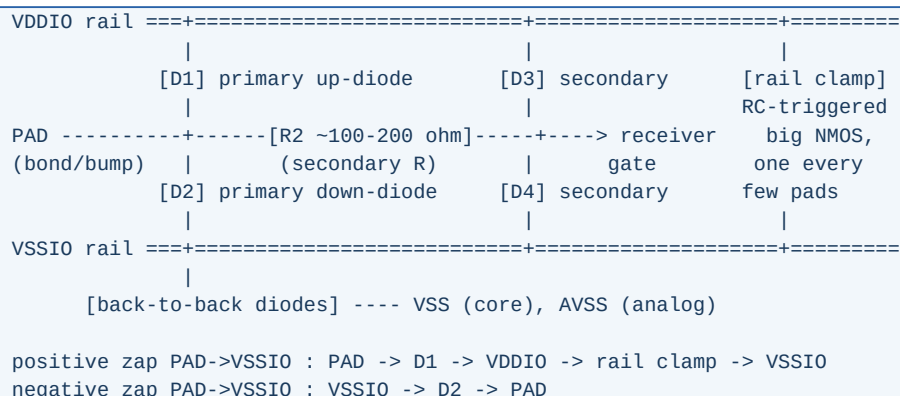


Figure 12.6 - Dual-diode protection with a power-rail clamp. Every pin-to-pin combination must have a low-impedance path through diodes and clamps; current paths across domains go through the ground diodes and the clamps of both domains.

- **Primary diodes** steer the current to the rails; the **rail clamp** is a large NMOS whose gate is driven by an RC detector: an ESD edge (ns) turns it on, a normal power-up (microseconds to milliseconds) does not. The RC time constant is typically chosen around a microsecond - long compared with HBM, short compared with power-up.
- The **secondary** stage (small resistor + small diodes/grounded-gate NMOS at the receiver) limits the voltage that reaches the thin gate oxide during fast CDM events.
- **Bus resistance matters**: the discharge path through the ring rails and clamps must be low-ohmic, so clamp spacing and rail widths are specified by the I/O library (for example 'one clamp every N pads' and a maximum resistance from any pad to its clamp).
- **Cross-domain CDM**: a signal crossing between two core supply domains can see a large voltage difference during CDM; the receiver gate needs local protection (a small clamp or resistor and diodes). Foundry ESD rule decks and dedicated tools (e.g. Calibre PERC, Pathfinder) check these paths at chip level (Chapter 18).

## Latch-up

A CMOS inverter contains a parasitic PNP thyristor (the PMOS source, n-well, p-substrate and NMOS source form a vertical PNP and a lateral NPN). If a transient injects enough current to forward-bias a well or substrate junction, the thyristor can latch into a low-impedance state between VDD and VSS and destroy the chip. Prevention is by layout: **well and substrate taps** close to every transistor (the tap cells of Chapter 14; the maximum tap distance is a foundry rule), **guard rings** around I/O devices and anything that can inject current, and spacing between NMOS and PMOS in I/O areas. Qualification follows JEDEC JESD78 (current injection of typically  $\pm 100$  mA into I/O pins and an over-voltage test on supplies, at maximum operating temperature for the higher classes).

## Simultaneous switching output (SSO) noise

When many output drivers switch together, their current flows through the inductance of the I/O supply bonds, balls and package planes, and the local VSSIO bounces up (ground bounce) while VDDIO droops.

$$V_{\text{bounce}} \sim L_{\text{eff}} * N * dI/dt$$

example: 16 drivers, each 20 mA in 1 ns,  $L_{\text{eff}} = 0.5$  nH (one bond/ball pair shared)

$$V = 0.5e-9 * 16 * 0.02 / 1e-9 = 0.16 \text{ V} \rightarrow \text{add supply pairs, slow the edges}$$

Equation 12.2 - Ground bounce. Real analysis uses IBIS models and package models; this estimate explains the 'N signal pads per supply pair' rule.

### WHO OWNS WHAT

The I/O library and ESD cells come from the foundry or IP vendor; the **pad-ring plan** (pad order, supply pads, domain breaks, clamps) is owned by the chip-level physical design team together with the package team; the **ESD sign-off** (all pin-pair paths, cross-domain checks) is usually owned by a chip-level ESD/reliability engineer using the foundry's ESD rules. Board-level SSO/SI is owned by the package/board SI team using IBIS models from the I/O library.

## 12.9 Summary

- Every SoC integrates PLLs/DLLs, oscillators, a bandgap, regulators, a POR, data converters, sensors and high-speed PHYs; the digital team owns their floorplan, supplies, timing interfaces and verification.
- A charge-pump PLL is a feedback loop (PFD, charge pump, loop filter, VCO, divider); bandwidth, damping and phase margin follow from  $I_{cp}$ ,  $K_{vco}$ ,  $N$  and the filter. PLL jitter enters STA as clock uncertainty.
- Analog design is schematic, SPICE over corners and Monte Carlo, custom layout and extraction; SoC verification uses real-number models validated against transistor-level simulation.
- Hard macros must come with GDS, CDL, LEF, Liberty, a behavioural model, UPF and an integration guide; digital-on-top vs analog-on-top defines who owns the top level.
- Noise isolation uses separate supplies, distance, guard rings, deep n-well, keep-outs and shielding.
- I/O cells shift levels and drive the board; the pad ring includes signal, supply, corner, filler and break cells. ESD protection (diodes, RC rail clamps, secondary stages) is designed against HBM and CDM; latch-up is prevented by taps and guard rings; SSO limits the number of signal pads per supply pair.

## 12.10 Exercises

- 1 In pll.py, double  $I_{cp}$ . Predict how  $\omega_n$ ,  $\zeta$  and the unity-gain frequency change, then verify. What happens to the phase margin if  $C_2$  is increased to 20 pF?
- 2 A PLL datasheet quotes 3 ps RMS period jitter. What peak-to-peak value would you use in `set_clock_uncertainty` for setup, and what else would you add to it?
- 3 Write a real-number model of an LDO whose output follows its input minus a 150 mV dropout and is 0.0 when disabled. How would you validate it against the schematic?
- 4 Draw the ESD current path for a positive HBM zap from a GPIO pad to a core VSS pin in a chip whose core and I/O grounds are separate but connected by back-to-back diodes. Which elements carry the current?

- 5 An interface has 32 outputs at 12 mA with 0.8 ns edges. The I/O library recommends at most 150 mV of bounce and each VSSIO ball contributes 0.8 nH. How many VSSIO balls are needed if only half the outputs switch in the same direction at once?

## 13. Physical Design Overview: Data, Flows and Design Planning

Physical design (PD) - also called implementation or the back end - turns a gate-level netlist into the geometric shapes of every transistor, wire and via on every mask layer, while meeting timing, power, signal-integrity, reliability and manufacturability constraints. This chapter is the map for Part IV: the steps, the data that flows between them, how big designs are split into manageable pieces, and how the die size is estimated long before anything is placed. Chapter 14 then begins the actual implementation with the floorplan.

### 13.1 From netlist to GDSII

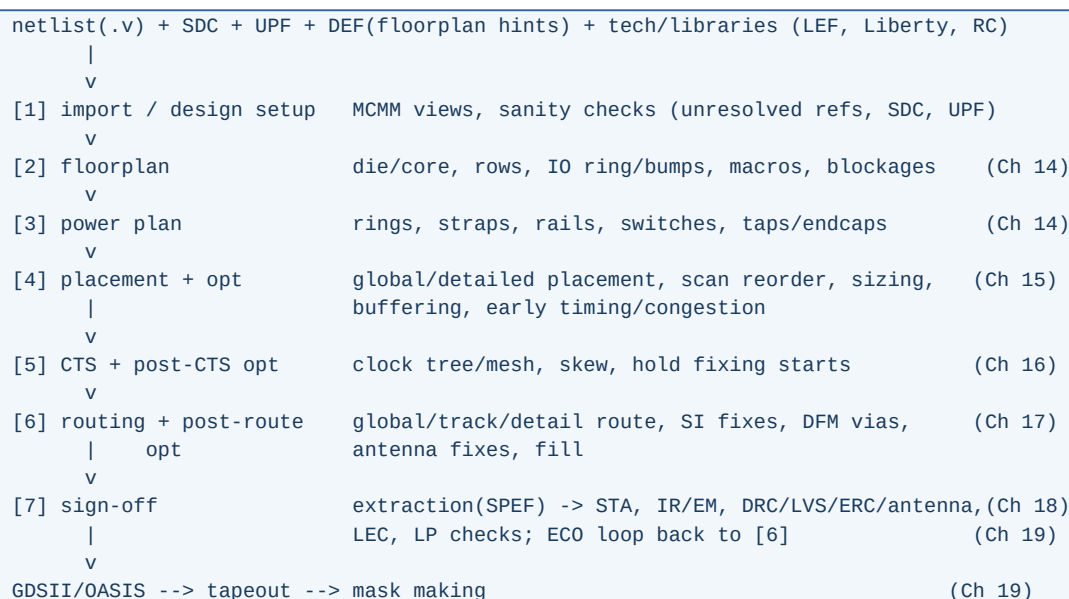


Figure 13.1 - The implementation flow. In practice steps 4-6 are iterated many times, and sign-off findings feed back through ECOS.

Modern tools blur the steps: placement runs with trial routing and clock estimates, CTS is concurrent with optimisation, and 'physically aware' synthesis (Genus iSpatial, Fusion Compiler, DC Topographical/NXT) places cells during synthesis so the netlist handed to PD already reflects real wire lengths. The conceptual sequence above is still how engineers reason about problems and how flows are scripted.

### 13.2 Inputs: the data PD needs

| Input                         | Format                                                      | Content                                                                        | Provided by                  |
|-------------------------------|-------------------------------------------------------------|--------------------------------------------------------------------------------|------------------------------|
| <b>Netlist</b>                | Verilog (structural)                                        | Instances of library cells and macros, connectivity, hierarchy                 | Synthesis / DFT team         |
| <b>Timing constraints</b>     | SDC (Tcl)                                                   | Clocks, generated clocks, I/O delays, exceptions, modes                        | Front-end + STA team         |
| <b>Power intent</b>           | UPF (IEEE 1801)                                             | Domains, supplies, strategies, states (Chapter 11)                             | Power architect              |
| <b>Technology LEF</b>         | LEF                                                         | Layers, directions, pitches, widths, spacing rules, vias, sites, antenna rules | Foundry PDK / library vendor |
| <b>Cell / macro LEF</b>       | LEF                                                         | Abstract of each cell: size, site, pins with shapes/layers, obstructions       | Library/IP vendors           |
| <b>Timing/power libraries</b> | Liberty (.lib, compiled .db / .ldb), with CCS/ECSM/LVF data | Delay/transition tables, constraints, power, pg_pins, per PVT corner           | Library/IP vendors           |



| Input                         | Format                                                                         | Content                                               | Provided by              |
|-------------------------------|--------------------------------------------------------------------------------|-------------------------------------------------------|--------------------------|
| <b>RC extraction data</b>     | QRC techfile (Cadence), TLU+ + layer map (Synopsys), captables; ITF/ICT source | Per-layer R and C vs width/spacing for each RC corner | Foundry PDK              |
| <b>Floorplan / partitions</b> | DEF, Tcl, or tool DB                                                           | Die/core, pins, macro positions from planning         | PD lead / top-level team |
| <b>Physical rules</b>         | DRC/LVS/antenna/fill decks (Calibre SVRF, Pegasus, ICV)                        | Sign-off rule checks                                  | Foundry PDK              |
| <b>Parasitics of blocks</b>   | SPEF                                                                           | Extracted RC of lower blocks for top-level timing     | Block PD teams           |

Table 13.1 - PD inputs. Liberty, LEF and the technology files are described in detail in Chapters 3 and 4 and Appendix A.

Here is what a real technology LEF says about the metal stack of the open SKY130 process (a 130 nm, five-metal-plus-local-interconnect process). A short script reads the routing layers from the file used by the open-source flows:

```
# techlef.py - summarise routing layers of the SKY130 HD technology LEF
import re
t = open("sky130_fd_sc_hd.tlef").read()
print("layer dir pitch min_w thick Rsq(ohm) R/um@min_w EM Iavg(mA/um)")
for m in re.finditer(r"LAYER (\w+)\s+TYPE ROUTING ;(.*?)END \1", t, re.S):
    n, b = m.group(1), m.group(2)
    g = lambda k: re.search(k + r"\s+([\d.E-]+)", b)
    d = re.search(r"DIRECTION (\w)", b).group(1)
    w = float(g("WIDTH").group(1)); r = float(g("RESISTANCE RPERSQ").group(1))
    em = g("DCCURRENTDENSITY AVERAGE")
    print(...)      # name, dir, PITCH, WIDTH, THICKNESS, Rsq, Rsq/w, EM
```

Listing 13.1 - Reading the tech LEF.

| layer | dir | pitch | min_w | thick | Rsq(ohm) | R/um@min_w | EM Iavg(mA/um) |
|-------|-----|-------|-------|-------|----------|------------|----------------|
| li1   | V   | 0.46  | 0.17  | 0.1   | 12.2000  | 71.765     | -              |
| met1  | H   | 0.34  | 0.14  | 0.35  | 0.1250   | 0.893      | 2.8            |
| met2  | V   | 0.46  | 0.14  | 0.35  | 0.1250   | 0.893      | 2.8            |
| met3  | H   | 0.68  | 0.30  | 0.8   | 0.0470   | 0.157      | 6.8            |
| met4  | V   | 0.92  | 0.30  | 0.8   | 0.0470   | 0.157      | 6.8            |
| met5  | H   | 3.4   | 1.60  | 1.2   | 0.0285   | 0.018      | 10.17          |

Output of techlef.py on the SKY130 HD technology LEF (units: um, ohm/square, ohm/um; EM limits are the LEF's average-current density at  $T_j = 90^\circ\text{C}$ ). The same file defines the standard-cell site `unithd`, 0.46 x 2.72 um.

Note the pattern that holds in every process: thin, dense lower layers for local routing (high resistance per um), thick, wide upper layers for power and long global wires (low resistance). Directions alternate (H/V) so that adjacent layers route orthogonally. Advanced nodes have 10-20 metal layers in several pitch classes (1x, 2x, 4x...) and add rules such as coloring for multi-patterning (Chapter 3).

### 13.3 Outputs and databases

| Output                               | Format                                    | Consumer                                                     |
|--------------------------------------|-------------------------------------------|--------------------------------------------------------------|
| <b>Layout</b>                        | GDSII or OASIS (stream)                   | Physical verification, merge with foundry cells, mask making |
| <b>Netlist (with and without PG)</b> | Verilog                                   | LVS (PG netlist), STA, LEC, gate-level simulation            |
| <b>Placement/routing</b>             | DEF (Design Exchange Format)              | Other tools, top-level integration, open-source flows        |
| <b>Parasitics</b>                    | SPEF (IEEE 1481) from sign-off extraction | STA, power, IR drop                                          |
| <b>Delays</b>                        | SDF (IEEE 1497)                           | Timing-annotated gate-level simulation                       |

| Output              | Format                         | Consumer                                                                 |
|---------------------|--------------------------------|--------------------------------------------------------------------------|
| <b>Block models</b> | LEF abstract, Liberty ETM, ILM | Top-level implementation and STA                                         |
| <b>Reports</b>      | text/HTML                      | Sign-off reviews: timing, power, IR/EM, DRC/LVS, utilisation, congestion |

Table 13.2 - PD outputs.

Each tool keeps the design in its own database: Cadence Innovus uses its native database and can read/write **OpenAccess (OA)**, the common database shared with Virtuoso for mixed-signal work; Synopsys IC Compiler II and Fusion Compiler use **NDM** (New Data Model) libraries; OpenROAD uses **OpenDB**, which is modelled on LEF/DEF. Between companies and tools, LEF/DEF/Verilog/SDC/SPEF are the lingua franca.

## A DEF excerpt, explained

```

VERSION 5.8 ;
DESIGN mac_rf ;
UNITS DISTANCE MICRONS 1000 ;                               # 1 DB unit = 1 nm
DIEAREA ( 0 0 ) ( 180000 180000 ) ;                           # 180 x 180 um
ROW ROW_0 unithd 5520 10880 N DO 367 BY 1 STEP 460 0 ;      # site, origin, orient, count
ROW ROW_1 unithd 5520 13600 FS DO 367 BY 1 STEP 460 0 ;      # flipped: shares rails
TRACKS X 230 DO 391 STEP 460 LAYER met2 ;                   # routing grid, per layer
TRACKS Y 170 DO 529 STEP 340 LAYER met1 ;
COMPONENTS 2163 ;
- _1034_ sky130_fd_sc_hd__nand2_1 + PLACED ( 41400 27200 ) FS ;
- acc_reg_7_ sky130_fd_sc_hd__dfrtp_1 + PLACED ( 52900 29920 ) N ;
- TAP_12 sky130_fd_sc_hd__tapvpwrvgn_1 + FIXED ( 12420 10880 ) N ;
END COMPONENTS
PINS 118 ;
- clk + NET clk + DIRECTION INPUT + USE CLOCK
  + LAYER met3 ( -150 -150 ) ( 150 150 ) + PLACED ( 0 90100 ) N ;
END PINS
SPECIALNETS 2 ;
- VPWR ( * VPWR ) + USE POWER
  + ROUTED met4 1600 + SHAPE STRIPE ( 20000 5000 ) ( 20000 175000 ) ;
END SPECIALNETS
NETS 1702 ;
- n_415 ( _1034_ Y ) ( acc_reg_7_ D )
  + ROUTED met1 ( 41630 27370 ) ( 52900 * ) M1M2_PR ( 52900 30090 ) ;
END NETS
END DESIGN

```

Listing 13.2 - A hand-written DEF excerpt (not tool output) using SKY130 names. # comments are added for explanation; DEF uses them too.

- **UNITS** sets database units per micron; every coordinate is an integer in these units.
- **ROW** lines define placement rows built from the site in the LEF; alternate rows are flipped (N / FS) so neighbouring rows share a VDD or VSS rail.
- **TRACKS** define the routing grid on each layer (start, count, pitch); detailed routers put wires on tracks.
- **COMPONENTS** are instances with a status: UNPLACED, PLACED (movable), FIXED (tool must not move) or COVER (physical only).
- **SPECIALNETS** hold power/ground and other pre-routed nets (shapes such as STRIPE, RING, FOLLOWPIN); **NETS** hold signal routing as wire segments and vias; \* repeats the previous coordinate.

## 13.4 Flat vs hierarchical implementation

A block of a few million instances can be implemented flat in one tool run. A modern SoC with hundreds of millions of instances cannot: run time, memory and team parallelism force a **hierarchical** flow, where the chip is partitioned into blocks (partitions, tiles) that are implemented independently and then assembled at the top.

|                           | Flat                                                                         | Hierarchical                                                                                                    |
|---------------------------|------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| <b>Capacity</b>           | Up to roughly several million instances per run (tool and machine dependent) | Unlimited: each block within capacity                                                                           |
| <b>QoR</b>                | Best: global optimisation across boundaries                                  | Boundary paths constrained by budgets; some pessimism                                                           |
| <b>Schedule</b>           | One team, serial                                                             | Parallel teams; blocks can close at different times; reuse of identical blocks (instantiate a CPU tile 8 times) |
| <b>ECO / late changes</b> | Whole-chip rerun                                                             | Only the affected block                                                                                         |
| <b>Overheads</b>          | -                                                                            | Budgeting, pin assignment, feedthroughs, abstracts, top-level integration and sign-off                          |

Table 13.3 - Flat vs hierarchical implementation.

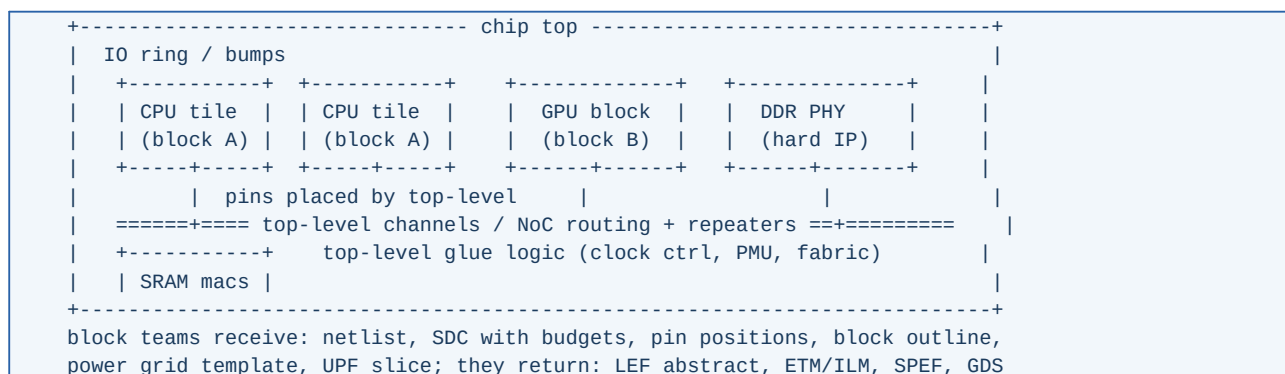


Figure 13.2 - Hierarchical implementation. Channel-less (abutted) floorplans route top-level nets through the blocks as feedthroughs instead of using channels.

## Partitioning, pin assignment and budgeting

- **Partitioning** follows the logical hierarchy where possible (ownership, reuse, UPF domains), sized so each block fits the tool and the schedule; highly connected logic stays together.
- **Pin assignment** places block pins where the top-level route wants them, aligned between neighbouring blocks for abutment.
- **Timing budgeting** splits the clock period of every cross-block path between the blocks and the top-level route: e.g. for a 1 ns path launched in block A and captured in block B through 300 ps of top-level wire, A gets an output delay and B an input delay that leave each block a fair share. Budgets are written as SDC `set_input_delay` / `set_output_delay` on the block ports and refined as blocks mature.
- **Block-level vs top-level**: blocks close their internal paths and their budgets; the top-level team closes the interface paths with the actual block timing models and the real top-level routing.

## Block models: abstracts, ETM and ILM

| Model                               | What it contains                                                                                                                                                             | Pros / cons                                                                         |
|-------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| <b>LEF abstract</b>                 | Outline, pins, routing obstructions per layer                                                                                                                                | Physical only; tiny                                                                 |
| <b>ETM (extracted timing model)</b> | A Liberty cell for the whole block: arcs from input pins to registers (setup/hold) and from clocks to outputs, generated by STA (e.g. PrimeTime <code>extract_model</code> ) | Compact, hides IP; loses SI and some context sensitivity; one model per corner/mode |
| <b>ILM (interface logic model)</b>  | The actual netlist and parasitics of only the interface logic (first/last register stages and the logic between them and the ports)                                          | Accurate, supports SI and OCV; bigger than ETM                                      |
| <b>Full netlist + SPEF</b>          | Everything (flat STA)                                                                                                                                                        | Exact; the final sign-off is often run flat on large servers                        |

Table 13.4 - Block models used at the top level.

## 13.5 Design planning and die-size estimation

Before RTL is final, the PD lead must commit to a die size (the package, the wafer cost and the number of dies per wafer depend on it; Chapter 23). The estimate combines the standard-cell area from synthesis (or from gate-count estimates), the hard-macro areas from the memory compiler and IP datasheets, a target **utilisation**, and the I/O ring or bump array.

```
core_area = std_cell_area / utilisation + sum(macro_area incl. halos)
die_side  = max( sqrt(core_area / AR) + 2 * io_ring_depth ,
                pads_per_side * pad_pitch + 2 * corner )    (wire-bond, in-line pads)
utilisation = std_cell_area / (core_area - macro_area) : typically 60-80%
```

Equation 13.1 - First-order die-size model (AR = aspect ratio). Routing-limited designs (many layers of dense wiring) need lower utilisation; regular datapaths can go higher.

To feed the model with a real number, a small MAC and 16x32 register file tile was synthesised with Yosys 0.33 against the SKY130 HD typical Liberty, with the low-power special cells removed from the library as Chapter 11 recommended:

```
// mac_rf.v: 16x32 register file + registered 16x16 multiply + 48-bit accumulate
module mac_rf (input clk, input rst_n, input we, input [3:0] wa, ra0, ra1,
               input [31:0] wd, input acc_clr, output reg [47:0] acc);
  reg [31:0] rf [0:15];
  reg [15:0] a_q, b_q; reg [31:0] p_q;
  always @(posedge clk) if (we) rf[wa] <= wd;
  always @(posedge clk or negedge rst_n)
    if (!rst_n) begin a_q <= 0; b_q <= 0; p_q <= 0; acc <= 0; end
    else begin
      a_q <= rf[ra0][15:0]; b_q <= rf[ra1][15:0];
      p_q <= a_q * b_q;
      acc <= acc_clr ? 48'd0 : acc + {16'd0, p_q};
    end
endmodule
# syn.py (Yosys): read_verilog mac_rf.v; synth -top mac_rf -flatten; memory_map
# dfflibmap -liberty hd_trim.lib; abc -liberty hd_trim.lib; opt_clean
# stat -liberty hd_trim.lib
```

Listing 13.3 - The tile and the Yosys script (hd\_trim.lib = the SKY130 HD tt\_025C\_1v80 Liberty with the 50 lpflow/probe/delay cells removed).

```
Number of cells:                2163
sky130_fd_sc_hd__a21oi_1        67
sky130_fd_sc_hd__a22oi_1        64
sky130_fd_sc_hd__dfrrtp_1       112
sky130_fd_sc_hd__dfxtp_1        256
sky130_fd_sc_hd__maj3_1         112
sky130_fd_sc_hd__mux2_1         256
sky130_fd_sc_hd__mux4_2         160
sky130_fd_sc_hd__nand2_1        166
sky130_fd_sc_hd__nor2_1         156
sky130_fd_sc_hd__xnor2_1        213
sky130_fd_sc_hd__xor2_1         203
... (28 further cell types with fewer instances)
Chip area for module '\mac_rf': 23825.350400
```

Excerpt of the Yosys **stat -liberty** report (real run; the '...' line is an elision). Area in  $\mu\text{m}^2$ . The 256 dfxtp + 256 mux2 are the register file (flops with recirculating write-enable muxes); maj3/xor/xnor are the adder cells of the multiplier and accumulator.

Flip-flops alone are  $112 \times 25.02 + 256 \times 20.02 = \text{about } 7,930 \mu\text{m}^2$ , one third of the tile - typical of register-heavy logic and a reason register files of this size are often replaced by a latch array or an SRAM macro. Now scale up to an accelerator of 64 such tiles plus glue logic, four SRAM macros and a PLL, and check both the core-limited and the pad-limited die size:

```
# die_size.py - core-limited vs pad-limited die estimate
tile_area = 23825.35          # um^2, parsed from the Yosys log above
std_area = tile_area * 64 * 1.25  # 64 tiles + 25% glue (control, DFT, CTS buffers)
macros = [("sram_8kx32", 4, 620.0, 410.0), ("pll", 1, 300.0, 300.0)] # w, h (um)
halo, util, aspect = 10.0, 0.65, 1.0
io_ring, n_pads, pad_pitch, corner = 250.0, 180, 80.0, 250.0
macro_area = sum(c*(w+2*halo)*(h+2*halo) for _, c, w, h in macros)
core_area = std_area/util + macro_area
core_w = math.sqrt(core_area/aspect)
die_core = core_w + 2*io_ring          # core-limited die side
side_pad = n_pads/4*pad_pitch + 2*corner # pad-limited die side
```

Listing 13.4 - Die-size estimate (macro sizes and pad parameters are assumed example values; print statements elided).

```
std-cell area (synthesis x 64 tiles x 1.25) = 1.906 mm^2
macro area incl. halos = 1.203 mm^2
core area @ 65% utilisation = 4.136 mm^2 (2034 x 2034 um)
die side, core-limited = 2534 um
die side, pad-limited (180 pads @ 80 um) = 4100 um
=> die 4100 x 4100 um = 16.81 mm^2, pad-limited
in-line 80 um : pad side 4100 um -> die 16.81 mm^2 (pad-limited)
staggered 2-row : pad side 2300 um -> die 6.42 mm^2 (core-limited)
util 0.55 -> core 4.67 mm^2, core-limited die 7.08 mm^2
util 0.65 -> core 4.14 mm^2, core-limited die 6.42 mm^2
util 0.75 -> core 3.74 mm^2, core-limited die 5.93 mm^2
util 0.85 -> core 3.45 mm^2, core-limited die 5.55 mm^2
```

Output of die\_size.py. With 180 in-line pads the die would be 2.6x larger than the logic needs; staggered pads (or flip-chip) make it core-limited again.

#### PAD-LIMITED VS CORE-LIMITED

A **pad-limited** die is larger than its logic requires because the perimeter must hold all the pads; the empty core area is wasted silicon (often filled with extra decap). A **core-limited** die is sized by its contents. Remedies for pad limitation: staggered or multi-row pads, fewer pads (serial interfaces instead of wide parallel buses), or flip-chip with an area bump array, where pad count scales with area instead of perimeter.

### Other planning decisions

- **Aspect ratio:** close to 1 unless the package, a wide PHY edge or a reticle-stitching plan demands otherwise; extreme aspect ratios lengthen global wires and clock trees.
- **Macro count and shapes:** memory compilers offer many configurations (words x bits x mux factor, banking); the choice changes both area and aspect (a tall thin SRAM may fit a channel better).
- **Metal stack:** how many layers (cost) and which are reserved for power, clock and top-level buses.
- **Package and bump/ball plan:** co-designed with the package team from the start (Chapter 20).
- **Utilisation targets per block:** lower for congested, timing-critical or heavily multi-voltage blocks.

## 13.6 MCMM: multi-corner, multi-mode setup

A design must meet timing in several **modes** (functional, scan shift, scan capture, low-power modes, each with its own SDC) at several **corners** (process/voltage/temperature for cells combined with RC corners for wires). Each mode-corner pair is a **scenario** or **analysis view**. P&R optimises all active scenarios concurrently: setup views at slow corners, hold views at fast corners (and at slow corners too, because hold can fail anywhere with OCV).

```
# mmmc.tcl - Innovus-style MMMC view definition (example, not run)
create_library_set -name LIB_SS -timing {lib/ss_0p72v_125c.lib lib/sram_ss.lib}
create_library_set -name LIB_FF -timing {lib/ff_0p88v_m40c.lib lib/sram_ff.lib}
create_rc_corner -name RC_CW -qrc_tech tech/cworst.qrc -temperature 125
create_rc_corner -name RC_CB -qrc_tech tech/cbest.qrc -temperature -40
create_rc_corner -name RC_RCW -qrc_tech tech/rcworst.qrc -temperature 125
create_delay_corner -name DC_SS_CW -library_set LIB_SS -rc_corner RC_CW
create_delay_corner -name DC_SS_RCW -library_set LIB_SS -rc_corner RC_RCW
create_delay_corner -name DC_FF_CB -library_set LIB_FF -rc_corner RC_CB
create_constraint_mode -name FUNC -sdc_files {sdc/func.sdc}
create_constraint_mode -name SHIFT -sdc_files {sdc/scan_shift.sdc}
create_analysis_view -name func_ss_cw -constraint_mode FUNC -delay_corner DC_SS_CW
create_analysis_view -name func_ss_rcw -constraint_mode FUNC -delay_corner DC_SS_RCW
create_analysis_view -name func_ff_cb -constraint_mode FUNC -delay_corner DC_FF_CB
create_analysis_view -name shift_ff_cb -constraint_mode SHIFT -delay_corner DC_FF_CB
set_analysis_view -setup {func_ss_cw func_ss_rcw} -hold {func_ff_cb shift_ff_cb}
```

Listing 13.5 - MMMC setup. Synopsys tools express the same idea with create\_mode / create\_corner / create\_scenario; OpenROAD reads multiple Liberty corners with define\_corners. Corner names and file names here are placeholders.

| RC corner               | Meaning                                            | Typically worst for                                       |
|-------------------------|----------------------------------------------------|-----------------------------------------------------------|
| <b>Cworst / Cbest</b>   | Maximum / minimum total capacitance                | Short, load-dominated paths (Cworst: setup); hold (Cbest) |
| <b>RCworst / RCbest</b> | Maximum / minimum R*C product (thin, narrow wires) | Long wire-dominated paths (RCworst: setup)                |
| <b>Typical</b>          | Nominal                                            | Power and typical-performance reporting                   |
| <b>CCworst variants</b> | Maximum coupling capacitance                       | SI / crosstalk analysis                                   |

Table 13.5 - Interconnect corners. Temperature inversion at advanced nodes (cells slower when cold at low voltage) means setup must be checked cold as well as hot.

#### PITFALL: TOO MANY SCENARIOS, OR THE WRONG ONES

Every active scenario costs run time and memory. Teams start with a small dominant set (e.g. 2 setup + 2 hold) during placement and add the full sign-off set (often dozens) for post-route optimisation. Dropping a scenario that later turns out to be limiting (a low-voltage DVFS point, a scan mode at a fast corner) is a classic cause of late surprises. Review the scenario list with the STA sign-off owner.

## 13.7 The PD tool landscape

| Task                              | Cadence          | Synopsys                                             | Siemens / others | Open source            |
|-----------------------------------|------------------|------------------------------------------------------|------------------|------------------------|
| <b>Physically aware synthesis</b> | Genus (iSpatial) | Fusion Compiler, DC NXT                              | -                | Yosys + ABC            |
| <b>Place and route</b>            | Innovus          | IC Compiler II, Fusion Compiler                      | Aprisa (Siemens) | OpenROAD               |
| <b>Extraction</b>                 | Quantus          | StarRC                                               | Calibre xACT     | OpenRCX                |
| <b>STA</b>                        | Tempus           | PrimeTime                                            | -                | OpenSTA                |
| <b>Power / IR / EM</b>            | Voltus           | PrimePower, RedHawk-SC (Ansys, now part of Synopsys) | mPower           | PDNSim (OpenROAD)      |
| <b>Physical verification</b>      | Pegasus          | IC Validator                                         | Calibre          | Magic, KLayout, Netgen |
| <b>Equivalence</b>                | Conformal        | Formality                                            | -                | Yosys eqy (limited)    |

Table 13.6 - Main tools by task (product names change; check current vendor offerings). Chapter 24 walks through the open-source flow.

## 13.8 Teams and hand-offs

| From                   | To                 | Deliverables                                                      | Acceptance checks                                                   |
|------------------------|--------------------|-------------------------------------------------------------------|---------------------------------------------------------------------|
| <b>RTL / front-end</b> | Synthesis          | Frozen RTL, SDC, UPF, file list                                   | Lint, CDC, UPF lint clean; SDC reviewed                             |
| <b>Synthesis + DFT</b> | PD                 | Scan-inserted netlist, SDC per mode, UPF, scan DEF, reports       | LEC RTL vs netlist; timing with reasonable slack; no unmapped cells |
| <b>PD lead</b>         | Block PD teams     | Floorplan/outline, pins, budgets, power-grid spec, partition UPF  | Agreed in a floorplan review                                        |
| <b>Block PD</b>        | Top-level PD / STA | LEF, ETM/ILM, netlist, SPEF, GDS, reports                         | Block sign-off checklist (Appendix A)                               |
| <b>Top-level PD</b>    | Sign-off / tapeout | Final GDS/OASIS, netlists, SPEF, all sign-off reports and waivers | Tapeout checklist (Chapter 19)                                      |
| <b>PD + package</b>    | Package team       | Bump/pad coordinates, die size, power map                         | Package-die co-design review (Chapter 20)                           |

Table 13.7 - Typical hand-offs around physical design.

### INTERVIEW INSIGHT: 'WHAT DO YOU CHECK FIRST WHEN YOU GET A NEW NETLIST?'

A strong answer: import it and look for unresolved references and black boxes, multiply-driven or floating nets, assign statements and tie-offs, dont\_use or unexpected cells, scan chain presence, the cell count/area against the synthesis report, and run a quick zero-wire-load and placed timing check with the SDC to find missing clocks, unconstrained endpoints and obviously infeasible paths before investing hours in placement.

## 13.9 Summary

- PD converts a netlist into mask geometry through setup, floorplan, power plan, placement, CTS, routing and sign-off, with ECO loops.
- Inputs: netlist, SDC, UPF, tech/cell LEF, Liberty per corner, RC techfiles, rule decks; outputs: GDS/OASIS, netlists, DEF, SPEF, SDF, block models and reports.
- Real LEF data show the universal metal-stack pattern: thin, resistive lower layers and thick, low-resistance upper layers for power and global routing.
- Large designs are implemented hierarchically with partitions, pin assignment, timing budgets and block models (LEF, ETM, ILM).
- Die size = max(core-limited, pad-limited); utilisation, macros with halos, aspect ratio and the I/O strategy decide it.
- MCM defines modes x corners as analysis views; choose the dominant scenarios early and the full set before sign-off.

## 13.10 Exercises

- 1 Using die\_size.py, find the utilisation at which the in-line-pad design stops being pad-limited if the pad count is reduced to 120.
- 2 Explain why the DEF in Listing 13.2 alternates N and FS rows. What would go wrong if all rows were N?
- 3 A path from block A to block B has a 1.25 GHz clock, 150 ps of top-level wire delay and 80 ps of clock uncertainty. Propose input/output delay budgets for both blocks and explain your split.
- 4 List the MMC views you would activate during placement and during post-route optimisation for a design with two DVFS points and a scan mode, at a node that shows temperature inversion.
- 5 In the SKY130 table, how much longer can a met5 wire be than a minimum-width met1 wire for the same resistance? Why is met5 not used for all signal routing?



## 14. Floorplanning, Power Planning and the I/O Ring

The floorplan is the single most leveraged decision in physical design. A good floorplan makes placement, clock tree synthesis and routing converge almost by themselves; a bad one produces congestion, long wires and timing violations that no amount of optimisation can fix, and it is expensive to change once the power grid, macros and pins are frozen. Experienced PD engineers spend days on a floorplan and review it with the architects, the package team and the power-integrity team before running a single full placement.

This chapter covers the physical skeleton of a chip or block: core and rows, the I/O ring or bump array, macro placement, blockages, the power delivery network (PDN) with its IR-drop and EM budgets, decoupling capacitance, power-switch placement, well taps and endcaps, pin assignment and the checks that say a floorplan is ready.

### 14.1 Die, core, rows and sites

```

+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| corner | IO  | IO  | IO  | IO  | IO  | IO  | IO  | IO  | corner |
+-----+-----+-----+-----+-----+-----+-----+-----+
| IO  |   core-to-IO spacing (power ring lives here)   | IO  |
+-----+-----+-----+-----+-----+-----+-----+
| IO  | | row 3 FS |VSS| cells cells cells cells | | IO  |
|    | | row 2 N  |VDD| <- rows share rails (flipped) | |   |
| IO  | | row 1 FS |VSS| +-----+ halo | | IO  |
|    | | row 0 N  |VDD| | SRAM | (no cells) | |   |
| IO  | |         +-----+ | | IO  |
|    | |         core area -----+ | |   |
+-----+-----+-----+-----+-----+-----+
| corner | IO  | IO  | IO  | IO  | IO  | IO  | IO  | IO  | corner |
+-----+-----+-----+-----+-----+-----+

```

Figure 14.1 - Die, I/O ring, core and rows. Rows are made of sites; alternate rows are flipped so VDD and VSS rails are shared.

- The **site** is the placement grid unit from the LEF: width = one placement pitch (often the contacted poly pitch), height = the cell height (in SKY130 HD `unithd` is 0.46 x 2.72 um). Every standard cell's width is an integer number of sites.
- **Rows** fill the core; each row has an orientation (N or FS). The core height is normally an integer number of rows, and the core edges are snapped to the site grid and to the manufacturing grid.
- The **core** must leave room between it and the I/O ring for the core power ring and for routing from the pads to the core pins.
- Multi-height cells (double-height flops, multi-row cells at advanced nodes) require rows to be built so that their rails line up.

```

rows      = floor(core_height / row_height)

sites/row = floor(core_width / site_width)

example SKY130 HD: core 2034 um x 2034 um -> 747 rows x 4421 sites

```

Equation 14.1 - Row arithmetic.

### 14.2 The I/O ring

In a wire-bond chip the I/O cells form a ring around the core; each signal and supply pad has a bond pad connected by a gold or copper wire to the package lead frame or substrate. Pad-limited vs core-limited sizing was quantified in Chapter 13; here is how the ring itself is built.

- **Pad order** comes from the package and board: the package team assigns balls/leads, the PD team maps them to pad positions so that bond wires do not cross and have acceptable lengths and angles.



- **Supply pads** are interleaved according to the I/O library's SSO and ESD rules (e.g. one VDDIO/VSSIO pair per N signal pads) and the core current (core VDD/VSS pads spread around the ring so the core ring is fed from all sides).
- **Corner cells** continue the pad-ring rails; **pad fillers** close the gaps so the I/O rails and the ESD bus are continuous; **break (cut) cells** separate I/O voltage banks.
- **Staggered pads** (two rows, inner and outer) halve the effective pitch at the cost of longer bond wires and bonding constraints.
- The I/O cells' core-side pins are connected to core logic with ordinary signal routing; timing to/from the pads is part of the I/O constraints (set\_input\_delay/set\_output\_delay, Chapter 9).

## Flip-chip: bumps and the redistribution layer

| top view (bump array)                                                                                                                                                                                                                                                                         | cross-section                                                                                                                                                                                                                               |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> o  o  o  o  o  o  o  o   V  G  V  G  V  G  V o  o  o  o  o  o  o  o   G  V  G  V  G  V  G S  S  o  o  o  o  S  S S  S  V  G  V  G  S  S S  S  o  o  o  o  S  S V/G: core power/ground bumps over the core S  : signal bumps near their I/O cells o  : further power/ground bumps </pre> | <pre> package substrate =====         solder bumps ----o--o--o--o----- RDL (redistribution layer, thick Al/Cu top metal) passivation + top vias ---- metal stack ----- ---- devices (face down) ---- silicon substrate (backside up) </pre> |

Figure 14.2 - Flip-chip: bumps over the whole die area. Power bumps are distributed over the core, so current enters the grid vertically close to where it is used.

- Bumps (C4 solder or copper pillar) sit on a regular array with a pitch typically in the range of about 100-200 um for package bumps (much finer for micro-bumps and hybrid bonding in 2.5D/3D, Chapter 20).
- The **RDL** (redistribution layer), a thick top metal (often aluminium), routes each bump to its I/O cell or power grid entry point. RDL routing is done by the P&R tool or a package co-design tool, with its own DRC.
- I/O cells can be placed in the core area ('area I/O') or still at the periphery; power bumps feed the grid directly from above, which makes IR drop far smaller than with a peripheral ring (see the IR-drop run later in this chapter).
- The bump map is a joint deliverable of the PD and package teams and is frozen early because the substrate design takes weeks.

## 14.3 Macro placement

Hard macros (SRAMs, ROMs, register files, PLLs, PHYs, analog IP) are big, rectangular, fixed obstacles with many pins on one or more edges. They are placed before the standard cells, usually by hand or with a macro placer followed by manual refinement.

| Guideline                                                                                                                                                   | Why                                                                              |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| Place macros at the core periphery, pins facing the core                                                                                                    | Keeps the centre free for standard cells and gives short, uncongested pin access |
| Follow the data flow: macros near the logic that uses them, and in the order data travels                                                                   | Shorter nets, less buffering and fewer long timing paths                         |
| Leave channels between macros sized for the pins and the routes through them (and for buffers)                                                              | Narrow channels with many pins become routing hot-spots                          |
| Use halos / keep-outs around macros                                                                                                                         | Room for pin access, power connections and to keep noisy cells away              |
| Orient macros so that pins face each other where they connect, and respect allowed orientations (some macros must not be rotated because of poly direction) | Legal and short connections                                                      |
| Align macros and avoid 'notches' and thin slivers of standard-cell area                                                                                     | Slivers become congested, poorly-placed regions                                  |

| Guideline                                                                                   | Why                                 |
|---------------------------------------------------------------------------------------------|-------------------------------------|
| Keep analog/sensitive macros at the die edge near their pads, away from high-activity logic | Noise isolation (Chapter 12)        |
| Group identical macros, about where the macro allows it (shared rings)                      | Area efficiency, regular power grid |

Table 14.1 - Macro-placement guidelines.

## Blockages

| Blockage                          | Effect                                                                | Typical use                                                  |
|-----------------------------------|-----------------------------------------------------------------------|--------------------------------------------------------------|
| <b>Hard placement blockage</b>    | No standard cells at all                                              | Macro halos, channels that must stay empty, analog keep-outs |
| <b>Soft placement blockage</b>    | No cells during initial placement; optimisation may put buffers there | Narrow channels between macros (buffers allowed, logic not)  |
| <b>Partial placement blockage</b> | Maximum cell density in the area (e.g. 40%)                           | Congested regions, around high-pin-density macros            |
| <b>Routing blockage</b>           | No routing on given layers                                            | Over analog macros, under bumps, for shielding               |

Table 14.2 - Placement and routing blockages.

## 14.4 Power delivery network design

The PDN carries current from the package (pads or bumps) through a ring, a mesh of wide straps on the upper metals, via stacks, and the thin **follow-pin rails** on metal 1 that run along every row to the cells' VDD/VSS pins. Its job is to keep the voltage at every cell within budget (IR drop), to keep current density in every wire and via below the electromigration limit, and to use as little routing resource as possible.

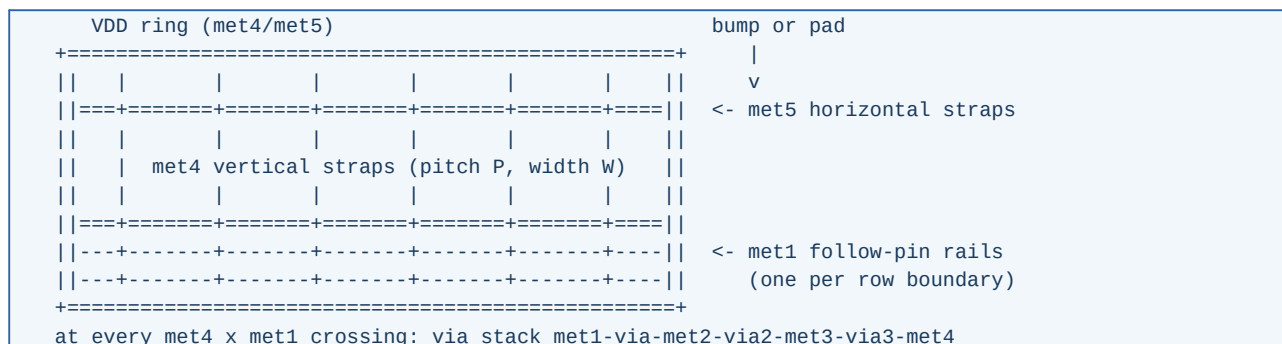


Figure 14.3 - A typical standard-cell PDN: ring, orthogonal upper-metal straps, via stacks and follow-pin rails. VSS is interleaved with VDD in the same pattern.

### Choosing pitch and width

- **Metal budget:** power straps typically take something like 10-30% of the tracks on the layers they use (more on the top layers, less on the signal-critical middle layers). Every strap track is a track signals cannot use.
- **IR drop** falls with more metal; for a mesh fed from its edges the drop depends mainly on the metal **density** (W/P), while the pitch sets the drop along the follow-pin rails between straps.
- **Via stacks** at every strap/rail crossing block routing tracks on the intermediate layers; very dense straps on a high layer multiply the via-stack blockage below it.
- **EM:** each strap segment and via array must carry its current within the average (and RMS/peak) current-density limits from the technology LEF / foundry EM rules at the target junction temperature and lifetime.
- **Alignment:** straps aligned to the placement/routing grid and (at advanced nodes) to the power-rail and via rules; standard cells often must not sit under certain strap/via positions (pin access).

## IR-drop budget

$$V_{\text{cell}} = V_{\text{pkg}} - (I * R)_{\text{package}} - (I * R)_{\text{grid}} - L * di/dt - (\text{dynamic droop})$$

typical budget: static (average-current) IR drop  $\leq \sim 2\text{-}3\%$  of VDD

dynamic (peak, time-domain) drop  $\leq \sim 5\text{-}10\%$  of VDD total

STA must then be signed off with the voltage actually seen, or with

a voltage-aware derate (IR-drop-aware STA, Chapter 18)

Equation 14.2 - IR-drop budgeting. Exact percentages are set per project; the library corner voltage (e.g. 0.72 V for a 0.8 V nominal at -10%) already includes part of the regulator tolerance.

To see how pitch, width and feed style interact, the script below solves the static IR drop of a VDD mesh numerically. It uses the real SKY130 sheet resistances and EM limits read from the technology LEF in Chapter 13: met4 vertical and met5 horizontal straps of equal width  $W$  at pitch  $P$  over a  $2 \times 2$  mm core drawing a uniform  $0.5 \text{ W/mm}^2$  ( $1.11 \text{ A}$  at  $1.8 \text{ V}$ ). The mesh is fed either by an ideal ring on all four sides (wire bond) or by ideal flip-chip bumps on a  $200 \text{ um}$  grid. Each strap crossing is a node that sinks the current of its  $P \times P$  tile; Kirchhoff's current law gives one linear equation per node, solved with numpy. The drop along the met1 rails between two straps is added analytically.

$$\text{node } k: \sum_j g_{kj} * (V_j - V_k) + g_{\text{feed}} * (VDD - V_k) = I_k, \quad I_k = J * P^2$$

$$g(\text{met4 segment}) = W / (R_{sq4} * P), \quad g(\text{met5 segment}) = W / (R_{sq5} * P)$$

$$\text{rail drop between straps (uniform load, fed at both ends)} = i' * r' * P^2 / 8$$

$$i' = J * \text{row\_height (A per um of rail)}, \quad r' = R_{sq1} / W_{\text{rail}} \text{ (ohm per um)}$$

Equation 14.3 - The mesh model (a 2D resistive network) and the rail drop.

```
# irdrop.py - static IR drop of a VDD mesh (SKY130 Rsq / EM from the tech LEF)
import numpy as np
VDD, CORE, J = 1.8, 2000.0, 0.5e-6 / 1.8 # V, um, A/um^2 (0.5 W/mm^2 at 1.8 V)
RSQ4, RSQ5, RSQ1 = 0.047, 0.0285, 0.125 # ohm/sq (met4, met5, met1)
EM4 = 6.8e-3 # A per um of met4 width (Iavg, 90 C)
ROW, W_RAIL, BUMP = 2.72, 0.48, 200.0 # row height, met1 rail width, bump pitch
def mesh_drop(P, W4, W5, feed):
    n = int(round(CORE / P)) - 1 # interior strap intersections per side
    A = np.zeros((n*n, n*n)); b = np.full(n*n, J * P * P) # node sink current
    g4, g5 = W4 / (RSQ4 * P), W5 / (RSQ5 * P) # segment conductances
    step = int(round(BUMP / P))
    for i in range(n):
        for j in range(n):
            k = i * n + j
            for di, dj, g in ((1, 0, g4), (-1, 0, g4), (0, 1, g5), (0, -1, g5)):
                ii, jj = i + di, j + dj
                if 0 <= ii < n and 0 <= jj < n:
                    A[k, k] -= g; A[k, ii * n + jj] += g
            elif feed == "ring": # tied to the ring
                A[k, k] -= g; b[k] -= g * VDD
            if feed == "bump" and (i + 1) % step == 0 and (j + 1) % step == 0:
                A[k, k] -= 1e3; b[k] -= 1e3 * VDD # ~ideal bump (1 mohm)
    V = np.linalg.solve(A, b).reshape(n, n)
    i_edge = ((VDD - V[0, :]) * g4).max() if feed == "ring" else 0.0
    return VDD - V.min(), i_edge
def rail_drop(P): # uniform load on a rail fed from both ends: i r P^2 / 8
    return J * ROW * (RSQ1 / W_RAIL) * P * P / 8
# ... sweep (P, W) for both feeds and print (see output)
```

Listing 14.1 - The IR-drop solver (the sweep and print loop is elided). The largest case has  $79 \times 79 = 6,241$  nodes; the whole sweep runs in a few seconds.

Core 2000 um square, 1.11 A total ( $0.5 \text{ W/mm}^2$  at  $1.8 \text{ V}$ )

| feed | pitch | W    | met4 | mesh  | rail | total | %VDD | ring-edge I / EM limit |
|------|-------|------|------|-------|------|-------|------|------------------------|
|      | [um]  | [um] | use% | [mV]  | [mV] | [mV]  |      | per strap [mA]         |
| ring | 400   | 8.0  | 2.0  | 131.1 | 3.9  | 135.0 | 7.5% | 40.2 / 54.4 ok         |
| ring | 200   | 8.0  | 4.0  | 71.7  | 1.0  | 72.7  | 4.0% | 25.6 / 54.4 ok         |
| ring | 100   | 8.0  | 8.0  | 36.1  | 0.2  | 36.3  | 2.0% | 14.1 / 54.4 ok         |
| ring | 50    | 8.0  | 16.0 | 18.1  | 0.1  | 18.1  | 1.0% | 7.4 / 54.4 ok          |
| ring | 400   | 40.0 | 10.0 | 26.2  | 3.9  | 30.1  | 1.7% | 40.2 / 272.0 ok        |
| ring | 200   | 20.0 | 10.0 | 28.7  | 1.0  | 29.7  | 1.6% | 25.6 / 136.0 ok        |
| ring | 100   | 10.0 | 10.0 | 28.9  | 0.2  | 29.1  | 1.6% | 14.1 / 68.0 ok         |
| ring | 50    | 5.0  | 10.0 | 28.9  | 0.1  | 29.0  | 1.6% | 7.4 / 34.0 ok          |
| bump | 100   | 8.0  | 8.0  | 2.7   | 0.2  | 3.0   | 0.2% | -                      |
| bump | 50    | 8.0  | 16.0 | 2.3   | 0.1  | 2.4   | 0.1% | -                      |
| bump | 25    | 8.0  | 32.0 | 1.6   | 0.0  | 1.6   | 0.1% | -                      |
| bump | 100   | 10.0 | 10.0 | 2.2   | 0.2  | 2.4   | 0.1% | -                      |
| bump | 50    | 5.0  | 10.0 | 3.7   | 0.1  | 3.8   | 0.2% | -                      |
| bump | 25    | 2.5  | 10.0 | 4.9   | 0.0  | 5.0   | 0.3% | -                      |

Output of irdrop.py (python3 + numpy 2.4). VDD network only; the VSS network adds a similar drop (ground bounce), so the total supply collapse is about twice these numbers.

What the run shows:

- **With a fixed strap width**, halving the pitch halves the mesh drop - but only because it doubles the metal used (2% -> 16% of met4).
- **At a fixed metal density** (10%), the ring-fed mesh drop is almost independent of pitch (about 29 mV): a fine mesh of narrow straps and a coarse mesh of wide straps have the same effective sheet resistance. The pitch then matters for the local met1 rail drop (3.9 mV at 400 um vs 0.1 mV at 50 um) and for how evenly cells are served - and a finer pitch costs more via stacks.
- **Feeding from bumps** instead of the periphery cuts the drop by roughly 10x for the same metal, because current travels at most about half a bump pitch laterally instead of half the die. This is why high-power chips are flip-chip. With bumps, finer, narrower straps at the same density show slightly *more* drop: the current crowds into the few straps next to each bump.
- **EM** is comfortably met on the met4 straps here; in real designs the limiting EM items are usually the via arrays at the ring and at the bump/pad connections and narrow lower-level straps, which this model does not include.
- Real IR analysis (Voltus, RedHawk-SC, PDNSim) extracts the whole grid including vias, uses the actual per-instance currents from power analysis, and includes the package model; this simple model is for sizing the grid before placement.

#### PITFALL: DESIGNING THE GRID FOR AVERAGE POWER ONLY

A grid that meets static IR drop with average current can still fail dynamic IR drop when a large block wakes up, when a clock starts, or during scan capture with all flops toggling in the same cycle. Size the grid with a power map that includes the realistic worst-case activity per region (from vectorless peak estimates or early FSDB) and plan decap for the transients.

## Decoupling capacitance

Decap cells (and the intrinsic capacitance of non-switching logic) act as a local charge reservoir that supplies current for the first nanoseconds of a transient, before the grid and package inductance can respond.

$$C_{\text{decap}} \geq Q / dV_{\text{allowed}} = I_{\text{step}} * t_{\text{response}} / dV_{\text{allowed}}$$

example: 0.5 A step in a region, package responds after ~1 ns,

$$\text{allowed droop } 40 \text{ mV} \rightarrow C \geq 0.5 * 1e-9 / 0.04 = 12.5 \text{ nF}$$

Equation 14.4 - Decap sizing, first order. Thin gate oxide gives on the order of 10 fF per um<sup>2</sup> of gate area (less per um<sup>2</sup> of decap cell), so 12.5 nF is on the order of a square millimetre - a real area cost, which is why decap is concentrated near high-activity regions and why MIM/deep-trench capacitors exist in some processes.

- Decap cells leak (they are large gate-oxide capacitors); use them where needed, not everywhere, and prefer thick-oxide or lower-leakage decap variants where the library offers them.
- Standard flows insert decap after placement and fill remaining gaps with decap-type filler cells in hot regions and plain fillers elsewhere.
- Pre-placed decap near clock roots, around large macros and in switched domains' always-on areas is part of the floorplan.

## 14.5 Power domains and power switches in the floorplan

Each UPF power domain (Chapter 11) becomes a physical region (voltage area / power domain region) with its own grid. Cells of a domain must be placed inside its region; always-on cells inside a switched domain need the real supply routed to their secondary pins.

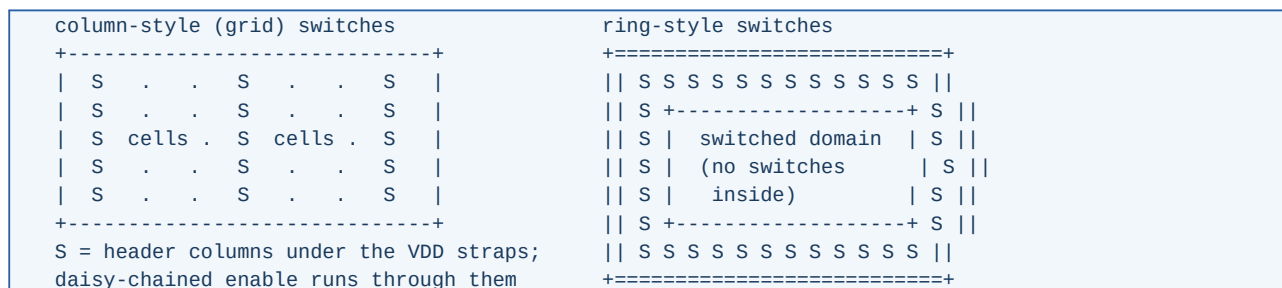


Figure 14.4 - Switch placement. Columns (a regular array under the power straps) give lower IR drop inside the domain and are the norm for standard-cell domains; rings suit macros and small domains.

- The number of switches is set by the domain's peak current and the allowed IR drop across the switches in the ON state; their chaining is set by the rush-current limit (Chapter 11).
- Level shifters and isolation cells are placed at the domain boundary (in the domain given by the UPF [-location](#)), often in dedicated rows or regions with both supplies available.
- Domain regions need spacing or a guard between them when their wells are at different potentials (different supply voltages require separate n-wells for the PMOS).

## 14.6 Well taps, endcaps and physical-only cells

- **Well-tap cells** connect the n-well to VDD and the p-substrate/p-well to VSS at regular intervals. Modern standard cells are 'tapless' (no taps inside each cell) to save area, so the flow must insert tap cells in a checkerboard at a maximum distance set by the foundry's latch-up rule; the open-source SKY130 flows place them every 13-14 um (the physical-only cell [tapvpwrvgnd\\_1](#) seen in Chapter 11).
- **Endcap (boundary) cells** terminate every row at the core and macro edges, completing the well and implant layers so that the last real cell sees a regular environment (required by DRC at advanced nodes).
- Other physical-only cells: **filler** cells (continuity of wells and rails in gaps), **decap** fillers, **tie-hi/tie-lo** cells (constant inputs never connect a gate directly to a rail, for ESD reasons), spare cells for ECOs (Chapter 19), and antenna diodes.
- All of these are inserted during floorplan (taps, endcaps) or at the end of placement/routing (fillers), are in the LEF but not (or only trivially) in Liberty, and appear in the netlist used for LVS.

## 14.7 Pin assignment

For a block, the ports must be placed on its boundary where the top-level route or the abutting neighbour expects them: on the correct layer (usually middle layers, with the preferred direction perpendicular to the edge), on the routing grid, with enough spacing, grouped by bus, and close to the logic that uses them. Top-down flows derive block pin positions from a top-level trial route; the positions are frozen early because they are shared by two teams. Clock pins are placed where the top-level clock can reach them with minimal insertion delay, and critical timing pins face their partners.

## 14.8 A floorplan script

```
# floorplan.tcl - OpenROAD-style floorplan + PDN for SKY130 HD (example, not run;
# option names vary between OpenROAD versions)
read_lef sky130_fd_sc_hd.tlef ; read_lef sky130_fd_sc_hd_merged.lef
read_liberty sky130_fd_sc_hd__tt_025C_1v80.lib
read_verilog accel.v ; link_design accel ; read_sdc accel.sdc
initialize_floorplan -die_area "0 0 2534 2534" -core_area "250 250 2284 2284" \
    -site unithd

make_tracks
place_pins -hor_layers met3 -ver_layers met2           ;# block-style pin placement
tapcell -distance 14 -tapcell_master sky130_fd_sc_hd__tapvpwrvgnd_1 \
    -endcap_master sky130_fd_sc_hd__decap_3
add_global_connection -net VDD -pin_pattern {^VPWR$} -power
add_global_connection -net VSS -pin_pattern {^VGND$} -ground
set_voltage_domain -name CORE -power VDD -ground VSS
define_pdn_grid -name core -voltage_domains CORE
add_pdn_ring -grid core -layers {met4 met5} -widths 10 -spacings 2 -core_offsets 5
add_pdn_stripe -grid core -layer met1 -width 0.48 -followpins
add_pdn_stripe -grid core -layer met4 -width 10 -pitch 100 -offset 20
add_pdn_stripe -grid core -layer met5 -width 10 -pitch 100 -offset 20
add_pdn_connect -grid core -layers {met1 met4}
add_pdn_connect -grid core -layers {met4 met5}
pdngen
```

Listing 14.2 - A floorplan and PDN script in OpenROAD Tcl (the 100 um / 10 um straps correspond roughly to the 1.6% ring-fed row of the IR-drop table). Commercial tools use equivalent commands (floorPlan, addRing, addStripe, srout in Innovus; initialize\_floorplan, create\_pg\_mesh\_pattern, compile\_pg in ICC2).

## 14.9 Floorplan quality checks

### Floorplan review checklist

- ☐ Die/core size, aspect ratio and utilisation agree with the plan; rows and sites legal.
- ☐ I/O ring or bump map matches the package; supply pads meet SSO/ESD/EM rules.
- ☐ Macros on grid, legal orientation, pins facing the core, halos and channels sized.
- ☐ No slivers or notches; standard-cell area contiguous; blockages intentional.
- ☐ Power grid complete: every macro and row connected; no floating PG shapes.
- ☐ Early static IR drop and EM on the grid within budget (with a realistic power map).
- ☐ Power domains, switches, always-on routes, level-shifter/isolation areas in place.
- ☐ Tap and endcap cells inserted; DRC clean on the floorplan (PG + macros + taps).
- ☐ Pins placed on legal layers/tracks, near their logic; feedthroughs planned.
- ☐ Trial placement + global route: congestion map and timing acceptable.

#### INTERVIEW INSIGHT: 'HOW DO YOU KNOW YOUR FLOORPLAN IS GOOD?'

Run a quick placement and global route on it and look at three things: the congestion map (hot-spots around macro corners and channels), the timing of the worst paths (are they long because of macro placement?) and the wire-length/flight-line picture between macros and logic. Then iterate the floorplan, not the optimisation settings. Also mention the power grid: early IR drop with a vectorless power map, and that the grid's track usage leaves enough routing resources.

## 14.10 Summary

- The floorplan sets die/core, rows and sites, the I/O ring or bump array, macro positions, blockages, power domains and pins; it is the most leveraged decision in PD.
- Wire-bond chips use a peripheral pad ring with corner, filler and break cells; flip-chip uses an area bump array and an RDL, and feeds power from above.
- Macros go to the periphery following the data flow, with halos, adequate channels and legal orientations; hard, soft and partial blockages shape placement.
- The PDN is rings, upper-metal straps, via stacks and follow-pin rails; the numerical mesh model shows that mesh IR drop tracks metal density, pitch controls the local rail drop, and bump feeding cuts the drop by about an order of magnitude.
- IR drop is budgeted (static around 2-3%, dynamic around 5-10% total, project-dependent) and EM limits come from the tech rules; decap supplies the first nanoseconds of transients.
- Switched domains get switch columns or rings, boundary cells and always-on routing; tap and endcap cells are inserted at floorplan; pins are assigned on legal layers near their logic.

## 14.11 Exercises

- 1 Compute the number of rows and sites per row for a 1.5 x 1.2 mm SKY130 HD core. How many tap cells are needed if one is placed every 14  $\mu\text{m}$  in every row (ignore the checkerboard offset)?
- 2 Extend irdrop.py to model the VSS grid and the package: add a 20 mohm resistance in series with every bump. How does the bump-fed drop change, and which term now dominates?
- 3 With the ring feed, find the strap width at 100  $\mu\text{m}$  pitch that keeps the total VDD drop below 1% of VDD. What fraction of met4 and met5 does it use?
- 4 A region of 0.25  $\text{mm}^2$  can see a current step of 150 mA. The package responds in 2 ns and the allowed droop is 30 mV. How much decap is needed, and what fraction of the region is that at 10 fF/ $\mu\text{m}^2$ ?
- 5 Sketch a floorplan for a block with four SRAM macros (each 600 x 400  $\mu\text{m}$ , pins on one long edge) and a datapath that reads all four every cycle. Justify your macro orientation and channel widths.
- 6 Explain why the mesh drop at 10% density with the ring feed is nearly independent of pitch, using the idea of an equivalent sheet resistance.

## PART IV

# Implementation and Sign-off

From a floorplanned block to a manufacturable mask set: placement and optimisation, clock tree synthesis, routing with signal integrity and DFM, the sign-off checks that decide whether a chip may be taped out, and the ECO, tapeout and mask-making steps that turn a database into physical photomasks.



## 15. Placement and Optimisation

Chapter 14 ended with a floorplan: a die outline, a core area cut into standard-cell rows, hard macros in fixed positions, an I/O ring and a power grid. What it does not yet contain is the bulk of the design - typically hundreds of thousands to tens of millions of standard cells that the synthesis netlist (Chapter 8) instantiates with no notion of where they will sit. **Placement** assigns every one of those cells a legal, non-overlapping location on a row site, and the **placement optimisation** that runs alongside it resizes, buffers and restructures the netlist so that, once cells are at their real positions, timing, congestion and power still close.

Placement is the step where the netlist first meets geometry, and it sets the ceiling for everything downstream. A poor placement cannot be rescued by clock tree synthesis or routing: the wires are already too long, the cells too crowded, the critical paths too spread out. That is why physical-design engineers spend far more iterations in placement than anywhere else, and why modern synthesis tools ("physical synthesis", Genus iSpatial, Fusion Compiler) run a placement engine internally before the netlist is ever handed over.

| Item                   | Content                                                                                                                                                                                                                                    |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Owner</b>           | Physical-design (PD/back-end) engineer for the block; timing sign-off engineer reviews the post-place timing reports; DFT engineer supplies scan-chain definitions.                                                                        |
| <b>Inputs</b>          | Floorplanned database (DEF/OpenDB or tool DB) with macros, blockages, power grid and pins; gate-level netlist; SDC; Liberty (.lib) for all corners; tech + cell LEF; scan DEF; UPF for multi-voltage designs; don't-touch/size-only lists. |
| <b>Outputs</b>         | Placed and optimised database; updated netlist (with new buffers and resized cells); reordered scan DEF; timing, congestion and utilisation reports that gate the move to CTS.                                                             |
| <b>Typical runtime</b> | Hours for a 1-5 M instance block on a modern multi-threaded tool; days for full-chip flat runs, which is one reason designs are partitioned into blocks (Chapter 13).                                                                      |

Placement at a glance

### 15.1 The placement problem

Formally, placement is an optimisation problem: find (x, y) for every movable cell such that cells do not overlap, every cell sits on a legal row site with the right orientation, and a cost function is minimised. The primary cost is **total wirelength**, because wire length drives wire capacitance (delay and dynamic power), routing demand (congestion) and coupling (crosstalk). Secondary costs are timing (weighted wirelength on critical nets), congestion, power (weighting high-activity nets) and, in multi-voltage designs, keeping cells inside their voltage areas.

Actual routed length is unknown until routing, so placers use cheap proxies. The industry-standard proxy is the **half-perimeter wirelength (HPWL)**: for each net, the half perimeter of the smallest rectangle enclosing all its pins. HPWL is exact for 2- and 3-pin nets routed as a Steiner tree and a good lower bound for larger nets.

$$\text{HPWL}(\text{net}) = (\max x_i - \min x_i) + (\max y_i - \min y_i) \quad \text{over pins } i \text{ of the net}$$

$$\text{Total cost} = \text{sum over nets of } w_{\text{net}} * \text{HPWL}(\text{net})$$

$w_{\text{net}} > 1$  for timing-critical nets, high-activity nets (power), clock-adjacent nets

Half-perimeter wirelength, the universal placement metric

The problem is NP-hard even for the pure wirelength objective, so every production placer is a stack of heuristics. The major algorithm families, historically and today, are:

| Family               | Idea                                                                                                 | Where it is used                                                                           |
|----------------------|------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| Min-cut partitioning | Recursively bisect the netlist and the die (Fiduccia-Mattheyses, hMETIS) so few nets cross each cut. | 1990s placers (Capo); still used for coarse clustering and hierarchical/multi-level flows. |

| Family                 | Idea                                                                                                                                                    | Where it is used                                                                                                         |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Simulated annealing    | Random swaps/moves accepted with probability $\exp(-dCost/T)$ ; temperature T slowly lowered (TimberWolf).                                              | Excellent quality on small problems; too slow for millions of cells; survives in detailed placement and macro placement. |
| Force-directed         | Nets act as springs pulling connected cells together; extra 'spreading forces' push cells out of dense regions.                                         | Conceptual basis of analytic placement (Kraftwerk).                                                                      |
| Analytic - quadratic   | Minimise $\sum w * (\text{squared distance})$ ; the optimum solves a sparse linear system $Ax = b$ . Iterate with spreading anchors (FastPlace, SimPL). | Fast global placement; initial placement in many commercial tools.                                                       |
| Analytic - nonlinear   | Smooth approximations of HPWL (log-sum-exp, weighted-average) plus a density penalty, minimised with gradient methods (NTUplace, APlace).               | Modern global placers.                                                                                                   |
| Electrostatic (ePlace) | Cells are positive charges; density overflow is the electric potential energy, solved by FFT on a grid (Poisson's equation); Nesterov gradient descent. | ePlace/RePlace - the global placer in OpenROAD; similar ideas in commercial engines.                                     |

Placement algorithm families

### Quadratic placement in one paragraph

Replace HPWL by squared Euclidean distance and model every k-pin net as a **clique** of edges with weight  $1/(k-1)$ . The cost  $\sum w_{ij} [(x_i - x_j)^2 + (y_i - y_j)^2]$  separates into x and y, and each is a convex quadratic whose minimum is where the gradient is zero:  $Ax = b_x$ ,  $Ay = b_y$ . A is a symmetric positive-definite Laplacian-like matrix (diagonal = total weight of edges at a cell, off-diagonal = minus the weight between two movable cells), and b collects the pull of fixed pins (I/O pads, macro pins). With no fixed pins A would be singular and all cells would collapse to one point - which is why I/O placement and macro placement (Chapter 14) must come first. Real solvers use conjugate gradient on millions of unknowns and replace the clique by the **Bound2Bound** net model, which reproduces HPWL much better for large nets.

The script below does exactly this for a six-cell netlist with four fixed pads, then runs a greedy legaliser that snaps cells to 10-um rows and 5-um sites without overlap. It prints the matrix, the positions and HPWL for a random placement, the quadratic optimum and the legal result.

```
# Quadratic (clique-model) placement of a tiny netlist, then row legalization
import numpy as np
rng = np.random.default_rng(7)
W, H = 100.0, 100.0 # core size in um
pads = {"IN0": (0, 20), "IN1": (0, 80), "OUT": (100, 50), "CLK": (50, 100)}
cells = ["u1", "u2", "u3", "u4", "u5", "u6"]
nets = [("IN0", "u1"), ("IN1", "u2"), ("u1", "u2", "u3"), ("u3", "u4"),
        ("u4", "u5", "u6"), ("u6", "OUT"), ("CLK", "u4", "u5"), ("u2", "u5")]
idx = {c: i for i, c in enumerate(cells)}
n = len(cells)

def hpwl(pos):
    tot = 0.0
    for net in nets:
        xs = [pos[t][0] for t in net]; ys = [pos[t][1] for t in net]
        tot += (max(xs) - min(xs)) + (max(ys) - min(ys))
    return tot

# Build A x = bx, A y = by: each k-pin net -> clique with weight 1/(k-1)
A = np.zeros((n, n)); bx = np.zeros(n); by = np.zeros(n)
for net in nets:
    w = 1.0 / (len(net) - 1)
    for i in range(len(net)):
        for j in range(i + 1, len(net)):
            a, b = net[i], net[j]
```

```

        for s, t in ((a, b), (b, a)):
            if s in idx:
                A[idx[s], idx[s]] += w
            if t in idx:
                A[idx[s], idx[t]] -= w
            else:
                bx[idx[s]] += w * pads[t][0]; by[idx[s]] += w * pads[t][1]
x = np.linalg.solve(A, bx); y = np.linalg.solve(A, by)

rand = {c: (rng.uniform(0, W), rng.uniform(0, H)) for c in cells}
rand.update(pads)
qp = {c: (x[idx[c]], y[idx[c]]) for c in cells}; qp.update(pads)

# Legalize: snap to 10-um rows, 5-um sites, cells 10 um wide, no overlap (greedy)
ROW, SITE, CW = 10.0, 5.0, 10.0
used = {}
leg = dict(pads)
for c in sorted(cells, key=lambda c: qp[c][0]):
    best = None
    for r in range(int(H // ROW)):
        yy = r * ROW + ROW / 2
        xx = max(0.0, min(W - CW, round((qp[c][0] - CW / 2) / SITE) * SITE))
        while any(abs(xx - u) < CW for u in used.get(r, [])):
            xx += SITE
        if xx > W - CW:
            continue
        d = abs(xx + CW / 2 - qp[c][0]) + abs(yy - qp[c][1])
        if best is None or d < best[0]:
            best = (d, r, xx, yy)
    d, r, xx, yy = best
    used.setdefault(r, []).append(xx)
    leg[c] = (xx + CW / 2, yy)

print("Laplacian-like matrix A (x and y share it):")
for i in range(n):
    print(" " + " ".join("%6.2f" % v for v in A[i]))
print("%-4s %-16s %-16s %-16s" % ("cell", "random (x,y)", "quadratic (x,y)", "legal (x,y)"))
for c in cells:
    print("%-4s (%5.1f,%5.1f) (%5.1f,%5.1f) (%5.1f,%5.1f)" %
          (c, rand[c][0], rand[c][1], qp[c][0], qp[c][1], leg[c][0], leg[c][1]))
print("HPWL random placement : %6.1f um" % hpwl(rand))
print("HPWL quadratic (overlap): %6.1f um" % hpwl(qp))
print("HPWL after legalization : %6.1f um" % hpwl(leg))

```

qplace.py - quadratic placement plus a greedy row legaliser (numpy)

```

Laplacian-like matrix A (x and y share it):
  2.00  -0.50  -0.50   0.00   0.00   0.00
 -0.50   3.00  -0.50   0.00  -1.00   0.00
 -0.50  -0.50   2.00  -1.00   0.00   0.00
  0.00   0.00  -1.00   3.00  -1.00  -0.50
  0.00  -1.00   0.00  -1.00   3.00  -0.50
  0.00   0.00   0.00  -0.50  -0.50   2.00
cell    random (x,y)  quadratic (x,y)    legal (x,y)
u1      ( 62.5, 89.7) ( 13.1, 43.8)   ( 15.0, 45.0)
u2      ( 77.6, 22.5) ( 21.5, 69.8)   ( 20.0, 65.0)
u3      ( 30.0, 87.4) ( 31.0, 65.3)   ( 30.0, 65.0)
u4      (  0.5, 82.1) ( 44.8, 73.7)   ( 50.0, 75.0)
u5      ( 79.7, 46.8) ( 42.4, 74.9)   ( 40.0, 75.0)
u6      ( 30.3, 27.8) ( 71.8, 62.1)   ( 70.0, 65.0)
HPWL random placement : 800.9 um
HPWL quadratic (overlap): 277.1 um

```

```
HPWL after legalization : 290.0 um
```

Real output of python3 qplace.py

Three lessons show up even at this toy scale. First, the analytic optimum cuts HPWL by roughly 3x versus random placement, and cells are pulled toward the pads they connect to (u1 toward IN0, u6 toward OUT). Second, the quadratic solution is **not legal**: u4 and u5 sit 2.4 um apart although each is 10 um wide. Third, legalisation costs wirelength (277 -> 290 um, about +5%); in a real design with millions of cells piled into the middle of the die, naive legalisation of an unspread solution would be catastrophic, which is why global placement must spread cells first.

#### WHY SQUARED LENGTH STILL WORKS

Squared distance over-penalises long nets and under-penalises short ones, so a pure quadratic placement clusters cells. Placers fix this by re-weighting each edge by  $1/(\text{current length})$  in successive solves (making the quadratic mimic linear length) and by adding pseudo-nets that anchor each cell to a spread-out target position (SimPL's 'lower bound / upper bound' iteration).

## 15.2 The placement flow: global, legalisation, detailed

```

floorplanned DB + netlist + SDC + libs
      |
      v
+-----+ coarse, overlapping positions; density-driven
| 1. global placement | spreading; timing/congestion weights on nets
+-----+
      | (interleaved with early optimisation: buffer long nets,
      v remove unneeded buffers from synthesis, resize drivers)
+-----+ snap to rows/sites, remove overlaps,
| 2. legalisation | honour blockages, fences, voltage areas,
+-----+ cell-edge/spacing and pin-access rules
      |
      v
+-----+ local swaps, flips, sliding windows,
| 3. detailed placement | small-window optimal reordering
+-----+
      |
      v
+-----+ sizing, buffering, Vt swap, pin swap,
| 4. pre-CTS optimise | cloning, restructuring (ideal clocks)
+-----+
      |
      v scan reorder, spare cells, tie cells, timing/congestion gate -> CTS

```

The placement stage inside a block implementation flow

### Global placement

Global placement finds approximate positions that minimise weighted wirelength subject to a **density constraint**: in every bin of a grid laid over the core, the total cell area must not exceed a target density (for example 0.6-0.8 of available area). Modern engines start with all cells clustered in the centre and iterate: each iteration reduces wirelength and increases a penalty on density overflow, so the design 'inflates' outward until overflow falls below a threshold (OpenROAD's RePlAce stops by default at about 10% overflow). Timing and congestion are fed back during this process, not only after it.

### Legalisation

Legalisation moves each cell the smallest possible distance to a legal location: on a row, aligned to the placement site grid, correctly oriented (alternate rows are flipped so VDD/VSS rails are shared), not overlapping, outside hard blockages, inside its fence or voltage area, and respecting advanced-node rules such as cell-edge spacing, implant/Vt-layer minimum width (neighbouring cells of different Vt may need filler or a minimum run), drain-sharing restrictions and pin-access constraints. Classic algorithms are Tetris (greedy, left to right) and Abacus (dynamic programming per row minimising squared displacement). Large

displacement is a warning sign: it means global placement produced regions that were too dense.

## Detailed placement

Detailed placement improves a legal placement with local moves that keep it legal: swapping two cells, sliding a cell along its row, flipping a cell to bring its pin closer to the connected net, and exhaustively reordering small windows of 3-4 adjacent cells. It typically recovers a few percent of wirelength and repairs the damage done by legalisation and by optimisation (which inserts and resizes cells and then needs a local re-legalise). Simulated annealing is still competitive here because the move set is small and local.

## 15.3 Timing-driven and congestion-driven placement

A wirelength-optimal placement is often timing-poor: one long net on a critical path matters far more than a hundred short nets on slack-rich paths. **Timing-driven placement** runs a fast internal STA (with a virtual-route or Steiner-tree wire model and ideal clocks) between placement iterations, computes slack per net, and raises the weight of nets on critical paths (net weighting), or adds explicit path-length constraints. Typical net weight:  $w = 1 + \alpha * (\text{criticality})^{\text{beta}}$ , where  $\text{criticality} = 1 - \text{slack}/\text{WNS}_{\text{ref}}$ . The side effect is a trade: critical logic is pulled together, raising local density and possibly congestion.

**Congestion-driven placement** does the opposite type of trade. The tool runs a fast global route (or a probabilistic estimate such as RUDY - rectangular uniform wire density) to find regions where routing demand exceeds supply, then inflates the cells in those regions (**cell padding / inflation**) so they spread, or lowers the target density locally. Congestion is typically caused by:

- **High-pin-density cells** clustered together (large AOI/OAI, multi-bit flops, wide muxes) - more pins per unit area than the local metal tracks can reach (pin-access congestion).
- **Macro corners and channels** - narrow channels between memories where many nets must squeeze through; always check the corners of large macros.
- **Logic structures with inherent global connectivity** - crossbars, large muxes, barrel shifters, FFT butterflies, wide AXI interconnects. These are an architecture/RTL problem (Chapter 6), not something the placer can fully fix.
- **Too high utilisation** - above roughly 70-80% cell utilisation in logic-dense blocks, routing typically fails at advanced nodes; the exact number depends on cell library track height and metal stack.

## Measuring congestion: GRC overflow

The global router divides the die into a grid of **global routing cells (GRCs, gcells)**, typically a few standard-cell rows high. For every gcell edge and every layer it knows the capacity (number of free tracks after blockages and pre-routes such as the power grid) and the demand (number of nets that the global route sends through). **Overflow = demand - capacity** when positive. Reports list total overflow, the worst overflow and the percentage of overflowing gcells per layer and direction, and a congestion map colours hot spots.

congestion map (H/V overflow per gcell, '.'=ok '1'..'9'=tracks over capacity)

```

+-----+
| . . . . . 1 2 1 . . . . . |
| . . +-----+ 2 4 3 1 . . . . +-----+ . . |
| . . | SRAM | 3 6 5 2 . . . . | SRAM | . . |
| . . | macro | 2 5 4 1 . . . . | macro | . . |
| . . +-----+ 1 2 1 . . . . +-----+ . . |
| . . . . . 1 1 . . . . . |
+-----+

```

hot spot in the channel beside the left macro -> add a partial placement blockage, widen the channel, or move pins on the macro

Reading a congestion map (schematic)

| Metric                               | Typically acceptable after placement                      | Action if worse                                               |
|--------------------------------------|-----------------------------------------------------------|---------------------------------------------------------------|
| Total GRC overflow (both directions) | well below 1% of gcells; no hot spots > 2-3 tracks        | cell padding, lower density, partial blockages, fix floorplan |
| Pin density (pins/um <sup>2</sup> )  | within library/tool guideline                             | spread high-pin cells, avoid big multi-bit cells in hot spots |
| Cell utilisation (whole block)       | roughly 60-75% at advanced nodes (higher at mature nodes) | grow block, remove redundant logic, move logic out            |
| Local density per bin                | no bins > ~90-95%                                         | density screens, tighter max-density setting                  |

Congestion targets (rules of thumb; every team calibrates its own)

#### PITFALL - CONGESTION THAT SHOWS UP ONLY AFTER ROUTING

A placement with clean global-route overflow can still fail detailed routing because the estimate ignores pin access, via rules and double-patterning colouring. If the detailed router leaves thousands of DRCs in one region, go back to placement: pad the cells in that region and re-place, rather than trying to fix it in routing.

## 15.4 Pre-CTS optimisation

Once cells have positions, wire parasitics can be estimated and STA becomes meaningful. **Pre-CTS optimisation** (place\_opt, repair\_design/repair\_timing in OpenROAD) fixes design-rule violations and setup timing with **ideal clocks** (zero skew, specified latency and uncertainty from the SDC). Hold is usually not fixed yet because real clock skew does not exist before CTS. The optimisation loop applies a menu of transforms, cheapest first:

| Transform                      | What it does                                                                                                                                            | Cost / side effects                                                                                                                         |
|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Gate sizing                    | Swap a cell for a stronger (or weaker) drive strength of the same function (NAND2_X1 -> NAND2_X4).                                                      | More area, more input cap (slows the previous stage), more leakage. Downsizing on non-critical paths recovers power.                        |
| Buffering / repeater insertion | Split long or high-fanout nets with buffers or inverter pairs; isolate critical sinks from non-critical load.                                           | Area, power; must be placed legally; long nets need buffers at regular spacing (Chapter 2 wire delay grows with L <sup>2</sup> unbuffered). |
| Vt swap                        | Replace an HVT/SVT cell by LVT/ULVT on a critical path, or LVT by HVT where slack allows.                                                               | Leakage rises roughly 3-10x per Vt step; same footprint, so no re-placement. Often capped by an LVT percentage budget.                      |
| Pin swap                       | Connect the latest-arriving signal to the fastest input pin of a cell with logically equivalent pins (e.g. the pin nearest the output in a NAND stack). | Free - no area; limited gain.                                                                                                               |
| Cloning (replication)          | Duplicate a driver so each copy drives part of the fanout, placed near its sinks.                                                                       | Area; breaks equivalence checking only if not recorded - tools keep it verifiable.                                                          |
| Logic restructuring            | Re-synthesise small cones (collapse, re-decompose, move late signals closer to the output).                                                             | Tool-dependent; may need LEC care.                                                                                                          |
| Useful movement                | Pull cells on critical paths closer together (incremental timing-driven placement).                                                                     | Local density.                                                                                                                              |

The pre-CTS optimisation toolbox

DRV (design-rule violation) fixing - max transition, max capacitance, max fanout from Liberty and SDC - comes before setup fixing, because a net with a 2 ns slew makes every timing number downstream meaningless. A useful order is: fix DRVs -> fix setup (WNS first, then TNS) -> recover area and leakage on slack-rich paths -> re-legalise -> incremental detailed placement.

```
# OpenROAD-style placement script (commands exist in OpenROAD; values are examples)
global_placement -density 0.65 -timing_driven -routability_driven
estimate_parasitics -placement           ;# Steiner-tree RC from layer R/C
repair_design                             ;# max slew/cap/fanout, long wires
detailed_placement
repair_timing -setup                     ;# sizing, buffering, Vt swap, cloning
detailed_placement
check_placement -verbose
report_design_area
report_checks -path_delay max -group_path_count 5
report_tns ; report_wns
```

A minimal open-source placement + optimisation sequence (OpenROAD commands; not run here - OpenROAD is covered in Chapter 24)

#### INTERVIEW INSIGHT - WHY NOT FIX HOLD BEFORE CTS?

Hold slack depends on the actual clock skew between launch and capture flops, which is only known after CTS. Fixing hold with ideal clocks inserts delay cells that may be unnecessary (wasting area and hurting setup) or insufficient. The exception is paths whose hold problem is independent of skew, such as scan shift paths between flops in the same leaf cluster with zero logic - some teams add a small hold margin there early.

#### PITFALL - OVER-OPTIMISTIC PRE-CTS TIMING

Ideal-clock timing uses the SDC `set_clock_uncertainty` to stand in for skew and jitter. If the pre-CTS uncertainty is smaller than the skew CTS will actually produce, setup looks clean after placement and collapses after CTS. Typical practice: pre-CTS uncertainty = expected skew + jitter + margin; post-CTS reduce it to jitter + margin because real skew is now computed.

## 15.5 Scan-chain reordering

Synthesis stitches scan chains (Chapter 10) in an order that knows nothing about placement - often alphabetical or by hierarchy - so the scan-in -> scan-out connections criss-cross the die. After placement the tool **reorders** each chain so consecutive flops are physically adjacent, cutting scan wirelength (and congestion) dramatically. The order is described by the **scan DEF** that DFT insertion writes: it defines each chain, its start and stop points, and which segments are **ordered** (must not change, e.g. a lockup latch pair or a shift register) versus **floating** (freely reorderable).

- Reordering is allowed only within a chain and only across flops in the same clock domain/edge, unless the tool inserts lockup latches; flops of different power domains need isolation-aware handling.
- Chain lengths may be rebalanced between chains of the same compression group if the DFT engineer allows it; the ATPG patterns must then be regenerated from the final netlist.
- The hand-off back to DFT is the updated scan DEF plus the final netlist; ATPG is run on the post-layout netlist, not the synthesis one.
- Short scan paths between adjacent flops create hold problems after CTS - expect hold buffers on scan paths.

## 15.6 Spare cells, tie cells and density screens

### Spare cells

A **spare cell** is an unconnected gate (NAND, NOR, inverter, flip-flop, mux, sometimes a small cluster called a spare module) placed throughout the design with inputs tied off. If a bug is found after the base layers are fabricated, a **metal-only ECO** (Chapter 19) can rewire spare cells using only upper metal and via masks - far cheaper and faster than a full respin. Typical practice is to reserve a few percent of cell area (often 1-5%) and scatter spare clusters uniformly so every region has one within reach. An alternative is **ECO gate-array filler cells**: programmable base cells placed as fillers that can be turned into logic by metal changes.



## Tie cells

Inputs that must be constant are not connected directly to the VDD/VSS rails; they go through **TIEHI/TIELO cells**, which protect the gate oxide from ESD events on the supply and make the constant visible to LVS and ECO. Tie cells are inserted after placement and should be spread, with a maximum fanout per tie cell.

## Density screens and blockages

| Construct                         | Effect                                                            | Typical use                                                                |
|-----------------------------------|-------------------------------------------------------------------|----------------------------------------------------------------------------|
| Hard placement blockage           | No standard cells at all                                          | Under macro halos, around analog blocks, channels reserved for routing     |
| Soft blockage                     | No cells during global placement; buffers allowed by optimisation | Narrow channels between macros                                             |
| Partial blockage (density screen) | Limit cell density in a region to e.g. 40-60%                     | Around congestion hot spots, near macro pins, high-pin-density logic       |
| Keep-out margin (halo)            | No cells within a distance of a macro or cell                     | Macro edges, to leave room for pin access and reduce proximity effects     |
| Fence / region / bound            | Constrain listed cells into (fence: exclusively) or near an area  | Voltage areas, clustering of timing-critical groups, partition preparation |

Placement constraints available in every commercial and open-source placer

## 15.7 Placement quality metrics and the go/no-go for CTS

Before moving on, the team reviews a fixed set of metrics. It is common to automate these into a 'QoR dashboard' compared run over run, because a regression (e.g. +3% wirelength after an innocent netlist drop) is much easier to diagnose early.

### Post-placement review

- ☐ All cells legally placed (check\_placement / verify placement shows 0 overlaps, 0 off-site cells, 0 cells in blockages).
- ☐ Setup WNS/TNS with ideal clocks within the agreed pre-CTS budget (e.g. WNS  $\geq$  -5% of the period, to be closed later); no max-transition/max-capacitance violations.
- ☐ Global-route overflow below threshold; congestion map reviewed around macros and channels.
- ☐ Utilisation and per-bin density within target; buffer and inverter count reasonable (a sudden jump means a timing or constraint problem, not a placement problem).
- ☐ HPWL and estimated total wirelength trended against previous runs.
- ☐ Leakage / Vt mix within budget (LVT %).
- ☐ Scan chains reordered; scan DEF written; spare and tie cells inserted.
- ☐ Multi-voltage checks: cells inside correct voltage areas, level shifters and isolation cells placed at domain boundaries.

### WHAT EXPERIENCED PD ENGINEERS LOOK AT FIRST

Not the WNS number but the **shape** of the failing paths: are they long wires between distant macros (floorplan problem), deep logic (RTL/synthesis problem), high-fanout nets (constraint or buffering problem), or paths through congested regions (detours)? Each category has a different owner and fix.



## 15.8 Summary

- Placement assigns legal row/site locations to every standard cell; it minimises weighted HPWL subject to density, then timing, congestion and power.
- Global placement (analytic: quadratic or electrostatic/ePlace) gives spread, overlapping positions; legalisation snaps them to rows and sites; detailed placement polishes locally.
- The quadratic formulation reduces to solving  $Ax = b$ ; fixed pins (I/O, macros) make it well-posed. Our 6-cell run cut HPWL from 801  $\mu\text{m}$  (random) to 277  $\mu\text{m}$ , and legalisation added ~5%.
- Timing-driven placement weights critical nets; congestion-driven placement pads cells in hot spots measured by GRC overflow.
- Pre-CTS optimisation (ideal clocks) fixes DRVs then setup via sizing, buffering, Vt swap, pin swap, cloning and restructuring; hold waits for CTS.
- Scan chains are reordered using the scan DEF; spare cells and tie cells are added; density screens, halos and fences steer the placer.

## 15.9 Exercises

- 1 Compute the HPWL of a 4-pin net with pins at (0,0), (30,10), (12,40) and (25,25). What is the rectilinear Steiner length, and why is HPWL a lower bound?
- 2 Modify qplace.py to remove all pads. Why does np.linalg.solve fail, and what does this say about floorplanning before placement?
- 3 A block at 82% utilisation shows 4% GRC overflow concentrated between two SRAMs. List four fixes in order of cost and name the owner of each.
- 4 Explain why a Vt swap is preferred to upsizing late in the flow, and give one case where upsizing is the only option.
- 5 Why must scan-chain reordering respect clock domains and edges? What cell fixes a chain that crosses from a negative-edge to a positive-edge flop?
- 6 Your pre-CTS WNS is +20 ps with 150 ps clock uncertainty. CTS achieves 60 ps skew and jitter is 30 ps. Estimate post-CTS WNS if you reduce uncertainty to jitter + 20 ps margin and skew happens to be fully against you on the critical path.

## 16. Clock Tree Synthesis and Clock Distribution

Up to placement, every clock in the design was **ideal**: the SDC said `create_clock -period 1.0 [get_ports clk]` and STA assumed the edge arrived at every flip-flop at the same instant (plus the uncertainty you declared). In silicon, one clock pin on the boundary must drive tens of thousands to millions of flop clock pins spread over square millimetres. **Clock tree synthesis (CTS)** builds the network of buffers, inverters, clock gates and wires that delivers the edge to every sink with controlled timing - and after CTS, STA switches to **propagated clocks** and the real skew becomes part of every path's slack.

The clock network is special in three ways. It switches every cycle (activity 1, or 2 transitions per cycle), so it is the single biggest consumer of dynamic power in most synchronous designs. It touches every sequential path, so its errors appear everywhere. And it is the reference for all timing, so its variability (OCV, jitter, crosstalk, IR drop) reduces the usable cycle time of the entire chip.

### 16.1 Clock tree goals and the vocabulary

```

                                source latency      network (insertion) latency
PLL/osc -----> clk port -----+-----> FF1/CK
(off-block)                        |   t1 = 410 ps
                                +-----> FF2/CK
                                t2 = 455 ps

latency (FF)  = source latency + network latency to that sink
global skew  = max(t_i) - min(t_i) over all sinks of the clock = 45 ps
local skew   = t_capture - t_launch for a pair of flops that share a path
transition   = clock slew at each sink (e.g. <= 60-100 ps at advanced nodes)

```

Latency and skew

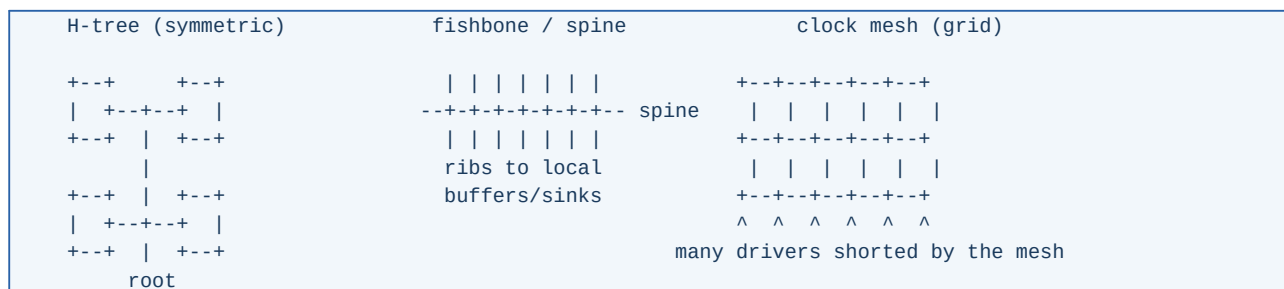
| Goal                      | Why                                                                                                               | Typical target (illustrative)                                              |
|---------------------------|-------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| Low skew                  | Skew is subtracted from setup slack (if capture is earlier) or hold slack (if capture is later).                  | tens of ps for a block at GHz rates; a few % of the period                 |
| Low / bounded latency     | Latency adds OCV-derated delay on the uncommon clock path, and must match between blocks that talk to each other. | a few hundred ps inside a block                                            |
| Sharp, bounded transition | Slow clock edges increase flop clk->Q and setup time, raise short-circuit power and sensitivity to noise.         | max clock transition set by the library/SDC; usually tighter than for data |
| Low power                 | Clock net + buffers + flop clock pins switch every cycle.                                                         | minimise buffer count, wire cap; gate as close to the root as possible     |
| Robustness                | Tolerate OCV, jitter, IR drop, crosstalk, EM on high-activity nets.                                               | NDRs, shielding, balanced cell types                                       |

What CTS optimises

#### SKEW IS ONLY BAD WHEN IT HURTS A PATH

Global skew between two flops that never exchange data is irrelevant to timing. What matters is **local skew on real timing paths**:  $\text{setup slack} = T - (\text{data arrival}) + (\text{capture clock latency} - \text{launch clock latency}) - \text{setup - uncertainty}$ . Positive skew (capture clock later) helps setup and hurts hold; negative skew does the opposite. Modern CTS therefore optimises skew per path group, not one global number.

### 16.2 Clock tree structures



Canonical clock distribution topologies

| Structure                                 | Strengths                                                                      | Weaknesses                                                                                    | Typical use                                       |
|-------------------------------------------|--------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|---------------------------------------------------|
| Conventional CTS (balanced buffered tree) | Automatic, low power, flexible with irregular sink placement                   | Skew and OCV grow with depth; common path short                                               | Most ASIC blocks                                  |
| H-tree                                    | Equal path length by construction -> near-zero nominal skew; long common path  | Needs regular sink distribution; wastes wire where sinks are sparse                           | Top levels of trees; regular arrays               |
| Spine / fishbone                          | Simple, low latency from a central trunk                                       | Skew grows along the spine unless driven from the centre                                      | Datapaths, bit-sliced structures                  |
| Clock mesh                                | Very low skew, insensitive to OCV and load changes (mesh shorts out variation) | High power (short-circuit currents between drivers), needs SPICE-level analysis, hard to gate | High-performance CPUs, GHz cores                  |
| Multi-source CTS (MSCTS) / structured     | H-tree or mesh at top driving many tap points, then conventional local trees   | More setup effort; tap planning                                                               | Large high-speed blocks; the modern middle ground |

Choosing a clock structure

## Elmore model of an H-tree versus an unbalanced tree

The Elmore delay (Chapter 2) of an RC tree is the sum, along the path from the root to a sink, of each segment resistance times all the capacitance downstream of it. It is an upper bound on the 50% delay for a step input and ranks tree topologies correctly, which is all we need. The script builds three unbuffered trees over the same 16 sinks on a 4x4 grid in a 1 mm x 1 mm block (upper-metal wire of 0.1 ohm/um and 0.2 fF/um, a 50-ohm driver and 5 fF per sink cluster) and reports latency and skew.

$$T_{\text{Elmore}}(\text{sink}) = R_{\text{drv}} * C_{\text{total}} + \sum \text{over segments } k \text{ on the root} \rightarrow \text{sink path of}$$

$$R_k * (C_k / 2 + C_{\text{downstream}}(k))$$

$$\text{skew} = \max \text{ over sinks } T - \min \text{ over sinks } T$$

Elmore delay with a pi-model per wire segment

```

# Elmore delay of a 16-sink H-tree vs an unbalanced "comb" tree on a 1 mm x 1 mm block
r, c = 0.10, 0.20e-15 # wire: 0.10 ohm/um, 0.20 fF/um (typical upper-metal order)
RDRV, CSINK = 50.0, 5e-15 # clock driver 50 ohm, 5 fF per sink (flop clock pins lumped)

class Node:
    def __init__(s, name, x, y):
        s.name, s.x, s.y, s.kids, s.len = name, x, y, [], 0.0

def link(par, ch, length=None):
    ch.len = length if length is not None else abs(par.x-ch.x) + abs(par.y-ch.y)
    par.kids.append(ch); return ch

def cdown(n):
    # total capacitance below and including node n's wire
    n.cd = (CSINK if not n.kids else 0.0) + sum(cdown(k) for k in n.kids)
    return n.cd + c * n.len

```

```

def elmore(n, t, out):      # pi-model per segment: R_seg * (C_seg/2 + C_downstream)
    R = r * n.len
    t = t + R * (c * n.len / 2 + n.cd)
    if not n.kids: out[n.name] = t
    for k in n.kids: elmore(k, t, out)
    return out

def analyse(root, label):
    ctot = cdown(root)
    d = elmore(root, RDRV * ctot, {})
    wl = sum_len(root)
    v = sorted(d.values())
    print("%-23s %5.0f %6.0f %8.1f %8.1f %7.1f" % (label, wl, ctot*1e15, v[0]*1e12,
                                                v[-1]*1e12, (v[-1]-v[0])*1e12))

    return d

def sum_len(n): return n.len + sum(sum_len(k) for k in n.kids)

def htree(par, x, y, half, level, tag):
    if level == 0:
        link(par, Node("ff" + tag, x, y)); return
    h = Node("h" + tag, x, y); link(par, h)
    for i, (dx, dy) in enumerate((( -1, -1), (-1, 1), (1, -1), (1, 1))):
        htree(h, x + dx*half, y + dy*half, half/2, level-1, tag + str(i))

sinks = [(125 + 250*i, 125 + 250*j) for i in range(4) for j in range(4)]

print("%-23s %5s %6s %8s %8s %7s" % ("16 sinks, 1x1 mm", "wire", "C(fF)", "min(ps)",
                                     "max(ps)", "skew"))

root = Node("root", 500, 500)
htree(root, 500, 500, 250, 2, "")
dh = analyse(root, "H-tree (balanced)")

# comb: driver at the left edge, one trunk along y=500, a branch per column
root2 = Node("root", 0, 500); prev = root2
for i, xcol in enumerate((125, 375, 625, 875)):
    t = link(prev, Node("t%d" % i, xcol, 500)); prev = t
    for j, ycol in enumerate((125, 375, 625, 875)):
        link(t, Node("ff%d%d" % (i, j), xcol, ycol))
dc = analyse(root2, "comb/fishbone (unbal.)")

# comb again, with the driver in the centre of the trunk
root3 = Node("root", 500, 500)
for i, xcol in enumerate((125, 375, 625, 875)):
    t = link(root3, Node("t%d" % i, xcol, 500))
    for j, ycol in enumerate((125, 375, 625, 875)):
        link(t, Node("ff%d%d" % (i, j), xcol, ycol))
analyse(root3, "comb, centre-driven")

print("\ncomb/fishbone arrival per sink column (ps), row = y position:")
for j in range(4):
    print("  y=%3d: " % (125+250*j) + " ".join("%6.1f" % (dc["ff%d%d" % (i, j)]*1e12)
                                                for i in range(4)))

```

ctree.py - Elmore latency and skew for three clock topologies

| 16 sinks, 1x1 mm       | wire | C(fF) | min(ps) | max(ps) | skew |
|------------------------|------|-------|---------|---------|------|
| H-tree (balanced)      | 6000 | 1280  | 78.2    | 78.2    | 0.0  |
| comb/fishbone (unbal.) | 4875 | 1055  | 66.0    | 106.0   | 40.0 |
| comb, centre-driven    | 5000 | 1080  | 57.1    | 65.2    | 8.1  |

comb/fishbone arrival per sink column (ps), row = y position:

|        |      |      |      |       |
|--------|------|------|------|-------|
| y=125: | 67.4 | 87.0 | 99.9 | 106.0 |
| y=375: | 66.0 | 85.6 | 98.5 | 104.6 |
| y=625: | 66.0 | 85.6 | 98.5 | 104.6 |
| y=875: | 67.4 | 87.0 | 99.9 | 106.0 |

Real output of python3 ctrees.py

The H-tree is perfectly balanced (0.0 ps skew) because every root-to-sink path has identical length and identical downstream load, but it uses 23% more wire (6000 vs 4875 um) and therefore more capacitance and power. The edge-driven fishbone has 40 ps of skew, almost entirely along the trunk: the last column sees all the trunk resistance. Driving the same comb from the centre cuts skew to 8 ps and even gives the lowest latency. Real CTS engines combine these ideas: a balanced structure at the top, then clustering of nearby sinks, buffer insertion to control slew, and **wire snaking** or buffer sizing to add delay on early branches.

#### WHAT THIS TOY MODEL LEAVES OUT

A real clock tree is buffered every few hundred um (an unbuffered 1 mm net would violate transition limits at advanced nodes), sink caps are unequal, and buffer delays vary with OCV. With buffers, skew comes mainly from unequal buffer stages and loads rather than from wire RC, and CTS balances by sizing buffers and adjusting the number of stages per branch.

## 16.3 The CTS flow

```
placed DB + SDC clocks + CTS spec (targets, cells, NDRs, exceptions)
    |
    v
1. clock tracing: find sinks, stop/ignore/exclude pins, generated clocks, muxes
2. clustering: group nearby sinks, respecting clock gates and max fanout
3. tree building: insert clock buffers/inverters level by level (bottom-up
   or top-down), route clock nets early with NDRs (clock pre-route)
4. balancing: skew groups, latency targets, snaking, sizing
5. clock-aware optimisation: useful skew (CCOpt), clock gate cloning/merging
6. switch STA to propagated clocks; update uncertainty; fix setup/hold
    |
    v
post-CTS DB -> routing (Chapter 17)
```

Clock tree synthesis steps

### Clock specifications

CTS is driven by the SDC clocks plus a tool-specific specification: target skew, maximum transition (often a separate, tighter value for the clock), maximum fanout, target latency, the list of allowed buffers and inverters, the routing rules for clock nets, and exceptions. Important exceptions are **sink/stop pins** (treat as a leaf and balance to it), **ignore/exclude pins** (drive but do not balance, e.g. a clock going to a data pin of a flop for a divider check), and **float pins with pre-defined insertion delay** for hard macros whose internal clock tree already has latency (an SRAM or a hardened sub-block).

```
# Illustrative CTS setup in generic Tcl (names differ by tool; values are examples)
set_ccopt_property target_skew          0.030    ;# ns
set_ccopt_property target_max_trans     0.060
set_ccopt_property buffer_cells         {CKBUF_X4 CKBUF_X8 CKBUF_X16}
set_ccopt_property inverter_cells      {CKINV_X4 CKINV_X8 CKINV_X16}
set_ccopt_property clock_gating_cells  {ICG_X2 ICG_X4}
# double-width, double-spacing rule on M5-M8 for trunk and leaf nets
create_route_rule -name CLK_2W2S -width_multiplier 2 -spacing_multiplier 2
create_route_type -name clk_trunk -route_rule CLK_2W2S -top M8 -bottom M5 \
                  -shield_net VSS
set_ccopt_property route_type clk_trunk -net_type trunk
# SRAM macro has 180 ps internal clock latency: balance to its CK pin minus that
set_ccopt_property insertion_delay 0.180 -pin u_sram0/CLK
```

A CTS specification (illustrative; property names modelled on Innovus CCOpt, not from a real run)

## Non-default routing rules (NDRs) and shielding

Clock nets are routed with **non-default rules**: double width lowers resistance (and EM current density), double spacing lowers coupling capacitance to neighbours and thus crosstalk-induced delay change. **Shielding** places VSS (or VDD) wires on both sides of a clock trunk so its neighbours are quiet by construction. Both cost routing tracks, so a common compromise is 2W2S plus shielding on trunks (few long nets, high impact) and default width with double spacing, or no NDR, on leaf nets (many short nets near the flops).

## Clock buffers versus clock inverters

Libraries provide dedicated clock cells (CKBUF, CKINV) with balanced rise/fall delays and strong, symmetric drive. Inverter-based trees use fewer transistors per stage and have lower latency and power than buffer trees; alternating polarities also help keep **duty cycle** intact, because a rise/fall delay imbalance in one stage is partly cancelled by the next. Inverter trees must have an even number of stages to each sink (or the tool must track polarity).

## Useful skew and concurrent clock/data optimisation

Zero skew is not optimal. If flop B has a critical input path and a slack-rich output path, delaying B's clock borrows time from the next stage: this is **useful skew**. Concurrent clock and data optimisation (CCOpt in Innovus, CCD in Fusion Compiler) builds the tree with per-sink latency targets computed from the slack of the paths entering and leaving each flop, optimising data-path sizing at the same time. It routinely recovers a few percent of frequency but must be constrained so hold does not explode and so it does not consume margin that sign-off OCV needs.

$$\text{setup slack}(A \rightarrow B) = T + (L_B - L_A) - t_{cq} - t_{logic} - t_{setup} - \text{uncertainty}$$

$$\text{hold slack}(A \rightarrow B) = t_{cq} + t_{logic\_min} - t_{hold} - (L_B - L_A) - \text{uncertainty\_hold}$$

Raising  $L_B$  by  $d$ :     $\text{setup slack}(A \rightarrow B) \ += \ d$      $\text{hold slack}(A \rightarrow B) \ -= \ d$

$\text{setup slack}(B \rightarrow C) \ -= \ d$      $\text{hold slack}(B \rightarrow C) \ += \ d$

Useful skew moves slack between adjacent stages ( $L$  = clock latency)

## 16.4 Clock gating inside the tree

Integrated clock-gating cells (ICGs, Chapter 11) sit **in** the clock tree: the ICG's clock input is a node of the tree and its gated output drives a sub-tree. CTS has to balance the whole thing so sinks behind a gate have the same latency as ungated sinks. Tools clone ICGs with high fanout (so each copy drives a local cluster) and merge ICGs with identical enables, and they place ICGs close to the root of their sub-tree so the gated portion - where power is saved - is as large as possible.

The ICG enable has a real setup check against the clock at the ICG (the **clock-gating check**, inferred automatically by STA for known ICG cells). Because the ICG is earlier in the tree than the flops that generate

the enable, the enable path sees a **negative skew**: the ICG clock arrives before the flop clocks. Placing ICGs too close to the root makes this check fail; too close to the leaves wastes power. Tools trade this automatically, but it is a classic place for late surprises.

#### PITFALL - CLOCK GATING CHECK AFTER CTS

Enable paths to ICGs often pass with ideal clocks and fail after CTS because the ICG's clock latency is, say, 250 ps smaller than the launching flop's. Budget for it pre-CTS (set\_clock\_gating\_check margins or tighter constraints on enable logic) and keep enable logic shallow.

## 16.5 OCV, CPPR, jitter and duty cycle

### On-chip variation on clock paths

Sign-off STA (Chapter 9) applies on-chip variation: for setup, the launch clock path and data path are computed late and the capture clock path early; for hold the opposite. With AOCV/POCV (Chapter 18) the derate depends on depth and distance, but the effect is the same: **every picosecond of clock latency not shared between launch and capture adds pessimism**. A 400 ps latency with a 5% derate difference costs up to about 20 ps of slack. This is why low latency and a long common path matter as much as low skew.

### Common path pessimism removal (CPPR / CRPR)

The part of the clock tree shared by launch and capture cannot be simultaneously fast and slow, yet OCV analysis derates it both ways. STA removes this double-count with **clock reconvergence pessimism removal (CRPR)**, also called CPPR: the difference between late and early arrival at the last common node is added back to the slack.

```

common path (derated late for launch AND early for capture)
clk --[B1]--[B2]---+---[B3]--[B4]--> FF_launch/CK    late : 0.52 ns
                  |
                  +---[B5]--[B6]--> FF_capture/CK    early : 0.44 ns

at the common node (after B2): late arrival 0.26, early arrival 0.22
CPPR credit = 0.26 - 0.22 = 0.04 ns added back to setup slack

```

CPPR in a picture (numbers illustrative)

### Jitter and duty-cycle distortion

**Jitter** is the time variation of the clock edge from its ideal position, generated by the PLL (reference noise, VCO phase noise) and added by the distribution (supply noise modulating buffer delay). **Cycle-to-cycle** and **period jitter** matter for setup between successive edges; they enter STA through set\_clock\_uncertainty. **Duty-cycle distortion** comes from unequal rise/fall delays accumulating along the tree; it matters for half-cycle paths (posedge to negedge), DDR interfaces and anything that uses both edges. Clock inverter trees and duty-cycle correctors control it; STA models it with set\_clock\_latency -rise/-fall or explicit half-period uncertainty.

**post-CTS setup uncertainty** ~ PLL jitter (period) + supply-noise jitter + margin

**pre-CTS setup uncertainty** ~ the above + expected skew

**hold uncertainty** ~ margin (+ skew pre-CTS); same-edge jitter cancels

How uncertainty is budgeted before and after CTS (typical practice)

## 16.6 Generated clocks, clock muxes and multiple clocks

Clock dividers (create\_generated\_clock -divide\_by 2 -source ...) and clock multiplexers complicate CTS. The generated clock's sinks must be balanced **through** the divider flop so that data paths between the master and divided domain see sensible skew; the CTS tool must therefore trace through the divider's clock pin to its output. For a **clock mux** (functional clock vs test clock, or two PLL outputs), the tree after the mux is

shared, and the tool must balance each clock through the mux while not balancing clocks that are never active together (logically/physically exclusive clock groups in the SDC). Scan test clocks share the functional tree in most designs; the test-mode skew targets may be looser, because shift is slow, but hold must be met in both modes.

#### INTERVIEW INSIGHT - BALANCING ACROSS DOMAINS

If clocks A and B are asynchronous, balancing their latencies to each other is pointless and wastes power; declare them in `set_clock_groups -asynchronous` and let CTS balance each on its own. If A and B are synchronous (same PLL, ratio clocks), their latencies must be balanced, or the inter-clock paths absorb the difference as skew. The question 'which clocks need to be balanced together?' is one of the first a CTS engineer asks the front-end team.

## 16.7 Clock power

Clock power is  $P = \alpha \times C \times V^2 \times f$  with activity  $\alpha = 1$  for the clock net (one full charge/discharge per cycle), where  $C$  includes clock wire, clock buffer input and output capacitance and every flop's clock pin (including the flop's internal clock inverters). In typical synchronous SoCs the clock network accounts for **roughly 30-40% of dynamic power** (sometimes more in flop-heavy designs), which makes it the first target of any power reduction.

$$P_{\text{clk}} = C_{\text{clk}} \times V^2 \times f$$

example:  $C_{\text{clk}} = 1280 \text{ fF}$  (the H-tree above, sinks included),  $V = 0.75 \text{ V}$ ,  $f = 1 \text{ GHz}$

$$P = 1.28\text{e-}12 \times 0.5625 \times 1\text{e}9 = 0.72 \text{ mW} \quad \text{for just 16 sink clusters}$$

a 1 M-flop block with  $\sim 1.5 \text{ fF/flop}$  of pin + wire + buffer cap:

$$C \sim 1.5 \text{ nF} \rightarrow P \sim 1.5\text{e-}9 \times 0.5625 \times 1\text{e}9 \sim 0.84 \text{ W}$$

Clock power estimate (the arithmetic is exact; the per-flop capacitance is an order-of-magnitude assumption)

- **Clock gating** (architectural, RTL-inferred ICGs, and gating close to the root) is the biggest lever.
- **Low-latency, low-buffer-count trees**: every avoided buffer stage removes its input/output cap and internal power.
- **Multi-bit flops** share the internal clock inverters and reduce the number of clock pins.
- **Avoid over-constraining skew**: tighter skew targets make trees deeper and fatter; ask for what the paths actually need.
- **NDR choice**: double spacing lowers coupling cap (and power); double width raises area cap slightly - wide wires are for resistance and EM, not for power.

## 16.8 Post-CTS optimisation and hold fixing

After CTS the STA mode switches to `set_propagated_clock`, the clock uncertainty is reduced (skew is now computed), and the timing picture changes: some setup paths improve (useful skew), others degrade. **Post-CTS optimisation** repeats setup fixing with real clocks and, for the first time, fixes **hold** violations.

Hold fixing inserts delay (delay cells, buffers, or downsized cells) on the data path of violating paths. The art is to fix hold without breaking setup: the tool fixes hold at the endpoint (capture side) of paths whose setup slack is large, uses multi-corner awareness (hold is usually worst at the fast corner, setup at the slow corner, and a delay cell that fixes hold at FF adds several times more delay at SS), and avoids fixing 'false' hold violations created by missing constraints. Thousands of hold buffers after CTS are common in large blocks - especially on scan shift paths - but tens of thousands suggest a clock-tree or constraint problem.

| Symptom after CTS                     | Likely cause                                                 | Fix                                                                 |
|---------------------------------------|--------------------------------------------------------------|---------------------------------------------------------------------|
| Massive hold violations on scan paths | Adjacent flops, zero logic, skew from different leaf buffers | Normal - hold buffers; or reorder chains so neighbours share a leaf |



| Symptom after CTS                   | Likely cause                                                         | Fix                                                           |
|-------------------------------------|----------------------------------------------------------------------|---------------------------------------------------------------|
| Setup degraded on inter-block paths | Latency mismatch with other blocks                                   | Latency targets; update block timing models                   |
| Clock transition violations         | Too few buffers, long clock routes, high fanout ICG                  | Tighter max-fanout, clone ICGs, NDR                           |
| Hold fixed at FF breaks setup at SS | Delay cell ratio across corners                                      | Fix with MCMM, prefer small buffers, move clock (useful skew) |
| Huge latency                        | Deep tree, many clock muxes/gates, macros with large insertion delay | Restructure, MSCTS, review macro insertion delays             |

Common post-CTS debugging patterns

## Reading a CTS report

```
# Clock tree summary (illustrative, not from a real run; generic format)
Clock      Sinks  Bufs  Invs  ICGs  Levels  Latency(min/max ps)  Skew(ps)  MaxTran
-----
core_clk   184312  212   3871  1406   11      412 / 448           36      58 ps
axi_clk    22140   31    402   188    8       301 / 322           21      55 ps
scan_clk   206452   -     -     -     -      (shares core/axi trees; test mode)
jtag_tck    314     6     12    0      4       140 / 151           11      40 ps

Skew groups      target  achieved  Worst sink pair
core_clk/func      40       36      u_alu/acc_reg[31]/CK vs u_lsu/q_reg[3]/CK
```

What a CTS summary typically contains (illustrative)

Read it top-down: sink count per clock (did every flop get found?), levels and latency (is the tree deeper than expected?), skew against target per skew group, maximum clock transition, and the worst sink pairs. A clock with far fewer sinks than the flop count in its domain usually means the SDC defines the clock on the wrong pin or a clock mux/gate is not traced; a very deep tree usually means a high-fanout ICG or an unnecessary balancing requirement between unrelated clocks.

| Hand-off        | From -> to                    | Content                                                                                                    |
|-----------------|-------------------------------|------------------------------------------------------------------------------------------------------------|
| Clock spec      | Front-end/architect -> PD     | Clocks, frequencies, which clocks balance together, exceptions, latency targets for inter-block interfaces |
| Post-CTS DB     | PD -> routing (same engineer) | Clock nets routed with NDRs, clock cells fixed/dont_touch                                                  |
| Clock latencies | Block PD -> top-level/SoC PD  | Achieved insertion delay per clock port for top-level balancing and for block timing models (ETM/ILM)      |
| Clock power     | PD -> power team              | Clock net/buffer power for the power budget                                                                |

CTS hand-offs

## CTS sign-off gate before routing

- ☐ All clocks built; no unbuffered/unbalanced sinks; report of ignore/exclude pins reviewed.
- ☐ Skew, latency and max transition within targets for every clock and every mode (functional and test).
- ☐ Setup and hold met (or within an agreed post-route budget) in all MCMM scenarios with propagated clocks.
- ☐ Clock nets routed (or pre-routed) with the specified NDR and shielding; clock cells are dont\_touch from here on.
- ☐ Clock power reported and compared to the power budget.

## 16.9 Summary

- CTS turns ideal clocks into a physical network of buffers, inverters, ICGs and NDR-routed wires; STA then uses propagated clocks.
- Goals: low skew on paths that matter, low latency (less OCV), sharp transitions, low power, robustness.
- Structures range from conventional balanced trees through H-trees and spines to meshes and multi-source CTS. Our Elmore model gave 0 ps skew for an H-tree, 40 ps for an edge-driven fishbone and 8 ps when the same fishbone is centre-driven.
- Useful skew/CCOpt moves slack between stages; clock-gating checks and generated clocks/muxes need special care.
- CPPR removes pessimism on the common path; jitter and duty-cycle distortion enter via uncertainty and half-cycle paths.
- Clock power is typically 30-40% of dynamic power; hold is fixed after CTS, multi-corner aware.

## 16.10 Exercises

- 1 Using `ctree.py`, add wire snaking to the edge-driven fishbone so every branch has the same Elmore delay. How much wire did you add and what happened to total capacitance?
- 2 A launch flop has clock latency 520 ps (late), capture 470 ps (early), and the common path ends at a node with late/early arrivals of 300/270 ps. Compute the skew seen by setup before and after CPPR.
- 3 Explain why inverter-based clock trees preserve duty cycle better than buffer-based ones.
- 4 A flop B has 80 ps negative setup slack on its input path and +150 ps on its output path. How much useful skew would you apply, and what hold checks must you re-verify?
- 5 Estimate clock power for 400 k flops at 1.2 GHz, 0.8 V, assuming 2 fF of clock capacitance per flop including wire and buffers. What fraction is saved if 60% of flops are gated off 70% of the time?

## 17. Routing, Signal Integrity and DFM

Routing creates the physical metal and vias that implement every connection of the netlist, obeying hundreds to thousands of design rules (Chapter 3), meeting timing with real parasitics, and producing geometry that the fab can print and polish reliably. After CTS the clock nets are already routed; routing now completes the remaining signal nets - often millions of them across 10-15+ metal layers - and then hands the design to extraction and sign-off. Three disciplines meet here: **combinatorial routing algorithms**, **signal integrity** (crosstalk, noise, electromigration) and **design for manufacturability** (fill, density, lithography hotspots, via redundancy).

### 17.1 Routing layers, tracks and preferred directions

Each routing layer has a **preferred direction** (horizontal or vertical, alternating layer to layer), a **pitch** (track spacing), and minimum width/spacing. Lower layers are thin and dense (short local connections, pin access), intermediate layers carry block-level routes, and thick upper layers carry long global wires, clock trunks and power. Wrong-way routing on a layer is allowed in older nodes but restricted or forbidden at advanced nodes, where lower metals are unidirectional and printed with multiple patterning.

| layer   | dir | relative pitch | typical role                                  |
|---------|-----|----------------|-----------------------------------------------|
| AP/RDL  | -   | very thick     | bumps, redistribution, package interface      |
| M13-M14 | H/V | ~ 20-50x       | top power grid, global clocks                 |
| M9-M12  | H/V | ~ 4-10x        | global signal routes, clock trunks, PG straps |
| M4-M8   | H/V | ~ 1.5-2x       | block-level signal routing                    |
| M1-M3   | H/V | 1x (tightest)  | cell pins, local routing, multi-patterned     |
| MEOL    | -   | -              | local interconnect / contacts to gate & S/D   |

(layer counts and ratios vary widely by node and foundry option)

A generic advanced-node metal stack (schematic)

The router works on a **track grid** derived from the LEF: each **LAYER** statement gives direction, pitch, width and spacing tables, and each **VIA/VIARULE** defines legal vias. Cell pins, defined in the cell LEF, must be reached from tracks - at advanced nodes **pin access** (how many distinct ways a router can land on a pin) is a first-order problem that library designers and placement tools (cell spacing, flipping) address together.

### 17.2 Global routing, track assignment and detailed routing

```

+-----+      gcell grid, capacity per edge/layer, route each net as
| global routing |--> a sequence of gcells (Steiner tree + maze/pattern routes);
+-----+      minimise overflow and wirelength; layer assignment
      |
      v
+-----+      assign each global segment to an actual track in its
| track assignment|--> panel; long straight segments first; avoid parallel
+-----+      runs of critical nets (early crosstalk control)
      |
      v
+-----+      exact geometry: pin access, vias, jogs; fix all spacing,
| detailed routing|--> end-of-line, min-area, via enclosure, cut spacing, colour
+-----+      rules; rip-up and reroute iterations until DRC count -> 0
      |
      v
search & repair, redundant vias, post-route opt, fill, extraction -> sign-off

```

The three routing stages

**Global routing** divides the die into gcells and decides, for each net, which gcells and layers it will use - without committing to tracks. Because it is fast, it is also the congestion estimator used during placement. **Track assignment** turns the coarse routes into segments on specific tracks within each panel (a row or column of gcells), which is where long parallel runs can be avoided. **Detailed routing** then produces exact DRC-correct

geometry, working in small windows and iterating **rip-up and reroute** until no violations remain.

## Maze routing: the Lee algorithm

The Lee algorithm (1961) is the textbook detailed router. It is a breadth-first search on a grid: from the source, label every reachable free cell with its distance (the **wave expansion**), stop when the target is labelled, then **backtrace** from target to source by always stepping to a neighbour whose label is one less. It is guaranteed to find a shortest path if one exists, but needs memory and time proportional to the grid area. Improvements include A\* search (expand toward the target first), bidirectional search, cost functions penalising vias, bends and congestion (Soukup, Hadlock), and line-probe routers (Hightower, Mikami-Tabuchi) that trace straight lines instead of cells - fast but not complete.

The script below routes two 2-pin nets on a 10 x 20 grid with obstacles (#). Each net's routed cells become blockages for the next net. Routing in two different orders shows the most important property of sequential routing: **the result depends on net order**, and an early net can make a later one unroutable.

```
# Lee maze router: BFS wave expansion + backtrace on a single-layer grid
from collections import deque
GRID = [
    ".....X.....",
    ".....",
    "..S....#.....",
    ".....#.....",
    ".....#.....####",
    "....###..#.....#..",
    "....#..#.....#..",
    "....#..#####....#T.",
    "....#.....#..",
    ".....Y.....",
]
R, C = len(GRID), len(GRID[0])
g = [list(row) for row in GRID]
find = lambda ch: next((r, c) for r in range(R) for c in range(C) if g[r][c] == ch)
S, T = find("S"), find("T")

def lee(src, dst, blocked):
    dist = {src: 0}; q = deque([src]); expanded = 0
    while q:
        cur = q.popleft(); expanded += 1
        if cur == dst: break
        r, c = cur
        for nr, nc in ((r-1, c), (r+1, c), (r, c-1), (r, c+1)):
            if 0 <= nr < R and 0 <= nc < C and (nr, nc) not in dist \
                and not blocked(nr, nc):
                dist[(nr, nc)] = dist[cur] + 1; q.append((nr, nc))
    if dst not in dist: return None, expanded
    path = [dst]
    while path[-1] != src:
        r, c = path[-1]
        nbrs = [(r-1, c), (r+1, c), (r, c-1), (r, c+1)]
        if len(path) > 1:
            pr, pc = path[-2]; nbrs.sort(key=lambda n: (n[0]-r, n[1]-c) != (r-pr, c-pc))
        path.append(next(n for n in nbrs if dist.get(n) == dist[(r, c)] - 1))
    return path[::-1], expanded

A = (find("S"), find("T")); B = (find("X"), find("Y"))

def route_all(order):
    occ = {}
    res = {}
    # grid cell -> net that owns it
```

```

for name, (s, t) in order:
    def blocked(r, c, name=name):
        return g0[r][c] == "#" or occ.get((r, c), name) != name
    path, exp = lee(s, t, blocked)
    res[name] = (path, exp)
    if path:
        for cell in path: occ[cell] = name
    return res, occ

g0 = [row[:] for row in g]
for label, order in (("order A then B", (("A", A), ("B", B))),
                    ("order B then A", (("B", B), ("A", A))):
    res, occ = route_all(order)
    pic = [row[:] for row in g0]
    for cell, name in occ.items():
        if pic[cell[0]][cell[1]] == ".": pic[cell[0]][cell[1]] = "*" if name == "A" else "+"
    print("%s: " % label + " ".join("%s len=%d expanded=%d" % (n, len(p)-1, e)) if p
        else ("%s UNROUTABLE (expanded=%d)" % (n, e)) for n, (p, e) in res.items()))
    print(" " + "".join(str(c % 10) for c in range(C)))
    for r in range(R): print("%2d " % r + "".join(pic[r]))

```

lee2.py - Lee maze router with BFS wave expansion, backtrace and net ordering

```

order A then B: A len=23 expanded=169  B len=27 expanded=116
01234567890123456789
0 .+++++++X.....
1 .+*****.
2 .+S....#.....*.
3 .+++++.#.....*.
4 .....+.#.....####*.
5 ..####+.#.....#*.
6 .....#+.#.....#*.
7 .....#+.#####....#T.
8 .....#++++++.#...
9 .....Y.....
order B then A: B len=15 expanded=145  A UNROUTABLE (expanded=66)
01234567890123456789
0 .....+++X.....
1 .....+.
2 ..S....+.#.....
3 .....+.#.....
4 .....+.#.....####..
5 ..####+.#.....#..
6 .....#+.#.....#..
7 .....#+#####....#T.
8 .....#++++++.#...
9 .....Y.....

```

Real output of python3 lee2.py (\* = net A, + = net B, # = blockage)

Routed A-then-B, both nets complete: A takes 23 grid units (Manhattan distance 21, a 2-unit detour over the wall) and B detours to 27 units around A. Routed B-then-A, B takes its shortest 15-unit path down column 7 and walls off the left half of the grid, so A becomes **unroutable**. A real router would now **rip up** B (or the nets blocking A), route A, and **reroute** B with a history cost that discourages the contested cells - negotiated congestion routing (PathFinder) formalises exactly this. Production routers iterate this on millions of nets while also checking every design rule, which is why detailed routing dominates back-end runtime.

## Design-rule-driven routing

Modern detailed routers are driven by far more than width and spacing: end-of-line spacing, minimum area and minimum enclosed area, minimum step, parallel-run-length-dependent spacing tables, via enclosure and cut-spacing rules, via-to-via spacing across layers, and multi-patterning colour rules (two same-colour shapes on a double-patterned layer must be further apart than two different-colour shapes). The router must also avoid

'odd cycles' that cannot be coloured. Rules come from the tech LEF (or the tool's tech file), which the foundry and tool vendor co-develop.

### 17.3 Vias, redundant vias and non-default rules

A single via is a reliability and yield weak point: a void or misalignment in one cut opens the net, and a single cut carries current density limited by EM. **Redundant (double-cut) via insertion** after routing replaces single-cut vias by two-cut or bar vias wherever space allows; typical targets are a high percentage (often 90%+ where possible) of vias doubled, especially on clock and power-critical nets. Foundry DFM rules may be 'recommended' rules that are scored rather than mandatory.

**Non-default rules (NDRs)** - introduced for clocks in Chapter 16 - are also applied to critical signal nets: wider wires for long high-speed routes (lower resistance), extra spacing for noise-sensitive nets (analog references, asynchronous resets), and multiple-cut vias for high-current nets. Each NDR consumes tracks, so they are used selectively.

### 17.4 Crosstalk: coupling, delta delay and glitches

At advanced nodes wires are tall and narrow and packed at minimum spacing, so a large fraction of each wire's capacitance - often more than half - is **coupling capacitance**  $C_c$  to its neighbours rather than to ground. When a neighbour (the **aggressor**) switches, charge is injected into the **victim** through  $C_c$ . Two effects follow:

- **Delta delay (crosstalk-induced delay change)**: if aggressor and victim switch at overlapping times, the effective coupling capacitance is multiplied by the **Miller factor**  $k$ :  $k = 0$  when they switch in the same direction (the victim speeds up),  $k = 1$  when the aggressor is quiet,  $k = 2$  when they switch in opposite directions (the victim slows down). STA with SI uses this for setup (slowdown on data, speed-up on the capture clock) and hold (the reverse).
- **Glitch (functional noise)**: if the victim is quiet (held by its driver), an aggressor transition creates a voltage bump. If the bump exceeds the receiver's noise margin and propagates to a flop, latch or asynchronous pin during its sensitive window, it corrupts state. Noise analysis compares glitch height and width against noise-immunity curves from the library.

$$C_{\text{eff}}(\text{victim}) = C_{\text{ground}} + k * C_c \quad k \text{ in } \{0, 1, 2\}$$

glitch peak (lumped, ramp aggressor of slew  $tr$ , victim holder  $R$ ,  $\tau = R * C_{\text{tot}}$ ):

$$V_{\text{peak}} = VDD * (C_c / C_{\text{tot}}) * (\tau / tr) * (1 - \exp(-tr / \tau))$$

-> bounded by the charge-sharing limit  $VDD * C_c / C_{\text{tot}}$  when  $\tau \gg tr$

First-order crosstalk models

The script estimates both effects for a 500-um victim on thin lower metal (2 ohm/um) with a weak 800-ohm driver, whose ground capacitance is 40 fF and coupling to each neighbour is 35 fF at minimum spacing - numbers typical in order of magnitude for an advanced node, but not for any particular process.

```
# Crosstalk on a 500-um victim between two parallel aggressors (lumped/Elmore estimates)
import math
VDD, L = 0.75, 500.0          # supply (V), victim length (um)
rw = 2.0                      # victim wire resistance, ohm/um (thin lower metal)
cg = 0.08e-15                 # victim area+fringe cap to ground, F/um
cc_min = 0.07e-15             # coupling cap to EACH neighbour at min spacing, F/um
cl = 3e-15                    # receiver pin cap
rdrv = 800.0                  # victim driver output resistance (weak driver)

def delay(k, cc):
    """Elmore delay (0.69 * sum RC) with Miller factor k applied to both coupling caps."""
    cw = (cg + 2 * k * cc) * L
    return 0.69 * (rdrv * (cw + cl) + rw * L * (cw / 2 + cl))
```

```

print("victim %d um, Rdrv=%d ohm: Cg=%1f fF, Cc=%1f fF per side (min spacing)"
      % (L, rdrv, cg * L * 1e15, cc_min * L * 1e15))
print("%-34s %9s %9s %9s %9s" % ("configuration", "k=0 (ps)", "k=1 (ps)", "k=2 (ps)",
                                  "window"))
for label, cc in (("min spacing", cc_min), ("2x spacing (Cc ~ 0.5x)", cc_min * 0.5),
                 ("3x spacing (Cc ~ 0.3x)", cc_min * 0.3)):
    d0, d1, d2 = (delay(k, cc) * 1e12 for k in (0, 1, 2))
    print("%-34s %9.1f %9.1f %9.1f %8.1f%%" % (label, d0, d1, d2, 100 * (d2 - d0) / d1))
# shielding: neighbours become VSS/VDD wires -> k is always 1, but Cc still loads the net
d_sh = delay(1, cc_min) * 1e12
print("%-34s %9s %9.1f %9s %8.1f%%" % ("shielded (VSS both sides)", "-", d_sh, "-", 0))

print("\nglitch on a quiet victim held low by its driver; one aggressor, parallel run Lc:")
for lc in (100.0, 500.0):
    cc = cc_min * lc
    ctot = (cg * L + cc_min * L * 2) + cl          # everything on the victim, others quiet
    for rhold in (300.0, 1500.0):
        for tr in (30e-12, 100e-12):
            tau = (rhold + rw * L / 2) * ctot
            vp = VDD * (cc / ctot) * (tau / tr) * (1 - math.exp(-tr / tau))
            print("  Lc=%3.0f um Rhold=%4.0f ohm slew=%3.0f ps -> peak=%3.0f mV (%2.0f%% VDD)"
                  % (lc, rhold, tr * 1e12, vp * 1e3, 100 * vp / VDD))

```

xtalk.py - Miller-factor delay window and lumped glitch estimate

```

victim 500 um, Rdrv=800 ohm: Cg=40.0 fF, Cc=35.0 fF per side (min spacing)
configuration          k=0 (ps)  k=1 (ps)  k=2 (ps)  window
min spacing            39.6      102.4     165.2     122.6%
2x spacing (Cc ~ 0.5x) 39.6      71.0      102.4      88.4%
3x spacing (Cc ~ 0.3x) 39.6      58.4      77.3       64.5%
shielded (VSS both sides) -       102.4      -         0.0%

glitch on a quiet victim held low by its driver; one aggressor, parallel run Lc:
Lc=100 um Rhold= 300 ohm slew= 30 ps -> peak= 40 mV ( 5% VDD)
Lc=100 um Rhold= 300 ohm slew=100 ps -> peak= 28 mV ( 4% VDD)
Lc=100 um Rhold=1500 ohm slew= 30 ps -> peak= 44 mV ( 6% VDD)
Lc=100 um Rhold=1500 ohm slew=100 ps -> peak= 38 mV ( 5% VDD)
Lc=500 um Rhold= 300 ohm slew= 30 ps -> peak=198 mV (26% VDD)
Lc=500 um Rhold= 300 ohm slew=100 ps -> peak=141 mV (19% VDD)
Lc=500 um Rhold=1500 ohm slew= 30 ps -> peak=218 mV (29% VDD)
Lc=500 um Rhold=1500 ohm slew=100 ps -> peak=188 mV (25% VDD)

```

Real output of python3 xtalk.py

At minimum spacing, the victim's delay can be anything from 40 to 165 ps depending on what its neighbours do - a window larger than the nominal delay itself. Doubling the spacing shrinks the window; shielding with ground wires removes the variation entirely (k is always 1) but does **not** reduce the delay, since the shield is still a capacitance. Glitch height scales with the length of the parallel run and the aggressor's slew: a 100-um overlap produces ~5% VDD, while a 500-um run with a sharp aggressor reaches ~26-29% VDD, enough to worry about on an asynchronous reset or a clock.

#### TIMING WINDOWS MAKE SI ANALYSIS TRACTABLE

Assuming every aggressor switches at the worst moment is hugely pessimistic. SI-aware STA (PrimeTime SI, Tempus SI) computes the **arrival windows** of victims and aggressors and only counts aggressors whose windows overlap, iterating because delta delays move the windows. Logical correlation (two nets that can never switch together) and aggressor filtering by coupling size reduce pessimism further.

## Fixing crosstalk

| Fix                    | How it helps                                                      | Cost           |
|------------------------|-------------------------------------------------------------------|----------------|
| Increase spacing / NDR | Reduces C <sub>c</sub> roughly in proportion to the extra spacing | Routing tracks |

| Fix                           | How it helps                                                                      | Cost                                                             |
|-------------------------------|-----------------------------------------------------------------------------------|------------------------------------------------------------------|
| Shielding                     | Neighbour becomes a quiet supply wire                                             | Two tracks per shielded net; slightly higher C <sub>ground</sub> |
| Layer promotion               | Move the net to a thicker/wider upper layer with lower R and different neighbours | Vias, upper-layer resources                                      |
| Upsize the victim driver      | Stronger holder: smaller glitch, faster transition, smaller delta delay           | Area, power, input cap upstream                                  |
| Buffer insertion              | Breaks a long parallel run into shorter coupled segments; restores slew           | Cells, placement                                                 |
| Downsize / slow the aggressor | Slower aggressor slew injects less charge                                         | Aggressor timing                                                 |
| Reroute (ECO route)           | Separate the two nets physically                                                  | Router iterations                                                |

SI repair techniques

## 17.5 Antenna effect

During fabrication, metal layers are etched with plasma. A long metal segment connected to a transistor gate - but not yet to any diffusion that could discharge it, because the connecting upper layer has not been built - collects charge from the plasma and can drive current through the thin gate oxide, damaging it (shifting V<sub>th</sub> or causing early breakdown). This is the **antenna effect** (plasma-induced gate-oxide damage). Foundries specify antenna rules as ratios of the metal (or via) area connected to a gate, layer by layer, to the gate area, often cumulative across layers and relaxed when a diode is connected.

| problem:                                                                                                                                                              | fixes:                                                                                                                                                                                               |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| M1 long wire (etched first, floating)<br>=====+<br> <br>[gate]<br><br>driver's drain connects<br>only through M5 (built later)<br>-> no discharge path during M1 etch | a) layer jump (bridge up to a higher layer near the gate, so the long piece is only connected once the gate's upper layer exists)<br><br>b) antenna diode (reverse-biased diffusion) on the gate net |
| $\text{antenna ratio} = (\text{area of metal Mx connected to gate}) / (\text{gate oxide area})$                                                                       |                                                                                                                                                                                                      |

The antenna effect and its standard fixes

- **Layer jumping (bridging):** break the offending lower-layer wire and jump up to a higher layer just before the gate, so during the etch of the lower layer only a short piece is attached to the gate. Routers do this automatically.
- **Antenna diodes:** an ANTENNA cell (a reverse-biased diode to substrate) near the gate gives the charge a discharge path; it adds a small capacitance and leakage.
- **Driver placement:** a diffusion (driver output) on the same net discharges it; nets with a driver close to the receiver rarely violate.
- Antenna checks are part of physical verification (Chapter 18); the LEF carries antenna data (ANTENNAGATEAREA, ANTENNADIFFAREA and per-layer ratio rules) so the router can fix them during routing.

## 17.6 Electromigration on signal nets

Electromigration (EM, Chapter 18 in detail) is metal atom transport by the electron wind; it is usually associated with power grids but also affects signal nets that carry high switching currents: clock nets, high-fanout drivers, long heavily loaded nets. Signal EM uses **RMS and peak current** limits (bidirectional AC current partially heals, so average-current limits apply mostly to unidirectional power currents) and depends on temperature. Fixes: widen the wire (NDR), add parallel routes or multi-cut vias, reduce the driver strength or its load, split the net.



## 17.7 Design for manufacturability (DFM)

A design that passes DRC can still yield poorly. DFM covers the rules and practices that make the layout robust to manufacturing variation. Most of it is driven by the physics of lithography and chemical-mechanical polishing (CMP) introduced in Chapter 3.

| DFM topic                   | Problem                                                                                                                                                                                         | Action in implementation                                                                                                                                                      |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Metal density rules         | CMP removes copper and dielectric at rates that depend on local pattern density: sparse areas dish, dense areas erode, giving thickness variation (and so R/C variation and planarity problems) | Min/max density per window (e.g. sliding windows tens of um wide) per layer; checked in DRC                                                                                   |
| Metal fill (dummy fill)     | Low-density windows                                                                                                                                                                             | Insert floating (or tied) fill shapes after routing; timing-aware fill keeps distance from critical nets because fill adds coupling capacitance; extraction must see the fill |
| Lithography hotspots        | Patterns that print poorly under process window variation (pinching, bridging) even though they pass DRC                                                                                        | Litho-friendly routing, pattern-matching DFM checks, foundry hotspot libraries, fixing in ECO                                                                                 |
| Via redundancy              | Single-cut via failures                                                                                                                                                                         | Double-cut/bar vias (see above)                                                                                                                                               |
| Wire spreading and widening | Critical-area defects: random particles short close wires or open thin ones                                                                                                                     | Spread wires where tracks are free; widen where possible                                                                                                                      |
| Recommended rules           | Foundry rules beyond the minimum that improve yield                                                                                                                                             | Scored DFM checks; fix where cheap                                                                                                                                            |
| Multi-patterning colouring  | Decomposition failures, overlay sensitivity                                                                                                                                                     | Colour-aware routing; balanced colour density                                                                                                                                 |

DFM measures applied during and after routing

### PITFALL - FILL AFTER TIMING SIGN-OFF

Metal fill adds capacitance to nearby wires. If fill is inserted after the final extraction and STA, the sign-off timing is no longer the timing of the taped-out design. Insert fill before the final extraction (or use a fill-aware extraction mode) and re-run SI/timing on the filled database.

## 17.8 Post-route optimisation

Once real routes exist, parasitics are accurate and SI effects appear. Post-route optimisation fixes the residual setup, hold, transition and SI violations with the same toolbox as before - sizing, buffer insertion, Vt swap, hold delay insertion - but every change now requires **ECO routing**: rip up and reroute the affected nets while leaving the rest untouched. Changes must be small and local; the goal is convergence, not re-optimisation. The loop 'extract -> STA with SI -> fix -> ECO route -> DRC check' runs until timing, SI and DRC are all clean.

```
# OpenROAD-style routing sequence (real OpenROAD command names; example values;
# not run here - see Chapter 24 for the open-source flow)
set_routing_layers -signal met1-met5 -clock met3-met5
global_route -congestion_iterations 50 -verbose
estimate_parasitics -global_routing
repair_timing ;# last chance with GR parasitics
detailed_route -output_drc route_drc.rpt -droute_end_iter 64
repair_antennas ;# insert diodes where jumping was not enough
density_fill -rules fill.json ;# metal fill
extract_parasitics -ext_model_file rcx_patterns.rules
write_spef top.spef
```

The routing stage as an open-source script

| Detailed-route DRCs left over         | Usual root cause                                             | Where to fix                                        |
|---------------------------------------|--------------------------------------------------------------|-----------------------------------------------------|
| Many shorts/spacing in one region     | Local congestion or pin access                               | Placement: pad cells, lower density, re-place       |
| Violations at macro edges             | Macro pins on blocked tracks, missing halo                   | Floorplan: halo, pin layer/placement                |
| Min-area / end-of-line on short stubs | Router corner cases                                          | Search-and-repair iterations, ECO route             |
| Violations inside or at IP boundaries | Abstract (LEF) does not match the GDS (missing obstructions) | Fix the LEF with the IP owner; not a router problem |
| Colour conflicts                      | Odd cycles on multi-patterned layers                         | Reroute locally; check cell library compliance      |

Triaging leftover routing DRCs

## 17.9 RC extraction and SPEF

**Parasitic extraction** computes the resistance and capacitance of every routed net from its geometry and the process's interconnect technology file (ITF/ICT, compiled into a tool-specific TLUplus/QRC tech file). Sign-off extractors (StarRC, Quantus, Calibre xACT, and in open source OpenRCX) use pattern-matching against field-solver-built tables, or a field solver directly for critical nets. Extraction must be done at each **RC corner** (typical, Cworst, Cbest, RCworst, RCbest - thickness and width variations pull R and C in correlated directions), and at the right temperature for resistance.

The standard output is **SPEF** (Standard Parasitic Exchange Format, IEEE 1481), a text format listing, per net, the total capacitance, each node's ground capacitance, coupling capacitances to other nets, and the resistor network.

```
*SPEF "IEEE 1481-1998"
*DESIGN "top"
*T_UNIT 1 NS
*C_UNIT 1 FF
*R_UNIT 1 OHM
*L_UNIT 1 HENRY
*NAME_MAP
*1 n42
*2 u7
*3 u9
*D_NET *1 5.13           // net n42, total cap 5.13 fF
*CONN
*I *2:Y 0 *L 0 *D INV_X2 // driver pin
*I *3:A I *L 1.2         // load pin, 1.2 fF pin cap
*CAP
1 *1:1 1.10             // ground cap at internal node
2 *1:2 1.45
3 *1:2 *17:3 1.38       // coupling cap to another net
*RES
1 *2:Y *1:1 12.5
2 *1:1 *1:2 40.2
3 *1:2 *3:A 8.1
*END
```

The structure of a SPEF file (hand-written illustrative fragment, not from a real run; comments added)

### SANITY-CHECK THE EXTRACTION

Before trusting sign-off numbers, check that every net was extracted (no 'unannotated nets' in STA), that the RC corner matches the timing corner, that coupling caps are present (SI needs them - a 'ground-only' extraction silently disables crosstalk analysis) and that fill was included.

## 17.10 Summary

---

- Routing is global routing (gcells, congestion), track assignment, and detailed routing with rip-up and reroute; layers have preferred directions and pitch, advanced lower layers are unidirectional and multi-patterned.
- The Lee maze router finds shortest paths by BFS and backtrace; our run showed that net order can make a net unroutable, which rip-up and reroute resolves.
- Crosstalk changes delay (Miller factor 0..2) and creates glitches; our 500-um example had a 40-165 ps delay window at minimum spacing. Fixes: spacing, shielding, layer promotion, sizing, buffering.
- Antenna violations are fixed by layer jumping and diodes; signal EM by widening and multi-cut vias.
- DFM: density rules and fill (CMP), litho hotspots, redundant vias, wire spreading; fill must be in the extracted database.
- Post-route optimisation uses ECO routing; extraction at every RC corner produces SPEF for sign-off STA.

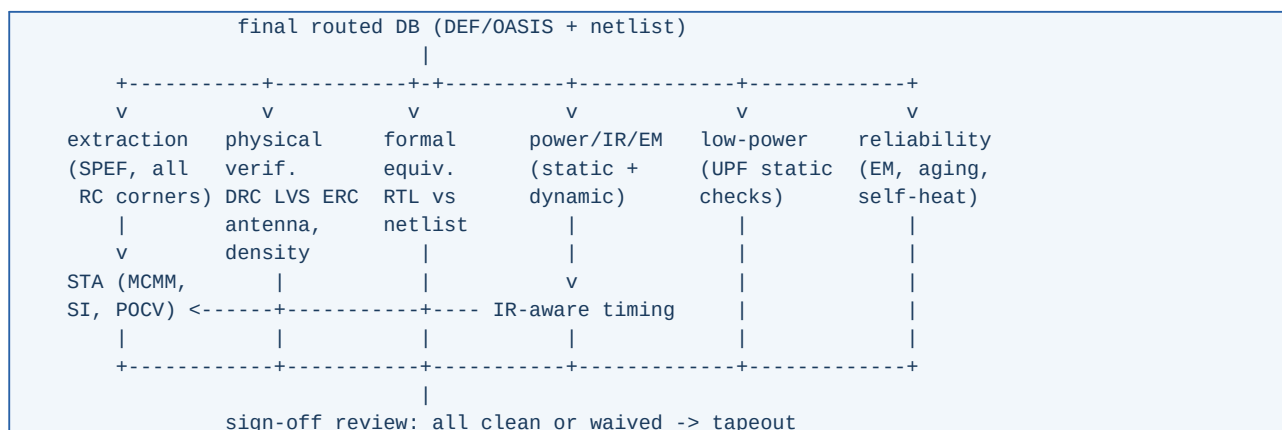
## 17.11 Exercises

---

- 1 Modify lee2.py to add a via cost: give the grid two layers (H-only and V-only) and count a layer change as cost 3. How does the route for net A change?
- 2 Why is  $k = 2$  the worst case for setup on a data path but the best case for hold? Which Miller factor would SI analysis use on the capture clock path for a setup check?
- 3 A 1.5-mm net on M2 fails an antenna check. Give two fixes and explain which one the router prefers and why.
- 4 Explain why metal fill changes timing and how you would make sure sign-off accounts for it.
- 5 List the five common RC extraction corners and state which one is typically worst for setup on long wire-dominated paths and which for short gate-dominated paths.

## 18. Sign-off: Timing, Power Integrity, Reliability and Physical Verification

**Sign-off** is the set of independent, golden-tool checks that must all pass before a design is allowed to tape out. The word matters: implementation tools (Innovus, Fusion Compiler, OpenROAD) have built-in timers, extractors and DRC checkers, but their results are estimates tuned for speed. Sign-off uses separately qualified tools and foundry-certified rule decks - PrimeTime or Tempus for timing, StarRC or Quantus for extraction, RedHawk or Voltus for power integrity, Calibre, Pegasus or IC Validator for physical verification, Conformal or Formality for equivalence - and every waiver is written down and approved. A chip fails in silicon because one of these checks was skipped, run on the wrong data, or waived without understanding.



The sign-off checks and how they feed each other

### 18.1 Timing sign-off

Chapter 9 covered STA theory; here we look at what makes sign-off STA different from the timing inside the implementation tool.

#### Multi-corner multi-mode (MCMM)

A **mode** is a set of constraints (functional, scan shift, scan capture, MBIST, low-power retention...). A **corner** is a combination of process (library corner: SS, TT, FF, and at advanced nodes SSG/FFG global corners), voltage, temperature and RC extraction corner. A **scenario** is a mode x corner pair. Sign-off runs every scenario that could be worst for some check. Temperature inversion at advanced nodes (cells can be slower at low temperature at low voltage) means both -40 C and 125 C (or the product's range) are needed for setup.

| Scenario (example)  | Library PVT         | RC corner        | Checks                                             |
|---------------------|---------------------|------------------|----------------------------------------------------|
| func_ss_cold        | SSG, 0.675 V, -40 C | Cworst / RCworst | setup (temperature inversion), max transition      |
| func_ss_hot         | SSG, 0.675 V, 125 C | RCworst          | setup, EM current (hot)                            |
| func_ff_cold        | FFG, 0.825 V, -40 C | Cbest / RCbest   | hold                                               |
| func_ff_hot         | FFG, 0.825 V, 125 C | Cbest            | hold, leakage                                      |
| func_tt             | TT, 0.75 V, 85 C    | typical          | power reporting, reference                         |
| shift_ss / shift_ff | SS / FF             | RCworst / Cbest  | scan shift setup (slow clock) and hold (critical!) |
| capture_ff          | FF                  | Cbest            | at-speed capture hold                              |

A typical sign-off corner list for a 0.75-V nominal design (values illustrative; the real list comes from the foundry and product requirements)

## Variation modelling: OCV, AOCV, POCV/LVF

| Model                          | How derate is applied                                                                                                   | Pessimism                                                    |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|
| Flat OCV                       | One late and one early derate (e.g. +/-5%) on all cells and nets                                                        | High for deep paths, possibly optimistic for very short ones |
| AOCV                           | Derate from tables indexed by path depth and distance: random variation averages out over many stages                   | Lower                                                        |
| POCV / SOCV                    | Each cell arc has a sigma; path sigma computed statistically (root-sum-square); check at mean +/- n*sigma (typically 3) | Lowest; standard at 16 nm and below                          |
| LVF (Liberty Variation Format) | Liberty tables of sigma per arc, slew and load (ocv_sigma_cell_rise etc.), including asymmetric early/late sigma        | Data format used by POCV                                     |

On-chip variation models in sign-off STA

## SI and the rest of the sign-off STA checklist

- **SI on:** coupling caps from SPEF, delta delay with timing windows, and noise/glitch analysis on all nets (Chapter 17).
- **All checks, not only setup/hold:** max transition, max capacitance, min pulse width (clock high/low), minimum period, recovery/removal on asynchronous resets, clock-gating checks, data-to-data checks, latch time borrowing.
- **Constraints are signed off too:** unconstrained endpoints = 0 (or reviewed), no unintended false paths or multicycle paths, clock-domain crossings handled by CDC sign-off (Chapter 6), IO constraints agreed with the SoC top.
- **Consistency:** the netlist in STA is exactly the netlist in LVS and LEC; the SPEF was extracted from the same layout; library versions match the foundry release.

```
# Illustrative PrimeTime-style sign-off script fragment (not from a real run)
set_app_var si_enable_analysis true
set_app_var timing_pocvm_enable_analysis true
set_app_var timing_remove_clock_reconvergence_pessimism true
read_verilog top.post_route.v ; link_design top
read_parasitics -keep_capacitive_coupling top.rcworst_m40.spef.gz
read_sdc top.func.sdc
update_timing -full
report_global_timing
report_constraint -all_violators -max_transition -max_capacitance
report_analysis_coverage           ;# untested / unconstrained checks
report_si_bottleneck               ;# nets with largest delta delay
report_noise -all_violators
```

Sign-off STA essentials (commands modelled on PrimeTime; illustrative)

## 18.2 Power integrity: IR drop and voltage-drop-aware timing

The power grid (Chapter 14) has resistance, so current flowing from the bumps to the cells causes a **voltage drop (IR drop)** on VDD and a **ground bounce** on VSS. A cell that sees 0.70 V instead of 0.75 V is slower - cell delay sensitivity to supply is typically several percent delay per few percent of voltage at advanced nodes - and severe drop can cause functional failure.

| Analysis                     | What it computes                                                                                                                     | When used                                                                                       |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Static IR drop               | Average current per cell (from average power at a given activity), DC solve of the resistive grid                                    | Early grid sizing; catches missing vias/straps, weak connections; typical budget a few % of VDD |
| Dynamic IR drop (vectorless) | Tool chooses switching scenarios statistically to meet a target toggle rate, solves the RLC grid in time including decap and package | Coverage without vectors; can be pessimistic or miss real hot spots                             |

| Analysis                       | What it computes                                                                                                | When used                                        |
|--------------------------------|-----------------------------------------------------------------------------------------------------------------|--------------------------------------------------|
| Dynamic IR drop (vector-based) | Uses real activity (VCD/FSDB/SAIF) from simulation or emulation of worst-case workloads, including scan capture | Sign-off for realistic peaks; needs good vectors |
| Voltage-drop-aware timing      | Annotates each cell's effective supply from IR analysis back into STA (or uses the IR-aware delay models)       | Critical paths through hot spots; clock trees    |
| Power-up / rush current        | Current when a power-gated domain turns on                                                                      | Power switch sequencing (Chapter 11)             |

Power-integrity analyses

**static:**  $V_{\text{drop}}(\text{node}) = \text{sum over grid resistors on the path of } I_{\text{branch}} * R_{\text{branch}}$   
**dynamic:**  $V_{\text{drop}}(t) = I(t) * R + L * dI/dt$  (grid, package and bump inductance),  
 mitigated by on-die decap: charge  $dQ = C_{\text{decap}} * dV$  supplies the fast peak

Static versus dynamic voltage drop

**PITFALL - SCAN CAPTURE IR DROP**

During at-speed scan capture, a large fraction of all flops can toggle in the same cycle - far more than in any functional mode. The resulting dynamic drop can make good silicon fail at-speed tests (yield loss from overkill). Run vector-based IR on real ATPG patterns and use low-power ATPG fill or staggered capture if needed (Chapter 10).

## 18.3 Reliability sign-off: EM, self-heating and aging

### Electromigration and Black's equation

Electromigration is the gradual transport of metal atoms by momentum transfer from electrons. Over years it creates voids (opens, resistance increase) and hillocks/extrusions (shorts). The median time to failure follows **Black's equation**. Foundries translate it into current-density limits per layer, width and temperature (average current for unidirectional flow, RMS for Joule heating, peak for signal nets), for a target lifetime (for example 10 years at a given junction temperature) and a failure-rate criterion.

$$\text{MTTF} = A * J^{(-n)} * \exp(E_a / (k * T))$$

$J$  = current density ( $A/cm^2$ )       $n \sim 1$  to  $2$  (fit;  $2$  often used)

$E_a$  = activation energy (eV)       $k = 8.617e-5$  eV/K       $T$  = absolute temperature

$A$  = technology/geometry constant (Cu interconnect:  $E_a$  typically  $\sim 0.8-0.9$  eV)

Black's equation for electromigration lifetime

The script normalises lifetime to 1.0 at  $1 \text{ MA/cm}^2$  and  $105 \text{ C}$ , using  $n = 2$  and  $E_a = 0.85 \text{ eV}$  (typical fit values; real ones are foundry-specific), and shows how quickly margin disappears with temperature and current density.

```
# Black's equation: MTTF = A * J^(-n) * exp(Ea / (k * T)); everything relative to a reference
import math
k = 8.617e-5 # Boltzmann constant, eV/K
n, Ea = 2.0, 0.85 # typical Cu-interconnect fit values (foundry-specific!)
Jref, Tref = 1.0, 105.0 # reference: 1.0 MA/cm^2 at 105 C -> MTTF = 1.00 (normalised)

def rel_mttf(J, Tc):
    return (J / Jref) ** (-n) * math.exp(Ea / k * (1 / (Tc + 273.15) - 1 / (Tref + 273.15)))

temps = (85, 105, 125, 150)
print("relative MTTF (1.00 = 1 MA/cm^2 at 105 C), n=%.1f, Ea=%.2f eV" % (n, Ea))
```

```

print("J (MA/cm^2) " + "".join("%10s" % ("%d C" % t) for t in temps))
for J in (0.5, 1.0, 1.5, 2.0, 3.0):
    print("    %4.1f      " % J + "".join("%10.2f" % rel_mttf(J, t) for t in temps))

print("\nallowed J for the SAME lifetime as 1 MA/cm^2 at 105 C:")
for t in temps:
    print("    %3d C : Jmax = %.2f MA/cm^2" % (t, math.sqrt(math.exp(Ea / k * (1 / (t + 273.15)
        - 1 / (Tref + 273.15)))))
print("\nself-heating: a signal wire at 105 C ambient that heats by dT")
for dT in (0, 5, 10, 20):
    print("    dT=%2d C -> lifetime x %.2f" % (dT, rel_mttf(1.0, Tref + dT)))

```

black.py - relative EM lifetime

```

relative MTTF (1.00 = 1 MA/cm^2 at 105 C), n=2.0, Ea=0.85 eV

```

| J (MA/cm^2) | 85 C  | 105 C | 125 C | 150 C |
|-------------|-------|-------|-------|-------|
| 0.5         | 17.17 | 4.00  | 1.08  | 0.25  |
| 1.0         | 4.29  | 1.00  | 0.27  | 0.06  |
| 1.5         | 1.91  | 0.44  | 0.12  | 0.03  |
| 2.0         | 1.07  | 0.25  | 0.07  | 0.02  |
| 3.0         | 0.48  | 0.11  | 0.03  | 0.01  |

```

allowed J for the SAME lifetime as 1 MA/cm^2 at 105 C:

```

```

    85 C : Jmax = 2.07 MA/cm^2
   105 C : Jmax = 1.00 MA/cm^2
   125 C : Jmax = 0.52 MA/cm^2
   150 C : Jmax = 0.25 MA/cm^2

```

```

self-heating: a signal wire at 105 C ambient that heats by dT

```

```

dT= 0 C -> lifetime x 1.00
dT= 5 C -> lifetime x 0.71
dT=10 C -> lifetime x 0.51
dT=20 C -> lifetime x 0.27

```

Real output of python3 black.py

Two lessons are worth memorising. Lifetime falls roughly 4x for every 20 C rise near 105 C with these parameters, so the **temperature** at which EM limits are specified matters as much as the current. And because lifetime goes as  $J^{-2}$ , a wire that is 50% over its limit loses more than half its life - EM violations are not 'small' just because the overshoot is small. **Self-heating** of wires (Joule heating from RMS current) and of FinFET/GAA devices (heat trapped in narrow fins) raises local temperature by several degrees above the ambient junction temperature, which the EM check must include.

## Device aging in timing

- **NBTI** (negative-bias temperature instability) raises the  $|V_{th}|$  of PMOS transistors held on (gate low) at high temperature; it partially recovers when stress is removed. Clock trees with gated (stopped) clocks age asymmetrically, distorting duty cycle.
- **HCI** (hot-carrier injection) degrades NMOS (and PMOS) by energetic carriers near the drain during switching; it grows with switching activity and is worst for high-activity, high-slew nets.
- **PBTI** matters for NMOS with high-k gate stacks; **TDDB** (time-dependent dielectric breakdown) is a gate-oxide and inter-metal dielectric lifetime issue set by voltage.
- In sign-off, aging is handled by **aged libraries** (Liberty characterised with end-of-life  $V_{th}$  shifts, e.g. after 10 years at the mission profile) or aging derates; automotive designs (AEC-Q100, ISO 26262 mission profiles) require it explicitly.

## 18.4 Physical verification: DRC

**Design rule checking (DRC)** verifies that every polygon of the final layout obeys the foundry's manufacturing rules: widths, spacings, enclosures, extensions, areas, densities, and advanced-node rules such as multi-patterning colouring, tip-to-tip, voltage-dependent spacing and via/cut rules. The sign-off rule deck (Calibre SVRF, Pegasus PVL, ICV runsets; KLayout and Magic decks for open PDKs) is released by the foundry and must be used unmodified. Sign-off DRC runs on the **merged GDSII/OASIS**: the block layout streamed out with all standard-cell, IP and memory layouts substituted for their abstracts - the implementation tool only ever saw LEF abstracts.

The demo uses the open-source **magic** layout tool with its built-in SCMOS (MOSIS scalable, lambda-based) technology: three metal-1 wires, one legal, one too narrow, and one too close to its neighbour, then a batch DRC.

```
# three metal1 shapes in the SCMOS (lambda-based) technology
cellname rename (UNNAMED) demo
box 0 0 3 20;      paint metal1      ;# OK: 3 lambda wide (min width 3)
box 10 0 12 20;    paint metal1      ;# too narrow: 2 lambda
box 14 0 17 20;    paint metal1      ;# 2 lambda from the previous wire (min space 3)
box 0 0 17 20
drc check
drc catchup
puts "DRC error count: [drc list count total]"
foreach {why boxes} [drc listall why] {
  puts "RULE: $why"
  foreach b $boxes { puts "    at (lambda) [lrange $b 0 3]" }
}
quit -noprompt
```

drc.tcl - run with: magic -dnul -noconsole -T scmos drc.tcl

```
DRC error count: 2
RULE: First-level metal spacing must be at least 3 (MOSIS rule #7.2)
    at (lambda) 14 0 15 20
RULE: First-level metal width must be at least 3 (MOSIS rule #7.1)
    at (lambda) 12 0 13 20
```

Real output (last lines) of magic 8.3.105 DRC on the deliberately broken layout

```
DRC error count: 0
No errors found.
```

After widening the middle wire to 3 lambda and moving the third wire to 3 lambda spacing (drc\_fix.tcl): clean

Magic reports each violation with the rule text from the technology file and the error region (the area where the rule fails, in lambda). A production DRC run on a large SoC reports results per rule in a database (Calibre RVE, KLayout marker browser), and the team drives the count to zero or to a list of formally waived violations (for example, known foundry-approved IP exceptions).

| DRC category          | Examples                                                     | Typical origin                              |
|-----------------------|--------------------------------------------------------------|---------------------------------------------|
| Base width/space      | M2.W.1, M2.S.1, via enclosure                                | Manual edits, pin access, IP abutment       |
| Density               | Metal/poly/diffusion density min/max per window              | Missing or blocked fill, large macros       |
| Antenna               | Metal-to-gate area ratios                                    | Long nets; checked in PV with antenna rules |
| Multi-patterning      | Colouring conflicts, same-colour spacing                     | Router corner cases, IP boundaries          |
| Off-grid / structural | Shapes off manufacturing grid, acute angles, non-rectilinear | Imported IP, logos                          |
| Boundary/ESD/latch-up | Well tap distance, guard rings, ESD rules                    | I/O ring, analog IP (Chapter 12)            |

Where DRC violations usually come from at sign-off



## 18.5 Physical verification: LVS

**Layout versus schematic (LVS)** proves that the layout implements the intended circuit. It has two stages:

- **Extraction:** from the layout polygons, the LVS rule deck recognises devices (a poly shape crossing diffusion is a MOSFET; its W and L are measured), resistors, capacitors and diodes, and traces connectivity through contacts and vias to build a **layout netlist**. Labels (text on pins) name the nets.
- **Comparison:** the layout netlist is compared with the reference (schematic) netlist - for a digital block, the post-route Verilog netlist converted to SPICE with the standard cells' transistor-level CDL netlists. Comparison is graph isomorphism: devices are matched by type and connectivity, iteratively refining partitions of nets and devices until everything is uniquely matched or a mismatch is isolated. Device properties (W, L, number of fins, multiplier, resistor length) are compared within tolerance.

The demo compares hand-written SPICE netlists of an AND2 cell (NAND2 + inverter) with the open-source LVS tool **netgen** (used in the OpenLane flow). The 'layout' netlists deliberately use different device names, internal net names, device order and port order, which LVS must see through.

```
* schematic: 2-input NAND followed by an inverter (AND2)
.subckt and2 A B Y VDD VSS
M1 n1 A VDD VDD pmos W=0.84u L=0.15u
M2 n1 B VDD VDD pmos W=0.84u L=0.15u
M3 n1 A x VSS nmos W=0.64u L=0.15u
M4 x B VSS VSS nmos W=0.64u L=0.15u
M5 Y n1 VDD VDD pmos W=1.00u L=0.15u
M6 Y n1 VSS VSS nmos W=0.65u L=0.15u
.ends
```

sch.spice - reference (schematic) netlist

```
* extracted layout netlist: device order and internal names differ
.subckt and2 VSS VDD Y B A
M10 Y net7 VSS VSS nmos W=0.65u L=0.15u
M11 Y net7 VDD VDD pmos W=1.00u L=0.15u
M12 net9 B VSS VSS nmos W=0.64u L=0.15u
M13 net7 A net9 VSS nmos W=0.64u L=0.15u
M14 net7 B VDD VDD pmos W=0.84u L=0.15u
M15 net7 A VDD VDD pmos W=0.84u L=0.15u
.ends
```

lay\_ok.spice - 'extracted' netlist, same circuit, different names and order

```
# minimal netgen setup: compare MOSFET W and L with a 1% tolerance
# (source/drain of built-in nmos/pmos devices are already permutable)
foreach dev {nmos pmos} {
    foreach c {-circuit1 -circuit2} {
        property "$c $dev" tolerance {w 0.01} {l 0.01}
    }
}
```

setup.tcl - tells netgen to compare W and L with 1% tolerance

```
Reading netlist file lay_ok.spice
Reading netlist file sch.spice
Reading setup file setup.tcl
Comparison output logged to file ok.out
Logging to file "ok.out" enabled
Contents of circuit 1: Circuit: 'and2'
Circuit and2 contains 6 device instances.
  Class: pmos           instances: 3
  Class: nmos           instances: 3
Circuit contains 7 nets.
Contents of circuit 2: Circuit: 'and2'
Circuit and2 contains 6 device instances.
  Class: pmos           instances: 3
```

```

Class: nmos              instances: 3
Circuit contains 7 nets.

Circuit 1 contains 6 devices, Circuit 2 contains 6 devices.
Circuit 1 contains 7 nets,    Circuit 2 contains 7 nets.

Netlists match uniquely.
Result: Circuits match uniquely.
Logging to file "ok.out" disabled
LVS Done.

```

Real output of: `netgen-lvs -batch lvs "lay_ok.spice and2" "sch.spice and2" setup.tcl ok.out (netgen 1.5.133)`

Now two classic layout bugs: the drain of the B-input PMOS is connected to Y instead of the internal node (a **short/miswire**), and the inverter NMOS is drawn narrower than intended.

```

* layout netlist, two bugs: B pmos drain shorted to Y; inverter nmos too narrow
.subckt and2 VSS VDD Y B A
M10 Y net7 VSS VSS nmos W=0.42u L=0.15u
M11 Y net7 VDD VDD pmos W=1.00u L=0.15u
M12 net9 B VSS VSS nmos W=0.64u L=0.15u
M13 net7 A net9 VSS nmos W=0.64u L=0.15u
M14 Y B VDD VDD pmos W=0.84u L=0.15u
M15 net7 A VDD VDD pmos W=0.84u L=0.15u
.ends

```

lay\_bad.spice - a miswire plus a device-size error

```

NET mismatches: Class fragments follow (with fanout counts):
Circuit 1: and2                                |Circuit 2: and2
-----|-----
Net: Y                                         |Net: n1
  nmos/drain = 1                             |  pmos/drain = 2
  pmos/drain = 2                             |  nmos/drain = 1
                                           |  pmos/gate = 1
                                           |  nmos/gate = 1
                                           |
Net: net7                                     |Net: Y
  nmos/gate = 1                             |  pmos/drain = 1
  pmos/gate = 1                             |  nmos/drain = 1
  nmos/drain = 1                             |
  pmos/drain = 1                             |
-----|-----
Netlists do not match.

```

Real netgen report (bad.out, excerpt): the net-fragment view that pinpoints the miswire

netgen cannot match nets Y (layout) and n1 (schematic) because their **fanout signatures** differ: layout net Y has two PMOS drains and one NMOS drain but no gate connections, while schematic n1 also drives a PMOS and an NMOS gate. This is how every LVS tool reports connectivity errors - as mismatched partitions, from which the engineer infers the physical short or open. Fixing only the miswire and leaving the narrow transistor gives a topological match with a **property error**:

```

* extracted layout netlist with one bug: the inverter nmos is drawn too narrow (0.42u vs 0.65u)
.subckt and2 VSS VDD Y B A
M10 Y net7 VSS VSS nmos W=0.42u L=0.15u
M11 Y net7 VDD VDD pmos W=1.00u L=0.15u
M12 net9 B VSS VSS nmos W=0.64u L=0.15u
M13 net7 A net9 VSS nmos W=0.64u L=0.15u
M14 net7 B VDD VDD pmos W=0.84u L=0.15u
M15 net7 A VDD VDD pmos W=0.84u L=0.15u
.ends

```

lay\_w.spice - connectivity fixed, W still wrong

```

Circuit 1 contains 6 devices, Circuit 2 contains 6 devices.
Circuit 1 contains 7 nets,    Circuit 2 contains 7 nets.

Netlists match uniquely.
There were property errors.
nmos10 vs. nmos6:
  W circuit1: 4.2e-07  circuit2: 6.5e-07  (delta=43%, cutoff=1%)
Result: Circuits match uniquely.
Property errors were found.
The following cells had property errors: and2

Logging to file "w.out" disabled
LVS Done.

```

Real netgen output: circuits match topologically but the W property differs by 43%

| LVS error                   | What you see                                                                                    | Typical cause                                                                    |
|-----------------------------|-------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| Short                       | Two named nets merged into one (e.g. 'VDD and n12 shorted', or a net with too many connections) | Overlapping metal, a stray via, fill touching a wire, mislabelled pin text       |
| Open                        | One schematic net split into two layout nets                                                    | Missing via, broken wire, missing connection to a well/substrate tap             |
| Missing / extra port        | Pin lists differ                                                                                | Pin not labelled in layout, top-level text on wrong layer, port renamed          |
| Device mismatch             | Extra or missing devices, wrong type (e.g. LVT vs SVT)                                          | Wrong cell variant, missing implant layer, decap/filler cells not in the netlist |
| Property mismatch           | W/L/nfin/multiplier out of tolerance                                                            | Wrong cell version, manual edit                                                  |
| Well/substrate connectivity | Bulk terminals on wrong net                                                                     | Missing taps, multi-voltage wells, tapless cells without tap cells               |

Common LVS errors

#### DEBUGGING LVS ON A REAL CHIP

Fix **shorts first** - a single short (for example a power-to-ground short through a mislabelled pin) merges thousands of nets and produces an avalanche of secondary mismatches. Then fix opens, then ports, then devices, then properties. Always check the texts/labels and the power connections before believing a large mismatch report.

## 18.6 ERC, antenna and density checks

- **ERC (electrical rule checks)**: floating gates, floating wells, nets with no driver or multiple drivers, gates tied directly to supply (instead of tie cells), wells connected to the wrong supply, missing ESD protection on pad nets, and in multi-voltage designs thin-oxide devices connected to high-voltage nets.
- **Antenna**: run with the foundry antenna deck on the final layout; must be clean (router fixes plus diodes, Chapter 17).
- **Density**: min/max density per window for every relevant layer after fill; the foundry also checks density gradient between neighbouring windows.
- **Other foundry checks** often run at tapeout: latch-up rules, ESD rules, reliability rules for thick-metal and bump layers, and DFM/recommended-rule scoring.

## 18.7 Formal equivalence checking

Every netlist transformation after RTL - synthesis, scan insertion, placement optimisation, CTS, hold fixing, ECOs - could introduce a functional change. Rerunning full simulation on each netlist is impossible, so **logic equivalence checking (LEC)** proves mathematically that two designs implement the same function. The tool maps **compare points** (primary outputs, flop and latch inputs, black-box inputs) between the two designs, then proves each compare point's logic cone equivalent using BDDs and SAT solvers.

|     |          |                                                                               |          |                                          |          |                                     |
|-----|----------|-------------------------------------------------------------------------------|----------|------------------------------------------|----------|-------------------------------------|
| RTL | --LEC--> | synthesis netlist                                                             | --LEC--> | DFT netlist                              | --LEC--> | post-route netlist                  |
|     |          | (retiming, clock gating,<br>multibit, constant-flop<br>removal need guidance) |          | (scan mux: constrain<br>scan_enable = 0) |          | (buffers, sizing,<br>cloning, ECOs) |

The chain of equivalence checks

- Non-equivalences are debugged by the counter-example (input vector) the tool produces.
- Synthesis optimisations that change the state encoding (retiming, FSM re-encoding, register merging, constant or duplicate register removal) need **guidance files** (SVF in Formality, or equivalent) or explicit setup, otherwise they appear as mismatches or unmapped points.
- Scan insertion is verified with scan-enable constrained to functional mode; clock-gating insertion needs clock-gate recognition.
- **Unmapped or 'unreachable' compare points must be reviewed**, not ignored; an aborted (inconclusive) cone is not a pass.
- The post-route netlist LEC is the sign-off check that a physical optimisation or ECO did not change function.

## 18.8 Low-power sign-off (UPF static checks)

For designs with power intent (Chapter 11), the final netlist must be checked against the UPF (IEEE 1801) with a static low-power verifier (VC LP, Conformal Low Power): every signal crossing from a switchable domain to an always-on domain has an isolation cell with the right clamp value and enable; every crossing between different voltages has a level shifter; retention flops are connected to the right save/restore controls and supplies; power switches are connected and chained; always-on cells in switchable domains take their supply from the always-on rail (secondary PG pin); and no path goes through a powered-down domain unprotected. The check runs on the power-aware netlist (with PG pins) that also goes to LVS.

## 18.9 The sign-off criteria checklist

### Tapeout sign-off (typical block/chip criteria)

- ☐ STA: setup/hold/DRV clean in all MCM scenarios with SI and POCV; zero unconstrained endpoints (or reviewed); constraints reviewed and frozen.
- ☐ Extraction: all nets annotated, all RC corners, fill included.
- ☐ IR drop: static and dynamic within budget (e.g. a few % static, ~10% dynamic peak - project-specific); IR-aware timing clean.
- ☐ EM: power and signal EM clean at the mission-profile temperature; self-heating included where the foundry requires.
- ☐ Aging: timing with aged libraries or derates, if required by the product.
- ☐ DRC (all decks: base, antenna, density, DFM-mandatory), LVS clean, ERC clean - on the final merged GDS/OASIS.
- ☐ LEC: RTL vs final netlist equivalent (possibly through the chain); no aborted points.
- ☐ Low-power static checks clean against the final UPF.
- ☐ DFT: final ATPG coverage on the post-layout netlist meets target; scan and MBIST simulations with SDF pass.
- ☐ Gate-level simulation with SDF (selected tests) passes at the timing corners.
- ☐ Every waiver documented, reviewed and signed by the owner.

## 18.10 Summary

---

- Sign-off uses independent, foundry-certified golden tools and rule decks; implementation-tool results are estimates.
- Timing sign-off: MCMM scenarios (mode x PVT x RC corner), SI, POCV/LVF, all check types, frozen constraints.
- Power integrity: static and dynamic (vectorless and vector-based) IR drop, voltage-drop-aware timing, scan-capture peaks.
- Reliability: EM via Black's equation (our run: lifetime  $\sim 4\times$  shorter per  $+20\text{ C}$ ,  $J^2$  dependence), self-heating, aging (NBTI/HCI) via aged libraries.
- Physical verification: DRC (magic flagged width and spacing errors), LVS (netgen matched renamed netlists, located a miswire and a W error), ERC, antenna, density.
- LEC proves each netlist equivalent to the previous; UPF static checks verify power intent on the final netlist.

## 18.11 Exercises

---

- 1 Why can a design fail setup at  $-40\text{ C}$  but pass at  $125\text{ C}$  at an advanced node? Which corners must therefore be in the sign-off list?
- 2 Using `black.py`, find the current-density limit at  $125\text{ C}$  that gives the same lifetime as  $1.5\text{ MA/cm}^2$  at  $85\text{ C}$ .
- 3 Edit `lay_ok.spice` to create an **open** (split `net7` into two nets) and run `netgen`. Describe the mismatch report and how you would locate the open in a layout viewer.
- 4 Explain why an LVS short between VDD and a signal net produces hundreds of mismatches, and in what order you would debug a report with 3 shorts, 12 opens and 40 property errors.
- 5 Which synthesis optimisations make LEC fail without guidance, and why?
- 6 A block passes vectorless dynamic IR but fails at-speed scan tests on the tester. What happened and what analysis would have caught it?

## 19. ECOs, Tapeout and Mask Making

No large chip reaches tapeout on the first pass of the flow. Bugs are found in late verification, timing fails in a corner that was added late, a customer changes a register default, or first silicon reveals a problem. An **engineering change order (ECO)** is a small, controlled modification of a design that is already implemented, done without re-running synthesis and place-and-route from scratch. This chapter covers ECOs, the tapeout process that hands the design to the foundry, and what the foundry and mask shop then do with it - the last steps under the design team's control and the most expensive ones to get wrong.

### 19.1 Why ECOs, and what kinds exist

Re-running the full flow after a one-line RTL fix would take days to weeks, would reshuffle every placement and route, and would invalidate all sign-off results. An ECO touches only what must change, so most sign-off results remain valid and only incremental checks are needed. ECOs are classified by what they change and by when they happen.

| ECO type                   | What changes                                                                        | Typical tool flow                                                                                                      |
|----------------------------|-------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| Functional ECO             | Logic function (a bug fix, a feature disable, a register reset value)               | RTL fix -> re-synthesise -> ECO tool computes a minimal patch -> apply to the implemented netlist -> ECO place & route |
| Timing ECO                 | No function change: sizing, buffering, Vt swap, hold fixing                         | Sign-off STA generates fixes (PrimeTime ECO, Tempus ECO) -> ECO route                                                  |
| DRC / SI / EM ECO          | Geometry: reroute, spacing, shielding, wire widening                                | Implementation tool ECO route; PV rerun                                                                                |
| Pre-mask ECO               | Before base layers are released; any layer may change                               | Full-layer ECO; new cells can be added anywhere                                                                        |
| Post-mask (metal-only) ECO | After base layers are fabricated or masks made; only metal and via masks may change | Spare cells or ECO gate-array cells rewired with metal                                                                 |

ECO types

### 19.2 Functional ECO flow

```

RTL_v1 --synth--> netlist_v1 --P&R, DFT, opt--> impl_netlist_v1 (+ layout)
|
| bug fix
v
RTL_v2 --synth--> netlist_v2 (reference: what we want)

ECO tool (Conformal ECO / Formality ECO):
  compare impl_netlist_v1 (revised) against RTL_v2 or netlist_v2 (golden),
  find the smallest set of cones that differ, synthesise a patch
  using available cells (or only spare cells for metal-only)
  |
  v
patch (Tcl/Verilog) -> apply to impl DB -> ECO place (legalise new cells
near their fanin/fanout, or map to spare cells) -> ECO route
  |
  v
LEC: impl_netlist_v2 vs RTL_v2 | STA | DRC/LVS | scan/ATPG update

```

A functional ECO with a formal ECO tool

The formal ECO tools (Cadence Conformal ECO, Synopsys Formality ECO) are built on equivalence-checking engines: they identify which compare points became non-equivalent and search the implemented netlist for **rectification points** - internal nets where a small new cone restores equivalence - re-using existing logic wherever possible. A good patch for a small RTL change is typically a few to a few dozen cells. Manual ECOs (the engineer edits the netlist by hand) are still used for tiny changes, especially metal-only, but must always be

followed by LEC.

#### PITFALL - THE ECO THAT WAS NOT VERIFIED

Every ECO must be followed by LEC against the new RTL, STA at all corners, DRC/LVS on the final layout and, for DFT-visible changes, updated ATPG. Teams under schedule pressure skip one of these 'because the change is tiny' - which is precisely how a tiny change breaks scan chains or introduces a hold violation at the fast corner.

#### FREEZING AND TRACKING

Late in a project, RTL is 'frozen' and every change becomes an ECO with a ticket: the RTL diff, the reason, the patch, and the LEC/STA/PV results. This record is invaluable when the same fix must be carried into the next chip revision or a derivative.

```
# Illustrative Conformal-ECO-style flow (command names modelled on the tool;
# not from a real run). G = golden (new RTL/netlist), R = revised (implemented)
read_design -golden rtl_v2/*.sv          ;# or the re-synthesised netlist_v2
read_design -revised impl_netlist_v1.v
set_system_mode lec
add_compared_points -all
compare                                     ;# -> list of non-equivalent points
analyze_eco -ecopin_dofile eco_pins.do    ;# find rectification points
optimize_patch -spare_file spares.txt -workdir eco_out ;# map patch to spares
write_eco_design -newfile impl_netlist_v2.v
```

Shape of a formal functional-ECO run (illustrative)

| Step               | Owner                      | Deliverable                                                  |
|--------------------|----------------------------|--------------------------------------------------------------|
| Bug fix in RTL     | RTL/design engineer        | RTL diff, updated tests, ECO ticket                          |
| Patch generation   | Synthesis/ECO engineer     | Patch netlist or Tcl, LEC report: patched netlist == new RTL |
| ECO implementation | PD engineer                | ECO-placed and routed DB, changed-cell list                  |
| Re-sign-off        | STA, PV, DFT, power owners | Incremental STA, DRC/LVS, updated ATPG and IR/EM reports     |

Who does what in a functional ECO

## 19.3 Timing ECOs

Near tapeout, the remaining timing violations are usually a handful of paths in specific corners. Sign-off STA tools can generate **timing ECO** fixes directly (PrimeTime ECO: `fix_eco_timing`, `fix_eco_drc`; Tempus ECO), because they have the accurate SI/POCV view of timing. They propose sizing, Vt swaps and buffer insertions with physical awareness (free space and legal sites from the DEF), write a change list, and the implementation tool applies it with ECO placement and routing. Several loops of 'sign-off STA -> ECO -> extract -> STA' are normal in the final weeks. Leakage and power recovery (swapping to HVT where slack allows) is done the same way.

## 19.4 Metal-only ECOs, spare cells and ECO cells

After the base (FEOL) layers of the wafers are processed - or at least after their masks have been made - changing a transistor costs a new full mask set and months. But changing only upper metal and via masks costs a fraction and can be done on **held wafers**: foundries can process wafers up to a chosen metal layer and park them ('metal hold' / 'pre-metal hold'), so a metal ECO resumes from there and first-fix silicon comes back in weeks rather than months.

| before the ECO:                                               | after a metal-only ECO:          |
|---------------------------------------------------------------|----------------------------------|
| A ----[AND2]---- Y (bug: needs A&B&C)                         | A ----[AND2]---+                 |
| B ----[ ]                                                     | B ----[ ] +---[spare AND2]---- Y |
|                                                               | C -----+ (inputs were            |
| spare AND2 (inputs tied to TIELO,                             | tied off; now                    |
| output floating) 40 um away                                   | rewired in M2-M5)                |
| only metal/via masks change; transistors stay where they were |                                  |

Metal-only ECO using a pre-placed spare cell

- **Spare cells** (Chapter 15) are placed during implementation: a mix of inverters, NAND/NOR, AOI/OAI, muxes and flops, with inputs tied off. The ECO tool maps the patch onto the nearest suitable spare cells; the distance matters for timing, so they are spread uniformly.
- **ECO gate-array cells (programmable fillers)**: base cells whose transistors are pre-built but unconnected, placed as filler in empty space. A metal ECO turns them into the needed gates by adding the intra-cell connections in metal. They give more flexibility than a fixed spare-cell mix at the cost of lower density and performance.
- Only layers from a chosen metal upward are changed ('M2+ ECO', 'via3+ ECO'); the fewer layers changed, the cheaper and faster the respin - but the harder the routing of the patch.
- Post-mask ECOs are also the standard way to fix bugs found on first silicon in a B0/B1 stepping ('metal fix' revisions).

## 19.5 The tapeout process

**Tapeout** is the release of the final layout database to the foundry (historically on magnetic tape). It is a formal process with a checklist, a data package and a sign-off meeting, typically owned by a tapeout/PD lead with representatives from every team, and it is scheduled months in advance because the foundry reserves capacity in its mask shop and fab.

```

final block DBs --> chip assembly --> final PV/STA/IR/LEC --> stream-out
                                     |
seal ring, logos, fill, marks <-----+
                                     |
                                     v
merged GDSII/OASIS --> foundry DRC/LVS at the foundry ('tapeout checks')
                                     |
                                     v
tapeout forms / jobview (layer list, mask options, CD bias, ID) --> mask shop

```

Tapeout data flow

### Final checks

- All sign-off criteria of Chapter 18 met or waived, on the exact database to be streamed out.
- Chip-level items: I/O ring and ESD connectivity, bump/pad coordinates matching the package design (Chapter 20), power domains, top-level timing between blocks, analog/IP integration reviews (Chapter 12), chip-level ERC.
- IP deliverables verified: every hard IP (memories, PHYs, PLLs) is the exact version the foundry and IP vendor approved, with the vendor's GDS merged, and any 'IP tag'/tracking layer requirements met.
- Documentation: DRC/LVS summaries, waiver list, top-level pin list, die size, layer usage.

### Stream-out: GDSII and OASIS

The final layout is written in **GDSII** (Stream format, the industry standard since the 1970s, a binary hierarchical format of cells, boundaries, paths, texts and cell references, with each shape tagged by a layer number and datatype - originally 0-63, commonly 0-255, and up to the 16-bit field limit in modern tools) or **OASIS** (SEMI P39, typically several to ten times more compact thanks to repetitions and delta encoding, and free of GDSII limits such as the 16-bit record length that caps the vertex count of a polygon). Advanced-node chips are delivered in OASIS because full-chip GDSII with fill can reach hundreds of GB.



A **layer map** file translates the tool's layer names to the foundry's layer/datatype numbers. Getting it wrong is catastrophic yet easy: a missing entry silently drops a layer, a swapped datatype turns drawing shapes into text or into a 'blockage' layer that is not manufactured.

```
# layer map (format and numbers illustrative; each PDK defines its own)
# tool_layer  purpose      gds_layer  gds_datatype
M1            drawing      31          0
M1            pin          31          2
M1            label        31          1
VIA1          drawing      51          0
M2            drawing      32          0
M2            fill         32          9
PR_BOUNDARY   drawing      235         0
```

A layer map, the Rosetta stone of stream-out

## Seal ring, logos, fill and marks

- **Seal ring:** a continuous ring of all metal and via layers (and diffusion) around the die edge, grounded, that stops cracks from the dicing saw and moisture/ion ingress through the dielectric stack from reaching the circuitry. Usually a foundry-provided structure; its corners are often chamfered.
- **Scribe line** (saw street) outside the seal ring holds the foundry's process-control monitors and alignment marks; designers normally do not draw it.
- **Logos, die ID, mask revision text:** drawn in top metal (and often in every metal layer's revision field) so a die can be identified under a microscope; must obey DRC and density rules.
- **Dummy fill** for all layers (front-end: diffusion and poly fill; back-end: metal fill), inserted with the foundry's fill utility or deck, then included in the final DRC/extraction.
- **Chip-level corner and CMP structures,** alignment and overlay marks where the foundry requires them in the die.

## Foundry checks, jobview and tapeout forms

The foundry reruns DRC with its own deck version on the received database (sometimes also LVS on provided netlists), checks the data against the order (die size, layer list, frame) and reports violations back. The customer fills the **tapeout forms** (often called a jobview or tape-out information sheet): product ID, process option and metal stack, list of layers to be made into masks and their revision, die and frame dimensions, fill and optical shrink options, bump/probe information and the order quantity. Errors on the forms (wrong metal option, a mask layer left off) are as fatal as layout errors.

## Tapeout-day checklist (condensed)

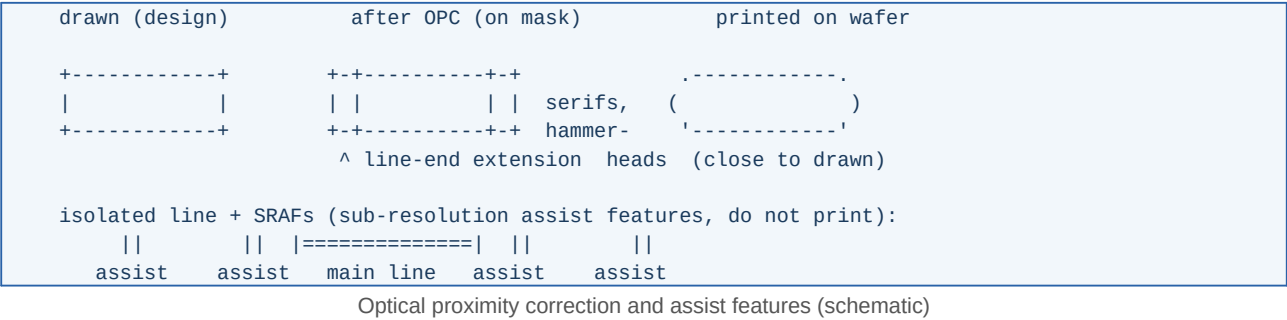
- ☐ Database frozen: tagged in revision control with the exact netlist, UPF, SDC, libraries, PDK and rule-deck versions used for sign-off.
- ☐ Final stream-out done from the frozen DB with the approved layer map; file checksums recorded.
- ☐ DRC/LVS/antenna/density/ERC rerun on the streamed-out file (not on the tool database) with the foundry-released decks.
- ☐ Seal ring, logo, die ID and mask revision text present and DRC clean; fill inserted and included in the last extraction.
- ☐ IP versions and vendor sign-offs checked against the foundry's approved list; required IP tag layers present.
- ☐ Die size, frame, scribe width and orientation match the package drawing and the tapeout form.
- ☐ Tapeout forms (layer list, mask revision per layer, process options) reviewed by two people; waiver list attached.
- ☐ Metal-hold instructions (which wafers to park before which layer) agreed with the foundry.

INTERVIEW INSIGHT - WHY DO MASKS GET REVISION LETTERS?

Each mask layer carries its own revision code. A metal-only ECO changes, say, only M3-M6 and V2-V5, so only those masks are re-made and get new revision letters, while the base-layer masks keep theirs. Chip steppings such as A0 -> A1 (metal fix) versus A0 -> B0 (base-layer change) follow the same logic, and the die ID text drawn in each layer lets failure analysis tell which combination a given part was built with.

19.6 Mask data preparation: OPC, RET, SRAF and fracturing

The shapes a designer draws are not the shapes that print. At advanced nodes, feature sizes are far below the 193-nm wavelength of ArF immersion lithography (and still close to the resolution limit for 13.5-nm EUV), so diffraction rounds corners, shortens line ends and makes printed width depend on neighbours. **Mask data preparation (MDP)** - performed by the foundry or mask shop - transforms the design into mask patterns that print as intended.



| Step                                    | Purpose                                                                                                                                                                                                           |
|-----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Retargeting / biasing                   | Adjust drawn sizes to the target that should print (rule-based corrections)                                                                                                                                       |
| OPC (optical proximity correction)      | Model-based: fragment every edge and move fragments so a lithography + resist model predicts the printed contour matches the target; adds serifs and hammerheads. Full-chip OPC takes very large compute clusters |
| RET (resolution enhancement techniques) | Umbrella term: OPC, SRAFs, phase-shift masks, off-axis illumination, source-mask optimisation (SMO), multi-patterning decomposition                                                                               |
| SRAF insertion                          | Assist features below the printing threshold make isolated features behave like dense ones, enlarging the process window                                                                                          |
| Multi-patterning decomposition          | Split a layer into 2-4 masks (LELE, SADP/SAQP cut/mandrel masks) with colour-aware rules                                                                                                                          |
| ILT (inverse lithography)               | Compute the mask as an inverse problem; curvilinear shapes, now practical with multi-beam writers                                                                                                                 |
| Verification (ORC/LFD)                  | Simulate the corrected mask over the process window to find hotspots before writing                                                                                                                               |
| Fracturing                              | Convert polygons into the primitive shapes (trapezoids/rectangles) of the mask writer's format (e.g. MEBES, VSB formats), with proximity-effect correction for the e-beam                                         |

Mask data preparation steps

Mask writing and mask sets

A photomask (reticle) is a quartz plate with a patterned absorber (chrome or MoSi for DUV; a multilayer reflector with an absorber for EUV), written by **electron-beam** writers (variable-shaped beam, or multi-beam for complex/curvilinear masks) or laser writers for non-critical layers. It is inspected for defects, repaired, and covered with a **pellicle** to keep particles out of focus. A scanner images the reticle at 4x reduction, so a 26 mm x 33 mm maximum field on the wafer corresponds to 104 mm x 132 mm on the mask.

A **mask set** is one mask per patterned layer (and per exposure for multi-patterned layers). The number and cost grow steeply with the node. The figures below are widely quoted industry orders of magnitude, not quotes from any foundry.

| Node (approx.)            | Masks in a set | Mask set cost (order of magnitude) |
|---------------------------|----------------|------------------------------------|
| 180-130 nm                | ~20-35         | ~\$0.1-0.3 M                       |
| 65-40 nm                  | ~35-45         | ~\$0.5-1.5 M                       |
| 28 nm                     | ~40-50         | ~\$1-3 M                           |
| 16/14/12 nm FinFET        | ~60-70         | ~\$4-7 M                           |
| 7 nm (DUV + EUV variants) | ~70-85         | ~\$10-15 M                         |
| 5 nm and below            | ~80-100+       | ~\$15-30+ M                        |

Mask sets by node (rough public estimates; actual numbers vary widely by foundry, options, EUV layer count and contract)

## 19.7 MPW shuttles

A **multi-project wafer (MPW)** run shares one mask set among many designs: each customer buys a slot (a few mm<sup>2</sup> to tens of mm<sup>2</sup>) on a shared reticle, and receives a small number of packaged or bare dies (tens to a few hundred). Foundries and brokers (for example Europractice, MOSIS historically, and foundry-run shuttle programs) schedule shuttles on fixed dates. MPWs are the standard way to prototype test chips, IP and academic designs, and open-source PDK programs have offered free or low-cost shuttles for SKY130 and GF180 designs. Limits: fixed schedule, few parts, no volume, and only the foundry's standard process options.

## 19.8 Reticles, dies per wafer and cost per die

The scanner exposes the wafer one **field** at a time; a field is at most 26 mm x 33 mm (858 mm<sup>2</sup>) on standard DUV/EUV scanners, which is the **reticle limit** for a single die. Small dies are arrayed several per field. The number of dies on a wafer is estimated by a classic formula that subtracts the partial dies at the edge:

$$\text{gross dies per wafer } DPW \approx \pi * (d/2)^2 / S - \pi * d / \sqrt{2 * S}$$

$d$  = usable wafer diameter (mm, after edge exclusion),

$S$  = die area including scribe (mm<sup>2</sup>)

$$\text{yield (Murphy): } Y = ((1 - \exp(-A * D0)) / (A * D0))^2 \quad A = \text{die area (cm}^2\text{)},$$

$D0$  = defects/cm<sup>2</sup>

$$\text{cost per good die} = \text{wafer cost} / (DPW * Y)$$

Dies per wafer and cost per good die (yield models: Chapter 21)

```
# Gross dies per wafer, yield and cost per good die on a 300-mm wafer
import math
D = 300.0          # wafer diameter, mm
EE = 3.0           # edge exclusion, mm
SCRIBE = 0.1       # scribe (saw) street added to each die edge, mm
D0 = 0.10          # defect density, defects/cm^2 (mature-process order of magnitude)
WAFER_COST = 15000.0 # USD per processed wafer (advanced-node order of magnitude)

def dpw_formula(w, h):
    s = (w + SCRIBE) * (h + SCRIBE); d = D - 2 * EE
    return math.pi * (d / 2) ** 2 / s - math.pi * d / math.sqrt(2 * s)

def dpw_count(w, h):
    """Exact count of whole dies on a grid (best of 4 grid offsets)."""
    R = D / 2 - EE; W, H = w + SCRIBE, h + SCRIBE; best = 0
    for ox in (0.0, W / 2):
        for oy in (0.0, H / 2):
            cnt = 0
            nx, ny = int(R // W) + 2, int(R // H) + 2
```

```

    for i in range(-nx, nx):
        for j in range(-ny, ny):
            x0, y0 = i * W + ox, j * H + oy
            if all(math.hypot(x, y) <= R for x in (x0, x0 + W) for y in (y0, y0 + H)):
                cnt += 1
        best = max(best, cnt)
    return best

print("300-mm wafer, %.0f-mm edge exclusion, %.2f-mm scribe, D0=%.2f/cm^2, wafer=$%.0f"
      % (EE, SCRIBE, D0, WAFER_COST))
print("%12s %8s %8s %8s %9s %11s" % ("die (mm)", "area", "formula", "counted",
                                     "Y(Murphy)", "$/good die"))
for w, h in ((2, 2), (5, 5), (10, 10), (15, 15), (20, 20), (26, 33)):
    a = w * h; A = a / 100.0
    y = ((1 - math.exp(-A * D0)) / (A * D0)) ** 2
    gross = dpw_count(w, h)
    print("%5.0f x %-5.0f %6.0f %9.0f %8d %8.1f%% %11.2f" % (w, h, a, dpw_formula(w, h), gross,
                                                            100 * y, WAFER_COST / (gross * y)))

```

dpw.py - gross dies per wafer by formula and by exact counting, yield and cost per good die

```

300-mm wafer, 3-mm edge exclusion, 0.10-mm scribe, D0=0.10/cm^2, wafer=$15000
die (mm)    area  formula  counted  Y(Murphy)  $/good die
2 x 2       4     15083    15110    99.6%      1.00
5 x 5       25     2482     2500     97.5%      6.15
10 x 10     100     601      612      90.6%     27.06
15 x 15     225     254      266      80.2%     70.32
20 x 20     400     136      148      67.9%    149.20
26 x 33     858      56       64      45.1%    520.06

```

Real output of python3 dpw.py

The formula agrees with exact grid counting to within a few percent (counting with the best grid offset gains a little). The economics are brutal for large dies: going from 10 x 10 mm to 20 x 20 mm quadruples area but cuts gross dies from 612 to 148 and yield from 91% to 68%, so cost per good die rises about 5.5x (\$27 -> \$149). A reticle-limit die costs over \$500 before packaging and test even with a mature defect density - one of the main arguments for chiplets (Chapter 20). The \$15k wafer cost and  $D0 = 0.1/\text{cm}^2$  are illustrative orders of magnitude.

#### AMORTISING THE MASK SET

Total cost per chip = (NRE) / volume + cost per good die + package + test. With a \$10 M mask set and 100 k units, masks add \$100 per chip - far more than the 10 x 10 mm die itself. At 10 M units they add \$1. This is why low-volume products use older nodes, MPWs or FPGAs (Chapter 23).

## 19.9 From tapeout to first silicon

|                                                                          |                                                                                                                                                            |
|--------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| week 0                                                                   | tapeout: GDS/OASIS + forms delivered                                                                                                                       |
| +1 to +2                                                                 | foundry checks, jobview sign-off, fix-and-resubmit if needed                                                                                               |
| +1 to +4                                                                 | MDP: OPC/RET, fracturing, mask writing, inspection, repair<br>(masks are written in layer order so fab can start early)                                    |
| +3 to +16                                                                | wafer fabrication: hundreds of process steps; roughly<br>1-1.5 days per mask layer is a common rule of thumb;<br>'hot lots' at premium priority run faster |
| +12 to +20                                                               | wafer test (probe) at foundry/OSAT, dicing, packaging<br>(engineering samples often in a fast-turn package)                                                |
| +14 to +22                                                               | first silicon on the bench -> bring-up (Chapter 22)                                                                                                        |
| totals: roughly 2-4 months at mature nodes, 4-6 months at advanced nodes |                                                                                                                                                            |

Typical timeline from tapeout to first silicon (orders of magnitude; strongly foundry-, node- and priority-dependent)

While the wafers are in the fab, the team is not idle: bring-up boards, test programs, silicon validation plans and software are prepared (Chapter 22), and a subset of wafers is typically held before the upper metals so that

metal-only ECO fixes can be processed quickly if bring-up finds a bug.

## 19.10 Summary

---

- ECOs are minimal, controlled changes after implementation: functional (formal ECO tools compute patches), timing (sign-off STA generates fixes) and DRC/Sl; every ECO needs LEC, STA and PV.
- Pre-mask ECOs may change any layer; post-mask ECOs change only metal/via masks, using spare cells or ECO gate-array fillers, and can use wafers held before metallisation.
- Tapeout: final checks, stream-out to GDSII/OASIS through a verified layer map, seal ring, logos, fill, foundry DRC and tapeout forms.
- Mask data preparation: retargeting, OPC, SRAFs, multi-patterning decomposition, ILT, verification, fracturing, e-beam writing.
- Mask sets range from ~\$0.1 M at mature nodes to tens of \$M at the leading edge; MPW shuttles share the cost.
- Dies per wafer  $\sim \pi (d/2)^2/S - \pi d/\sqrt{2S}$ ; our run showed cost per good die rising 5.5x when die area quadruples from 100 to 400 mm<sup>2</sup>.

## 19.11 Exercises

---

- 1 A bug requires changing an AND to an AND-OR function after base layers are fabricated. Which spare cells would you want nearby, and what checks must be rerun?
- 2 Explain why a missing entry in the layer map might not be caught by the design team's own DRC but would be caught by LVS - or might not be caught at all.
- 3 Use dpw.py to find the die size (square) at which the formula and the exact count differ by more than 10%. Why does the formula break down for large dies?
- 4 Your product needs 50 k units per year. Compare total cost per unit for a 100-mm<sup>2</sup> die at 28 nm (mask set \$2 M, wafer \$3 k) versus a 40-mm<sup>2</sup> die at 7 nm (mask set \$12 M, wafer \$10 k), ignoring package and test.
- 5 List three things that can go wrong on tapeout forms and their consequence.

## PART V

# After Tapeout

What happens once the GDSII has gone to the foundry: packaging, 2.5D/3D integration and chiplets; manufacturing test, yield and reliability physics; silicon bring-up, validation, characterisation and qualification; and the methodology, project management and economics that decide whether an ASIC is worth building at all.

## 20. Packaging, 2.5D/3D Integration and Chiplets

The GDSII you signed off in Chapter 19 describes a sliver of silicon a fraction of a millimetre thick that cannot survive a fingerprint, cannot be soldered to a board and would cook itself within seconds at full power. The **package** turns that die into a product: it carries signals and power between micron-scale pads on the die and millimetre-scale balls on a printed circuit board, it pulls heat out, and it protects the die mechanically and chemically for ten or more years. For modern high-performance parts the package has become a design problem as hard as the die itself: a large AI accelerator is no longer one chip but a system of several compute dies, stacks of HBM memory and a silicon interposer, all assembled in one package.

Packaging is often treated as somebody else's problem until it is too late. In reality the package decisions - bump map, pad ring, power delivery, thermal solution - must be made during floorplanning (Chapter 14), and the package engineer is part of the core team from architecture onward. This chapter gives the ASIC engineer enough depth to have that conversation.

### 20.1 What a package does

| Role                          | What it means                                                                   | Typical owner concerns                                                                                       |
|-------------------------------|---------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| <b>Electrical: signals</b>    | Route every die I/O to a board ball with controlled impedance and low crosstalk | Trace length, impedance (50 ohm single-ended, 85-100 ohm differential), return paths, loss at SerDes Nyquist |
| <b>Electrical: power</b>      | Deliver tens to hundreds of amps with millivolts of droop                       | Number of power balls/bumps, plane layers, loop inductance, decoupling capacitors in the package             |
| <b>Thermal</b>                | Move watts from the junction to ambient                                         | theta_JC, lid and thermal interface material (TIM), heat spreader, hot spots                                 |
| <b>Mechanical</b>             | Hold the die flat, survive assembly and temperature cycling                     | Warping, coefficient of thermal expansion (CTE) mismatch, underfill, solder joint fatigue                    |
| <b>Environmental</b>          | Keep moisture, ions and light out                                               | Moisture sensitivity level (MSL), mould compound, corrosion                                                  |
| <b>Interface to the board</b> | Fit the customer's PCB rules and assembly line                                  | Ball pitch (e.g. 0.4-1.0 mm), body size, footprint, rework                                                   |

Table 20.1 - The jobs of a package.

### 20.2 Package families

Package types differ in how the die is connected (wire bond or flip chip), what it is mounted on (lead frame, laminate substrate, redistribution layers) and how the package attaches to the board (leads, lands or solder balls). The main families an ASIC team chooses from are:

| Package                        | Construction                                                                     | I/O range | Typical use                                                      |
|--------------------------------|----------------------------------------------------------------------------------|-----------|------------------------------------------------------------------|
| <b>QFN (quad flat no-lead)</b> | Die on a copper lead frame, wire bonded, moulded; exposed thermal pad underneath | 8-100     | Low-cost MCUs, PMICs, sensors, RF                                |
| <b>QFP / LQFP</b>              | Lead frame with gull-wing leads on four sides                                    | 32-256    | Legacy MCUs, industrial parts that need visual solder inspection |
| <b>Wire-bond BGA (PBGA)</b>    | Die wire bonded to a laminate substrate with solder balls on the bottom          | 100-700   | Consumer SoCs, networking, cost-sensitive mid-range              |
| <b>Flip-chip BGA (FCBGA)</b>   | Die flipped face-down onto a build-up substrate through bumps; lid or bare die   | 500-5000+ | CPUs, GPUs, FPGAs, switches, AI accelerators                     |
| <b>WLCSP (fan-in)</b>          | Balls placed directly on the die through an RDL; package = die size              | 4-400     | Mobile: PMICs, RF, audio, sensors                                |

| Package                                             | Construction                                                                   | I/O range                                 | Typical use                                                      |
|-----------------------------------------------------|--------------------------------------------------------------------------------|-------------------------------------------|------------------------------------------------------------------|
| <b>Fan-out wafer/panel level (FOWLP, InFO-like)</b> | Dies embedded in a reconstituted mould wafer; RDL fans out beyond the die edge | 100-2000+                                 | Mobile application processors, RF modules, multi-die integration |
| <b>2.5D / 3D</b>                                    | Several dies on an interposer or stacked (see 20.5-20.6)                       | Thousands to 100,000s of die-to-die wires | HPC, AI, HBM-based products                                      |

Table 20.2 - Main package families (I/O ranges are typical, not limits).

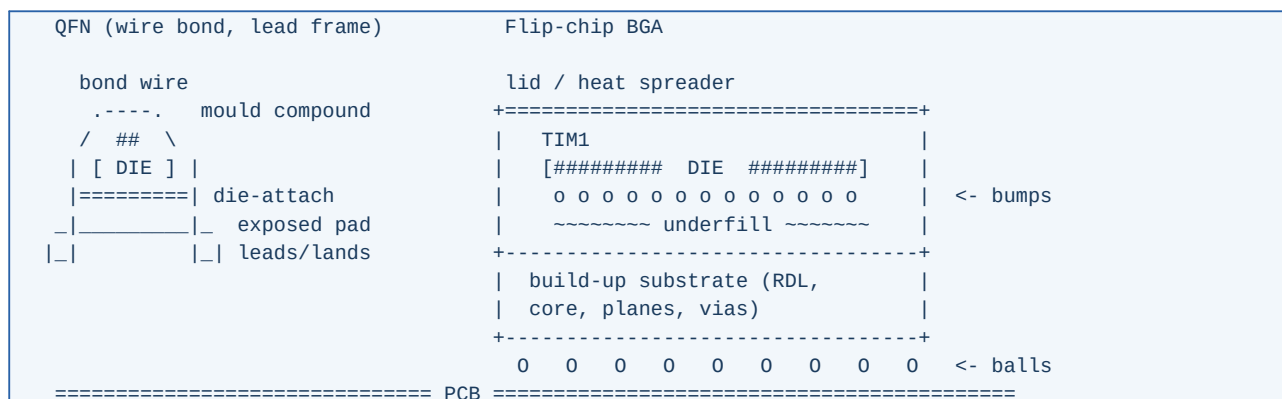


Figure 20.1 - Cross-sections: a wire-bonded OFN and a lidded flip-chip BGA (not to scale).

### 20.3 Wire bond versus flip chip

In **wire bonding** the die sits face-up and fine gold or copper wires (typically 15-25  $\mu\text{m}$  diameter) connect pads on the die periphery to the lead frame or substrate. It is cheap, mature and flexible, but the I/O count is limited by the die perimeter, each wire adds roughly 1 nH of inductance per millimetre, and all power has to enter from the edge - which becomes an IR-drop problem for large, power-hungry dies.

In **flip chip** the die is turned face-down and connected through an **area array** of bumps across its whole surface. Classic **C4** (controlled collapse chip connection) solder bumps are on a pitch of roughly 130-200  $\mu\text{m}$ ; **copper pillar** bumps with a solder cap allow finer pitch (tens of  $\mu\text{m}$ ) with less collapse; **micro-bumps** on 2.5D/3D interposers reach roughly 25-55  $\mu\text{m}$  pitch. After reflow the gap between die and substrate is filled with **underfill** epoxy, which shares the thermo-mechanical strain so that the bumps do not crack when the silicon (CTE about 2.6 ppm/K) and the organic substrate (CTE roughly 15-17 ppm/K) expand by different amounts.

| Aspect            | Wire bond                                          | Flip chip                                                         |
|-------------------|----------------------------------------------------|-------------------------------------------------------------------|
| I/O placement     | Periphery only (pad ring)                          | Whole die area (bump array); I/O can sit near the logic it serves |
| Power delivery    | From the edges; long wires; IR drop in the centre  | Power bumps everywhere over the grid; much lower R and L          |
| Inductance        | About 1 nH/mm of wire; few nH per connection       | Tens of pH per bump; loop L dominated by substrate                |
| Heat path         | Through die back into the lead frame / board       | Through the die backside into lid and heat sink                   |
| Cost              | Lowest for low pin counts                          | Higher: bumping, build-up substrate, underfill                    |
| Die design impact | Pad-limited vs core-limited trade-off (Chapter 14) | Bump plan, RDL, bump-to-I/O assignment are part of the floorplan  |

Table 20.3 - Wire bond and flip chip compared.



PAD-LIMITED DIES

If a wire-bonded design needs 400 signal and power pads at a 60 um pad pitch, the perimeter must be at least 24 mm, i.e. a die of 6 x 6 mm even if the logic fits in 2 x 2 mm. The die is **pad-limited** and you pay for empty silicon. Moving to flip chip (or staggered pads) turns it into a core-limited die and can halve its cost.

20.4 Substrates, RDL and the package design flow

A flip-chip **substrate** is a small, very dense PCB. A typical build-up substrate has a glass-reinforced core with plated through holes and several thin build-up layers on each side (written e.g. 4-2-4: four build-up layers, a two-layer core, four more), with laser-drilled micro-vias and line/space down to roughly 8-15 um in advanced substrates. Layers are allocated to signal routing, power planes and ground planes, much like a board stack-up. Substrates are a major part of the cost of a large FCBGA and have their own supply constraints (ABF build-up film shortages have delayed products in the past).

A **redistribution layer (RDL)** is one or more thin metal layers (copper, polyimide dielectric) patterned on top of a die or a reconstituted wafer to move connections from where the pads are to where the bumps or balls must be. Fan-in WLCSP uses RDL on the die itself; fan-out uses RDL that extends beyond the die edge onto the mould compound, which is how InFO-style packages achieve many I/Os and thin profiles without a laminate substrate.

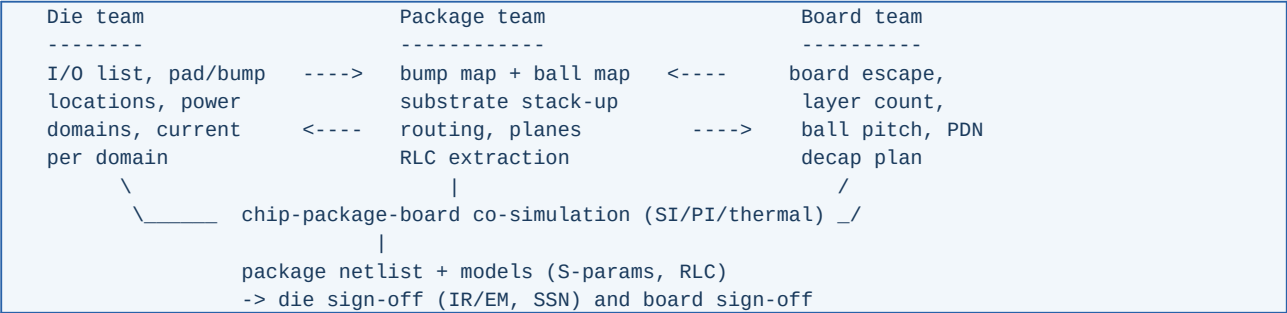


Figure 20.2 - Package design is a three-way negotiation that starts at floorplanning.

Package design is done in dedicated tools (Cadence Allegro Package Designer / SiP Layout, Siemens Xpedition Substrate Integrator, Synopsys 3DIC Compiler for multi-die). The deliverables that come back to the die team are a **bump/ball map**, a package netlist, extracted package models (RLC per net or S-parameters) for signal integrity, and a package power model for chip-package power integrity analysis (Chapter 18).

20.5 2.5D integration: interposers and bridges

**2.5D** places several dies side by side on a common high-density interconnect layer that routes thousands of fine wires between them. The dies are not stacked on each other - hence '2.5'.

- **Silicon interposer (CoWoS-like).** A large passive silicon die, made on an older process, carrying several metal layers with sub-micron wires and **through-silicon vias (TSVs)** down to the package substrate. Logic dies and HBM stacks sit on it via micro-bumps. It gives the densest die-to-die wiring but the interposer can exceed the lithography reticle size (stitched exposures) and is costly.
- **Silicon bridge (EMIB-like).** Instead of a full interposer, a small silicon bridge die is embedded in the organic substrate only under the edges of the dies it connects. Only the die-to-die wires use fine pitch; everything else uses ordinary substrate bumps. Cheaper and scales to large packages.
- **Organic / RDL interposer (CoWoS-R, InFO-type).** A fan-out RDL layer stack acts as the interposer: coarser than silicon (line widths of a couple of um) but cheaper, larger and mechanically more compliant.

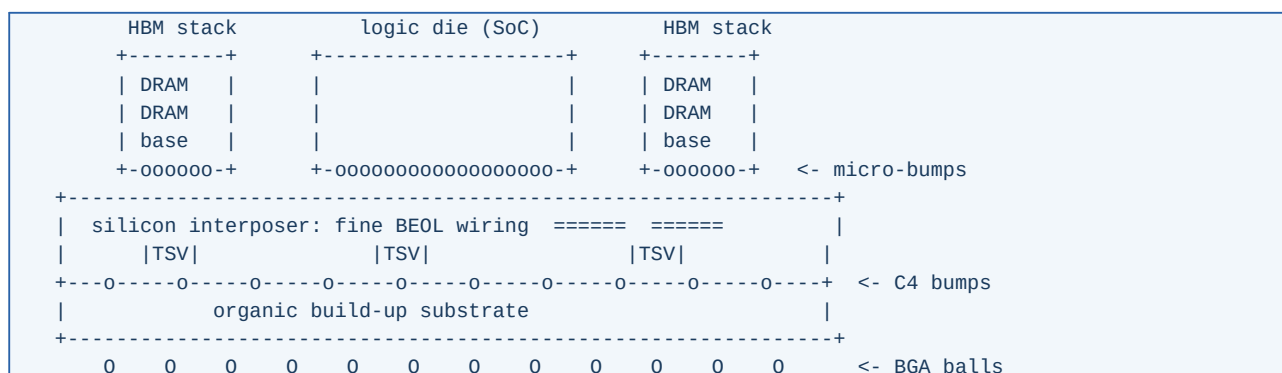


Figure 20.3 - A 2.5D package: logic die and HBM on a silicon interposer (CoWoS-like).

## 20.6 3D integration: TSVs and hybrid bonding

**3D** stacks dies vertically. Vertical connections pass through the silicon in **TSVs** - copper-filled holes typically a few microns to ~10 um in diameter - or directly between face-to-face bonded dies. Two bonding technologies dominate:

- **Micro-bump (solder) stacking.** Dies are thinned, TSVs expose on the backside and the next die is soldered on with micro-bumps (pitch of tens of um). HBM DRAM stacks are built this way (or with newer hybrid bonding in future generations).
- **Hybrid bonding (Cu-Cu direct bond, SoIC-like, Foveros Direct-like).** Polished copper pads embedded in oxide are bonded directly without solder, at pitches of roughly 10 um down to a few um. This gives 10-100x the connection density of micro-bumps and very low energy per bit, at the cost of extreme surface flatness and cleanliness requirements.

| Interconnect                     | Pitch (typical order) | Density per mm <sup>2</sup> | Energy/bit (order)            |
|----------------------------------|-----------------------|-----------------------------|-------------------------------|
| Package substrate trace (2D MCM) | 100-150 um bump       | ~50-100                     | ~1-2 pJ/bit (SerDes-less D2D) |
| Silicon interposer micro-bump    | 40-55 um              | ~400-600                    | ~0.3-0.5 pJ/bit               |
| Hybrid bond                      | 3-10 um               | 10,000-100,000              | <0.1 pJ/bit                   |

Table 20.4 - Orders of magnitude of die-to-die connection technologies (values vary widely by vendor and generation).

### PITFALL: 3D IS A THERMAL PROBLEM FIRST

Stacking a hot logic die under (or over) another die multiplies power density and puts silicon, bond layers and possibly DRAM in the heat path. DRAM retention degrades at high temperature, so a memory-on-logic stack may force refresh-rate increases or power caps. Always run a thermal model of the stack before committing to a 3D architecture - and check that TSV keep-out zones (stress affects nearby transistor mobility) are in the floorplan.

## 20.7 Chiplets, UCle and HBM

A **chiplet** architecture splits what would have been one large monolithic SoC into several smaller dies assembled in one package. The motivations are economic and practical:

- **Yield.** Yield falls roughly exponentially with die area (Chapter 21). Four 150 mm<sup>2</sup> dies can yield far better than one 600 mm<sup>2</sup> die, and bad chiplets are discarded before assembly.
- **Reticle limit.** A single exposure field is about 26 x 33 mm (~858 mm<sup>2</sup>); beyond that you must use multiple dies.
- **Mixed nodes.** Put compute on the leading-edge node and I/O, SerDes and analog - which scale poorly - on a cheaper mature node.
- **Reuse and product variants.** One compute chiplet plus a different I/O die yields a family of products with one tapeout of the expensive part.

The price is die-to-die (D2D) interfaces on every chiplet (area, power, latency), assembly cost and yield, a far harder test problem (below) and multi-die sign-off. Standard D2D interfaces make chiplets from different teams or vendors interoperable: **UCIe** (Universal Chiplet Interconnect Express) defines a physical layer for standard (organic substrate) and advanced (interposer/bridge) packages, a die-to-die adapter with CRC/retry, and protocol mapping for PCIe, CXL and streaming traffic; BoW (Bunch of Wires) is a simpler open alternative. The protocol details are covered in Chapter 22 of the companion Communication Protocols guide; here the point is that the D2D PHY is placed at the die edge facing its partner, its bump region is fixed by the standard, and it must be floorplanned together with the package.

**HBM (High Bandwidth Memory)** stacks several DRAM dies on a base (logic) die connected by TSVs, and exposes a very wide interface (1024 data bits per stack in HBM2/HBM3, doubled to 2048 in HBM4) at moderate per-pin rates. Because thousands of wires are needed, HBM requires an interposer or bridge. The ASIC team integrates an HBM controller and PHY IP, places the PHY at the die edge with its fixed bump footprint aligned to the HBM stack, and co-designs the interposer routing (short, length-matched, typically a few mm). Memory interfaces including HBM are covered in Chapter 20 of the companion Communication Protocols guide.

#### KNOWN-GOOD DIE (KGD)

In a multi-die package, one bad die scraps the whole assembly - including several expensive good dies and the interposer. If each of  $N$  dies has probability  $p$  of being good after wafer test, the assembled package is good with probability about  $p^N$  (ignoring assembly yield). Four dies at 98% give 92%; at 90% only 66%. So chiplets must be tested to **known-good-die** quality at wafer sort: full-speed tests, burn-in-like screens where needed and access to D2D interfaces through loopback or DFT (IEEE 1838 defines a test access architecture for 3D stacks), because micro-bumps are too small and fragile for conventional probing at full count.

## 20.8 Package-chip co-design: bumps, power and signal integrity

### Bump planning

The **bump map** assigns every bump site on a flip-chip die to a signal, a power net or ground. Signal bumps sit close to the I/O cells they serve, high-speed differential pairs get shielding ground bumps, and power/ground bumps are spread across the core in a regular pattern that aligns with the top-metal power grid (Chapter 14). The number of power bumps is set by the current each bump can carry without electromigration failure and by the IR drop budget. A first-order budget is easy to compute:

```
# Rough power-bump budget for a flip-chip die (all inputs are assumptions)
P, VDD = 40.0, 0.75          # W, V core supply
I = P / VDD                  # A total core current
I_bump = 0.15                # A allowed per bump (EM-limited; foundry/package specific)
margin = 1.5                  # derate for non-uniform current sharing
n_pwr = I / I_bump * margin   # VDD bumps; the same number again for VSS
pitch_um = 150.0              # C4 bump pitch
die_mm = 10.0
sites = int(die_mm * 1000 / pitch_um) ** 2
print(f"Core current          : {I:6.1f} A")
print(f"VDD bumps needed      : {n_pwr:6.0f} (+ same for VSS)")
print(f"Bump sites on {die_mm:.0f}x{die_mm:.0f} mm : {sites:6d} at {pitch_um:.0f} um pitch")
print(f"Fraction used by power   : {2 * n_pwr / sites * 100:5.1f} %")
```

bumps.py - first-order power bump count (all inputs are assumptions).

```
Core current          : 53.3 A
VDD bumps needed      : 533 (+ same for VSS)
Bump sites on 10x10 mm : 4356 at 150 um pitch
Fraction used by power : 24.5 %
```

Output of bumps.py (python3).

A quarter of all bump sites go to one supply pair, before counting I/O supplies, other core domains and signals. This is why large flip-chip dies often have far more power and ground bumps than signal bumps, and why the bump plan must be frozen early: every change moves top-level power straps.

## Power delivery and simultaneous switching noise

The power delivery network (PDN) runs from the voltage regulator on the board, through board planes and decoupling capacitors, package balls, package planes and package capacitors, bumps and finally the on-die grid and on-die decap. Each stage has resistance and inductance, and together with the capacitances they form resonances. The designer targets a **PDN impedance** below  $Z_{\text{target}} = (\text{allowed ripple}) / (\text{transient current})$  across frequency, typically from DC up to hundreds of MHz: the VRM covers low frequencies, board and package capacitors the middle, and on-die capacitance the highest frequencies. The worst droop is usually at the **first droop** resonance (tens to ~200 MHz) between package inductance and on-die capacitance.

$$Z_{\text{target}} = dV_{\text{allowed}} / dI_{\text{step}} \quad \text{e.g. } 0.05 \text{ V} \times 0.75 \text{ V} / 20 \text{ A} \sim 1.9 \text{ mohm}$$

$$V_{\text{noise}} \sim L_{\text{loop}} \times dI/dt \quad (\text{inductive, 'L di/dt' noise})$$

$$f_{\text{res}} = 1 / (2 \times \pi \times \sqrt{L_{\text{pkg}} \times C_{\text{die}}}) \quad \text{first-droop resonance}$$

Power integrity rules of thumb.

**Simultaneous switching noise (SSN)**, also called ground bounce, occurs when many output drivers switch at once and their combined current ramp flows through the shared inductance of the package power and ground connections. The script below shows why output banks need enough dedicated power/ground pins:

```
# Simultaneous switching noise: dV = N * L_eff * dI/dt, with shared return pins
L_pin = 1.0e-9      # H, effective loop inductance of one ground pin+bond wire
I_drv = 0.020       # A, peak current per output driver
t_r   = 0.5e-9      # s, output edge (current ramp) time
for n_out, n_gnd in [(8, 1), (32, 1), (32, 4), (32, 8)]:
    didt = n_out * I_drv / t_r          # total current slew on the return path
    dv = (L_pin / n_gnd) * didt         # ground pins in parallel
    print(f"{n_out:2d} outputs, {n_gnd} gnd pins: ground bounce ~ {dv*1000:6.0f} mV")
```

ssn.py - ground bounce of an output bank.

```
8 outputs, 1 gnd pins: ground bounce ~ 320 mV
32 outputs, 1 gnd pins: ground bounce ~ 1280 mV
32 outputs, 4 gnd pins: ground bounce ~ 320 mV
32 outputs, 8 gnd pins: ground bounce ~ 160 mV
```

Output of ssn.py (python3).

32 outputs on a single ground wire would bounce the local ground by over a volt - enough to cause false switching in quiet inputs. Remedies: more power/ground pins per I/O bank (I/O libraries specify a maximum number of simultaneously switching outputs per supply pair), slew-rate-controlled drivers, staggered switching, differential signalling, on-package decoupling and flip chip.

## Package parasitics and signal integrity

Every package net has series resistance and inductance and capacitance to its neighbours and planes. For slow I/O a lumped RLC per pin (typical orders: a few nH and ~1 pF for wire-bond BGA nets, a fraction of a nH for flip-chip nets) is enough. For multi-gigabit SerDes, DDR and HBM the package is characterised as **S-parameters** (insertion loss, return loss, crosstalk) and simulated end to end with the die I/O model (IBIS or IBIS-AMI) and the board channel. Via stubs, ball-field breakout and plane changes create impedance discontinuities that show up as reflections in the eye diagram.

## 20.9 Thermal design

Heat flows from the transistor junctions through the silicon to the package and on to ambient. Engineers model this path as a chain of **thermal resistances** (in deg C per watt), just as they would resistors in series:

$$T_j = T_a + P \times \theta_{JA}$$

$$\theta_{JA} \sim \theta_{JC} + \theta_{CS} + \theta_{SA} \quad (\text{junction} \rightarrow \text{case} \rightarrow \text{sink} \rightarrow \text{ambient})$$

$$T_j = T_c + P \times \theta_{JC} \quad (\text{when the case temperature is known})$$

Steady-state junction temperature.

$\theta_{JA}$  is not a property of the package alone: JEDEC JESD51 defines how it is measured on standard test boards (1s or 2s2p) in still air or a wind tunnel, and the real value on your board can differ substantially.  $\theta_{JC}$  measures the path to the top of the package (important when a heat sink is attached);  $\psi_{JT}$  and  $\psi_{JB}$  are characterisation parameters used to estimate  $T_j$  from a measured case-top or board temperature. The calculation itself is simple enough to script:

```
# Junction temperature for a few package / cooling options (illustrative thetas)
P = 6.0          # W, chip power
T_amb = 45.0     # deg C, ambient inside the enclosure
cases = [        # name, theta_JA (C/W) or (theta_JC, theta_CS, theta_SA)
    ("QFN-48, no heatsink, 2s2p board", 28.0, None),
    ("FC-BGA 23mm, no heatsink", 14.0, None),
    ("FC-BGA + small passive sink", None, (0.3, 0.2, 5.5)),
    ("FC-BGA + fan heatsink", None, (0.3, 0.2, 1.8)),
]
for name, tja, stack in cases:
    if stack:
        tja = sum(stack)
    tj = T_amb + P * tja
    ok = "OK" if tj <= 105 else "TOO HOT"
    print(f"{name:34s} theta_JA={tja:5.1f} C/W Tj={tj:6.1f} C {ok}")
# Max power allowed for Tj_max = 105 C with the fan heatsink
tja = 0.3 + 0.2 + 1.8
print(f"Max power with fan sink at Tj<=105C: {(105 - T_amb) / tja:.1f} W")
```

tj.py - junction temperature for several package and cooling options (illustrative theta values).

```
QFN-48, no heatsink, 2s2p board    theta_JA= 28.0 C/W Tj= 213.0 C TOO HOT
FC-BGA 23mm, no heatsink          theta_JA= 14.0 C/W Tj= 129.0 C TOO HOT
FC-BGA + small passive sink       theta_JA=  6.0 C/W Tj=  81.0 C OK
FC-BGA + fan heatsink             theta_JA=  2.3 C/W Tj=  58.8 C OK
Max power with fan sink at Tj<=105C: 26.1 W
```

Output of tj.py (python3).

A 6 W chip is far too hot for a small QFN and even for a bare BGA in a warm enclosure; with a heat sink it is comfortable. Typical maximum junction temperatures are 105-125 deg C for commercial and industrial parts and up to 150 deg C for automotive grade 0. Remember that leakage rises strongly with temperature, which raises power, which raises temperature: at high power densities the calculation must iterate to check for **thermal runaway**.

- **Heat spreaders and lids** flatten hot spots; **TIM1** (between die and lid) and **TIM2** (between lid and heat sink) are often the largest resistances in the stack.
- **Bare-die (lidless)** packages remove one TIM layer and are used for the highest-power parts, with careful mechanical design of the heat-sink pressure.

- **Hot spots** matter as much as total power: a 1 mm<sup>2</sup> block at several W/mm<sup>2</sup> can exceed  $T_{j\_max}$  even when the average is fine. Thermal-aware floorplanning spreads hot blocks apart and places on-die temperature sensors at predicted hot spots.
- **Transient thermal** behaviour (seconds of time constant) is what dynamic thermal management (throttling, DVFS) exploits: turbo modes run above sustainable power for short bursts.

## 20.10 Package reliability and qualification

The package fails in different ways from the die. The dominant mechanisms are thermo-mechanical: every power cycle and ambient temperature swing strains the joints because materials expand by different amounts.

| Mechanism                          | Cause                                                                       | Test / mitigation                                                                                                   |
|------------------------------------|-----------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| <b>Solder joint fatigue</b>        | CTE mismatch between package and board under temperature cycling            | Temperature cycling (e.g. JESD22-A104, -55/+125 deg C), board-level reliability tests; underfill, ball alloy choice |
| <b>Bump cracking / white bumps</b> | Die-substrate CTE mismatch; stress on fragile low-k dielectrics under bumps | Underfill, chip-package interaction (CPI) rules, bump keep-outs                                                     |
| <b>Warpage</b>                     | Asymmetric layer stack and CTE; large bodies bow during reflow              | Balanced stack-up, stiffener rings, warpage measured versus temperature (shadow moire)                              |
| <b>Delamination / popcorn</b>      | Absorbed moisture turns to steam in reflow                                  | MSL rating (J-STD-020) and dry-pack; bake before reflow                                                             |
| <b>Wire bond failures</b>          | Intermetallic growth, corrosion (Cu wire), wire sweep                       | High-temperature storage, HAST, bond process control                                                                |
| <b>Electromigration in bumps</b>   | High current density in bumps and under-bump metallisation                  | Current-per-bump limits in the bump plan (see above)                                                                |

Table 20.5 - Package failure mechanisms.

### INTERVIEW INSIGHT: WHY NOT ALWAYS USE FLIP CHIP?

A good answer weighs cost against need: flip chip requires wafer bumping, a more expensive build-up substrate and underfill, and its benefits (area I/O, low inductance, backside heat path) only pay off above roughly a few hundred I/Os, a few watts, or with high-speed interfaces. A 50-pin sensor hub belongs in a QFN or WLCSP; a 300 W accelerator has no choice. Mention that the decision constrains the floorplan (pad ring versus bump array) and must be made at architecture time.

### PITFALL: FREEZING THE DIE BEFORE THE PACKAGE

Teams that finish the floorplan first and 'throw it over the wall' to packaging often discover that the ball map cannot escape on the customer's board in the allowed layer count, that differential pairs cross, or that one supply has too few balls. The fix then needs a die re-floorplan late in the schedule. Hold a joint die/package/board review before floorplan sign-off.

## 20.11 Summary

- The package provides electrical, thermal, mechanical and environmental functions; it is designed concurrently with the floorplan, not after tapeout.
- Families range from lead-frame QFN/QFP through wire-bond and flip-chip BGA to WLCSP and fan-out; flip chip gives area I/O, low inductance and a backside heat path at higher cost.
- 2.5D uses silicon interposers, embedded bridges or RDL interposers for dense die-to-die wiring; 3D stacks dies with TSVs, micro-bumps or hybrid bonding.
- Chiplets trade D2D interface overhead and assembly complexity for yield, reticle-limit relief and mixed-node reuse; they require known-good-die testing and standards such as UCle.
- Bump planning, PDN impedance, SSN and package parasitics are co-designed with the chip;  $T_j = T_a + P \times \theta_{JA}$  sets the thermal solution.

- Package reliability is dominated by CTE-driven fatigue, warpage and moisture; it is qualified with temperature cycling, MSL and related JEDEC tests.

## 20.12 Exercises

---

- 1 A wire-bonded die needs 320 pads at 50  $\mu\text{m}$  pitch in a single row. What is the minimum square die size? If the logic needs only 4  $\text{mm}^2$ , estimate how much silicon area is wasted.
- 2 A 15 W chip in an FCBGA has  $\theta_{\text{JC}} = 0.2 \text{ C/W}$ ,  $\text{TIM2} = 0.15 \text{ C/W}$  and a heat sink of 3.0  $\text{C/W}$ . What is  $T_{\text{j}}$  at 50 deg C ambient? What sink resistance is needed to keep  $T_{\text{j}} \leq 95 \text{ deg C}$ ?
- 3 Modify bumps.py for a 1.0 V, 100 W die of 20 x 20 mm at 130  $\mu\text{m}$  bump pitch. What fraction of bump sites go to VDD/VSS?
- 4 Four chiplets are assembled in one package, and 1% of the dies that pass wafer sort are actually defective (test escapes). Estimate the package yield from escapes alone, then repeat for an escape rate of 0.1%. Why does the escape rate matter more than the raw wafer-sort yield for multi-die packages?
- 5 Explain, with a sketch, why an HBM stack must be on an interposer or bridge rather than an ordinary organic substrate.
- 6 List three pieces of information the die team must give the package team before a first bump map can be drawn.



## 21. Manufacturing Test, Yield and Reliability

Every die the foundry ships is a gamble: some fraction carry defects - a particle that shorted two wires, a void in a via, a transistor with a weak gate oxide. **Manufacturing test** separates good dies from bad ones using the DFT structures designed in Chapter 10. **Yield** is the fraction that pass, and it sets the cost per good die. **Reliability** is about the dies that pass today but might fail after months or years in the field. The three are tightly linked: test quality determines how many defective parts escape to customers (DPPM), and screens such as burn-in exist to catch parts that would fail early in life.

Ownership: the **product/test engineering** team writes and maintains test programs and runs production test; **yield engineering** (with the foundry) analyses failures and drives yield up; **reliability/quality engineering** plans qualification and owns field-return analysis. The design team delivers the DFT patterns, test specification and failure diagnosis support.

### 21.1 The test flow: from wafer to shipped part

| FAB                                                                             | WAFER SORT (probe)                                                                                                                         | ASSEMBLY                                 | FINAL TEST (FT)                                                                                          | SHIP           |
|---------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------|----------------------------------------------------------------------------------------------------------|----------------|
| -----                                                                           | -----                                                                                                                                      | -----                                    | -----                                                                                                    | -----          |
| wafer out                                                                       | -> probe card touches pads or bumps of each die<br>- continuity, IDDQ<br>- scan/ATPG, MBIST<br>- analog trim, fuses<br>-> wafer map (bins) | -> saw, pick good dies, package (Ch. 20) | -> handler + socket<br>full test at hot/cold, speed bin, burn-in (if any), SLT (if any)<br>-> bin + mark | -> tape & reel |
| Parametric test (PCM/WAT) on scribe-line structures runs in the fab before sort |                                                                                                                                            |                                          |                                                                                                          |                |

Figure 21.1 - A typical production test flow.

#### Wafer sort

At **wafer sort** (also called wafer probe or CP, chip probe) a **probe card** with hundreds to tens of thousands of needles or MEMS probes contacts the pads or bumps of one or several dies at once. A **prober** steps the wafer under the card. The tester applies power, checks continuity and shorts, measures supply currents (including IDDQ where meaningful), runs scan/ATPG and memory BIST, and trims analog blocks, programming the results into eFuses or OTP. Each die gets a **bin** number recorded in a **wafer map**; bad dies are inked or, more commonly today, tracked electronically so that assembly picks only good dies. Sort cannot usually run everything at full speed or temperature (contact resistance, probe inductance), which is why a final test follows.

#### Final test

After packaging, a **handler** places each part in a **socket** on a **load board** attached to the tester. Final test repeats the structural tests (the package may have introduced defects), adds at-speed functional tests, I/O parametric tests (VOH/VOL, leakage, timing), SerDes loopback and eye tests, and runs at the specified temperature corners (e.g. hot at 85-125 deg C, sometimes cold). Parts are sorted into bins by result and speed, then marked and packed.

### 21.2 ATE architecture and the test program

**Automatic test equipment (ATE)** - families such as Advantest V93000/T2000 or Teradyne UltraFLEX/J750 - consists of a test head with many **pin electronics** channels (each able to drive and compare a digital waveform with programmable levels and timing), **device power supplies (DPS)** with current measurement, **parametric measurement units (PMU)** for force-voltage/measure-current tests, and optional analog, RF and high-speed instruments. A workstation runs the **test program**.



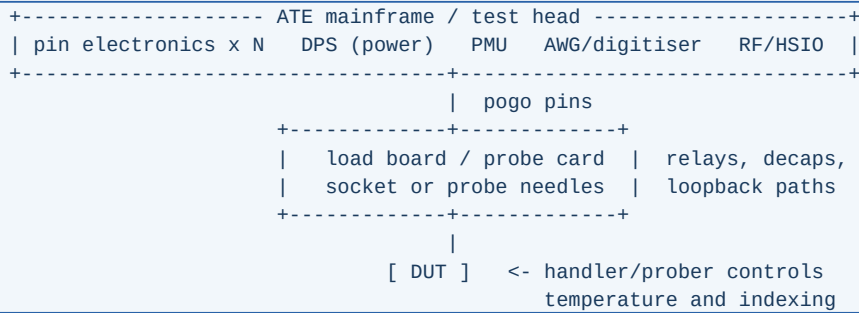


Figure 21.2 - ATE, interface hardware and the device under test.

A test program is an ordered **test flow** of test suites: continuity, leakage, power-up and IDD, scan chain integrity, stuck-at ATPG, transition (at-speed) ATPG, memory BIST, analog/PLL tests, functional vectors, I/O specs. Each test has limits and a failing **soft bin** that maps to a **hard bin** (the physical sort category). Patterns come from the DFT tools as STIL or WGL files (Chapter 10) and are converted to the tester's format. The program also logs data (datalogs, STDF format) for yield analysis.

#### THE TEST HAND-OFF PACKAGE

Design/DFT delivers to test engineering: ATPG patterns (STIL) with expected responses and fault coverage reports; MBIST controller operation and expected signatures; a pin list with levels and timing; power-up sequence; analog trim procedures; the JTAG/IJTAG access description (BSDI for boundary scan, ICL/PDL for IEEE 1687 networks); and diagnosis support (failing-pattern to fault mapping). Missing any of these costs weeks on the tester.

## 21.3 Test time and test cost

Test is expensive capital equipment amortised over seconds of tester time. The cost of test per good die is roughly the tester cost per second, times the test time, divided by the number of sites tested in parallel and by the yield. Scan test time is set by the number of patterns and the length of the longest scan chain - the motivation for scan compression (Chapter 10).

$$t_{\text{scan}} \sim N_{\text{patterns}} \times (L_{\text{chain}} + \text{capture cycles}) / f_{\text{shift}}$$

$$\text{Cost}_{\text{test/good die}} \sim (\text{rate } \$/\text{s}) \times t_{\text{test}} / (N_{\text{sites}} \times \text{site efficiency} \times \text{yield})$$

Test time and cost.

```

# Test cost per good die: ATE cost/second x test time / (sites x yield x efficiency)
ate_rate = 0.03          # $/s fully loaded tester + handler cost (assumption)
def cost_per_good(t_test, sites, yld, eff=0.9):
    # eff: multi-site efficiency (index time, retests, uneven site times)
    t_per_die = t_test / (sites * eff)
    return ate_rate * t_per_die / yld
print(" test_s  sites  yield  $/good die")
for t, s, y in [(2.0, 1, 0.90), (2.0, 4, 0.90), (2.0, 16, 0.90),
                (8.0, 4, 0.90), (8.0, 4, 0.60), (0.5, 32, 0.95)]:
    print(f"{t:8.1f} {s:6d} {y:6.2f} {cost_per_good(t, s, y):12.4f}")
# Scan test time: patterns x (chain length + capture) / shift frequency
patterns, chain_len, f_shift = 5000, 2000, 50e6
t_scan = patterns * (chain_len + 3) / f_shift
print(f"Scan time: {patterns} patterns x {chain_len} bits @ {f_shift/1e6:.0f} MHz = "
      f"{t_scan*1000:.1f} ms")

```

testcost.py - cost of test per good die (the \$/s rate is an assumption).

| test_s | sites | yield | \$/good die |
|--------|-------|-------|-------------|
| 2.0    | 1     | 0.90  | 0.0741      |
| 2.0    | 4     | 0.90  | 0.0185      |
| 2.0    | 16    | 0.90  | 0.0046      |
| 8.0    | 4     | 0.90  | 0.0741      |
| 8.0    | 4     | 0.60  | 0.1111      |
| 0.5    | 32    | 0.95  | 0.0005      |

Scan time: 5000 patterns x 2000 bits @ 50 MHz = 200.3 ms

Output of testcost.py (python3).

Multi-site testing is the biggest lever: 16 sites cut cost per die 16-fold if the tester has enough channels and the sites run concurrently. That is why test engineers push the design team to reduce the number of pins that must be contacted (reduced pin-count test through JTAG or a few scan pins), and why a 200 ms scan test matters: it is often the largest item in the budget.

## 21.4 Binning, speed binning and adaptive test

**Binning** sorts tested parts into categories: hard bin 1 = good; other bins record the failure class (opens/shorts, leakage, scan, MBIST, functional). Many products also **speed bin**: parts are tested at several frequencies (or at several voltages for a fixed frequency) and sold as different SKUs; a CPU line might sell the fastest parts at a premium. Related techniques include **fusing off** defective cores or cache ways to sell partially good dies (harvesting), and programming per-part voltage tables (**adaptive voltage scaling**, AVS) into fuses so that fast, leaky parts run at lower voltage and slow parts at higher voltage.

**Adaptive test** uses data to change the test flow on the fly: skipping tests that never fail for a stable lot (test time reduction), adding tests when a wafer looks unusual, or rejecting **outliers** - dies that pass all limits but whose measurements sit far from their neighbours (part average testing, PAT, and nearest-neighbour residual methods, mandatory practice for automotive per AEC-Q001). A die that passes but has an IDDQ three sigma above its neighbours is more likely to fail in the field.

## 21.5 Burn-in and system-level test

**Burn-in** operates parts at elevated voltage and temperature (e.g. 125 deg C and 1.1-1.4x nominal voltage) for hours to accelerate latent defects to failure before shipment - pushing the population past the infant mortality region of the bathtub curve (below). It is expensive (ovens, burn-in boards, hours of time, and some lifetime consumed), so many consumer products skip it or use short **wafer-level burn-in** or high-voltage stress tests instead, while automotive, server and aerospace products commonly use it.

**System-level test (SLT)** runs the part in an application-like board, booting firmware or an operating system and running real workloads. It catches defects that structural tests miss - marginal timing on paths not covered by ATPG, power-delivery interactions, analog/digital integration issues. SLT is now common for high-end SoCs despite its long test time (minutes), because structural coverage alone cannot reach single-digit DPPM for billion-transistor designs.

## 21.6 Yield: defect density and yield models

**Yield** at wafer sort is the fraction of dies that pass. It combines **systematic** losses (design-process interactions, lithography hotspots, parametric shifts - addressed by DFM and process tuning) and **random defect** losses (particles). Random defect yield is modelled from the die's **critical area** and the **defect density** D0 (defects per cm<sup>2</sup>). If defects landed independently (a Poisson process) the probability of a die with zero defects would be  $\exp(-A \times D0)$ . In reality defects cluster (some wafer regions are dirtier), which improves yield for large dies relative to Poisson. Several classic models capture this:

**Poisson:**  $Y = \exp(-A D0)$

**Murphy:**  $Y = ((1 - \exp(-A D0)) / (A D0))^2$

**Seeds:**  $Y = 1 / (1 + A D0)$

**Negative binomial:**  $Y = (1 + A D0 / \alpha)^{-\alpha}$       $\alpha = \text{clustering factor}$

$Y_{\text{total}} = Y_{\text{systematic}} \times Y_{\text{random}}$      ( $\alpha \rightarrow \text{infinity}$  gives Poisson)

Random-defect yield models (A = die area, D0 = defect density).

The negative binomial model is the industry workhorse: alpha (typically quoted around 1-5) tunes how clustered defects are; alpha -> infinity gives Poisson and alpha = 1 gives Seeds. The following script compares them against die area for a mature process and an early-ramp process:

```
import math
D0 = 0.1          # defects per cm^2 (mature-ish process, assumption)
alpha = 3.0       # negative-binomial clustering parameter
def poisson(A):   return math.exp(-A * D0)
def murphy(A):    x = A * D0; return ((1 - math.exp(-x)) / x) ** 2
def seeds(A):     return 1 / (1 + A * D0)
def negbin(A):    return (1 + A * D0 / alpha) ** (-alpha)
print(f"D0 = {D0} /cm^2, alpha = {alpha}")
print(" Area(mm2) Poisson Murphy Seeds NegBin(a=3)")
for a_mm2 in [10, 25, 50, 100, 200, 400, 600, 800]:
    A = a_mm2 / 100.0          # mm^2 -> cm^2
    print(f"{a_mm2:9d} {poisson(A):7.3f} {murphy(A):7.3f} {seeds(A):7.3f} {negbin(A):10.3f}")
```

yield\_models.py - four yield models versus die area.

```
D0 = 0.1 /cm^2, alpha = 3.0
Area(mm2) Poisson Murphy Seeds NegBin(a=3)
10 0.990 0.990 0.990 0.990
25 0.975 0.975 0.976 0.975
50 0.951 0.951 0.952 0.952
100 0.905 0.906 0.909 0.906
200 0.819 0.821 0.833 0.824
400 0.670 0.679 0.714 0.687
600 0.549 0.565 0.625 0.579
800 0.449 0.474 0.556 0.492
```

Output of yield\_models.py (python3).

```
import math
D0, alpha = 0.3, 3.0          # early-ramp defect density (assumption)
models = {"N": lambda x: (1 + x / alpha) ** (-alpha), # drawn first = wins ties
          "M": lambda x: ((1 - math.exp(-x)) / x) ** 2,
          "S": lambda x: 1 / (1 + x),
          "P": lambda x: math.exp(-x)}
rows, cols = 16, 80          # ASCII canvas: yield 0..1 vs area 0..800 mm^2
grid = [[" "] * cols for _ in range(rows)]
for c in range(cols):
    A_cm2 = (c + 1) / cols * 8.0      # 0..8 cm^2 = 0..800 mm^2
    for tag, f in models.items():
        y = f(A_cm2 * D0)
        r = rows - 1 - int(round(y * (rows - 1)))
        if grid[r][c] == " ":
            grid[r][c] = tag
for r in range(rows):
    label = f"{1 - r / (rows - 1):4.2f} |"
    print(label + "".join(grid[r]))
print("    +" + "-" * cols)
print("    0" + "".join(f"{a:>10d}" for a in range(100, 900, 100)) + " mm^2")
```

```
print(f"D0={D0}/cm^2 P=Poisson M=Murphy S=Seeds N=neg.binomial(alpha={alpha:.0f})")
```

yield\_plot.py - ASCII plot of the models for an early-ramp D0 of 0.3/cm<sup>2</sup>.

```

1.00 | N
0.93 | NN
0.87 | NNN
0.80 | NNNS
0.73 | PNNNSS
0.67 | PNNNNS
0.60 | MNNNNS
0.53 | PMNNNNS
0.47 | PMNNNNS SSSSSSS
0.40 | PPPMMNNN SSSSSSSSS
0.33 | PPPMMNNN SSSSSSSSSSS
0.27 | PPPPPMMNNN SSSSSSSSSSSSS
0.20 | PPPPPPMNNN SSSSSSSSSSSSSSS
0.13 | PPPPPPMNNN SSSSSSSSSSSSSSSSS
0.07 | PPPPPPMNNN SSSSSSSSSSSSSSSSSS
0.00 |
      +-----+
      0      100    200    300    400    500    600    700    800 mm^2
D0=0.3/cm^2 P=Poisson M=Murphy S=Seeds N=neg.binomial(alpha=3)

```

Output of yield\_plot.py (python3). Where curves coincide only one letter is drawn.

Small dies yield well under every model. Beyond a few hundred mm<sup>2</sup> the models diverge: Poisson predicts ~9% yield at 800 mm<sup>2</sup> while Seeds predicts ~29% - a threefold difference in cost per die. This is why yield models must be calibrated with the foundry's data for the specific process and die size, and it is the quantitative reason behind chiplets (Chapter 20).

#### CRITICAL AREA

Not every defect kills a die: a particle smaller than the space between two wires does not short them. The **critical area**  $A_c(r)$  is the area in which the centre of a defect of radius  $r$  causes a fault; integrating  $A_c(r)$  over the defect size distribution gives the effective area to use in the models. Dense layouts (minimum spacing everywhere) have larger critical area, which is what DFM wire spreading and via doubling (Chapter 17) attack.

### Parametric versus functional yield

**Functional (defect) yield** loss is caused by hard faults: the die simply does not work. **Parametric yield** loss comes from dies that work but miss a specification - too slow at Fmax, too leaky for the standby-power spec, an ADC with insufficient linearity - because of process variation. Parametric yield is controlled at design time by sign-off margins (Chapter 9 corners and OCV) and on silicon by AVS, trimming and binning. A design with no timing margin can have perfect defect yield and still lose 30% of dies to speed.

### Redundancy and repair

Large SRAMs dominate critical area in many SoCs, so they are built with **redundant rows and columns**. MBIST finds failing cells, a built-in redundancy analysis (BIRA) engine computes a repair solution, and the repair is programmed into eFuses (built-in self repair, BISR). Without repair a chip with 100 Mbit of SRAM would lose a large fraction of dies to single-bit failures; with repair most of those dies are saved. Designs also harvest defective cores or channels (see binning above).

## 21.7 Test quality: coverage and DPPM

Customers specify quality as **DPPM** (defective parts per million shipped). Consumer products may tolerate hundreds of DPPM; automotive customers demand single-digit DPPM or a zero-defect programme. The classic Williams-Brown relation links the defect level to yield  $Y$  and fault coverage  $T$ :

$$DL = 1 - Y^{(1 - T)}$$

(defect level: fraction of shipped parts that are bad)

Williams-Brown defect level model.

```
# Williams-Brown: defect level DL = 1 - Y^(1 - T) (Y = yield, T = fault coverage)
print(" yield coverage   DPPM")
for y in (0.90, 0.50):
    for t in (0.90, 0.95, 0.99, 0.995, 0.999):
        dl = 1 - y ** (1 - t)
        print(f"{y:6.2f} {t:9.3f} {dl*1e6:8.0f}")
```

dppm.py - defect level versus yield and fault coverage.

| yield | coverage | DPPM  |
|-------|----------|-------|
| 0.90  | 0.900    | 10481 |
| 0.90  | 0.950    | 5254  |
| 0.90  | 0.990    | 1053  |
| 0.90  | 0.995    | 527   |
| 0.90  | 0.999    | 105   |
| 0.50  | 0.900    | 66967 |
| 0.50  | 0.950    | 34064 |
| 0.50  | 0.990    | 6908  |
| 0.50  | 0.995    | 3460  |
| 0.50  | 0.999    | 693   |

Output of dppm.py (python3).

Even 99.9% stuck-at coverage leaves ~100 DPPM at 90% yield and ~700 DPPM at 50% yield under this model. Real escapes are dominated by defects that stuck-at faults do not model (resistive opens, bridges, small delay defects), which is why production test adds transition, path-delay, cell-aware and small-delay-defect ATPG, IDDQ, outlier screening and SLT.

## 21.8 Reliability physics

Reliability failures are wear-out or random failures that occur after the part has passed test. The main silicon mechanisms are:

| Mechanism                                         | Physics                                                                                                                                                  | Accelerated by                                      | Design mitigation                                                               |
|---------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|---------------------------------------------------------------------------------|
| <b>Electromigration (EM)</b>                      | Momentum transfer from electrons moves metal atoms; voids (opens) and hillocks (shorts). Black's equation: $MTTF \sim A J^n \exp(E_a/kT)$ , $n \sim 1-2$ | Current density, temperature                        | EM sign-off (Chapter 18): wider wires, more vias, current limits per layer      |
| <b>Time-dependent dielectric breakdown (TDDB)</b> | Traps accumulate in gate oxide (or inter-metal low-k) until a conductive path forms                                                                      | Electric field (voltage), temperature               | Voltage limits per oxide; overdrive rules; spacing for BEOL TDDB                |
| <b>Hot carrier injection (HCI)</b>                | High-energy carriers near the drain damage the oxide interface; $V_{th}$ shifts, drive falls                                                             | High $V_{ds}$ , fast edges, switching activity      | Limit overdrive, slew constraints, ageing-aware timing                          |
| <b>Bias temperature instability (NBTI/PBTI)</b>   | Interface traps under gate bias at temperature raise $ V_{th} $ (NBTI in PMOS, PBTI in NMOS with high-k gates); partially recovers                       | Gate bias, temperature, duty cycle                  | Ageing derates in STA (end-of-life libraries), guard-bands, AVS                 |
| <b>Soft errors (SER)</b>                          | Alpha particles (from package/solder impurities) and atmospheric neutrons deposit charge and flip stored bits; not permanent damage                      | Altitude, sensitive node charge, low supply voltage | ECC on memories, parity, hardened flip-flops, interleaving, low-alpha materials |

Table 21.1 - Silicon reliability mechanisms.

Soft error rates are expressed in **FIT** (failures in time: one failure per  $10^9$  device-hours). SRAM SER is often quoted per Mbit (typical orders of hundreds to a few thousand FIT/Mbit for unprotected SRAM at sea level, depending on node). A chip with 100 Mbit of unprotected SRAM therefore has a raw SER of tens of thousands

of FIT - one upset every few years per chip, but many per day across a data-centre fleet. SECCDED ECC reduces the visible rate by orders of magnitude, leaving multi-bit upsets, which bit interleaving addresses.

## 21.9 The bathtub curve, FIT and MTBF

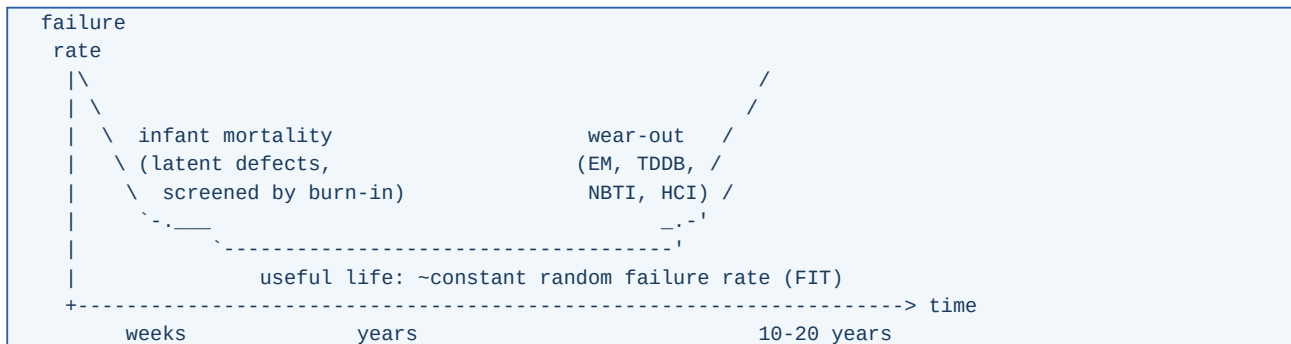


Figure 21.3 - The bathtub curve.

During useful life the failure rate  $\lambda$  is approximately constant and failures follow an exponential distribution, so rates add across components and  $MTBF = 1/\lambda$ . Wear-out must be pushed beyond the product lifetime (e.g. 10 years for consumer at a defined mission profile, 15 years or more for automotive) by the reliability sign-off rules.

**1 FIT = 1 failure /  $10^9$  device-hours**

**$\lambda_{\text{system}} = \text{sum of } \lambda_i$  (series system, constant rates)**

**$MTBF = 10^9 / FIT_{\text{total}}$  hours  $R(t) = \exp(-\lambda t)$**

FIT and MTBF arithmetic.

```
# FIT and MTBF arithmetic for a system built from several chips
parts = [("SoC (logic + SRAM)", 1, 25.0), ("DRAM", 4, 10.0),
         ("PMIC", 1, 5.0), ("Flash", 1, 8.0)] # name, count, FIT each (assumptions)
total = sum(n * f for _, n, f in parts)
mtbf_h = 1e9 / total
print(f"System FIT = {total:.0f} -> MTBF = {mtbf_h:.3g} h = {mtbf_h/8760:.0f} years")
fleet, years = 1_000_000, 1
fails = total * 1e-9 * fleet * 8760 * years
print(f"Expected failures in a fleet of {fleet:,} for {years} year: {fails:.0f}")
```

fit.py - system FIT, MTBF and fleet failures (FIT values are assumptions).

```
System FIT = 78 -> MTBF = 1.28e+07 h = 1464 years
Expected failures in a fleet of 1,000,000 for 1 year: 683
```

Output of fit.py (python3).

### PITFALL: MTBF IS NOT LIFETIME

An MTBF of 1464 years does not mean a unit lasts 1464 years. It means that during the useful-life region, a large population fails at a rate of one unit per 1464 unit-years - here about 683 failures a year in a fleet of a million. Wear-out (the right side of the bathtub) will still end every unit's life after its design lifetime. Quote FIT for components and state the mission profile (temperature, voltage, duty cycle) the number assumes.

## 21.10 Accelerated testing and the Arrhenius model

Nobody can wait ten years to see if a product survives ten years. Reliability tests therefore **accelerate** failure mechanisms with temperature, voltage, humidity or temperature swing, and use physical models to translate stress time into use time. For thermally activated mechanisms the **Arrhenius** acceleration factor is:

$AF = \exp[ (E_a / k) \times (1/T_{use} - 1/T_{stress}) ]$        $T$  in kelvin  
 $k = 8.617e-5 \text{ eV/K}$        $E_a$  = activation energy (mechanism-specific, ~0.3-1.0+ eV)

**Voltage acceleration (e.g. TDDb):**  $AF_V = \exp[ \gamma \times (V_{stress} - V_{use}) ]$

**Temperature cycling (Coffin-Manson):**  $AF = (dT_{stress} / dT_{use})^m$

Acceleration models.

JEDEC JESD47 uses a default  $E_a$  of 0.7 eV for HTOL when the actual mechanism is unknown. The script computes acceleration factors and sizes an HTOL test that demonstrates a FIT target with zero failures, using the chi-square bound for the failure rate:

```

import math
k = 8.617e-5 # Boltzmann constant, eV/K
def af_thermal(Ea, T_use_C, T_stress_C):
    Tu, Ts = T_use_C + 273.15, T_stress_C + 273.15
    return math.exp(Ea / k * (1 / Tu - 1 / Ts))
print("Ea(eV) Tuse Tstress AF")
for Ea in (0.5, 0.7, 1.0):
    print(f"{Ea:5.2f} {55:5d} {125:8d} {af_thermal(Ea, 55, 125):9.1f}")
# HTOL sizing: how many device-hours prove a FIT target (zero failures, 60% conf.)
Ea, AF = 0.7, af_thermal(0.7, 55, 125)
chi2_60_2dof = 1.833 # chi-square(0.60, 2*0+2)
fit_target = 10.0 # failures per 1e9 device-hours at use conditions
dev_hours_use = chi2_60_2dof / 2 / (fit_target * 1e-9)
dev_hours_stress = dev_hours_use / AF
print(f"AF(0.7 eV, 55->125C) = {AF:.1f}")
print(f"Zero-fail proof of {fit_target:.0f} FIT @60%: {dev_hours_use:.3g} use device-hours")
print(f" = {dev_hours_stress:.3g} device-hours at 125C, e.g. "
      f"{dev_hours_stress/1000:.0f} parts x 1000 h")

```

arrhenius.py - Arrhenius acceleration and HTOL sample-size arithmetic.

```

Ea(eV) Tuse Tstress AF
0.50 55 125 22.4
0.70 55 125 77.7
1.00 55 125 501.5
AF(0.7 eV, 55->125C) = 77.7
Zero-fail proof of 10 FIT @60%: 9.16e+07 use device-hours
= 1.18e+06 device-hours at 125C, e.g. 1180 parts x 1000 h

```

Output of arrhenius.py (python3).

At 0.7 eV one hour at 125 deg C is worth about 78 hours at 55 deg C. Demonstrating 10 FIT at 60% confidence with zero failures needs about 1.2 million stress device-hours - e.g. 1180 parts for 1000 hours. Standard HTOL (three lots of 77 parts for 1000 hours) therefore proves a much looser bound on its own; low-FIT claims rely on larger samples, longer tests, cumulative data across products and physics-of-failure models of each mechanism.

**PITFALL: OVER-ACCELERATING**

Acceleration is valid only while the stress triggers the same failure mechanism as use. Too high a temperature or voltage can activate new mechanisms (package materials degrade above their glass transition temperature, oxides break down by a different process), producing failures that would never happen in the field - or masking ones that would. Also, a single  $E_a$  cannot describe a product whose failures come from several mechanisms.



## 21.11 Qualification: JEDEC, ESD, latch-up and AEC-Q100

**Qualification** is the formal demonstration, on parts from multiple production-representative lots, that the product meets its reliability targets. For commercial and industrial ICs the reference is **JEDEC JESD47** (stress-test-driven qualification of integrated circuits), which calls out the individual test methods. **AEC-Q100** defines the stricter automotive qualification, with temperature grades from grade 0 (-40 to +150 deg C ambient) to grade 3 (-40 to +85 deg C), with grade 1 (-40 to +125 deg C) the most common.

| Test                                          | Standard (typical)                    | Purpose / typical conditions                                                                               |
|-----------------------------------------------|---------------------------------------|------------------------------------------------------------------------------------------------------------|
| <b>HTOL - high-temperature operating life</b> | JESD22-A108                           | Biased, exercised parts at ~125 deg C (Tj higher) for ~1000 h; 3 lots x 77 parts is the common sample plan |
| <b>ELFR - early life failure rate</b>         | JESD74 / AEC-Q100                     | Hundreds to thousands of parts for 48-168 h to measure infant mortality                                    |
| <b>HTSL - high-temperature storage</b>        | JESD22-A103                           | Unbiased bake, e.g. 150 deg C for 1000 h                                                                   |
| <b>Temperature cycling (TC)</b>               | JESD22-A104                           | e.g. -55/+125 deg C, hundreds to 1000+ cycles; package integrity                                           |
| <b>THB / HAST / uHAST</b>                     | JESD22-A101 / A110 / A118             | Humidity (85 deg C / 85% RH, or pressurised HAST at 110-130 deg C) with or without bias; corrosion         |
| <b>Preconditioning / MSL</b>                  | JESD22-A113, J-STD-020                | Moisture soak and reflow before the other stresses                                                         |
| <b>ESD - HBM</b>                              | ANSI/ESDA/JEDEC JS-001                | Human body model; commonly $\geq 1$ -2 kV target                                                           |
| <b>ESD - CDM</b>                              | ANSI/ESDA/JEDEC JS-002                | Charged device model; commonly 250-500 V target (AEC-Q100 asks 750 V on corner pins)                       |
| <b>Latch-up</b>                               | JESD78                                | Current injection (e.g. +/-100 mA) and supply overvoltage at max temperature; no latch-up allowed          |
| <b>EM, TDDb, HCI, NBTI</b>                    | JEP / foundry wafer-level reliability | Process qualification by the foundry; the product inherits it if design rules are met                      |

Table 21.2 - Common qualification stresses (check the current revision of each standard for exact conditions).

### INTERVIEW INSIGHT: YIELD VERSUS RELIABILITY VERSUS QUALITY

Interviewers like to probe whether you separate these. **Yield** is about the factory: the fraction of dies that pass test (drives cost). **Quality** is about the customer on day one: the fraction of shipped parts that are defective (DPPM, driven by test coverage). **Reliability** is about time: the failure rate in the field (FIT) and the lifetime before wear-out. A product can have poor yield but excellent quality (strict test throws many dies away) - and good quality but poor reliability (it passes test but its oxide wears out in two years).

## 21.12 Summary

- Production test runs at wafer sort (probe card) and final test (handler, socket); ATE provides pin electronics, power supplies and measurement units driven by a test program of binned test suites.
- Test cost ~ tester \$/s x time / (sites x yield); multi-site test, scan compression and reduced pin-count test are the main levers.
- Speed binning, harvesting, AVS fuses and adaptive/outlier test turn test data into revenue and quality.
- Random-defect yield falls with area x D0; Poisson, Murphy, Seeds and negative-binomial models diverge for large dies. Parametric yield is set by margins and variation; memory repair recovers defect yield.
- DPPM depends on yield and coverage ( $DL = 1 - Y^{(1-T)}$ ); low DPPM needs more than stuck-at coverage.
- EM, TDDb, HCI, BTI and soft errors are the main reliability mechanisms; FIT/MTBF describe the useful-life region of the bathtub curve; Arrhenius and related models convert stress time to use time.
- Qualification follows JESD47 (commercial/industrial) or AEC-Q100 (automotive): HTOL, ELFR, HTSL, TC, humidity, ESD (HBM/CDM) and latch-up (JESD78).



## 21.13 Exercises

- 1 A test takes 3.5 s on a \$0.04/s tester with 8 sites at 92% multi-site efficiency and 85% yield. Compute the test cost per good die. What test time would halve it?
- 2 Using the negative binomial model with  $D0 = 0.2/\text{cm}^2$  and  $\alpha = 2$ , compute yield for one  $600 \text{ mm}^2$  die and for four  $150 \text{ mm}^2$  chiplets (assume perfect KGD test and 99% assembly yield). Which costs less per good product if silicon cost is proportional to area?
- 3 Using Williams-Brown, what fault coverage is needed for 10 DPPM at 80% yield? Why might real DPPM still be higher?
- 4 Compute the Arrhenius acceleration factor from 150 deg C stress to 85 deg C use for  $E_a = 0.7 \text{ eV}$ . How many hours of HTOL at 150 deg C represent 10 years at 85 deg C?
- 5 A server has 32 DIMMs at 15 FIT each, 2 CPUs at 40 FIT each and other parts totalling 200 FIT. Compute its MTBF and expected failures per year in a fleet of 50,000 servers.
- 6 Explain why NBTI requires 'end-of-life' timing libraries, and which team owns the decision on the ageing guard-band.

## 22. Silicon Bring-up, Validation, Characterisation and Qualification

Eight to sixteen weeks after tapeout (longer for advanced nodes, shorter with hot-lot processing) the first wafers come out of the fab, a few are packaged on an expedited line, and the most intense phase of the project begins. **Bring-up** answers 'is it alive?', **post-silicon validation** answers 'does it do what the spec says, in every mode?', **characterisation** answers 'how much margin do we have across process, voltage and temperature?', and **qualification** answers 'will it keep working for its lifetime?'. Only when all four pass does the part move from engineering samples (ES) to production.

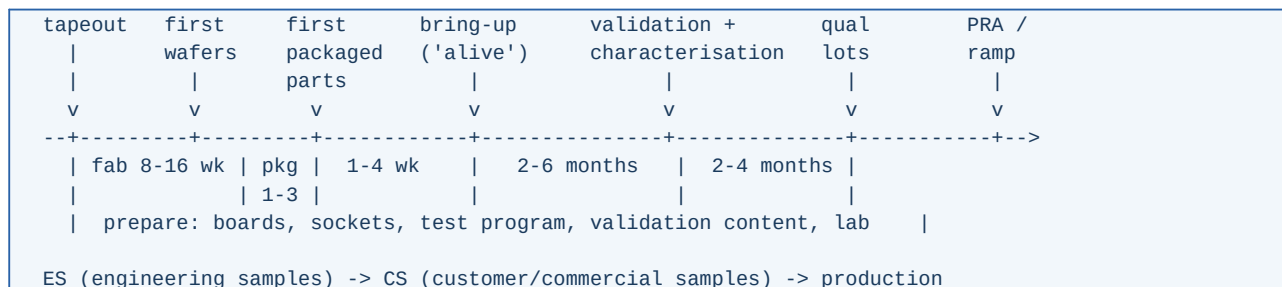


Figure 22.1 - Post-silicon timeline (durations are typical and vary widely).

### 22.1 Preparing for silicon: the bring-up plan

Bring-up is won or lost before tapeout. While the wafers are in the fab, the team prepares a written **bring-up plan** that lists, in order, every step from applying power to running the first application, the expected result of each step, who owns it, and what to do if it fails. The plan relies on design-for-debug features that had to be in the RTL months earlier.

#### Bring-up readiness checklist

- ☐ Bring-up (validation) boards designed, fabricated and assembled; at least one spare; power rails individually controllable and measurable (sense resistors, test points).
- ☐ Sockets for packaged parts (so parts can be swapped without rework); probe-able test points for clocks and resets.
- ☐ Bench equipment: programmable supplies with current limits, oscilloscope, logic analyser, spectrum analyser / BERT for SerDes, thermal forcing unit (thermostream) or chamber.
- ☐ JTAG debugger and scripts that can read the IDCODE and access internal registers without software running.
- ☐ Pre-silicon collateral ported to silicon: boot ROM image, first firmware, register access scripts, the same tests run in emulation/FPGA prototyping so that failures can be compared.
- ☐ A test program on ATE able to run scan and MBIST on the first parts, in parallel with the lab effort.
- ☐ Defined owners and a daily triage meeting; a bug tracker for silicon issues separate from pre-silicon bugs.

#### BRING-UP BOARD DESIGN

Design the first board for **observability and control**, not cost: separate regulators (or at least jumpers and sense resistors) on every supply rail so you can measure each current and power rails up in any order; an external clock input as an alternative to on-chip PLLs; switches for every boot strap; headers for JTAG, UART and trace; margining circuits that let software move each supply by +/-10%; and thermal sensors near the part. The cost-reduced customer reference board comes later.

## 22.2 First power-on and smoke tests

The first power-on is deliberately slow and paranoid. A typical sequence:

- 1 **Visual and continuity.** Check the board, check the part is seated correctly and orientation is right, measure rail-to-ground resistances with power off to catch shorts.
- 2 **Current-limited ramp.** Raise each supply in its specified order with a tight current limit, watching current. A short (hundreds of mA at a few hundred mV) means stop. Compare the static current with the predicted leakage for the corner and temperature.
- 3 **Clocks and reset.** Check the reference clock arrives, release reset, confirm the PLL locks (lock bit via JTAG or a debug pin), measure clock outputs if a debug clock-out mux exists.
- 4 **JTAG alive.** Read the IDCODE through the TAP (IEEE 1149.1). This is the first proof that digital logic works: the TAP controller, instruction register and IDCODE path are functioning.
- 5 **Register access.** Through the debug port (e.g. an Arm CoreSight DAP or RISC-V debug module) read and write on-chip registers and SRAM. Run MBIST through JTAG.
- 6 **Boot.** Release the CPU from reset, fetch from the boot ROM, load code from external flash or through JTAG into SRAM, print the first message on a UART. The 'hello world' on the console is the traditional bring-up milestone.
- 7 **Interfaces and power-up of domains.** Bring up DDR (training results), SerDes links, peripherals, then the power management modes (Chapter 11) one by one.

### PITFALL: NO ESCAPE PATH WHEN THE BOOT FAILS

If the boot ROM has a bug, or the external flash interface does not work, a chip with no alternative boot source is a brick. Always provide at least two independent paths to get code running: boot straps selecting UART/USB/JTAG download, JTAG access to SRAM and CPU halt/run, and ROM patch registers. The same applies to PLLs (an external clock bypass) and to any analog block the digital logic depends on (bypass modes, fuse overrides).

JTAG and debug architecture (IEEE 1149.1 boundary scan, IEEE 1500 core wrappers, IEEE 1687 IJTAG, CoreSight and RISC-V debug) is covered in Chapter 10 and in Chapter 23 of the companion Communication Protocols guide.

## 22.3 Post-silicon validation

Pre-silicon verification (Chapter 7) runs at simulation speed and sees only the modelled world. Silicon runs a billion times faster, at real voltages and temperatures, connected to real devices. **Post-silicon validation** exercises the part to find the bugs and margins that pre-silicon missed. It is organised by domain:

| Domain                   | What is checked                                                                                                | Typical methods                                                                                                    |
|--------------------------|----------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| <b>Functional</b>        | Every feature and mode against the spec; corner-case interactions; power-mode transitions                      | Directed tests, random instruction generators and self-checking tests running at speed, OS boot, compliance suites |
| <b>Electrical</b>        | I/O levels and timing, SerDes eye and jitter, DDR margins, PLL jitter, ADC/DAC performance, supply sensitivity | Scope/BERT measurements, eye scans with on-die margining, supply and temperature sweeps                            |
| <b>Performance</b>       | Throughput, latency, bandwidth versus the architecture model                                                   | Benchmarks, performance counters, comparison with the pre-silicon performance model                                |
| <b>Power and thermal</b> | Dynamic and leakage power per mode, wake-up latency, thermal behaviour                                         | Per-rail current measurement, power virus tests, thermal camera/sensors                                            |
| <b>Compliance</b>        | Standard interfaces meet their specs (USB, PCIe, Ethernet)                                                     | Compliance test equipment, plug-fests                                                                              |
| <b>Software/system</b>   | Drivers, firmware, OS and customer workloads                                                                   | Reference board with the software stack                                                                            |

Table 22.1 - Post-silicon validation domains.

**WHY BUGS SURVIVE TO SILICON**

Silicon runs as many cycles in one second as a simulator runs in weeks. Rare coincidences - a cache eviction racing with a power-state change during a specific interrupt - become near-certainties. Analog/digital interfaces are often only modelled behaviourally. And real software does things no test writer imagined. A clean pre-silicon sign-off lowers the number of silicon bugs; it never makes it zero.

## 22.4 Debug infrastructure: seeing inside the silicon

On silicon you cannot add a `$display`. Observability must be designed in. The main mechanisms, from cheapest to richest:

- **Debug registers and status bits** readable over JTAG/APB: state machine states, error flags, sticky exception bits, performance counters.
- **Scan dump.** Stop the clocks at a trigger point, switch the flops into scan mode and shift out the entire state of the chip through the scan chains. With the scan chain map from DFT insertion, a script turns the bit stream back into register names. It is destructive (the run ends) but gives total visibility at one instant - often combined with clock-stop on a trigger condition.
- **Trace.** Processor trace (Arm ETM/PTM, RISC-V E-Trace/N-Trace) compresses the program flow into a trace stream stored in an on-chip buffer or exported through a trace port; bus trace records transactions on selected interconnect links.
- **On-chip logic analysers (embedded trace/signal-capture buffers).** A trigger unit and SRAM buffer capture selected internal signals around an event, like an FPGA logic analyser; muxes select which signal groups are observed. Area is spent on silicon only where the designers predicted bugs might be.
- **Debug muxes to pins.** A few spare GPIOs that can be switched to observe internal clocks or state bits on a scope.
- **Focused ion beam (FIB) edits and physical probing.** Failure analysis labs can cut and reconnect metal lines on a die to test a fix, or probe internal nodes (laser voltage probing, emission microscopy) to localise a fault.

**INTERVIEW INSIGHT: DEBUGGING A SILICON HANG**

A classic question: 'the chip hangs after a few hours of a stress test - how do you debug it?' A strong answer: reproduce and reduce (find the minimal test, check sensitivity to voltage, frequency and temperature - a sensitivity to V/F suggests a timing path, insensitivity suggests a logic bug); read status registers via JTAG while hung; use trace to find the last instructions and bus transactions; set up a trigger and scan dump at the hang; map the state back to RTL; reproduce the scenario in simulation or emulation using the dumped state; then decide on a software workaround, a metal fix or both.

## 22.5 Characterisation: shmoo plots, Vmin and Fmax

**Characterisation** measures how the silicon's behaviour varies with voltage, frequency, temperature and process, on enough parts to be statistically meaningful. The basic tool is the **shmoo plot**: a pass/fail grid obtained by running a test at every combination of two parameters, usually supply voltage (x) and clock frequency or period (y). The shape reveals the physics: a diagonal boundary is the normal speed-voltage relationship; a vertical wall at low voltage is a Vmin limit that does not depend on frequency (typically SRAM bit-cell stability or a retention problem); holes or 'floating' fails inside the passing region point to noise, resonances or a logic bug triggered at specific settings.

The script below builds a simple model of a chip: an alpha-power law path delay (Chapter 2) whose threshold voltage and mobility depend on the process corner and on temperature, plus an SRAM that fails below a fixed Vmin. It then prints an ASCII shmoo, exactly as a characterisation engineer would print one from ATE data.

```
# Toy silicon model: alpha-power-law path delay with corner and temperature effects
ALPHA, K = 1.3, 0.38          # toy constants: TT part ~1.16 GHz at 0.80 V, 25 C
CORNER = {"SS": (+0.04, 0.90), "TT": (0.0, 1.0), "FF": (-0.04, 1.10)} # dvth, mobility
```

```

SRAM_VMIN = 0.62          # below this the (toy) SRAM bit-cells fail at any speed
def fmax_mhz(v, corner="TT", t_c=25.0):
    dvth, mob = CORNER[corner]
    vth = 0.35 + dvth - 0.0015 * (t_c - 25)          # Vth falls as T rises
    mob *= ((t_c + 273.15) / 298.15) ** -1.5         # mobility falls as T rises
    if v <= vth:
        return 0.0
    return 1000.0 * mob * (v - vth) ** ALPHA / (K * v)
def passes(v, f, corner="TT", t_c=25.0):
    return v >= SRAM_VMIN and f <= fmax_mhz(v, corner, t_c)
def shmoo(corner, t_c):
    volts = [0.60 + 0.025 * i for i in range(17)]      # 0.600 .. 1.000 V
    print(f"Shmoo: corner={corner} T={t_c:.0f}C   '*' = pass  '.' = fail")
    for f in range(1400, 399, -100):
        row = "".join("* " if passes(v, f, corner, t_c) else ". " for v in volts)
        print(f"{f:5d} MHz | {row}")
    print("      +" + "---" * len(volts))
    print("VDD(V)  " + "".join(f"{v:<6.2f}" for v in volts[:2]))

```

shmoo.py - a toy silicon model (constants are illustrative, not from any process).

```

from shmoo import *
shmoo("TT", 25)

```

run\_shmoo.py

```

Shmoo: corner=TT T=25C   '*' = pass  '.' = fail
1400 MHz | . . . . . . . . . . . . . . . * * *
1300 MHz | . . . . . . . . . . . . . . * * * *
1200 MHz | . . . . . . . . . . . . . * * * * *
1100 MHz | . . . . . . . . . . . . * * * * * *
1000 MHz | . . . . . . . . . . . * * * * * * *
 900 MHz | . . . . * * * * * * * * * * * * * *
 800 MHz | . . * * * * * * * * * * * * * * * *
 700 MHz | . * * * * * * * * * * * * * * * * *
 600 MHz | . * * * * * * * * * * * * * * * * *
 500 MHz | . * * * * * * * * * * * * * * * * *
 400 MHz | . * * * * * * * * * * * * * * * * *
      +-----+
VDD(V)  0.60  0.65  0.70  0.75  0.80  0.85  0.90  0.95  1.00

```

Output of run\_shmoo.py (python3): shmoo of the toy model, typical corner at 25 deg C.

The diagonal boundary is the timing limit; the failing column at 0.60 V is the SRAM Vmin wall (0.62 V in the model) - no frequency, however low, passes there. On real silicon the plot would come from the test program looping over DPS voltage and pattern timing, and each cell might show a count of passing parts instead of a single character.

## Vmin and Fmax across corners and temperature

Production parts come from anywhere in the process distribution and operate from cold to hot. Characterisation therefore measures **Vmin** (the lowest voltage that passes at a target frequency) and **Fmax** (the highest frequency that passes at a given voltage) on parts from each **corner lot** and across temperature:

```

from shmoo import *
# Vmin at two target clocks, and Fmax at a low and a high VDD, per corner and temperature
def vmin(f, corner, t_c):
    v = 0.50
    while v <= 1.20 and not passes(v, f, corner, t_c):
        v += 0.005
    return v
print("corner T(C) Vmin@500MHz Vmin@1GHz Fmax@0.65V Fmax@0.90V")
for corner in ("SS", "TT", "FF"):
    for t_c in (-40, 25, 125):
        print(f"{corner:>6} {t_c:5d} {vmin(500, corner, t_c):11.3f} "
              f"{vmin(1000, corner, t_c):10.3f} {fmax_mhz(0.65, corner, t_c):11.0f} "
              f"{fmax_mhz(0.90, corner, t_c):11.0f}")

```

run\_vmin.py - Vmin and Fmax of the toy model across corners and temperature.

| corner | T(C) | Vmin@500MHz | Vmin@1GHz | Fmax@0.65V | Fmax@0.90V |
|--------|------|-------------|-----------|------------|------------|
| SS     | -40  | 0.655       | 0.825     | 496        | 1204       |
| SS     | 25   | 0.620       | 0.845     | 632        | 1097       |
| SS     | 125  | 0.620       | 0.910     | 741        | 994        |
| TT     | -40  | 0.620       | 0.730     | 734        | 1508       |
| TT     | 25   | 0.620       | 0.720     | 846        | 1344       |
| TT     | 125  | 0.620       | 0.710     | 929        | 1192       |
| FF     | -40  | 0.620       | 0.645     | 1021       | 1852       |
| FF     | 25   | 0.620       | 0.620     | 1096       | 1620       |
| FF     | 125  | 0.620       | 0.620     | 1142       | 1409       |

Output of run\_vmin.py (python3).

Three effects are visible, all of which occur on real silicon. First, the SS corner at hot needs far more voltage for 1 GHz than FF - the reason for AVS and speed binning. Second, at 500 MHz most corners are pinned at the 0.62 V SRAM limit: memory, not logic, sets Vmin. Third, the model shows **temperature inversion**: at 0.65 V parts are faster hot than cold (threshold voltage falls with temperature and dominates at low overdrive), while at 0.90 V they are faster cold (mobility loss dominates). In low-voltage designs the worst-case timing corner can therefore be cold, which is why sign-off includes both temperature extremes (Chapter 9).

## Process split lots and corner lots

To characterise across process without waiting for years of natural variation, the team orders **split lots** or **corner lots** from the foundry: wafers deliberately processed with key parameters (implant doses, gate length) shifted to the edges of the process window - e.g. SS, FF, SF, FS wafers alongside TT. Measuring Vmin/Fmax, leakage and analog performance on these wafers verifies that the design works across the whole window the foundry guarantees. Correlation between measured silicon and sign-off corners (silicon-to-model correlation) is checked with on-die **ring oscillators** and **process monitors**, which also provide per-die speed data for binning and AVS.

| Parameter                    | Typical sweep                                              | Deliverable                                     |
|------------------------------|------------------------------------------------------------|-------------------------------------------------|
| <b>Fmax vs VDD</b>           | Voltage in 10-25 mV steps x frequency, at cold/room/hot    | Shmoo plots, guard-band for datasheet           |
| <b>Vmin</b>                  | Per part, per corner, per temperature, per memory instance | Vmin distribution; AVS table; retention voltage |
| <b>Leakage / IDDQ</b>        | Temperature and voltage                                    | Standby power spec, leakage-based screens       |
| <b>I/O timing and levels</b> | Setup/hold, VIH/VIL, VOH/VOL, drive strength               | Datasheet AC/DC tables                          |
| <b>Analog</b>                | PLL jitter, ADC ENOB/INL/DNL, LDO regulation, SerDes eye   | Datasheet analog tables, trim targets           |
| <b>ESD / latch-up</b>        | Per qualification standard                                 | Qualification report (Chapter 21)               |

Table 22.2 - What is characterised.

## From characterisation to speed bins and AVS tables

Characterisation data on a few hundred parts is used to predict what millions of production parts will do. The script below adds random per-part variation (threshold-voltage shift and mobility) to the same toy model, 'tests' 10,000 parts at 0.80 V and 85 deg C with a 3% tester guard-band, sorts them into speed bins, and then compares a single fixed supply voltage with **adaptive voltage scaling**, where each part is fused with the lowest voltage that meets 1 GHz:

```
import random
from shmoo import ALPHA, K, SRAM_VMIN
random.seed(1)
# Monte Carlo "production": each part gets its own Vth shift and mobility factor
def fmax(v, dvth, mob, t_c=85.0):
    vth = 0.35 + dvth - 0.0015 * (t_c - 25)
    mob *= ((t_c + 273.15) / 298.15) ** -1.5
    return 1000.0 * mob * (v - vth) ** ALPHA / (K * v)
parts = [(random.gauss(0, 0.02), random.gauss(1.0, 0.04)) for _ in range(10000)]
# 1) Speed binning at a fixed 0.80 V, 85 C (with a 3% tester guard-band)
bins = {"1200 MHz": 0, "1050 MHz": 0, "900 MHz": 0, "reject": 0}
for dvth, mob in parts:
    f = fmax(0.80, dvth, mob) * 0.97
    key = ("1200 MHz" if f >= 1200 else "1050 MHz" if f >= 1050
          else "900 MHz" if f >= 900 else "reject")
    bins[key] += 1
for k, n in bins.items():
    print(f"bin {k:9s}: {n:5d} parts {n / len(parts) * 100:5.1f}% " + "#" * (n // 200))
# 2) AVS: per-part voltage for 1000 MHz at 85 C versus one fixed voltage for all parts
def v_needed(dvth, mob, f=1000 / 0.97):
    v = SRAM_VMIN
    # memories set the floor
    while fmax(v, dvth, mob) < f:
        v += 0.0025
    return v
vs = sorted(v_needed(d, m) for d, m in parts)
v_fixed = vs[int(0.999 * len(vs))]
# fixed VDD must cover the 99.9th percentile part
avg_p = sum((v / v_fixed) ** 2 for v in vs) / len(vs)
# dynamic power ~ V^2
print(f"Fixed VDD for 99.9% of parts : {v_fixed:.3f} V")
print(f"AVS voltages: min {vs[0]:.3f} median {vs[len(vs) // 2]:.3f} max {vs[-1]:.3f} V")
print(f"Average dynamic power with AVS: {avg_p * 100:.1f}% of the fixed-VDD power")
```

bins.py - Monte Carlo speed binning and AVS (toy model, seeded random variation).

```
bin 1200 MHz :   584 parts    5.8% ##
bin 1050 MHz :  6561 parts   65.6% #####
bin  900 MHz :  2828 parts   28.3% #####
bin reject   :    27 parts    0.3%
Fixed VDD for 99.9% of parts : 0.900 V
AVS voltages: min 0.620 median 0.735 max 0.945 V
Average dynamic power with AVS: 67.1% of the fixed-VDD power
```

Output of bins.py (python3).

Two business decisions fall straight out of such data. The bin split decides which SKUs marketing can promise in which quantities (here only ~6% of parts reach the top bin, so a 1.2 GHz premium SKU must be priced and forecast accordingly). And AVS lets the typical part run at ~0.74 V instead of the 0.90 V that the slowest 0.1% would need, cutting average dynamic power by about a third; the cost is per-part test time to find the voltage, fuses to store it, and a power-management interface (PMIC or on-die regulator) that honours it.

## 22.6 Managing validation: bug tracking and exit criteria

Post-silicon work is run like a project within the project. Every anomaly is logged as a **sighting**, reproduced, root-caused (design bug, board issue, test issue, software bug or specification ambiguity) and only then promoted to a silicon bug with a severity:

| Severity                      | Definition                                                                    | Typical action                                      |
|-------------------------------|-------------------------------------------------------------------------------|-----------------------------------------------------|
| <b>Critical (showstopper)</b> | No workaround; blocks a key feature or causes data corruption                 | Respin (metal or full); may delay customer sampling |
| <b>High</b>                   | Workaround exists but costs performance, power or significant software effort | Workaround in firmware/SDK; fix in next stepping    |
| <b>Medium</b>                 | Workaround is simple; limited impact                                          | Erratum; fix if a respin happens anyway             |
| <b>Low / documentation</b>    | Behaviour differs from the spec but is harmless, or the spec was wrong        | Update the datasheet or reference manual            |

Table 22.3 - Silicon bug severity classes (names vary by company).

Validation exit criteria mirror pre-silicon sign-off: the planned test content has run on the target number of parts across corners and temperatures; the rate of new sightings has flattened over several weeks; no critical bugs are open; every high bug has a documented workaround; compliance tests have passed; and power and performance meet the datasheet targets. Each silicon bug also feeds back to the pre-silicon team as an **escape analysis**: why did verification miss it, and what test, assertion or coverage point would have caught it? That analysis is what improves the next project's methodology (Chapter 23).

## 22.7 Errata, workarounds and respins

Every complex chip has bugs in silicon. Each confirmed bug is root-caused and then classified: can it be worked around in software, firmware, board design or by avoiding a mode? Workable bugs are documented as **errata** (published to customers with workarounds) and fixed in the next revision if cheap. Bugs with no acceptable workaround force a **respin**.

| Respin type                       | What changes                                                                                                | Cost and time (typical order)                                                                                                            | When possible                                                                    |
|-----------------------------------|-------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| <b>Metal-only (ECO) respin</b>    | Only some metal and via masks; fixes use spare cells or gate-array ECO cells placed at tapeout (Chapter 19) | A fraction of a full mask set; weeks saved because base-layer wafers held at the fab ('wafer bank') can be finished with new metal masks | Logic fixes that fit in spare resources; small analog tweaks                     |
| <b>Full (all-layer) respin</b>    | All masks                                                                                                   | Full mask cost and full fab cycle time (months)                                                                                          | Transistor-level changes, large logic changes, new IP                            |
| <b>Software / fuse workaround</b> | Nothing on silicon                                                                                          | Engineering time only; possible performance cost                                                                                         | Bug has a trigger that firmware can avoid, or a chicken bit disables the feature |

Table 22.4 - Ways to fix a silicon bug.

### DESIGN FOR FIXABILITY

Experienced teams tape out with **spare cells** sprinkled throughout the logic, **chicken bits** (register bits that disable new or risky features or select conservative behaviour), ROM **patch** mechanisms, programmable delay taps on critical interfaces, and a policy of **holding wafers** before metal layers so that a metal fix can be finished quickly. Revision naming conventions track this: A0 is first silicon; A1 a metal respin; B0 a full respin.



## 22.8 From samples to production: ramp, qualification and datasheets

When validation and characterisation are complete and the silicon revision is frozen, the part goes through **product qualification** (the JEDEC or AEC-Q100 stresses of Chapter 21 on three lots of the final revision), and the organisation holds a **production release** (PRA) review. The **ramp to production** then scales volume: the test program is optimised for test time, yield learning continues with the foundry, test limits are tightened from characterisation data, and statistical process control monitors every lot.

The **datasheet** is generated from characterisation: each guaranteed specification is set from the measured distribution with a guard-band (for example, the measured mean minus several sigma across corners and temperature, further tightened for tester accuracy), and each value is marked as either tested in production or guaranteed by design/characterisation. Preliminary datasheets with target values are issued earlier and must be updated once silicon data exist.

### Lifecycle and product change notification

After release, a product is managed through its **lifecycle**: active, not recommended for new designs (NRND), last-time buy and end of life (EOL). Any change that could affect form, fit, function or reliability - a new fab or assembly site, a mask change, a new mould compound, a test flow change - requires a **product change notification (PCN)** to customers, often with requalification data (JEDEC J-STD-046 describes the process; automotive customers require it per AEC-Q100 change guidelines). Automotive and industrial customers often require 10-15 years of supply, which shapes the choice of process node and IP from the start.

#### PITFALL: CHARACTERISING TOO FEW PARTS

Datasheet limits derived from ten engineering samples from one lot will not hold across millions of production parts. Characterise hundreds of parts from multiple lots including corner lots, check distributions (not just pass/fail), and keep monitoring after release - the first production months often reveal a tail that samples missed.

## 22.9 Summary

- Bring-up is planned before tapeout: boards built for observability, debug access, bench equipment, ported pre-silicon tests and an ordered, owned plan.
- First power-on proceeds from current-limited rail ramps through clocks, JTAG IDCODE and register access to boot and interfaces; always design alternative boot and clock paths.
- Post-silicon validation covers functional, electrical, performance, power, compliance and software domains; debug infrastructure (scan dump, trace, on-chip logic analysers) must be designed in.
- Characterisation uses shmoo plots, Vmin/Fmax distributions, corner lots and temperature sweeps; watch for Vmin walls set by memory and for temperature inversion.
- Bugs are handled as errata with workarounds, metal-only respins (spare cells, held wafers) or full respins.
- Production release requires qualification, a datasheet derived from characterisation data with guard-bands, and lifecycle management with PCNs.

## 22.10 Exercises

- 1 Write a 12-step bring-up plan for a small microcontroller with one PLL, a boot ROM, SPI flash and a UART. For each step state the pass criterion and the fallback if it fails.
- 2 Modify shmoo.py so that a resonance causes failures only between 900 and 1000 MHz at voltages below 0.75 V. Print the shmoo and describe how a validation engineer would recognise this pattern.
- 3 Using run\_vmin.py, determine the Vmin at 800 MHz for all nine corner/temperature combinations. Which combination should AVS fuse tables be built around, and why?
- 4 A silicon bug corrupts data when two masters write the same cache line within 3 cycles. List three possible workarounds and discuss whether a metal-only fix is plausible.
- 5 Explain the difference between a value 'tested in production' and 'guaranteed by characterisation' in a datasheet. Which would you choose for leakage current, and why?

## 23. Methodology, Project Management and the Economics of an ASIC

The previous 22 chapters described what must be done to turn a specification into a shipping chip. This chapter is about how an organisation does it on time and on budget: the phases and milestones that structure a project, the reviews that gate each transition, the people and tools involved, the methodology that keeps hundreds of runs reproducible, and the cost model that decides whether the ASIC should be built at all. Technical leads who understand these topics make better engineering decisions, because almost every technical trade-off is ultimately a schedule, cost or risk trade-off.

### 23.1 Project phases and milestones

Company vocabularies differ, but every ASIC project passes through the same sequence of **freezes** - points after which a class of change becomes expensive and requires formal approval.

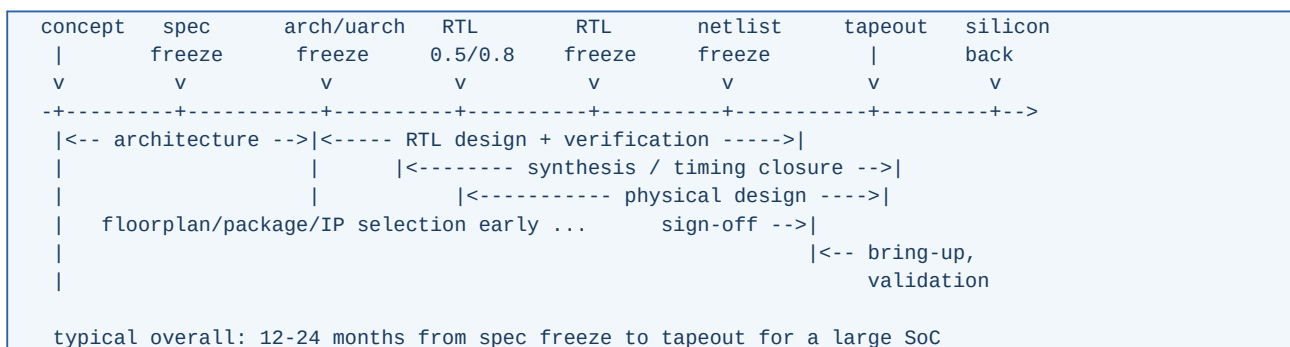


Figure 23.1 - Phases overlap; freezes mark where change control tightens.

| Milestone                                       | Meaning                                                                      | Exit criteria (examples)                                                                        |
|-------------------------------------------------|------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| <b>Spec freeze</b>                              | Features, interfaces, PPA targets and package agreed with customer/marketing | Signed product requirements; architecture spec reviewed; IP list chosen                         |
| <b>Architecture / micro-architecture freeze</b> | Block partitioning, clocks, power domains, memory map fixed                  | Block specs, performance model results, area/power estimates, floorplan sketch, package concept |
| <b>RTL 0.5 / 0.8 (feature complete)</b>         | All blocks coded; integration running                                        | Lint/CDC clean trend; basic tests pass; first synthesis and trial floorplan                     |
| <b>RTL freeze (1.0)</b>                         | No functional changes without an ECO process                                 | Coverage targets met; bug rate down; no open critical bugs; DFT inserted; UPF final             |
| <b>Netlist freeze</b>                           | Synthesised, DFT-inserted netlist handed to physical design (or final)       | Equivalence checks pass; timing met at block level; ATPG coverage met                           |
| <b>Tapeout</b>                                  | GDSII (OASIS) delivered to the foundry                                       | All sign-off checklists (Chapter 18-19) closed; waivers approved; tapeout review signed         |
| <b>Silicon back / production release</b>        | First parts in the lab; later, release to volume                             | Bring-up plan executed; qualification and characterisation complete (Chapters 21-22)            |

Table 23.1 - Standard milestones.

### 23.2 Gating reviews

Each milestone is gated by a **review** with a checklist and named sign-off owners. Good reviews are data-driven: they look at coverage reports, lint and CDC waiver lists, timing summaries, power estimates, DRC/LVS results and the open-bug list, not at slide decks full of green traffic lights. Typical reviews are the architecture review, RTL freeze review, DFT review, floorplan and package review, pre-tapeout (sign-off)

review and production release review. The output is a go/no-go decision plus an action list, and any **waiver** (a known violation accepted with justification) is recorded with an owner.

#### PITFALL: MOVING A FREEZE WITHOUT MOVING THE PLAN

The most common schedule failure is not a slow team but a late change that is accepted without replanning: 'just one more feature' after RTL freeze reopens verification, synthesis, timing closure and DFT, and the tapeout date silently slips by months. Every post-freeze change needs an impact assessment from verification, implementation and DFT before it is approved.

## 23.3 Team structure and headcount

A large SoC project involves dozens to hundreds of engineers across many disciplines, and the mix changes over time: architecture peaks early, verification in the middle, physical design towards tapeout, and validation after silicon returns.

| Role                                             | Main responsibility                                                              | Peak phase                                     |
|--------------------------------------------------|----------------------------------------------------------------------------------|------------------------------------------------|
| <b>Architects</b>                                | Spec, performance/power models, trade-offs                                       | Concept to arch freeze                         |
| <b>RTL designers</b>                             | Micro-architecture and RTL for blocks; integration                               | Arch freeze to RTL freeze                      |
| <b>Verification engineers</b>                    | Testbenches, coverage, formal, emulation; usually 1.5-3x the number of designers | RTL 0.5 to RTL freeze                          |
| <b>DFT engineers</b>                             | Scan, compression, MBIST, JTAG, ATPG                                             | RTL freeze to tapeout                          |
| <b>Physical design (PD)</b>                      | Synthesis, floorplan, P&R, timing closure                                        | Netlist to tapeout                             |
| <b>Sign-off / STA / PI</b>                       | Timing, IR/EM, physical verification                                             | Last months before tapeout                     |
| <b>Analog / mixed-signal</b>                     | PLLs, ADCs, I/O, custom blocks                                                   | Throughout; tapes out early test chips         |
| <b>CAD / methodology</b>                         | Flows, scripts, licences, compute                                                | Throughout                                     |
| <b>Package, SI/PI, test, product engineering</b> | Package design, test program, bring-up, characterisation                         | Tapeout to production                          |
| <b>Software / firmware</b>                       | Boot, drivers, SDK; pre-silicon on emulation/FPGA                                | Grows through the project; peaks after silicon |
| <b>Programme/project management</b>              | Schedule, dependencies, risk, reporting                                          | Throughout                                     |

Table 23.2 - ASIC team roles.

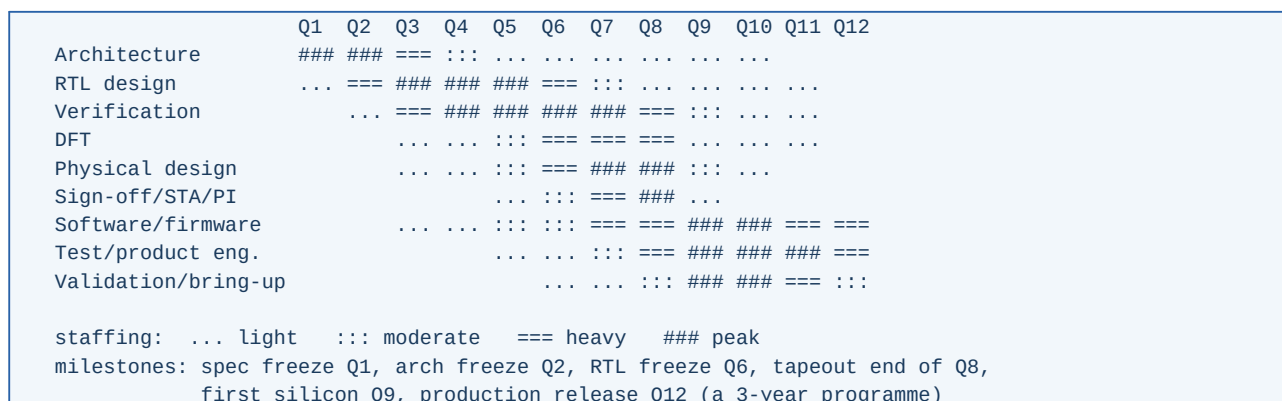


Figure 23.2 - Qualitative staffing by discipline over a three-year programme (illustrative).

## 23.4 Schedule risks

| Risk                         | Symptom                                                                                               | Mitigation                                                                                     |
|------------------------------|-------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| <b>Late or unstable spec</b> | Features added after RTL freeze                                                                       | Spec freeze discipline, change control board, contract terms                                   |
| <b>Third-party IP</b>        | IP late, buggy or missing views (LEF, Liberty, models); hardened IP not qualified on your metal stack | Qualify IP early; demand full deliverable lists; test chips; back-up vendors                   |
| <b>Verification closure</b>  | Bug rate not converging; coverage stuck                                                               | Track bug curves; emulation; formal on control logic; early integration                        |
| <b>Timing closure</b>        | Many iterations between synthesis and P&R                                                             | Early trial floorplans, physically aware synthesis, architecture changes rather than heroic PD |
| <b>Congestion / area</b>     | Die size grows after floorplan                                                                        | Early placement trials; utilisation targets; memory floorplanning                              |
| <b>Power / IR</b>            | Late IR-drop violations, thermal limits                                                               | Early power estimation, power grid sign-off per iteration                                      |
| <b>Foundry / PDK changes</b> | New PDK versions late in the project                                                                  | Lock PDK version; assess each update                                                           |
| <b>Tools and compute</b>     | Licence shortages near tapeout, long runtimes                                                         | Licence planning; cloud burst; hierarchical flows                                              |
| <b>People</b>                | Key-person dependencies, attrition                                                                    | Documentation, pairing, reviews                                                                |

Table 23.3 - Common schedule risks.

## 23.5 The EDA tool landscape and licensing

Three large vendors supply most commercial EDA, each with a complete digital flow, plus specialists. Product names and ownership change frequently through acquisitions, so treat the table as orientation and check current offerings.

| Flow step                      | Synopsys                            | Cadence            | Siemens EDA            | Open source            |
|--------------------------------|-------------------------------------|--------------------|------------------------|------------------------|
| <b>Simulation</b>              | VCS                                 | Xcelium            | Questa                 | Verilator, Icarus      |
| <b>Formal / equivalence</b>    | VC Formal, Formality                | Jasper, Conformal  | Questa Formal          | SymbiYosys, Yosys eqy  |
| <b>Emulation / prototyping</b> | ZeBu, HAPS                          | Palladium, Protium | Veloce                 | FPGA boards            |
| <b>Synthesis</b>               | Design Compiler, Fusion Compiler    | Genus              | (Oasys; HLS: Catapult) | Yosys + ABC            |
| <b>Place &amp; route</b>       | IC Compiler II, Fusion Compiler     | Innovus            | Aprisa                 | OpenROAD               |
| <b>STA</b>                     | PrimeTime                           | Tempus             | -                      | OpenSTA                |
| <b>DFT / ATPG</b>              | TestMAX                             | Modus              | Tessent                | (Fault, limited)       |
| <b>Physical verification</b>   | IC Validator                        | Pegasus            | Calibre                | Magic, KLayout, Netgen |
| <b>Power integrity</b>         | PrimePower, RedHawk-SC (from Ansys) | Voltus             | mPower                 | OpenROAD PDNSim        |
| <b>Custom / analog</b>         | Custom Compiler, PrimeSim           | Virtuoso, Spectre  | Solido, AFS            | Xschem, ngspice, Magic |

Table 23.4 - Representative tools by flow step (not exhaustive).

**OWNERSHIP CHANGES: THE ANSYS CAUTION**

Synopsys completed its acquisition of Ansys in 2025, so tools such as RedHawk-SC and Totem (power integrity) and the Ansys thermal and electromagnetic solvers now sit in the Synopsys portfolio; as a condition of approval some products were divested (for example Ansys PowerArtist went to Keysight). Many foundry sign-off certifications, older documents and job adverts still use the Ansys names. Always check current product names, ownership and foundry certification status rather than relying on this or any book.

**Licensing** is a major cost and a scheduling constraint. Licences are typically **time-based** (term licences of 1-3 years) and counted as simultaneous users per feature, managed by a licence server (FlexNet/FlexLM). Large companies negotiate **token** or 'all-you-can-eat' pools; start-ups buy bundles at a discount for their first tapeout. Place-and-route and sign-off licences are the most expensive, and demand peaks in the months before tapeout, when many parallel runs compete. Foundry **sign-off certification** matters: DRC/LVS decks, extraction rule files and EM rules are released for specific tools, which can lock the flow.

**Open-source tools** (Chapter 24) - Yosys, OpenROAD/OpenLane, Magic, KLayout, Netgen, Verilator - are production-capable for open PDKs (SkyWater 130, GF180, IHP SG13G2) and are widely used for education, research and small commercial designs. They are not sign-off certified for leading-edge commercial nodes, whose PDKs are under NDA.

## 23.6 Compute and infrastructure

An ASIC project is also a data-centre project. Full-chip P&R and sign-off jobs need hundreds of GB to multiple TB of RAM and run for hours to days; regressions run tens of thousands of simulations per night; a tapeout database with all its derived data runs to many terabytes. Typical infrastructure includes a compute farm with a job scheduler (LSF, Slurm, Grid Engine), high-performance NFS storage with snapshots, licence servers, emulators, and increasingly **cloud** capacity for peaks, which requires foundry approval for PDK data to leave the premises (most major foundries now have cloud-ready programmes).

## 23.7 Methodology: flows, scripts, CI and version control

**Methodology** is the set of flows, scripts, conventions and checks that make results reproducible. A mature team can re-run any block from RTL to GDSII with one command and get the same result; an immature one relies on a single engineer's shell history. Key elements:

- **Reference flows.** Scripted, parameterised Tcl/Python flows for each step (lint, CDC, synthesis, DFT, P&R, STA, PV), usually starting from vendor reference flows and adapted by the CAD team. Block owners change configuration, not the flow code.
- **Version control.** RTL, testbenches, constraints (SDC, UPF), scripts and flow configuration live in git (or Perforce). Large binary design data (libraries, layouts, databases) use design data management systems (e.g. ClioSoft SOS, IC Manage, Perforce/Helix, git-LFS) with check-in/check-out and tagging; every milestone and tapeout is a **tagged, reproducible release** including tool versions and PDK version.
- **Continuous integration.** Every commit triggers lint, a smoke simulation and possibly a quick synthesis; nightly regressions run full test suites, formal proofs, synthesis and trial P&R, tracking coverage, QoR (area, timing, power) and runtimes over time. Dashboards expose trends so that a timing or area regression is caught the day it is introduced.
- **Checklists and sign-off databases.** Each gating review pulls results from the regression database rather than hand-edited slides.
- **Consistency checks.** The same SDC and UPF used in synthesis, P&R and sign-off; automated checks that every block uses the approved library and PDK versions.

```
# nightly.yml - sketch of a nightly design regression (CI job definitions)
stages: [lint, sim, formal, synth, pnr_trial, report]
lint:    run: make lint          # Verilator --lint-only / commercial lint
sim:     run: make regress SEEDS=500 COV=1
formal:  run: make formal        # property proofs, CDC, connectivity
synth:   run: make synth CORNER=ss_0p72v_125c
pnr_trial: run: make pnr STAGE=place_opt # congestion and timing trend
report:  run: python qor_dashboard.py --compare last_good
```

An illustrative CI pipeline for a design repository (tool invocations abstracted behind make targets).

## 23.8 The cost model: NRE

ASIC economics has two parts: **non-recurring engineering (NRE)** - the one-time cost to design and bring the chip to production - and **unit cost** - the cost of each part shipped. NRE is spread over the lifetime volume, which is why volume decides whether an ASIC beats an FPGA or an off-the-shelf part. The script below builds up an NRE estimate for a mid-size 16 nm-class SoC. Every number is an assumption chosen to be plausible, not data from a real project or foundry.

```
# ILLUSTRATIVE NRE build-up for a mid-size 16nm-class SoC (all numbers are assumptions)
eng = [ # phase, months, average engineers
    ("Spec + architecture", 4, 10), ("RTL + verification", 12, 45),
    ("Physical design", 8, 25), ("Bring-up + qual", 6, 20)]
LOADED = 250e3 # $ per engineer-year incl. overhead
people = sum(m / 12 * n * LOADED for _, m, n in eng)
items = {
    "Engineering (people)": people,
    "EDA licences": 9e6,
    "Third-party IP": 12e6, # SerDes/DDR PHY, CPU, PLLs, memory compilers
    "Mask set (1 full + 1 metal respin)": 5e6 + 1.5e6,
    "Test chip / MPW": 1e6,
    "Package + test dev": 2.5e6, # substrate design, probe card, load boards
    "Compute + infra": 3e6,
}
total = sum(items.values())
for k, v in items.items():
    print(f"{k:36s} ${v/1e6:6.1f}M {v/total*100:5.1f}%")
print(f"'TOTAL NRE':36s ${total/1e6:6.1f}M")
print(f"Engineer-years: {sum(m / 12 * n for _, m, n in eng):.0f}")
```

nre.py - an illustrative NRE build-up (assumptions only).

|                                    |    |       |       |
|------------------------------------|----|-------|-------|
| Engineering (people)               | \$ | 18.8M | 35.5% |
| EDA licences                       | \$ | 9.0M  | 17.1% |
| Third-party IP                     | \$ | 12.0M | 22.7% |
| Mask set (1 full + 1 metal respin) | \$ | 6.5M  | 12.3% |
| Test chip / MPW                    | \$ | 1.0M  | 1.9%  |
| Package + test dev                 | \$ | 2.5M  | 4.7%  |
| Compute + infra                    | \$ | 3.0M  | 5.7%  |
| TOTAL NRE                          | \$ | 52.8M |       |
| Engineer-years: 75                 |    |       |       |

Output of nre.py (python3).

Even in this modest example people are only about a third of the cost; licences and IP together exceed them. At leading-edge nodes public industry estimates for a complex SoC run into hundreds of millions of dollars once software, validation and IP are included, and a full mask set alone is in the multi-million-dollar range, rising steeply with node because of EUV and multi-patterning layers. **Multi-project wafers (MPW)** or shuttles share one mask set among many designs and cut prototype costs to tens of thousands of dollars at mature nodes - the route most start-ups and universities use for test chips.

## 23.9 The cost model: unit cost and cost per good die

Unit cost is built from the wafer cost, how many dies fit on a wafer, how many of those work, and the package, test and assembly-yield costs:

$$DPW \sim \pi (d/2)^2 / A - \pi d / \sqrt{2A} \quad d = \text{usable wafer diameter}$$

$$\text{Cost\_die} = C_{\text{wafer}} / (DPW \times Y_{\text{die}})$$

$$\text{Cost\_unit} = (\text{Cost\_die} + C_{\text{package}} + C_{\text{test}}) / Y_{\text{final}}$$

$$\text{Cost\_total}(V) = NRE / V + \text{Cost\_unit} \quad V = \text{lifetime volume}$$

Unit cost model.

The first term of DPW is the wafer area divided by the die area; the second subtracts partial dies around the edge. The following calculator applies the model to the same product implemented in five nodes. The design is assumed to shrink with each node (less than ideally, because SRAM, analog and I/O scale poorly), while wafer price and NRE rise. **All inputs are illustrative assumptions** in the rough range of public estimates; real wafer prices depend on volume, customer and negotiation and are confidential.

```
import math
# ILLUSTRATIVE assumptions only - real wafer prices, D0 and NRE are negotiated and confidential
# node wafer$ D0/cm2 die_mm2 pkg+test$ NRE($M: masks, EDA, IP, people)
NODES = [
    ("65nm", 2000, 0.08, 160.0, 1.50, 15),
    ("28nm", 3000, 0.09, 60.0, 1.50, 35),
    ("16nm", 6000, 0.10, 35.0, 1.80, 55),
    ("7nm", 10000, 0.12, 16.0, 2.00, 200),
    ("5nm", 17000, 0.15, 11.0, 2.20, 400),
]
WAFER_MM, EDGE_MM, ALPHA = 300.0, 3.0, 3.0
def dies_per_wafer(a_mm2):
    d = WAFER_MM - 2 * EDGE_MM # usable diameter
    return int(math.pi * (d / 2) ** 2 / a_mm2 - math.pi * d / math.sqrt(2 * a_mm2))
def die_yield(a_mm2, d0):
    return (1 + a_mm2 / 100 * d0 / ALPHA) ** (-ALPHA) # negative binomial
print(" node mm2 DPW yield good/wfr $/good_die unit$ | unit$ + NRE share at volume")
print(" " * 48 + "| 1M 10M 100M")
for name, wafer, d0, area, pkg, nre in NODES:
    dpw = dies_per_wafer(area)
    y = die_yield(area, d0)
    good = dpw * y
    die_cost = wafer / good
    unit = die_cost + pkg / 0.98 # 98% final-test yield
    amort = [unit + nre * 1e6 / vol for vol in (1e6, 1e7, 1e8)]
    print(f"{name:>5} {area:4.0f} {dpw:5d} {y:6.3f} {good:8.0f} {die_cost:9.2f} {unit:5.2f} |"
          + "".join(f"{c:7.2f}" for c in amort))
```

cost.py - cost per good die and per unit across nodes (illustrative assumptions).

| node | mm2 | DPW  | yield | good/wfr | \$/good_die | unit\$ |  | unit\$ + NRE | share at volume |      |
|------|-----|------|-------|----------|-------------|--------|--|--------------|-----------------|------|
|      |     |      |       |          |             |        |  | 1M           | 10M             | 100M |
| 65nm | 160 | 372  | 0.882 | 328      | 6.09        | 7.62   |  | 22.62        | 9.12            | 7.77 |
| 28nm | 60  | 1047 | 0.948 | 992      | 3.02        | 4.55   |  | 39.55        | 8.05            | 4.90 |
| 16nm | 35  | 1829 | 0.966 | 1766     | 3.40        | 5.23   |  | 60.23        | 10.73           | 5.78 |
| 7nm  | 16  | 4079 | 0.981 | 4002     | 2.50        | 4.54   |  | 204.54       | 24.54           | 6.54 |
| 5nm  | 11  | 5974 | 0.984 | 5877     | 2.89        | 5.14   |  | 405.14       | 45.14           | 9.14 |

Output of cost.py (python3).



```
# Break-even volume between two nodes: NRE_a + V*unit_a = NRE_b + V*unit_b
def breakeven(nre_a, unit_a, nre_b, unit_b):
    return (nre_b - nre_a) * 1e6 / (unit_a - unit_b)
# numbers taken from the cost table above (illustrative)
print(f"65nm -> 28nm pays off above {breakeven(15, 7.62, 35, 4.55)/1e6:5.1f} M units")
print(f"28nm -> 7nm  pays off above {breakeven(35, 4.55, 200, 4.54)/1e6:8.0f} M units"
      " (unit cost barely moves)")
```

breakeven.py - volume at which a more advanced node becomes cheaper overall.

```
65nm -> 28nm pays off above 6.5 M units
28nm -> 7nm  pays off above 16500 M units (unit cost barely moves)
```

Output of breakeven.py (python3).

Under these assumptions the silicon cost per good die is lowest at 7 nm, but only slightly below 28 nm (16 nm is dearer than 28 nm because its wafer price rises faster than the die shrinks), and once NRE is included the advanced nodes are more expensive at every volume in the table. For a small die the package and test costs are comparable to the die cost, so shrinking the die buys little. The economic case for an advanced node is therefore **performance and energy efficiency** (and, for large dies, fitting the design at all), not cost - unless volumes reach the hundreds of millions. This is the quantitative core of many real node-selection decisions.

### SENSITIVITY ANALYSIS

Cost models are only as good as their inputs. Always run sensitivities: +/-20% on wafer price, D0 doubled (early ramp), die area +15% (typical growth from the first estimate to tapeout), package cost for a larger body, and a lower volume forecast. If the decision flips under plausible changes, it was not a robust decision.

## 23.10 Make versus buy

Before committing to an ASIC, compare the alternatives over the product lifetime:

| Option                                   | NRE                         | Unit cost                            | Time to market         | Best when                                                          |
|------------------------------------------|-----------------------------|--------------------------------------|------------------------|--------------------------------------------------------------------|
| <b>Off-the-shelf chip (ASSP/MCU)</b>     | Lowest (board and software) | Vendor's price                       | Fastest                | A standard part meets the needs                                    |
| <b>FPGA</b>                              | Low                         | High (often tens to thousands of \$) | Fast; reprogrammable   | Low volume, changing requirements, prototyping                     |
| <b>Structured ASIC / FPGA conversion</b> | Medium                      | Medium                               | Medium                 | Moderate volume, design proven in an FPGA                          |
| <b>Full-custom ASIC (in-house)</b>       | Highest                     | Lowest                               | Slowest (18-36 months) | High volume or a PPA/IP differentiation that nothing else provides |
| <b>ASIC via design-services partner</b>  | High (partner margin)       | Low                                  | Medium-slow            | Team lacks back-end, packaging or foundry access                   |

Table 23.5 - Make-versus-buy options.

Within an ASIC project the same question repeats for each block: license a CPU, SerDes, DDR PHY or PLL, or design it in-house? Licensed hard IP costs licence fees and royalties but carries silicon proof; in-house design costs engineers and schedule risk but may differentiate the product. The usual rule is to buy anything that is a standard interface or not differentiating, and to invest in-house effort where the product competes.

## 23.11 Risk management

Risk management runs throughout the project. The team keeps a **risk register** of identified risks, each with a likelihood, impact, owner, mitigation and trigger. Typical technical mitigations are **test chips** for new analog IP or new process nodes, early package and thermal studies, emulation and FPGA prototyping to start software early, conservative timing margins on first silicon, and design-for-fixability features (Chapter 22). Commercial mitigations include second-source IP, MPW slots booked in advance and foundry capacity reservations.



**INTERVIEW INSIGHT: 'SHOULD WE BUILD THIS ASIC?'**

A senior candidate answers with a model, not an opinion: estimate NRE (people x time, IP, masks, EDA), unit cost at the realistic volume (die size -> DPW -> yield -> package/test), compare the total cost of ownership with the FPGA/ASSP alternative over the product lifetime, then weigh non-cost factors - power, form factor, IP protection, supply security, time to market - and name the top three risks with mitigations. Mentioning the break-even volume and its sensitivity to schedule slip is a strong signal.

## 23.12 Functional safety and security certification

Chips for cars, factories, medical devices and payment systems must be developed under certified processes. Certification is largely about **process and evidence**: documented requirements traced to design and verification, analyses of how faults are detected, and independent assessment. Retro-fitting it to a finished design is extremely expensive.

### ISO 26262 (road vehicles)

ISO 26262 defines **Automotive Safety Integrity Levels** ASIL A (lowest) to D (highest), derived from hazard analysis of the vehicle function. Part 5 covers hardware development and Part 11 gives guidance specific to semiconductors. A chip developed out of context of a specific car is a **Safety Element out of Context (SEooC)** with assumed safety requirements. Hardware must meet quantitative metrics:

| Metric                                                   | ASIL B              | ASIL C              | ASIL D             |
|----------------------------------------------------------|---------------------|---------------------|--------------------|
| Single-point fault metric (SPFM)                         | $\geq 90\%$         | $\geq 97\%$         | $\geq 99\%$        |
| Latent fault metric (LFM)                                | $\geq 60\%$         | $\geq 80\%$         | $\geq 90\%$        |
| Probabilistic metric for random hardware failures (PMHF) | $< 100 \text{ FIT}$ | $< 100 \text{ FIT}$ | $< 10 \text{ FIT}$ |

Table 23.6 - ISO 26262 hardware architectural metric targets (item level).

The chip team performs an **FMEDA** (failure modes, effects and diagnostic analysis), estimating base failure rates (e.g. from IEC TR 62380 / SN 29500 / IEC 61709 models, or measured SER and FIT) and the diagnostic coverage of each safety mechanism: lockstep CPUs, ECC on memories and buses, parity, watchdogs, clock and voltage monitors, logic BIST and memory BIST at key-on. **Fault injection** campaigns in simulation verify the claimed coverage. Development follows a safety plan with a safety manager, tool confidence levels for EDA tools, confirmation reviews and a **safety case**. Production quality requirements (AEC-Q100, near-zero DPPM, PAT) come on top.

### IEC 61508 and derivatives

IEC 61508 is the generic functional safety standard for electrical/electronic/programmable systems, with **Safety Integrity Levels** SIL 1 to SIL 4. It is the parent of sector standards (ISO 26262 for automotive, IEC 62304 for medical software, EN 50128/50129 for railway, IEC 61511 for process industries). Its hardware metrics use the **safe failure fraction (SFF)** and hardware fault tolerance to set allowable SILs, and IEC 61508-2 Annex E gives specific requirements for on-chip redundancy.

### Security: Common Criteria and related schemes

**Common Criteria** (ISO/IEC 15408) evaluates security products against a **Protection Profile** or Security Target, at **Evaluation Assurance Levels** EAL1-EAL7; smart-card and secure element chips are typically evaluated at EAL4+ to EAL6+ with 'AVA\_VAN.5' vulnerability analysis (resistance to high-attack-potential attackers, including side-channel and fault-injection attacks) in an accredited laboratory. Other schemes include FIPS 140-3 for cryptographic modules, EMVCo for payment chips, SESIP and PSA Certified for IoT, and ISO/SAE 21434 for automotive cybersecurity engineering. For the chip team this means secure development lifecycle processes, secure site certification for design and manufacturing locations, countermeasures in the design (masking, sensors, secure boot, key storage) and extensive documentation.

**PITFALL: BOLTING ON SAFETY OR SECURITY LATE**

Safety and security requirements change the architecture: lockstep cores double CPU area, ECC widens every memory, secure key storage needs special memories and fuses, and the development process itself must be audited from the start. A team that decides to 'get ASIL B' after RTL freeze usually has to redo large parts of the project.

### 23.13 Summary

- Projects progress through spec, architecture, RTL, netlist and tapeout freezes, each gated by a data-driven review; late changes must be replanned, not squeezed in.
- Team composition shifts from architects to verification to physical design to validation; verification is typically the largest group.
- Synopsys, Cadence and Siemens supply most commercial EDA (Synopsys now also owns Ansys); licences, compute and foundry sign-off certification shape the flow; open-source tools cover open PDKs.
- Reproducible methodology means scripted flows, version-controlled and tagged design data, CI regressions and QoR dashboards.
- $NRE = \text{people} + \text{EDA} + \text{IP} + \text{masks} + \text{test chips} + \text{package/test development}$ ;  $\text{unit cost} = \text{wafer cost} / (\text{DPW} \times \text{yield}) + \text{package} + \text{test}$ , divided by final yield. Advanced nodes pay off in performance and energy, and in cost only at very high volume.
- Make-versus-buy and risk registers structure the business decisions; ISO 26262, IEC 61508 and Common Criteria impose process, metrics and evidence that must be planned from day one.

### 23.14 Exercises

- 1 Draw a milestone plan for a 40 mm<sup>2</sup> SoC with a 20-month schedule from spec freeze to tapeout. Place each freeze and review and state its exit criteria.
- 2 Change cost.py to a 400 mm<sup>2</sup> design at 28 nm (scale the other nodes by the same ratios). Where is the cheapest node now, and why does the answer differ from the small-die case?
- 3 Using nre.py, estimate the NRE if verification takes six months longer with the same team, and the resulting increase in cost per unit at 2 million units.
- 4 An FPGA solution costs \$80 per unit with \$2M NRE; an ASIC costs \$9 per unit with \$30M NRE. Compute the break-even volume. How does a 9-month later market entry for the ASIC change the decision?
- 5 For an ASIL D microcontroller, list five on-chip safety mechanisms and the fault classes each detects. Which of them contribute to the SPFM and which to the LFM?

## PART VI

# Practice and Roadmap

The open-source ASIC flow - Yosys, OpenROAD, OpenLane/LibreLane, Magic, KLayout, Netgen and the open SKY130, GF180MCU and IHP SG13G2 PDKs - run for real on the SkyWater libraries; a complete case study that takes an int8 MAC block from specification through simulation, lint, synthesis, multi-corner timing, floorplan and power numbers to the GDSII hand-off; and a career roadmap for every ASIC role, with a 12-month study plan and interview preparation.

## 24. The Open-Source ASIC Flow: Yosys, OpenROAD/OpenLane and Open PDKs

For forty years the ASIC flow was closed at both ends: the process design kit (PDK) was under a foundry NDA and the tools were licensed from three EDA vendors at prices only companies could pay. A student could write RTL and simulate it, but could not legally see a real design rule, a real Liberty file or a real standard-cell layout, let alone tape out. Since 2020 that has changed. There are now manufacturable open PDKs, an open RTL-to-GDSII tool chain that has produced hundreds of working chips, and shuttle programmes that put a small design on silicon for the price of a laptop.

This chapter explains what exists, how the pieces fit together and where the open flow stops being a substitute for the commercial one. Everything that can run in a plain Linux container is run for real here against the SkyWater SKY130 files: the Liberty corners, the technology LEF, a standard-cell GDS read through the KLayout API, and Yosys synthesis. OpenROAD itself is described, not run; the commands you would type are given exactly. Chapter 25 then pushes a complete block through every step.

### 24.1 Why an open flow matters, and who uses it

| User                                    | What the open flow gives them                                                                                                                                                             |
|-----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Students and self-learners</b>       | Real PDK data (Liberty, LEF, GDS, DRC decks, SPICE models) and every step of the flow, inspectable down to the source code. The mapping from Chapters 8-19 to real files becomes concrete |
| <b>Universities and research groups</b> | Tape-outs for courses and papers without NDAs; placement, routing and CTS algorithms can be modified and published (OpenROAD itself came out of research)                                 |
| <b>Start-ups and hobbyists</b>          | A cheap path to a first mature-node chip: a sensor interface, a small RISC-V microcontroller, a mixed-signal test chip                                                                    |
| <b>Industry engineers</b>               | Scriptable reference flows for experiments, CI regression of RTL through a real synthesis and place-and-route, design-space exploration without licence limits, and interview portfolios  |
| <b>Tool builders</b>                    | A common, open database (OpenDB) and test designs to benchmark new algorithms - including machine-learning driven placement and routing research                                          |

#### THE ONE-SENTENCE SUMMARY

The open flow is a **complete, working, mature-node RTL-to-GDSII flow** - good enough to tape out 130/180 nm chips that work - but it is **not a sign-off substitute** for a commercial flow at advanced nodes, where foundry-certified extraction, timing, power-integrity and physical-verification tools are contractually required.

### 24.2 A short history: from closed PDKs to open silicon

| Year    | Event                                                                                                                                      | Why it mattered                                                                                                |
|---------|--------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| 1980s   | Berkeley <b>Magic</b> layout editor (John Ousterhout and others); later maintained for decades by Tim Edwards                              | A free layout editor with interactive DRC and extraction - the backbone of academic VLSI courses               |
| 2000s   | Berkeley <b>ABC</b> logic synthesis and verification (Alan Mishchenko)                                                                     | The technology-mapping engine used inside Yosys and many research tools                                        |
| 2012-13 | <b>Yosys</b> open synthesis suite (Claire Wolf)                                                                                            | A production-quality Verilog front end and synthesis script engine                                             |
| 2018    | DARPA <b>IDEA</b> programme funds <b>OpenROAD</b> (led by UC San Diego) with the goal of a no-human-in-the-loop, 24-hour RTL-to-GDSII flow | Placement, CTS, routing, extraction and STA released as open source; <b>OpenSTA</b> (Parallax Software) opened |

| Year    | Event                                                                                                                                                     | Why it mattered                                                                                        |
|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| 2020    | Google and SkyWater release the <b>SKY130</b> open PDK; Google-sponsored <b>Open MPW</b> shuttles run by <b>Efabless</b> ; <b>OpenLane</b> flow published | For the first time a manufacturable PDK is public (Apache-2.0) and designs can be fabricated for free  |
| 2022    | <b>GF180MCU</b> open PDK (GlobalFoundries with Google); <b>Tiny Tapeout</b> starts (Matt Venn and collaborators)                                          | A second foundry; tiny designs on silicon for a small fee                                              |
| 2022-24 | <b>IHP SG13G2</b> 130 nm SiGe BiCMOS open PDK; <b>OpenLane 2</b> (Python-based, step API)                                                                 | A European research foundry with fast HBTs and a thick top-metal stack; a more flexible flow framework |
| 2025    | Efabless ceases operations; OpenLane 2 continues as <b>LibreLane</b> under the FOSSi Foundation; shuttle access moves to other providers                  | The ecosystem survived its main shuttle operator - but access routes changed and are still settling    |

Dates are approximate to the year. Treat the 2025 row as a snapshot: check the project websites for the current state.

#### PITFALL - STALE TUTORIALS

Most tutorials online were written for OpenLane 1 (a Docker/Tcl flow, `flow.tcl -design ...`) and for the Efabless Open MPW `caravel_user_project`. Variable names, directory layouts, the PDK installer (volare, now a successor called ciel in LibreLane) and shuttle logistics have all changed since. Always read the current documentation of the exact tool version you install.

## 24.3 The open PDKs

A PDK (Chapter 3) is the foundry's contract with the designer: device models, design rules and verification decks, and usually standard-cell and I/O libraries. The open PDKs contain the same kinds of files as a commercial one - only the licence differs.

|                         | SkyWater SKY130                                                                                                                                           | GlobalFoundries GF180MCU                                     | IHP SG13G2                                         |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|----------------------------------------------------|
| Node / type             | 130 nm bulk CMOS (derived from Cypress S8)                                                                                                                | 180 nm bulk CMOS                                             | 130 nm SiGe:C BiCMOS                               |
| Core / I/O voltage      | 1.8 V core; 3.3 V and 5 V devices                                                                                                                         | 3.3 V (also 5 V / 6 V devices)                               | 1.2 V core; 3.3 V I/O                              |
| Interconnect            | Local interconnect <code>li1</code> + 5 Al metals (met1-met5)                                                                                             | Up to 6 metals (3-6 configurable)                            | 5 thin metals + 2 thick top metals (TopMetal1/2)   |
| Open std-cell libraries | <code>sky130_fd_sc_hd</code> (high density), <code>hs</code> , <code>ms</code> , <code>ls</code> , <code>lp</code> , <code>hdl1</code> , <code>hvl</code> | <code>gf180mcu_fd_sc_mcu7t5v0</code> , <code>mcu9t5v0</code> | <code>sg13g2_stdcell</code>                        |
| Special devices         | MiM caps, ReRAM option, SONOS, precision resistors                                                                                                        | MiM caps, poly resistors                                     | HBTs with fT in the hundreds of GHz, MiM caps      |
| Typical use             | Digital, mixed-signal, first tape-outs                                                                                                                    | Higher-voltage, analog, MCU-style designs                    | RF/mm-wave, analog, digital with thick-metal power |

Key facts of the three open PDKs; all numbers are the published nominal values.

### How an open PDK is organised

|                                     |                                |
|-------------------------------------|--------------------------------|
| skywater-pdk (raw foundry data)     | open_pdk (Tim Edwards)         |
| libraries/sky130_fd_pr devices      | ++-> builds a tool-ready PDK:  |
| libraries/sky130_fd_sc_hd std cells |                                |
| cells/<cell>/ .gds .lef .v          | \$PDK_ROOT/sky130A/            |
| timing/*.lib.json (per corner)      | libs.tech/ <- tool setup files |
| libraries/sky130_fd_io I/O cells    | magic/ sky130A.tech, .magirc   |
| libraries/sky130_fd_pr_reram ... -+ | klayout/ DRC/LVS decks, .lyp   |
|                                     | netgen/ LVS setup              |
| volare / ciel: download a PREBUILT  | ngspice/ model includes        |
| open_pdk build by version hash      | openlane/ flow config          |
|                                     | libs.ref/ <- design views      |
|                                     | sky130_fd_sc_hd/               |
|                                     | lib/ lef/ gds/ verilog/        |
|                                     | techlef/ spice/ cdl/           |

From raw foundry repositories to the libs.tech / libs.ref layout that every open tool expects. sky130A and sky130B differ in the back-end stack (sky130B adds the ReRAM layers).

**libs.ref** holds **design views** of each library (the same split as any commercial kit: Liberty for timing, LEF for abstracts, GDS for layout, Verilog for simulation, CDL/SPICE for LVS); **libs.tech** holds **tool set-ups** (the Magic tech file, KLayout DRC/LVS scripts, Netgen set-up, flow configuration). Because building open\_pdk from source takes a long time, a version manager (**volare**, and for LibreLane its successor **ciel**) downloads a prebuilt PDK pinned to a commit hash - exactly the reproducibility a real project needs.

### Run: what is inside the SKY130 hd Liberty files

The raw **sky130\_fd\_sc\_hd** repository stores timing as per-cell JSON; the assembled **.lib** files ship inside open\_pdk and inside the OpenROAD and OpenROAD-flow-scripts platform directories, which is where the files used in this chapter were fetched from. First, what a synthesis tool is allowed to use. Real flows remove cells that must not appear in ordinary logic (power-domain **lpflow** cells, probe cells, clock-tree and delay cells reserved for CTS and hold fixing); OpenROAD-flow-scripts does it with **DONT\_USE\_CELLS**, this script does the same by deleting the cell groups:

```
# Remove cells a synthesis run must not use (power-domain, probe, clock-tree,
# delay and tri-state cells) - the same idea as ORFS DONT_USE_CELLS.
import re, sys
DONT = re.compile(r'sky130_fd_sc_hd__(lpflow|probe|clkdiv|clkbuf|clkdiv|dlygate|'
r'dlymetal|dlclkp|sdclkp|einv|ebuf|macro|dlx|dlrt|dlrnb|dlrbp)')
for src, dst in zip(sys.argv[1:2], sys.argv[2:2]):
    lines = open(src).read().split('\n'); out = []; i = 0; kept = drop = 0
    while i < len(lines):
        m = re.match(r'\s*cell\s*\(\s*"?(w+)"?\s*\)', lines[i])
        if m and DONT.search(m.group(1)):
            d = 0
            while True:
                d += lines[i].count('{') - lines[i].count('}'); i += 1
                if d == 0: break
            drop += 1; continue
        if m: kept += 1
        out.append(lines[i]); i += 1
    open(dst, 'w').write('\n'.join(out))
    print("%-36s kept %3d cells, dropped %3d" % (src, kept, drop))
```

duflit.py - removing dont-use cells from a Liberty file before synthesis (run with python3).

|                                  |                            |
|----------------------------------|----------------------------|
| sky130_fd_sc_hd_tt_025C_1v80.lib | kept 329 cells, dropped 99 |
|----------------------------------|----------------------------|

Real output: 428 cells in the tt file, 329 left for logic synthesis.

Next, a summary of the library and the delay of four cells at five published PVT corners, looked up from the NLDM tables with bilinear interpolation (the same lookup an STA tool does; Chapter 9). Three corners are the assembled **.lib** files; **ss\_100C\_1v60** and **tt\_100C\_1v80** were rebuilt from the per-cell JSON files of the SkyWater repository, and the rebuild was validated by reproducing the tt results exactly.

```
tt_025C_1v80: 428 cells in 158 functional families
inverter drive strengths: [1, 2, 4, 6, 8, 12, 16]
corner      inv_1  nand2_1  xor2_1  dfxtp_1  (ns, input slew 0.1 ns, load 0.01 pF)
ss_n40C_1v40 0.291  0.355  0.827  1.418
ss_100C_1v60 0.153  0.194  0.400  0.715
tt_100C_1v80 0.106  0.114  0.231  0.365
tt_025C_1v80 0.114  0.123  0.241  0.368
ff_n40C_1v95 0.093  0.099  0.163  0.238
```

Real output of libsum.py: the SKY130 hd library and cell delays across corners (NLDM lookup at 0.1 ns input slew, 10 fF load).

- **Voltage dominates.** From ff 1.95 V to ss 1.40 V the flip-flop clock-to-Q grows about 6x (0.238 -> 1.418 ns); the XOR about 5x. A corner list is a statement about the supply range your product guarantees, not a formality.
- **Temperature is not monotonic.** In this data **tt\_100C** is slightly **faster** than **tt\_025C** for the inverter and NAND. Threshold voltage falls with temperature while mobility also falls; which effect wins depends on overdrive (Chapter 2). Never assume the hot corner is the slow one - that is why sign-off covers both temperature extremes.
- **Complex cells degrade more.** The XOR (stacked, internally buffered) loses more speed at low voltage than the inverter, so the critical path can **change** between corners.

## Run: the technology LEF - rows, sites and the metal stack

Place-and-route never sees transistors. It sees the **technology LEF** (sites, routing layers, pitches, widths, via rules) and the **cell LEF** (each cell's size, pins and blockages). A dozen lines of Python print the parts that matter for floorplanning and routing:

```
# lefstack.py - summarise the SKY130 routing stack from the technology LEF
import re
t = open("sky130_fd_sc_hd.tlef").read()
site = re.search(r"SITE unithd\s+(.*?)END unithd", t, re.S).group(1)
print("SITE unithd:", " ".join(re.search(r"SIZE ([\d.]+) BY ([\d.]+)", site).groups()), "um")
print("%-6s %-8s %-10s %7s %7s %9s"
      % ("layer", "type", "direction", "pitch", "width", "min area"))
for m in re.finditer(r"\nLAYER (\w+)\s+(.*?)\nEND \1", t, re.S):
    name, body = m.groups()
    typ = re.search(r"TYPE (\w+)", body).group(1)
    if typ not in ("ROUTING", "CUT"): continue
    d = re.search(r"DIRECTION (\w+)", body)
    p = re.search(r"\n\s*PITCH ([\d.]+)", body)
    w = re.search(r"\n\s*WIDTH ([\d.]+)", body)
    a = re.search(r"\n\s*AREA ([\d.]+)", body)
    print("%-6s %-8s %-10s %7s %7s %9s" % (name, typ, d.group(1) if d else "-",
      p.group(1) if p else "-", w.group(1) if w else "-", a.group(1) if a else "-"))
```

lefstack.py - summarise the SKY130 hd technology LEF.

```
SITE unithd: 0.46 2.72 um
layer type direction pitch width min area
li1 ROUTING VERTICAL 0.46 0.17 0.0561
mcon CUT - - 0.17 -
met1 ROUTING HORIZONTAL 0.34 0.14 0.083
via CUT - - 0.15 -
met2 ROUTING VERTICAL 0.46 0.14 0.0676
via2 CUT - - 0.2 -
met3 ROUTING HORIZONTAL 0.68 0.3 0.24
via3 CUT - - 0.2 -
met4 ROUTING VERTICAL 0.92 0.3 0.24
via4 CUT - - 0.8 -
met5 ROUTING HORIZONTAL 3.4 1.6 4
```

Real output: the unithd site and the routing stack (um). li1 has a two-value pitch (0.46 0.34); the first is shown.



Every hd cell is an integer number of 0.46 um sites wide and exactly one 2.72 um row tall; **SIZE 1.38 BY 2.72** for `sky130_fd_sc_hd__inv_1` is three sites. The direction alternates layer by layer (li1 and met2 and met4 vertical, met1, met3 and met5 horizontal), and the pitch grows towards the top of the stack - thin, dense wires below for local routing, thick wide wires above for power and long nets (Chapters 14 and 17). `li1` is SKY130's peculiar local interconnect: a resistive titanium-nitride layer that the cells use internally and the router uses only for very short hops.

### Run: looking inside a standard-cell GDS with KLayout

KLayout is both a layout viewer/editor and a Python-scriptable geometry engine; the `klayout` package on PyPI exposes the same database API without the GUI (`pip install klayout`). The script below reads the real GDS of two cells from the SkyWater repository, merges the shapes on each drawing layer and reports their area and extent, and checks the placement boundary against the LEF.

```
# gdsrep.py - inspect SKY130 standard-cell GDS files with the KLayout Python API
import sys, re
import klayout.db as db
NAMES = {(64, 20): "nwell", (65, 20): "diff", (66, 20): "poly", (66, 44): "licon1",
         (67, 20): "li1", (67, 44): "mcon", (68, 20): "met1", (78, 44): "hvtp",
         (81, 4): "areaid.sc", (93, 44): "nsdm", (94, 20): "psdm", (95, 20): "npc",
         (236, 0): "prBoundary"}
lef = open("sky130_fd_sc_hd_merged.lef").read()
for f in sys.argv[1:]:
    ly = db.Layout(); ly.read(f); top = ly.top_cell()
    size = re.search(r"MACRO %s\s.*?SIZE\s+([\d.]+) BY\s+([\d.]+)" % top.name, lef, re.S)
    w, h = float(size.group(1)), float(size.group(2))
    print("%s: LEF SIZE %.2f x %.2f um" % (top.name, w, h))
    rows = []
    for li in ly.layer_indexes():
        inf = ly.get_info(li); key = (inf.layer, inf.datatype)
        if key not in NAMES: continue # skip pin/label purposes
        r = db.Region(top.begin_shapes_rec(li)); n = r.count(); r.merge()
        bb = r.bbox()
        rows.append((key, NAMES[key], n, r.area() * ly.dbu ** 2,
                    bb.width() * ly.dbu, bb.height() * ly.dbu))
    for key, nm, n, a, w, h in sorted(rows):
        print(" %3d/%-2d %-10s %3d shapes %7.3f um^2 extent %5.2f x %4.2f"
              % (key[0], key[1], nm, n, a, w, h))
```

gdsrep.py - read SKY130 cell GDS with the KLayout Python API (klayout 0.30).

```
sky130_fd_sc_hd__inv_1: LEF SIZE 1.38 x 2.72 um
 64/20 nwell      1 shapes    2.825 um^2 extent 1.76 x 1.60
 65/20 diff       2 shapes    1.105 um^2 extent 0.67 x 2.25
 66/20 poly       1 shapes    0.469 um^2 extent 0.43 x 2.51
 66/44 licon1     11 shapes    0.318 um^2 extent 0.59 x 2.11
 67/20 li1        6 shapes    1.646 um^2 extent 1.38 x 2.89
 67/44 mcon       6 shapes    0.173 um^2 extent 1.09 x 2.89
 68/20 met1       2 shapes    1.325 um^2 extent 1.38 x 3.20
 78/44 hvtp       1 shapes    2.029 um^2 extent 1.38 x 1.47
 81/4  areaid.sc  1 shapes    3.754 um^2 extent 1.38 x 2.72
 93/44 nsdm       1 shapes    1.663 um^2 extent 1.38 x 1.21
 94/20 psdm       1 shapes    2.146 um^2 extent 1.38 x 1.55
 95/20 npc        1 shapes    0.511 um^2 extent 1.38 x 0.37
236/0  prBoundary 1 shapes    3.754 um^2 extent 1.38 x 2.72
sky130_fd_sc_hd__dfxtp_1: LEF SIZE 7.36 x 2.72 um
 66/20 poly      14 shapes    5.511 um^2 extent 6.88 x 2.51
 66/44 licon1     50 shapes    1.445 um^2 extent 6.99 x 2.13
236/0  prBoundary 1 shapes   20.019 um^2 extent 7.36 x 2.72
```

Real output for `inv_1` (full) and `dfxtp_1` (selected layers). GDS layer/datatype numbers are the SKY130 layer map.



- The **prBoundary** (236/0) and **areaid.sc** layers are exactly the LEF size - 1.38 x 2.72 um for inv\_1, 7.36 x 2.72 um for the flip-flop. The abstract and the layout agree by construction; LVS and the GDS merge rely on it.
- **met1** extends to 3.20 um in height: the VPWR/VGND rails are 0.48 um wide and centred on the row edges, so abutting rows share them (the same 0.48 um width the ORFS PDN script uses for its met1 follow-pin stripes).
- **nwell** overhangs the cell (1.76 um wide versus 1.38) - wells of abutting cells merge. **hvt** marks high-Vth PMOS: the hd library uses HVT PFETs throughout, part of why its leakage is so low.
- The flop has 14 poly shapes and 50 contacts against 1 and 11 for the inverter: sequential cells are the densest part of any standard-cell library, and at 20 um<sup>2</sup> one dftxp\_1 is 5.3 inverters.

#### INSTALLING THE LAYOUT TOOLS

Debian/Ubuntu package **klayout**, **magic** and **netgen-lvs** (`apt-get install klayout magic netgen-lvs`), but distribution versions are often years old; the flows pin specific versions. For scripting, `pip install klayout` is the quickest start. For everything at once, the **IIC-OSIC-TOOLS** Docker image (JKU Linz) bundles Yosys, OpenROAD, OpenLane, Magic, KLayout, Netgen, ngspice, xschem and the PDKs.

## 24.4 The tools and what each one does

| Tool                                     | Role in the flow                                                                                                                       | Commercial counterpart                   |
|------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------|
| <b>Yosys + ABC</b>                       | RTL elaboration, generic synthesis, technology mapping to Liberty cells, netlist export; also formal (Symbyosys) and equivalence (EQY) | Design Compiler / Fusion Compiler, Genus |
| <b>OpenROAD</b> (one binary, many tools) | Floorplan, PDN, placement, resizing, CTS, global/detailed routing, extraction, STA, IR drop - on one database (OpenDB)                 | Innovus, ICC2 / Fusion Compiler          |
| <b>OpenSTA</b>                           | Static timing analysis: Liberty, Verilog, SDC, SPEF, SDF; multi-corner                                                                 | PrimeTime, Tempus                        |
| <b>OpenRCX</b>                           | Parasitic extraction from routed DEF to SPEF                                                                                           | StarRC, Quantus                          |
| <b>Magic</b>                             | Layout editor, DRC, extraction (for LVS), GDS read/write, antenna checks                                                               | Virtuoso (layout), Calibre (DRC)         |
| <b>KLayout</b>                           | Viewer/editor, scripted DRC and LVS decks, GDS/OASIS merge, XOR                                                                        | Calibre DESIGNrev, Calibre DRC/LVS       |
| <b>Netgen</b>                            | LVS: compare layout-extracted netlist to the schematic/netlist                                                                         | Calibre LVS, Pegasus                     |
| <b>Icarus / Verilator</b>                | Event-driven and cycle-based simulation, lint                                                                                          | VCS, Xcelium, Questa; SpyGlass lint      |
| <b>ngspice / Xyce, xschem</b>            | Circuit simulation and schematic capture for analog blocks                                                                             | Spectre, HSPICE; Virtuoso schematic      |

### Inside OpenROAD

OpenROAD is one executable with a Tcl (and Python) shell. Each sub-tool reads and writes the same in-memory OpenDB, so there are no file hand-offs between floorplan, placement, CTS and routing - the same architecture as the commercial implementation tools.

| Module | Command(s)                                                      | What it does (chapter)                                     |
|--------|-----------------------------------------------------------------|------------------------------------------------------------|
| ifp    | <code>initialize_floorplan</code> ,<br><code>make_tracks</code> | Die/core size, rows from the site, routing tracks (Ch. 14) |
| ppl    | <code>place_pins</code>                                         | I/O pin placement on the block edge                        |
| tap    | <code>tapcell</code>                                            | Well-tap and end-cap cells in every row                    |

| Module            | Command(s)                                           | What it does (chapter)                                                                   |
|-------------------|------------------------------------------------------|------------------------------------------------------------------------------------------|
| pdn               | <code>pdngen, add_pdn_stripe</code>                  | Power grid: rails, stripes, rings, vias (Ch. 14)                                         |
| gpl (RePlAce)     | <code>global_placement</code>                        | Analytic, electrostatics-based global placement, timing- and routability-driven (Ch. 15) |
| rsz (Resizer)     | <code>repair_design, repair_timing</code>            | Buffering, gate sizing, max slew/cap/fanout repair, setup and hold fixing (Ch. 15, 16)   |
| dpl (OpenDP)      | <code>detailed_placement, check_placement</code>     | Legalisation onto sites and rows                                                         |
| cts (TritonCTS)   | <code>clock_tree_synthesis</code>                    | H-tree/clustering based clock tree (Ch. 16)                                              |
| grt (FastRoute)   | <code>global_route</code>                            | Global routing on a GCell grid; congestion maps (Ch. 17)                                 |
| drt (TritonRoute) | <code>detailed_route</code>                          | Track assignment and DRC-aware detailed routing                                          |
| ant               | <code>check_antennas, repair_antennas</code>         | Antenna ratio check and diode insertion (Ch. 18)                                         |
| rcx (OpenRCX)     | <code>extract_parasitics, write_spef</code>          | Pattern-based RC extraction                                                              |
| sta (OpenSTA)     | <code>report_checks, report_wns, report_power</code> | Timing and power analysis, built in                                                      |
| psm (PDNSim)      | <code>analyze_power_grid</code>                      | Static IR drop and EM of the power grid (Ch. 18)                                         |
| mpl / mpl2        | <code>rtl_macro_placer</code>                        | Automatic macro placement                                                                |

OpenROAD modules and their main commands (names as used in recent OpenROAD releases).

## 24.5 The flows: OpenLane, OpenLane 2 / LibreLane and ORFS

A flow is the script that calls the tools in the right order with the right files and checks each step. There are two mainstream open flows, and they share almost all tools.

```

RTL (.v/.sv) + config + SDC
|
[1] Yosys + ABC synthesis -----> netlist.v    (+ pre-PnR STA with OpenSTA)
|
[2] Floorplan (ifp) -> I/O pins (ppl) -> tap/endcap -> PDN (pdngen)
|
[3] Global placement (gpl) -> repair_design (rsz) -> detailed placement (dpl)
|
[4] CTS (TritonCTS) -> repair_timing -setup/-hold (rsz) -> dpl
|
[5] Global route (FastRoute) -> antenna repair -> detailed route (TritonRoute)
|
[6] Fill / decap -> RC extraction (OpenRCX) -> multi-corner STA (OpenSTA)
|                                     -> IR drop (PDNSim)
[7] GDS stream-out (Magic and/or KLayout) + merge std-cell GDS
|
[8] Sign-off: DRC (Magic + KLayout), LVS (Netgen), antenna, XOR
|
final/ gds def odb spef sdf lib netlist metrics.json

```

The open RTL-to-GDSII flow. Steps 2-6 run inside OpenROAD on one database; steps 7-8 use the layout tools of the PDK.

|               | OpenLane 2 / LibreLane                                                               | OpenROAD-flow-scripts (ORFS)                                        |
|---------------|--------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| Origin        | Efabless (OpenLane 1: 2020; OpenLane 2: 2023-24); continued as LibreLane             | The OpenROAD project (UCSD)                                         |
| Written in    | Python: every step is a class; flows are sequences of steps                          | GNU make + Tcl + shell                                              |
| Configuration | <code>config.json</code> / <code>config.yaml</code> with typed, documented variables | <code>config.mk</code> (make variables) per design and per platform |

|                | OpenLane 2 / LibreLane                                                      | OpenROAD-flow-scripts (ORFS)                                              |
|----------------|-----------------------------------------------------------------------------|---------------------------------------------------------------------------|
| PDks           | sky130, gf180mcu (IHP support added more recently)                          | sky130hd/hs, gf180, ihp-sg13g2, asap7 (predictive 7 nm), nangate45        |
| Sign-off steps | Built in: Magic and KLayout DRC, Netgen LVS, antenna, XOR, multi-corner STA | Mostly up to routed GDS; DRC and LVS support vary per platform            |
| Best for       | Taping out on open shuttles (shuttle precheck expects its outputs)          | Tool research, PPA experiments, trying many designs and platforms quickly |

## What a real platform configuration looks like

ORFS keeps everything technology-specific in a platform directory. This excerpt is from the real [flow/platforms/sky130hd/config.mk](#) fetched for this chapter; every variable is one decision from Chapters 13-18 made once for the whole technology:

```
export TECH_LEF ?= $(PLATFORM_DIR)/lef/sky130_fd_sc_hd.tlef
export SC_LEF ?= $(PLATFORM_DIR)/lef/sky130_fd_sc_hd_merged.lef
export LIB_FILES ?= $(PLATFORM_DIR)/lib/sky130_fd_sc_hd_tt_025C_1v80.lib
# *probe* cells have metal on all layers; *lpflow* are multi-power-domain
export DONT_USE_CELLS += sky130_fd_sc_hd__probe_p_8 ... (36 cells)
export FILL_CELLS ?= sky130_fd_sc_hd__fill_1 sky130_fd_sc_hd__fill_2 ...
export TIEHI_CELL_AND_PORT = sky130_fd_sc_hd__conb_1 HI
export TIELO_CELL_AND_PORT = sky130_fd_sc_hd__conb_1 LO
export ABC_DRIVER_CELL = sky130_fd_sc_hd__buf_1
export PLACE_SITE = unithd
export IO_PLACER_H ?= met3
export IO_PLACER_V ?= met2
export TAP_CELL_NAME = sky130_fd_sc_hd__tapvpwrvgnd_1
export PLACE_DENSITY ?= 0.60
export MIN_ROUTING_LAYER ?= met1
export MIN_CLK_ROUTING_LAYER ?= met3
export MAX_ROUTING_LAYER ?= met5
```

Excerpt of ORFS flow/platforms/sky130hd/config.mk (abridged; the DONT\_USE list is shortened).

The companion [pdn.tcl](#) of the same platform defines the power grid as met1 follow-pin rails (0.48 um wide, pitch 5.44 um = two rows), met4 vertical stripes (1.6 um wide, 27.14 um pitch) and met5 horizontal stripes (1.6 um, 27.2 um pitch), connected met1-met4 and met4-met5; [tapcell.tcl](#) inserts [tapvpwrvgnd\\_1](#) well taps with [-distance 14](#) (um), which keeps every point of every well within the latch-up tap-distance rule (Chapter 12).

## OpenLane 2 / LibreLane configuration

```
{
  "DESIGN_NAME": "mac_int8",
  "VERILOG_FILES": "dir::src/mac_int8.sv",
  "CLOCK_PORT": "clk",
  "CLOCK_PERIOD": 20.0,
  "PNR_SDC_FILE": "dir::mac_int8.sdc",
  "SIGNOFF_SDC_FILE": "dir::mac_int8.sdc",
  "FP_SIZING": "absolute",
  "DIE_AREA": [0, 0, 125, 130],
  "PL_TARGET_DENSITY_PCT": 60
}
```

config.json for the Chapter 25 block (OpenLane 2 variable names; not run here - check the documentation of your LibreLane version, names do change).

```
# OpenLane 2 / LibreLane (inside the Nix shell, or with Docker)
openlane config.json          # or: librelane config.json
openlane --dockerized config.json  # run the tools from the container
openlane --flow Classic --to OpenROAD.STAPostPNR config.json  # stop early
openlane --last-run --flow OpenInKLayout config.json          # view layout

# ORFS
cd OpenROAD-flow-scripts/flow
make DESIGN_CONFIG=./designs/sky130hd/mac_int8/config.mk      # full flow
make DESIGN_CONFIG=./designs/sky130hd/mac_int8/config.mk gui_final # GUI
```

How the flows are invoked (commands shown, not run in this book's container).

## WHERE THE RESULTS LAND

OpenLane 2 creates `runs/RUN_<timestamp>/`, one numbered directory per step (with its log, reports and the design state) and a `final/` directory with GDS, DEF, ODB, netlists, SPEF per corner, SDF and `metrics.json`. ORFS writes `logs/`, `reports/`, `results/` and `objects/` under `<platform>/<design>/<variant>/`, with stage prefixes `1_synth ... 6_final` (for example `6_final.gds`). Learn to read the **metrics** first (WNS/TNS per corner, DRC count, LVS status, utilisation, wirelength) and open the logs only for what is red.

## 24.6 Run: synthesis with Yosys on the real SKY130 library

Synthesis is the one implementation step that runs in seconds on a laptop, so it is the best place to start. An 8-bit counter with asynchronous reset and enable:

```
module counter (input clk, input rst_n, input en, output reg [7:0] q);
    always @(posedge clk or negedge rst_n)
        if (!rst_n) q <= 8'd0;
        else if (en) q <= q + 8'd1;
endmodule
```

counter.v

```
read_liberty -lib ../tt.lib
read_verilog counter.v
synth -top counter
dfflibmap -liberty ../tt.lib
abc -liberty ../tt.lib
opt_clean
stat -liberty ../tt.lib
write_verilog -noattr counter_net.v
```

syn.py - the minimal Yosys script: generic synthesis, flop mapping, ABC logic mapping, statistics.

```
=== counter ===
Number of wires:          19
Number of wire bits:      26
Number of public wires:   4
Number of public wire bits: 11
Number of cells:          23
  sky130_fd_sc_hd__a21oi_1    1
  sky130_fd_sc_hd__a31oi_1    1
  sky130_fd_sc_hd__and3_1     1
  sky130_fd_sc_hd__and4_1     1
  sky130_fd_sc_hd__dfirtp_1   8
  sky130_fd_sc_hd__nand2_1    2
  sky130_fd_sc_hd__nand3_1    1
  sky130_fd_sc_hd__nor2_1     2
  sky130_fd_sc_hd__xnor2_1    3
  sky130_fd_sc_hd__xor2_1     3
Chip area for module '\counter': 299.036800
```

Real output of **yosys syn.py** (Yosys 0.33) against the dont-use-filtered sky130\_fd\_sc\_hd tt\_025C\_1v80 Liberty.

Eight `dfrtp_1` (D flop, async active-low reset, 25.02  $\mu\text{m}^2$  each) hold the count, and 15 small gates form the incrementer: the flops are two-thirds of the area (200 of 299  $\mu\text{m}^2$ ), which is typical of control-dominated logic in this library. `read_liberty -lib` loads the cells as black boxes so the netlist can be checked; `dfflibmap` and `abc -liberty` do the mapping (Chapter 8 explains what happens inside); `stat -liberty` sums the `area` attribute of every cell. Chapter 25 extends this script with an ABC delay target and timing constraints.

#### PITFALL - YOSYS STAT IS CELL AREA, NOT BLOCK AREA

The 299  $\mu\text{m}^2$  is the sum of cell footprints. The placed block needs whitespace for routing and buffering (utilisation 50-70 %), well taps, fillers, a power grid and pin access - so the core is roughly 1.5-2x larger, and the die larger again. Chapter 25 does the arithmetic.

## 24.7 Tiny Tapeout, shuttles and getting a design onto silicon

**Tiny Tapeout** aggregates hundreds of tiny designs onto one chip. Each project gets one or more **tiles** (on the SKY130 shuttles a single tile was on the order of 160 x 100  $\mu\text{m}$ , room for a few hundred to a thousand-odd standard cells) with a fixed interface: 8 dedicated inputs, 8 dedicated outputs, 8 bidirectional I/Os, clock, reset and an enable. An on-chip multiplexer selects which project drives the pins, and a demo board lets you exercise the chip after delivery. You write Verilog (or draw an analog block), push to a GitHub template, and a GitHub Action runs the whole open flow and produces the GDS and a viewer of your tile.

| Route to silicon                           | What you get                                                                                                                        | Notes                                                                                                                                                                                                                           |
|--------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Tiny Tapeout</b>                        | A tile on a shared chip, a demo board                                                                                               | Cheapest and quickest; a complete learning experience including bring-up. Shuttles have run on SKY130, IHP SG13G2 and GF180; the schedule and technologies change - check <a href="https://tinytapeout.com">tinytapeout.com</a> |
| <b>Full MPW slot</b>                       | A die area of several $\text{mm}^2$ (formerly the Caravel harness: RISC-V management core plus about 10 $\text{mm}^2$ of user area) | The Efabless Open MPW/chiplgnite programmes ended with Efabless in 2025; successor programmes (for example ChipFoundry for SKY130, wafer.space for GF180) and IHP's own research MPWs have appeared. Verify current offers      |
| <b>University programmes</b>               | Europractice, MOSIS-style brokers, IHP research runs                                                                                | Commercial PDKs under NDA, or open PDKs; funded for education and research                                                                                                                                                      |
| <b>Commercial MPW / dedicated mask set</b> | Your own reticle or a share of one                                                                                                  | The production path of Chapters 19 and 23                                                                                                                                                                                       |

Access routes change year to year; this is a map of the kinds of options, not a price list.

#### INTERVIEW INSIGHT - WHAT SHUTTLE PRECHECK TEACHES

Shuttle operators run a **precheck** before accepting a design: correct top-level name and pin positions, no DRC or LVS errors, consistent power connections, no layer outside the allowed set, GDS size and hash. This is a miniature version of a foundry's tapeout intake (Chapter 19). Saying in an interview that you have closed a precheck - and what failed the first time - is a strong signal that you understand the sign-off hand-off, not just RTL.

## 24.8 Installing the flow

| Method                                              | Commands (outline)                                                                                                                                                            | Pros and cons                                                              |
|-----------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| <b>Nix</b> (recommended for OpenLane 2 / LibreLane) | Install Nix with the project's cache; <code>nix-shell</code> in the cloned repository; then <code>openlane --smoke-test</code>                                                | Bit-exact reproducible tool versions; first download is large              |
| <b>Docker</b>                                       | <code>pip install openlane</code> (or <code>librelane</code> ), then <code>openlane --dockerized --smoke-test</code> ; ORFS: <code>docker run</code> the published ORFS image | Works on any OS with Docker; file permissions and GUI forwarding need care |

| Method                       | Commands (outline)                                                                                                 | Pros and cons                                                              |
|------------------------------|--------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| <b>Build ORFS locally</b>    | <code>git clone --recursive OpenROAD-flow-scripts, ./build_openroad.sh --local, source env.sh, make -C flow</code> | Fastest iteration for tool developers; long compile, many dependencies     |
| <b>All-in-one image</b>      | IIC-OSIC-TOOLS Docker/VNC image                                                                                    | Everything including analog tools and PDKs; large                          |
| <b>Distribution packages</b> | <code>apt install yosys iverilog verilator klayout magic netgen-lvs ngspice</code>                                 | Good for learning individual tools; versions usually too old for the flows |

Commands outline the documented procedures and are not run here; they change between releases.

#### THE SMOKE TEST FIRST

Before your own design, run the flow's smoke test (OpenLane 2 ships one; ORFS has [designs/sky130hd/gcd](#)). If the reference design does not reach a clean GDS, the problem is your installation or PDK, not your RTL - and you have saved a day of confused debugging.

## 24.9 Open-source versus commercial: an honest gap analysis

| Area                  | Open flow today                                                                         | Commercial flow                                                                  | Consequence                                            |
|-----------------------|-----------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|--------------------------------------------------------|
| Nodes                 | 130/180 nm production PDKs; ASAP7 and FreePDK45/Nangate45 predictive kits for research  | Every node down to the leading edge, foundry-certified                           | No open path to a production FinFET/GAA tape-out       |
| QoR (PPA)             | Good at mature nodes; timing-driven placement, resizing and CTS work                    | Tighter optimisation, concurrent clock/data optimisation, more mature algorithms | Expect larger area or lower frequency for the same RTL |
| Timing sign-off       | OpenSTA: NLDM, multi-corner, SPEF; limited CCS/AOCV/POCV, SI                            | PrimeTime/Tempus with CCS, POCV/LVF, SI crosstalk delta-delay and noise          | Add margin; no crosstalk sign-off                      |
| Extraction            | OpenRCX pattern-based                                                                   | Field-solver-correlated, foundry-certified (StarRC, Quantus)                     | Accuracy good enough for 130 nm, not certified         |
| Power integrity       | PDNSim static IR                                                                        | Dynamic/vectorless IR, EM, RedHawk/Voltus                                        | Over-design the grid                                   |
| Physical verification | Magic, KLayout DRC; Netgen LVS - foundry-quality decks for the open PDKs                | Calibre/Pegasus/ICV, certified decks                                             | Workable at 130/180 nm                                 |
| DFT                   | Scan insertion in OpenROAD/Yosys plug-ins; limited ATPG (for example the Fault project) | Full compression, ATPG, BIST, diagnosis                                          | Production test is the weakest link                    |
| Verification          | Icarus, Verilator, cocotb, SymbiYosys, EQY; partial SystemVerilog/UVM support           | Full SV/UVM, formal apps, emulation                                              | Use Verilator + cocotb, or commercial simulators       |
| Support               | Community, GitHub issues                                                                | Vendor AEs, contractual support                                                  | You own the debugging                                  |

#### WHERE THE OPEN FLOW IS GENUINELY EXCELLENT

Learning (everything is inspectable), reproducible research, mature-node test chips, CI of RTL through a real back end, and **scripting culture**: engineers who learned on OpenROAD are usually strong at the Tcl/Python automation every commercial flow also needs.

## 24.10 Summary

---

- Open PDKs (SKY130, GF180MCU, IHP SG13G2) contain real device models, rules, decks and libraries; open\_pdk builds the tool-ready libs.tech/libs.ref layout and volare/ciel pin versions.
- Yosys/ABC synthesise; OpenROAD (ifp, pdn, gpl, rsz, dpl, cts, grt, drt, rcx, sta, psm) implements on one database; Magic, KLayout and Netgen stream out and verify.
- OpenLane 2 / LibreLane is the tape-out-oriented Python flow; ORFS is the make-based research flow; both are configured, not programmed, for a design.
- Real SKY130 data shows the physics of earlier chapters: flop clock-to-Q 0.24-1.42 ns across corners, 0.46 x 2.72 um sites, alternating routing directions, 0.48 um met1 rails.
- Tiny Tapeout and successor shuttle services make a first tape-out possible for an individual; check the current schedule.
- The gap to commercial tools is in advanced nodes, certified sign-off (SI, CCS/POCV, dynamic IR), DFT/ATPG and QoR.

## 24.11 Exercises

---

- 1 Run `dufilt.py` and `libsum.py` on the SKY130 `hs` (high-speed) library. How much faster and larger is `dfxtp_1` than in `hd`?
- 2 Extend `lefstack.py` to print the via rules (`VIARULE ... GENERATE`) and compute the maximum number of met2 tracks over a 100 um wide block.
- 3 Use `gdsrep.py` on `sky130_fd_sc_hd__nand2_1`. Predict the number of poly shapes before you run it, then explain the result.
- 4 Install OpenLane 2 or ORFS and run the `gcd` or smoke-test design. Record WNS, cell count, utilisation and DRC count from the metrics, then halve the clock period and record them again.
- 5 Write a one-page comparison of the three open PDKs for a 1 mm<sup>2</sup> sensor-interface chip with a 5 V output: which would you choose and why?



## 25. Case Study: A Block from RTL to GDSII

This chapter takes one small, realistic block through the entire flow of this book. The block is a **streaming int8 multiply-accumulate (MAC) engine** - the inner loop of every quantized neural-network accelerator, and a natural fit for a repository about on-device training, quantization and pruning. It is small enough to reason about completely (a few hundred cells) and rich enough to show every real issue: a valid/ready handshake, signed arithmetic, overflow sizing, a carry chain that fails timing at the slow corner, a high-fanout reset, clock power, and hold at the fast corner.

Every result in a green card was produced for this chapter: Icarus Verilog 12 simulation, Verilator 5.020 lint, Yosys 0.33 synthesis against the real SKY130 `sky130_fd_sc_hd` Liberty files, a zero-delay gate-level simulation, and Python models for multi-corner timing, floorplan and power that read the same Liberty and netlist files. Placement, CTS, routing and sign-off were **not run** (no OpenROAD in this environment); for those steps the exact commands and the checks to make are given and clearly marked.

| Step                            | Owner                | Tool used here                     | Status in this chapter              |
|---------------------------------|----------------------|------------------------------------|-------------------------------------|
| Spec and micro-architecture     | Architect / designer | -                                  | Written                             |
| RTL                             | RTL designer         | SystemVerilog                      | Written                             |
| Simulation                      | DV                   | Icarus Verilog 12                  | <b>Run</b>                          |
| Lint                            | RTL designer         | Verilator 5.020                    | <b>Run</b>                          |
| Synthesis                       | Synthesis / RTL      | Yosys 0.33 + ABC, SKY130 hd        | <b>Run</b>                          |
| Gate-level simulation           | DV                   | Yosys + Icarus                     | <b>Run</b> (zero delay)             |
| Multi-corner STA estimate       | STA                  | Python NLDLM model on real Liberty | <b>Run</b> (pre-layout)             |
| SDC                             | Designer / STA       | -                                  | Written, not run in a real STA tool |
| Floorplan sizing                | PD                   | Python on real cell area and site  | <b>Run</b>                          |
| Power estimate                  | Power / designer     | Python on real Liberty + VCD       | <b>Run</b>                          |
| Place, CTS, route, extraction   | PD                   | OpenROAD (ORFS / LibreLane)        | Commands given, <b>not run</b>      |
| DRC, LVS, antenna, sign-off STA | PV / STA             | Magic, KLayout, Netgen, OpenSTA    | Checklist, <b>not run</b>           |

### 25.1 Step 1 - Specification

A block spec (Chapter 5) states behaviour, interfaces, performance and the conditions under which the numbers must hold. Writing the corners and the overflow rule **before** the RTL is what makes the later steps checkable.

| Item                    | Specification                                                                                                                                                                                                      |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Function</b>         | Signed int8 dot product: $out = \sum(a[i] * b[i])$ over a vector of 1 to 511 element pairs; <code>in_last</code> marks the final pair                                                                              |
| <b>Input interface</b>  | valid/ready stream: <code>in_valid</code> , <code>in_ready</code> , <code>in_a[7:0]</code> , <code>in_b[7:0]</code> (two's complement), <code>in_last</code> ; a beat transfers when valid and ready are both high |
| <b>Output interface</b> | valid/ready stream: <code>out_valid</code> , <code>out_ready</code> , <code>out_acc[23:0]</code> (two's complement); the result is held stable while <code>out_valid &amp;&amp; !out_ready</code>                  |
| <b>Throughput</b>       | One element pair per clock when not back-pressured                                                                                                                                                                 |
| <b>Latency</b>          | Two-stage pipeline: <code>out_valid</code> rises on the clock edge after the edge that accepts the last pair                                                                                                       |
| <b>Clock / reset</b>    | One clock <code>clk</code> ; <code>rst_n</code> asynchronous assert, synchronous de-assert (synchroniser outside the block)                                                                                        |



| Item         | Specification                                                                                                                                       |
|--------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| Frequency    | 50 MHz sign-off (20 ns); 100 MHz desirable at typical                                                                                               |
| PVT          | SKY130, 1.8 V nominal, 1.60-1.95 V at the cells, -40 to 100 deg C junction: setup at ss_100C_1v60, hold at ff_n40C_1v95, tt_025C_1v80 for reporting |
| Area / power | < 0.02 mm <sup>2</sup> die; < 0.5 mW at 50 MHz, typical                                                                                             |
| Test         | Scan-insertable (all flops on <b>clk</b> , no internal resets or gated clocks)                                                                      |

### SIZING THE ACCUMULATOR

The largest product magnitude is  $(-128) \times (-128) = +16384 = 2^{14}$ . A signed accumulator of W bits holds at most  $2^{(W-1)} - 1$ . With  $W = 24$ :  $2^{23} - 1 = 8,388,607$ , and  $8,388,607 / 16,384 = 511.99$ , so **511 worst-case terms fit and 512 overflow** by exactly one LSB. The spec therefore says 1 to 511, and the testbench checks the  $511 \times (-128 \times -128) = 8,372,224$  corner directly. In general  $W = 16 + \text{ceil}(\log_2(N))$  for N terms, minus one bit if the  $-128 \times -128$  case is excluded.

## 25.2 Step 2 - Micro-architecture and RTL

```

stage 1 (multiply)          stage 2 (accumulate)          output
in_a --+                    register
in_b --+--> [ 8x8 signed ] --> p_q[15:0] --+--> (+) --> sum --+--> out_acc[23:0]
      | multiplier          p_last |      ^      | out_valid
in_last+-----> p_valid |      |      +--> acc_q[23:0]
                        |      +--- acc_q --+ (cleared on last)
in_ready <-- s1_ready = !p_valid | s2_fire |
              s2_fire = p_valid & (!p_last | !out_valid | out_ready)
out_ready -----^

```

Two pipeline stages. A non-last product always retires into the accumulator; only a last product needs the output register to be free.

The multiplier and the adder are in separate stages so that each stage has one arithmetic operator in its critical path. The back-pressure logic is the **pipeline-ready** form from the companion RTL Design guide: stage 1 may load when it is empty or when stage 2 consumes its content in the same cycle. Data flops (**p\_q**, **p\_last**) have no reset - their contents are qualified by **p\_valid** - which saves area and keeps them off the reset tree.

```

`timescale 1ns/1ps
// mac_int8: streaming signed int8 dot-product engine (Chapter 25 case study)
// One (a,b) pair per accepted input beat; in_last marks the final pair of a
// vector. The block returns sum(a*b) on the output channel, then restarts.
module mac_int8 #(
    parameter int ACC_W = 24          // 24 bits: <= 511 terms never overflow
) (
    input logic clk,
    input logic rst_n,      // async assert, sync deassert
    // input stream (valid/ready)
    input logic in_valid,
    output logic in_ready,
    input logic signed [7:0] in_a,
    input logic signed [7:0] in_b,
    input logic in_last,
    // output stream (valid/ready)
    output logic out_valid,
    input logic out_ready,
    output logic signed [ACC_W-1:0] out_acc
);
    // ---- stage 1: registered product -----
    logic p_valid, p_last;
    logic signed [15:0] p_q;
    // ---- stage 2: accumulator and output register -----

```

```

logic signed [ACC_W-1:0] acc_q;
logic signed [ACC_W-1:0] sum;
logic                s2_fire, s1_ready;

// a non-last product can always retire; a last one needs the output slot
assign s2_fire = p_valid && (!p_last || !out_valid || out_ready);
assign s1_ready = !p_valid || s2_fire;
assign in_ready = s1_ready;
assign sum      = acc_q + ACC_W'(p_q);    // sign-extending add

always_ff @(posedge clk or negedge rst_n)
  if (!rst_n)      p_valid <= 1'b0;
  else if (s1_ready) p_valid <= in_valid;

// datapath flops without reset: smaller cells, no reset-tree load
always_ff @(posedge clk)
  if (s1_ready && in_valid) begin
    p_q    <= in_a * in_b;                // 8x8 signed -> 16 bits
    p_last <= in_last;
  end

always_ff @(posedge clk or negedge rst_n)
  if (!rst_n) begin
    acc_q    <= '0;
    out_valid <= 1'b0;
    out_acc  <= '0;
  end else begin
    if (out_valid && out_ready) out_valid <= 1'b0;
    if (s2_fire) begin
      if (p_last) begin
        out_acc <= sum;
        out_valid <= 1'b1;
        acc_q    <= '0;
      end else begin
        acc_q    <= sum;
      end
    end
  end
end
endmodule

```

mac\_int8.sv - the complete RTL of the case-study block.

## 25.3 Step 3 - Simulation

The testbench is self-checking (the companion SystemVerilog guide covers the techniques): a producer task with random idle gaps, a consumer that de-asserts `out_ready` a quarter of the time, and a queue of expected results computed with 32-bit integer arithmetic. Directed corners come first: a one-element vector, the 511-term worst case, and a mixed-sign pair; then 200 random vectors of 1 to 64 elements.

```

`timescale 1ns/1ps
module tb_mac;
  logic clk = 0, rst_n = 0;
  always #5 clk = ~clk;                // 100 MHz
  logic in_valid = 0, in_last = 0, out_ready = 0;
  logic signed [7:0] in_a = 0, in_b = 0;
  logic in_ready, out_valid;
  logic signed [23:0] out_acc;
  mac_int8 dut (.*);

  integer exp_q [0:1023];                // expected-result FIFO
  integer wr = 0, rd = 0, errors = 0, beats = 0, stalls = 0;

```

```

integer n, len, i, sum, maxv = 0;

// consumer: random back-pressure, checks every result in order
always @(posedge clk) begin
    out_ready <= ($urandom % 4) != 0;          // ready 75% of cycles
    if (out_valid && !out_ready) stalls = stalls + 1;
    if (out_valid && out_ready) begin
        if (out_acc != exp_q[rd]) begin
            errors = errors + 1;
            $display("MISMATCH vec %0d: got %0d exp %0d", rd, out_acc, exp_q[rd]);
        end
        if (out_acc > maxv) maxv = out_acc;
        rd = rd + 1;
    end
end

task automatic send(input logic signed [7:0] a, b, input logic last);
    in_valid <= 1; in_a <= a; in_b <= b; in_last <= last;
    @(posedge clk);
    while (!in_ready) @(posedge clk);
    beats = beats + 1;
    in_valid <= 0;
    if ($urandom % 3 == 0) @(posedge clk);      // random idle gaps
endtask

initial if ($test$plusargs("vcd")) begin      // activity for power estimation
    $dumpfile("mac.vcd"); $dumpvars(1, dut);
end

initial begin
    repeat (3) @(posedge clk);
    rst_n <= 1;
    // directed corners: 1-term vector, worst-case 511 x (-128 * -128)
    send(-128, -128, 1); exp_q[wr] = 16384; wr = wr + 1;
    for (i = 0; i < 511; i = i + 1) send(-128, -128, i == 510);
    exp_q[wr] = 511 * 16384; wr = wr + 1;
    send(127, -128, 0); send(-1, 1, 1); exp_q[wr] = -16257; wr = wr + 1;
    // 200 random vectors of random length 1..64
    for (n = 0; n < 200; n = n + 1) begin
        len = 1 + $urandom % 64; sum = 0;
        for (i = 0; i < len; i = i + 1) begin
            in_a = $urandom; in_b = $urandom;          // blocking: sample now
            sum = sum + in_a * in_b;
            send(in_a, in_b, i == len - 1);
        end
        exp_q[wr] = sum; wr = wr + 1;
    end
    wait (rd == wr);
    repeat (2) @(posedge clk);
    $display("vectors=%0d beats=%0d out_stall_cycles=%0d errors=%0d",
        wr, beats, stalls, errors);
    $display("largest result checked: %0d (2^23-1 = %0d)", maxv, 2**23-1);
    if (errors == 0) $display("TEST PASSED"); else $display("TEST FAILED");
    $finish;
end
endmodule

```

tb\_mac.sv - self-checking testbench with random back-pressure (the +vcd switch dumps activity for Step 9).

```
$ iverilog -g2012 -Wall -o sim tb_mac.sv mac_int8.sv && vvp -n sim +vcd
vectors=203 beats=7430 out_stall_cycles=70 errors=0
largest result checked: 8372224 (2^23-1 = 8388607)
TEST PASSED
tb_mac.sv:66: $finish called at 98925000 (1ps)
```

Real output: 203 vectors, 7430 accepted beats, 70 cycles of output back-pressure, no mismatches.

To see what the checker catches, one keyword was removed: `p_q` declared without `signed`. The product is still computed correctly (the operands are signed) but `ACC_W'(p_q)` now **zero-extends** a negative product before the add:

```
MISMATCH vec 2: got 114815 exp -16257
MISMATCH vec 3: got 193344 exp -3264
MISMATCH vec 4: got 185953 exp -10655
MISMATCH vec 5: got 1275441 exp 30257
...
vectors=203 beats=7430 out_stall_cycles=70 errors=201
largest result checked: 8372224 (2^23-1 = 8388607)
TEST FAILED
tb_mac.sv:66: $finish called at 98925000 (1ps)
```

Real output with the bug injected: 201 of 203 results wrong. Only the two all-positive-product vectors pass.

#### PITFALL - SIGNEDNESS BUGS ARE INVISIBLE TO LINT

Verilator `-Wall` reports **nothing** for the unsigned `p_q` (it is legal, width-consistent code). Only a checker with negative operands finds it. A directed test with only positive values, or an FPGA demo that happened to use unsigned data, would have passed and the bug would have reached silicon.

## 25.4 Step 4 - Lint

Lint is the cheapest static check (Chapter 7 and the companion RTL guide). The final RTL is clean:

```
$ verilator --lint-only -Wall mac_int8.sv ; echo exit=$?
exit=0
```

Real output: no warnings with `-Wall`.

An earlier draft (kept to show what lint is for) added the product without the explicit sign-extending cast and left a debug signal behind:

```
%Warning-WIDTHEXPAND: mac_int8.sv:32:27: Operator ADD expects 24 bits on the RHS, but RHS's
    VARREF 'p_q' generates 16 bits.
                                : ... note: In instance 'mac_int8'
32 |   assign sum      = acc_q + p_q;
    |                   ^
%Warning-UNUSED SIGNAL: mac_int8.sv:21:40: Signal is not driven, nor used: 'dbg'
                                : ... note: In instance 'mac_int8'
21 |   logic           p_valid, p_last, dbg;
    |                   ^~~
%Error: Exiting due to 2 warning(s)
```

Real Verilator output on the draft (long lines wrapped). `WIDTHEXPAND` flags an implicit extension - here it happens to be correct because both operands are signed, but the reviewer should not have to work that out.

#### LINT WAIVERS ARE DESIGN DECISIONS

Fix, do not waive, width warnings in arithmetic. Waive only with a comment explaining why (for example an intentionally unused bit of a standard bus), and review the waiver file like RTL.

## 25.5 Step 5 - Synthesis on SKY130

Synthesis uses the dont-use-filtered `tt_025C_1v80` Liberty (Chapter 24). The ABC constraint file sets the input drive (`set_driving_cell sky130_fd_sc_hd__buf_1`) and output load (`set_load 0.005`, in pF); `-D 10000` gives ABC a 10 ns delay target.

```
# syn.py - Yosys 0.33 synthesis of mac_int8 onto SKY130 hd (tt_025C_1v80)
# ../tt.lib = sky130_fd_sc_hd__tt_025C_1v80.lib with dont-use cells removed
read_verilog -sv mac_int8.sv
synth -top mac_int8 -flatten          # generic synthesis: RTL -> $_AND_/$_DFF_ cells
dfflibmap -liberty ../tt.lib          # map flops onto real Liberty flip-flops
abc -liberty ../tt.lib -constr abc.constr -D 10000  # map logic, 10 ns target
opt_clean
hilomap -hicell sky130_fd_sc_hd__conb_1 HI -locell sky130_fd_sc_hd__conb_1 L0
stat -liberty ../tt.lib               # cell count and area from Liberty 'area'
write_verilog -noattr mac_net.v        # netlist for GLS and for the P&R flow
write_json mac_net.json               # same netlist, for the Python STA/power models
```

syn.py - synthesis script for mac\_int8.

```
=== mac_int8 ===
Number of wires:          591
Number of wire bits:      666
Number of public wires:   15
Number of public wire bits: 90
Number of cells:          644
  sky130_fd_sc_hd__a2111oi_0      1
  sky130_fd_sc_hd__a211oi_1       1
  sky130_fd_sc_hd__a21boi_0       2
  sky130_fd_sc_hd__a21o_1         6
  sky130_fd_sc_hd__a21oi_1       20
  sky130_fd_sc_hd__a22o_1         8
  sky130_fd_sc_hd__a22oi_1        3
  sky130_fd_sc_hd__a31oi_1        4
  sky130_fd_sc_hd__and2_1         8
  sky130_fd_sc_hd__and3_1         9
  sky130_fd_sc_hd__and4_1         3
  sky130_fd_sc_hd__buf_1         29
  sky130_fd_sc_hd__dfirtp_1       50
  sky130_fd_sc_hd__dfxtp_1       17
  sky130_fd_sc_hd__inv_1         10
  sky130_fd_sc_hd__maj3_1        41
  sky130_fd_sc_hd__mux2_2         4
  sky130_fd_sc_hd__mux2i_1        1
  sky130_fd_sc_hd__nand2_1       121
  sky130_fd_sc_hd__nand2b_1       5
  sky130_fd_sc_hd__nand3_1        6
  sky130_fd_sc_hd__nand3b_1       1
  sky130_fd_sc_hd__nand4_1        4
  sky130_fd_sc_hd__nand4b_1       1
  sky130_fd_sc_hd__nor2_1        44
  sky130_fd_sc_hd__nor2b_1        9
  sky130_fd_sc_hd__nor3_1         5
  sky130_fd_sc_hd__o211a_1        1
  sky130_fd_sc_hd__o21a_1         3
  sky130_fd_sc_hd__o21ai_0       66
  sky130_fd_sc_hd__o21bai_1       2
  sky130_fd_sc_hd__o22ai_1        1
  sky130_fd_sc_hd__o311a_1        2
  sky130_fd_sc_hd__o31ai_1        7
  sky130_fd_sc_hd__o32ai_1        1
  sky130_fd_sc_hd__o41ai_1        1
```

```
sky130_fd_sc_hd__or2_2      5
sky130_fd_sc_hd__or3_1      8
sky130_fd_sc_hd__xnor2_1    76
sky130_fd_sc_hd__xnor3_1     6
sky130_fd_sc_hd__xor2_1     44
sky130_fd_sc_hd__xor3_1      8
Chip area for module '\mac_int8': 5365.145600
```

Real output: Yosys 0.33 stat -liberty on sky130\_fd\_sc\_hd\_tt\_025C\_1v80 - 644 cells, 5365 um^2.

```
ABC: WireLoad = "none" Gates = 577 ( 6.8 %) Cap = 5.5 ff ( 6.0 %) Area = 3773.62 ( 82.5 %)
Delay = 6882.77 ps ( 7.6 %)
```

Real output: ABC's own summary of the combinational part (flops excluded; its internal load model, not an STA).

| Group                             | Cells      | Area (um^2)   | Share        | Comment                                                             |
|-----------------------------------|------------|---------------|--------------|---------------------------------------------------------------------|
| Reset flops <code>dfrrtp_1</code> | 50         | 1251.2        | 23.3 %       | p_valid, acc_q, out_acc, out_valid                                  |
| Plain flops <code>dfxtp_1</code>  | 17         | 340.3         | 6.3 %        | p_q, p_last                                                         |
| Combinational                     | 577        | 3773.6        | 70.3 %       | Multiplier (xor/xnor/maj3/nand) and 24-bit adder (maj3 carry chain) |
| <b>Total</b>                      | <b>644</b> | <b>5365.1</b> | <b>100 %</b> | ABC 3773.6 + flops 1591.5 = stat total                              |

Area breakdown derived from the real stat output above.

- **maj3\_1 x 41** is the tell-tale of ripple-carry adders: a full adder's carry is majority(a, b, cin). Yosys' default `alumacc` mapping produces ripple adders; ABC restructures logic but cannot invent a parallel-prefix adder. Tightening `-D` from 10000 to 2000 was tried: area rose slightly (5391 um^2) and ABC's delay improved only from 6.88 to 5.89 ns. **Architecture, not synthesis effort, sets this critical path** (Chapter 8).
- **Reset cost:** `out_acc` has a reset it does not need (it is qualified by `out_valid`). Removing it turns 24 `dfrrtp_1` (25.02 um^2) into `dfxtp_1` (20.02 um^2) and saves 24 x 5.0 = 120 um^2, 2.2 % of the block - and 24 loads on the reset tree.
- **Scan cost** (DFT, Chapter 10): replacing every flop with its scan equivalent (`sdfrrtp_1` 31.28 um^2, `sdfxtp_1` 26.28 um^2) adds 50 x 6.26 + 17 x 6.26 = 419 um^2, about 7.8 %.

## Gate-level simulation of the netlist

Before trusting a netlist, simulate it with the same testbench. Yosys can import the Liberty `function` and `ff` groups of the cells it used as behavioural modules and flatten them into a plain-Verilog netlist:

```
read_liberty -ignore_miss_func used_tt.lib
read_verilog mac_net.v
hierarchy -top mac_int8
flatten
opt_clean
write_verilog -noattr gls_flat.v
```

`gls.py` - build a flattened functional netlist (`used_tt.lib` is the tt Liberty reduced to the 42 cell types used).

```
$ iverilog -g2012 -o gsim tb_mac.sv gls_flat.v && vvp -n gsim
vectors=203 beats=7430 out_stall_cycles=70 errors=0
largest result checked: 8372224 (2^23-1 = 8388607)
TEST PASSED
tb_mac.sv:66: $finish called at 98925000 (1ps)
```

Real output: the synthesized netlist passes the same 203 vectors (zero-delay GLS).

## WHAT ZERO-DELAY GLS DOES AND DOES NOT PROVE

It shows that synthesis preserved function for these vectors and that no X appears after reset in the netlist - a common failure when datapath flops have no reset. It proves nothing about timing, and it is not exhaustive: **formal equivalence checking** (Chapter 8; Yosys `equiv_*` or EQY in the open flow, Conformal or Formality commercially) is the sign-off check that netlist == RTL. Timing-annotated GLS with SDF comes after routing.

## 25.6 Step 6 - Constraints (SDC)

```
# mac_int8.sdc - timing intent for the case-study block
create_clock -name core_clk -period 20.0 [get_ports clk]      ;# 50 MHz
set_clock_uncertainty -setup 0.50 [get_clocks core_clk]      ;# jitter + pre-CTS skew
set_clock_uncertainty -hold 0.10 [get_clocks core_clk]
set_clock_transition 0.10 [get_clocks core_clk]              ;# ideal clock until CTS

# I/O budget: 20 % of the period is used outside the block on each side
set_input_delay 4.0 -clock core_clk [all_inputs -no_clocks]
set_output_delay 4.0 -clock core_clk [all_outputs]
set_driving_cell -lib_cell sky130_fd_sc_hd_buf_1 [all_inputs -no_clocks]
set_load 0.005 [all_outputs]                                ;# pF

# rst_n is asserted asynchronously but released synchronously by a reset
# synchroniser outside the block, so recovery/removal ARE timed (no false path)
set_max_fanout 16 [current_design]
set_max_transition 1.0 [current_design]                      ;# ns, below lib 1.5
```

mac\_int8.sdc - written for this block (standard SDC as read by OpenSTA). Not run through a real STA tool here; the Python model in Step 7 applies the same period, uncertainty and I/O budget.

- The I/O delays spend 20 % of the period outside the block on each side - a common starting budget until the SoC integrator provides real numbers (Chapter 9).
- Setup uncertainty 0.5 ns before CTS covers jitter plus an allowance for skew; after CTS it is reduced to jitter plus margin because skew is then computed from the real tree.
- `rst_n` stays timed. Its de-assertion must meet **recovery** and **removal** at 50 flops; with a fanout of 50 it will be buffered by `repair_design`. Falsely pathing a synchronously released reset is a classic silicon bug (Chapter 12 of the companion RTL guide).

## 25.7 Step 7 - Timing across corners (pre-layout estimate)

OpenSTA is not available in this container, so a compact NLDM timing analyser was written for this chapter. It reads the Yosys netlist (JSON) and a Liberty corner, computes each net's load from the input-pin capacitances plus the Liberty 'Small' wire-load model, propagates late and early arrival times and slews through the `cell_rise/cell_fall` and transition tables (worst of rise and fall), starts paths at flop clock-to-Q arcs and ends them at the `setup_rising/hold_rising` constraint tables - exactly the algorithm of Chapter 9, minus crosstalk, CRPR, OCV derates and unateness tracking. It was validated by reproducing identical results from the assembled `tt.lib` and from the per-cell JSON rebuild of the same corner.

```
# minista.py - a teaching-size NLDM static timing analyser (NOT sign-off STA)
# Reads the Yosys-mapped netlist (JSON) and a SKY130 Liberty corner, propagates
# late/early arrival and slew through the NLDM tables and checks setup/hold.
...
def look(tab, x, y):
    # bilinear interpolation / extrapolation
    i1, i2, v = tab
    def seg(ax, a):
        if len(ax) == 1: return 0, 0.0
        k = min(max(bisect.bisect(ax, a) - 1, 0), len(ax) - 2)
        return k, (a - ax[k]) / (ax[k + 1] - ax[k])
    r, fx = seg(i1, x); c, fy = seg(i2, y)
    if len(i2) == 1: return v[r][0] * (1 - fx) + v[r + 1][0] * fx
    return (v[r][c] * (1 - fx) * (1 - fy) + v[r + 1][c] * fx * (1 - fy)
            + v[r][c + 1] * (1 - fx) * fy + v[r + 1][c + 1] * fx * fy)
    ...
def arc_eval(a, slew, cap, late):
    f = max if late else min
    d = f(look(a["cell_rise"], slew, cap), look(a["cell_fall"], slew, cap))
    s = f(look(a["rise_transition"], slew, cap), look(a["fall_transition"], slew, cap))
    return d, s
```

```

...
    if a["type"] == "setup_rising":
        at, sl, _ = LATE[b]
        su = max(look(a["rise_constraint"], clk_slew, sl),
                 look(a["fall_constraint"], clk_slew, sl))
        ends.append(("setup", T - unc_setup - su - at, n + "/" + p, b, at, su))
    else:
        at, sl, _ = EARLY[b]
        ho = max(look(a["rise_constraint"], clk_slew, sl),
                 look(a["fall_constraint"], clk_slew, sl))
        ends.append(("hold", at - ho - unc_hold, n + "/" + p, b, at, ho))

```

Excerpts of minista.py (about 200 lines in full): table lookup, arc evaluation and the setup/hold checks.

|              |        |           |         |     |          |      |    |  |          |       |
|--------------|--------|-----------|---------|-----|----------|------|----|--|----------|-------|
| ss_n40C_1v40 | T=10.0 | setup WNS | -30.663 | TNS | -805.230 | viol | 53 |  | hold WNS | 1.201 |
| ss_100C_1v60 | T=10.0 | setup WNS | -8.478  | TNS | -144.605 | viol | 34 |  | hold WNS | 0.782 |
| tt_100C_1v80 | T=10.0 | setup WNS | 1.117   | TNS | 0.000    | viol | 0  |  | hold WNS | 0.366 |
| tt_025C_1v80 | T=10.0 | setup WNS | 0.829   | TNS | 0.000    | viol | 0  |  | hold WNS | 0.343 |
| ff_n40C_1v95 | T=10.0 | setup WNS | 4.191   | TNS | 0.000    | viol | 0  |  | hold WNS | 0.176 |
| ss_n40C_1v40 | T=20.0 | setup WNS | -20.663 | TNS | -373.744 | viol | 38 |  | hold WNS | 1.201 |
| ss_100C_1v60 | T=20.0 | setup WNS | 1.522   | TNS | 0.000    | viol | 0  |  | hold WNS | 0.782 |
| tt_100C_1v80 | T=20.0 | setup WNS | 10.452  | TNS | 0.000    | viol | 0  |  | hold WNS | 0.366 |
| tt_025C_1v80 | T=20.0 | setup WNS | 10.166  | TNS | 0.000    | viol | 0  |  | hold WNS | 0.343 |
| ff_n40C_1v95 | T=20.0 | setup WNS | 11.850  | TNS | 0.000    | viol | 0  |  | hold WNS | 0.176 |

Real output: worst negative slack (WNS), total negative slack (TNS), number of violating endpoints and worst hold slack at five SKY130 corners, for a 10 ns and a 20 ns clock (ns; ideal clock, pre-layout wire-load estimate).

```

ss_100C_1v60  T=20.0  setup WNS  1.522  TNS  0.000  viol  0  | hold WNS  0.782
worst setup endpoint acc_q[23]/D: arrival 17.599, setup 0.379, slack 1.522
  0.815  slew 0.166  dfrtp_1  CLK->Q
  1.192  slew 0.318  xnor2_1  A->Y
  1.751  slew 0.513  o21ai_0  A2->Y
  2.751  slew 0.171  maj3_1  C->X
  3.615  slew 0.172  maj3_1  C->X
  4.478  slew 0.172  maj3_1  C->X
  5.342  slew 0.172  maj3_1  C->X
  6.205  slew 0.172  maj3_1  C->X
  7.069  slew 0.172  maj3_1  C->X
  7.932  slew 0.172  maj3_1  C->X
  8.796  slew 0.172  maj3_1  C->X
  9.659  slew 0.172  maj3_1  C->X
 10.523  slew 0.172  maj3_1  C->X
 11.386  slew 0.172  maj3_1  C->X
 12.250  slew 0.172  maj3_1  C->X
 13.113  slew 0.172  maj3_1  C->X
 14.038  slew 0.232  maj3_1  C->X
 14.300  slew 0.221  nand4_1  A->Y
 15.487  slew 1.176  a211oi_0  A1->Y
 16.370  slew 0.583  o21ai_0  A1->Y
 17.038  slew 0.437  a211oi_1  A2->Y
 17.599  slew 0.336  o32ai_1  A2->Y
worst hold endpoint p_q[5]/D: early arrival 0.819, hold req -0.063, slack 0.782

```

Real output: the worst setup path at the sign-off corner ss\_100C\_1v60, 20 ns clock.

The table is a design review in itself:

- **100 MHz passes only at typical.** At `ss_100C_1v60` the same netlist misses 10 ns by 8.5 ns with 34 failing endpoints. A block that is 'fast enough in simulation at tt' is not a product.
- **The critical path is the accumulator carry chain:** clock-to-Q of `acc_q`, one XNOR and OAI for the first bit, then 14 `maj3` carry stages at about 0.86 ns each at ss (0.40 ns at tt) and a multiplexing tail that selects between `sum`, hold and clear. The `a211oi_0` in that tail has a 1.18 ns output slew at ss - close to the 1.5 ns library limit and above the 1.0 ns SDC limit - so `repair_design` will upsize it.



- **50 MHz meets the spec corner** with 1.52 ns of setup slack and hold is clean everywhere, smallest at ff (0.18 ns) as expected.
- **The ss\_n40C\_1v40 corner fails by 20 ns** - but 1.40 V is outside this spec's 1.60 V minimum, so it is **not a sign-off corner here**. It would be for a product whose regulator can droop that far. Choosing the corner list is a specification decision (Chapter 9).
- Pre-layout numbers ignore real wires and the clock tree. Keep **10-20 % of the period as margin** at this stage for a block of this size; at advanced nodes wires dominate and the margin must be larger.

#### HOW TO REACH 100 MHZ AT SS\_100C\_1V60

Options, in order of preference: (1) a **parallel-prefix or carry-select adder** for the 24-bit accumulate (log-depth carry, roughly 5-8 stages instead of 24); (2) **carry-save accumulation** - keep the accumulator in redundant form and resolve it once per vector, which removes the carry chain from the loop entirely; (3) split the add across two cycles and accumulate two partial sums (changes latency, not throughput); (4) a faster library ([sky130\\_fd\\_sc\\_hs](#)) at the cost of area and leakage. A good interview answer names (2) - it is how real MAC arrays close timing.

## 25.8 Step 8 - Floorplan arithmetic

From the synthesized cell area, the site from the technology LEF and the tap-cell rule of the platform, the core and die size follow directly (Chapter 14):

```
# floorplan.py - size the core of mac_int8 from the real synthesized cell area
import math
CELL_AREA = 5365.1456          # um^2, Yosys 'stat -liberty' on tt_025C_1v80
SITE_W, ROW_H = 0.46, 2.72    # unithd site from sky130_fd_sc_hd.tlef
TAP_W, TAP_PITCH = 0.46, 14.0 # tapvpwrvrnd_1, ORFS tapcell -distance 14 (um)
MARGIN = 10.0                 # core-to-die margin for pins and power ring (um)
print("cell area from synthesis: %.1f um^2" % CELL_AREA)
print(" util core W x H (um)  rows  sites/row  taps  tap area  die W x H (um)  die mm^2")
for util in (0.40, 0.50, 0.60, 0.70):
    side = math.sqrt(CELL_AREA / util)          # aspect ratio 1
    rows = math.ceil(side / ROW_H)
    h = rows * ROW_H
    sites = math.ceil((CELL_AREA / util / h) / SITE_W)
    w = sites * SITE_W
    taps = rows * (math.floor(w / TAP_PITCH) + 1)
    dw, dh = w + 2 * MARGIN, h + 2 * MARGIN
    print(" %3.0f%%  %6.2f x %6.2f  %4d  %9d  %4d  %8.1f  %6.2f x %6.2f  %8.4f"
          % (util * 100, w, h, rows, sites, taps, taps * TAP_W * ROW_H, dw, dh, dw * dh / 1e6))

floorplan.py
```

```
cell area from synthesis: 5365.1 um^2
 util core W x H (um)  rows  sites/row  taps  tap area  die W x H (um)  die mm^2
 40% 115.00 x 116.96   43      250    387    484.2  135.00 x 136.96   0.0185
 50% 101.20 x 106.08   39      220    312    390.4  121.20 x 126.08   0.0153
 60%  94.30 x  95.20   35      205    245    306.5  114.30 x 115.20   0.0132
 70%  85.56 x  89.76   33      186    231    289.0  105.56 x 109.76   0.0116
```

Real output: core and die size, row and site counts at four utilisation targets. Tap counts assume a tap every 14 um in every row - an upper bound, because OpenROAD's tapcell staggers taps between rows.

```

die 121.2 x 126.1 um (50 % utilisation)
+-----+
| 10 um margin: pins on met2 (top/bottom) and met3 (sides) |
| +-----+ |
| | row 39  VPWR ===== | |
| | T cells T cells T cells T cells T cells | |
| | row 38  VGND ===== | |
| | :      met4 stripes every 27.14 um (vertical) | |
| | :      met5 stripes every 27.2 um (horizontal) | |
| | row 1   VGND ===== | |
| +--- core 101.2 x 106.1 um: 39 rows x 220 sites -----+ |
+-----+
T = tapvpwrvgnd_1 well tap  ==== = met1 follow-pin rail (0.48 um)

```

The 50 % floorplan of mac\_int8: 39 rows of 2.72 um, 220 sites of 0.46 um, met1 rails, met4/met5 grid from the sky130hd PDN script.

- At 50 % the block needs about **0.015 mm<sup>2</sup>** of die, meeting the 0.02 mm<sup>2</sup> spec; the core itself is 0.0107 mm<sup>2</sup>.
- Utilisation is a trade-off, not a target: 70 % saves 25 % of die but leaves little room for the buffers `repair_design` and CTS will add (tens of cells here) and risks congestion over the multiplier. For a block this small, 50-60 % with `PLACE_DENSITY` 0.6 is the usual starting point.
- Only about four met4 and four met5 stripes cross a 100 um core: IR drop is not an issue for 0.25 mW, but check it anyway - the same script on a 5 mm block is where grids fail.

## 25.9 Step 9 - Power estimate

Power (Chapter 11) is estimated from real Liberty data and real switching activity: the RTL simulation's VCD gives toggle rates per bit; the netlist and Liberty pin capacitances plus the wire-load model give the capacitance of every net; the Liberty `internal_power` tables on the flops' CLK pins give the energy each flop burns per clock cycle; `cell_leakage_power` gives leakage.

```

# power.py - first-order power estimate for mac_int8 (pre-layout, labelled assumptions)
...
reg = [tg[s] for s in ("p_q", "acc_q", "out_acc", "p_valid", "p_last", "out_valid")]
alpha_reg = sum(n for n, w in reg) / sum(w for n, w in reg) / cycles / 2 # 0->1 per cycle
alpha_cmb = tg["sum"][0] / tg["sum"][1] / cycles / 2
...
wire = lambda k: wlc * (fl[min(k, 6)] + max(k - 6, 0) * 8.3631) if k else 0.0
c_clk = loads[clk] + wire(fo[clk])
c_reg = sum(loads[b] + wire(fo[b]) for b in drv if drv[b])
c_cmb = sum(loads[b] + wire(fo[b]) for b in drv if not drv[b])
p_clk = c_clk * v * v * f # alpha = 1 for a clock
p_reg = alpha_reg * c_reg * v * v * f
p_cmb = alpha_cmb * c_cmb * v * v * f
...
p_int = e_int * f # pJ * GHz = mW

```

Excerpts of power.py: activity from the VCD,  $C V^2 f$  alpha switching power per net class, flop CLK internal energy.

```

simulated cycles: 9892
in_a    toggles/bit/cycle = 0.351
p_q     toggles/bit/cycle = 0.344
acc_q   toggles/bit/cycle = 0.224
sum     toggles/bit/cycle = 0.299
out_acc toggles/bit/cycle = 0.010
alpha (0->1 per cycle): registers 0.087 combinational (sum bus) 0.149
tt_025C_1v80 @ 50 MHz: C_clk 0.128 pF, C_reg 0.483 pF, C_comb 3.493 pF
    clock-net 0.0208 mW flop-CLK internal 0.1351 mW reg nets 0.0068 mW comb nets 0.0846 mW
    leakage 2.21 nW total (excl. comb internal power) 0.2473 mW
tt_100C_1v80 @ 50 MHz: C_clk 0.132 pF, C_reg 0.494 pF, C_comb 3.567 pF
    clock-net 0.0214 mW flop-CLK internal 0.1372 mW reg nets 0.0070 mW comb nets 0.0864 mW
    leakage 684.05 nW total (excl. comb internal power) 0.2526 mW

```

```

ss_100C_1v60 @ 50 MHz: C_clk 0.121 pF, C_reg 0.481 pF, C_comb 3.454 pF
  clock-net 0.0155 mW  flop-CLK internal 0.1036 mW  reg nets 0.0054 mW  comb nets 0.0661 mW
  leakage 3173.22 nW  total (excl. comb internal power) 0.1937 mW
ff_n40C_1v95 @ 50 MHz: C_clk 0.134 pF, C_reg 0.488 pF, C_comb 3.536 pF
  clock-net 0.0255 mW  flop-CLK internal 0.1578 mW  reg nets 0.0081 mW  comb nets 0.1005 mW
  leakage 3.80 nW  total (excl. comb internal power) 0.2918 mW

```

Real output at 50 MHz. Assumptions: activity from the RTL VCD (no glitches), wire-load-model wires, no clock tree (ideal clock), combinational-cell internal power omitted.

- **The clock dominates:** clock-net switching plus flop clock-pin internal power is 0.156 of 0.247 mW at  $t_t$  - 63 % - for a block whose data toggles only 9-15 % of the time. Each flop burns about 40 fJ per clock edge pair just to exist. Clock gating the accumulator when `in_valid` is low (Chapter 11) is the first optimisation; with `dlclkp` integrated clock-gating cells it remains scan-friendly.
- **Energy per MAC:** 0.247 mW at 50 MHz with 75 % input utilisation (7430 beats in 9892 cycles) is about  $0.247\text{e-}3 / (50\text{e}6 \times 0.751) = \mathbf{6.6 \text{ pJ per int8 MAC}}$  at 130 nm and 1.8 V - before clock tree and glitch power. Modern 5-7 nm accelerators report well under 1 pJ per int8 MAC; scaling (Chapter 2) and aggressive data reuse are why.
- **Leakage is negligible at 25 deg C** (2.2 nW) and grows about 300x at 100 deg C (0.68 uW at  $t_t$ \_100C). The SKY130 data gives an even higher value at ss\_100C\_1v60 (3.2 uW) than at  $t_t$ \_100C - counter-intuitive, since slow corners usually leak less; treat open-PDK leakage numbers as approximate and always check leakage at the hottest corner your spec allows.
- A real sign-off power run (OpenSTA `report_power` with a SAIF/VCD from gate-level simulation, or PrimePower/Voltus) includes the clock tree, glitches and internal power of every cell; expect it to be noticeably higher than this estimate.

## 25.10 Step 10 - Place, CTS and route (commands, not run)

The next steps need OpenROAD. With ORFS the whole back end is one `make`; the design configuration is a few lines:

```

export DESIGN_NAME      = mac_int8
export PLATFORM          = sky130hd
export VERILOG_FILES     = $(DESIGN_HOME)/src/mac_int8/mac_int8.sv
export SDC_FILE          = $(DESIGN_HOME)/$(PLATFORM)/mac_int8/constraint.sdc
export CORE_UTILIZATION = 50
export CORE_ASPECT_RATIO = 1
export CORE_MARGIN       = 10
export PLACE_DENSITY     = 0.60

```

designs/sky130hd/mac\_int8/config.mk for ORFS (not run here). The SDC from Step 6 goes in constraint.sdc.

Doing the same steps by hand in the OpenROAD shell is the best way to learn what the flow does. The commands below are the real OpenROAD commands in their usual order; option details differ between versions, and the flows add many checks around them.

```

# ---- set-up -----
read_lef sky130_fd_sc_hd.tlef ; read_lef sky130_fd_sc_hd_merged.lef
read_liberty sky130_fd_sc_hd_tt_025C_1v80.lib
read_verilog mac_net.v ; link_design mac_int8 ; read_sdc mac_int8.sdc
# ---- floorplan and power -----
initialize_floorplan -utilization 50 -aspect_ratio 1 -core_space 10 -site unithd
source sky130hd.tracks ;# make_tracks per routing layer
place_pins -hor_layers met3 -ver_layers met2
tapcell -distance 14 -tapcell_master sky130_fd_sc_hd__tapvpwrvgn1
source pdn.tcl ; pdngen
# ---- placement -----
global_placement -density 0.60 -timing_driven
repair_design ;# max slew/cap/fanout: buffers rst_n etc.
detailed_placement ; check_placement
# ---- clock tree -----
clock_tree_synthesis -root_buf sky130_fd_sc_hd__clkbuf_16 \

```

```

        -buf_list {sky130_fd_sc_hd__clkbuf_4 sky130_fd_sc_hd__clkbuf_8}
set_propagated_clock [all_clocks]
repair_timing -setup ; repair_timing -hold
detailed_placement
# ---- routing -----
global_route -congestion_report_file congestion.rpt
repair_antennas ; detailed_route -output_drc route_drc.rpt
filler_placement {sky130_fd_sc_hd__fill_*} ;# decap/fill in the gaps
# ---- extraction and timing -----
extract_parasitics -ext_model_file rcx_patterns.rules ; write_spef mac_int8.spef
report_checks -path_delay min_max ; report_wns ; report_tns ; report_power
write_def mac_int8.def ; write_db mac_int8.odb

```

The back end of mac\_int8 as OpenROAD Tcl commands (illustrative sequence, not run here; see the ORFS scripts for the production version).

| After...           | Check                                                                                                  | Good result for this block                                                                                      |
|--------------------|--------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Floorplan          | Core area, rows, pin placement, PDN connectivity ( <a href="#">check_power_grid</a> or the PDN report) | 39 rows, all pins on legal tracks, no unconnected rails                                                         |
| Global placement   | Overflow, density map, timing estimate ( <a href="#">report_wns</a> with placement parasitics)         | Overflow < 10 %; WNS close to the Step 7 estimate                                                               |
| repair_design      | Max slew/cap/fanout violations, buffers added                                                          | 0 violations; <a href="#">rst_n</a> split into a small buffer tree                                              |
| Detailed placement | <a href="#">check_placement</a> , displacement                                                         | Legal, small average displacement                                                                               |
| CTS                | Skew, insertion latency, clock buffers, hold after <a href="#">set_propagated_clock</a>                | Skew well under 0.1 ns for 67 sinks; hold fixed by <a href="#">repair_timing</a>                                |
| Global route       | GCell overflow, congestion report                                                                      | 0 overflow                                                                                                      |
| Detailed route     | DRC count in <a href="#">route_drc.rpt</a> , antenna                                                   | 0 DRCs, 0 antenna violations                                                                                    |
| Extraction + STA   | WNS/TNS setup at ss, hold at ff, max slew/cap, per corner with SPEF                                    | Setup slack > 0 at ss_100C_1v60 (wires and the clock tree will eat part of the 1.5 ns estimate), hold > 0 at ff |

## 25.11 Step 11 - Sign-off and the GDSII hand-off (checklist, not run)

Sign-off (Chapter 18) is where the open flows call the PDK's layout tools. OpenLane 2 / LibreLane runs all of these automatically; in ORFS some are separate targets. What each check proves:

| Check                | Tool (open flow)                                                   | Pass criterion                                                        |
|----------------------|--------------------------------------------------------------------|-----------------------------------------------------------------------|
| GDS stream-out       | Magic and KLayout, merging the standard-cell GDS from the PDK      | Both produce a GDS; <b>XOR</b> of the two is empty                    |
| DRC                  | Magic ( <a href="#">drc check</a> ) and KLayout decks from the PDK | 0 violations (density rules may need fill)                            |
| LVS                  | Magic extraction + Netgen <a href="#">lvs</a> against the netlist  | 'Circuits match uniquely'; no pin or property mismatches              |
| Antenna              | OpenROAD <a href="#">check_antennas</a> , Magic                    | 0 violations after diode insertion                                    |
| Multi-corner STA     | OpenSTA with SPEF per corner                                       | Setup and hold met at all sign-off corners; max slew/cap/fanout clean |
| IR drop              | PDNSim <a href="#">analyze_power_grid</a>                          | Worst static drop a few percent of VDD at most                        |
| Timing-annotated GLS | Icarus/Verilator with SDF (or commercial)                          | Reset and a few vectors pass with back-annotated delays               |
| Equivalence          | Yosys <a href="#">equiv_*</a> / EQY between RTL and final netlist  | Proven equivalent (catches ECO and optimisation errors)               |

## mac\_int8 tapeout checklist

- ☐ Spec frozen: interfaces, overflow rule ( $\leq 511$  terms), corners, clock, area and power targets signed off by the architect
- ☐ RTL reviewed; lint clean with -Wall and no unexplained waivers
- ☐ Regression passes with random back-pressure; coverage of first/last beat, single-element vector, 511-term maximum, both stall types
- ☐ Synthesis log reviewed: no latches, no unmapped cells, no dont-use cells, tie cells for constants
- ☐ Formal equivalence RTL vs final netlist proven
- ☐ SDC reviewed: clock, I/O budget agreed with the integrator, reset recovery/removal timed, no unintended false paths
- ☐ Setup met at ss\_100C\_1v60 and hold at ff\_n40C\_1v95 with extracted parasitics and propagated clock
- ☐ Max transition / capacitance / fanout clean at all corners
- ☐ DRC, LVS, antenna clean; Magic/KLayout XOR empty
- ☐ IR drop and EM within limits; power estimate updated with the routed netlist and gate-level activity
- ☐ Scan inserted (or waiver agreed with the DFT owner) and test coverage reported
- ☐ Deliverables archived with tool and PDK versions: GDS, LEF abstract, Liberty model of the block, netlist, SPEF, SDC, reports, run configuration and git hash
- ☐ Shuttle/foundry precheck passed (names, pins, layers, GDS size)

## 25.12 What the case study teaches

| Lesson                                                         | Where it showed up                                           |
|----------------------------------------------------------------|--------------------------------------------------------------|
| Write the overflow rule and the corners into the spec first    | Step 1: 511 terms, ss_100C_1v60 setup / ff hold              |
| Only self-checking random tests catch data-dependent bugs      | Step 3: the missing <b>signed</b> was lint-clean             |
| Area is mostly logic, but flops and resets are not free        | Step 5: 30 % flops; 120 $\mu\text{m}^2$ of unnecessary reset |
| Architecture sets the critical path; synthesis effort does not | Steps 5 and 7: ripple carry, ABC -D barely helps             |
| Corners change the answer by 2-6x                              | Step 7: +0.83 ns slack at tt vs -8.5 ns at ss for 100 MHz    |
| Clock power dominates lightly active blocks                    | Step 9: 63 % of the estimate is clock                        |
| Floorplan numbers are arithmetic, not magic                    | Step 8: rows, sites and die size from three inputs           |

## 25.13 Summary

- A 644-cell, 5365  $\mu\text{m}^2$  int8 MAC with valid/ready interfaces was specified, written, simulated (203 vectors), linted, synthesized on SKY130 hd and re-simulated at gate level - all for real.
- Multi-corner NLDM analysis on the real Liberty files shows 100 MHz at typical but only 50 MHz at the ss\_100C\_1v60 sign-off corner, limited by a 24-bit ripple-carry accumulator.
- At 50 % utilisation the block fits in a 121 x 126  $\mu\text{m}$  die with 39 rows of 220 sites; the pre-layout power estimate is about 0.25 mW at 50 MHz, two-thirds of it in the clock.
- Placement, CTS, routing, extraction and sign-off are driven by ORFS or LibreLane with the configuration and commands shown; each step has a concrete check before moving on.
- The tapeout checklist is the hand-off contract: frozen spec, clean static checks, timing at the right corners, clean DRC/LVS, and an archive that can reproduce the GDS.

## 25.14 Exercises

- 1 Remove the reset from `out_acc`, re-run synthesis and GLS, and confirm the 120  $\mu\text{m}^2$  saving. Does any X appear in simulation? Why not?
- 2 Replace the accumulator with a carry-save form (keep sum and carry vectors, resolve on `in_last`). Estimate the new critical path and the area cost.
- 3 Change `ACC_W` to 20. What is the new maximum vector length? Add a sticky `overflow` output and a test that provokes it.
- 4 Using the floorplan script, find the utilisation at which the die is exactly 0.0125  $\text{mm}^2$ , and argue whether you would accept it.
- 5 Install ORFS or LibreLane, run the block through with the SDC of Step 6, and compare post-route WNS at `ss_100C_1v60` with the 1.52 ns pre-layout estimate. Explain the difference.
- 6 Add clock gating for the accumulator using an integrated clock-gating cell and recompute the power estimate.

## 26. The ASIC Career Roadmap and Interview Preparation

An ASIC is built by a dozen specialisms that rarely sit in the same room: architects, RTL designers, verification engineers, DFT, synthesis and timing, physical design, physical verification, CAD, analog, test and post-silicon. This book walked through all of them in the order the chip needs them. This chapter turns the same map into a career: what each role does all day, what it takes to be hired into it, how people grow from a first job to principal engineer, a 12-month study plan that uses this book and its three companions, portfolio projects that prove the skills, and how interviews for each role are actually run.

Titles, levels and salaries differ between companies and countries; the **scope** of each stage - a task, a block, a subsystem, a chip, a product line - is remarkably consistent, and that is what this chapter uses.

### 26.1 The roles on an ASIC team

| Role                               | What you own                                                                             | Main deliverables                                                 | Chapters         |
|------------------------------------|------------------------------------------------------------------------------------------|-------------------------------------------------------------------|------------------|
| <b>Architecture</b>                | Performance, power and area targets; block partitioning; memory and interconnect choices | Architecture spec, performance model, PPA budget                  | 1, 5, 23         |
| <b>RTL design</b>                  | Micro-architecture and RTL of blocks or subsystems                                       | Micro-arch spec, lint/CDC-clean RTL, SDC and UPF inputs           | 6, 11; RTL guide |
| <b>Design verification (DV)</b>    | Proving the RTL does what the spec says                                                  | Test plan, UVM/cocotb environment, coverage closure, sign-off     | 7; SV guide      |
| <b>DFT</b>                         | Making the chip testable in production                                                   | Scan, compression, MBIST, JTAG/IEEE 1500, ATPG patterns, coverage | 10, 21           |
| <b>Synthesis / STA</b>             | Turning RTL into a netlist that meets timing; timing sign-off                            | Synthesis scripts, constraints, timing reports, ECO lists         | 8, 9, 18         |
| <b>Physical design (PD)</b>        | Floorplan to routed database                                                             | Floorplan, PDN, placed/CTS/routed DB, closure at all corners      | 13-17, 19        |
| <b>Physical verification (PV)</b>  | DRC, LVS, ERC, antenna, DFM sign-off                                                     | Clean decks, waivers, tapeout GDS                                 | 3, 18, 19        |
| <b>CAD / methodology</b>           | Flows, scripts, tool qualification, compute and licence efficiency                       | Reference flows, regression infrastructure, QoR dashboards        | 13, 23, 24       |
| <b>Analog / mixed-signal (AMS)</b> | PLLs, ADCs, SerDes, regulators, I/O                                                      | Schematics, layout, characterisation, behavioural models          | 2, 12            |
| <b>Test / product engineering</b>  | Production test programs, yield, binning, cost                                           | ATE programs, yield reports, test-time reduction                  | 21, 22           |
| <b>Post-silicon validation</b>     | Bring-up and validation of real chips                                                    | Bring-up plan, characterisation data, errata                      | 22               |

The ASIC roles, what each owns and where this book covers it. 'RTL guide', 'SV guide' and 'Protocols guide' are the companion books.

#### PICK A HOME, KEEP A MAP

Nearly everyone specialises - a PD engineer does not write UVM and a DV engineer does not close hold at ff. But the best engineers in every role understand the stage before and after theirs: the RTL designer who can read a timing report, the PD engineer who can read RTL, the DV engineer who knows what DFT did to the netlist. That breadth is what this book is for.

## 26.2 Skills tables by role

### Front-end roles

| Skill                          | Architecture    | RTL design | DV                     | DFT               |
|--------------------------------|-----------------|------------|------------------------|-------------------|
| Digital logic, CMOS basics     | Strong          | Strong     | Good                   | Strong            |
| Verilog / SystemVerilog design | Read            | Expert     | Good                   | Good (netlists)   |
| SV verification, UVM, SVA      | -               | Good (SVA) | Expert                 | Basic             |
| C/C++ and Python               | Expert (models) | Good       | Strong                 | Good              |
| Performance modelling          | Expert          | Good       | -                      | -                 |
| Bus protocols (AXI, PCIe...)   | Strong          | Strong     | Strong                 | Basic             |
| CDC / RDC, lint                | Aware           | Expert     | Good                   | Good              |
| STA and SDC                    | Aware           | Good       | Basic                  | Good (test modes) |
| Low power / UPF                | Strong          | Good       | Good (power-aware sim) | Good              |
| Scan, ATPG, MBIST, JTAG        | Aware           | Aware      | Basic                  | Expert            |
| Tcl scripting                  | -               | Good       | Good                   | Strong            |

### Back-end and silicon roles

| Skill                        | Synth/STA | PD     | PV     | CAD    | AMS    | Test/Post-Si |
|------------------------------|-----------|--------|--------|--------|--------|--------------|
| Device physics, process      | Good      | Good   | Strong | Basic  | Expert | Strong       |
| Liberty, LEF/DEF, SPEF, SDF  | Expert    | Expert | Good   | Expert | Good   | Basic        |
| SDC and STA                  | Expert    | Expert | Basic  | Strong | Basic  | Good         |
| Floorplan, PDN, CTS, routing | Good      | Expert | Good   | Strong | Basic  | -            |
| DRC/LVS decks, DFM           | -         | Good   | Expert | Good   | Strong | -            |
| IR drop, EM, SI              | Good      | Strong | Good   | Good   | Good   | -            |
| Tcl and Python               | Expert    | Expert | Strong | Expert | Good   | Strong       |
| SPICE, schematics            | Basic     | Basic  | Good   | Basic  | Expert | Good         |
| ATE, lab equipment, stats    | -         | -      | -      | -      | Good   | Expert       |

#### THE SKILL EVERY TABLE LEAVES OUT

Scripting. Every role above spends a large part of its time writing Tcl and Python to drive tools, parse reports and automate checks. Being the person who turns a two-day manual review into a ten-minute script is the fastest way to be noticed in your first year - and it is exactly what the Python models of this book practise.

## 26.3 Where ASIC engineers work

| Employer type                                 | Examples of the work                                              | What it means for you                                                               |
|-----------------------------------------------|-------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| <b>Fabless semiconductor companies</b>        | Design chips and sell them; manufacturing outsourced to a foundry | Every role from architecture to post-silicon; the classic ASIC career               |
| <b>Integrated device manufacturers (IDMs)</b> | Design and manufacture (own fabs)                                 | Closer to process, device and test engineering; strong in analog, power, automotive |



| Employer type                            | Examples of the work                                                                 | What it means for you                                                                 |
|------------------------------------------|--------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| <b>System companies and hyperscalers</b> | Custom silicon for their own products (phones, servers, AI accelerators, networking) | Architecture tightly coupled to software; large teams, often with outsourced back end |
| <b>Foundries</b>                         | Process development, PDKs, libraries, design enablement                              | PDK, library, DRC/LVS deck and reference-flow work (Chapters 3-4)                     |
| <b>EDA vendors</b>                       | Build the tools: synthesis, P&R, STA, verification, PV                               | CAD, algorithms and application-engineer (AE) roles supporting customers              |
| <b>IP vendors</b>                        | Processors, interfaces (PCIe, DDR, USB), memories, PHYs                              | Deep specialisation; the Protocols guide is directly relevant                         |
| <b>Design-services companies</b>         | Execute parts of other companies' chips (PD, DV, DFT)                                | Many projects quickly; excellent for breadth early in a career                        |
| <b>Start-ups</b>                         | First chip of a new architecture                                                     | Wide scope and ownership; schedule and funding risk                                   |

## 26.4 A staged roadmap: student to principal

| Stage                                        | Scope                                                                 | Expected skills                                                                                                     | Ready for the next stage when                                                                            |
|----------------------------------------------|-----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| <b>Student / beginner</b>                    | Coursework and personal projects                                      | Digital logic, one HDL, simulation, Linux, git, Python; one complete project through synthesis                      | You can take a small spec to verified RTL and a synthesized netlist with a timing report you can explain |
| <b>Junior engineer</b><br>(about 0-3 years)  | Tasks and small blocks, reviewed by seniors                           | The tools and methodology of one role; reading reports; debugging your own failures; team conventions               | You deliver blocks or partitions with few review findings and close your own lint/timing/DRC issues      |
| <b>Senior engineer</b><br>(about 3-8 years)  | A whole IP, partition or verification environment, through tapeout    | Owning trade-offs, estimating schedules, interfacing with adjacent teams, mentoring, silicon debug                  | Others come to you for reviews; you have taken something through tapeout and bring-up                    |
| <b>Staff / lead</b> (about 8+ years)         | A subsystem, a chip's implementation or verification strategy, a team | Methodology decisions, cross-team planning, risk management, hiring                                                 | Your decisions shape more than one project                                                               |
| <b>Principal / architect / distinguished</b> | Product lines, company technology direction                           | Architecture, technology roadmaps, customer and foundry relationships, deep expertise that the company is known for | -                                                                                                        |

Years are indicative only; scope, independence and judgement define the stage.

### WHAT CHANGES AT EACH STEP

Junior -> senior is mostly **technical depth plus ownership**: you stop asking what to check and start deciding what matters. Senior -> staff is mostly **influence**: writing the methodology others follow, spotting schedule risk early, making other engineers productive. Staff -> principal is mostly **judgement under uncertainty**: choosing a node, an architecture or a vendor with incomplete data and being right more often than not.

## 26.5 A 12-month study plan

The plan assumes about 10 hours a week alongside a degree or a job. Each month pairs reading with a hands-on deliverable; the deliverables add up to the portfolio of the next section. 'RTL' is the companion *RTL Design for SoC & ASIC*, 'SV' the *Verilog & SystemVerilog for SoC & ASIC* guide and 'Proto' the *Communication Protocols for SoC & ASIC* guide.

| Month | Read (this book)                               | Read (companions)                          | Build / practise                                                                                                           |
|-------|------------------------------------------------|--------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| 1     | Ch. 1-2: landscape, devices                    | RTL Ch. 1-4; SV Ch. 1-6                    | Install Icarus, Verilator, Yosys; write and simulate a counter, an FSM and a FIFO                                          |
| 2     | Ch. 3-4: process, cells, Liberty               | RTL Ch. 5-10; SV Ch. 7-11                  | Re-run the Chapter 24 Liberty and LEF scripts; synthesize your FIFO on SKY130 and explain every cell                       |
| 3     | Ch. 5-6: spec, RTL for ASIC                    | RTL Ch. 11-16; SV Ch. 12-17; Proto Ch. 1-8 | Write a spec and RTL for an APB peripheral (UART or timer) with CDC-safe reset                                             |
| 4     | Ch. 7: verification                            | SV Ch. 18-26; RTL Ch. 22-26                | A self-checking, constrained-random testbench with assertions and coverage for month 3's block (UVM or cocotb)             |
| 5     | Ch. 8-9: synthesis, STA                        | RTL Ch. 17-18, 21                          | Constraints for your block; sweep clock period and ABC -D; write a small STA script like Chapter 25's and compare corners  |
| 6     | Ch. 10-11: DFT, low power                      | RTL Ch. 19-20                              | Insert scan in your netlist (by hand or a tool), compute the area cost; add clock gating and measure power change from VCD |
| 7     | Ch. 12-14: AMS/IO, PD setup, floorplan and PDN | Proto Ch. 23-24                            | Install ORFS or LibreLane; run the reference design; floorplan your block by hand, then with the flow                      |
| 8     | Ch. 15-17: placement, CTS, routing             | Proto Ch. 9-16                             | Take your block to a routed GDS; vary density and clock period; record WNS, skew, DRC count                                |
| 9     | Ch. 18-19: sign-off, ECO, tapeout              | Proto Ch. 25-27                            | DRC/LVS with Magic, KLayout and Netgen; fix one real violation; do a functional ECO by hand and re-verify                  |
| 10    | Ch. 20-22: packaging, test, bring-up           | Proto Ch. 17-22                            | Submit a Tiny Tapeout design (or prepare one to shuttle-precheck quality); write its bring-up plan                         |
| 11    | Ch. 23: methodology, economics                 | RTL Ch. 27-30; SV Ch. 27-28                | Pick a specialisation; do a deep project in it (see portfolio) and one open-source contribution                            |
| 12    | Ch. 24-26; Appendices A-D                      | Protocol and RTL appendices                | Mock interviews with Appendix C questions; polish portfolio, write-ups and resume                                          |

A 12-month plan mapped to this book and its three companion books. Compress it if you already work in one area.

#### PITFALL - READING WITHOUT BUILDING

Twelve months of reading produces an engineer who can recite the definitions of skew and recovery time. Interviewers hire the one who has **closed** a hold violation and can show the report. Every month above ends in something you ran; keep the logs.

## 26.6 Portfolio projects that get a resume read

| Project                                                                                                                        | Demonstrates                                               | Target roles                     | Effort    |
|--------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|----------------------------------|-----------|
| <b>Tiny Tapeout design</b> (for example a small RISC-V ALU, a PWM/servo controller, a CORDIC, a tiny int8 MAC array)           | The whole flow to real silicon; shuttle precheck; bring-up | All digital roles, PD especially | 4-8 weeks |
| <b>Chapter 25 extended:</b> carry-save MAC, clock gating, scan, routed with ORFS; a write-up comparing estimate and post-route | PPA reasoning, timing closure, power analysis              | RTL, synthesis/STA, PD           | 3-6 weeks |
| <b>UVM or cocotb environment</b> for an open IP (a UART, an AXI-Lite slave) with coverage closure and bugs found               | Verification methodology, coverage-driven thinking         | DV                               | 4-6 weeks |

| Project                                                                                                                      | Demonstrates                                             | Target roles      | Effort         |
|------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|-------------------|----------------|
| <b>FPGA project</b> (camera pipeline, audio DSP, soft-core SoC running software)                                             | Real clocks, I/O, CDC, debug on hardware                 | RTL, post-silicon | 4-10 weeks     |
| <b>Open-source contribution</b> (a Yosys pass fix, an OpenROAD issue with a test case, a PDK DRC deck fix, cocotb extension) | Working in a large code base, code review, communication | CAD, PD, DV       | Ongoing        |
| <b>Analysis scripts</b> (Liberty explorer, timing-report parser, IR-drop model, yield model)                                 | Automation, data skills                                  | CAD, STA, test    | 1-3 weeks each |
| <b>Analog block on an open PDK</b> (bandgap, ring oscillator, comparator) with xschem + ngspice + Magic, LVS clean           | Full-custom flow                                         | AMS, PV           | 6-12 weeks     |

### HOW TO PRESENT A PROJECT

A public repository with a README that states the spec, a block diagram, how to run it in one command, the results (area, timing at named corners, coverage, power) and **what went wrong and how you fixed it**. Reviewers read the last section first.

## 26.7 How ASIC interviews are structured

Most companies run a similar pipeline: a recruiter screen, one or two technical phone/video screens, then an on-site or virtual loop of four to six interviews (mostly technical, one or two on behaviour and project depth), and a debrief where the panel decides level as well as yes/no. Expect whiteboard (or shared-document) problem solving rather than trivia, and expect every answer to be followed by 'why?' and 'what if?'.

| Role                         | Typical technical rounds                                                                   | Sample topics                                                                                                                                                                             |
|------------------------------|--------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>RTL design</b>            | Digital logic; RTL coding live; micro-architecture design; CDC; resume deep-dive           | Design a synchronous/asynchronous FIFO; valid/ready pipeline with back-pressure; clock-domain crossing of a multi-bit value; arbiter; count ones in a word; pipelining a MAC (Chapter 25) |
| <b>DV</b>                    | SystemVerilog/OOP; testbench architecture; constrained random and coverage; debugging; SVA | Build a scoreboard for a FIFO; write constraints for a packet; coverage plan for an arbiter; race conditions; UVM phases and sequences; assertion for a handshake                         |
| <b>DFT</b>                   | Scan and ATPG theory; test modes; MBIST; JTAG                                              | Stuck-at vs transition faults; scan chain shift/capture; compression ratio; at-speed test with launch-on-capture; test-mode timing constraints                                            |
| <b>Synthesis / STA</b>       | Timing theory; SDC; report reading; ECOs                                                   | Setup/hold equations with skew; multicycle and false paths; OCV/CRPR; fix a hold violation without breaking setup; generated clocks; why it passes and ss fails                           |
| <b>Physical design</b>       | Flow and each stage; floorplan and PDN reasoning; CTS; congestion and timing closure       | Size a floorplan from cell area; macro placement guidelines; IR-drop fixes; useful skew; routing congestion causes; antenna fixes; hold fixing after CTS                                  |
| <b>Physical verification</b> | DRC/LVS concepts; debugging; DFM; decks                                                    | LVS short/open debug; soft connections; latch-up rules; density and fill; antenna ratio                                                                                                   |
| <b>CAD / methodology</b>     | Coding (Python/Tcl) live; flow design; tool knowledge                                      | Parse a timing report and summarise by clock group; design a regression system; data structures for a netlist                                                                             |
| <b>AMS</b>                   | Circuit analysis; device physics; layout; specific block design                            | Op-amp stability; PLL loop dynamics; matching and layout; bandgap; ADC architectures; noise                                                                                               |
| <b>Test / post-silicon</b>   | Test theory; statistics; lab debugging; ATE                                                | Yield models; DPPM vs coverage; shmoo interpretation; binning; debugging a failing part                                                                                                   |

**THE FOLLOW-UP QUESTION IS THE REAL QUESTION**

Almost every interviewer starts easy ('what is setup time?') and then pushes until you reach the edge of your knowledge ('...and with a negative-edge capture flop? With a clock-gating cell on the path? At the ff corner?'). The goal is to find **where** your understanding stops and **how you reason** there. Say what you know, state assumptions, reason aloud, and say 'I would check X' rather than guessing. Appendix C collects 100 such questions with answers.

## 26.8 Mini-problems by role, with model answers

Short numerical problems are a staple of phone screens because they separate people who have used a concept from people who have read about it. Four examples, one per major area:

### STA: setup and hold with skew

*Launch and capture flops on the same 2.0 ns clock. Clock-to-Q 0.15 ns (min 0.12 ns), maximum combinational delay 1.40 ns, minimum 0.02 ns, setup 0.10 ns, hold 0.05 ns. The capture clock arrives 0.10 ns later than the launch clock. Setup uncertainty 0.10 ns, hold uncertainty 0.05 ns. Compute both slacks.*

$$\begin{aligned}\text{setup slack} &= (T + \text{skew} - \text{unc}_s) - (t_{\text{cq,max}} + t_{\text{comb,max}} + t_{\text{su}}) \\ &= (2.00 + 0.10 - 0.10) - (0.15 + 1.40 + 0.10) = 2.00 - 1.65 = +0.35 \text{ ns} \\ \\ \text{hold slack} &= (t_{\text{cq,min}} + t_{\text{comb,min}}) - (t_h + \text{skew} + \text{unc}_h) \\ &= (0.12 + 0.02) - (0.05 + 0.10 + 0.05) = 0.14 - 0.20 = -0.06 \text{ ns}\end{aligned}$$

Positive (late-capture) skew helps setup and hurts hold by the same amount (Chapter 9).

The hold violation is independent of the clock period, so slowing the clock cannot fix it. Fix it with about 0.1 ns of delay on **that** data path (a delay cell or a sized-down buffer, checked at the fast corner), or by reducing the skew; then re-check setup, which has 0.35 ns to give.

### Physical design: sizing a core with macros

*A block has 1.2 mm<sup>2</sup> of standard cells and four SRAM macros of 0.2 mm<sup>2</sup> each. Target standard-cell utilisation is 70 % and each macro needs a 10 % halo. What core area do you start with?*

$$\begin{aligned}\text{core} &= \text{std-cell area} / \text{utilisation} + \text{macro area} \times (1 + \text{halo}) \\ &= 1.2 / 0.70 + 4 \times 0.2 \times 1.10 = 1.714 + 0.880 = 2.59 \text{ mm}^2\end{aligned}$$

Then check the macro aspect ratios fit, pins face the logic that uses them, and channels between macros are routable (Chapter 14).

### DV: a weighted constraint

```
class pkt;
  rand bit [6:0] len;           // 1..64 bytes
  constraint c_len {
    len inside {[1:64]};
    len dist { 64 := 10, [1:63] := 90 }; // 10 % maximum-size packets
  }
endclass
```

`:=` gives the weight to each value, `:=` spreads it over the whole range - a favourite interview trap (the SV guide covers dist in detail). Written for this chapter, not run.

### DFT: test time with and without compression

1,000,000 scan flops, 8 scan-in pins, 50 MHz shift, 2000 patterns

No compression: chain length  $1e6 / 8 = 125,000 \rightarrow 2.5 \text{ ms/pattern} \rightarrow 5.0 \text{ s per die}$

100x compression: chain length 1,250  $\rightarrow 25 \text{ us/pattern} \rightarrow 0.05 \text{ s per die}$

Shift dominates test time; compression (Chapter 10) is what makes large SoCs economic to test (Chapter 21).

### A worked interview answer

'Your block meets timing at typical but fails at the slow corner by 8 ns at 100 MHz. What do you do?' A strong answer, based on Chapter 25, goes in layers:

- **Confirm the problem:** is ss\_100C\_1v60 really a sign-off corner for this product (supply range, temperature)? Is the SDC right - I/O budget, uncertainty, no missing multicycle path?
- **Characterise the path:** it is a 24-bit ripple-carry accumulator loop, 14 majority cells deep; the failure is structural, so sizing and buffering will not recover 8 ns.
- **Offer architecture options with costs:** carry-save accumulation (best; adds a resolve step and area), a parallel-prefix adder (log depth, more area and wiring), a two-cycle accumulate (latency), a faster library (leakage and area), or a lower clock (throughput).
- **Pick and justify:** carry-save, because it removes the carry from the recurrence entirely; show the estimated new critical path and area; re-run the corners.
- **Close the loop:** update the spec and the SDC, re-verify, and record the decision.

## 26.9 Soft skills that decide careers

| Skill                               | Why it matters in ASIC work                                                                                             | How to practise                                                                                    |
|-------------------------------------|-------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| <b>Written communication</b>        | Specs, review comments, bug reports and tapeout sign-offs are the permanent record; a misunderstood spec costs a respin | Write a one-page spec and a results summary for every project; have someone else implement from it |
| <b>Estimation and planning</b>      | Tapeout dates are fixed by shuttle and mask schedules; slips cost money (Chapter 23)                                    | Estimate every task before starting; record actuals; learn your bias                               |
| <b>Reviewing and being reviewed</b> | Most bugs are caught by people, not tools                                                                               | Review open-source pull requests; ask for reviews and thank reviewers                              |
| <b>Debugging discipline</b>         | Silicon debug has few observables and no second chance                                                                  | Hypothesis, cheapest falsifying experiment, one change at a time, log everything                   |
| <b>Cross-team empathy</b>           | Every hand-off (RTL $\rightarrow$ PD, PD $\rightarrow$ PV, design $\rightarrow$ test) is a negotiation                  | Spend a week learning the adjacent role's tools; read their reports                                |
| <b>Saying 'I do not know yet'</b>   | Wrong confident answers cause respins                                                                                   | Practise stating assumptions and confidence levels                                                 |

## 26.10 Your first 90 days on an ASIC team

- **Weeks 1-2:** get the flow running end to end on a known-good block; learn where logs, reports and the bug tracker live; read the chip spec and your block's history.
- **Weeks 3-6:** own a small, real task (a lint cleanup, a timing partition, a coverage hole) and close it completely, including the report nobody asked for.
- **Weeks 7-12:** find one repetitive manual step and automate it; present it at a team meeting. Ask to shadow the adjacent role for a day.
- **Throughout:** keep a notebook of every command, every mistake and every answer a senior gave you. It becomes your personal methodology guide - and the source of your first promotion case.

**RESUME LINES THAT WORK**

Quantified and specific: 'Closed timing on a 1.2 GHz CPU partition (WNS -180 ps to 0) by restructuring two paths and adding useful skew' beats 'Responsible for timing closure'. For students: 'Designed and taped out (Tiny Tapeout) an 8-bit MAC; 644 cells, 50 MHz at ss\_100C\_1v60 on SKY130' beats a list of courses.

## 26.11 Final checklist

### Before you apply

- ☐ I can explain the complete ASIC flow from specification to qualification and name the owner and hand-off of each step
- ☐ I have taken at least one design through synthesis with a real library and can explain every cell type and the critical path
- ☐ I have written SDC for a block and can derive setup and hold slack by hand from a timing report
- ☐ I have run (or can describe step by step) floorplan, placement, CTS, routing and sign-off in an open flow, with the checks after each step
- ☐ I have a self-checking testbench with random stimulus and coverage for one of my designs
- ☐ I understand scan, stuck-at and transition faults, and what DFT costs in area and timing
- ☐ I can estimate dynamic and leakage power and explain why the clock dominates many blocks
- ☐ My portfolio has 2-3 projects with READMEs, results at named corners and a 'what went wrong' section
- ☐ I have answered the Appendix C questions for my target role aloud, and reviewed the ones I got wrong
- ☐ I can describe one hard problem I solved, what I tried, and what I would do differently

**THE LAST WORD**

Chips are built by teams under immovable deadlines with no undo button. The engineers who thrive are the ones who **check before they trust**: they run the corner, read the report, simulate the negative operand and ask the adjacent team. Everything in this book - every run, every checklist - is practice for that habit.

## 26.12 Summary

- An ASIC team spans architecture, RTL, DV, DFT, synthesis/STA, PD, PV, CAD, AMS, test and post-silicon; each owns specific deliverables and hands off to the next.
- Careers progress by scope and judgement: task, block, subsystem, chip, product line.
- A 12-month plan pairs this book's chapters and the three companion books with a monthly hands-on deliverable, ending in a portfolio.
- Tiny Tapeout designs, open-source contributions, FPGA projects and extended versions of Chapter 25 make a portfolio that proves skills rather than listing them.
- Interviews probe the edge of your knowledge with follow-up questions; reasoning aloud from first principles and knowing what to check matter more than recall.
- Communication, estimation, review and debugging discipline decide careers as much as technical depth.

## 26.13 Exercises

- 1 Choose a target role. Fill in its skills table row by row with your own honest level and list the three biggest gaps.
- 2 Turn the 12-month plan into a dated plan for your situation, with one measurable deliverable per month.
- 3 Write the README for a portfolio project you have done (or will do) using the structure of this chapter: spec, how to run, results at named corners, what went wrong.
- 4 Pick five Appendix C questions for your role, answer each aloud in under three minutes, then write down the follow-up question an interviewer would ask and answer that too.

- 5 Find an open issue in Yosys, OpenROAD, cocotb or an open PDK repository, reproduce it, and write up what you found - even if you do not fix it.

## APPENDICES

# Appendices

Sign-off checklists and a file-formats reference with real SKY130 and tool excerpts, a glossary of about 190 terms, 100 interview questions with worked answers, and the standards, tools and resources of the ASIC world.



## Appendix A. Sign-off Checklists and File Formats Reference

Part one condenses the sign-off gates of Chapters 5 to 19 into checklists you can copy into a project tracker. Part two is a field guide to the files that move between tools and teams, with annotated excerpts. Green cards are real: real SKY130 library files and real output from Yosys, Icarus Verilog, KLayout and small Python tools; grey cards are hand-written examples, and are labelled as such.

### How to use the sign-off checklists

Every gate below is a **hand-off**: one team stops changing an input and another team starts depending on it. A checklist item is only "done" when there is evidence - a clean report, a waiver with a named approver and a reason, or a review record. Waivers are part of the deliverable; an unexplained waiver is a failed item. Adapt the numbers (coverage targets, IR-drop budget, utilisation) to your node, foundry and product - the values shown are typical starting points, not rules.

| Gate                         | Typical owner                    | Main deliverables handed on                                                 | Chapter |
|------------------------------|----------------------------------|-----------------------------------------------------------------------------|---------|
| <b>Spec freeze</b>           | Architecture / product           | Frozen spec, PPA targets, IP list, pin list, power budget                   | 5, 23   |
| <b>RTL freeze</b>            | Design lead                      | Tagged RTL, lint/CDC reports, coverage report, SDC draft, UPF               | 6, 7    |
| <b>Synthesis hand-off</b>    | Synthesis / implementation       | Netlist, SDC, UPF, reports, LEC result                                      | 8       |
| <b>DFT sign-off</b>          | DFT                              | Scan-inserted netlist, test protocol, ATPG patterns (STIL), coverage report | 10      |
| <b>Floorplan review</b>      | Physical design                  | Floorplan DEF, pin placement, power grid, macro placement                   | 13, 14  |
| <b>Pre-CTS</b>               | Physical design                  | Placed DEF, timing/congestion reports, CTS spec                             | 15, 16  |
| <b>Post-route</b>            | Physical design                  | Routed DEF, SPEF, DRC/timing snapshots                                      | 17      |
| <b>Timing sign-off</b>       | STA                              | MCMM STA reports, SDF, waivers                                              | 9, 18   |
| <b>Power/IR/EM sign-off</b>  | Power integrity                  | Static and dynamic IR maps, EM report, power numbers                        | 11, 18  |
| <b>Physical verification</b> | PV                               | DRC/LVS/ERC/antenna/density clean reports                                   | 18      |
| <b>LEC</b>                   | Synthesis / PD                   | RTL-vs-netlist and netlist-vs-netlist equivalence results                   | 8, 19   |
| <b>Tapeout</b>               | Project lead + foundry interface | Final GDSII/OASIS, tapeout form, sign-off package                           | 19      |

### Front-end gates

#### Spec freeze

- ☐ Functional spec reviewed and signed by architecture, design, verification, software and product owners; open issues listed with owners and dates.
- ☐ PPA targets written down: frequency per clock and mode, power per use-case (dynamic, leakage, standby), area/die-size budget, package and pin count.
- ☐ Process node, foundry, metal stack, standard-cell library (track height, Vt flavours) and PVT sign-off corners agreed.
- ☐ IP list complete (memories, PLLs, SerDes, PHYs, I/O library, analog blocks) with vendor, version, deliverables and licence status.
- ☐ Clock and reset architecture, power domains and operating modes defined (feeds SDC and UPF).

- ☐ DFT strategy agreed: scan style, compression ratio, MBIST, JTAG/IJTAG, target coverage and test-time budget.
- ☐ Safety/security requirements (ISO 26262 ASIL, secure boot, fuse/OTP) and qualification target (JEDEC, AEC-Q100 grade) stated.
- ☐ Schedule and tapeout date agreed with the foundry shuttle or production slot; change-control process defined.

### RTL freeze

- ☐ All features implemented; no open P1/P2 bugs; bug trend flat.
- ☐ Code and functional coverage at target (for example line/branch near 100 % with justified exclusions, all coverpoints hit), assertions passing, regression green on the frozen tag.
- ☐ Lint clean (or waived) with the agreed rule set; no latches, combinational loops or multi-driven nets unless intended.
- ☐ CDC and RDC analysis clean: every crossing uses an approved synchroniser; reset-domain crossings reviewed.
- ☐ Low-power checks: UPF written, simulated with power-aware simulation; isolation, retention and level shifting verified.
- ☐ SDC draft reviewed against the clock spec; trial synthesis meets timing within an agreed margin.
- ☐ RTL tagged in revision control; file list and include paths frozen; no simulation-only constructs in synthesisable files.
- ☐ Memories and IP instantiated with final configurations; behavioural models match vendor views.

### Synthesis hand-off

- ☐ Correct library set per corner (.lib/.db), including memories and IP; `dont_use` list applied (e.g. weak, special or low-power-only cells).
- ☐ Synthesis log reviewed: no unresolved references, no black boxes except intended macros, no inferred latches, no assign-to-constant surprises.
- ☐ `check_design` / `check_timing` clean: every endpoint constrained, no unlocked flops, no combinational loops.
- ☐ Timing met with the agreed pre-layout margin (setup WNS, TNS); area and leakage within budget; clock gating inserted as intended.
- ☐ UPF applied: isolation, level shifters, retention flops inserted and checked (static low-power verification).
- ☐ Netlist naming readable; hierarchies to keep are preserved (ungroup policy documented).
- ☐ Formal equivalence (RTL vs netlist) passed with no aborted points.
- ☐ Hand-off package: netlist, SDC, UPF, SVF/guide file for LEC, area/timing/power reports, scan-ready flag.

### DFT sign-off

- ☐ All flops scannable (or excluded with reason); scan chains balanced; lock-up latches between clock domains.
- ☐ DFT DRC clean: no uncontrollable clocks or asynchronous sets/resets in test mode, no tri-state contention, X sources blocked.
- ☐ Stuck-at coverage and transition coverage at target (commonly > 99 % and > 90-95 % respectively; set per product and DPPM goal).
- ☐ Test compression ratio and pattern count fit the tester memory and test-time budget.
- ☐ MBIST on every memory with repair (redundancy) if required; BISR/fuse flow verified.
- ☐ JTAG (1149.1) boundary scan and IJTAG (1687) networks verified; BSDL generated and checked.
- ☐ Test-mode SDC (shift and capture modes, at-speed) delivered to STA.
- ☐ Patterns simulated on the netlist with SDF (zero-delay and timing) and the test protocol reviewed with the test engineer.

## Back-end gates

### Floorplan review

- ☐ Die/core size, aspect ratio and target utilisation agreed; I/O ring or bump map matches the package design.
- ☐ Macro placement reviewed: memories at the edges, pins facing the logic, channels wide enough for routing, halos/keep-outs set.
- ☐ Pin placement follows the top-level plan (for blocks) and signal flow; critical interfaces short.
- ☐ Power grid planned per domain: ring, straps, rails, via density; early IR-drop estimate within budget.
- ☐ Power switches, always-on routing channels and isolation cell locations defined for each switchable domain.
- ☐ Tap cells, end caps, boundary cells and decap budget inserted per PDK rules.
- ☐ Placement and routing blockages, and routing layer limits per block, documented.
- ☐ Floorplan DRC clean; early congestion and timing estimate acceptable.

### Pre-CTS

- ☐ Placement legal (no overlaps, on-site, on-row); density hot spots below limit; congestion map acceptable (overflow under a few %).
- ☐ Setup timing met or within what CTS/useful skew can recover; transition, capacitance and fanout violations fixed.
- ☐ Scan chains re-ordered by placement; spare cells distributed.
- ☐ High-fanout nets (resets, enables) buffered; don't-touch nets respected.
- ☐ CTS specification ready: target skew/latency, max transition, cell list (clock buffers/inverters), NDRs (double width/spacing, shielding), exclusion and stop pins, generated clocks.
- ☐ Clock gating cells placed close to their sinks where possible.

### Post-route

- ☐ Routing complete with zero opens and shorts; DRC count in the tool near zero before sign-off PV.
- ☐ Setup and hold met across all MCMM scenarios with SI enabled.
- ☐ Crosstalk delay and noise glitches fixed (spacing, shielding, resizing, layer promotion).
- ☐ Antenna violations fixed (diodes, jumpers).
- ☐ Redundant (double) vias inserted where possible; metal fill inserted and timing re-checked with fill.
- ☐ Filler and decap cells inserted; no gaps in rows.
- ☐ Final DEF, netlist (with power/ground for LVS) and SPEF written.

## Sign-off gates

### Timing sign-off

- ☐ Parasitics extracted with the sign-off extractor at every RC corner (Cworst, Cbest, RCworst, RCbest, typical) with fill.
- ☐ All MCMM scenarios run: functional and test modes x PVT corners x RC corners, with OCV/AOCV/POCV derates and CRPR.
- ☐ Setup, hold, recovery/removal, min pulse width, clock gating, max transition/capacitance checks clean or waived.
- ☐ SI (crosstalk) analysis enabled; noise glitches clean.
- ☐ Unconstrained endpoints = 0; check\_timing reviewed; every exception (false/multicycle path) reviewed and signed.

- ☐ I/O timing checked against interface spec and board budget.
- ☐ SDF written for gate-level simulation; GLS run on critical modes.

### Power, IR-drop and EM sign-off

- ☐ Total power per use-case within budget (vector-based with SAIF/FSDDB for key modes, vectorless for the rest).
- ☐ Static IR drop within budget (often a few % of VDD) and dynamic (voltage drop during switching) within budget (often ~10 % of VDD, lower at advanced nodes); hot spots reviewed.
- ☐ Timing re-checked with IR-aware delays where drop is significant.
- ☐ Power and signal EM within foundry limits at the mission-profile temperature and lifetime; via EM checked.
- ☐ Power-up/ramp-up (in-rush current) of switched domains analysed; switch sequencing defined.
- ☐ ESD paths verified (clamp placement, bus resistance).
- ☐ Thermal map and junction temperature consistent with the package thermal model.

### Physical verification

- ☐ DRC clean with the foundry sign-off deck and the exact deck version named in the tapeout form.
- ☐ LVS clean: layout vs source netlist, including power/ground, well/substrate connections and macros (black-boxed only where the IP vendor supplies LVS-clean views).
- ☐ ERC clean (floating gates, floating wells, missing taps, power-ground shorts).
- ☐ Antenna rule check clean.
- ☐ Density (metal, poly, diffusion) within min/max windows after fill.
- ☐ Seal ring, logo, alignment marks, chip ID and text labels per foundry requirements.
- ☐ DFM/litho checks (recommended rules, litho hot spots, CMP) reviewed; critical ones fixed.

### Logic equivalence (LEC)

- ☐ RTL vs synthesised netlist: equivalent, no aborts, all key points mapped; guide/SVF file used only with review.
- ☐ Synthesised vs post-scan netlist: equivalent with scan disabled (test inputs constrained).
- ☐ Post-layout netlist (after CTS, optimisation and ECOs) vs pre-layout netlist: equivalent.
- ☐ Low-power-aware LEC (with UPF) confirms isolation/retention do not change function in the on state.
- ☐ Every non-equivalent point and every constraint used is reviewed and documented.

### Tapeout

- ☐ All sign-off items above closed; waiver list approved by the project lead and, where required, by the foundry.
- ☐ Final GDSII/OASIS merged (standard cells, IP, memories, I/O, seal ring); layer map correct; top cell name and unit/precision checked.
- ☐ Stream-out vs database comparison (XOR) clean; file checksum recorded.
- ☐ Tapeout form: node, mask layers and versions, die size, frame, shuttle or production, optional layers, ECO spare strategy.
- ☐ Test program, probe card and load board ordered; package substrate design released.
- ☐ Design database, scripts, tool versions and reports archived for ECOs and metal-only respins.
- ☐ Bring-up plan and silicon validation plan ready before wafers come back.

## File formats at a glance

| Format                              | Purpose                                                                                                                                                                        | Producer                           | Consumer                                          |
|-------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------|---------------------------------------------------|
| <b>Verilog netlist (.v)</b>         | Gate-level connectivity: cell instances and nets                                                                                                                               | Synthesis, DFT insertion, P&R, ECO | P&R, STA, LEC, GLS, LVS (via conversion)          |
| <b>Liberty (.lib; compiled .db)</b> | Cell timing, power, noise, function, area per PVT corner                                                                                                                       | Library characterisation (SPICE)   | Synthesis, P&R, STA, power                        |
| <b>LEF (tech + cell/macro)</b>      | Abstract physical view: layers, vias, sites, pin shapes, blockages                                                                                                             | Foundry/library/IP vendor          | P&R, extraction, DEF readers                      |
| <b>DEF</b>                          | Placed and routed design: rows, components, pins, nets, wiring                                                                                                                 | P&R                                | P&R, extraction, PV, other tools                  |
| <b>SDC (Tcl)</b>                    | Timing constraints: clocks, I/O delays, exceptions                                                                                                                             | Designer / STA engineer            | Synthesis, P&R, STA                               |
| <b>SPEF</b>                         | Extracted R and C parasitics per net                                                                                                                                           | Extraction                         | STA, power, SI, IR tools                          |
| <b>SDF</b>                          | Annotated cell and interconnect delays, timing checks                                                                                                                          | STA / delay calculator             | Gate-level simulation                             |
| <b>UPF (IEEE 1801)</b>              | Power intent: domains, supplies, switches, isolation, retention                                                                                                                | Architect / designer               | Simulation, synthesis, P&R, LEC, static LP checks |
| <b>SAIF</b>                         | Switching activity summary (time at 0/1, toggle counts)                                                                                                                        | Simulator / converter              | Power analysis, power-aware synthesis             |
| <b>VCD (IEEE 1364) / FSDB</b>       | Full value-change waveform                                                                                                                                                     | Simulator, emulator                | Debug, power (time-based), SAIF conversion        |
| <b>GDSII / OASIS</b>                | Final mask geometry by layer, hierarchical                                                                                                                                     | P&R stream-out, custom layout      | PV, foundry mask data prep                        |
| <b>Tcl scripts</b>                  | Tool control and flow automation                                                                                                                                               | Engineers                          | Every major EDA tool                              |
| <b>Also common</b>                  | STIL/WGL (test patterns), BSDL (boundary scan), SVRF/rule decks (PV), ITF/nxtgrd/QRC tech (extraction), OpenAccess/NDM/Milkyway (design databases), CPF (Cadence power intent) | various                            | various                                           |

## Gate-level Verilog netlist

The simplest file in the flow and the one every tool agrees on: a structural Verilog module that instantiates library cells by name and connects their pins with named-port connections. No behaviour, no always blocks - the behaviour lives in the Liberty **function** attribute and in the simulation models. The example is a 3-bit counter with enable and active-low asynchronous reset, synthesised by Yosys 0.33 against the real SKY130 **sky130\_fd\_sc\_hd** typical Liberty.

```

module cnt3 (input wire clk, input wire rst_n, input wire en,
             output reg [2:0] q);
    always @(posedge clk or negedge rst_n)
        if (!rst_n) q <= 3'd0;
        else if (en) q <= q + 3'd1;
endmodule

# syn.py - Yosys script
read_verilog cnt.v
synth -top cnt3 -flatten
dfflibmap -liberty tt_nolp.lib      # map flops to dfrtp etc.
abc -liberty tt_nolp.lib           # map logic to library gates
opt_clean
stat -liberty tt_nolp.lib
write_verilog -noattr cnt3_netlist.v

```

RTL and synthesis script

```

/* Generated by Yosys 0.33 (git sha1 2584903a060) */

module cnt3(clk, rst_n, en, q);
    wire _00_;
    wire _01_;
    wire _02_;
    wire _03_;
    wire _04_;
    input clk;
    wire clk;
    input en;
    wire en;
    output [2:0] q;
    wire [2:0] q;
    input rst_n;
    wire rst_n;
    sky130_fd_sc_hd__nand2_1 _05_ (
        .A(q[0]),
        .B(en),
        .Y(_04_)
    );
    sky130_fd_sc_hd__xor2_1 _06_ (
        .A(q[0]),
        .B(en),
        .X(_00_)
    );
    sky130_fd_sc_hd__nand3_1 _07_ (
        .A(q[0]),
        .B(q[1]),
        .C(en),
        .Y(_03_)
    );
    sky130_fd_sc_hd__xnor2_1 _08_ (
        .A(q[1]),
        .B(_04_),
        .Y(_01_)
    );
    sky130_fd_sc_hd__xnor2_1 _09_ (
        .A(q[2]),
        .B(_03_),
        .Y(_02_)
    );
    sky130_fd_sc_hd__dfrtp_1 _10_ (
        .CLK(clk),

```

```

        .D(_00_),
        .Q(q[0]),
        .RESET_B(rst_n)
    );
    sky130_fd_sc_hd__dfrtp_1 _11_ (
        .CLK(clk),
        .D(_01_),
        .Q(q[1]),
        .RESET_B(rst_n)
    );
    sky130_fd_sc_hd__dfrtp_1 _12_ (
        .CLK(clk),
        .D(_02_),
        .Q(q[2]),
        .RESET_B(rst_n)
    );
endmodule

```

cnt3\_netlist.v as written by Yosys 0.33 (complete file, real run)

```

=== cnt3 ===
Number of wires:          9
Number of wire bits:      11
Number of public wires:   4
Number of public wire bits: 6
Number of cells:          8
    sky130_fd_sc_hd__dfrtp_1    3
    sky130_fd_sc_hd__nand2_1    1
    sky130_fd_sc_hd__nand3_1    1
    sky130_fd_sc_hd__xnor2_1    2
    sky130_fd_sc_hd__xor2_1     1
Chip area for module 'cnt3': 110.105600

```

Area report from **stat -liberty** (real run; area in um^2 from the Liberty **area** attributes)

- **\_10...\_12** are **dfrtp\_1** flops: D flip-flop, positive edge, active-low async reset (**RESET\_B**), single Q output.
- Bit 1's next state is **q[1] XNOR NAND(q[0], en) = q[1] XOR (q[0] AND en)** - the increment carry chain, mapped to real gates.
- Power pins (VPWR/VGND/VPB/VNB) are absent: the logical netlist leaves them implicit. P&R adds them for the LVS/physical netlist (**write\_verilog -pg** style options in commercial tools).
- The escaped names **\_05\_** etc. are tool-generated; keep useful RTL names on registers so STA reports and ECOs stay readable.

#### PITFALL - UNWANTED SPECIAL CELLS

On the first run ABC mapped part of the logic to **sky130\_fd\_sc\_hd\_\_lpflow\_isobufsrc\_1**, an isolation cell meant for power-gating boundaries. Real flows pass a *dont\_use* list (lpflow, probe, clock-only, weak drive, delay cells) to synthesis and P&R; here the same effect was obtained by removing those cells from the Liberty before mapping. Always read the cell-usage table of your first netlist.

## Liberty (.lib)

Liberty is a nested **group(name) { attribute : value; }** text format describing one library at one PVT corner: units, operating conditions, table templates, then one **cell** group per cell with **pin** groups holding capacitance, function and timing/power arcs. Delay tables are indexed by input transition (**index\_1**) and output load (**index\_2**) - the NLDM model of Chapter 4. Below are excerpts of the real SKY130 HD typical library (tt, 25 C, 1.80 V); long table rows are wrapped at commas, otherwise the text is verbatim.

```

library ("sky130_fd_sc_hd__tt_025C_1v80") {
    technology("cmos");
    delay_model : "table_lookup";
    bus_naming_style : "%s[%d]";

```

```

time_unit : "1ns";
voltage_unit : "1V";
leakage_power_unit : "1nW";
current_unit : "1mA";
pulling_resistance_unit : "1kohm";
capacitive_load_unit(1.0000000000, "pf");
revision : 1.0000000000;
operating_conditions ("tt_025C_1v80") {
    voltage : 1.8000000000;
    process : 1.0000000000;
    temperature : 25.000000000;
    tree_type : "balanced_tree";
}
...
lu_table_template ("del_1_7_7") {
    variable_1 : "input_net_transition";
    variable_2 : "total_output_net_capacitance";
    index_1("1, 2, 3, 4, 5, 6, 7");
    index_2("1, 2, 3, 4, 5, 6, 7");
}
lu_table_template ("vio_3_3_1") {
    variable_1 : "related_pin_transition";
    variable_2 : "constrained_pin_transition";
    index_1("1, 2, 3");
    index_2("1, 2, 3");
}
...

```

sky130\_fd\_sc\_hd\_\_tt\_025C\_1v80.lib - library header (real file, excerpt)

```

cell ("sky130_fd_sc_hd__nand2_1") {
    ...
    area : 3.7536000000;
    cell_footprint : "sky130_fd_sc_hd__nand2";
    cell_leakage_power : 0.0021179600;
    pin ("A") {
        capacitance : 0.0023150000;
        clock : "false";
        direction : "input";
        fall_capacitance : 0.0022540000;
        ...
        rise_capacitance : 0.0023750000;
    }
    pin ("Y") {
        direction : "output";
        function : "(!A) | (!B)";
        ...
        max_capacitance : 0.1666360000;
        max_transition : 1.4963760000;
        timing () {
            cell_fall ("del_1_7_7") {
                index_1("0.0100000000, 0.0230506000, 0.0531329000, 0.1224740000,
                    0.2823110000, 0.6507430000, 1.5000000000");
                index_2("0.0005000000, 0.0013165500, 0.0034665900, 0.0091278700,
                    0.0240345000, 0.0632852000, 0.1666360000");
                values("0.0206305000, 0.0250594000, 0.0363371000, 0.0651531000,
                    0.1403625000, 0.3379392000, 0.8628026000", \
                    ...
            }
            ...
            related_pin : "A";
            timing_sense : "negative_unate";
        }
    }
}

```



```

        timing_type : "combinational";
    }
    ...

```

The nand2\_1 cell: area, input pin capacitance, function and one timing arc (real file, excerpt)

| Attribute                        | Meaning in the excerpt                                                                                           |
|----------------------------------|------------------------------------------------------------------------------------------------------------------|
| time_unit / capacitive_load_unit | 1 ns and 1 pF: the A-pin capacitance 0.002315 means 2.3 fF                                                       |
| area                             | 3.7536 um^2 = 1.38 um x 2.72 um (the LEF SIZE below)                                                             |
| function                         | Boolean output function used by synthesis, LEC and simulation-model generation                                   |
| cell_rise / cell_fall            | Propagation delay tables (ns); the first cell_fall entry 0.0206 ns = 20.6 ps at 10 ps input slew and 0.5 fF load |
| rise/fall_transition             | Output slew table - fed to the next stage's delay lookup                                                         |
| timing_sense                     | <b>negative_unate</b> : an input rise can only cause an output fall                                              |
| max_capacitance / max_transition | Design-rule limits the tools must respect (DRV fixing)                                                           |

```

pin ("D") {
    capacitance : 0.0019770000;
    clock : "false";
    direction : "input";
    ...
    timing () {
        fall_constraint ("vio_3_3_1") {
            index_1("0.0100000000, 0.5000000000, 1.5000000000");
            index_2("0.0100000000, 0.5000000000, 1.5000000000");
            values("0.1069805000, 0.3212057000, 0.6280091000", \
                "-0.004705700, 0.2034160000, 0.5028952000", \
                "-0.090561800, 0.1126771000, 0.4084941000");
        }
        ...
        related_pin : "CLK";
        timing_type : "setup_rising";
    }
    ...
}

```

dfrtp\_1 pin D: the setup constraint table (real file, excerpt)

Constraint tables are indexed by the clock (related pin) slew and the data (constrained pin) slew. Negative values in hold tables (and even in setup tables for some slew combinations, as above) are normal: they mean the internal clock path is slower than the data path inside the cell.

#### BEYOND NLDL

Advanced-node libraries add CCS (composite current source) or ECSM current-based timing, noise (CCSN) and power models, and LVF (Liberty Variation Format) sigma tables for POCV/SOCV. Commercial tools compile .lib to a binary (.db for Synopsys) for speed.

## LEF (Library Exchange Format)

LEF has two halves. The **technology LEF** defines units, manufacturing grid, placement sites, routing layers (direction, pitch, width, spacing, R and C per unit, antenna and current-density rules) and vias. The **cell/macro LEF** gives each cell's size, site, symmetry, pin shapes and obstructions - an abstract that is enough to place and route without the full GDS. Both excerpts are the real SKY130 HD files.

```

VERSION 5.7 ;

BUSBITCHARS "["] ;
DIVIDERCHAR "/" ;

```

```

UNITS
  TIME NANoseconds 1 ;
  CAPACITANCE PICOfarads 1 ;
  RESISTANCE OHMS 1 ;
  DATABASE MICRONS 1000 ;
END UNITS

SITE unithd
  SYMMETRY Y ;
  CLASS CORE ;
  SIZE 0.46 BY 2.72 ;
END unithd

LAYER met1
  TYPE ROUTING ;
  DIRECTION HORIZONTAL ;
  PITCH 0.34 ;
  OFFSET 0.17 ;
  WIDTH 0.14 ; # Met1 1
  # SPACING 0.14 ; # Met1 2
  # SPACING 0.28 RANGE 3.001 100 ; # Met1 3b
  SPACINGTABLE
    PARALLELRUNLENGTH 0
    WIDTH 0 0.14
    WIDTH 3 0.28 ;
  AREA 0.083 ; # Met1 6
  THICKNESS 0.35 ;
  ANTENNA_MODEL OXIDE1 ;
  ANTENNA_DIFF_SIDE_AREA_RATIO PWL ( ( 0 400 ) ( 0.0125 400 ) ( 0.0225 2609 ) ( 22.5 11600 ) ) ;
  EDGE_CAPACITANCE 40.567E-6 ;
  CAPACITANCE_CPERSQDIST 25.7784E-6 ;
  DCCURRENTDENSITY_AVERAGE 2.8 ; # mA/um Iavg_max at Tj = 90oC
  ACCURRENTDENSITY_RMS 6.1 ; # mA/um Irms_max at Tj = 90oC
  MAXIMUMDENSITY 70 ;
  DENSITYCHECKWINDOW 700 700 ;
  DENSITYCHECKSTEP 70 ;
  RESISTANCE_RPERSQ 0.125 ;
END met1

```

sky130\_fd\_sc\_hd.tlef - units, the unithd site and the met1 layer (real file, excerpt)

```

VERSION 5.7 ;
NOWIREEXTENSIONATPIN ON ;
DIVIDERCHAR "/" ;
BUSBITCHARS "[" ;
MACRO sky130_fd_sc_hd__nand2_1
  CLASS CORE ;
  FOREIGN sky130_fd_sc_hd__nand2_1 ;
  ORIGIN 0.000000 0.000000 ;
  SIZE 1.380000 BY 2.720000 ;
  SYMMETRY X Y R90 ;
  SITE unithd ;
  PIN A
    ANTENNA_GATEAREA 0.247500 ;
    DIRECTION INPUT ;
    USE SIGNAL ;
  PORT
    LAYER li1 ;
    RECT 0.940000 1.075000 1.275000 1.325000 ;
  END

```

```

END A
PIN B ... (same form as A) ... END B
PIN Y
  ANTENNADIFFAREA 0.439000 ;
  DIRECTION OUTPUT ;
  USE SIGNAL ;
  PORT
    LAYER li1 ;
    RECT 0.535000 1.485000 0.865000 2.465000 ;
    RECT 0.600000 0.255000 1.295000 0.885000 ;
    RECT 0.600000 0.885000 0.770000 1.485000 ;
  END
END Y
PIN VGND
  DIRECTION INOUT ;
  SHAPE ABUTMENT ;
  USE GROUND ;
  PORT
    LAYER met1 ;
    RECT 0.000000 -0.240000 1.380000 0.240000 ;
  END
END VGND
PIN VNB
  DIRECTION INOUT ;
  USE GROUND ;
  PORT
    LAYER pwell ;
    RECT 0.140000 -0.085000 0.310000 0.085000 ;
  END
END VNB
PIN VPB
  DIRECTION INOUT ;
  USE POWER ;
  PORT
    LAYER nwell ;
    RECT -0.190000 1.305000 1.570000 2.910000 ;
  END
END VPB
PIN VPWR
  DIRECTION INOUT ;
  SHAPE ABUTMENT ;
  USE POWER ;
  PORT
    LAYER met1 ;
    RECT 0.000000 2.480000 1.380000 2.960000 ;
  END
END VPWR
OBS
  LAYER li1 ;
  RECT 0.000000 -0.085000 1.380000 0.085000 ;
  RECT 0.000000 2.635000 1.380000 2.805000 ;
  ...
END
END sky130_fd_sc_hd__nand2_1
END LIBRARY

```

sky130\_fd\_sc\_hd\_\_nand2\_1.magic.lef - cell abstract (real file; pin B and part of OBS elided)

- **SIZE 1.38 BY 2.72** = 3 sites of 0.46 um by one row of 2.72 um; the Liberty area (3.7536) is the same rectangle.
- Pins are on **li1** (SKY130's local interconnect) - the router must reach them with an li1-met1 contact (mcon); OBS tells it where not to route.

- **ANTENNAGATEAREA** and **ANTENNADIFFAREA** feed the antenna checker (Chapter 17); **SHAPE ABUTMENT** marks the rail pins that connect by abutting cells in a row.
- met1 is 0.14 um wide on a 0.34 um pitch, 0.125 ohm/sq: a 100 um minimum-width wire is about  $0.125 \times 100 / 0.14 = 89$  ohm.

## DEF (Design Exchange Format)

DEF carries the design itself: die area, rows, tracks, components with placement, I/O pins, special (power) nets and regular nets with optional routing. Coordinates are integers in database units (**UNITS DISTANCE MICRONS 1000** = 1 nm). The example places the eight cells of the netlist above; it was hand-written and then read by KLayout's LEF/DEF reader together with the real SKY130 LEFs.

```

VERSION 5.8 ;
DIVIDERCHAR "/" ;
BUSBITCHARS "[" ;
DESIGN cnt3 ;
UNITS DISTANCE MICRONS 1000 ;
DIEAREA ( 0 0 ) ( 27600 16320 ) ;
ROW ROW_0 unithd 5520 2720 N DO 36 BY 1 STEP 460 0 ;
ROW ROW_1 unithd 5520 5440 FS DO 36 BY 1 STEP 460 0 ;
ROW ROW_2 unithd 5520 8160 N DO 36 BY 1 STEP 460 0 ;
ROW ROW_3 unithd 5520 10880 FS DO 36 BY 1 STEP 460 0 ;
TRACKS X 230 DO 60 STEP 460 LAYER met1 ;
TRACKS Y 170 DO 48 STEP 340 LAYER met1 ;
COMPONENTS 8 ;
- _10_ sky130_fd_sc_hd__dfrrtp_1 + PLACED ( 5520 2720 ) N ;
- _05_ sky130_fd_sc_hd__nand2_1 + PLACED ( 14720 2720 ) N ;
- _06_ sky130_fd_sc_hd__xor2_1 + PLACED ( 16100 2720 ) N ;
- _11_ sky130_fd_sc_hd__dfrrtp_1 + PLACED ( 5520 5440 ) FS ;
- _07_ sky130_fd_sc_hd__nand3_1 + PLACED ( 14720 5440 ) FS ;
- _08_ sky130_fd_sc_hd__xnor2_1 + PLACED ( 16560 5440 ) FS ;
- _12_ sky130_fd_sc_hd__dfrrtp_1 + PLACED ( 5520 8160 ) N ;
- _09_ sky130_fd_sc_hd__xnor2_1 + PLACED ( 14720 8160 ) N ;
END COMPONENTS
PINS 6 ;
- clk + NET clk + DIRECTION INPUT + USE CLOCK
  + LAYER met2 ( -70 -300 ) ( 70 300 ) + PLACED ( 2530 300 ) N ;
- rst_n + NET rst_n + DIRECTION INPUT + USE SIGNAL
  + LAYER met2 ( -70 -300 ) ( 70 300 ) + PLACED ( 4370 300 ) N ;
- en + NET en + DIRECTION INPUT + USE SIGNAL
  + LAYER met2 ( -70 -300 ) ( 70 300 ) + PLACED ( 6210 300 ) N ;
- q[0] + NET q[0] + DIRECTION OUTPUT + USE SIGNAL
  + LAYER met2 ( -70 -300 ) ( 70 300 ) + PLACED ( 20930 16020 ) S ;
- q[1] + NET q[1] + DIRECTION OUTPUT + USE SIGNAL
  + LAYER met2 ( -70 -300 ) ( 70 300 ) + PLACED ( 22770 16020 ) S ;
- q[2] + NET q[2] + DIRECTION OUTPUT + USE SIGNAL
  + LAYER met2 ( -70 -300 ) ( 70 300 ) + PLACED ( 24610 16020 ) S ;
END PINS
SPECIALNETS 2 ;
- VPWR ( * VPWR ) + USE POWER ;
- VGND ( * VGND ) + USE GROUND ;
END SPECIALNETS
NETS 11 ;
- clk ( PIN clk ) ( _10_ CLK ) ( _11_ CLK ) ( _12_ CLK ) + USE CLOCK ;
- rst_n ( PIN rst_n ) ( _10_ RESET_B ) ( _11_ RESET_B ) ( _12_ RESET_B ) ;
- en ( PIN en ) ( _05_ B ) ( _06_ B ) ( _07_ C ) ;
- q[0] ( PIN q[0] ) ( _10_ Q ) ( _05_ A ) ( _06_ A ) ( _07_ A ) ;
- q[1] ( PIN q[1] ) ( _11_ Q ) ( _07_ B ) ( _08_ A ) ;
- q[2] ( PIN q[2] ) ( _12_ Q ) ( _09_ A ) ;
- _00_ ( _06_ X ) ( _10_ D )

```

```

+ ROUTED met1 ( 18630 3230 ) ( 7130 * ) ;
- _01_ ( _08_ Y ) ( _11_ D ) ;
- _02_ ( _09_ Y ) ( _12_ D ) ;
- _03_ ( _07_ Y ) ( _09_ B ) ;
- _04_ ( _05_ Y ) ( _08_ B ) ;
END NETS
END DESIGN

```

cnt3.def - small hand-written DEF 5.8 (syntax checked by KLayout)

```

top cell : cnt3 bbox (um): (0,0;27.6,16.32)
instances : 8
  master : sky130_fd_sc_hd__dfrtp_1
  master : sky130_fd_sc_hd__nand2_1
  master : sky130_fd_sc_hd__nand3_1
  master : sky130_fd_sc_hd__xnor2_1
  master : sky130_fd_sc_hd__xor2_1
layers : OUTLINE li1.LABEL li1.OBS li1.PIN mcon.OBS met1
        met1.LABEL met1.OBS met1.PIN met2.LABEL met2.PIN

```

KLayout 0.30 (Python API) reading cnt3.def with the tech LEF and five cell LEFs (real run)

| Section        | What to notice                                                                                            |
|----------------|-----------------------------------------------------------------------------------------------------------|
| ROW ... N / FS | Alternate rows are flipped (FS) so VPWR and VGND rails are shared between neighbouring rows               |
| TRACKS         | Routing grid per layer; must match the LEF pitch/offset                                                   |
| COMPONENTS     | Instance, master, status (PLACED, FIXED, COVER, UNPLACED), location of the lower-left corner, orientation |
| PINS           | Top-level ports with layer, shape and location                                                            |
| SPECIALNETS    | Power/ground; in a real DEF these carry the rails and straps as geometry                                  |
| NETS + ROUTED  | Connectivity, then wiring as layer + centre-line points; * repeats the previous coordinate                |

## SDC (Synopsys Design Constraints)

SDC is a Tcl subset (the specification is published openly by Synopsys) read by synthesis, P&R and STA. The order matters little, the completeness matters a lot: every clock, every I/O and every exception. The file below constrains the counter at 100 MHz.

```

# cnt3.sdc - timing constraints for the 3-bit counter (units from the Liberty: ns, pF)
set_units -time ns -capacitance pF
create_clock -name clk -period 10.0 -waveform {0 5} [get_ports clk]
set_clock_uncertainty -setup 0.25 [get_clocks clk]    ;# jitter + margin before CTS
set_clock_uncertainty -hold 0.05 [get_clocks clk]
set_clock_transition 0.15 [get_clocks clk]           ;# ideal-clock slew (pre-CTS only)
set_clock_latency -source 0.40 [get_clocks clk]       ;# PLL/board to the clock port

set_input_delay -clock clk -max 4.0 [get_ports {en rst_n}]
set_input_delay -clock clk -min 0.5 [get_ports {en rst_n}]
set_output_delay -clock clk -max 3.0 [get_ports {q[*]}] ;# braces: q[*] is not a command
set_output_delay -clock clk -min 0.0 [get_ports {q[*]}]

set_driving_cell -lib_cell sky130_fd_sc_hd__buf_2 -pin X [get_ports {en rst_n}]
set_load 0.02 [get_ports {q[*]}]
set_max_transition 1.0 [current_design]
set_max_fanout 16 [current_design]

# Exceptions: always justify in a comment and review with the designer.
# set_multicycle_path 2 -setup -from [get_cells u_slow/*] -to [get_cells u_acc/*]
# set_multicycle_path 1 -hold -from [get_cells u_slow/*] -to [get_cells u_acc/*]
# set_false_path -from [get_clocks clk_a] -to [get_clocks clk_b] ;# only with a CDC sync

```

**PITFALL - SDC MISTAKES THAT PASS SILENTLY**

`[get_ports q[*]]` without braces makes Tcl try to run a command named `*`; always brace bus names.

A `set_false_path` on an asynchronous reset input also removes recovery/removal checks at the flops - use a proper reset synchroniser and constrain the synchronised reset instead.

`set_clock_transition` and ideal latency must be removed (or ignored via propagated clocks) after CTS; leaving them in makes post-CTS timing optimistic or pessimistic.

Every `set_multicycle_path -setup N` needs a matching `-hold N-1`, otherwise the hold check moves too and hold fixing inserts hundreds of useless buffers.

**SPEF (Standard Parasitic Exchange Format)**

SPEF (defined in IEEE 1481) is written by the extractor and read by STA and power/SI tools. A header fixes units and delimiters; a name map shortens names to `*n` indexes; each `*D_NET` (detailed net) lists its total capacitance, connections, ground and coupling capacitors and resistors between nodes. The example is written by hand for net `_00_` of the DEF above using the tech-LEF values (met1: 0.0258 fF/um<sup>2</sup> area and 0.0406 fF/um edge capacitance, 0.125 ohm/sq).

```
*SPEF "IEEE 1481-1998"
*DESIGN "cnt3"
*DATE "Sat Sep 26 2026"
*VENDOR "hand-written example"
*PROGRAM "none"
*VERSION "1.0"
*DESIGN_FLOW "EXTERNAL_LOADS" "EXTERNAL_SLEWS"
*DIVIDER /
*DELIMITER :
*BUS_DELIMITER [ ]
*T_UNIT 1 NS
*C_UNIT 1 FF
*R_UNIT 1 OHM
*L_UNIT 1 HENRY

*NAME_MAP
*1 _00_
*2 _06_
*3 _10_
*4 q[0]

*PORTS
*4 0

// Net _00_: xor2 output _06_/X drives flop input _10_/D along ~11.5 um of met1
*D_NET *1 1.00
*CONN
*I *2:X 0 *C 18.63 3.23 *D sky130_fd_sc_hd__xor2_1
*I *3:D I *C 7.13 3.23 *D sky130_fd_sc_hd__dfrtp_1
*CAP
1 *2:X 0.12
2 *1:1 0.48
3 *3:D 0.24
4 *1:1 *4:2 0.16
*RES
1 *2:X *1:1 12.5
2 *1:1 *3:D 10.3
*END

// ... one *D_NET (or *R_NET for reduced models) per net; q[0] (name *4) owns node *4:2
```

Small, syntactically complete SPEF example (hand-written, not from an extractor)

- **\*D\_NET \*1 1.00**: total capacitance of net **\_00\_** = 1.00 fF = sum of the ground caps (0.12 + 0.48 + 0.24) + the coupling cap (0.16).
- **\*CONN: \*I** = instance pin, direction, optional coordinates (**\*C**) and driving cell (**\*D**); **\*P** would be a top-level port.
- **\*CAP** lines with two nodes are coupling capacitors to another net (**\*4:2** = node 2 of **q[0]**) - the input to crosstalk analysis.
- **\*RES**: 12.5 ohm for the li1/mcon pin access at the driver and 10.3 ohm for ~11.5 um of 0.14 um met1 (82 squares x 0.125).
- Pin capacitances are **not** in SPEF: STA takes them from the Liberty.

## SDF (Standard Delay Format)

SDF (IEEE 1497) carries delays **out of** STA into gate-level simulation: **IOPATH** cell delays, **INTERCONNECT** wire delays and **TIMINGCHECK** limits (SETUP, HOLD, RECOVERY, WIDTH...). Each delay is a (min:typ:max) triple, given separately for rise and fall. The small file below was written by hand and annotated onto a two-NAND netlist with Icarus Verilog's **\$sdf\_annotate**.

```
(DELAYFILE
  (SDFVERSION "3.0")
  (DESIGN "top")
  (VENDOR "hand-written example")
  (DIVIDER /)
  (TIMESCALE 1ns)
  (CELL
    (CELLTYPE "top")
    (INSTANCE)
    (DELAY
      (ABSOLUTE
        (INTERCONNECT u1/Y u2/A (0.010:0.015:0.022))
      )
    )
  )
  (CELL
    (CELLTYPE "NAND2")
    (INSTANCE u1)
    (DELAY
      (ABSOLUTE
        (IOPATH A Y (0.031:0.045:0.066) (0.024:0.035:0.052))
        (IOPATH B Y (0.034:0.049:0.071) (0.026:0.038:0.056))
      )
    )
  )
  (CELL
    (CELLTYPE "NAND2")
    (INSTANCE u2)
    (DELAY
      (ABSOLUTE
        (IOPATH A Y (0.031:0.045:0.066) (0.024:0.035:0.052))
        (IOPATH B Y (0.034:0.049:0.071) (0.026:0.038:0.056))
      )
    )
  )
)
```

top.sdf - hand-written SDF 3.0

```
`timescale 1ns/1ps
module NAND2 (input A, input B, output Y);
    nand g (Y, A, B);
    specify (A => Y) = (0, 0); (B => Y) = (0, 0); endspecify
endmodule
// top: u1 = NAND2(a, b) -> n1; u2 = NAND2(n1, c) -> y
// tb: b = c = 1, a steps 0 -> 1 at t = 1 ns;
//     initial $sdf_annotate("top.sdf", dut);
$ iverilog -gspecify -o sim tb.v cells.v && vvp sim -sdf-verbose
```

Cell model with a specify block and the run command

```
top.sdf:3:SDF INFO: Design: top
top.sdf:4:SDF INFO: Vendor: hand-written example
top.sdf:6:SDF INFO: TIMESCALE : 1ns
top.sdf:12: SDF WARNING: INTERCONNECT not supported.
t=0.000 ns  a=0  y=x
t=0.080 ns  a=0  y=0
t=1.000 ns  a=1  y=0
t=1.080 ns  a=1  y=1
tb.v:9: $finish called at 2000 (1ps)
```

Icarus Verilog 12 applying the SDF (real run)

At  $t = 1$  ns **a** rises, so **n1** falls after u1's A->Y **fall** typical delay (0.035 ns) and **y** rises after u2's A->Y **rise** typical delay (0.045 ns): 0.080 ns, exactly as annotated. Icarus uses the typical value by default and warns that it ignores **INTERCONNECT** - a sign-off GLS simulator (VCS, Xcelium, Questa) would add the 0.015 ns wire delay too.

## UPF (Unified Power Format, IEEE 1801)

UPF is Tcl-based power intent that travels with the RTL through simulation, synthesis, P&R and LEC. It declares supply ports/nets and supply sets, power domains, power switches, isolation, retention, level shifters and the legal power states. The example describes a CPU domain that can be switched off while the top stays on (Chapter 11).

```
# cpu_pd.upf - IEEE 1801 power intent: a switchable CPU domain inside an always-on top
create_supply_port VDD
create_supply_port VSS
create_supply_net VDD
create_supply_net VSS
create_supply_net VDD_CPU ;# switched rail, driven by SW_CPU
connect_supply_net VDD -ports VDD
connect_supply_net VSS -ports VSS
create_supply_set SS_AON -function {power VDD} -function {ground VSS}
create_supply_set SS_CPU -function {power VDD_CPU} -function {ground VSS}

create_power_domain PD_TOP -include_scope -supply {primary SS_AON}
create_power_domain PD_CPU -elements {u_cpu} -supply {primary SS_CPU}

create_power_switch SW_CPU -domain PD_CPU \
    -input_supply_port {vin VDD} \
    -output_supply_port {vout VDD_CPU} \
    -control_port {sleep u_pmu/cpu_sleep} \
    -on_state {on_s vin {!sleep}} \
    -off_state {off_s {sleep}}

set_isolation ISO_CPU -domain PD_CPU -applies_to outputs \
    -isolation_supply_set SS_AON -clamp_value 0 \
    -isolation_signal u_pmu/cpu_iso -isolation_sense high -location parent

set_retention RET_CPU -domain PD_CPU -retention_supply_set SS_AON \
```



```

    -save_signal {u_pmu/cpu_save high} -restore_signal {u_pmu/cpu_restore low}

# Legal power states (UPF 2.0 port-state / PST style; 1801-2013+ also offers
# add_power_state on supply sets and domains)
add_port_state VDD -state {ON 0.8}
add_port_state SW_CPU/vout -state {CPU_ON 0.8} -state {CPU_OFF off}
create_pst pst_top -supplies {VDD VDD_CPU}
add_pst_state RUN -pst pst_top -state {ON CPU_ON}
add_pst_state SLEEP -pst pst_top -state {ON CPU_OFF}

```

cpu\_pd.upf - hand-written example

- Isolation clamps the CPU outputs to 0 while its supply is off; the isolation cells are powered by the always-on set SS\_AON.
- Retention flops keep state on SS\_AON; **save** and **restore** are sequenced by the PMU.
- The PST lists legal combinations; the static checker uses it to decide where isolation and level shifting are required.
- No level shifters are needed here because both rails are 0.8 V when on; with different voltages add **set\_level\_shifter**.

## VCD (Value Change Dump) and SAIF (Switching Activity)

VCD (defined in IEEE 1364) records every value change with a timestamp: a header declares scopes and variables with short identifier codes, then **#time** lines are followed by changes (**1!** = scalar **!** becomes 1, **b10** **\$** = vector **\$** becomes 2). The dump below is the complete file from an Icarus run of the counter (clock period 10 ns, reset released at 12 ns, enable at 22 ns).

```

$date
    Sat Sep 26 17:13:29 2026
$end
$version
    Icarus Verilog
$end
$timescale
    1ps
$end
$scope module tb $end
$scope module dut $end
$var wire 1 ! clk $end
$var wire 1 " en $end
$var wire 1 # rst_n $end
$var reg 3 $ q [2:0] $end
$upscope $end
$upscope $end
$enddefinitions $end
$comment Show the parameter values. $end
$dumppall
$end
#0
$dumppvars
b0 $
0#
0"
0!
$end
#5000
1!
#10000
0!
#12000
1#

```

```
#15000
1!
#20000
0!
#22000
1"
#25000
b1 $
1!
#30000
0!
#35000
b10 $
1!
#40000
0!
#45000
b11 $
1!
#50000
0!
#52000
```

cnt3.vcd from Icarus Verilog 12 (complete file, real run)

SAIF condenses a waveform into per-net statistics - time at 0 (T0), at 1 (T1), at X (TX), toggle count (TC) and glitch count (IG) - which is all a power tool needs to compute average switching power. The file below was produced from the VCD above by a small Python converter ([DURATION](#) and times in the VCD time unit, 1 ps).

```
(SAIFFILE
(SAIFVERSION "2.0")
(DIRECTION "backward")
(DESIGN )
(VENDOR "example")
(PROGRAM_NAME "vcd2saif.py")
(VERSION "0.1")
(DIVIDER / )
(TIMESCALE 1 ps)
(DURATION 52000)
(INSTANCE tb
  (INSTANCE dut
    (NET
      (clk (T0 27000) (T1 25000) (TX 0) (TC 10) (IG 0))
      (en (T0 22000) (T1 30000) (TX 0) (TC 1) (IG 0))
      (q\[0\] (T0 35000) (T1 17000) (TX 0) (TC 3) (IG 0))
      (q\[1\] (T0 35000) (T1 17000) (TX 0) (TC 1) (IG 0))
      (q\[2\] (T0 52000) (T1 0) (TX 0) (TC 0) (IG 0))
      (rst_n (T0 12000) (T1 40000) (TX 0) (TC 1) (IG 0))
    )
  )
)
)
```

cnt3.saif derived from cnt3.vcd (real conversion output)

- clk: 10 toggles in 52 ns, T1 = 25 ns (five full high phases).
- q[0] toggles 3 times (0->1->0->1), q[1] once, q[2] never: activity halves per bit, as expected for a counter.
- Toggle rate of a net = TC / DURATION; a power tool multiplies it by 0.5 C V<sup>2</sup> per transition to get switching power.
- Real flows use FSDB or SAIF from the simulator/emulator directly; SAIF files for large SoCs are far smaller than their VCDs.

## GDSII and OASIS

GDSII (Stream format, from Calma in the 1970s) is a binary sequence of records. Each record starts with a 2-byte length, a 1-byte record type and a 1-byte data type (0 none, 1 bit array, 2 16-bit int, 3 32-bit int, 5 8-byte real in IBM excess-64 format, 6 ASCII). A library holds structures (cells); a structure holds elements - BOUNDARY (polygon), PATH, TEXT, SREF (cell instance) and AREF (array instance) - each with a LAYER/DATATYPE pair and XY coordinates in database units. The dump below is a real two-cell file written by gdstk: a 1.38 x 0.48 um shape on SKY130 met1 drawing (68/20) with a VGND label (68/5), instantiated once in [top](#).

| offset | len | rec | dt | name         | payload                                   |
|--------|-----|-----|----|--------------|-------------------------------------------|
| 0      | 6   | 00  | 2  | HEADER       | [600]                                     |
| 6      | 28  | 01  | 2  | BGNLIB       | [126, 1, 1, 0, 0, 0, 126, 1, '...']       |
| 34     | 12  | 02  | 6  | LIBNAME      | 'demo_lib'                                |
| 46     | 20  | 03  | 5  | UNITS        | ['1e-03', '1e-09']                        |
| 66     | 28  | 05  | 2  | BGNSTR       | [126, 1, 1, 0, 0, 0, 126, 1, '...']       |
| 94     | 10  | 06  | 6  | STRNAME      | 'pad_m1'                                  |
| 104    | 4   | 08  | 0  | BOUNDARY     |                                           |
| 108    | 6   | 0D  | 2  | LAYER        | [68]                                      |
| 114    | 6   | 0E  | 2  | DATATYPE     | [20]                                      |
| 120    | 44  | 10  | 3  | XY           | [0, 0, 1380, 0, 1380, 480, 0, 480, '...'] |
| 164    | 4   | 11  | 0  | ENDEL        |                                           |
| 168    | 4   | 0C  | 0  | TEXT         |                                           |
| 172    | 6   | 0D  | 2  | LAYER        | [68]                                      |
| 178    | 6   | 16  | 2  | TEXTTYPE     | [5]                                       |
| 184    | 6   | 17  | 1  | PRESENTATION | 0x0005                                    |
| 190    | 12  | 10  | 3  | XY           | [690, 240]                                |
| 202    | 8   | 19  | 6  | STRING       | 'VGND'                                    |
| 210    | 4   | 11  | 0  | ENDEL        |                                           |
| 214    | 4   | 07  | 0  | ENDSTR       |                                           |
| 218    | 28  | 05  | 2  | BGNSTR       | [126, 1, 1, 0, 0, 0, 126, 1, '...']       |
| 246    | 8   | 06  | 6  | STRNAME      | 'top'                                     |
| 254    | 4   | 0A  | 0  | SREF         |                                           |
| 258    | 10  | 12  | 6  | SNAME        | 'pad_m1'                                  |
| 268    | 12  | 10  | 3  | XY           | [10000, 5000]                             |
| 280    | 4   | 11  | 0  | ENDEL        |                                           |
| 284    | 4   | 07  | 0  | ENDSTR       |                                           |
| 288    | 4   | 04  | 0  | ENDLIB       |                                           |

Record-by-record dump of demo.gds (real file, 292 bytes; parser written for this appendix)

- UNITS: 1e-3 user units per database unit and 1e-9 m per database unit - a 1 nm grid, displayed in um.
- BGNLIB/BGNSTR hold modification/access dates (year 126 = years since 1900 = 2026).
- The polygon has 5 XY points (10 integers; the dump shows the first 8): GDS polygons repeat the first point to close the shape.
- Classic GDSII limits: 32-bit coordinates, polygons of up to 8191 vertices, layer and datatype numbers traditionally 0-255 (many tools accept up to 65535); files of advanced SoCs reach tens to hundreds of GB.

**OASIS** (SEMI P39, 2004) is the successor for large layouts: variable-length integers, relative coordinates, repetitions (regular and irregular arrays of any element), modal variables (repeat the previous layer/width without storing it), a string/name table and optional compression per cell. Files are typically 10x or more smaller than the equivalent GDSII. OASIS.MASK (SEMI P44) is the restricted subset used for mask data preparation. Foundries accept both; tapeout forms record the format, top cell, layer map and checksum.

## Tcl in EDA tools

Practically every commercial EDA tool (Design Compiler, Fusion Compiler, Genus, Innovus, PrimeTime, Tempus, Calibre's batch mode, Tessent) and OpenROAD uses Tcl as its command language; SDC and UPF are Tcl dialects. Flow scripts set variables, loop over files and corners, run tool commands and parse reports. Yosys can also run Tcl, which makes a small but real demonstration.

```
# flow.tcl - run with: yosys -c flow.tcl
yosys -import                ;# make Yosys commands callable as Tcl procs
set top      cnt3
set liberty  tt_nolp.lib
foreach f {cnt.v} { read_verilog $f }
synth -top $top -flatten
dfflibmap -liberty $liberty
abc -liberty $liberty
opt_clean
tee -o area.rpt stat -liberty $liberty
write_verilog -noattr ${top}_netlist.v
# pull one number out of the report with plain Tcl
set fh [open area.rpt r]; set rpt [read $fh]; close $fh
regexp {Chip area for module '^[^']*': ([0-9.]+)} $rpt -> area
regexp {Number of cells:\s+([0-9]+)} $rpt -> ncells
puts [format "RESULT: %s area = %.2f um^2 (%d cells)" $top $area $ncells]
```

flow.tcl - the synthesis above driven from Tcl

```
$ yosys -q -c flow.tcl 2>&1 | grep -v Warning
RESULT: cnt3 area = 110.11 um^2 (8 cells)
```

Real run (Yosys 0.33 with its built-in Tcl interpreter)

| Idiom                                                       | Why                                                              |
|-------------------------------------------------------------|------------------------------------------------------------------|
| <code>foreach corner {ss tt ff} { ... }</code>              | Run the same step per corner or mode (MCMC set-up, report loops) |
| <code>[get_cells -hier -filter "ref_name =~ *DFF*"]</code>  | Collections (SDC/STA object queries) instead of string lists     |
| <code>redirect -file x.rpt {report_timing} / tee -o</code>  | Save reports for regression diffing                              |
| <code>if {[sizeof_collection \$viol] &gt; 0} { ... }</code> | Gate the flow on a check (stop if unconstrained endpoints exist) |
| <code>source -echo -verbose</code>                          | Log every command - essential for reproducing a run months later |

### REPRODUCIBILITY

Keep flow scripts in revision control with the tool versions, library versions and PDK revision they were run with. At tapeout, archive scripts, inputs and logs together - an ECO a year later depends on it (Chapter 19).

## Appendix B. Glossary

About 190 terms used in the book, alphabetically. Numbers in parentheses point to the chapter (or appendix) where the concept is treated in depth.

| Term                      | Definition                                                                                                                                           |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>ABC</b>                | Berkeley logic synthesis and verification system; the technology mapper inside Yosys (24).                                                           |
| <b>Abutment</b>           | Placing cells or blocks edge to edge so that rails or pins connect without routing (4, 15).                                                          |
| <b>AEC-Q100</b>           | Automotive Electronics Council stress-test qualification for ICs; temperature grades 0-3 (22).                                                       |
| <b>Always-on domain</b>   | Power domain that stays powered in all modes; hosts the PMU, isolation and retention control (11).                                                   |
| <b>Antenna effect</b>     | Charge collected on a long metal wire during plasma etch damaging a connected gate oxide; fixed with diodes or layer jumpers (17, 18).               |
| <b>AOCV / POCV / SOCV</b> | Advanced / parametric / statistical on-chip variation: depth- and distance-aware or sigma-based derates that are less pessimistic than flat OCV (9). |
| <b>ASIC</b>               | Application-specific integrated circuit: a chip designed for one product or function (1).                                                            |
| <b>ASIL</b>               | Automotive Safety Integrity Level (A-D) from ISO 26262; sets the required diagnostic coverage and failure-rate targets (22).                         |
| <b>ATE</b>                | Automatic test equipment: the tester that applies patterns at wafer sort and final test (21).                                                        |
| <b>ATPG</b>               | Automatic test pattern generation: computes scan patterns that detect modelled faults (10).                                                          |
| <b>BEOL</b>               | Back end of line: the metal interconnect layers and vias built above the transistors (3).                                                            |
| <b>Binning</b>            | Sorting good dies by measured speed or power into product grades (21).                                                                               |
| <b>BIST</b>               | Built-in self-test: on-chip pattern generation and response checking (LBIST for logic, MBIST for memories) (10).                                     |
| <b>Blockage</b>           | Region where placement (hard/soft/partial) or routing on given layers is forbidden or limited (14).                                                  |
| <b>Body bias</b>          | Voltage on the transistor body/well to shift $V_{th}$ : forward bias for speed, reverse for leakage; strong on FD-SOI (2, 11).                       |
| <b>Boundary scan</b>      | IEEE 1149.1 shift-register cells at the I/Os for board interconnect test (10).                                                                       |
| <b>BSDL</b>               | Boundary Scan Description Language: describes a chip's 1149.1 implementation for board test tools (10).                                              |
| <b>BTI / NBTI</b>         | (Negative) bias temperature instability: $V_{th}$ shift of transistors under bias and heat, an ageing mechanism (21).                                |
| <b>Bump</b>               | Solder or copper-pillar connection of a flip-chip die to the package substrate or interposer (20).                                                   |
| <b>Burn-in</b>            | Operating parts at elevated voltage and temperature to screen out infant-mortality failures (21).                                                    |
| <b>CCS / ECSM</b>         | Current-source timing/noise/power models in Liberty, more accurate than NLDM at advanced nodes (4).                                                  |
| <b>CDC</b>                | Clock-domain crossing: a signal passing between asynchronous clocks; needs synchronisers and structural checks (6).                                  |
| <b>CDM / HBM (ESD)</b>    | Charged-device model and human-body model: the two standard ESD qualification tests (12, 22).                                                        |
| <b>Characterisation</b>   | SPICE simulation of cells or IP across slews, loads and PVT to build Liberty models (4).                                                             |
| <b>Chiplet</b>            | A die designed to be combined with others in one package over a die-to-die interface such as UCIe (20).                                              |
| <b>Clock gating (ICG)</b> | Stopping the clock to idle registers with an integrated clock-gating cell (latch + AND) to save dynamic power (11).                                  |

| Term                              | Definition                                                                                                                                 |
|-----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Clock latency</b>              | Delay from the clock source to a register clock pin: source latency (outside) plus network latency (the tree) (9, 16).                     |
| <b>Clock mesh</b>                 | Grid of shorted clock wires driven by many buffers; low skew and variation tolerance at the cost of power (16).                            |
| <b>Clock skew</b>                 | Difference in clock arrival time between two registers (9, 16).                                                                            |
| <b>Clock uncertainty</b>          | Margin in SDC for jitter and not-yet-built clock tree skew (9).                                                                            |
| <b>CMP</b>                        | Chemical-mechanical polishing: planarisation step; drives metal density rules and fill (3).                                                |
| <b>Congestion</b>                 | Routing demand exceeding available tracks in a region; leads to detours, DRCs and timing loss (15, 17).                                    |
| <b>Core-limited / pad-limited</b> | Die size set by the core logic area or by the number of I/O pads around the edge (14).                                                     |
| <b>Corner</b>                     | A combination of process, voltage and temperature (and RC extraction) used for analysis (3, 9).                                            |
| <b>Coupling capacitance</b>       | Capacitance between neighbouring wires; causes crosstalk delay and noise (17).                                                             |
| <b>CPF</b>                        | Common Power Format: Cadence-originated power-intent format, largely superseded by UPF (11).                                               |
| <b>Critical area</b>              | Layout area in which a defect of given size causes a fault; basis of yield modelling (21).                                                 |
| <b>Critical path</b>              | The path with the worst (least) slack (9).                                                                                                 |
| <b>Crosstalk</b>                  | Aggressor-net switching coupling into a victim: delay change or glitch (17).                                                               |
| <b>CRPR / CPPR</b>                | Clock reconvergence (common path) pessimism removal: credits back derate applied to the shared part of launch and capture clock paths (9). |
| <b>CTS</b>                        | Clock tree synthesis: building the buffered clock distribution to meet skew, latency and slew targets (16).                                |
| <b>Decap</b>                      | Decoupling capacitor cell between VDD and VSS that supplies local charge and reduces dynamic IR drop (14, 18).                             |
| <b>DEF</b>                        | Design Exchange Format: placed/routed design data (13, A).                                                                                 |
| <b>Defect density (D0)</b>        | Killer defects per unit area; with die area sets yield (21).                                                                               |
| <b>Delay calculation</b>          | Computing cell and wire delays from Liberty tables and parasitics before STA propagates them (9).                                          |
| <b>Derate</b>                     | Multiplier applied to delays to model on-chip variation or ageing (9).                                                                     |
| <b>DFM</b>                        | Design for manufacturability: layout practices and checks that improve yield (17).                                                         |
| <b>DFT</b>                        | Design for test: structures such as scan, BIST and test access that make manufacturing test possible (10).                                 |
| <b>Die</b>                        | One chip cut from a wafer (3).                                                                                                             |
| <b>dont_touch / dont_use</b>      | Tool attributes: do not modify an object / never pick a library cell (8).                                                                  |
| <b>DPPM</b>                       | Defective parts per million shipped; the quality metric test coverage drives (21).                                                         |
| <b>DRC</b>                        | Design rule check: geometric rule verification of layout against foundry rules (18).                                                       |
| <b>DRV</b>                        | Design-rule violations in the timing sense: max transition, max capacitance, max fanout (15).                                              |
| <b>DVFS</b>                       | Dynamic voltage and frequency scaling (11).                                                                                                |
| <b>Dynamic power</b>              | Power from charging and discharging capacitance ( $\alpha C V^2 f$ ) plus short-circuit current (2, 11).                                   |
| <b>ECO</b>                        | Engineering change order: a late, incremental netlist/layout change; metal-only ECOs reuse base layers (19).                               |
| <b>Electromigration (EM)</b>      | Metal atom transport by current, forming voids and hillocks; limited by current-density rules (18).                                        |
| <b>Elmore delay</b>               | First-order RC-tree delay: sum over resistors of R times downstream capacitance (2, 17).                                                   |

| Term                           | Definition                                                                                                               |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <b>End cap / boundary cell</b> | Cells at row ends that satisfy well and implant rules (14).                                                              |
| <b>Endpoint</b>                | A timing path end: register data pin or output port (9).                                                                 |
| <b>ERC</b>                     | Electrical rule check: floating gates, missing taps, shorted supplies and similar (18).                                  |
| <b>ESD</b>                     | Electrostatic discharge; I/O and supply clamps protect the core (12).                                                    |
| <b>EUV</b>                     | Extreme-ultraviolet lithography (13.5 nm wavelength) used at 7 nm-class nodes and below (3).                             |
| <b>False path</b>              | A structurally present path that can never be functionally exercised; excluded from timing (9).                          |
| <b>Fanout</b>                  | Number of inputs driven by one output (4).                                                                               |
| <b>Fault coverage</b>          | Fraction of modelled faults detected by the test set (10).                                                               |
| <b>Fault model</b>             | Abstraction of defects: stuck-at, transition, path delay, bridging, cell-aware (10).                                     |
| <b>FEOL</b>                    | Front end of line: transistor formation (wells, gates, source/drain) (3).                                                |
| <b>Filler cell</b>             | Cell that fills row gaps to keep wells and implants continuous (15).                                                     |
| <b>FinFET</b>                  | 3-D transistor whose channel is a vertical fin wrapped by the gate on three sides; mainstream from 22/16 nm to 3 nm (2). |
| <b>FIT</b>                     | Failures in time: failures per $10^9$ device-hours (21).                                                                 |
| <b>Flip-chip</b>               | Die mounted face down with bumps over its area (20).                                                                     |
| <b>Floorplan</b>               | Die/block shape, macro and pin placement, power plan and regions before placement (14).                                  |
| <b>FO4</b>                     | Delay of an inverter driving four copies of itself; a technology-neutral speed unit (2).                                 |
| <b>Foundry</b>                 | Company that manufactures wafers for others (TSMC, Samsung, GlobalFoundries, ...) (1, 3).                                |
| <b>GAA / nanosheet</b>         | Gate-all-around transistor with stacked horizontal sheets fully surrounded by the gate; 3 nm/2 nm class (2).             |
| <b>GDSII</b>                   | Binary stream format for final layout geometry (19, A).                                                                  |
| <b>Generated clock</b>         | A clock derived from another (dividers, muxes) declared with <code>create_generated_clock</code> (9).                    |
| <b>Glitch power</b>            | Power spent on spurious transitions before a node settles (11).                                                          |
| <b>Global routing</b>          | Coarse assignment of nets to routing tiles and layers before detailed routing (17).                                      |
| <b>GLS</b>                     | Gate-level simulation of the netlist, often with SDF (7).                                                                |
| <b>Guard ring</b>              | Diffusion ring tied to a supply that collects injected carriers; prevents latch-up and isolates noise (12).              |
| <b>Halo / keep-out</b>         | Placement-free margin around a macro (14).                                                                               |
| <b>Hard macro</b>              | Pre-laid-out block (memory, PLL, PHY) placed as a unit (4, 14).                                                          |
| <b>HBM (memory)</b>            | High Bandwidth Memory: stacked DRAM on an interposer next to the logic die (20).                                         |
| <b>HCI</b>                     | Hot-carrier injection: ageing of transistors by energetic carriers near the drain (21).                                  |
| <b>Hierarchical design</b>     | Implementing a chip as separately closed blocks integrated at top level with timing/physical models (13).                |
| <b>HKMG</b>                    | High-k dielectric and metal gate stack, introduced at 45/32 nm to cut gate leakage (2, 3).                               |
| <b>Hold time</b>               | Time data must remain stable after the capturing clock edge (9).                                                         |
| <b>HTOL</b>                    | High-temperature operating life test (JESD22-A108) used in qualification (22).                                           |
| <b>I/O ring</b>                | Row of pad cells around the die with supplies, ESD and level shifting (12, 14).                                          |
| <b>IDDQ</b>                    | Quiescent supply current test; detects defects that cause leakage (21).                                                  |
| <b>IJTAG</b>                   | IEEE 1687: network for accessing embedded instruments through JTAG (10).                                                 |
| <b>Interposer</b>              | Silicon, glass or organic layer with fine wiring that connects several dies side by side (2.5D) (20).                    |

| Term                        | Definition                                                                                                          |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------|
| <b>IP</b>                   | Intellectual property block licensed or reused: soft (RTL) or hard (layout) (4).                                    |
| <b>IR drop</b>              | Supply voltage loss across the power network resistance: static (average) and dynamic (transient) (14, 18).         |
| <b>Isolation cell</b>       | Clamps outputs of a powered-down domain to a known value (11).                                                      |
| <b>JTAG</b>                 | IEEE 1149.1 test access port (TCK, TMS, TDI, TDO, optional TRST) (10).                                              |
| <b>KGD</b>                  | Known good die: die fully tested before multi-die assembly (20, 21).                                                |
| <b>Latch-up</b>             | Parasitic PNP thyristor turn-on shorting VDD to VSS; prevented by taps and guard rings, tested per JESD78 (12, 22). |
| <b>Leakage power</b>        | Static power from subthreshold, gate and junction leakage (2, 11).                                                  |
| <b>LEC</b>                  | Logic equivalence checking between two representations (8).                                                         |
| <b>LEF</b>                  | Library Exchange Format: technology and abstract cell views (4, A).                                                 |
| <b>Level shifter</b>        | Cell translating a signal between voltage domains (11).                                                             |
| <b>Liberty</b>              | Text library format for timing, power and function (4, A).                                                          |
| <b>Lithography</b>          | Printing patterns on the wafer with light through masks (3).                                                        |
| <b>Logical effort</b>       | Method for sizing gate chains: delay = effort (g h) + parasitic (p) (2).                                            |
| <b>LVS</b>                  | Layout versus schematic: extracting the layout netlist and comparing it to the source netlist (18).                 |
| <b>MBIST</b>                | Memory BIST: march algorithms run by on-chip controllers, often with repair (10).                                   |
| <b>MCMM</b>                 | Multi-corner multi-mode analysis/optimisation (9, 15).                                                              |
| <b>MEOL / MOL</b>           | Middle of line: local contacts between transistors and the first metal (3).                                         |
| <b>Metal fill</b>           | Dummy metal added to meet density rules for CMP (17).                                                               |
| <b>Metal stack</b>          | The set of metal layers, their thickness and pitch (3).                                                             |
| <b>Metastability</b>        | Unresolved intermediate state of a flop after a setup/hold violation (6).                                           |
| <b>MPW</b>                  | Multi-project wafer: many designs share one mask set (1, 24).                                                       |
| <b>Multi-bit flop</b>       | Several flops sharing clock circuitry in one cell; saves clock power and area (11, 15).                             |
| <b>Multi-patterning</b>     | Splitting a layer over several masks (LELE, SADP, SAQP) to print pitches below the single-exposure limit (3).       |
| <b>Multi-Vt</b>             | Libraries with low/standard/high threshold flavours to trade speed against leakage (4, 11).                         |
| <b>Multicycle path</b>      | Path allowed more than one clock period (9).                                                                        |
| <b>NDR</b>                  | Non-default routing rule: wider/spaced wires for clocks and critical nets (16).                                     |
| <b>Netlist</b>              | Connection description of cells and nets (8).                                                                       |
| <b>NLDM</b>                 | Non-linear delay model: 2-D table lookup of delay and slew (4).                                                     |
| <b>NRE</b>                  | Non-recurring engineering cost: design, masks, IP, tools (1, 23).                                                   |
| <b>OASIS</b>                | Compact successor to GDSII for layout data (SEMI P39) (19).                                                         |
| <b>OCV</b>                  | On-chip variation: derates for local differences in delay (9).                                                      |
| <b>OPC</b>                  | Optical proximity correction: pre-distorting mask shapes so the printed shape matches the design (3, 19).           |
| <b>Operating mode</b>       | Functional or test configuration with its own constraints (functional, scan shift, capture, sleep) (9).             |
| <b>OTP / eFuse</b>          | One-time programmable storage for trim, IDs, keys and memory repair (12, 21).                                       |
| <b>Parasitic extraction</b> | Computing R, C (and sometimes L) of the layout interconnect (17, 18).                                               |
| <b>PDK</b>                  | Process design kit: rules, device models, decks, tech files and often libraries for a process (3).                  |



| Term                         | Definition                                                                                                |
|------------------------------|-----------------------------------------------------------------------------------------------------------|
| <b>Pin access</b>            | Ability of the router to connect to cell pins without violations; a key issue at advanced nodes (15, 17). |
| <b>PLL</b>                   | Phase-locked loop: clock generation and multiplication (12).                                              |
| <b>Power domain</b>          | Group of logic sharing a supply and power state (11).                                                     |
| <b>Power gating</b>          | Switching off a domain's supply with header/footer switches (11).                                         |
| <b>Power grid</b>            | Rings, straps, rails and vias that distribute supply (14).                                                |
| <b>PPA</b>                   | Power, performance, area: the three competing targets (1, 5).                                             |
| <b>Probe card</b>            | Fixture of needles contacting die pads at wafer sort (21).                                                |
| <b>PVT</b>                   | Process, voltage, temperature (2, 9).                                                                     |
| <b>Qualification</b>         | Stress testing to prove a product meets reliability requirements (JESD47, AEC-Q100) (22).                 |
| <b>RC corner</b>             | Extraction corner: Cworst, Cbest, RCworst, RCbest, typical (9, 18).                                       |
| <b>RDL</b>                   | Redistribution layer: extra metal that reroutes pads to bump locations (20).                              |
| <b>Recovery / removal</b>    | Setup- and hold-like checks for asynchronous reset/set deassertion relative to the clock (9).             |
| <b>Redundant via</b>         | Second via inserted in parallel to improve yield and EM (17).                                             |
| <b>Retention flop</b>        | Flop with a shadow latch on an always-on supply that keeps state while the domain is off (11).            |
| <b>Reticle</b>               | The mask field printed per exposure, at most about 26 x 33 mm on the wafer (3).                           |
| <b>Row / site</b>            | Horizontal placement rows made of unit sites; cells are multiples of the site width (13).                 |
| <b>SAIF</b>                  | Switching Activity Interchange Format (11, A).                                                            |
| <b>Scan chain</b>            | Flops connected as a shift register in test mode (10).                                                    |
| <b>Scan compression</b>      | On-chip decompressor/compactor that feeds many short chains from few pins (10).                           |
| <b>SDC</b>                   | Synopsys Design Constraints (8, 9, A).                                                                    |
| <b>SDF</b>                   | Standard Delay Format (IEEE 1497) (9, A).                                                                 |
| <b>Seal ring</b>             | Metal and via ring around the die against cracks and moisture (19).                                       |
| <b>Self-heating</b>          | Local temperature rise in FinFET/GAA devices that worsens EM and ageing (18).                             |
| <b>SER</b>                   | Soft error rate: upsets from alpha particles and neutrons, in FIT (21).                                   |
| <b>Setup time</b>            | Time data must be stable before the capturing clock edge (9).                                             |
| <b>Shielding</b>             | Grounded wires beside a sensitive net to block crosstalk (16, 17).                                        |
| <b>Shmoo plot</b>            | Pass/fail map over two parameters, such as voltage vs frequency (22).                                     |
| <b>Sign-off</b>              | Final verification with golden tools and decks before tapeout (18).                                       |
| <b>Signal integrity (SI)</b> | Crosstalk, noise and IR effects on signal timing and levels (17).                                         |
| <b>Slack / WNS / TNS</b>     | Required minus arrival time; worst negative and total negative slack (9).                                 |
| <b>Slew (transition)</b>     | Signal rise/fall time, typically 10-90 % or 20-80 % (4).                                                  |
| <b>SoC</b>                   | System on chip: processors, memory, interconnect and peripherals on one die (1).                          |
| <b>Spare cells</b>           | Unused gates sprinkled in the layout for metal-only ECOs (19).                                            |
| <b>SPEF</b>                  | Standard Parasitic Exchange Format (IEEE 1481) (17, A).                                                   |
| <b>SPICE</b>                 | Transistor-level circuit simulator (HSPICE, Spectre, ngspice) (2, 4).                                     |
| <b>SRAM compiler</b>         | Generator of memory macros of chosen size with all views (4).                                             |
| <b>STA</b>                   | Static timing analysis: exhaustive path-based timing without vectors (9).                                 |
| <b>Standard cell</b>         | Pre-designed fixed-height logic cell (4).                                                                 |

| Term                         | Definition                                                                                                                    |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| <b>STIL</b>                  | Standard Test Interface Language (IEEE 1450) for test patterns (10, 21).                                                      |
| <b>Stuck-at fault</b>        | A node permanently at 0 or 1 (10).                                                                                            |
| <b>Substrate noise</b>       | Switching noise coupled through the silicon substrate into sensitive analog blocks (12).                                      |
| <b>Tap cell (well tap)</b>   | Cell connecting n-well and p-substrate to supplies at regular spacing; prevents latch-up (14).                                |
| <b>Tapeout</b>               | Release of final layout data to the foundry (19).                                                                             |
| <b>TDDB</b>                  | Time-dependent dielectric breakdown of gate or inter-metal dielectric (21).                                                   |
| <b>Temperature inversion</b> | At low supply voltages, cells get faster when hot because $V_{th}$ drops more than mobility; worst corner may be cold (2, 9). |
| <b>Tie cell</b>              | Cell providing constant 0/1 so gate inputs are not tied directly to rails (15).                                               |
| <b>Timing arc</b>            | Pin-to-pin delay or check relation in a cell (4, 9).                                                                          |
| <b>Timing closure</b>        | Iterating until all timing checks pass in all scenarios (9, 15).                                                              |
| <b>Timing window</b>         | Interval in which a net may switch; used to filter crosstalk aggressors (17).                                                 |
| <b>Track</b>                 | Legal routing centre line on a metal layer; cell height is quoted in tracks (4).                                              |
| <b>TSV</b>                   | Through-silicon via for vertical connection in 3-D stacks (20).                                                               |
| <b>UPF</b>                   | Unified Power Format, IEEE 1801 (11, A).                                                                                      |
| <b>Useful skew</b>           | Intentional clock skew that borrows time from a path with slack (16).                                                         |
| <b>Utilisation</b>           | Cell area divided by available placement area (14).                                                                           |
| <b>VCD</b>                   | Value Change Dump waveform format (IEEE 1364) (A).                                                                            |
| <b>Via</b>                   | Vertical connection between metal layers (3).                                                                                 |
| <b><math>V_{th}</math></b>   | Threshold voltage of a MOSFET (2).                                                                                            |
| <b>Wafer</b>                 | Silicon disc (300 mm today for leading nodes) on which dies are built (3).                                                    |
| <b>Wafer sort</b>            | Testing each die on the wafer before dicing (21).                                                                             |
| <b>Wire-load model</b>       | Statistical fanout-based wire capacitance estimate used by synthesis before layout; replaced by physical synthesis (8).       |
| <b>Yield</b>                 | Fraction of good dies (or parts) - line, die and test yield (21).                                                             |

## Appendix C. 100 ASIC Design Interview Questions with Answers

Grouped from device physics to project economics, ending with a few puzzles. Numeric answers are worked out step by step (and were checked in Python); try each one on paper first. Chapter numbers point to the full treatment.

### Device physics and technology

#### Q1. Why does a MOSFET leak when it is "off"?

Below  $V_{th}$  the channel current does not stop; it falls exponentially with  $V_{gs}$  (subthreshold conduction). Add gate tunnelling through thin oxide, junction leakage and GIDL. Subthreshold leakage dominates in most logic and grows exponentially as  $V_{th}$  is lowered or temperature rises (Chapter 2).

#### Q2. What is the subthreshold slope and why can it not beat 60 mV/decade at room temperature?

It is the gate voltage needed to change subthreshold current by 10x:  $S = n (kT/q) \ln 10$ . With the ideal body factor  $n = 1$  at 300 K,  $S = 0.0259 \text{ V} \times 2.303 = 59.5 \text{ mV/dec}$ . Real planar devices have  $n$  of roughly 1.2-1.5 (70-90 mV/dec); FinFET and GAA get closer to 60 through better gate control. Beating it needs a different injection mechanism (tunnel FETs, negative capacitance).

#### Q3. Lowering $V_{th}$ by 40 mV on a process with $S = 80 \text{ mV/dec}$ : what happens to leakage?

Leakage scales by  $10^{dV_{th}/S} = 10^{40/80} = 10^{0.5} = 3.16x$ . This is why an LVT cell can leak several times more than the SVT equivalent while being only 10-20 % faster.

#### Q4. With the alpha-power law ( $\alpha = 1.3$ , $V_{th} = 0.3 \text{ V}$ ), how much slower is a gate at 0.7 V than at 0.8 V?

Delay is roughly proportional to  $C V / I_{on}$  and  $I_{on} \sim (V - V_{th})^{\alpha}$ . Current ratio =  $(0.5/0.4)^{1.3} = 1.337$ . Delay ratio =  $(0.7/0.8) \times 1.337 = 1.17$ , so about 17 % slower for a 12.5 % voltage cut, while dynamic energy per operation drops by  $(0.7/0.8)^2 = 0.77$ .

#### Q5. Why did the industry move from planar to FinFET and then to GAA?

Short-channel effects: as gate length shrank, the drain started controlling the channel (DIBL,  $V_{th}$  roll-off, high leakage). A fin wrapped on three sides restores electrostatic control; nanosheets wrapped on all four sides improve it further and allow a continuous choice of sheet width instead of quantised fin counts.

#### Q6. What is temperature inversion and how does it change sign-off?

At low supply voltage, the  $V_{th}$  decrease with temperature dominates the mobility decrease, so cells become **faster** when hot. The slow corner is then cold (for example -40 C) instead of hot, so sign-off must include both temperature extremes at the slow process corner.

#### Q7. What is the difference between global (die-to-die) and local (within-die) variation?

Global variation shifts all devices on a die together and is covered by PVT corners. Local variation (random dopant fluctuation, line-edge roughness, layout-dependent effects) differs between neighbouring transistors and is covered by OCV, AOCV or POCV/LVF derates in STA (Chapters 3, 9).

#### Q8. Name three BEOL reliability mechanisms and what the designer does about each.

Electromigration (keep current density under the rules: widen wires, add vias), TDDB of inter-metal dielectrics (respect spacing/voltage rules), and stress migration/via voiding (use redundant vias). Device-level: BTI and HCI - handled with ageing derates or aged libraries.

## CMOS circuits and logic

### Q9. Why are NAND gates preferred over NOR gates in CMOS?

In a NAND the slow PMOS devices are in parallel and the fast NMOS in series; in a NOR the PMOS are in series. Series PMOS must be made very wide for equal drive, so a NOR is larger and slower (logical effort 5/3 vs 4/3 for 2 inputs).

### Q10. Size an inverter chain to drive a load 64 times its input capacitance. How many stages and what delay?

Path effort  $F = 64$  (inverters:  $g = 1$ , no branching). Best stage effort is about 4, so  $N = \log_4 64 = 3$  stages with stage effort 4 (sizes 1, 4, 16). Delay =  $N(f + p) = 3 \times (4 + 1) = 15 \tau$ , i.e. 3 FO4 delays.

### Q11. What is FO4 delay and why is it useful?

The delay of an inverter driving four identical inverters. It normalises speed across processes: a pipeline stage of about 12-20 FO4 is typical for high-performance cores, 30-60 FO4 for low-power designs.

### Q12. Write the dynamic power equation and name each term.

$P_{\text{dyn}} = \alpha \times C \times V^2 \times f$  (+ short-circuit power).  $\alpha$  is the activity (probability of a 0->1 transition per cycle),  $C$  the switched capacitance,  $V$  the supply and  $f$  the clock frequency. A clock net has  $\alpha = 1$  in this convention (one rising transition per cycle).

### Q13. A block switches 500 pF with $\alpha = 0.15$ at 1 GHz and 0.8 V. What is its dynamic power, and what happens at 0.72 V and 0.9 GHz?

$0.15 \times 500 \times 10^{-12} \times 0.64 \times 1 \times 10^9 = 48 \text{ mW}$ . Scaling:  $(0.72/0.8)^2 \times (0.9/1) = 0.81 \times 0.9 = 0.729$ , so about 35 mW (27 % lower). This is the core argument for DVFS.

### Q14. What is latch-up and how is it prevented?

The parasitic PNP and NPN bipolar transistors formed by the wells make a thyristor; triggered by injection it shorts VDD to VSS. Prevention: frequent well and substrate taps (tap cells), guard rings around I/O, spacing rules, and qualification per JESD78.

### Q15. What does a standard-cell's "track height" mean?

Cell height expressed in routing tracks of the lowest horizontal metal, e.g. a 9-track library at 0.34  $\mu\text{m}$  pitch would be about 3  $\mu\text{m}$  high. Fewer tracks = denser but weaker cells and harder pin access; advanced nodes use 5-7 track libraries.

### Q16. Why is a flip-flop's setup time sometimes negative?

Setup is measured at the cell pins. If the internal clock path to the master latch is slower than the data path, data may arrive after the external clock edge and still be captured. Negative hold is similarly common. The real SKY130 dfrtp\_1 table in Appendix A shows both.

## Logic synthesis

### Q17. What are the main steps inside logic synthesis?

Elaboration (RTL to generic operators and registers), high-level optimisation (constant propagation, resource sharing, FSM extraction), technology-independent logic optimisation, technology mapping to library cells, then timing/area/power optimisation (sizing, buffering, restructuring) and DFT/clock-gating insertion (Chapter 8).

### Q18. What is a wire-load model, and why do modern flows avoid it?

A table from fanout to estimated wire capacitance/resistance. It ignores placement, so at deep-submicron nodes (wires dominate delay) it is badly wrong. Physical (topographical/placement-aware) synthesis uses the floorplan and a quick placement instead.

### Q19. Synthesis meets timing but the design fails after placement. List likely causes.

Optimistic wire model; high-fanout nets not buffered; macros far apart in the floorplan; congestion forcing detours; missing `set_load/set_driving_cell`; clock uncertainty too small for real CTS skew; too-high utilisation leaving no room for buffering.

#### Q20. What is a `dont_use` list and why is it needed?

A list of library cells synthesis and P&R may not use: cells with poor characterisation, very weak or very strong drives, special cells (clock-only, isolation, delay, low-power-only) or cells with known DFM issues. Without it tools happily pick them - Appendix A shows Yosys using an isolation cell for ordinary logic.

#### Q21. How does synthesis insert clock gating, and what must the designer check?

It finds registers with a common enable (`if (en) q <= d`) and replaces the feedback mux with an ICG cell on their clock. Check the minimum bank size, the clock-gating setup/hold check, that test mode forces the ICG enable (test\_en pin), and that CTS treats the ICG correctly.

#### Q22. What is retiming?

Moving registers across combinational logic without changing cycle-level behaviour at the outputs, to balance stage delays. It changes state encoding, so LEC needs sequential-equivalence support or guidance, and DFT/debug must accept renamed flops.

#### Q23. What does logic equivalence checking prove, and what does it not?

That two designs (RTL and netlist, or netlist and netlist) produce the same outputs and next-states for all inputs, after mapping registers (key points). It does not check timing, and it trusts the constraints you give it - a wrong constant on a test pin can hide a real difference.

#### Q24. Why can unintended latches appear, and why are they a problem in an ASIC flow?

Incomplete assignment in a combinational block (missing else/default). Latches complicate STA (time borrowing), DFT (not scannable, need transparency control) and CDC analysis, and usually indicate a bug.

#### Q25. What is multi-Vt optimisation?

Using high-Vt cells everywhere by default (low leakage) and swapping to standard/low-Vt only on critical paths. Typical results: large leakage savings for a few % of cells on low Vt. Done in synthesis and again after routing (leakage recovery).

## Static timing analysis

#### Q26. State the setup and hold inequalities with clock latencies.

Setup:  $L_{\text{launch}} + t_{\text{cq}} + t_{\text{data,max}} \leq T + L_{\text{capture}} - t_{\text{setup}} - U_{\text{setup}}$ . Hold:  $L_{\text{launch}} + t_{\text{cq}} + t_{\text{data,min}} \geq L_{\text{capture}} + t_{\text{hold}} + U_{\text{hold}}$ . Slack = required - arrival (setup) or arrival - required (hold).

#### Q27. Tclk = 2.0 ns, launch latency 0.50 ns, capture latency 0.42 ns, t\_cq = 0.15 ns, logic = 1.40 ns, t\_setup = 0.10 ns, setup uncertainty 0.05 ns. What is the setup slack?

Arrival = 0.50 + 0.15 + 1.40 = 2.05 ns. Required = 2.0 + 0.42 - 0.10 - 0.05 = 2.27 ns. Slack = 2.27 - 2.05 = **+0.22 ns** (met). Note that the capture clock arriving 80 ps early costs 80 ps.

#### Q28. Launch latency 0.40 ns, capture latency 0.55 ns, t\_cq\_min = 0.12 ns, min logic 0.03 ns, t\_hold = 0.08 ns, hold uncertainty 0.03 ns. Hold slack? How do you fix it?

Arrival = 0.40 + 0.12 + 0.03 = 0.55 ns. Required = 0.55 + 0.08 + 0.03 = 0.66 ns. Slack = 0.55 - 0.66 = **-0.11 ns** (violation).

Fix: add at least 110 ps of delay on the data path (delay cells or buffers at the capture end, checking that the setup of that path still has room), or reduce the capture-clock lateness. Changing the clock period does not help.

#### Q29. t\_cq = 80 ps, worst logic = 620 ps, t\_setup = 50 ps and the capture clock arrives 30 ps later than the launch clock. What is f\_max?

$T_{\min} = 80 + 620 + 50 - 30 = 720$  ps, so  $f_{\max} = 1/720$  ps = 1.39 GHz. The positive (useful) skew helped setup by 30 ps - and costs 30 ps of hold margin on the same pair.

**Q30. What is CRPR? Compute it for a 0.6 ns common clock path with late/early derates of 1.08/0.92.**

OCV derates make the shared part of the launch and capture clock paths simultaneously slow and fast, which is physically impossible. The removed pessimism =  $0.6 \times (1.08 - 0.92) = 0.096$  ns, added back to the slack.

**Q31. With the same derates, Tclk = 1.0 ns, launch clock = 0.6 (common) + 0.2 ns, data (t\_cq + logic) = 0.55 ns, capture clock = 0.6 + 0.25 ns, setup 0.05 ns. Setup slack with and without CRPR?**

Late arrival =  $(0.8 + 0.55) \times 1.08 = 1.458$  ns. Early required =  $1.0 + 0.85 \times 0.92 - 0.05 = 1.732$  ns. Slack = 0.274 ns; with CRPR (+0.096) = **0.370 ns**.

**Q32. What happens to hold when you write `set_multicycle_path 2 -setup`?**

The setup capture edge moves one cycle later, and the hold check (by default one edge before the setup edge) moves with it, so hold is now checked against the edge one cycle after launch - a much harder check. Add `set_multicycle_path 1 -hold` (same from/to) to move it back to the launch edge.

**Q33. A path launches on a rising edge and captures on the falling edge of a 1 GHz, 50 % duty clock. How much time does it have?**

Half a period, 0.5 ns, minus setup and uncertainty; duty-cycle distortion reduces it further. Such half-cycle paths appear with negative-edge flops and latch-based designs.

**Q34. Explain time borrowing with latches.**

A positive latch is transparent while the clock is high. Data arriving after the opening edge passes straight through; the excess is borrowed from the next stage, up to the closing edge minus setup. STA reports the borrowed time and checks the next path with the reduced budget.

**Q35. Why do we need both max (Cworst/RCworst) and min (Cbest/RCbest) extraction corners?**

Setup is worst with slow cells and maximum wire delay; hold is worst with fast data and maximum clock skew, which can come from a different RC corner. Coupling-dominated wires are worst at Cw, resistance-dominated long wires at RCw.

**Q36. What is SI (crosstalk) delay and how is it bounded in STA?**

When an aggressor switches opposite to the victim, the coupling capacitance is effectively doubled (Miller), slowing the victim; same-direction switching speeds it up. STA computes delta delays using timing windows so only aggressors that can overlap are counted; setup uses slowdowns, hold uses speedups.

**Q37. What are the main timing checks besides setup and hold?**

Recovery/removal (async reset release), minimum pulse width, clock-gating setup/hold, data-to-data checks, max transition/capacitance/fanout, and for latches borrow limits.

## Low power

**Q38. List the low-power techniques in order of typical impact and implementation cost.**

Architecture (do less work, lower frequency, parallelism + lower V) > power gating and DVFS > clock gating (almost free, big dynamic saving) > multi-Vt and cell sizing > operand isolation, memory banking and low-power data encoding.

**Q39. What cells does a switchable domain need at its boundary?**

Isolation cells on outputs going to powered-on logic (clamp 0/1 or latch), level shifters if voltages differ, retention flops for state to keep, power switches (header or footer), and always-on buffers for control signals that route through it.

**Q40. Header or footer switches - which is more common and why?**

Headers (PMOS between VDD and the virtual VDD) are more common: the domain's ground stays shared, which simplifies level shifting, substrate connections and multiple-voltage schemes. Footers (NMOS) are smaller for the same resistance but make the virtual ground float.

#### Q41. Why must a power-gated domain be woken up gradually?

Turning all switches on at once charges the whole domain capacitance instantly; the in-rush current causes a large IR drop on the always-on neighbours. Switches are daisy-chained (a weak trickle chain, then the main chain) to limit  $di/dt$ .

#### Q42. What does UPF describe that RTL cannot?

Supply networks, which logic is in which domain, when domains are on or off (power states), and the protection strategies (isolation, retention, level shifting, switches). This keeps the RTL power-agnostic and lets simulation, synthesis and P&R share one intent file.

#### Q43. A chip's leakage is fine at 25 C but too high at 125 C. Why, and what can you do?

Subthreshold leakage grows exponentially with temperature ( $V_{th}$  falls and  $kT/q$  rises) - often an order of magnitude or more between room temperature and 125 C. Options: more high- $V_t$  cells, power gating idle blocks, body bias, lower standby voltage, better cooling to keep junction temperature down.

#### Q44. Why is glitch power significant in arithmetic blocks?

Unequal path depths in adders and multipliers make intermediate nodes toggle several times per cycle. Balanced trees, pipelining and operand isolation reduce it; vector-based power analysis with timing (not zero-delay) is needed to see it.

#### Q45. What is the difference between vectorless and vector-based power analysis?

Vectorless assumes default activity (for example 10-20 % toggle rate) propagated statistically - quick but approximate. Vector-based uses real activity from simulation or emulation (SAIF for average, FSDB/VCD for time-based peak power) - accurate for the chosen workload only.

## Design for test

#### Q46. How does a scan flop work?

A mux in front of D selects functional data or the scan input (scan\_enable). In shift mode flops form shift registers to load/unload state; in capture mode one or two functional clocks capture the response.

#### Q47. 10 000 flops in 100 balanced chains, 2 000 patterns, shift at 50 MHz. Estimate the test time.

Chain length  $L = 100$ . With load of the next pattern overlapping unload of the previous one: cycles =  $P(L + 1) + L = 2000 \times 101 + 100 = 202\,100$  cycles, i.e.  $202\,100 / 50 \text{ MHz} = 4.04 \text{ ms}$ .

#### Q48. Why is scan compression used, and roughly how does it work?

Test time and tester memory scale with chain length  $\times$  patterns. A decompressor expands a few scan-in pins into hundreds of short internal chains and a compactor (XOR tree/MISR) merges their outputs, giving 10-100x+ reductions. X values must be masked or they corrupt the compacted response.

#### Q49. Stuck-at vs transition fault models: what does each detect?

Stuck-at models a node fixed at 0 or 1 (static defects). Transition (at-speed) faults model slow-to-rise/fall nodes and need launch-on-capture or launch-on-shift patterns at functional speed; they catch resistive opens and delay defects that stuck-at tests miss.

#### Q50. Yield is 90 % and fault coverage is 99 %. Estimate the defect level; what if coverage were 99.9 %?

Williams-Brown:  $DL = 1 - Y^{(1 - T)} = 1 - 0.9^{0.01} = 0.00105$ , about **1 050 DPPM**. At  $T = 99.9 \%$ :  $1 - 0.9^{0.001} = 105 \text{ DPPM}$ . Each extra nine of coverage cuts escapes about 10x, which is why automotive products demand very high coverage plus cell-aware and IDDQ tests.

#### Q51. What are lock-up latches for?



When a scan chain crosses from one clock domain (or tree branch) to another with skew, a negative-level latch inserted between them adds half a cycle of hold margin during shift.

#### Q52. What does MBIST do that scan does not?

Memories are not made of scan flops. MBIST controllers run march algorithms (e.g. March C-) at speed to detect cell, address-decoder and coupling faults, and with repair analysis they program redundant rows/columns via fuses (BISR).

#### Q53. What is IEEE 1500 and how does it relate to 1149.1 and 1687?

1500 defines a wrapper around embedded cores (wrapper boundary register, WIR) so each core can be tested in isolation. 1149.1 provides chip-level test access and boundary scan; 1687 (JTAG) describes networks of embedded instruments reachable through that access port.

## Floorplanning and power grid

#### Q54. What are the inputs and outputs of floorplanning?

Inputs: netlist, SDC, LEF/Liberty, UPF, I/O or pin list, package bump map, target utilisation. Outputs: die/core size, I/O and pin placement, macro placement, power grid, blockages, domain regions, tap/end-cap cells - a floorplan DEF (Chapter 14).

#### Q55. How do you place memories (hard macros) in a block?

Usually at the periphery, pins facing the core logic, aligned to the power grid and track pitch, with halos and routing channels sized by the number of pins; group macros by data-flow; avoid narrow channels and notches that trap cells.

#### Q56. A block draws 1.25 A (1 W at 0.8 V). What maximum effective grid resistance keeps static IR drop at 5 %?

5 % of 0.8 V = 40 mV.  $R_{\max} = 0.040 / 1.25 = \mathbf{32 \text{ milliohm}}$  from the bumps to the worst cell. Dynamic drop needs decap and grid inductance on top of this.

#### Q57. How much decap is needed to hold a 1 A current step for 1 ns with at most 50 mV droop (ignoring the grid)?

$C = I \, dt / dV = 1 \times 1\text{e-}9 / 0.05 = \mathbf{20 \text{ nF}}$ . That is a lot of on-die decap - which is why package and board decoupling handle the slower part of the step and on-die decap only the first nanoseconds.

#### Q58. Why is the power grid usually drawn on the top metals?

Top metals are thick with low sheet resistance and wide pitch, so they carry current across the die with little drop; lower metals are reserved for signal routing, with stacked vias bringing power down to the cell rails.

#### Q59. What is utilisation and what target would you start with?

Standard-cell area divided by available placement area. Typical starting points are about 60-75 %: higher saves area but leaves no room for buffering, CTS and ECOs and increases congestion.

#### Q60. What is the difference between a pad-limited and a core-limited die?

Pad-limited: the perimeter needed for the I/O pads sets the die size and the core has spare area (consider staggered pads, flip-chip or area I/O). Core-limited: the logic sets the size and the pads fit easily.

## Placement and clock tree synthesis

#### Q61. What happens during placement?

Global placement (analytic/quadratic wirelength and density optimisation), legalisation onto rows and sites, detailed placement, then timing-driven optimisation: sizing, buffering, Vt swap, restructuring, plus scan re-ordering and congestion repair (Chapter 15).

#### Q62. What are the targets of CTS?



Low skew and insertion delay within limits, clean transitions (max slew), low power, robustness to variation, and meeting setup/hold after propagation. Structure options: balanced trees (H-tree), mesh, spine, multi-source CTS.

**Q63. Compute the Elmore delay of a clock branch: driver resistance 1 kohm, two segments of 200 ohm and 20 fF each, and a 10 fF sink at the end.**

Each resistor times all capacitance downstream of it:  $1000 \times (20 + 20 + 10) \text{ fF} + 200 \times (20 + 20 + 10) \text{ fF} + 200 \times (20 + 10) \text{ fF} = 50 + 10 + 6 \text{ ps} = \mathbf{66 \text{ ps}}$ .

**Q64. Why does CTS use non-default routing rules for clock nets?**

Double width lowers resistance and EM risk; double spacing or shielding reduces coupling, making the clock less sensitive to crosstalk and variation. The cost is routing resources.

**Q65. Why do clock trees use inverters rather than buffers at advanced nodes?**

An inverter is one stage (a buffer is two), giving less delay and less variation per level, and pairs of inverters keep rise/fall duty cycle balanced.

**Q66. What is useful skew and when is it dangerous?**

Delaying the capture clock of a critical path (or advancing launch) to lend it time from neighbouring paths with slack. It is dangerous for hold (the same skew hurts the hold check) and across corners, where the skew can scale differently.

**Q67. Why is hold fixing done after CTS and not before?**

Hold depends on actual clock skew; before CTS the clocks are ideal and hold numbers are meaningless. After CTS (propagated clocks) real hold violations appear and are fixed while protecting setup.

**Q68. What is a clock-gating check?**

For an AND-type gate on a clock, the enable must be stable while the clock is high: STA checks enable setup before the rising edge and hold after the falling edge (the reverse for OR-type). ICG cells have these checks in their Liberty.

## Routing, signal integrity and DFM

**Q69. What are the stages of routing?**

Global routing (assign nets to tiles and layers, estimate congestion), track assignment, detailed routing (exact wires and vias obeying design rules), and search-and-repair iterations to remove DRCs.

**Q70. What is the resistance and approximate delay of a 1 mm met1 wire 0.14 um wide at 0.125 ohm/sq with 0.2 fF/um?**

$R = 0.125 \times 1000 / 0.14 = 893 \text{ ohm}$ ;  $C = 200 \text{ fF}$ . Distributed RC: Elmore delay  $RC/2 = 89 \text{ ps}$ , 50 % step-response delay about  $0.38 RC = 68 \text{ ps}$  (plus driver and load terms). Long wires belong on thicker upper metals with repeaters.

**Q71. Why does wire delay grow quadratically with length and how do repeaters help?**

Both  $R$  and  $C$  are proportional to length, so  $RC \sim L^2$ . Splitting the wire into  $k$  repeated segments makes each segment's  $RC \sim (L/k)^2$ ; the total becomes linear in  $L$  with an optimum repeater spacing and size.

**Q72. What is the antenna effect and how is it fixed?**

During manufacture, a long metal wire connected only to a gate (the driver's diffusion is connected through higher metal not yet built) collects plasma charge that can damage the gate oxide. Fixes: layer hopping (jumper to a higher metal near the gate) or an antenna diode near the gate.

**Q73. How do you fix a crosstalk noise (glitch) violation?**

Increase victim drive strength, reduce aggressor drive/slew, increase spacing, shield with ground wires, move to another layer, or insert a buffer to shorten the coupled length.

**Q74. Why do advanced nodes need multi-patterning and what does it mean for routing?**

193 nm immersion lithography cannot print pitches below about 80 nm in one exposure, so a layer is split over two or more masks (LELE) or made with spacers (SADP/SAQP). Routers must respect colouring, unidirectional layers, line-end and via rules, which reduces flexibility.

**Q75. What is metal fill and why re-time after fill?**

Dummy shapes added to meet min/max density windows for uniform CMP. Fill adds coupling capacitance, so sign-off extraction and STA must include it (or model it); timing-aware fill keeps distance from critical nets.

## Sign-off and physical verification

**Q76. What does LVS compare and what are typical LVS errors?**

It extracts devices and connectivity from the layout and compares them with the source netlist. Typical errors: shorts (often power-ground or label shorts), opens, missing or extra devices, wrong device sizes, unconnected wells/substrate, mismatched pin names on macros.

**Q77. What is the difference between DRC in the P&R tool and sign-off DRC?**

The router uses LEF rules (a simplified subset). Sign-off DRC runs the foundry's full rule deck on the merged GDS including cell internals, IP, fill and seal ring. Clean in P&R does not mean clean at sign-off.

**Q78. What is an MCMM scenario count, and how do you keep it manageable?**

Scenarios = modes x corners (PVT x RC). A modern SoC can have dozens to hundreds. Tools prune dominated scenarios, and teams pick a representative subset for optimisation and run the full set only at sign-off.

**Q79. What is dynamic IR drop and why is it harder than static?**

The transient droop when many cells switch simultaneously; it depends on the switching pattern, decap placement and package inductance, so it needs vectors (or vectorless worst-case estimates) and time-domain analysis instead of average current.

**Q80. An EM limit is 2.8 mA/um of width for DC current in met1. What average current can a 0.14 um wire carry?**

$2.8 \times 0.14 = 0.39 \text{ mA}$ , at the rule's reference temperature. Higher temperature or longer lifetime targets reduce the limit; wider wires or parallel straps raise the allowed current.

**Q81. Why are ERC checks needed when DRC and LVS are clean?**

Some electrical errors are geometrically legal and match the netlist: floating gates, wells without taps, gate oxide exposed to the wrong voltage domain, missing ESD paths. ERC catches these.

**Q82. What is an ECO and what is a metal-only ECO?**

A late incremental change to the netlist and layout. After base layers are fabricated, a metal-only ECO rewires pre-placed spare cells (or gate-array filler) using only metal/via masks, saving time and mask cost (Chapter 19).

**Q83. What does the foundry typically require with the tapeout database?**

The final GDSII/OASIS with top cell name, layer map, DRC/LVS reports with the exact deck version, a completed tapeout form (node, masks, optional layers, die size, frame), waivers, and for MPW the slot and die coordinates.

## Packaging, manufacturing test, yield and reliability

### Q84. How many 100 mm<sup>2</sup> dies fit on a 300 mm wafer (gross)?

Common estimate:  $DPW = \pi (d/2)^2 / A - \pi d / \sqrt{2A} = 70\,686 / 100 - 942.5 / 14.14 = 706.9 - 66.6 = \text{about } 640$  (ignoring scribe lines and edge exclusion).

### Q85. With $D_0 = 0.1$ defects/cm<sup>2</sup>, what is the Poisson yield of a 1 cm<sup>2</sup> die and a 2 cm<sup>2</sup> die?

$Y = \exp(-A D_0)$ :  $\exp(-0.1) = 90.5\%$  and  $\exp(-0.2) = 81.9\%$ . Larger dies lose yield exponentially - a key argument for chiplets. Negative-binomial models with clustering give somewhat higher yields.

### Q86. A wafer costs \$10 000 and has 640 gross 1 cm<sup>2</sup> dies at 90.5 % yield. What is the cost per good die before test and packaging?

Good dies =  $640 \times 0.905 = 579$ ; cost =  $10\,000 / 579 = \$17.27$ .

### Q87. What is the difference between wafer sort and final test?

Wafer sort (probe) tests each die on the wafer with a probe card to avoid packaging bad dies; final test runs after packaging, often at temperature, with fuller coverage, speed binning and package-related tests (opens/shorts of bumps and balls).

### Q88. HTOL at 125 C must represent use at 55 C. With $E_a = 0.7$ eV, what is the acceleration factor and what do 1 000 h represent?

$AF = \exp[(E_a/k)(1/T_{\text{use}} - 1/T_{\text{stress}})] = \exp[(0.7 / 8.617e-5) \times (1/328.15 - 1/398.15)] = 77.7$ . 1 000 h  $\times$  77.7 = 77 700 h, about 8.9 years of use (voltage acceleration comes on top).

### Q89. Zero failures in 231 devices after 1 000 h HTOL with $AF = 77.7$ . What FIT can you claim at 60 % confidence?

Device-hours =  $231 \times 1\,000 \times 77.7 = 1.79e7$ . For zero failures,  $\chi^2(60\%, 2 \text{ dof})/2 = 0.916$ .  $\lambda = 0.916 / 1.79e7 = 5.1e-8$  per hour = **about 51 FIT** (about 128 FIT at 90 % confidence).

### Q90. What is 2.5D vs 3D integration?

2.5D places dies side by side on an interposer (silicon with TSVs, or organic/bridge) with dense wiring between them - e.g. logic plus HBM. 3D stacks dies on top of each other connected by TSVs or hybrid bonding. 3D gives shorter connections but harder thermal, test and power delivery.

### Q91. A 2 W chip has $\theta_{JA} = 25$ C/W in an 85 C ambient. Is it within a 125 C junction limit?

$T_J = 85 + 2 \times 25 = 135$  C: **no**. Needed  $\theta_{JA} \leq (125 - 85)/2 = 20$  C/W (heat spreader, better board, heat sink) or lower power.

## Project, methodology and economics

### Q92. NRE is \$5 M; the ASIC costs \$5 per unit, an FPGA solution \$80. Break-even volume?

$5\,000\,000 / (80 - 5) = 66\,667$  units. Below that the FPGA is cheaper (ignoring power, size and schedule); above it the ASIC wins.

### Q93. What drives the cost of an advanced-node tapeout?

Mask sets (tens of masks, many multi-patterned or EUV), IP licences and royalties, EDA licences, engineering headcount over years, prototype wafers, packaging and test development, and the risk of a respin. Leading-edge full-mask sets are commonly quoted in the multi-million-dollar range.

### Q94. Why is a metal-only respin much cheaper and faster than a full respin?

Only the metal/via masks change (fewer, cheaper masks) and wafers held before metallisation ("base layer hold") can be finished in weeks instead of months.

### Q95. What is the role of a design methodology (CAD) team?

It owns the flow: tool versions, scripts, reference constraints, PDK and library installation, QoR tracking, regression and sign-off checklists, so that design teams spend time on the design and not on the plumbing.

**Q96. How would you track whether a physical design is converging?**

Trend WNS/TNS, number of failing endpoints, DRC count, congestion overflow, utilisation, power and IR drop per run and per block; compare against schedule milestones and stop adding features (ECO freeze) when trends flatten.

## Puzzles

---

**Q97. Design a divide-by-3 clock with 50 % duty cycle.**

Build a mod-3 counter on the rising edge; make a signal high for 1.5 cycles by OR-ing a rising-edge decode with a copy of it retimed on the falling edge (count == 0 on posedge, delayed half a cycle by a negedge flop). The OR is high for 1.5 of 3 cycles. Declare it with `create_generated_clock -divide_by 3` and review the glitch-free structure.

**Q98. A chip fails at -40 C but passes at 125 C at the slow corner. Name two plausible reasons.**

Temperature inversion (the slow corner is cold at low voltage) not covered in sign-off, or hold violations that appear when fast clock buffers and slow data combine; also analog blocks (PLL, bias) with a cold-temperature margin problem.

**Q99. Same RTL, two synthesis runs, one netlist is 10 % larger. What do you check first?**

Constraints (clock period, uncertainty, I/O delays, `dont_use`), library set and corner, tool version and options (flatten, retiming, effort), and whether a black box or constant input changed. Diff the reports (cell-type histogram, area per hierarchy) before touching the RTL.

**Q100. You have one hour before a tapeout deadline and one hold violation of -5 ps at one fast corner. What do you do?**

Do not waive it silently. Check if it is real (CRPR, correct corner, correct SDC). If real, fix with a single delay cell or resize in an ECO, re-run LVS/DRC/STA incrementally on the changed area and LEC, then document. A known hold violation is a functional failure at every frequency.

## Appendix D. Standards, Tools and Resources

A map of the standards, tools, foundries, books and communities an ASIC engineer meets. Tool names and ownership change through mergers and rebranding; check vendor sites for the current product names. Revision years are those current at the time of writing.

### Design-language and design-data standards

| Standard                | Scope                                                      | Notes                                                                                                                                                 |
|-------------------------|------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>IEEE 1364</b>        | Verilog HDL (last revision 2005)                           | Merged into SystemVerilog; VCD is defined here                                                                                                        |
| <b>IEEE 1800</b>        | SystemVerilog (design, assertions, verification)           | Revisions 2005, 2009, 2012, 2017, 2023                                                                                                                |
| <b>IEEE 1800.2</b>      | UVM class library                                          | Accellera UVM standardised                                                                                                                            |
| <b>IEEE 1076</b>        | VHDL                                                       | Still common in Europe, defence, FPGA                                                                                                                 |
| <b>IEEE 1685</b>        | IP-XACT: XML description of IP, registers and connectivity | Used for register generation and SoC assembly                                                                                                         |
| <b>IEEE 1735</b>        | Encryption and rights management for IP                    | Lets vendors ship encrypted RTL/netlists to simulators and synthesis                                                                                  |
| <b>IEEE 1801</b>        | UPF power intent                                           | Revisions from 2009 (UPF 2.0) onward; 2018 = UPF 3.1, newer revisions exist                                                                           |
| <b>IEEE 1497</b>        | SDF - Standard Delay Format                                | SDF 3.0 is the common version                                                                                                                         |
| <b>IEEE 1481</b>        | Delay and power calculation (DPCS/OLA); also defines SPEF  | SPEF is the parasitic exchange everyone uses                                                                                                          |
| <b>IEEE 2416</b>        | Power modelling for system-level analysis (2019)           | Parameterised, process-/voltage-aware power models                                                                                                    |
| <b>Liberty</b>          | Library timing/power/noise format                          | Originated at Synopsys; published through the Open Source Liberty program and steered by the Liberty Technical Advisory Board (LTAB, under IEEE-ISTO) |
| <b>SDC</b>              | Timing constraints                                         | Synopsys-originated, published openly; a Tcl subset                                                                                                   |
| <b>LEF/DEF</b>          | Physical library and design exchange                       | Cadence-originated, distributed by Si2; 5.8 is the common version                                                                                     |
| <b>OpenAccess</b>       | Design database API and reference implementation           | Si2 OpenAccess Coalition; used by custom/AMS tools                                                                                                    |
| <b>GDSII / OASIS</b>    | Layout stream formats                                      | GDSII: de facto (Calma); OASIS: SEMI P39, OASIS.MASK: SEMI P44                                                                                        |
| <b>IEEE 1450 (STIL)</b> | Standard Test Interface Language                           | Test patterns from ATPG to ATE; 1450.6 = CTL for cores                                                                                                |

### Test, reliability and safety standards

| Standard           | Scope                                   | Notes                                                                     |
|--------------------|-----------------------------------------|---------------------------------------------------------------------------|
| <b>IEEE 1149.1</b> | JTAG test access port and boundary scan | 1149.6 adds AC-coupled/differential I/O; 1149.10 high-speed packet access |
| <b>IEEE 1500</b>   | Embedded core test wrappers             | WIR, WBR, WBY; core isolation for SoC test                                |
| <b>IEEE 1687</b>   | IJTAG: access to embedded instruments   | ICL/PDL languages, SIB-based networks                                     |
| <b>IEEE 1838</b>   | Test access for 3-D stacked dies        | Die wrappers for stacked-die test                                         |

| Standard                              | Scope                                                     | Notes                                                                                                               |
|---------------------------------------|-----------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| <b>JEDEC JESD47</b>                   | Stress-test-driven qualification of ICs                   | Defines HTOL, temperature cycling, humidity, ESD and latch-up requirements                                          |
| <b>JEDEC JESD22 series</b>            | Individual reliability test methods                       | e.g. A108 HTOL, A104 temperature cycling, A110 HAST, A113 preconditioning                                           |
| <b>JEDEC JESD78</b>                   | IC latch-up test                                          | Current and voltage injection at temperature                                                                        |
| <b>ANSI/ESDA/JEDEC JS-001, JS-002</b> | HBM and CDM ESD testing                                   | Replace the older JESD22-A114 (HBM) and C101 (CDM)                                                                  |
| <b>JEDEC JESD89</b>                   | Soft-error rate measurement                               | Alpha and neutron SER testing, reported in FIT                                                                      |
| <b>AEC-Q100</b>                       | Automotive IC qualification                               | Grades: 0 (-40 to +150 C), 1 (-40 to +125 C), 2 (-40 to +105 C), 3 (-40 to +85 C); AEC-Q104 for multi-chip modules  |
| <b>ISO 26262</b>                      | Automotive functional safety (2nd ed. 2018)               | Part 11: semiconductors. ASIL B/C/D targets: SPFM $\geq 90/97/99$ %, LFM $\geq 60/80/90$ %, PMHF $< 100/100/10$ FIT |
| <b>IEC 61508, DO-254</b>              | Industrial functional safety; airborne hardware assurance | Similar safety-lifecycle ideas outside automotive                                                                   |
| <b>ISO/SAE 21434</b>                  | Automotive cybersecurity engineering                      | Drives secure boot, key storage and debug locking requirements                                                      |

## Commercial EDA tools by flow step

The three large vendors cover almost the whole digital flow; Ansys (power integrity, thermal, multiphysics) was acquired by Synopsys, with the deal completed in 2025, so some products below may be rebranded. Keysight focuses on RF/microwave and data management.

| Flow step                      | Synopsys                               | Cadence                       | Siemens EDA                    | Others                                                                                  |
|--------------------------------|----------------------------------------|-------------------------------|--------------------------------|-----------------------------------------------------------------------------------------|
| <b>RTL simulation</b>          | VCS                                    | Xcelium                       | Questa                         | Aldec Riviera-PRO                                                                       |
| <b>Emulation / prototyping</b> | ZeBu, HAPS                             | Palladium, Protium            | Veloce                         | -                                                                                       |
| <b>Formal / lint / CDC</b>     | VC Formal, VC SpyGlass                 | Jasper (formal apps, CDC/RDC) | Questa Formal, Questa CDC/Lint | Real Intent (Ascent, Meridian), Blue Pearl                                              |
| <b>HLS</b>                     | -                                      | Stratus                       | Catapult                       | -                                                                                       |
| <b>Logic synthesis</b>         | Design Compiler (NXT), Fusion Compiler | Genus                         | -                              | -                                                                                       |
| <b>DFT / ATPG</b>              | TestMAX (DFTMAX, TetraMAX)             | Modus                         | Tessent                        | -                                                                                       |
| <b>Equivalence (LEC)</b>       | Formality                              | Conformal LEC                 | -                              | -                                                                                       |
| <b>Place and route</b>         | IC Compiler II, Fusion Compiler        | Innovus                       | Aprisa                         | -                                                                                       |
| <b>Extraction</b>              | StarRC                                 | Quantus                       | Calibre xACT                   | -                                                                                       |
| <b>STA</b>                     | PrimeTime                              | Tempus                        | -                              | -                                                                                       |
| <b>Power / IR / EM</b>         | PrimePower, RedHawk-SC (ex-Ansys)      | Voltus, Joules                | -                              | PowerArtist (RTL power; ownership changed during the merger - check the current vendor) |
| <b>Physical verification</b>   | IC Validator                           | Pegasus                       | Calibre                        | -                                                                                       |

| Flow step                       | Synopsys                             | Cadence                                   | Siemens EDA              | Others                                                             |
|---------------------------------|--------------------------------------|-------------------------------------------|--------------------------|--------------------------------------------------------------------|
| <b>Custom / AMS layout</b>      | Custom Compiler                      | Virtuoso                                  | Tanner, Solido           | Keysight ADS/RFPro (RF)                                            |
| <b>Circuit simulation</b>       | HSPICE, PrimeSim                     | Spectre                                   | Analog FastSPICE, Solido | Keysight (RF, EM: Momentum, EMPro)                                 |
| <b>Library characterisation</b> | PrimeLib                             | Liberate                                  | Solido Characterization  | -                                                                  |
| <b>Package / 3D-IC</b>          | 3DIC Compiler                        | Integrity 3D-IC, Allegro Package Designer | Xpedition, HyperLynx     | -                                                                  |
| <b>Thermal</b>                  | RedHawk-SC Electrothermal (ex-Ansys) | Celsius                                   | Simcenter Flotherm       | -                                                                  |
| <b>Design data management</b>   | -                                    | -                                         | -                        | Cliosoft SOS/HUB (Cliosoft is now part of Keysight), Perforce, git |

#### READING THE TOOL TABLE

"-" means the vendor has no widely used product in that step, not that it has none at all. Most teams mix vendors: for example one vendor's P&R with another's STA and a third's PV, because sign-off tools are chosen by foundry certification and customer trust.

## Open-source tools

| Tool                                     | Role                                                                   |
|------------------------------------------|------------------------------------------------------------------------|
| <b>Icarus Verilog, Verilator</b>         | Simulation (event-driven; cycle-based compiled C++)                    |
| <b>cocotb</b>                            | Python testbenches for any simulator                                   |
| <b>GTKWave, Surfer</b>                   | Waveform viewers                                                       |
| <b>Verible, sv2v, slang</b>              | SystemVerilog lint/format, SV-to-Verilog conversion, fast SV front end |
| <b>Yosys + ABC</b>                       | Synthesis and technology mapping; SymbiYosys for formal                |
| <b>OpenROAD</b>                          | Floorplan, placement, CTS, routing, STA (OpenSTA), parasitics, IR drop |
| <b>OpenLane / OpenLane 2 / LibreLane</b> | Complete RTL-to-GDS flows built around Yosys and OpenROAD              |
| <b>Magic, KLayout, Netgen</b>            | Layout editing, DRC, extraction; layout viewing and DRC scripting; LVS |
| <b>ngspice, Xyce, Xschem</b>             | SPICE simulation and schematic capture                                 |
| <b>OpenRAM</b>                           | SRAM compiler                                                          |
| <b>Fault, OpenTimer, iEDA, Coriolis</b>  | Open DFT, timing, and alternative physical design tool sets            |

## Foundries, open PDKs and prototyping routes

| Name                   | Notes                                                                                                    |
|------------------------|----------------------------------------------------------------------------------------------------------|
| <b>TSMC</b>            | Largest pure-play foundry; leads at 3 nm and 2 nm-class nodes; broad mature-node and specialty portfolio |
| <b>Samsung Foundry</b> | FinFET and the first GAA (3 nm-class) in production                                                      |
| <b>Intel Foundry</b>   | Intel's own advanced nodes (18A with RibbonFET GAA and backside power) offered to external customers     |
| <b>GlobalFoundries</b> | FinFET down to 12 nm, FD-SOI (22FDX), RF and specialty processes                                         |



| Name                                          | Notes                                                                                                                   |
|-----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <b>UMC, SMIC, Tower, X-FAB, SkyWater, IHP</b> | Mature, specialty, analog/RF, automotive and research processes                                                         |
| <b>Rapidus</b>                                | Japanese 2 nm-class foundry start-up (pilot line started 2025)                                                          |
| <b>SkyWater SKY130</b>                        | Open 130 nm PDK (Google + SkyWater); used in this book's examples                                                       |
| <b>GF180MCU</b>                               | Open 180 nm PDK (Google + GlobalFoundries)                                                                              |
| <b>IHP SG13G2</b>                             | Open 130 nm SiGe BiCMOS PDK with open MPW runs                                                                          |
| <b>ASAP7, FreePDK45 / Nangate45</b>           | Predictive academic PDKs - good for research flows, not manufacturable                                                  |
| <b>Europractice, CMC, MOSIS</b>               | University and low-volume MPW access to commercial processes (under NDA)                                                |
| <b>Tiny Tapeout</b>                           | Very low-cost educational shared chips; moved to other fabrication routes (including IHP) after Efabless closed in 2025 |

## Books

| Book                                                                                                                 | Why read it                                                                              |
|----------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| N. Weste, D. Harris - <i>CMOS VLSI Design: A Circuits and Systems Perspective</i>                                    | The standard text: devices, logical effort, circuits, power, testing, design methodology |
| J. Rabaey, A. Chandrakasan, B. Nikolic - <i>Digital Integrated Circuits: A Design Perspective</i>                    | Circuit-level depth on delay, interconnect, timing and memories                          |
| A. Kahng, J. Lienig, I. Markov, J. Hu - <i>VLSI Physical Design: From Graph Partitioning to Timing Closure</i>       | Algorithms behind partitioning, floorplanning, placement, CTS, routing                   |
| N. Sherwani - <i>Algorithms for VLSI Physical Design Automation</i>                                                  | Classic algorithm reference for PD tools                                                 |
| J. Bhasker, R. Chadha - <i>Static Timing Analysis for Nanometer Designs: A Practical Approach</i>                    | The practical STA book: SDC, OCV, SI, reports                                            |
| M. Bushnell, V. Agrawal - <i>Essentials of Electronic Testing for Digital, Memory and Mixed-Signal VLSI Circuits</i> | Fault models, ATPG, DFT, BIST, IDDQ                                                      |
| M. Keating, D. Flynn, R. Aitken, A. Gibbons, K. Shi - <i>Low Power Methodology Manual</i>                            | Clock gating, power gating, retention, multi-voltage in practice (free from Synopsys)    |
| V. Chakravarthi - <i>A Practical Approach to VLSI System on Chip (SoC) Design</i>                                    | End-to-end SoC flow from spec to silicon                                                 |
| H. Kaeslin - <i>Top-Down Digital VLSI Design</i>                                                                     | Architecture to gates with a strong methodology focus                                    |
| L.-T. Wang, C.-W. Wu, X. Wen - <i>VLSI Test Principles and Architectures</i>                                         | Deeper DFT: compression, logic BIST, diagnosis                                           |

## Courses and communities

- **University courses (many with public material):** UC Berkeley EECS 151/251A (ASIC lab track), UIUC/Coursera *VLSI CAD: Logic to Layout*, UCSD physical-design courses (A. Kahng), NPTEL (IIT) courses on VLSI physical design and testing.
- **Hands-on:** Zero to ASIC course and Tiny Tapeout (first chip for beginners), OpenROAD/OpenLane tutorials on SKY130 and IHP, VSD (VLSI System Design) workshops.
- **Conferences:** DAC, ICCAD, DATE, ISPD (physical design), ITC (test), ISSCC and VLSI Symposium (circuits), IRPS (reliability), plus vendor user groups (SNUG, CadenceLIVE, User2User).
- **Communities:** FOSSi Foundation (ORConf, Latch-Up), OpenROAD and Yosys GitHub discussions, the Tiny Tapeout and open-source silicon chat servers, r/chipdesign, EDABoard, SemiWiki.
- **Standards bodies to follow:** IEEE SA, Accellera, Si2, JEDEC, AEC, SEMI, ESDA.



#### A LEARNING PATH THROUGH THESE RESOURCES

Read Weste & Harris alongside Chapters 2-4, Bhasker & Chadha with Chapter 9 and Kahng et al. with Chapters 13-17. Then take a small block through OpenLane on SKY130 (Chapter 24) and compare every report against the checklists in Appendix A.